

# Computer Science XI

Learn every chapter on your own — explanations, worked examples, memory tricks, practice questions and full answer keys.

Bilingual support: English + اردو

کمپیوٹر سائنس — گیارہویں جماعت: خود سیکھیں، خود پرکھیں

01 Computer Systems

02 Computational Thinking & Algorithm

03 Programming Fundamentals

04 Data and Analysis

05 Application and Impacts of Computing

06 Digital Literacy

XI

# Contents

## Chapter 1: Computer Systems

- 1.1.1 Discrete and Continuous Quantities
- 1.1.3 Analog and Digital Signals
- 1.1.5 Boolean Operations
- 1.1.7 Logic Gates (Basic Gates: AND, OR, NOT)
- 1.1.7 Advanced Logic Gates (XOR and XNOR)
- 1.1.11 Logic Diagrams
- LogiSim Evolution: Simulation Tool
- 1.2.1 – 1.2.3 Software Development Life Cycle (SDLC)
- 1.2.4 The Agile Model
- 1.1.2 Introduction to Digital Systems
- 1.1.4 Boolean Algebra
- 1.1.6 Truth Tables
- 1.1.7 Universal Logic Gates (NAND and NOR)
- 1.1.8 – 1.1.10 Logic Expressions, Minterms and Maxterms
- 1.1.13 Karnaugh Maps (K-Maps)
- Building a Circuit in LogiSim
- 1.2.4 The Waterfall Model
- 1.2.5 – 1.2.7 Waterfall vs Agile: Comparison and Case Studies
- 1.3.1 – 1.3.4 Communication Models and the OSI 7-Layer Model
- 1.3.5 – 1.3.6 TCP/IP Model vs OSI Model

## Chapter 2: Computational Thinking & Algorithm

- 2.1 Computational Thinking
- 2.2.2 Algorithm vs Pseudocode
- 2.3.2 Pattern Recognition
- 2.4 Sorting and Searching Algorithms
- 2.4.1 Sorting Algorithms: Selection Sort
- 2.4.2 Searching Algorithms: Binary Search
- 2.6 How to Decide Which Algorithm to Use
- 2.2 Algorithm
- 2.3.1 Decomposition
- 2.3.3 Abstraction
- 2.4.1 Sorting Algorithms: Bubble Sort
- 2.4.2 Searching Algorithms: Linear Search
- 2.5 Evaluation Criteria of Algorithm

## Chapter 3: Programming Fundamentals

- 3.1 Programming
- 3.2 Career Growth with Python
- 3.3.1 Visual Studio Code (VS Code)
- 3.4.1 Variables
- 3.4.2 Input and Output Handling
- 3.4.3 Relational, Logical and Membership Operators
- 3.5 Control Structures
- 3.5.2 Selection Statements: if and if-else
- 3.5.3 Repetition Statements: the for Loop and range()
- 3.5.3 Nested Loops and for vs while
- 3.6 Libraries in Python
- 3.6.2 Third-Party Libraries
- 3.1.1 Programming Language Types
- 3.3 Integrated Development Environment (IDE)
- 3.4 Fundamentals of Python Programming
- 3.4.1 Data Types in Python
- 3.4.3 Operators and Operands: Arithmetic and Assignment
- 3.4.3 Bitwise Operators
- 3.5.1 Sequence
- 3.5.2 Selection Statements: if-elif-else and Nested if
- 3.5.3 The while Loop, break and continue
- Practical Activity: Shopping Bill Calculator
- 3.6.1 Built-in Libraries
- 3.7 Debugging Code in Python

## Chapter 4: Data and Analysis

- 4.1.1 Data, Information and Database
- 4.2 Database Components (Tables, Records, Fields)
- 4.3 Relational Database Management System (RDBMS)
- 4.5 Referential Integrity
- 4.7 Case Study — Library Management System
- 4.9 Creation of Tables in Microsoft Access
- 4.11 Creating Queries in MS Access
- 4.11.6 Data Visualization in MS Access (Charts)
- 4.1.2 Database Management System (DBMS)
- 4.3.1 Keys and Integrity Constraints
- 4.4 Entity Relationship Model (ER-Model)
- 4.6 Relational Schema Development
- 4.8 Database Objects (Tables, Forms, Queries, Reports)
- 4.10 Designing Forms for Data Manipulation
- 4.11.4 Data Summarization and Statistical Analysis

## Chapter 5: Application and Impacts of Computing

- 5.1 Introduction to Computing Impacts
- 5.1.2 Internet of Things (IoT) & Components
- Comparison: IoT vs AI vs Data Analytics
- 5.2.1 Scope of AI in Education in Pakistan
- 5.3 Information Sources (Primary, Secondary, Tertiary)
- 5.5 Assistive Technologies
- 5.5.2 Career Connection in Assistive Tech
- 5.1.1 Artificial Intelligence (AI)
- 5.1.3 Data Analytics & Components
- 5.2 Uses of IoT in Different Areas
- 5.2.2 Career Connection in IoT and AI
- 5.4 Impacts of Computing in Various Fields
- 5.5.1 Importance of Assistive Technologies

## Chapter 6: Digital Literacy

- 6.1 Introduction to Digital Literacy
- 6.3 Types of Data
- 6.4.1 – 6.4.2 Interviews and Surveys
- 6.5 Primary and Secondary Data
- 6.7 Presenting Data Using Digital Tools: 6.7.1 Spreadsheets and 6.7.2 Presentations
- 6.8 Case Study: Digital Inquiry Project
- 6.8 Steps 4 – 6: Survey Preparation and Primary Data Collection
- 6.8 Steps 10 – 12: Data Analysis, Conclusion and Recommendations
- Careers in Digital Research
- 6.2 Digital Literacy in the Information Age
- 6.4 Data-Collection Strategies
- 6.4.3 – 6.4.5 Prototypes, Observation and Simulations
- 6.6 Designing a Data-Collection Approach
- 6.7.3 – 6.7.4 Infographics and Reports
- 6.8 Steps 1 – 3: Advanced Search and Methodology
- 6.8 Steps 7 – 9: Secondary Data and Spreadsheet Organisation
- 6.8 The Final Digital Artefact

**Final Revision:** one-page summaries, mock paper and answers

## How to use this booklet

This booklet is written so that you can learn every chapter **on your own**, without a teacher sitting next to you. Each topic explains the idea in simple English, shows a worked example, gives you a memory trick, warns you about common mistakes, explains it again in Urdu, and ends with quick questions so you can check that you really understood.

یہ کتابچہ اس طرح لکھا گیا ہے کہ آپ استاد کے بغیر خود ہر باب سیکھ سکیں۔ ہر موضوع پہلے آسان انگریزی میں سمجھایا گیا ہے، پھر حل شدہ مثال، یاد رکھنے کا طریقہ، عام غلطیاں، اردو میں وضاحت اور آخر میں خود کو پرکھنے کے سوالات دیے گئے ہیں۔

## What the coloured boxes mean

### LEARN IT

The main explanation. Read it slowly and make sure every sentence makes sense.

### WORKED EXAMPLE

A solved example, step by step. Cover the solution and try it yourself first.

### REMEMBER IT

A short trick, analogy or mnemonic that helps you recall the idea in the exam.

### COMMON MISTAKE

Mistakes students often make. Read these before every test.

اردو میں سمجھیں

اسی موضوع کی اردو میں آسان وضاحت۔

### EXAM TIP

How Golden topics are asked in board exams and what examiners look for.

### CHECK YOURSELF

Two or three quick questions. Answer them before looking at the answers underneath.

### GOLDEN TOPIC

★ **Golden** marks a high-yield topic that appears often in examinations. Give these topics extra time.

## The 5-step self-study method

1. **Preview (5 minutes).** Read the chapter opener and the "What you will learn" list. Turn each objective into a question you want answered.
2. **Learn one topic at a time.** Read the explanation, then work through the example with a pen and paper. Do not just read the solution — reproduce it.
3. **Check yourself straight away.** Answer the "Check yourself" questions from memory. If you get one wrong, re-read that topic before moving on.
4. **Practise at the end of the chapter.** Do the multiple-choice, short and long questions in exam conditions (no notes), then mark your work using the answer key.
5. **Review and repeat.** Tick the "I can..." checklist. Revisit anything you could not tick after 1 day, 3 days and 7 days. Spaced repetition makes knowledge stick.

## Tips for board exam answers

- **Define, explain, example.** A strong short answer gives a clear definition, one or two lines of explanation and a real-life example.
- **Use tables for "differentiate" questions.** Compare at least four points side by side.
- **Draw and label.** Logic gates, diagrams, charts and data structures earn marks when drawn neatly and labelled.
- **Show every step.** In K-maps, sorting traces, trace tables, statistics and code output questions, examiners give marks for the working, not only the final answer.
- **Write correct code.** Indent Python carefully, use meaningful variable names, and write the expected output under your program.

### CHAPTER 1

## Computer Systems

This chapter shows you how a computer thinks in 0s and 1s, how logic gates turn those bits into decisions, how software is planned and built step by step, and how data travels across a network. These three ideas — digital logic, software development and communication models — are the foundation of everything else in Computer Science.

اس باب میں آپ سیکھیں گے کہ کمپیوٹر صفر اور ایک کی زبان میں کیسے کام کرتا ہے، لاجک گیٹس فیصلے کیسے کرتے ہیں، سافٹ ویئر مرحلہ وار کیسے بنایا جاتا ہے، اور نیٹ ورک پر ڈیٹا ایک جگہ سے دوسری جگہ کیسے پہنچتا ہے۔

## What you will learn

- Tell the difference between discrete and continuous quantities, and between analog and digital signals, with examples.
- Use the Boolean operations AND, OR and NOT, and apply the order of precedence.
- Draw the symbol, write the expression and give the truth table of AND, OR, NOT, NAND, NOR, XOR and XNOR gates.
- Simplify Boolean expressions using 2-variable and 3-variable Karnaugh maps.
- Describe the six phases of the SDLC and compare the Waterfall and Agile models using case studies.
- State the advantages of digital systems.
- Build a complete truth table for any expression with 2 or 3 inputs.
- Write minterms (SOP) and maxterms (POS) from a truth table, and draw a logic diagram from an expression.
- Build and test a simple circuit in LogiSim Evolution.
- Explain the seven layers of the OSI model and map them to the four layers of the TCP/IP model.

### 1.1.1 Discrete and Continuous Quantities

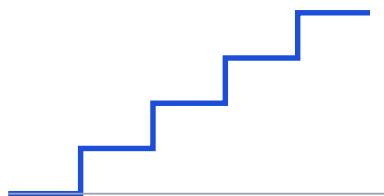
#### LEARN IT

Everything around you that can be measured or counted is a **quantity**. Quantities come in two kinds.

**Discrete quantities** are separate, countable items with specific values. You can count them one by one: 1, 2, 3 ... There is nothing "in between". You cannot have 2.5 students in a classroom.

**Continuous quantities** can take *any* value within a range. They flow smoothly without breaks. Water can be 30 °C, 30.5 °C, 30.51 °C, and so on — there is always another value in between.

| Discrete (counted)                | Continuous (measured)    |
|-----------------------------------|--------------------------|
| Number of students in a classroom | Temperature of water     |
| Number of cars in a parking lot   | Speed of a moving car    |
| Has fixed, separate values        | Has any value in a range |



Discrete (Stairs)



Continuous (Ramp)

Stairs have fixed steps (discrete). A ramp lets you stand at any point (continuous).

#### REMEMBER IT

**Stairs vs Ramp.** On stairs you can only stand on step 1, 2 or 3 — that is discrete data. On a ramp you can stand at any height — that is continuous data. Ask yourself: "Do I *count* it or *measure* it?" Count → discrete. Measure → continuous.

#### COMMON MISTAKE

Students sometimes say "money is continuous because amounts are large". No — rupees and paise are counted in fixed units, so money is discrete. Size does not matter; what matters is whether values in between are possible.

منفصل (Discrete) مقداریں وہ ہیں جنہیں ایک ایک کر کے گنا جا سکے، جیسے کلاس میں طلبہ کی تعداد۔ مسلسل (Continuous) مقداریں کسی حد کے اندر کوئی بھی قیمت لے سکتی ہیں، جیسے پانی کا درجہ حرارت۔ سیڑھیاں منفصل ڈیٹا کی مثال ہیں اور ڈھلوان (ریمپ) مسلسل ڈیٹا کی۔

#### CHECK YOURSELF

1. Is the number of pages in a book discrete or continuous?
2. Is your height discrete or continuous?
3. In the stairs/ramp analogy, which one shows continuous data?

Answers: 1. Discrete (you count pages). 2. Continuous (height is measured and can be any value). 3. The ramp.

## 1.1.2 Introduction to Digital Systems

#### LEARN IT

A **digital system** is an electronic system that uses only two values: **0** and **1**. These two values are called **binary digits**, or **bits** for short.

- **0** represents OFF, LOW, or FALSE.
- **1** represents ON, HIGH, or TRUE.

Why only two values? Because it is easy for an electronic circuit to tell "voltage present" from "no voltage". Every number, letter, photo and song in a computer is finally stored as a long pattern of these bits.

### Advantages of digital systems

| Advantage      | Detail                                   |
|----------------|------------------------------------------|
| Reliable       | Less affected by noise and interference. |
| Accurate       | No loss of quality when copying data.    |
| Easy to design | Only two states (0 and 1) to handle.     |
| Programmable   | Can be easily controlled by software.    |

### Examples of digital systems

- **Computers:** process data as 0s and 1s.
- **Smartphones:** store photos and music as binary data.
- **Digital cameras:** capture images as binary code.

#### WORKED EXAMPLE

##### Why is copying digital data "accurate"?

Step 1: A song is stored as bits, for example 10110010...

Step 2: When you copy it to a USB, the computer copies each bit exactly: 1 stays 1, 0 stays 0.

Step 3: Even after 100 copies the bits are identical, so the song sounds the same. An old cassette tape (analog) loses a little quality every time it is copied.

### REMEMBER IT

Think of a bit as a light switch: it is either ON (1) or OFF (0). A digital system is a huge panel of such switches.

### اردو میں سمجھیں

ڈیجیٹل نظام صرف دو قیمتیں استعمال کرتا ہے: 0 اور 1، جنہیں بٹ (Bit) کہتے ہیں۔ 0 کا مطلب بند، کم یا غلط ہے اور 1 کا مطلب چالو، زیادہ یا درست ہے۔ ڈیجیٹل نظام قابلِ بھروسا، درست، بنانے میں آسان اور سافٹ ویئر سے قابو میں رہنے والے ہوتے ہیں۔

### CHECK YOURSELF

1. What does a 1 represent in a digital system?
2. Give two advantages of digital systems.

Answers: 1. ON, HIGH or TRUE. 2. Any two of: reliable (less affected by noise), accurate (no loss when copying), easy to design (two states), programmable (controlled by software).

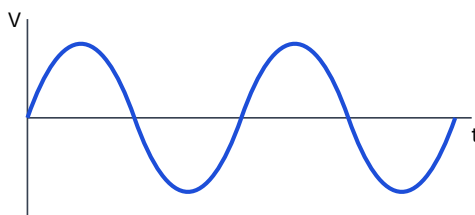
## 1.1.3 Analog and Digital Signals ★ Golden

### LEARN IT

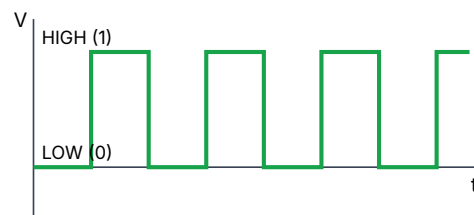
A **signal** is a changing voltage (or other quantity) that carries information.

**Analog signals** are continuous signals that change smoothly over time, like a **sine wave**. Examples: the human voice, a traditional mercury thermometer.

**Digital signals** have only two levels: **HIGH (1)** and **LOW (0)**. They jump instantly between these values, which creates a **square wave**. Example: computer data.



Analog Signal (Sine Wave)



Digital Signal (Square Wave)

An analog signal changes smoothly; a digital signal jumps between only two levels.

| Feature     | Analog data                           | Digital data                      |
|-------------|---------------------------------------|-----------------------------------|
| Nature      | Continuous flow.                      | Discrete, separate values (bits). |
| Wave shape  | Sine wave (smooth curve).             | Square wave (sudden jumps).       |
| Reliability | Low (affected by noise).              | High (resistant to noise).        |
| Examples    | Human voice, traditional thermometer. | Computer data.                    |

### WORKED EXAMPLE

#### Why is digital more resistant to noise?

Step 1: Suppose a wire carries 5 V for HIGH and 0 V for LOW.

Step 2: Noise adds a small error, so 5 V arrives as 4.6 V and 0 V arrives as 0.3 V.

Step 3: The receiver only asks "Is it closer to HIGH or LOW?" 4.6 V is still read as 1 and 0.3 V is still read as 0. The data is perfect.

Step 4: In an analog signal, every exact value carries meaning, so 4.6 V instead of 5 V really changes the sound or reading. The noise cannot be removed.

#### REMEMBER IT

**Analog = A smooth curve. Digital = Doorsteps.** Analog looks like ocean waves; digital looks like a castle wall (square teeth).

#### COMMON MISTAKE

Do not draw a digital signal with sloping sides. A digital signal jumps *instantly*, so its edges are straight vertical lines. Also do not confuse "digital" with "electronic": an analog signal can also be electronic.

#### اردو میں سمجھیں

اینا لاگ سگنل وقت کے ساتھ ہموار انداز میں بدلتا ہے، جیسے سائن ویو (Sine Wave)؛ انسانی آواز اس کی مثال ہے۔ ڈیجیٹل سگنل کی صرف دو سطحیں ہوتی ہیں، HIGH (1) اور LOW (0)، اور یہ اچانک ایک سے دوسری پر جاتا ہے، اس لیے مربع لہر (Square Wave) بنتی ہے۔ ڈیجیٹل سگنل شور سے کم متاثر ہوتا ہے، اس لیے زیادہ قابلِ بھروسہ ہے۔

#### EXAM TIP

Common questions: "Differentiate between analog and digital signals" (3–4 marks) and "Draw an analog and a digital signal." Write the comparison as a table with at least three features (nature, wave shape, reliability, example). Always label your diagrams: axes V and t, and HIGH(1)/LOW(0) on the square wave.

#### CHECK YOURSELF

1. What shape of wave does a digital signal make?
2. Which type of signal is more affected by noise?
3. Give one example of an analog signal.

Answers: 1. A square wave. 2. Analog. 3. Human voice (or a traditional thermometer reading).

## 1.1.4 Boolean Algebra

#### LEARN IT

**Boolean algebra** was developed by the English mathematician **George Boole**. It is a kind of algebra that deals with only two values: **TRUE (1)** and **FALSE (0)**. It is the foundation of digital electronics and computer programming — every `if` condition in a program is a small piece of Boolean algebra.

| Concept     | Description                               | Example              |
|-------------|-------------------------------------------|----------------------|
| Variables   | Letters that hold the value 0 or 1.       | A, B, C, X, Y        |
| Operations  | Actions performed on variables.           | AND, OR, NOT         |
| Expressions | Combinations of variables and operations. | A AND B, $Y = A + B$ |

Some simple rules (laws) follow directly from the meaning of AND, OR and NOT. They are useful when you check or simplify answers:



| OR rules    | AND rules       | NOT rules        |
|-------------|-----------------|------------------|
| $A + 0 = A$ | $A \cdot 1 = A$ | $(A')' = A$      |
| $A + 1 = 1$ | $A \cdot 0 = 0$ | $A + A' = 1$     |
| $A + A = A$ | $A \cdot A = A$ | $A \cdot A' = 0$ |

#### WORKED EXAMPLE

**Using a law:** Simplify  $Y = A'B + AB$ .

Step 1: Both terms contain B, so take B out:  $Y = B(A' + A)$ .

Step 2:  $A' + A = 1$  (a variable OR its opposite is always 1).

Step 3:  $Y = B \cdot 1 = B$ .

#### COMMON MISTAKE

In Boolean algebra "+" means OR, not addition. So  $1 + 1 = 1$  (TRUE OR TRUE is TRUE), not 2. There is no value bigger than 1.

#### اردو میں سمجھیں

بولین الجبرا جارج بول (George Boole) نے بنایا۔ اس میں صرف دو قیمتیں ہوتی ہیں: درست (1) اور غلط (0)۔ متغیرات (Variables) جیسے A اور B صرف 0 یا 1 رکھتے ہیں، آپریشنز AND، OR اور NOT ہیں، اور ان کے ملاپ سے ایکسپریشن (Expression) بنتا ہے۔

#### CHECK YOURSELF

1. Who developed Boolean algebra?

2. In Boolean algebra, what is  $1 + 1$ ?

Answers: 1. George Boole. 2. 1 (because + means OR).

## 1.1.5 Boolean Operations

#### LEARN IT

There are three basic Boolean operations. Every other operation can be made from them.

**1. AND operation (  $\cdot$  )** — gives output 1 **only when ALL inputs are 1**. Expression:  $Y = A \cdot B$  (often written just AB).

**2. OR operation ( + )** — gives output 1 **when ANY input is 1**. Expression:  $Y = A + B$ .

**3. NOT operation ( ' or  $\bar{\phantom{x}}$  )** — **inverts** (flips) the input. If the input is 1 the output is 0, and if the input is 0 the output is 1. It is a **unary** operation, meaning it works on only one operand. Expression:  $Y = A'$  (also written as A with a bar on top,  $\bar{A}$ ).

| Operation | Real-life analogy                                                                                                                  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------|
| AND       | A door with two locks. The door opens only when <b>both</b> locks are unlocked.                                                    |
| OR        | A room with two doors. You can enter if <b>either</b> door is open.                                                                |
| NOT       | A room light switch. When the switch is ON the room is lit; when it is OFF the room is dark — the state is the opposite of "dark". |

#### WORKED EXAMPLE

Evaluate each expression for  $A = 1$ ,  $B = 0$ .

- (a)  $A \cdot B = 1 \cdot 0 = 0$  (not all inputs are 1).
- (b)  $A + B = 1 + 0 = 1$  (at least one input is 1).
- (c)  $B' = 0' = 1$  (NOT flips 0 to 1).
- (d)  $A \cdot B' = 1 \cdot 1 = 1$ .

#### REMEMBER IT

AND is "strict" — everyone must agree (all 1s). OR is "easy-going" — one 1 is enough. NOT is "the rebel" — it always says the opposite.

#### اردو میں سمجھیں

AND آپریشن تب ہی 1 دیتا ہے جب تمام ان پٹ 1 ہوں (دو تالوں والا دروازہ). OR آپریشن تب 1 دیتا ہے جب کوئی ایک ان پٹ بھی 1 ہو (دو دروازوں والا کمرہ). NOT آپریشن ان پٹ کو الٹ دیتا ہے اور صرف ایک ان پٹ پر کام کرتا ہے، اس لیے اسے یونری (Unary) کہتے ہیں۔

#### CHECK YOURSELF

1. What symbol is used for the OR operation?
2. Why is NOT called a unary operation?
3. Find  $A + B'$  when  $A = 0$ ,  $B = 1$ .

Answers: 1. "+". 2. Because it works on only one operand (one input). 3.  $B' = 0$ , so  $0 + 0 = 0$ .

## 1.1.6 Truth Tables

#### LEARN IT

A **truth table** shows **all possible input combinations** and the output for each one, for a logic gate or a circuit. It is the "complete record" of how the circuit behaves.

#### How to create a truth table:

1. Count the number of inputs ( $n$ ).
2. Calculate total rows =  $2^n$ .
3. List all input combinations in binary counting order (0, 1, 2, 3 ... in binary).
4. Calculate the output for each combination.

#### 2-input example ( $n = 2$ , rows = 4)

| A | B | Combination |
|---|---|-------------|
| 0 | 0 | Row 1       |
| 0 | 1 | Row 2       |
| 1 | 0 | Row 3       |
| 1 | 1 | Row 4       |

#### 3-input example ( $n = 3$ , rows = 8)

| A | B | C | Decimal |
|---|---|---|---------|
| 0 | 0 | 0 | 0       |
| 0 | 0 | 1 | 1       |
| 0 | 1 | 0 | 2       |
| 0 | 1 | 1 | 3       |
| 1 | 0 | 0 | 4       |
| 1 | 0 | 1 | 5       |
| 1 | 1 | 0 | 6       |
| 1 | 1 | 1 | 7       |

### REMEMBER IT

**Quick fill trick for 3 inputs:** In column C write 0,1,0,1... (change every row). In column B write 0,0,1,1... (change every 2 rows). In column A write 0,0,0,0,1,1,1,1 (change every 4 rows). For  $n$  inputs the leftmost column changes every  $2^{n-1}$  rows.

### WORKED EXAMPLE

**Make the truth table of a 3-input AND:  $Y = A \cdot B \cdot C$ .**

Step 1:  $n = 3$ , so rows =  $2^3 = 8$ .

Step 2: List the 8 combinations in binary order.

Step 3: AND gives 1 only when all inputs are 1, so only row 111 gives 1.

| A | B | C | $Y = A \cdot B \cdot C$ |
|---|---|---|-------------------------|
| 0 | 0 | 0 | 0                       |
| 0 | 0 | 1 | 0                       |
| 0 | 1 | 0 | 0                       |
| 0 | 1 | 1 | 0                       |
| 1 | 0 | 0 | 0                       |
| 1 | 0 | 1 | 0                       |
| 1 | 1 | 0 | 0                       |
| 1 | 1 | 1 | 1                       |

### COMMON MISTAKE

Do not list rows in a random order — you may miss or repeat a combination. Always count in binary from 000 to 111. Also remember rows =  $2^n$ , not  $2 \times n$ : 3 inputs give 8 rows, not 6.

### اردو میں سمجھیں

ثروتھ ٹیبل (Truth Table) کسی گیٹ یا سرکٹ کے تمام ممکنہ ان پٹ اور ان کے آؤٹ پٹ دکھاتا ہے۔ اگر ان پٹ  $n$  ہوں تو قطاریں (Rows) 2 کی طاقت  $n$  ہوں گی؛ دو ان پٹ کے لیے 4 اور تین ان پٹ کے لیے 8 قطاریں۔ ان پٹ ہمیشہ بائری ترتیب میں لکھیں۔

### CHECK YOURSELF

1. How many rows does a truth table with 4 inputs have?
2. In a 3-input table, what is the 6th row (counting from row 1 = 000)?

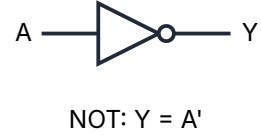
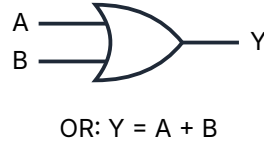
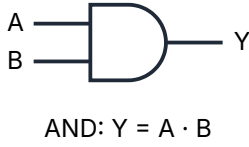
Answers: 1.  $2^4 = 16$ . 2. 101.

## 1.1.7 Logic Gates (Basic Gates: AND, OR, NOT) ★ Golden

### LEARN IT

**Logic gates** are electronic circuits that perform Boolean operations. They take one or more input bits and produce one output bit. They are the **building blocks of all digital systems** — a processor contains billions of them.

The three basic gates match the three basic operations: the **AND gate**, the **OR gate** and the **NOT gate** (also called an **inverter**).



Symbols of the basic gates. AND has a flat back and round front ("D" shape); OR has a curved back and pointed front; NOT is a triangle with a small circle (bubble).

**AND gate** —  $Y = A \cdot B$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

**OR gate** —  $Y = A + B$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**NOT gate** —  $Y = A'$

| A | Y |
|---|---|
| 0 | 1 |
| 1 | 0 |

#### WORKED EXAMPLE

**A school bell circuit.** The bell (Y) should ring when the timer says "period over" ( $A = 1$ ) **and** the school is open ( $B = 1$ ).

Step 1: The bell needs both conditions, so use an AND gate:  $Y = A \cdot B$ .

Step 2: Sunday, timer fires:  $A = 1$ ,  $B = 0 \rightarrow Y = 1 \cdot 0 = 0$ . No bell.

Step 3: Monday, timer fires:  $A = 1$ ,  $B = 1 \rightarrow Y = 1$ . The bell rings.

If instead the bell should ring when *either* the timer fires *or* the principal presses a button, you would use an OR gate.

#### REMEMBER IT

**Shortcut:** AND output is 0 if *any* input is 0. OR output is 1 if *any* input is 1. Learn just the "special row": AND is 1 only at 11; OR is 0 only at 00.

#### COMMON MISTAKE

Do not mix up the AND and OR symbols. AND has a **flat** input side (like the letter D). OR has a **curved** input side and a **pointed** output, like a shield or arrowhead.

#### اردو میں سمجھیں

لاجک گیٹس الیکٹرانک سرکٹ ہیں جو بولین آپریشن انجام دیتے ہیں اور تمام ڈیجیٹل نظاموں کی بنیادی اینٹیں ہیں۔ AND گیٹ صرف تب 1 دیتا ہے جب دونوں ان پٹ 1 ہوں، OR گیٹ صرف تب 0 دیتا ہے جب دونوں ان پٹ 0 ہوں، اور NOT گیٹ ان پٹ کو الٹ دیتا ہے۔

### EXAM TIP

You are often asked to "draw the symbol, write the expression and truth table" of a gate (2–3 marks each). Draw neat symbols with labelled inputs A, B and output Y. Write the full truth table in binary order. A frequent MCQ: "Which gate gives 1 only when all inputs are 1?" (AND).

### CHECK YOURSELF

1. What is the output of an OR gate when  $A = 0$  and  $B = 0$ ?
2. What is another name for the NOT gate?
3. For which input does a 2-input AND gate output 1?

Answers: 1. 0. 2. Inverter. 3. Only  $A = 1$ ,  $B = 1$ .

## 1.1.7 Universal Logic Gates (NAND and NOR)

### LEARN IT

**NAND** means "NOT AND": it is an AND gate followed by a NOT. **NOR** means "NOT OR": an OR gate followed by a NOT. On the symbol, the NOT is shown as a small bubble at the output.

NAND and NOR are called **universal gates** because **any logic circuit in the world can be built using only NAND gates (or only NOR gates)**. Factories like this: they can mass-produce one type of chip and build everything from it.



$$\text{NAND: } Y = (A \cdot B)'$$



$$\text{NOR: } Y = (A + B)'$$

NAND = AND symbol + bubble. NOR = OR symbol + bubble.

**NAND (NOT AND) —  $Y = (A \cdot B)'$**

| A | B | $A \cdot B$ | Y |
|---|---|-------------|---|
| 0 | 0 | 0           | 1 |
| 0 | 1 | 0           | 1 |
| 1 | 0 | 0           | 1 |
| 1 | 1 | 1           | 0 |

**NOR (NOT OR) —  $Y = (A + B)'$**

| A | B | $A + B$ | Y |
|---|---|---------|---|
| 0 | 0 | 0       | 1 |
| 0 | 1 | 1       | 0 |
| 1 | 0 | 1       | 0 |
| 1 | 1 | 1       | 0 |

### WORKED EXAMPLE

**Show that a NAND gate can work as a NOT gate.**

Step 1: Connect the same input A to both inputs of the NAND gate. Then  $Y = (A \cdot A)'$ .

Step 2: Since  $A \cdot A = A$ , we get  $Y = A'$ .

Step 3: Check:  $A = 0 \rightarrow (0 \cdot 0)' = 0' = 1$ .  $A = 1 \rightarrow (1 \cdot 1)' = 1' = 0$ . This is exactly the NOT truth table.

In the same way, a NOR gate with both inputs tied together gives  $(A + A)' = A'$ . And a NAND followed by a NAND-as-NOT gives  $((A \cdot B)')' = A \cdot B$ , an AND gate. This is why they are "universal".

#### REMEMBER IT

NAND is the exact **opposite** of AND (0 only at 11). NOR is the exact opposite of OR (1 only at 00). Just flip the AND/OR output column.

#### COMMON MISTAKE

Do not forget the bubble. Without the small circle, a NAND symbol becomes an AND symbol and your answer is wrong. Also,  $(A \cdot B)'$  is **not** the same as  $A' \cdot B'$ : for  $A = 1, B = 0$ ,  $(1 \cdot 0)' = 1$  but  $0 \cdot 1 = 0$ .

#### اردو میں سمجھیں

NAND کا مطلب ہے NOT AND اور NOR کا مطلب ہے NOT OR۔ انہیں یونیورسل (عالمگیر) گیٹ کہتے ہیں کیونکہ صرف NAND یا صرف NOR گیٹس سے دنیا کا کوئی بھی لاجک سرکٹ بنایا جا سکتا ہے۔ NAND صرف تب 0 دیتا ہے جب دونوں ان پٹ 1 ہوں، اور NOR صرف تب 1 دیتا ہے جب دونوں ان پٹ 0 ہوں۔

#### CHECK YOURSELF

1. Why are NAND and NOR called universal gates?
2. What is the NOR output for  $A = 0, B = 1$ ?
3. How do you make a NOT gate from a NAND gate?

Answers: 1. Any logic circuit can be built using only NAND gates or only NOR gates. 2. 0. 3. Join both inputs together and feed A into them:  $(A \cdot A)' = A'$ .

## 1.1.7 Advanced Logic Gates (XOR and XNOR)

#### LEARN IT

**XOR (Exclusive OR)** gives output 1 when the inputs are **different**. Expression:  $Y = A \oplus B$ . In terms of basic operations,  $A \oplus B = A'B + AB'$ .

**XNOR (Exclusive NOR)** is XOR followed by NOT. It gives output 1 when the inputs are the **same**. Expression:  $Y = (A \oplus B)' = A'B' + AB$ .

The XOR symbol is an OR symbol with an extra curved line at the input side. XNOR adds a bubble at the output.



$$\text{XOR: } Y = A \oplus B$$



$$\text{XNOR: } Y = (A \oplus B)'$$

XOR = OR symbol with an extra curve at the back. XNOR = XOR symbol + bubble.

$$\text{XOR — } Y = A \oplus B$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

$$\text{XNOR — } Y = (A \oplus B)'$$

| A | B | Y |
|---|---|---|
| 0 | 0 | 1 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

## All seven gates at a glance

| A | B | AND | OR | NAND | NOR | XOR | XNOR |
|---|---|-----|----|------|-----|-----|------|
| 0 | 0 | 0   | 0  | 1    | 1   | 0   | 1    |
| 0 | 1 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 0 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 1 | 1   | 1  | 0    | 0   | 0   | 1    |

(NOT has one input:  $A = 0 \rightarrow 1$ ,  $A = 1 \rightarrow 0$ .)

### WORKED EXAMPLE

**A staircase light.** A light has one switch at the bottom (A) and one at the top (B) of a staircase. Flipping either switch should change the light.

Step 1: Start with both switches down:  $A = 0$ ,  $B = 0$ , light off ( $Y = 0$ ).

Step 2: Flip the bottom switch:  $A = 1$ ,  $B = 0 \rightarrow$  light on ( $Y = 1$ ).

Step 3: Walk up and flip the top switch:  $A = 1$ ,  $B = 1 \rightarrow$  light off ( $Y = 0$ ).

Step 4: The pattern 0, 1, 1, 0 is the XOR truth table. So the light is  $Y = A \oplus B$ .

### REMEMBER IT

**Memory trick:** XOR outputs 1 when inputs are "eXclusively" different. XNOR outputs 1 when inputs are the same. XNOR is sometimes called the "equality gate".

### COMMON MISTAKE

XOR is not the same as OR. They differ only in the last row: for  $A = 1$ ,  $B = 1$ , OR gives 1 but XOR gives 0.

### اردو میں سمجھیں

XOR گیٹ تب 1 دیتا ہے جب دونوں ان پٹ مختلف ہوں، اور XNOR گیٹ تب 1 دیتا ہے جب دونوں ان پٹ ایک جیسے ہوں۔ XNOR دراصل XOR کے بعد NOT ہے۔ OR اور XOR میں فرق صرف آخری قطار میں ہے جہاں دونوں ان پٹ 1 ہوں۔

### CHECK YOURSELF

- What is  $1 \oplus 1$ ?
- Which gate outputs 1 when both inputs are the same?
- Write XOR using AND, OR and NOT.

Answers: 1. 0. 2. XNOR. 3.  $A \oplus B = A'B + AB'$ .

## 1.1.8 – 1.1.10 Logic Expressions, Minterms and Maxterms

### LEARN IT

A **logic expression** (Boolean expression) combines variables with operations, for example  $Y = A + B' \cdot C$ . To evaluate it correctly you must follow the **order of precedence**, just like BODMAS in maths:

1. Parentheses ( ) → 2. NOT ( ' ) → 3. AND ( · ) → 4. OR ( + )

### WORKED EXAMPLE

#### Worked example 1: truth table of $Y = A + B' \cdot C$

Step 1: 3 inputs, so 8 rows.

Step 2: Precedence: first find  $B'$  (NOT), then  $B' \cdot C$  (AND), then  $A + (B' \cdot C)$  (OR). Make a column for each step.

| A | B | C | $B'$ | $B' \cdot C$ | $Y = A + B' \cdot C$ |
|---|---|---|------|--------------|----------------------|
| 0 | 0 | 0 | 1    | 0            | 0                    |
| 0 | 0 | 1 | 1    | 1            | 1                    |
| 0 | 1 | 0 | 0    | 0            | 0                    |
| 0 | 1 | 1 | 0    | 0            | 0                    |
| 1 | 0 | 0 | 1    | 0            | 1                    |
| 1 | 0 | 1 | 1    | 1            | 1                    |
| 1 | 1 | 0 | 0    | 0            | 1                    |
| 1 | 1 | 1 | 0    | 0            | 1                    |

Step 3:  $Y = 1$  in rows 1, 4, 5, 6, 7 (decimal). Check one row by hand:  $A=0, B=0, C=1 \rightarrow B' = 1, B' \cdot C = 1, Y = 0 + 1 = 1$ . ✓

### WORKED EXAMPLE

#### Worked example 2: truth table of $Y = (A + B) \cdot C'$

Step 1: Parentheses first: find  $A + B$ . Then NOT:  $C'$ . Then AND the two.

| A | B | C | $A+B$ | $C'$ | Y |
|---|---|---|-------|------|---|
| 0 | 0 | 0 | 0     | 1    | 0 |
| 0 | 0 | 1 | 0     | 0    | 0 |
| 0 | 1 | 0 | 1     | 1    | 1 |
| 0 | 1 | 1 | 1     | 0    | 0 |
| 1 | 0 | 0 | 1     | 1    | 1 |
| 1 | 0 | 1 | 1     | 0    | 0 |
| 1 | 1 | 0 | 1     | 1    | 1 |
| 1 | 1 | 1 | 1     | 0    | 0 |

Step 2:  $Y = 1$  in rows 2, 4 and 6 (decimal).

## Minterms (SOP) and Maxterms (POS)

### LEARN IT

Sometimes you are given a truth table and must *find* the expression. There are two standard ways.

**Minterm (used in SOP — Sum of Products):** a **product (AND)** of all the variables, written for a row whose output is 1. A variable is **complemented ( $A'$ )** if its value is 0, and **uncomplemented ( $A$ )** if its value is 1. OR-ing all the minterms gives the SOP expression.

**Maxterm (used in POS — Product of Sums):** a **sum (OR)** of all the variables, written for a row whose output is 0. A variable is **complemented ( $A'$ )** if its value is 1, and **uncomplemented ( $A$ )** if its value is 0. AND-ing all the maxterms gives the POS expression.



| A | B | Output (Y) | Minterm (SOP) | Maxterm (POS) |
|---|---|------------|---------------|---------------|
| 0 | 0 | 0          | –             | $A + B$       |
| 0 | 1 | 1          | $A'B$         | –             |
| 1 | 0 | 0          | –             | $A' + B$      |
| 1 | 1 | 1          | $AB$          | –             |

**Final SOP expression:**  $Y = A'B + AB$ . **Final POS expression:**  $Y = (A + B)(A' + B)$ . Both describe the same table. (Both simplify to  $Y = B$  — look at the table: Y always equals B. You will confirm this with a K-map later.)

#### WORKED EXAMPLE

**Worked example 3: write the SOP for a 3-input "majority" circuit.**  $Y = 1$  when at least two of A, B, C are 1.

| Row | A | B | C | Y | Minterm |
|-----|---|---|---|---|---------|
| 0   | 0 | 0 | 0 | 0 | –       |
| 1   | 0 | 0 | 1 | 0 | –       |
| 2   | 0 | 1 | 0 | 0 | –       |
| 3   | 0 | 1 | 1 | 1 | $A'BC$  |
| 4   | 1 | 0 | 0 | 0 | –       |
| 5   | 1 | 0 | 1 | 1 | $AB'C$  |
| 6   | 1 | 1 | 0 | 1 | $ABC'$  |
| 7   | 1 | 1 | 1 | 1 | $ABC$   |

Step 1: Pick the rows with  $Y = 1$ : rows 3, 5, 6, 7.

Step 2: Row 3 is  $A=0, B=1, C=1 \rightarrow A$  is 0 so write  $A'$ ; B and C are 1 so write B, C  $\rightarrow A'BC$ .

Step 3: Do the same for the other rows and OR them:  **$Y = A'BC + AB'C + ABC' + ABC$** . In short form:  $Y = \Sigma m(3, 5, 6, 7)$ . We will simplify this with a K-map in 1.1.13.

#### REMEMBER IT

**SOP = 1s, POS = 0s.** Minterm: 0  $\rightarrow$  bar, 1  $\rightarrow$  plain. Maxterm: the reverse (1  $\rightarrow$  bar, 0  $\rightarrow$  plain). Think "Min likes 1s, Max likes 0s".

#### COMMON MISTAKE

Two classic errors: (1) writing minterms for the 0-rows — minterms come only from rows where  $Y = 1$ ; (2) forgetting precedence — in  $A + B \cdot C$  you must do  $B \cdot C$  first.  $A + B \cdot C$  is **not**  $(A + B) \cdot C$ : for  $A=1, B=0, C=0$  the first gives 1 and the second gives 0.

#### اردو میں سمجھیں

ایکسپریشن حل کرتے وقت ترتیب یہ ہے: پہلے قوسین، پھر NOT، پھر AND اور آخر میں OR۔ منٹرم (Minterm) اُن قطاروں سے بنتا ہے جن کا آؤٹ پٹ 1 ہو؛ اس میں 0 والے متغیر پر بار ('A) لگتی ہے۔ میکس ٹرم (Maxterm) اُن قطاروں سے بنتا ہے جن کا آؤٹ پٹ 0 ہو؛ اس میں 1 والے متغیر پر بار لگتی ہے۔

### CHECK YOURSELF

1. Which comes first: AND or OR?
2. Write the minterm for  $A = 1, B = 0, C = 1$ .
3. Write the maxterm for  $A = 0, B = 1$ .

Answers: 1. AND. 2.  $AB'C$ . 3.  $A + B'$ .

## 1.1.11 Logic Diagrams ★ Golden

### LEARN IT

A **logic diagram** shows how logic gates are connected to create a circuit. It is a "picture" of a Boolean expression. To draw one, use the order of precedence: the operation with the **highest priority is drawn first** (nearest the inputs), and the last operation is the gate that gives the final output.

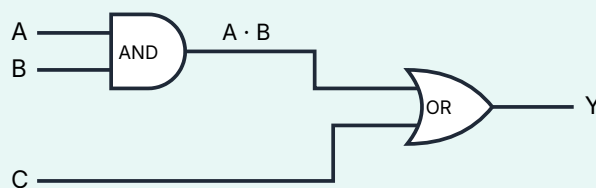
### WORKED EXAMPLE

**Example 1: Draw the logic diagram for  $Y = A \cdot B + C$ .**

Step 1: Find the operations: AND ( $\cdot$ ) and OR ( $+$ ).

Step 2: AND has higher precedence than OR, so draw the AND gate first, with inputs A and B. Its output is  $A \cdot B$ .

Step 3: Draw an OR gate. Feed it  $A \cdot B$  and C. Its output is Y.



$Y = A \cdot B + C$ : AND first, then OR.

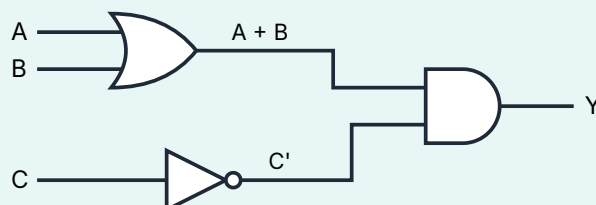
### WORKED EXAMPLE

**Example 2: Draw the logic diagram for  $Y = (A + B) \cdot C'$ .**

Step 1: Parentheses have the highest priority, so draw the OR gate for  $A + B$  first.

Step 2: NOT is next: draw a NOT gate on input C to get  $C'$ .

Step 3: The last operation is AND: feed  $(A + B)$  and  $C'$  into an AND gate to get Y.



$Y = (A + B) \cdot C'$ : OR and NOT first, then AND.

You can check this circuit against the truth table in 1.1.8 (worked example 2):  $Y = 1$  only for rows 010, 100 and 110.

#### REMEMBER IT

Read the expression and ask "What is done **last**?" That gate goes on the far right, next to Y. Work backwards to the inputs. Count gates:  $Y = A \cdot B + C$  needs 2 gates;  $Y = (A+B) \cdot C'$  needs 3.

#### COMMON MISTAKE

Do not draw a separate wire for a variable each time it appears and then leave one unconnected. Every gate input must be joined to an input variable or to another gate's output, and every wire should be clearly labelled in exams.

#### اردو میں سمجھیں

لاجک ڈایاگرام دکھاتا ہے کہ گیٹس آپس میں کیسے جڑ کر سرکٹ بناتے ہیں۔ ترجیحی ترتیب کے مطابق جو آپریشن پہلے ہوتا ہے اس کا گیٹ پہلے (ان پٹ کے قریب) بنایا جاتا ہے، اور آخری آپریشن کا گیٹ آؤٹ پٹ Y دیتا ہے۔ مثلاً  $Y = A \cdot B + C$  میں پہلے AND گیٹ اور پھر OR گیٹ بنتا ہے۔

#### EXAM TIP

Typical question: "Draw the logic diagram of  $Y = \dots$ " and sometimes "also make its truth table" (4–5 marks). Write the precedence steps in one line, draw neat gate symbols with labelled intermediate outputs (like  $A \cdot B$ ), and finish with the truth table. Marks are lost for wrong gate shapes and missing bubbles.

#### CHECK YOURSELF

1. In  $Y = A + B \cdot C$ , which gate is drawn first?
2. How many gates are needed for  $Y = A' \cdot B$ ?

Answers: 1. The AND gate (for  $B \cdot C$ ). 2. Two: one NOT gate (for  $A'$ ) and one AND gate.

### 1.1.13 Karnaugh Maps (K-Maps) ★ Golden

#### LEARN IT

A **Karnaugh map (K-map)** is a graphical tool used to **simplify Boolean expressions**. It is a grid of cells — one cell for each row of the truth table. The cells are arranged so that **neighbouring (adjacent) cells differ by only ONE bit**. This ordering is called **Gray code**.

You write the output (1 or 0) of each row into its cell, then **group the 1s** in rectangles of size 1, 2, 4 or 8 (powers of 2). Each group becomes one simple term. This simplifies the equation without long algebra.

#### Two-variable K-map (A, B)

2 variables need  $2^2 = 4$  cells.

|       | B = 0             | B = 1            |
|-------|-------------------|------------------|
| A = 0 | Cell 0 ( $A'B'$ ) | Cell 1 ( $A'B$ ) |
| A = 1 | Cell 2 ( $AB'$ )  | Cell 3 ( $AB$ )  |

### Three-variable K-map (A, B, C)

3 variables need  $2^3 = 8$  cells. The columns follow the Gray code sequence **00, 01, 11, 10** (not 00, 01, 10, 11). This makes adjacent cells differ by only one variable.

|       | BC = 00        | BC = 01       | BC = 11      | BC = 10       |
|-------|----------------|---------------|--------------|---------------|
| A = 0 | $m_0$ (A'B'C') | $m_1$ (A'B'C) | $m_3$ (A'BC) | $m_2$ (A'BC') |
| A = 1 | $m_4$ (AB'C')  | $m_5$ (AB'C)  | $m_7$ (ABC)  | $m_6$ (ABC')  |

#### LEARN IT

##### Grouping rules

1. Group only 1s. Groups must be rectangles of size 1, 2, 4 or 8 — never 3, 5 or 6.
2. Make groups **as large as possible**. Bigger groups give simpler terms.
3. Use **as few groups as possible**, but every 1 must be in at least one group.
4. **Overlapping is allowed** — a 1 may be used in more than one group.
5. **Wrapping around the edges is allowed**: a cell in the far-left column is adjacent to the cell in the far-right column (BC = 00 is next to BC = 10).
6. To read a group: variables that **change** (0 to 1) inside the group are **eliminated**. Variables that stay the same are kept — plain if they stay 1, complemented if they stay 0.

Group size tells you how many variables disappear: in a 3-variable map, a group of 2 leaves 2 variables, a group of 4 leaves 1 variable, and a group of 8 means  $Y = 1$ .

#### WORKED EXAMPLE

##### K-map example 1 (2 variables): simplify $Y = A'B + AB$ .

Step 1: The minterms are A'B (cell 1) and AB (cell 3). Put 1 in those cells, 0 elsewhere.

|       | B = 0 | B = 1 |
|-------|-------|-------|
| A = 0 | 0     | 1     |
| A = 1 | 0     | 1     |

Step 2: The two 1s are in the same column, one above the other. Make one group of 2.

Step 3: Inside the group, A changes (0 then 1) → eliminate A. B stays 1 → keep B.

**Result:  $Y = B$ .** This agrees with the algebra in 1.1.4 and the minterm table in 1.1.8.

#### WORKED EXAMPLE

##### K-map example 2 (2 variables, overlapping): simplify $Y = A'B' + A'B + AB$ .

Step 1: Put 1s in cells 0, 1 and 3.

|       | B = 0 | B = 1 |
|-------|-------|-------|
| A = 0 | 1     | 1     |
| A = 1 | 0     | 1     |

Step 2: Group 1 = the top row (cells 0 and 1). A stays 0, B changes → term **A'**.

Step 3: Group 2 = the right column (cells 1 and 3). A changes, B stays 1 → term **B**. Cell 1 is used twice — overlapping is allowed.

**Result:  $Y = A' + B$ .** Check:  $A' + B$  is 0 only when  $A = 1$  and  $B = 0$ , which is exactly the one 0 cell (cell 2). ✓

### WORKED EXAMPLE

**K-map example 3 (3 variables):** simplify the majority function  $Y = \Sigma m(3, 5, 6, 7) = A'BC + AB'C + ABC' + ABC$ .

Step 1: Fill the map. Careful — the columns are in Gray order 00, 01, 11, 10, so  $m_3$  is in the third column and  $m_2$  in the fourth.

|       | BC = 00 | BC = 01     | BC = 11     | BC = 10     |
|-------|---------|-------------|-------------|-------------|
| A = 0 | 0       | 0           | 1 ( $m_3$ ) | 0           |
| A = 1 | 0       | 1 ( $m_5$ ) | 1 ( $m_7$ ) | 1 ( $m_6$ ) |

Step 2: No group of 4 is possible (there is no  $2 \times 2$  block or full row of 1s). So make groups of 2, each using  $m_7$ :

- Group 1:  $m_3 + m_7$  (column BC = 11). A changes; B = 1, C = 1 stay  $\rightarrow$  **BC**.
- Group 2:  $m_5 + m_7$  (row A = 1, columns 01 and 11). B changes; A = 1, C = 1 stay  $\rightarrow$  **AC**.
- Group 3:  $m_7 + m_6$  (row A = 1, columns 11 and 10). C changes; A = 1, B = 1 stay  $\rightarrow$  **AB**.

**Result:  $Y = AB + BC + AC$ .** Four 3-variable terms (needing 4 three-input AND gates) became three 2-variable terms.

Check one row: A=1, B=0, C=1  $\rightarrow$  AB = 0, BC = 0, AC = 1  $\rightarrow$  Y = 1. In the truth table, row 5 is 1. ✓

### WORKED EXAMPLE

**K-map example 4 (3 variables, wrap-around):** simplify  $Y = \Sigma m(0, 2, 4, 6)$ .

|       | BC = 00 | BC = 01 | BC = 11 | BC = 10 |
|-------|---------|---------|---------|---------|
| A = 0 | 1       | 0       | 0       | 1       |
| A = 1 | 1       | 0       | 0       | 1       |

Step 1: The 1s are in the far-left column (BC = 00) and the far-right column (BC = 10).

Step 2: Because the map wraps around, these two columns are adjacent. Make **one group of 4**:  $m_0, m_4, m_2, m_6$ .

Step 3: In this group A changes (0, 1) and B changes (0, 1). Only C stays the same: C = 0  $\rightarrow$  term **C'**.

**Result:  $Y = C'$ .** Without wrapping you would get two groups ( $B'C' + BC'$ ), which is correct but not the simplest.

### WORKED EXAMPLE

**K-map example 5 (3 variables):** simplify  $Y = \Sigma m(1, 4, 5, 6, 7)$  — this is the output of  $Y = A + B'C$  from 1.1.8.

|       | BC = 00 | BC = 01 | BC = 11 | BC = 10 |
|-------|---------|---------|---------|---------|
| A = 0 | 0       | 1       | 0       | 0       |
| A = 1 | 1       | 1       | 1       | 1       |

Step 1: Group of 4: the whole row A = 1 ( $m_4, m_5, m_7, m_6$ ). B and C both change; A stays 1  $\rightarrow$  **A**.

Step 2:  $m_1$  is still alone. Pair it with  $m_5$  below it (column BC = 01). A changes; B = 0, C = 1 stay  $\rightarrow$  **B'C**.

**Result:  $Y = A + B'C$**  — the K-map confirms the original expression was already in its simplest form.

### WORKED EXAMPLE

**Going further — a 4-variable K-map: simplify  $Y = \Sigma m(0, 2, 5, 7, 8, 10, 13, 15)$ .** (Not required by the slides, but a good test of the same rules.) 4 variables need 16 cells; both rows (AB) and columns (CD) use Gray order 00, 01, 11, 10.

|         | CD = 00     | CD = 01        | CD = 11        | CD = 10        |
|---------|-------------|----------------|----------------|----------------|
| AB = 00 | 1 ( $m_0$ ) | 0              | 0              | 1 ( $m_2$ )    |
| AB = 01 | 0           | 1 ( $m_5$ )    | 1 ( $m_7$ )    | 0              |
| AB = 11 | 0           | 1 ( $m_{13}$ ) | 1 ( $m_{15}$ ) | 0              |
| AB = 10 | 1 ( $m_8$ ) | 0              | 0              | 1 ( $m_{10}$ ) |

Step 1: The centre  $2 \times 2$  block ( $m_5, m_7, m_{13}, m_{15}$ ) is a group of 4. A and C change; B = 1 and D = 1 stay  $\rightarrow$  **BD**.

Step 2: The four corners ( $m_0, m_2, m_8, m_{10}$ ) are adjacent through wrap-around (left-right and top-bottom). A and C change; B = 0 and D = 0 stay  $\rightarrow$  **B'D'**.

**Result:  $Y = BD + B'D'$**  (this is B XNOR D). Eight 4-variable minterms became two 2-variable terms.

### REMEMBER IT

**Gray code song:** "zero-zero, zero-one, one-one, one-zero" — only one bit flips at each step. Imagine the K-map is drawn on a tube, so the left and right edges touch. For reading groups: "**If it changes, it goes.**"

### COMMON MISTAKE

Most common K-map mistakes: (1) writing the columns as 00, 01, 10, 11 — this puts  $m_2$  and  $m_3$  in the wrong places; (2) making groups of 3 or 6 — only 1, 2, 4, 8 are allowed; (3) grouping diagonally — groups must be rectangles of side-by-side cells; (4) stopping at small groups when a bigger group (or a wrap-around group) is possible.

### اردو میں سمجھیں

کارنو میپ (K-Map) بولین ایکسپریشن کو آسان بنانے کا تصویری طریقہ ہے۔ اس کے ساتھ والے خانوں میں صرف ایک بٹ کا فرق ہوتا ہے (گرے کوڈ: 00, 01, 10, 11)۔ 1 والے خانوں کو 0, 1, 2, 4 یا 8 کے گروپوں میں جتنا بڑا ممکن ہو جوڑیں؛ گروپ ایک دوسرے پر چڑھ سکتے ہیں اور کناروں سے گھوم کر بھی بن سکتے ہیں۔ گروپ کے اندر جو متغیر بدلتا ہے وہ ختم ہو جاتا ہے۔

### EXAM TIP

K-maps are a favourite long question (5–6 marks): "Simplify  $F(A,B,C) = \Sigma m(\dots)$  using a K-map." Draw the map with labelled Gray-code headings, fill every cell (write the 0s too), circle the groups clearly, write the term for each group beside it, and give the final simplified expression. Examiners often add "draw the logic diagram of the result" — so practise drawing it too.

### CHECK YOURSELF

1. In a 3-variable K-map, which column is next to BC = 01 on the right?
2. Simplify  $Y = \Sigma m(2, 3)$  for two variables A, B.
3. Simplify  $Y = \Sigma m(4, 5, 6, 7)$  for three variables.

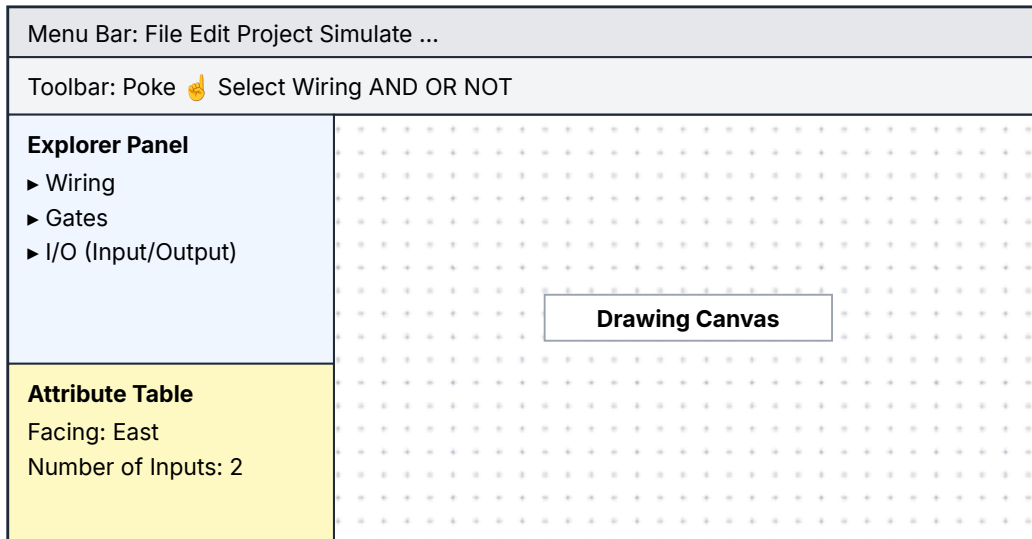
Answers: 1. BC = 11. 2. Cells 2 and 3 are the row A = 1; B changes  $\rightarrow Y = A$ . 3. The whole row A = 1 is a group of 4  $\rightarrow Y = A$ .

# LogiSim Evolution: Simulation Tool

## LEARN IT

**LogiSim Evolution** is a free graphical tool for **designing and simulating logic circuits**. "Simulating" means the computer pretends to be the real circuit: you can switch inputs on and off and watch the output change. It lets you see the truth tables and Boolean expressions you learn in theory.

## 1. The interface areas



The main areas of the LogiSim Evolution window.

| Area                          | What it does                                                                           |
|-------------------------------|----------------------------------------------------------------------------------------|
| Menu Bar (top)                | File, Edit, Project, Simulate menus — open/save files and control the simulation.      |
| Toolbar                       | Quick access to standard tools: Wiring, Poke, and AND, OR, NOT gates.                  |
| Explorer Panel (left)         | Contains the component libraries: Gates, I/O, Wiring.                                  |
| Attribute Table (bottom left) | Adjust the properties of the selected gate, e.g. Number of Inputs or Facing direction. |
| Drawing Canvas (right)        | The dotted grid where you place and wire the gates.                                    |

## Wiring colours during simulation

| Colour       | Meaning                   |
|--------------|---------------------------|
| Bright green | Logic 1 (HIGH)            |
| Dark green   | Logic 0 (LOW)             |
| Blue         | Unknown / unconnected pin |
| Red          | Error / short circuit     |

## REMEMBER IT

**Colours like a traffic light:** Green means "working" (bright = 1, dark = 0). Blue means "the wire is lonely — not connected". Red means "stop, something is wrong".

### COMMON MISTAKE

If your output wire is blue, the circuit is not wrong in logic — a wire is simply not touching a pin. Zoom in and make sure every wire end sits exactly on a connection point.

### اردو میں سمجھیں

لاجی سیم ایوولوشن (LogiSim Evolution) لاجک سرکٹ بنانے اور چلا کر دیکھنے کا سافٹ ویئر ہے۔ اس میں مینو بار، ٹول بار، ایکسپلورر پینل، ایٹری بیوٹ ٹیبل اور ڈرائنگ کینوس ہوتے ہیں۔ چمکدار سبز تار 1، گہری سبز 0، نیلی تار غیر منسلک اور سرخ تار غلطی کو ظاہر کرتی ہے۔

### CHECK YOURSELF

1. Where do you change the number of inputs of a gate?
2. What does a red wire mean?

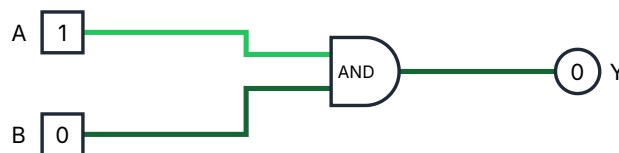
Answers: 1. In the Attribute Table (bottom left). 2. An error or short circuit.

## Building a Circuit in LogiSim

### WORKED EXAMPLE

#### Step by step: building $Y = A \cdot B$

1. Select the **AND gate** from the Toolbar (or from the Gates library) and place it on the canvas.
2. Select the **Input Pin** (square icon with a 0) and place two pins to the left of the gate. Label them A and B.
3. Select the **Output Pin** (circle icon with a 0) and place it to the right. Label it Y.
4. Use the **Wiring Tool** to click and drag wires from the input pins to the AND gate's input ports. Wires must be horizontal or vertical.
5. Wire the gate's output to the output pin Y.
6. Select the **Poke Tool** (pointing finger) and click inputs A and B to toggle them between 0 and 1. Test the circuit!



Bright green = 1, dark green = 0

Testing with the Poke Tool: A = 1 (bright green wire), B = 0 (dark green), so  $Y = 1 \cdot 0 = 0$ .

### WORKED EXAMPLE

**Testing the circuit = making its truth table.** Poke the inputs through all four combinations and write what Y shows:

| Poke A | Poke B | Y shows | Output wire colour |
|--------|--------|---------|--------------------|
| 0      | 0      | 0       | Dark green         |
| 0      | 1      | 0       | Dark green         |
| 1      | 0      | 0       | Dark green         |
| 1      | 1      | 1       | Bright green       |

This matches the AND truth table, so the circuit is correct. Try the same with  $Y = A \cdot B + C$  from 1.1.11 (add a second gate and a third input pin).



#### REMEMBER IT

Input pin = **square** (you push it). Output pin = **circle** (it shows a light). Poke = finger — you "poke" inputs to change them.

#### COMMON MISTAKE

Students often try to poke inputs while the Wiring or Select tool is active — then clicking moves the pin instead of changing it. Switch to the Poke Tool first.

#### اردو میں سمجھیں

لاجی سیم میں  $Y = A \cdot B$  بنانے کے لیے AND گیٹ رکھیں، بائیں جانب دو ان پٹ پن (چوکور) اور دائیں جانب ایک آؤٹ پٹ پن (گول) رکھیں، پھر وائرنگ ٹول سے تاریں جوڑیں۔ آخر میں پوک ٹول (انگلی) سے ان پٹ کو 0 اور 1 کر کے سرکٹ کو جانچیں۔

#### CHECK YOURSELF

1. Which icon is the input pin: square or circle?
2. Which tool do you use to change an input between 0 and 1?

Answers: 1. Square (output pin is the circle). 2. The Poke Tool (pointing finger).

## 1.2.1 – 1.2.3 Software Development Life Cycle (SDLC) ★ Golden

#### LEARN IT

The **Software Development Life Cycle (SDLC)** is a structured process that guides software development from beginning to end. It ensures **clear goals, better planning, risk reduction and high quality**. Without it, teams forget features, go over budget and deliver buggy software.

SDLC has **6 phases**:



### Phases 1–3 in detail

| Phase                   | Activities and details                                                                                                                        | Deliverable (what it produces)                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 1. Requirement Analysis | Meet clients, understand their needs, list the features. This is the <b>most important</b> phase — mistakes here spread to every later phase. | SRS Document (Software Requirements Specification) |
| 2. System Design        | Create the High-Level Design (overall architecture), the Low-Level Design (database, UI screens) and flowcharts.                              | Design Document                                    |
| 3. Implementation       | Write the actual code in programming languages (Python, Java, etc.) based on the design documents.                                            | Working Source Code                                |

## Phases 4–6 in detail

| Phase          | Activities and details                                               | Deliverable                     |
|----------------|----------------------------------------------------------------------|---------------------------------|
| 4. Testing     | Find and fix bugs (errors), using Black Box and White Box testing.   | Test Reports, bug-free software |
| 5. Deployment  | Release the software to end-users: Direct, Phased, or Pilot release. | Live Software                   |
| 6. Maintenance | Provide support and updates, and fix post-release issues.            | Patches, Updated Versions       |

**Deployment methods:** In a **direct** release the old system is switched off and the new one switched on at once. In a **phased** release the new system is introduced part by part. In a **pilot** release one small group (for example one branch or one class) uses it first before everyone else.

## Testing methods

### Black Box Testing

- The tester does **not** see the code.
- Tests functionality: give inputs, check outputs.
- Done by dedicated testers.

### White Box Testing

- The tester **can** see the code.
- Tests the internal logic and structure.
- Done by developers.

#### WORKED EXAMPLE

##### Building a house = SDLC.

Phase 1 (Requirements): you ask the family what rooms they need — 3 bedrooms, a kitchen, a garage.

Phase 2 (Design): the architect draws the blueprints.

Phase 3 (Implementation): the builders lay the bricks.

Phase 4 (Testing): an inspector checks the wiring and the plumbing.

Phase 5 (Deployment): the family moves in.

Phase 6 (Maintenance): repairs, painting and a new room added later.

#### WORKED EXAMPLE

**Black box vs white box on a login screen.** A black-box tester types a correct password and a wrong password and checks that the first is accepted and the second rejected — without looking at the code. A white-box tester (the developer) reads the code and checks that every `if` branch, for example the "account locked after 3 tries" branch, actually runs correctly.

#### REMEMBER IT

Remember the order with "**R**ead **D**evelopers **I**mplement, **T**est, **D**eploy, **M**aintain" (R-D-I-T-D-M). Black box = a closed box (you can't see inside). White box = a glass box (you can see inside).

#### COMMON MISTAKE

Do not say the SDLC ends at deployment. Maintenance is a full phase and often lasts the longest. Also, the SRS is the deliverable of Requirement Analysis, not of Design.

ایس ڈی ایل سی (SDLC) سافٹ ویئر بنانے کا منظم طریقہ ہے جس کے چھ مراحل ہیں: ضروریات کا تجزیہ، سسٹم ڈیزائن، کوڈنگ (Implementation)، ٹیسٹنگ، تنصیب (Deployment) اور دیکھ بھال (Maintenance)۔ پہلا مرحلہ سب سے اہم ہے اور اس سے SRS دستاویز بنتی ہے۔ بلیک باکس ٹیسٹنگ میں کوڈ نظر نہیں آتا جبکہ وائٹ باکس ٹیسٹنگ میں کوڈ دیکھ کر اندرونی منطق جانچی جاتی ہے۔

#### EXAM TIP

"Explain the phases of SDLC" is a very common long question (5–8 marks). Name all 6 phases in order, write 1–2 lines of activities for each, and state the deliverable of each (SRS, Design Document, Source Code, Test Reports, Live Software, Patches). A short question often asks you to differentiate black box and white box testing — answer in a 3-row table (sees code?, what is tested?, who does it?).

#### CHECK YOURSELF

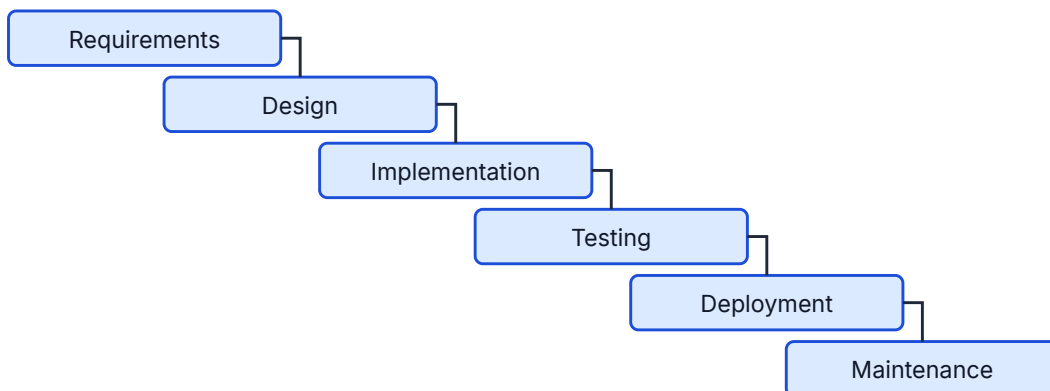
1. Which phase produces the SRS document?
2. Who usually performs white box testing?
3. Name the three ways to deploy software.

Answers: 1. Requirement Analysis. 2. Developers. 3. Direct, Phased and Pilot release.

## 1.2.4 The Waterfall Model ★ Golden

#### LEARN IT

The **Waterfall model** is a **linear, sequential** approach to the SDLC. Each phase must be completed before the next one begins, like water falling down steps. There is no easy way to go back to a previous phase.



The Waterfall model: each phase "falls" into the next, in one direction only.

### When to use it

- Requirements are clear, fixed and well understood.
- The project is small to medium in size.
- The technology is stable (not new or experimental).

| Advantages (Pros)                | Disadvantages (Cons)                      |
|----------------------------------|-------------------------------------------|
| Simple to understand and manage. | Not flexible to changes.                  |
| Well documented at every step.   | Testing happens very late in the process. |

#### WORKED EXAMPLE

**Case study: Al-Noor Library Management System.** The textbook uses this system as a Waterfall case study because the requirements were **100% fixed before coding began**.

Step 1: The librarian listed exactly what was needed: add books, issue books, return books, calculate fines. Nothing would change.

Step 2: Because the list was fixed, the team could finish the full design, then all the coding, then all the testing, in order.

Step 3: Every phase produced a document, so the library has a complete record for future maintenance.

#### REMEMBER IT

**Waterfall = water only flows downwards.** Once water has fallen it cannot climb back up — once a phase is finished you do not go back.

#### COMMON MISTAKE

Do not choose Waterfall for a project where the customer is still unsure what they want. A late change means going back through design, coding and testing, which is slow and costly.

#### اردو میں سمجھیں

وائرفال (آبشار) ماڈل ایک سیدھا اور ترتیب وار طریقہ ہے؛ ہر مرحلہ مکمل ہونے کے بعد ہی اگلا شروع ہوتا ہے اور پیچھے جانا مشکل ہوتا ہے۔ یہ تب بہتر ہے جب ضروریات واضح اور طے شدہ ہوں، جیسے النور لائبریری مینجمنٹ سسٹم میں۔ یہ سمجھنے میں آسان ہے مگر تبدیلیوں کے لیے لچکدار نہیں اور ٹیسٹنگ دیر سے ہوتی ہے۔

#### EXAM TIP

Expect "Explain the Waterfall model with its advantages and disadvantages" or "When should the Waterfall model be used?" Draw the step diagram, list the three "when to use" conditions, and give at least two pros and two cons. Mention the Al-Noor Library case study as your example.

#### CHECK YOURSELF

1. Why is the Waterfall model called "sequential"?
2. Why did the Al-Noor Library system use Waterfall?

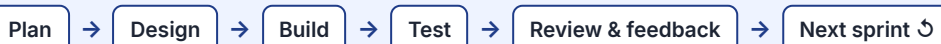
Answers: 1. Because each phase must be completed before the next begins, in a fixed order. 2. Because its requirements were 100% fixed before coding began.

## 1.2.4 The Agile Model ★ Golden

#### LEARN IT

The **Agile model** is an **iterative** approach: the software is built in small, repeated cycles called **sprints** (usually 2–4 weeks long). Each sprint plans, builds and tests a small part of the software, and then shows it to the customer.

**Customer feedback is gathered continuously** and used in the next sprint.



## When to use it

- Requirements are unclear or may change.
- The customer wants frequent progress updates (early delivery).
- The project is complex or innovative.

| Advantages (Pros)                   | Disadvantages (Cons)             |
|-------------------------------------|----------------------------------|
| Highly flexible to changes.         | Less documentation.              |
| Early delivery of working software. | Unpredictable end-time and cost. |

### WORKED EXAMPLE

**Case study: Al-Noor Student Mobile Application.** This app used Agile because teachers, parents and students **constantly requested new features** during development.

Sprint 1 (2 weeks): build login and the timetable screen. Show it to users.

Feedback: parents ask, "Can we see attendance too?"

Sprint 2: add attendance. Teachers now ask for homework notifications.

Sprint 3: add notifications. Each sprint delivers a working app, and new ideas are welcomed instead of causing a restart.

### REMEMBER IT

**Agile = agile athlete:** quick and able to change direction. Think of making a pizza where the customer tastes each slice and you adjust the next one.

### COMMON MISTAKE

"Less documentation" does not mean "no documentation" or "no planning". Agile teams still plan every sprint; they simply value working software over long documents.

### اردو میں سمجھیں

ایجائل (Agile) ماڈل میں سافٹ ویئر چھوٹے چھوٹے ادوار میں بنتا ہے جنہیں اسپرینٹ (Sprint) کہتے ہیں، جو عموماً دو سے چار ہفتے کے ہوتے ہیں۔ ہر اسپرینٹ کے بعد گاہک کی رائے لی جاتی ہے۔ یہ تبدیلیوں کے لیے بہت لچکدار ہے اور جلد کام کرنے والا سافٹ ویئر دیتا ہے، مگر اس میں دستاویزات کم ہوتی ہیں اور وقت و لاگت کا اندازہ مشکل ہوتا ہے۔ النور اسٹوڈنٹ موبائل ایپ اس کی مثال ہے۔

### EXAM TIP

Questions ask you to define Agile, explain sprints, list pros/cons, or say when to use it. Always mention the words "iterative", "sprints (2–4 weeks)" and "continuous customer feedback". Use the Al-Noor Student Mobile App as your case study.

### CHECK YOURSELF

1. What is a sprint and how long does it usually last?
2. Give one disadvantage of Agile.

Answers: 1. A short development cycle that delivers a working part of the software; usually 2–4 weeks. 2. Less documentation, or unpredictable end-time and cost.

## 1.2.5 – 1.2.7 Waterfall vs Agile: Comparison and Case Studies

### LEARN IT

Both models follow the same SDLC phases. The difference is **how** they move through them: Waterfall goes through each phase **once**, in a straight line; Agile goes through all phases **many times**, in small loops.

| Dimension            | Waterfall model                                          | Agile model                                                 |
|----------------------|----------------------------------------------------------|-------------------------------------------------------------|
| Flexibility          | Rigid. Hard to make changes later.                       | Very flexible. Adapts to changes easily in the next sprint. |
| Customer involvement | Only at the start (requirements) and the end (delivery). | Continuous throughout the project (after every sprint).     |
| Delivery of software | Single delivery at the very end.                         | Frequent, partial deliveries after every sprint.            |
| Testing              | Done late, in a dedicated testing phase.                 | Continuous testing within every sprint.                     |
| Documentation        | Heavy documentation required.                            | Light documentation. Focus is on working software.          |

### WORKED EXAMPLE

**Choosing a model — think like the textbook.**

**Case A:** Al-Noor Library Management System. Requirements fixed 100% before coding → no changes expected → **Waterfall**.

**Case B:** Al-Noor Student Mobile Application. Teachers, parents and students keep asking for features → requirements change → **Agile**.

**Practice case:** A bank wants a salary-calculation program for a fixed government formula that will not change. Requirements are clear and fixed, the project is medium-sized, and the technology is stable → **Waterfall**.

**Practice case:** A start-up wants a new food-delivery app and is not sure which features customers will like. Requirements are unclear and the project is innovative → **Agile**.

### REMEMBER IT

One question decides it: **"Will the requirements change?"** No → Waterfall. Yes / not sure → Agile.

### COMMON MISTAKE

Do not say one model is always "better". Each suits a different situation. In exams, justify your choice using the project's features (fixed vs changing requirements, size, customer involvement).

### اردو میں سمجھیں

وائرفال ماڈل سخت (Rigid) ہے، گاہک صرف شروع اور آخر میں شامل ہوتا ہے، سافٹ ویئر ایک ہی بار آخر میں ملتا ہے اور دستاویزات زیادہ ہوتی ہیں۔ ایجائل ماڈل لچکدار ہے، گاہک ہر اسپرنٹ کے بعد شامل ہوتا ہے، سافٹ ویئر بار بار حصوں میں ملتا ہے اور ٹیسٹنگ مسلسل ہوتی ہے۔ اگر ضروریات طے شدہ ہوں تو وائرفال اور اگر بدلتی رہیں تو ایجائل منتخب کریں۔

#### CHECK YOURSELF

1. In which model is the customer involved only at the start and the end?
2. Which model delivers software in frequent, partial deliveries?
3. A school wants an exam-result system with rules that never change. Which model?

Answers: 1. Waterfall. 2. Agile. 3. Waterfall (fixed, clear requirements).

### 1.3.1 – 1.3.4 Communication Models and the OSI 7-Layer Model ★ Golden

#### LEARN IT

A **communication model** explains how data travels from one device to another through a network. Because sending data is complicated, the job is split into **layers**; each layer does one part of the job and passes the data to the next layer.

The **OSI (Open Systems Interconnection) model** was developed by **ISO** (International Organization for Standardization) in **1984**. It divides communication into **7 layers**. When you send data, it goes **down** the layers on your device (7 → 1), travels as bits over the cable or air, and goes **up** the layers on the receiving device (1 → 7).



The 7 layers of the OSI model with the mnemonic "Please Do Not Throw Sausage Pizza Away".

#### REMEMBER IT

**Mnemonic (bottom to top, 1 → 7):** "Please Do Not Throw Sausage Pizza Away" = Physical, Data Link, Network, Transport, Session, Presentation, Application. (Top to bottom you can use "All People Seem To Need Data Processing".)

#### OSI layers — top layers (7 to 4)

| Layer           | Function                                                                          | Protocols / examples        |
|-----------------|-----------------------------------------------------------------------------------|-----------------------------|
| 7. Application  | Provides network services directly to user applications (browser, email program). | HTTP, HTTPS, FTP, SMTP, DNS |
| 6. Presentation | Data translation, encryption (security) and compression.                          | SSL/TLS, JPEG, MP3, ASCII   |
| 5. Session      | Opens, maintains and closes communication sessions.                               | NetBIOS, RPC                |
| 4. Transport    | Reliable end-to-end delivery, error checking, port numbering.                     | TCP (reliable), UDP (fast)  |

## OSI layers — bottom layers (3 to 1)

| Layer        | Function                                                               | Protocols / hardware      |
|--------------|------------------------------------------------------------------------|---------------------------|
| 3. Network   | Logical addressing (IP addresses) and routing packets across networks. | IP, ICMP (Ping), Routers  |
| 2. Data Link | Physical addressing (MAC addresses) and error detection in frames.     | Ethernet, MAC, Switches   |
| 1. Physical  | Transmits raw bits (0, 1) over cables or wireless media.               | Cables, Hubs, Wi-Fi radio |

## TCP vs UDP at the Transport layer

|              | TCP                                                             | UDP                                           |
|--------------|-----------------------------------------------------------------|-----------------------------------------------|
| Main feature | Reliable — checks that all data arrives correctly and in order. | Fast — sends data without waiting for checks. |
| Use when     | Accuracy matters.                                               | Speed matters more than perfection.           |
| Examples     | Web browsing, emails                                            | Video streaming, online gaming                |

### WORKED EXAMPLE

#### Tracing an email through the OSI layers (sender side).

Layer 7 (Application): you press Send; your email program uses **SMTP**.

Layer 6 (Presentation): the text is encoded (ASCII) and may be encrypted with TLS.

Layer 5 (Session): a session is opened with the mail server.

Layer 4 (Transport): **TCP** breaks the data into segments and numbers them, so nothing is lost (email needs accuracy).

Layer 3 (Network): each packet gets the **IP address** of the destination; **routers** choose the path.

Layer 2 (Data Link): packets are put in **frames** with **MAC addresses**; a **switch** forwards them on the local network.

Layer 1 (Physical): the frames are sent as raw **bits** over the cable or Wi-Fi radio.

At the receiver, the same steps happen in reverse order (1 → 7).

### COMMON MISTAKE

Do not mix up the two addresses: **IP address** = logical address = **Network layer (3)**; **MAC address** = physical address = **Data Link layer (2)**. Also, routers work at layer 3, switches at layer 2 and hubs at layer 1.

### اردو میں سمجھیں

کمپونیکیشن ماڈل بتاتا ہے کہ ڈیٹا نیٹ ورک کے ذریعے ایک ڈیوائس سے دوسری تک کیسے پہنچتا ہے۔ او ایس آئی (OSI) ماڈل 1984 میں آئی ایس او (ISO) نے بنایا اور اس کی سات تہیں (Layers) ہیں: فزیکل، ڈیٹا لنک، نیٹ ورک، ٹرانسپورٹ، سیشن، پریزنٹیشن اور ایپلی کیشن۔ ہر تہ کا الگ کام ہے، جیسے نیٹ ورک تہ IP ایڈریس اور روٹنگ سنبھالتی ہے۔ TCP درستگی کے لیے اور UDP رفتار کے لیے استعمال ہوتا ہے۔



#### EXAM TIP

A guaranteed long question: "Explain the seven layers of the OSI model." Draw the 7-layer stack (numbered, layer 7 on top), then write one function and one or two protocols/devices for each layer. MCQs often ask: "Which layer handles routing?" (Network), "Which layer does encryption?" (Presentation), "Who developed OSI and when?" (ISO, 1984), "Which protocol is used for video streaming?" (UDP).

#### CHECK YOURSELF

1. Which OSI layer is responsible for encryption and compression?
2. At which layer do switches and MAC addresses work?
3. Should online gaming use TCP or UDP? Why?

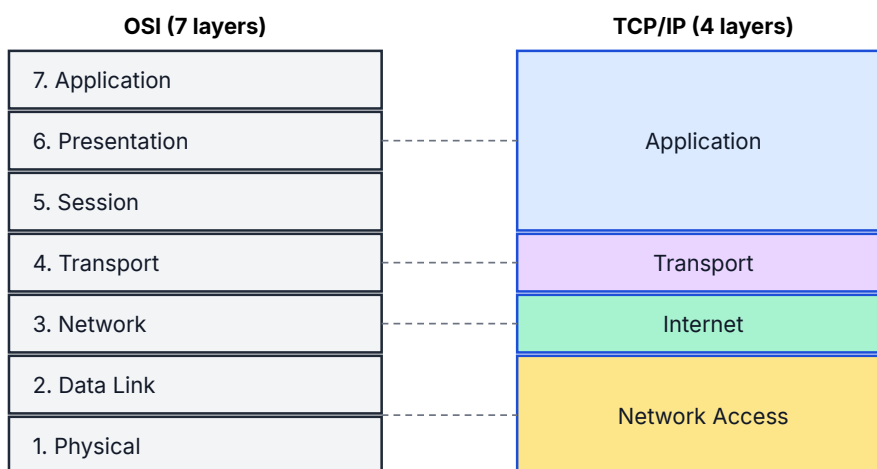
Answers: 1. Presentation layer (6). 2. Data Link layer (2). 3. UDP, because speed matters more than perfect accuracy.

## 1.3.5 – 1.3.6 TCP/IP Model vs OSI Model

#### LEARN IT

The OSI model is mainly a **reference** model used for teaching and design. The **TCP/IP model** is the **practical model used on the real Internet**. It condenses the 7 OSI layers into just **4 layers**.

| TCP/IP model (4 layers) | Corresponds to OSI layers                    |
|-------------------------|----------------------------------------------|
| 1. Application          | Application, Presentation, Session (7, 6, 5) |
| 2. Transport            | Transport (4)                                |
| 3. Internet             | Network (3)                                  |
| 4. Network Access       | Data Link, Physical (2, 1)                   |



How the 7 OSI layers map onto the 4 TCP/IP layers.

| Point            | OSI model                                             | TCP/IP model                                  |
|------------------|-------------------------------------------------------|-----------------------------------------------|
| Number of layers | 7                                                     | 4                                             |
| Use              | Reference / teaching model                            | Practical model used on the real Internet     |
| Top layers       | Separate Application, Presentation and Session layers | All three combined into one Application layer |
| Bottom layers    | Separate Data Link and Physical layers                | Combined into one Network Access layer        |

#### WORKED EXAMPLE

##### Where does each protocol/device go in TCP/IP?

HTTP (OSI layer 7) → TCP/IP **Application**.

SSL/TLS (OSI layer 6) → TCP/IP **Application** (layers 7, 6, 5 are merged).

UDP (OSI layer 4) → TCP/IP **Transport**.

IP and routers (OSI layer 3) → TCP/IP **Internet**.

Ethernet, switches, cables (OSI layers 2 and 1) → TCP/IP **Network Access**.

#### REMEMBER IT

Count it as **3-1-1-2**: the TCP/IP Application layer takes 3 OSI layers, Transport takes 1, Internet takes 1, Network Access takes 2. Total  $3+1+1+2 = 7$ .

#### COMMON MISTAKE

The OSI "Network" layer (3) matches the TCP/IP "**Internet**" layer, not the TCP/IP "Network Access" layer. The similar names confuse many students.

#### اردو میں سمجھیں

ٹی سی پی/آئی پی (TCP/IP) ماڈل حقیقی انٹرنیٹ پر استعمال ہونے والا عملی ماڈل ہے اور اس کی صرف چار تہیں ہیں۔ اس کی ایپلی کیشن تہہ OSI کی 7، 6 اور 5 تہوں کے برابر ہے، ٹرانسپورٹ تہہ 4 کے، انٹرنیٹ تہہ 3 کے، اور نیٹ ورک ایکسیس تہہ 2 اور 1 کے برابر ہے۔

#### CHECK YOURSELF

1. How many layers does the TCP/IP model have?
2. Which TCP/IP layer corresponds to OSI layers 2 and 1?
3. Which TCP/IP layer contains the OSI Session layer?

Answers: 1. Four. 2. Network Access. 3. Application.

## Chapter 1 Review

## Chapter summary

- Discrete quantities are counted (students, cars); continuous quantities are measured and can take any value (temperature, speed). Stairs vs ramp.
- A digital system uses only two values, 0 (OFF/LOW/FALSE) and 1 (ON/HIGH/TRUE), called bits. It is reliable, accurate, easy to design and programmable.
- Analog signals change smoothly (sine wave) and are affected by noise; digital signals jump between HIGH and LOW (square wave) and resist noise.
- Boolean algebra, by George Boole, works with only TRUE (1) and FALSE (0) using variables, operations and expressions.
- AND (·) gives 1 only when all inputs are 1; OR (+) gives 1 when any input is 1; NOT (') inverts its single input (unary).
- A truth table lists all  $2^n$  input combinations in binary order with their outputs.
- Logic gates are the building blocks of digital systems: AND, OR, NOT (basic); NAND, NOR (universal); XOR, XNOR (advanced).
- NAND and NOR are universal: any circuit can be built using only one of them.
- XOR gives 1 when inputs are different; XNOR gives 1 when inputs are the same.
- Order of precedence: Parentheses → NOT → AND → OR. A logic diagram is drawn in the same order.
- Minterms (SOP) come from rows with output 1; maxterms (POS) come from rows with output 0.
- K-maps use Gray code (00, 01, 11, 10); group 1s in sizes 1, 2, 4, 8, as large as possible; overlap and wrap-around are allowed; changing variables are eliminated.
- LogiSim Evolution simulates circuits: input pins are squares, output pins circles; the Poke Tool toggles inputs; bright green = 1, dark green = 0, blue = unconnected, red = error.
- SDLC has 6 phases: Requirement Analysis (SRS), Design, Implementation, Testing (black box/white box), Deployment (direct/phased/pilot), Maintenance.
- Waterfall is linear and suits fixed requirements (AI-Noor Library); Agile is iterative with 2–4 week sprints and suits changing requirements (AI-Noor Student App).
- The OSI model (ISO, 1984) has 7 layers — "Please Do Not Throw Sausage Pizza Away"; the practical TCP/IP model has 4 layers: Application, Transport, Internet, Network Access.

## Key terms

| Term                 | Meaning                                                                   | اردو                       |
|----------------------|---------------------------------------------------------------------------|----------------------------|
| Discrete quantity    | A countable quantity with separate, fixed values.                         | منفصل مقدار                |
| Continuous quantity  | A quantity that can take any value within a range.                        | مسلسل مقدار                |
| Bit                  | A binary digit, 0 or 1.                                                   | بائری ہندسہ                |
| Digital system       | An electronic system that uses only the values 0 and 1.                   | ڈیجیٹل نظام                |
| Analog signal        | A continuous signal that changes smoothly, like a sine wave.              | اینالوگ سگنل (مسلسل اشارہ) |
| Digital signal       | A signal with only two levels, HIGH and LOW (square wave).                | ڈیجیٹل سگنل                |
| Noise                | Unwanted interference that disturbs a signal.                             | شور (مداخلت)               |
| Boolean algebra      | Algebra of two values, TRUE (1) and FALSE (0), developed by George Boole. | بولین الجبرا               |
| Truth table          | A table of all input combinations and their outputs.                      | ٹروٹھ ٹیبل (صداقت جدول)    |
| Logic gate           | An electronic circuit that performs a Boolean operation.                  | منطقی گیٹ                  |
| Universal gate       | A gate (NAND or NOR) from which any circuit can be built.                 | عالمگیر گیٹ                |
| Unary operation      | An operation with only one operand, e.g. NOT.                             | یک عددی عمل                |
| Minterm              | A product (AND) of all variables for a row with output 1.                 | منٹرم                      |
| Maxterm              | A sum (OR) of all variables for a row with output 0.                      | میکس ٹرم                   |
| SOP                  | Sum of Products: minterms joined with OR.                                 | حاصل ضرب کا مجموعہ         |
| POS                  | Product of Sums: maxterms joined with AND.                                | مجموعوں کا حاصل ضرب        |
| Karnaugh map (K-map) | A grid used to simplify Boolean expressions by grouping 1s.               | کارنو نقشہ                 |
| Gray code            | An ordering where neighbouring values differ by only one bit.             | گرے کوڈ                    |
| Simulation           | Running a computer model of a circuit to test it.                         | نقلی عمل (سمیولیشن)        |

| Term              | Meaning                                                                          | اردو                            |
|-------------------|----------------------------------------------------------------------------------|---------------------------------|
| SDLC              | Software Development Life Cycle: the 6-phase process for building software.      | سافٹ ویئر کی تیاری کا دورِ حیات |
| SRS               | Software Requirements Specification: document listing what the software must do. | سافٹ ویئر ضروریات کی دستاویز    |
| Black box testing | Testing inputs and outputs without seeing the code.                              | بلیک باکس جانچ                  |
| White box testing | Testing the internal logic while seeing the code.                                | وائٹ باکس جانچ                  |
| Waterfall model   | A linear SDLC model; each phase is finished before the next.                     | آبشار ماڈل                      |
| Agile model       | An iterative SDLC model using short sprints and constant feedback.               | ایجائل (لچکدار) ماڈل            |
| Sprint            | A short development cycle in Agile, usually 2–4 weeks.                           | اسپرینٹ (مختصر ترقیاتی دور)     |
| Protocol          | A set of rules for communication, e.g. HTTP, TCP.                                | پروٹوکول (ضوابطِ ابلاغ)         |
| OSI model         | ISO's 7-layer reference model of network communication (1984).                   | او ایس آئی ماڈل                 |
| TCP/IP model      | The practical 4-layer model used on the Internet.                                | ٹی سی پی/آئی پی ماڈل            |

## Practice: multiple choice

1. Which of the following is a continuous quantity?
  - a. Number of students in a class
  - b. Number of cars in a parking lot
  - c. Temperature of water
  - d. Number of books on a shelf
2. In a digital system, the value 1 represents:
  - a. OFF
  - b. LOW
  - c. FALSE
  - d. ON / HIGH / TRUE
3. A square wave represents a:
  - a. Digital signal
  - b. Analog signal
  - c. Sine wave
  - d. Noise signal
4. Boolean algebra was developed by:
  - a. Charles Babbage
  - b. George Boole
  - c. Alan Turing
  - d. Maurice Karnaugh
5. How many rows does a truth table with 4 inputs have?
  - a. 4
  - b. 8
  - c. 16
  - d. 32
6. Which gate gives output 1 only when all inputs are 1?
  - a. OR
  - b. AND
  - c. NOR
  - d. XOR
7. Which pair of gates are called universal gates?
  - a. AND and OR
  - b. XOR and XNOR
  - c. NOT and AND
  - d. NAND and NOR
8. What is the output of a NOR gate when  $A = 0$  and  $B = 0$ ?
  - a. 1
  - b. 0
  - c. Unknown
  - d. Depends on the gate size
9. An XOR gate gives output 1 when:
  - a. Both inputs are 1
  - b. Both inputs are 0
  - c. The inputs are different
  - d. The inputs are the same
10. After parentheses, which operation has the highest precedence?
  - a. OR
  - b. NOT
  - c. AND
  - d. XOR
11. The minterm for the row  $A = 1, B = 0$  is:
  - a.  $AB'$
  - b.  $A'B$
  - c.  $A + B'$
  - d.  $A' + B$
12. The columns of a 3-variable K-map (BC) are written in the order:
  - a. 00, 01, 10, 11
  - b. 00, 10, 01, 11
  - c. 11, 10, 01, 00
  - d. 00, 01, 11, 10
13. In black box testing, the tester:
  - a. Reads the source code line by line
  - b. Does not see the code and tests inputs/outputs
  - c. Is always the developer
  - d. Tests only the internal structure
14. Which OSI layer handles logical (IP) addressing and routing?
  - a. Data Link
  - b. Transport
  - c. Network
  - d. Session
15. The TCP/IP Network Access layer corresponds to which OSI layers?
  - a. 7, 6 and 5
  - b. 4
  - c. 3
  - d. 2 and 1

### Practice: short questions

1. Define discrete and continuous quantities. Give two examples of each.
2. State four advantages of digital systems.
3. Differentiate between analog and digital signals (any three points).
4. Why is NOT called a unary operation? Write its truth table.
5. How many rows are needed in a truth table for 3 inputs? List all the input combinations.
6. Why are NAND and NOR called universal gates? Show how a NAND gate can be used as a NOT gate.
7. A 2-input truth table has  $Y = 1$  only for rows  $A=0, B=1$  and  $A=1, B=0$ . Write the SOP expression and name the gate.
8. Name the deliverable of each SDLC phase.
9. Differentiate between black box testing and white box testing.
10. When should TCP be used and when should UDP be used? Give one example of each.

### Practice: long / problem questions

1. Draw the symbol, write the Boolean expression and give the truth table for each of these gates: AND, OR, NOT, NAND, NOR, XOR and XNOR.
2. A function is given as  $F(A, B, C) = \sum m(1, 3, 5, 6, 7)$ . (a) Make its truth table. (b) Simplify it using a 3-variable K-map, explaining each group. (c) Draw the logic diagram of the simplified expression.
3. For  $Y = A \cdot B' + C$ : (a) state the order in which the operations are done; (b) make the complete truth table; (c) draw the logic diagram.
4. Explain the six phases of the Software Development Life Cycle with the activities and deliverable of each phase. Use the house-building analogy.
5. Compare the Waterfall and Agile models under five headings. Explain why the Al-Noor Library Management System used Waterfall and the Al-Noor Student Mobile Application used Agile.
6. Explain the seven layers of the OSI model with the function and one protocol/device for each layer. Then show how they map onto the four layers of the TCP/IP model.

## Answer key

---

### Multiple choice

1. c
2. d
3. a
4. b
5. c
6. b
7. d
8. a
9. c
10. b
11. a
12. d
13. b
14. c
15. d

## Short questions — model answers

1. Discrete quantities are separate, countable items with specific values (number of students in a classroom, number of cars in a parking lot). Continuous quantities can take any value within a range and flow without breaks (temperature of water, speed of a moving car).
2. Reliable (less affected by noise and interference); accurate (no loss of quality when copying); easy to design (only two states, 0 and 1); programmable (easily controlled by software).
3. Nature: analog is a continuous flow; digital has discrete values (bits, HIGH/LOW). Shape: analog is a sine wave; digital is a square wave. Reliability: analog is low (affected by noise); digital is high (resistant to noise). Examples: human voice vs computer data.
4. NOT works on only one operand (one input), so it is unary. Truth table:  $A = 0 \rightarrow Y = 1$ ;  $A = 1 \rightarrow Y = 0$ .
5.  $2^3 = 8$  rows: 000, 001, 010, 011, 100, 101, 110, 111.
6. Any logic circuit can be built using only NAND gates, or only NOR gates. NAND as NOT: join both inputs and feed A:  $Y = (A \cdot A)' = A'$ . Check:  $A = 0 \rightarrow 1$ ,  $A = 1 \rightarrow 0$ .
7. Minterms:  $A'B$  (row 01) and  $AB'$  (row 10).  $Y = A'B + AB'$ . This is the XOR gate,  $Y = A \oplus B$ .
8. Requirement Analysis  $\rightarrow$  SRS document; System Design  $\rightarrow$  Design Document; Implementation  $\rightarrow$  Working Source Code; Testing  $\rightarrow$  Test Reports / bug-free software; Deployment  $\rightarrow$  Live Software; Maintenance  $\rightarrow$  Patches / Updated Versions.
9. Black box: tester does not see the code, tests functionality (inputs/outputs), done by dedicated testers. White box: tester can see the code, tests internal logic/structure, done by developers.
10. Use TCP when accuracy matters, e.g. web browsing or email. Use UDP when speed matters more than perfection, e.g. video streaming or online gaming.



## Long questions — model answers

- Symbols as drawn in 1.1.7 (AND = D shape; OR = curved back, pointed front; NOT = triangle + bubble; NAND = AND + bubble; NOR = OR + bubble; XOR = OR with extra back curve; XNOR = XOR + bubble). Expressions: AND  $Y = A \cdot B$ ; OR  $Y = A + B$ ; NOT  $Y = A'$ ; NAND  $Y = (A \cdot B)'$ ; NOR  $Y = (A + B)'$ ; XOR  $Y = A \oplus B$ ; XNOR  $Y = (A \oplus B)'$ . Outputs for inputs 00, 01, 10, 11: AND 0,0,0,1; OR 0,1,1,1; NAND 1,1,1,0; NOR 1,0,0,0; XOR 0,1,1,0; XNOR 1,0,0,1. NOT: 0  $\rightarrow$  1, 1  $\rightarrow$  0.
- (a) Truth table ( $ABC \rightarrow F$ ): 000 $\rightarrow$ 0, 001 $\rightarrow$ 1, 010 $\rightarrow$ 0, 011 $\rightarrow$ 1, 100 $\rightarrow$ 0, 101 $\rightarrow$ 1, 110 $\rightarrow$ 1, 111 $\rightarrow$ 1.

(b) K-map:

|       | BC = 00 | BC = 01 | BC = 11 | BC = 10 |
|-------|---------|---------|---------|---------|
| A = 0 | 0       | 1       | 1       | 0       |
| A = 1 | 0       | 1       | 1       | 1       |

Group 1 (size 4): the two middle columns,  $m_1, m_3, m_5, m_7$ . A changes and B changes; C = 1 stays  $\rightarrow$  **C**. Group 2 (size 2):  $m_7$  and  $m_6$  in row A = 1. C changes; A = 1, B = 1 stay  $\rightarrow$  **AB** ( $m_7$  is shared — overlapping is allowed). **F = C + AB**. Check: C + AB is 1 for rows 1, 3, 5, 7 (C = 1) and 6, 7 (AB = 1) = {1, 3, 5, 6, 7}. ✓

(c) Logic diagram: an AND gate with inputs A and B gives AB; an OR gate takes AB and C and gives F.

- (a) First NOT ( $B'$ ), then AND ( $A \cdot B'$ ), then OR ( $+ C$ ).

(b) Truth table:

| A | B | C | B' | A·B' | Y |
|---|---|---|----|------|---|
| 0 | 0 | 0 | 1  | 0    | 0 |
| 0 | 0 | 1 | 1  | 0    | 1 |
| 0 | 1 | 0 | 0  | 0    | 0 |
| 0 | 1 | 1 | 0  | 0    | 1 |
| 1 | 0 | 0 | 1  | 1    | 1 |
| 1 | 0 | 1 | 1  | 1    | 1 |
| 1 | 1 | 0 | 0  | 0    | 0 |
| 1 | 1 | 1 | 0  | 0    | 1 |

(c) Logic diagram: input B goes through a NOT gate to give  $B'$ . A and  $B'$  go into an AND gate to give  $A \cdot B'$ .  $A \cdot B'$  and C go into an OR gate to give Y. (3 gates.)

- Requirement Analysis: meet clients, understand needs, list features; most important phase; deliverable SRS (house: asking the family what rooms they need). System Design: high-level design (architecture), low-level design (database, UI screens), flowcharts; deliverable Design Document (architect draws blueprints). Implementation: write code in Python, Java etc. from the design; deliverable Working Source Code (builders lay bricks). Testing: find and fix bugs using black box and white box testing; deliverable Test Reports / bug-free software (inspector checks the house). Deployment: release to end-users by direct, phased or pilot release; deliverable Live Software (family moves in). Maintenance: support, updates, fixing post-release issues; deliverable Patches / Updated Versions (repairs and extensions).
- Comparison: Flexibility — Waterfall rigid, Agile very flexible. Customer involvement — Waterfall only at start and end, Agile continuous after every sprint. Delivery — Waterfall single delivery at the end, Agile frequent partial deliveries. Testing — Waterfall late in a dedicated phase, Agile continuous in every sprint. Documentation — Waterfall heavy, Agile light. The Al-Noor Library system used Waterfall because its requirements were 100% fixed before coding began, so a simple, well-documented linear process fitted. The Al-Noor Student App used Agile because teachers, parents and students kept requesting new features, so short sprints with continuous feedback were needed.
- OSI (ISO, 1984), top to bottom: 7 Application — network services to user applications (HTTP, HTTPS, FTP, SMTP, DNS); 6 Presentation — translation, encryption, compression (SSL/TLS, JPEG, MP3, ASCII); 5 Session — opens, maintains, closes sessions (NetBIOS, RPC); 4 Transport — reliable end-to-end delivery, error checking, port numbers (TCP, UDP); 3 Network — logical IP addressing and routing (IP, ICMP, routers); 2 Data Link — physical MAC addressing and error detection in frames (Ethernet, switches); 1 Physical — raw bits over cables/wireless (cables, hubs, Wi-Fi radio). Mnemonic bottom to top: "Please Do Not Throw Sausage Pizza Away". TCP/IP mapping: Application = OSI 7, 6, 5; Transport = 4; Internet = 3; Network Access = 2, 1.

### Self-assessment: I can...

- ☐ ...tell discrete from continuous quantities and analog from digital signals, and draw both waves.
- ☐ ...evaluate AND, OR and NOT expressions and follow the order of precedence.
- ☐ ...draw all seven gate symbols and write their truth tables from memory.
- ☐ ...write the SOP (minterms) and POS (maxterms) from a truth table.
- ☐ ...simplify expressions with 2- and 3-variable K-maps, including overlap and wrap-around groups.
- ☐ ...explain the six SDLC phases and choose between Waterfall and Agile for a project.
- ☐ ...list four advantages of digital systems.
- ☐ ...build a complete truth table for 2 or 3 inputs without missing a row.
- ☐ ...explain why NAND and NOR are universal gates.
- ☐ ...draw a logic diagram from an expression.
- ☐ ...build and test a simple circuit in LogiSim and read the wire colours.
- ☐ ...name the 7 OSI layers in order, state their functions, and map them to the 4 TCP/IP layers.

## CHAPTER 2

# Computational Thinking & Algorithm

This chapter teaches you how to think like a computer scientist: break a problem into parts, spot patterns, ignore what does not matter, and write clear step-by-step algorithms. You will then learn four famous algorithms — Bubble Sort, Selection Sort, Linear Search and Binary Search — and how to judge and choose between them.

اس باب میں آپ سیکھیں گے کہ کسی مسئلے کو کمپیوٹر سائنس دان کی طرح کیسے سوچ کر حل کیا جاتا ہے: مسئلے کو چھوٹے حصوں میں توڑنا، پیٹرن پہچاننا، غیر ضروری باتوں کو نظر انداز کرنا اور مرحلہ وار الگورتھم لکھنا۔ اس کے بعد آپ ترتیب دینے (Sorting) اور تلاش کرنے (Searching) کے چار اہم الگورتھم سیکھیں گے۔

### What you will learn

- Define computational thinking and name its four parts: decomposition, pattern recognition, abstraction and algorithms.
- Compare an algorithm with pseudocode on at least five points.
- Dry-run Bubble Sort and Selection Sort pass by pass on a given list, and write them in pseudocode and Python.
- List the advantages and disadvantages of each sorting and searching algorithm.
- Evaluate an algorithm for correctness (using a trace table), efficiency and clarity.
- Define an algorithm and state five reasons why algorithms are important.
- Apply decomposition, pattern recognition and abstraction to real-life problems.
- Trace Linear Search step by step and Binary Search using a low / mid / high table.
- State the best-case and worst-case time complexity of the four algorithms.
- Choose the right algorithm for a situation and justify your choice.

## 2.1 Computational Thinking

### LEARN IT

**Computational Thinking (CT)** is a systematic approach to solving problems. "Systematic" means you follow a fixed, organised method instead of guessing. CT involves four activities:

- **Decomposition** — breaking a big problem down into smaller parts.
- **Pattern recognition** — identifying things that repeat or follow a rule.
- **Abstraction** — hiding (ignoring) details that do not matter.
- **Algorithms** — developing clear step-by-step instructions to solve the problem.

CT is not "thinking like a computer". It is thinking in a way that lets you describe a solution so clearly that a computer (or another person) could carry it out.

Problem



Decomposition



Pattern recognition



Abstraction



Algorithm



Program

You already use computational thinking in daily life. When you plan your exam preparation, you split the syllabus into chapters (decomposition), notice that every chapter needs the same routine of reading, making notes and practising (pattern recognition), ignore things that are not in the syllabus (abstraction), and make a day-by-day timetable (algorithm).

#### LEARN IT

**Did you know? Pseudocode** is a human-readable representation of an algorithm using plain English and structured programming keywords. It is the bridge between human logic and machine code. You will study it in detail in section 2.2.2.

#### REMEMBER IT

Remember the four parts of CT with the phrase "**Don't Pick Any Apple**": **D**ecomposition, **P**attern recognition, **A**bstraction, **A**lgorithm.

#### COMMON MISTAKE

Students sometimes think computational thinking means "writing code". It does not. CT happens *before* coding. You can do all four steps with pen and paper, without a computer.

#### اردو میں سمجھیں

کمپیوٹیشنل تھنکنگ (Computational Thinking) مسائل حل کرنے کا ایک منظم طریقہ ہے۔ اس کے چار حصے ہیں: مسئلے کو چھوٹے حصوں میں توڑنا (Decomposition)، دہرائی جانے والی باتوں کو پہچاننا (Pattern Recognition)، غیر ضروری تفصیلات کو چھپانا (Abstraction) اور مرحلہ وار حل لکھنا (Algorithm)۔ یہ سوچنے کا طریقہ ہے، کوڈ لکھنا نہیں۔

#### CHECK YOURSELF

1. Name the four parts of computational thinking.
2. Is computational thinking done before or after writing code?
3. What is pseudocode called a "bridge" between?

Answers: 1. Decomposition, pattern recognition, abstraction, algorithms. 2. Before coding (it is planning the solution). 3. Between human logic and machine code.

## 2.2 Algorithm ★ Golden

#### LEARN IT

An **algorithm** is a step-by-step logical procedure used to solve a problem. It is **conceptual** in nature (it is an idea, a plan) and completely **independent of any programming language**. The same algorithm can be written later in Python, C++ or Java.

A good algorithm has:

- **Input** — the data it starts with.
- **Clear steps** — each step has only one meaning.
- **A definite end** — it must finish after a limited number of steps.
- **Output** — the result it produces.

### WORKED EXAMPLE

#### Algorithm to find the larger of two numbers

Step 1: Start  
Step 2: Read A and B  
Step 3: If  $A > B$ , print "A is larger"  
Step 4: Else print "B is larger or equal"  
Step 5: End

Notice: there is no Python or C++ here. Anyone — even a non-programmer — can follow these steps. That is what "language-independent" means.

### 2.2.1 Importance of Algorithms

| Reason                   | What it means for you                                                                                                            |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Structured approach      | Algorithms provide a structured approach to problem-solving, making complex tasks manageable.                                    |
| Optimisation             | They optimise time and memory usage, which is crucial for large-scale applications (for example, searching millions of records). |
| Reusability              | They can be reused for similar problems, saving time and effort. A sorting algorithm for marks can also sort prices.             |
| Accuracy and consistency | They ensure accuracy and consistency, reducing human error. The same steps always give the same result.                          |
| Blueprint                | They act as a blueprint (a plan) before actual code implementation, just like an architect's map before building a house.        |

#### REMEMBER IT

Remember the five reasons as **S-O-R-A-B**: **S**tructured, **O**ptimised, **R**eusable, **A**ccurate, **B**lueprint.

#### COMMON MISTAKE

Do not write "an algorithm is a program". A program is the algorithm *written in a programming language* so that a computer can run it. The algorithm itself is only the logic.

#### اردو میں سمجھیں

الگورتھم (Algorithm) کسی مسئلے کو حل کرنے کا مرحلہ وار منطقی طریقہ ہے۔ یہ ایک تصور ہے اور کسی بھی پروگرامنگ زبان پر منحصر نہیں۔ الگورتھم کام کو منظم بناتے ہیں، وقت اور میموری بچاتے ہیں، دوبارہ استعمال ہو سکتے ہیں، غلطیاں کم کرتے ہیں اور کوڈ لکھنے سے پہلے نقشے کا کام دیتے ہیں۔

#### EXAM TIP

Very common short question: "Define algorithm and write its importance." Write the definition word for word — *a step-by-step logical procedure used to solve a problem; conceptual and independent of any programming language* — then list at least four of the five importance points. If asked to "write an algorithm", always begin with **Step 1: Start** and finish with **End**, and number every step.

### CHECK YOURSELF

1. Why is an algorithm called "language-independent"?
2. Give two reasons why algorithms are important.
3. Write a 5-step algorithm to find the sum of two numbers.

Answers: 1. Because it describes only the logic; it is not tied to Python, C++ or any other language. 2. Any two: structured approach, optimise time/memory, reusable, accurate and consistent, blueprint before coding. 3. Step 1: Start; Step 2: Read A and B; Step 3: SUM = A + B; Step 4: Print SUM; Step 5: End.

## 2.2.2 Algorithm vs Pseudocode ★ Golden

### LEARN IT

Both algorithms and pseudocode are used in problem-solving, but they serve different purposes in the design phase. The **algorithm** says *what* must be done. The **pseudocode** shows *how* it could be implemented, using programming-like keywords such as IF, FOR and WHILE, but it still cannot be run on a computer.

| Point        | Algorithm                                                              | Pseudocode                                                                      |
|--------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Definition   | A step-by-step logical procedure to solve a problem.                   | A human-readable representation of an algorithm using plain English & keywords. |
| Nature       | Conceptual and language-independent.                                   | Semi-formal and structured, but not executable code.                            |
| Purpose      | To define <b>what</b> needs to be done logically.                      | To present <b>how</b> it can be implemented in a readable format.               |
| Form         | Usually described in plain steps, bullets or numbered points.          | Written using structured constructs (IF, FOR, WHILE) like programming.          |
| Focus        | Focuses on the logic and steps of the problem.                         | Focuses on representing steps clearly before coding.                            |
| Audience     | For anyone trying to understand the logic (including non-programmers). | Mainly for programmers or students preparing for coding.                        |
| Format rules | No fixed format; can be informal.                                      | Follows standard pseudocode conventions for clarity.                            |

### WORKED EXAMPLE

The same problem written both ways: print whether a number is even or odd.

#### Algorithm

Step 1: Start  
Step 2: Read a number N  
Step 3: Divide N by 2 and find the remainder  
Step 4: If the remainder is 0, print "Even", otherwise print "Odd"  
Step 5: End

#### Pseudocode

```
START
INPUT N
SET R TO N MOD 2
IF R = 0 THEN
    PRINT "Even"
ELSE
    PRINT "Odd"
END IF
END
```

The equivalent Python program (real, executable code):

```
n = 7
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

OUTPUT  
Odd

Problem → Algorithm (what) → Pseudocode (how) → Program (code) → Output

#### REMEMBER IT

Teacher's analogy: an **algorithm** is like a recipe ("Bake a cake at 350 degrees"). **Pseudocode** is like a structured kitchen manual using standard verbs (SET temperature TO 350; WHILE cake is not baked, WAIT). The program is the oven actually doing it.

#### COMMON MISTAKE

Pseudocode is **not** executable. If you type pseudocode into Python, it will give errors. Also, do not mix exact Python syntax (colons, brackets) into pseudocode answers unless asked — use keywords like IF ... THEN ... END IF.

#### اردو میں سمجھیں

الگورتھم بتاتا ہے کہ کیا کرنا ہے، جبکہ سوڈو کوڈ (Pseudocode) بتاتا ہے کہ اسے کیسے کیا جا سکتا ہے۔ الگورتھم سادہ مرحلوں میں لکھا جاتا ہے اور ہر کوئی اسے سمجھ سکتا ہے۔ سوڈو کوڈ میں IF، FOR اور WHILE جیسے الفاظ استعمال ہوتے ہیں اور یہ زیادہ تر پروگرامرز کے لیے ہوتا ہے، لیکن اسے کمپیوٹر پر چلایا نہیں جا سکتا۔

#### EXAM TIP

"Differentiate between algorithm and pseudocode" is a favourite 4–5 mark question. Draw a two-column table and give at least **five** points (definition, nature, purpose, form, audience). Adding the recipe vs kitchen-manual analogy and one small example of each earns full marks.

#### CHECK YOURSELF

1. Which one — algorithm or pseudocode — uses keywords like FOR and WHILE?
2. Which one is meant for non-programmers too?
3. Can pseudocode be executed by a computer?

Answers: 1. Pseudocode. 2. Algorithm. 3. No; it is semi-formal and not executable.

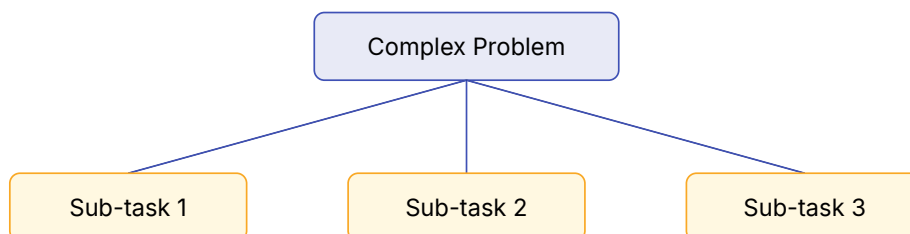
## 2.3.1 Decomposition ★ Golden

**2.3 Algorithmic Strategies for Problem Solving.** An **algorithmic strategy** is a general method that helps you design an algorithm. The three strategies in your syllabus are decomposition, pattern recognition and abstraction.

## LEARN IT

**Decomposition** is a problem-solving technique in which a complex task or system is broken down into smaller, manageable parts. By dividing a big problem into simpler steps, it becomes easier to understand, develop and solve.

Each small part is called a **sub-task** (or sub-problem). You solve each sub-task separately, then join the solutions to solve the whole problem.



Decomposition: one complex problem is split into smaller sub-tasks.

## WORKED EXAMPLE

### Real-life example: Learning a new language

"Learn English" (or any language) is too big to do in one go. Decompose it:

| Sub-task          | Description                                                                 |
|-------------------|-----------------------------------------------------------------------------|
| Learn vocabulary  | Learn the words for people, actions and things (e.g., "I", "eat", "apple"). |
| Grammar rules     | Learn the sentence structure: Subject → Verb → Object.                      |
| Learn tenses      | Present, past, future (e.g., "I eat" vs. "I ate").                          |
| Sentence building | Combine words using the rules to create valid sentences: "I ate an apple."  |

Vocabulary → Grammar rules → Tenses → Sentence building

## WORKED EXAMPLE

### Computing example: a "Student Result" program

Problem: read the marks of a student in 5 subjects and print the total, percentage and grade.

- Sub-task 1: Input marks of 5 subjects.
- Sub-task 2: Calculate total = sum of the 5 marks.
- Sub-task 3: Calculate percentage =  $\text{total} \div 500 \times 100$ .
- Sub-task 4: Decide the grade from the percentage.
- Sub-task 5: Print the result card.

Each sub-task is now so small that you can write it in two or three lines of code.

## Why is decomposition useful?

- **Reduces complexity:** you focus on one piece at a time.
- **Helps in debugging:** you can find exactly where the mistake is (e.g., in the language example, is the error in the verb tense or the word order?).
- **Promotes reusability:** once you learn or build a small part, you can apply it elsewhere (the "calculate percentage" part can be reused in an attendance program).

#### REMEMBER IT

Think of eating a big *roti*: you cannot eat it in one bite, so you break it into small pieces.  
Decomposition = "break the roti".

#### COMMON MISTAKE

Do not break a problem into random pieces. Each sub-task must be a meaningful, clear job, and all sub-tasks together must solve the *whole* problem. If one part is missing (e.g., "print result"), the solution is incomplete.

#### اردو میں سمجھیں

ڈیکمپوزیشن (Decomposition) کا مطلب ہے کسی بڑے اور پیچیدہ مسئلے کو چھوٹے اور آسان حصوں میں تقسیم کرنا۔ مثلاً نئی زبان سیکھنے کو الفاظ، گرامر، زمانے (Tenses) اور جملہ سازی میں تقسیم کیا جا سکتا ہے۔ اس سے پیچیدگی کم ہوتی ہے، غلطی ڈھونڈنا آسان ہوتا ہے اور حصے دوبارہ استعمال ہو سکتے ہیں۔

#### EXAM TIP

Expect "Define decomposition with a real-life example" or "Explain the benefits of decomposition". Give the definition, draw the small tree (Complex Problem → Sub-task 1, 2, 3), show the language-learning table, and list the three benefits: reduces complexity, helps debugging, promotes reusability.

#### CHECK YOURSELF

1. What is each smaller part of a decomposed problem called?
2. Decompose "Making a school timetable" into three sub-tasks.
3. State the three benefits of decomposition.

Answers: 1. A sub-task (sub-problem). 2. Example: list subjects and teachers; decide periods per subject per week; place periods in days so no teacher has two classes at once. 3. Reduces complexity, helps in debugging, promotes reusability.

## 2.3.2 Pattern Recognition

#### LEARN IT

**Pattern recognition** is the process of identifying regularities, repetitions or rules in data. It helps in reducing the overall complexity of an algorithm and forms the foundation of concepts such as **loops**.

How it works — ask yourself three questions:

- What is repeating?
- What changes in a predictable way?
- Can I generalise this (write one rule that works for every case)?



## WORKED EXAMPLE

### Example: Triangle of stars

#### OUTPUT

```
*
**
***
****
*****
```

**Step 1 — What repeats?** Every row prints stars.

**Step 2 — What changes predictably?** Each row adds 1 star to the previous row's count.

| Row | Stars |
|-----|-------|
| 1   | 1     |
| 2   | 2     |
| 3   | 3     |
| ... | ...   |
| N   | N     |

**Step 3 — Generalise:** Row N = N stars. Recognising this pattern lets us write one generic loop instead of 5 separate print statements.

#### Without the pattern (5 statements)

```
print("*")
print("**")
print("***")
print("****")
print("*****")
```

#### With the pattern (one loop)

```
n = 5
for row in range(1, n + 1):
    print("*" * row)
```

Both programs print exactly the same triangle. But if you want 100 rows, the loop version needs only one change (  $n = 100$  ), while the first version needs 100 lines.

## REMEMBER IT

Whenever you see "the same thing again and again, with a small change each time", think **loop**. Pattern recognition is the habit of spotting that.

## COMMON MISTAKE

Do not generalise from too few cases. Check your rule on several rows (or items) before trusting it. "Row N = N stars" is correct here; a different triangle (1, 3, 5, 7 stars) follows the rule "Row N = 2N - 1 stars".

## اردو میں سمجھیں

پیٹرن ریکگنیشن (Pattern Recognition) کا مطلب ہے ڈیٹا میں دہرائی جانے والی چیزوں یا اصولوں کو پہچاننا۔ ستاروں کی تکیوں میں ہر قطار میں پچھلی قطار سے ایک ستارہ زیادہ ہے، یعنی قطار N میں N ستارے۔ یہ پیٹرن پہچان کر ہم پانچ الگ پرنٹ کے بجائے ایک لوپ (Loop) لکھ سکتے ہیں۔

### CHECK YOURSELF

1. Which programming concept is based on pattern recognition?
2. How many stars will row 8 of the triangle have?
3. Find the rule: 2, 4, 6, 8, ...

Answers: 1. Loops. 2. 8 stars (Row N = N stars). 3. Term N =  $2 \times N$  (each term adds 2).

## 2.3.3 Abstraction

### LEARN IT

**Abstraction** means hiding unnecessary features and focusing only on the essential ones needed to solve a problem. It reduces complexity by ignoring what is not important (the **noise**).

When you apply abstraction you sort details into three groups:

- **Necessary steps** — the core logic that never changes.
- **Changes based on the situation** — these become **variables** (inputs) in the algorithm.
- **Abstraction items (noise)** — details you can ignore completely.

### WORKED EXAMPLE

#### Example: applying abstraction to "Making Tea"

To make an algorithm for tea, we separate the necessary steps from the situational changes and the noise.

| 1. Necessary steps (core logic)                                                                      | 2. Changes based on situation (variables)                                               | 3. Abstraction items (ignore / noise)                                          |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| 1. Boil water<br>2. Add tea leaves/bag<br>3. Steep for a few minutes<br>4. Pour into cup<br>5. Serve | Ingredients (black, green, milk)<br>Additions (sugar, lemon)<br>Tools (stove vs kettle) | Brand of kettle<br>Cup colour or design<br>Where tea is made (kitchen, office) |

The core abstracted logic, independent of cup colour or kettle brand:



This one algorithm now works for green tea in an office kettle and for milk tea on a kitchen stove. The middle column simply becomes inputs: `tea_type`, `addition`, `tool`.

Computers use abstraction everywhere. When you call `print("Hello")` in Python, you do not think about how the screen lights up each pixel — those details are hidden from you. A map of Karachi shows roads and landmarks but hides every tree and car: it is an abstraction of the city.

### REMEMBER IT

Abstraction = "**Keep the skeleton, drop the decoration.**" Ask: "If I remove this detail, does the solution still work?" If yes, it is noise.

### COMMON MISTAKE

Do not confuse abstraction with decomposition. Decomposition *splits* a problem into parts. Abstraction *removes* unimportant details. Also, do not throw away details that really matter — "boil water" can never be ignored, but kettle brand can.

ایبسٹریکشن (Abstraction) کا مطلب ہے غیر ضروری تفصیلات کو چھپا کر صرف ضروری باتوں پر توجہ دینا۔ چائے بنانے کی مثال میں پانی ابالنا، پتی ڈالنا، دم دینا، کپ میں ڈالنا اور پیش کرنا ضروری مراحل ہیں؛ چائے کی قسم یا چینی متغیرات (Variables) ہیں؛ جبکہ کیتلی کا برانڈ یا کپ کا رنگ فالتو باتیں (Noise) ہیں جنہیں نظر انداز کیا جاتا ہے۔

#### CHECK YOURSELF

1. What is "noise" in abstraction?
2. In the tea example, is "sugar or lemon" a necessary step, a variable, or noise?
3. How is abstraction different from decomposition?

Answers: 1. Unnecessary details that do not affect the solution. 2. A variable (it changes with the situation). 3. Decomposition breaks a problem into parts; abstraction hides details that are not important.

## 2.4 Sorting and Searching Algorithms

#### LEARN IT

Sorting and searching are both fundamental techniques of computer science. They work on **lists of the same data type** (for example, a list of all integers or a list of all names).

| Sorting                                                                                                                                           | Searching                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Arranging a collection of items in an order. The order may be <b>ascending</b> (0–9, A–Z) or <b>descending</b> (9–0, Z–A). Used to organise data. | Locating a specific element within a list to find its <b>position</b> or check its <b>existence</b> (is it there or not?). |
| Examples: Bubble Sort, Selection Sort                                                                                                             | Examples: Linear Search, Binary Search                                                                                     |
| Real life: merit list arranged from highest to lowest marks.                                                                                      | Real life: finding your roll number in the merit list.                                                                     |

#### Notation you will see

- **DATA** — the list (array). `DATA[1]` is the 1st element in the textbook algorithms (counting starts at 1).
- In **pseudocode and Python** counting starts at **0**: `DATA[0]` is the 1st element. This is called an **index**.
- **N** — the number of elements in the list.
- **PTR** (pointer) — a variable that holds the current position.
- **temp** — a temporary variable used to **swap** (exchange) two values.
- **Pass** — one complete run of the inner loop over the list.

#### WORKED EXAMPLE

**How a swap works with temp.** Suppose  $A = 8$  and  $B = 4$ . To exchange them:

`temp = A (temp = 8)` → `A = B (A = 4)` → `B = temp (B = 8)`

Without `temp`, writing `A = B` first would destroy the 8. In Python you may also write `a, b = b, a`.

#### COMMON MISTAKE

Position 1 in the textbook algorithm is index 0 in Python. In exams, if a question uses `DATA[1]`, count from 1; if it uses Python, count from 0. State clearly which one you use.

سارٹنگ (Sorting) کا مطلب ہے ڈیٹا کو صعودی (چھوٹے سے بڑے) یا نزولی (بڑے سے چھوٹے) ترتیب میں رکھنا۔ سرچنگ (Searching) کا مطلب ہے فہرست میں کسی خاص چیز کو تلاش کرنا تاکہ اس کی جگہ یا موجودگی معلوم ہو سکے۔ دونوں ایک ہی قسم کے ڈیٹا کی فہرست پر کام کرتے ہیں۔

#### CHECK YOURSELF

1. Arrange C, A, D, B in descending order.
2. What is the purpose of the variable temp?

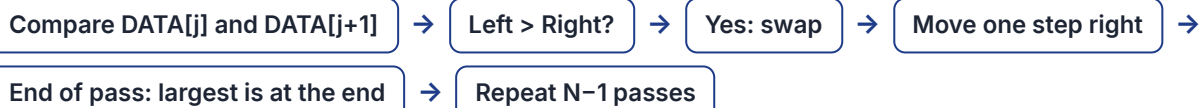
Answers: 1. D, C, B, A. 2. It temporarily holds one value so that two values can be swapped without losing either.

## 2.4.1 Sorting Algorithms: Bubble Sort ★ Golden

#### LEARN IT

**Bubble Sort** is an elementary sorting algorithm that works by repeatedly comparing **adjacent** (side-by-side) elements and swapping them if they are in the wrong order.

In each pass, the largest remaining value "bubbles up" to the end of the list, like an air bubble rising to the top of water. So after pass 1 the largest value is in the last place, after pass 2 the second largest is in second-last place, and so on. A list of  $N$  items needs  $N - 1$  passes, and pass  $K$  makes  $N - K$  comparisons.



#### Algorithm (textbook)

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N-1
Step 5: Set PTR := 1
Step 6: Repeat while PTR ≤ N-K:
    a: If DATA[PTR] > DATA[PTR+1]:
        1. Set temp = DATA[PTR]
        2. Set DATA[PTR] = DATA[PTR+1]
        3. Set DATA[PTR+1] = temp
    b: Set PTR := PTR + 1
Step 7: End
  
```

#### Pseudocode

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End
  
```

### WORKED EXAMPLE

**Worked example: sort 8, 4, 1, 9, 3 in ascending order.** The pair in brackets [ ] is being compared. Positions are counted from 1.

#### Pass 1 (K = 1, 4 comparisons)

| Comparison | Pair            | Left > Right? | Action    | List after           |
|------------|-----------------|---------------|-----------|----------------------|
| 1st vs 2nd | [8, 4], 1, 9, 3 | 8 > 4 Yes     | Swap      | 4, 8, 1, 9, 3        |
| 2nd vs 3rd | 4, [8, 1], 9, 3 | 8 > 1 Yes     | Swap      | 4, 1, 8, 9, 3        |
| 3rd vs 4th | 4, 1, [8, 9], 3 | 8 > 9 No      | No change | 4, 1, 8, 9, 3        |
| 4th vs 5th | 4, 1, 8, [9, 3] | 9 > 3 Yes     | Swap      | 4, 1, 8, 3, <b>9</b> |

9 is now at its correct (final) place.

#### Pass 2 (K = 2, 3 comparisons)

| Comparison | Pair            | Left > Right? | Action    | List after           |
|------------|-----------------|---------------|-----------|----------------------|
| 1st vs 2nd | [4, 1], 8, 3, 9 | 4 > 1 Yes     | Swap      | 1, 4, 8, 3, 9        |
| 2nd vs 3rd | 1, [4, 8], 3, 9 | 4 > 8 No      | No change | 1, 4, 8, 3, 9        |
| 3rd vs 4th | 1, 4, [8, 3], 9 | 8 > 3 Yes     | Swap      | 1, 4, 3, <b>8, 9</b> |

#### Pass 3 (K = 3, 2 comparisons)

| Comparison | Pair            | Left > Right? | Action    | List after           |
|------------|-----------------|---------------|-----------|----------------------|
| 1st vs 2nd | [1, 4], 3, 8, 9 | 1 > 4 No      | No change | 1, 4, 3, 8, 9        |
| 2nd vs 3rd | 1, [4, 3], 8, 9 | 4 > 3 Yes     | Swap      | 1, 3, <b>4, 8, 9</b> |

#### Pass 4 (K = 4, 1 comparison)

| Comparison | Pair            | Left > Right? | Action    | List after           |
|------------|-----------------|---------------|-----------|----------------------|
| 1st vs 2nd | [1, 3], 4, 8, 9 | 1 > 3 No      | No change | <b>1, 3, 4, 8, 9</b> |

**Sorted list: 1, 3, 4, 8, 9.** Totals:  $4 + 3 + 2 + 1 = 10$  comparisons and **6 swaps**.

| Pass  | List after pass | Fixed at the end |
|-------|-----------------|------------------|
| Start | 8, 4, 1, 9, 3   | —                |
| 1     | 4, 1, 8, 3, 9   | 9                |
| 2     | 1, 4, 3, 8, 9   | 8, 9             |
| 3     | 1, 3, 4, 8, 9   | 4, 8, 9          |
| 4     | 1, 3, 4, 8, 9   | all              |

### WORKED EXAMPLE

**Bubble Sort in Python** (follows the pseudocode exactly; Python indexes start at 0):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

#### OUTPUT

```
After pass 1 : [4, 1, 8, 3, 9]
After pass 2 : [1, 4, 3, 8, 9]
After pass 3 : [1, 3, 4, 8, 9]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

`range(n - 1 - outer)` gives `inner = 0 ... N-2-outer`, exactly as in the pseudocode. Notice the list was already sorted after pass 3, but the plain algorithm still does pass 4.

## Advantages and disadvantages

| Advantages                                         | Disadvantages                                                                               |
|----------------------------------------------------|---------------------------------------------------------------------------------------------|
| Easiest algorithm to visualise and explain.        | Uses nested loops, so the time grows very fast as the list grows (about $N^2$ comparisons). |
| Does not need extra memory (in-place sorting).     | Performs a massive number of swaps.                                                         |
| Stable (equal items stay in their original order). | Useless for real-world "Big Data".                                                          |

### REMEMBER IT

**Bubble = neighbours.** Bubble Sort only ever compares *neighbours* ( $j$  and  $j+1$ ). Each pass pushes the biggest bubble to the end. For **descending** order, just change `>` to `<` in the comparison.

### COMMON MISTAKE

The slides say the time grows "exponentially". Strictly, Bubble Sort's time grows **quadratically**: for  $N$  items it makes  $N(N-1)/2$  comparisons, which is  $O(N^2)$ . Doubling the list makes it about 4 times slower — bad, but not exponential. Another common mistake: students forget that pass  $K$  does only  $N - K$  comparisons; the sorted tail is never compared again.

### اردو میں سمجھیں

بل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کی جگہ بدل دی جاتی ہے۔ ہر پاس (Pass) کے بعد سب سے بڑا عدد آخر میں پہنچ جاتا ہے۔  $N$  عناصر کے لیے  $N-1$  پاس درکار ہوتے ہیں۔ یہ سمجھنے میں آسان اور مستحکم (Stable) ہے لیکن بڑے ڈیٹا کے لیے بہت سست ہے۔

### EXAM TIP

The most common question: "Sort the list ... using Bubble Sort. Show each pass." Draw one table per pass with columns *Pair*, *Compare*, *Action*, *List after*. Mark the element fixed at the end after every pass. Also be ready to write the algorithm (with PTR and K) or the pseudocode (with outer/inner), and to list two advantages and two disadvantages.

### CHECK YOURSELF

1. How many passes and comparisons does Bubble Sort need for 6 elements?
2. Show the list 5, 1, 4 after pass 1 of Bubble Sort.
3. Sort 2, 9, 4, 7 in descending order with Bubble Sort: give the list after each pass.

Answers: 1. 5 passes;  $5 + 4 + 3 + 2 + 1 = 15$  comparisons. 2. (5,1) swap  $\rightarrow$  1, 5, 4; (5,4) swap  $\rightarrow$  1, 4, 5. 3. Pass 1: 9, 4, 7, 2; Pass 2: 9, 7, 4, 2; Pass 3: 9, 7, 4, 2.

## 2.4.1 Sorting Algorithms: Selection Sort ★ Golden

### LEARN IT

**Selection Sort** is a simple sorting algorithm that works by repeatedly **selecting the smallest element** from the unsorted portion of the list and moving it to its correct position at the front.

Think of it as filling seats from the front: first find who belongs in position 1, then position 2, and so on. In the textbook version, the element at position  $K$  is compared with every element after it; whenever a smaller one is found, the two are swapped. So at the end of pass  $K$ , the  $K$ -th smallest value sits in position  $K$ .



### Algorithm (textbook)

```
Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N
Step 5: Set PTR := K+1
Step 6: Repeat while PTR ≤ N:
    a: If DATA[K] > DATA[PTR]:
        1. Set temp = DATA[K]
        2. Set DATA[K] = DATA[PTR]
        3. Set DATA[PTR] = temp
    b: Set PTR := PTR + 1
Step 7: End
```

### Pseudocode

```
Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = outer+1 to N-1
        IF DATA[outer] > DATA[inner]
            Temp = DATA[outer]
            DATA[outer] = DATA[inner]
            DATA[inner] = Temp
        End If
    End For
End For
End
```

### REMEMBER IT

**Teacher's tip — spot the difference:** in Selection Sort the inner loop starts at `outer + 1` and compares with the *fixed* position `DATA[outer]`. In Bubble Sort the inner loop always starts at `0` and compares *neighbours* `DATA[inner]` and `DATA[inner+1]`. Bubble pushes the largest to the **end**; Selection brings the smallest to the **front**.

### WORKED EXAMPLE

**Worked example: sort 8, 4, 1, 9, 3 in ascending order (textbook method).** The two positions in brackets are being compared.

#### Pass 1 — find the smallest for Position 1

| Compare      | List              | Pos1 > PosX? | Action    | List after            |
|--------------|-------------------|--------------|-----------|-----------------------|
| Pos1 vs Pos2 | [8], [4], 1, 9, 3 | 8 > 4 Yes    | Swap      | 4, 8, 1, 9, 3         |
| Pos1 vs Pos3 | [4], 8, [1], 9, 3 | 4 > 1 Yes    | Swap      | 1, 8, 4, 9, 3         |
| Pos1 vs Pos4 | [1], 8, 4, [9], 3 | 1 > 9 No     | No change | 1, 8, 4, 9, 3         |
| Pos1 vs Pos5 | [1], 8, 4, 9, [3] | 1 > 3 No     | No change | <b>1</b> , 8, 4, 9, 3 |

1 is now permanently in Position 1.

#### Pass 2 — find the smallest for Position 2

| Compare      | List              | Pos2 > PosX? | Action    | List after            |
|--------------|-------------------|--------------|-----------|-----------------------|
| Pos2 vs Pos3 | 1, [8], [4], 9, 3 | 8 > 4 Yes    | Swap      | 1, 4, 8, 9, 3         |
| Pos2 vs Pos4 | 1, [4], 8, [9], 3 | 4 > 9 No     | No change | 1, 4, 8, 9, 3         |
| Pos2 vs Pos5 | 1, [4], 8, 9, [3] | 4 > 3 Yes    | Swap      | <b>1, 3</b> , 8, 9, 4 |

#### Pass 3 — find the smallest for Position 3

| Compare      | List              | Pos3 > PosX? | Action    | List after            |
|--------------|-------------------|--------------|-----------|-----------------------|
| Pos3 vs Pos4 | 1, 3, [8], [9], 4 | 8 > 9 No     | No change | 1, 3, 8, 9, 4         |
| Pos3 vs Pos5 | 1, 3, [8], 9, [4] | 8 > 4 Yes    | Swap      | <b>1, 3, 4</b> , 9, 8 |

#### Pass 4 — find the smallest for Position 4

| Compare      | List              | Pos4 > PosX? | Action | List after           |
|--------------|-------------------|--------------|--------|----------------------|
| Pos4 vs Pos5 | 1, 3, 4, [9], [8] | 9 > 8 Yes    | Swap   | <b>1, 3, 4, 8, 9</b> |

**Sorted list: 1, 3, 4, 8, 9.** Totals: 4 + 3 + 2 + 1 = **10 comparisons, 6 swaps** in this textbook method.

| Pass  | List after pass | Fixed at the front |
|-------|-----------------|--------------------|
| Start | 8, 4, 1, 9, 3   | —                  |
| 1     | 1, 8, 4, 9, 3   | 1                  |
| 2     | 1, 3, 8, 9, 4   | 1, 3               |
| 3     | 1, 3, 4, 9, 8   | 1, 3, 4            |
| 4     | 1, 3, 4, 8, 9   | all                |



### WORKED EXAMPLE

**Selection Sort in Python** (follows the textbook pseudocode):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(outer + 1, n):
        if data[outer] > data[inner]:
            temp = data[outer]
            data[outer] = data[inner]
            data[inner] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

#### OUTPUT

```
After pass 1 : [1, 8, 4, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

### WORKED EXAMPLE

**The "classic" Selection Sort (one swap per pass).** Many books first *find the position of the smallest value* in the unsorted part, and swap only once at the end of each pass. The result is the same; only the swaps are fewer.

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    smallest = outer
    for inner in range(outer + 1, n):
        if data[inner] < data[smallest]:
            smallest = inner
    data[outer], data[smallest] = data[smallest], data[outer]
    print("After pass", outer + 1, ":", data)
```

#### OUTPUT

```
After pass 1 : [1, 4, 8, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
```

Here only 4 swaps are made (at most  $N - 1$ ). This is why Selection Sort is said to perform fewer swaps than Bubble Sort. In exams, use the textbook method unless the question asks otherwise.

## Advantages and disadvantages

| Advantages                                            | Disadvantages                                                                |
|-------------------------------------------------------|------------------------------------------------------------------------------|
| Very easy to code.                                    | Time grows very fast with list size (nested loops, about $N^2$ comparisons). |
| Performs fewer swaps overall compared to Bubble Sort. | Must finish every single loop — it is "blind" to an already sorted list.     |
| Does not need extra memory.                           | Not stable (it might change the relative positions of equal items).          |

### WORKED EXAMPLE

**Why "not stable"?** Take two equal marks,  $5_a$  (Ali) and  $5_b$  (Bilal), in the list  $5_a, 5_b, 2$ . Pass 1 compares  $5_a$  with  $5_b$  (no swap), then  $5_a$  with 2  $\rightarrow$  swap  $\rightarrow 2, 5_b, 5_a$ . Now Bilal is before Ali, although Ali came first originally. Bubble Sort never does this, because it swaps only neighbours that are strictly out of order.

### COMMON MISTAKE

In the textbook method, the comparison is always with the **fixed position**  $DATA[K]$  (Pos1 in pass 1, Pos2 in pass 2), not with the neighbour. Students often mix this up with Bubble Sort and compare neighbours. Also, after a swap, the *new* value at Pos K is used for the next comparison.

### اردو میں سمجھیں

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔ ہر پاس میں ایک مقررہ جگہ (Position) کا موازنہ اس کے بعد والے تمام عناصر سے ہوتا ہے۔ اس کا کوڈ آسان ہے اور ببل سارٹ سے کم تبادلے (Swaps) ہوتے ہیں، لیکن یہ مستحکم (Stable) نہیں اور ترتیب شدہ فہرست پر بھی پورے لوپ چلاتا ہے۔

### EXAM TIP

Typical questions: "Sort ... using Selection Sort, showing all steps", "Differentiate between Bubble Sort and Selection Sort" and "Why is Selection Sort not stable?" In the difference question mention: what is compared (neighbours vs fixed position), where the inner loop starts (0 vs outer+1), where items settle (end vs front), stability, and number of swaps.

### CHECK YOURSELF

1. In Selection Sort, where does the inner loop start?
2. Using the textbook method, show 7, 2, 5 after pass 1.
3. Which of the two sorts is stable?

Answers: 1. At outer + 1 (PTR = K + 1). 2. Pos1 vs Pos2:  $7 > 2$  swap  $\rightarrow 2, 7, 5$ ; Pos1 vs Pos3:  $2 > 5$  no  $\rightarrow 2, 7, 5$ . 3. Bubble Sort.

## 2.4.2 Searching Algorithms: Linear Search

### LEARN IT

**Linear Search**, also known as **sequential search**, works by examining each element in the list one by one, from the start, until the target is found (or the list ends). It works on both **sorted and unsorted** data.

A **flag** variable (here `ValueFound`) starts at 0 and becomes 1 when the item is found. After the loop, the flag tells us whether to print the item or "Not Found".



## Algorithm (textbook)

```
Step 1: Start
Step 2: DATA := {8,3,6,1,7,2,4,5}
Step 3: N := 8
Step 4: FindValue := 7
Step 5: ValueFound := 0
Step 6: Repeat for PTR = 1 to N
Step 7:   If DATA[PTR] = FindValue:
           a. ValueFound = 1
           b. Goto Step 8
Step 8: If ValueFound = 1:
           Print DATA[PTR]
       Else:
           Print "Not Found"
Step 9: End
```

## Pseudocode

```
Start
DATA = {8, 3, 6, 1, 7, 2, 4, 5}
N = 8
FindValue = 7
ValueFound = 0
For PTR = 0 to N-1
    If DATA[PTR] = FindValue then
        ValueFound = 1
        Exit Loop
    End If
End For
If ValueFound = 1 then
    Print DATA[PTR]
Else
    Print "Data Not Found"
End If
End
```

### WORKED EXAMPLE

**Worked example.** Data list: 8, 3, 6, 1, 7, 2, 4, 5. Searching item: 7.

| Step | Element checked | Condition (== 7) | Result             |
|------|-----------------|------------------|--------------------|
| 1    | 8               | 8 == 7 ?         | Not found          |
| 2    | 3               | 3 == 7 ?         | Not found          |
| 3    | 6               | 6 == 7 ?         | Not found          |
| 4    | 1               | 1 == 7 ?         | Not found          |
| 5    | 7               | 7 == 7 ?         | Found! (Exit loop) |

7 is found at position 5 (index 4) after 5 comparisons. The remaining items 2, 4, 5 are never checked.

**If the item is missing** — e.g., search for 9 — all 8 elements are checked, the flag stays 0, and "Data Not Found" is printed. This is the **worst case**: N comparisons.

### WORKED EXAMPLE

#### Linear Search in Python

```
data = [8, 3, 6, 1, 7, 2, 4, 5]
find_value = 7
value_found = 0
for ptr in range(len(data)):
    print("Step", ptr + 1, ": checking", data[ptr])
    if data[ptr] == find_value:
        value_found = 1
        break
if value_found == 1:
    print("Found", data[ptr], "at position", ptr + 1)
else:
    print("Data Not Found")
```

#### OUTPUT

```
Step 1 : checking 8
Step 2 : checking 3
Step 3 : checking 6
Step 4 : checking 1
Step 5 : checking 7
Found 7 at position 5
```

break is Python's "Exit Loop".

## Advantages and disadvantages

| Advantages                                                    | Disadvantages                                                             |
|---------------------------------------------------------------|---------------------------------------------------------------------------|
| Does not care whether data is scrambled or sorted.            | If the target is at the end of 1 million items, it does 1 million checks. |
| Extremely simple code (one loop, one if statement).           | Time grows linearly (double the list = double the time).                  |
| Faster for very small lists (no setup time, such as sorting). | Inefficient if you need to search the same list thousands of times.       |
| Does not require any extra space.                             |                                                                           |

### REMEMBER IT

Linear Search is like looking for your friend in a queue by checking every face from the front. Best case: 1 check (first person). Worst case: N checks (last person or not there).

### COMMON MISTAKE

Do not print "Not Found" inside the loop after every mismatch — that would print it many times. Decide only **after** the loop ends, using the flag. And remember to **exit the loop** once the item is found, or you waste comparisons.

### اردو میں سمجھیں

لینیئر سرچ (Linear Search) کو ترتیب وار تلاش (Sequential Search) بھی کہتے ہیں۔ اس میں فہرست کے ہر عنصر کو شروع سے ایک ایک کر کے چیک کیا جاتا ہے جب تک مطلوبہ چیز نہ مل جائے۔ یہ ترتیب شدہ اور غیر ترتیب شدہ دونوں قسم کے ڈیٹا پر کام کرتی ہے اور بہت سادہ ہے، لیکن بڑی فہرست میں بہت وقت لیتی ہے۔

### CHECK YOURSELF

1. What is another name for Linear Search?
2. In 8, 3, 6, 1, 7, 2, 4, 5, how many comparisons are needed to find 2?
3. What is the purpose of the variable ValueFound?

Answers: 1. Sequential search. 2. 6 comparisons (2 is in position 6). 3. It is a flag: 0 means not found yet, 1 means found; it decides what to print after the loop.

## 2.4.2 Searching Algorithms: Binary Search ★ Golden

### LEARN IT

**Binary Search** is a highly efficient search algorithm that works by repeatedly dividing the search interval in half.

**Crucial limitation: it only works on a SORTED list.**

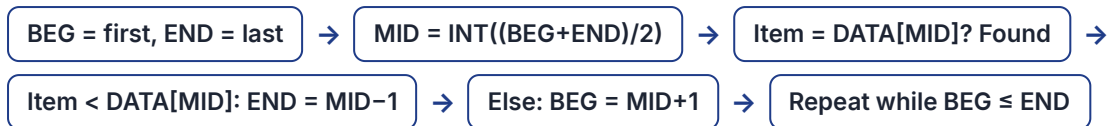
Three variables mark the part of the list still being searched:

- **BEG** (low) — first position of the current part.
- **END** (high) — last position of the current part.
- **MID** =  $\text{INT}((\text{BEG} + \text{END}) / 2)$  — the middle position. INT means "drop the decimal part".

At each step compare the item with `DATA[MID]`:

- Equal → found, stop.
- Item < `DATA[MID]` → the item can only be in the **left** half: set `END = MID - 1`.
- Item > `DATA[MID]` → the item can only be in the **right** half: set `BEG = MID + 1`.

If `BEG` becomes greater than `END`, the part is empty and the item is **not in the list**.



### Algorithm (textbook summary)

```

Step 1: Start
Step 2: DATA := {1,2,3,4,5,6,7,8,9}
         (Sorted)
Step 3: BEG = 1, END = 9
Step 4: MID = INT((BEG+END)/2)
Step 5: Item = 7
Step 6: LOC = 0
Step 7: Repeat while BEG ≤ END AND
        DATA[MID] ≠ Item
Step 8:   If Item < DATA[MID]:
            END = MID - 1
        Else:
            BEG = MID + 1
Step 9:   MID = INT((BEG+END)/2)
Step 10: [End loop]
Step 11: If DATA[MID] = Item:
            LOC = MID
        Else:
            LOC = 0
Step 12: Print result based on LOC
Step 13: End
  
```

### Pseudocode

```

Start
DATA = {1,2,3,4,5,6,7,8,9}
BEG = 1, END = 9
MID = INT((BEG+END)/2)
Item = 7
LOC = NULL
While BEG ≤ END AND DATA[MID] ≠ Item
  If Item < DATA[MID] then
    END = MID - 1
  Else
    BEG = MID + 1
  End If
  MID = INT((BEG+END)/2)
End While
If DATA[MID] = Item then
  LOC = MID
Else
  LOC = NULL
End If
If LOC ≠ NULL then
  Print DATA[MID]
Else
  Print "Not Found"
End If
End
  
```

### COMMON MISTAKE

The slide's pseudocode writes the loop condition as `BEG ≤ END OR DATA[MID] ≠ Item`. That is a slip: with OR the loop would never stop once the item is found while `BEG ≤ END`. The textbook algorithm (Step 7) correctly uses **AND** — keep going only while the part is not empty *and* the item is not yet found. Likewise, the slide sets `LOC = NULL` but then tests "`LOC ≥ 0`"; the check should simply be "`LOC is not NULL`" (or, with `LOC = 0` meaning not found, "`LOC > 0`"). The pseudocode above shows the corrected form.

### WORKED EXAMPLE

**Worked example 1 (textbook algorithm).** DATA = 1, 2, 3, 4, 5, 6, 7, 8, 9; Item = 7. Positions 1 to 9.

| Step | BEG (low) | END (high) | MID           | DATA[MID] | Compare | Action             |
|------|-----------|------------|---------------|-----------|---------|--------------------|
| 1    | 1         | 9          | INT(10/2) = 5 | 5         | 7 > 5   | BEG = 6 (go right) |
| 2    | 6         | 9          | INT(15/2) = 7 | 7         | 7 = 7   | Found, LOC = 7     |

Only **2 comparisons**. Linear Search would need 7.

### WORKED EXAMPLE

**Worked example 2 (slide example).** Sorted list: 1, 2, 3, 4, 5, 6, 7, 8. Searching item: 3.

| Step | BEG (low) | END (high) | MID                   | DATA[MID] | Compare | Action              |
|------|-----------|------------|-----------------------|-----------|---------|---------------------|
| 1    | 1         | 8          | $\text{INT}(9/2) = 4$ | 4         | $3 < 4$ | END = 3 (go left)   |
| 2    | 1         | 3          | $\text{INT}(4/2) = 2$ | 2         | $3 > 2$ | BEG = 3 (go right)  |
| 3    | 3         | 3          | $\text{INT}(6/2) = 3$ | 3         | $3 = 3$ | FOUND at position 3 |

Picture the list shrinking (the highlighted cell is MID):

| Step | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|------|---|---|---|---|---|---|---|---|
| 1    | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
| 2    | 1 | 2 | 3 |   |   |   |   |   |
| 3    |   |   | 3 |   |   |   |   |   |

The slide describes the same steps using the size of each sub-list: Step 1  $\text{MID} = 8/2 = 4$  (value 4),  $3 < 4$ , go left; Step 2 sub-list [1, 2, 3], middle =  $3/2 = 1.5 \rightarrow$  2nd item (value 2),  $3 > 2$ , go right; Step 3 sub-list [3], middle = 1st item (value 3), FOUND. Both ways land on the same values  $4 \rightarrow 2 \rightarrow 3$ .

### WORKED EXAMPLE

**Binary Search in Python** (index starts at 0, so positions 1–8 become indexes 0–7):

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
item = 3
beg = 0
end = len(data) - 1
loc = -1
step = 0
while beg ≤ end:
    mid = (beg + end) // 2
    step = step + 1
    print("Step", step, ": beg =", beg, "mid =", mid, "end =", end, "value =", data[mid])
    if data[mid] == item:
        loc = mid
        break
    elif item < data[mid]:
        end = mid - 1
    else:
        beg = mid + 1
if loc ≥ 0:
    print("Found", item, "at index", loc)
else:
    print("Not Found")
```

#### OUTPUT

```
Step 1 : beg = 0 mid = 3 end = 7 value = 4
Step 2 : beg = 0 mid = 1 end = 2 value = 2
Step 3 : beg = 2 mid = 2 end = 2 value = 3
Found 3 at index 2
```

// is integer division, the same as `INT()`. Index 2 is position 3. Here `loc = -1` means "not found", so the test `loc ≥ 0` is correct.

## Why is it so fast?

Every comparison throws away half of the remaining list.  $1,000,000 \rightarrow 500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$  takes about 20 halvings, because  $2^{20} = 1,048,576$ . So Binary Search finds an item among 1 million elements in at most about **20 steps**, while Linear Search may need 1,000,000.

## Advantages and disadvantages

| Advantages                                                                     | Disadvantages                                                                     |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Extremely fast for "Big Data": finds an item in 1M elements in just ~20 steps. | The list must be sorted first; it fails on scrambled data.                        |
| Massive time savings for static lists that are searched repeatedly.            | Requires random access to elements (works on arrays, not easily on linked lists). |
|                                                                                | More complex code, prone to logic errors (off-by-one).                            |

### REMEMBER IT

Binary Search is how you find a word in a dictionary: open the middle, decide "before or after?", and throw away half the book each time. **"Sorted first, then split."**

### COMMON MISTAKE

Common errors: (1) using Binary Search on an unsorted list — the answer will be wrong; (2) setting  $END = MID$  instead of  $MID - 1$  (or  $BEG = MID$  instead of  $MID + 1$ ), which can cause an infinite loop — this is the "off-by-one" error; (3) forgetting  $INT$ , so  $MID$  becomes a decimal like 4.5.

### اردو میں سمجھیں

بائنری سرچ (Binary Search) ایک تیز رفتار طریقہ ہے جو ہر قدم پر تلاش کے حصے کو آدھا کر دیتا ہے۔ یہ صرف ترتیب شدہ (Sorted) فہرست پر کام کرتا ہے۔ درمیانی عنصر (MID) سے موازنہ کیا جاتا ہے: اگر مطلوبہ چیز چھوٹی ہو تو بائیں آدھے حصے میں، اور بڑی ہو تو دائیں آدھے حصے میں تلاش جاری رہتی ہے۔ دس لاکھ عناصر میں بھی تقریباً بیس قدموں میں نتیجہ مل جاتا ہے۔

### EXAM TIP

You will usually be given a sorted list and an item and asked to "search using Binary Search, showing all steps". Always draw a table with **BEG, END, MID, DATA[MID], comparison, action**. Show the  $INT$  calculation for  $MID$ . If the item is absent, continue until  $BEG > END$  and write "Not Found". Also learn: "Why must the list be sorted?" and "Compare Linear and Binary Search".

### CHECK YOURSELF

1. What is the crucial condition for Binary Search to work?
2.  $BEG = 6$ ,  $END = 9$ . What is  $MID$ ?
3. In 1, 2, 3, 4, 5, 6, 7, 8 (positions 1–8), list the  $MID$  values visited when searching for 8.

Answers: 1. The list must be sorted. 2.  $INT(15/2) = 7$ . 3.  $MID = 4$  (value 4, go right,  $BEG = 5$ )  $\rightarrow MID = INT(13/2) = 6$  (value 6, go right,  $BEG = 7$ )  $\rightarrow MID = INT(15/2) = 7$  (value 7, go right,  $BEG = 8$ )  $\rightarrow MID = 8$  (value 8, found). Four comparisons.

## 2.5 Evaluation Criteria of Algorithm

### LEARN IT

Many algorithms can solve the same problem. To judge which is better, we evaluate them on three criteria: **correctness**, **efficiency** and **clarity**.

### 2.5.1 Correctness

An algorithm is **correct** if:

- its output matches the expected outcomes;
- it handles **edge cases** gracefully (unusual inputs such as an empty list, 0, a negative number, or an item that is not present);
- it is guaranteed to finish — no infinite loops.

We test correctness with a **trace table** (dry run): we feed sample inputs and record every value by hand, then compare the result with what we expect.

### WORKED EXAMPLE

**Trace table example: Even/Odd checker.** Rule: divide the number by 2; if the remainder is 0 it is Even, otherwise Odd.

| Input | Operation (divide by 2, find remainder) | Remainder == 0? | Result |
|-------|-----------------------------------------|-----------------|--------|
| 3     | 3 / 2 (Quotient = 1, Remainder = 1)     | No              | Odd    |
| 6     | 6 / 2 (Quotient = 3, Remainder = 0)     | Yes             | Even   |
| 15    | 15 / 2 (Quotient = 7, Remainder = 1)    | No              | Odd    |

All three results match what we expect, so the algorithm is correct for these inputs. The same check in Python:

```
for number in [3, 6, 15]:
    remainder = number % 2
    if remainder == 0:
        print(number, "remainder", remainder, "→ Even")
    else:
        print(number, "remainder", remainder, "→ Odd")
```

### OUTPUT

```
3 remainder 1 → Odd
6 remainder 0 → Even
15 remainder 1 → Odd
```

A good tester would also try the edge case 0 (remainder 0 → Even, which is correct).

### 2.5.2 Efficiency

We measure algorithm efficiency through **time** (speed) and **space** (memory). The key question is: how fast does it run as the input size grows — does it **scale** well?



Computer scientists describe growth with **Big O notation**. It counts roughly how many steps are needed for N items:

- **O(1)** — constant: same time, whatever N is.
- **O(log N)** — logarithmic: grows very slowly (Binary Search).
- **O(N)** — linear: double N, double the time (Linear Search).
- **O(N<sup>2</sup>)** — quadratic: double N, four times the time (Bubble and Selection Sort).

#### WORKED EXAMPLE

How the worst-case number of comparisons grows

| N (items) | Linear Search (N) | Binary Search (about $\log_2 N$ ) | Bubble/Selection Sort $N(N-1)/2$ |
|-----------|-------------------|-----------------------------------|----------------------------------|
| 10        | 10                | 4                                 | 45                               |
| 100       | 100               | 7                                 | 4,950                            |
| 1,000     | 1,000             | 10                                | 499,500                          |
| 1,000,000 | 1,000,000         | 20                                | 499,999,500,000                  |

This is why Binary Search is called excellent for Big Data and the simple sorts are called useless for it.

### 2.5.3 Clarity (Readability)

An algorithm must be easy to read, understand and modify. It should use **clear steps**, **meaningful variable names** and **good indentation**.

#### Unclear

```
a=[3,6,15]
for x in a:
    if x%2==0:print(x,"E")
    else:print(x,"O")
```

#### Clear

```
numbers = [3, 6, 15]
for number in numbers:
    if number % 2 == 0:
        print(number, "Even")
    else:
        print(number, "Odd")
```

Both work, but the second is far easier for another person (or you, a month later) to understand and change.

#### REMEMBER IT

Remember **C-E-C**: **C**orrect (right answer, always finishes), **E**fficient (fast and light), **C**lear (easy to read).

#### COMMON MISTAKE

Testing only "normal" inputs is not enough to prove correctness. Always include edge cases in your trace table, e.g., the first item, the last item, a missing item, or an empty list in a search.

#### اردو میں سمجھیں

کسی الگورتھم کو تین معیارات پر پرکھا جاتا ہے: درستگی (Correctness) یعنی درست نتیجہ دے، غیر معمولی صورتوں کو سنبھالے اور ہمیشہ ختم ہو؛ کارکردگی (Efficiency) یعنی کم وقت اور کم میموری استعمال کرے؛ اور وضاحت (Clarity) یعنی پڑھنے، سمجھنے اور تبدیل کرنے میں آسان ہو۔ درستگی جانچنے کے لیے ٹریس ٹیبل (Trace Table) بنایا جاتا ہے۔

### CHECK YOURSELF

1. Name the three evaluation criteria of an algorithm.
2. Give one example of an edge case for a search algorithm.
3. Efficiency is measured in terms of which two resources?

Answers: 1. Correctness, efficiency, clarity. 2. The item is not in the list (or the list is empty, or the item is the last element). 3. Time (speed) and space (memory).

## 2.6 How to Decide Which Algorithm to Use

### LEARN IT

Before choosing an algorithm, ask three questions:

- **What is the input size?** (10 items or 10 million?)
- **Is the data sorted?**
- **Do we need speed or memory efficiency?**

**Simple decision guide:**

- If searching on **unsorted** data → use **Linear Search**.
- If searching on **sorted** data → use **Binary Search**.
- If data needs **ordering** → use a **sorting** algorithm.

What is the task? → Need order? → Sorting algorithm → Need to find? Is data sorted? → No → Linear Search

→ Yes → Binary Search

### Comparison of the four algorithms

| Algorithm      | Type      | Best case                                            | Worst case                  | Extra memory  | Needs sorted data? | Stable? |
|----------------|-----------|------------------------------------------------------|-----------------------------|---------------|--------------------|---------|
| Bubble Sort    | Sorting   | $O(N^2)$ as written; $O(N)$ with an early-stop check | $O(N^2)$                    | None — $O(1)$ | —                  | Yes     |
| Selection Sort | Sorting   | $O(N^2)$                                             | $O(N^2)$                    | None — $O(1)$ | —                  | No      |
| Linear Search  | Searching | $O(1)$ (first item)                                  | $O(N)$ (last item / absent) | None — $O(1)$ | No                 | —       |
| Binary Search  | Searching | $O(1)$ (item at first MID)                           | $O(\log N)$                 | None — $O(1)$ | Yes                | —       |

| Point                | Bubble Sort                   | Selection Sort                            |
|----------------------|-------------------------------|-------------------------------------------|
| What is compared     | Adjacent elements (j and j+1) | Fixed position K with every later element |
| Inner loop starts at | 0                             | outer + 1                                 |
| After each pass      | Largest moves to the end      | Smallest moves to the front               |
| Swaps                | Many                          | Fewer                                     |
| Stable               | Yes                           | No                                        |

| Point                 | Linear Search                             | Binary Search                           |
|-----------------------|-------------------------------------------|-----------------------------------------|
| Method                | Checks one by one                         | Splits the list in half each step       |
| Data required         | Sorted or unsorted                        | Sorted only                             |
| Worst case (1M items) | 1,000,000 checks                          | About 20 checks                         |
| Code                  | Very simple                               | More complex, off-by-one risk           |
| Best for              | Small or unsorted lists, one-off searches | Large, sorted lists searched many times |

#### WORKED EXAMPLE

##### Choosing in practice

- **A shopkeeper's list of 15 items, written in random order, searched once:** Linear Search — the list is unsorted and tiny; sorting first would waste time.
- **NADRA looking up a CNIC number among millions of sorted records, thousands of times a day:** Binary Search — data is sorted, huge, and searched repeatedly.
- **A teacher arranging 40 students' marks into a merit list:** a sorting algorithm; for 40 items even Bubble or Selection Sort is acceptable.
- **An unsorted list of 1 million records that will be searched thousands of times:** sort it once, then use Binary Search for every search.

#### REMEMBER IT

Two keywords decide most exam answers: "**sorted?**" and "**how big?**". Unsorted or small → Linear. Sorted and big → Binary.

#### COMMON MISTAKE

"Binary Search is always better" is wrong. On unsorted data it does not work at all, and for a single search on a small unsorted list, sorting first costs more than a simple Linear Search.

#### اردو میں سمجھیں

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔ غیر ترتیب شدہ ڈیٹا میں تلاش کے لیے لینیئر سرچ، ترتیب شدہ ڈیٹا میں تلاش کے لیے بائنری سرچ، اور ڈیٹا کو ترتیب دینے کے لیے سارٹنگ الگورتھم استعمال کریں۔

#### CHECK YOURSELF

1. Which search would you use on an unsorted list of phone contacts?
2. What is the worst-case time complexity of Binary Search?
3. Which sort has the same time complexity in its best and worst case?

Answers: 1. Linear Search. 2.  $O(\log N)$ . 3. Selection Sort ( $O(N^2)$  in both).

## Chapter 2 Review

## Chapter summary

- **Computational thinking** is systematic problem solving using decomposition, pattern recognition, abstraction and algorithms.
- An **algorithm** is a step-by-step logical procedure to solve a problem; it is conceptual and language-independent.
- Algorithms give structure, optimise time and memory, are reusable, ensure accuracy and act as a blueprint for code.
- **Pseudocode** is a human-readable, structured representation of an algorithm using keywords like IF, FOR and WHILE; it is not executable.
- An algorithm defines *what* to do; pseudocode shows *how* to implement it.
- **Decomposition** breaks a big problem into smaller sub-tasks; it reduces complexity, helps debugging and promotes reusability.
- **Pattern recognition** finds repetitions and rules (Row N = N stars) and is the foundation of loops.
- **Abstraction** hides noise and keeps only essential steps; situational details become variables.
- Sorting arranges data (ascending or descending); searching locates an element's position or existence.
- **Bubble Sort** compares and swaps adjacent elements; the largest value moves to the end each pass; N – 1 passes; stable;  $O(N^2)$ .
- **Selection Sort** brings the smallest remaining value to the front each pass; inner loop starts at outer + 1; fewer swaps; not stable;  $O(N^2)$ .
- **Linear Search** checks items one by one; works on unsorted data;  $O(N)$  worst case.
- **Binary Search** halves the search range using BEG, MID and END; needs sorted data;  $O(\log N)$  — about 20 steps for 1 million items.
- Algorithms are evaluated on **correctness** (checked with trace tables and edge cases), **efficiency** (time and space) and **clarity**.
- Choose by asking: input size, sorted or not, speed or memory? Unsorted → Linear; sorted → Binary; needs order → sort.

## Key terms

| Term                   | Meaning                                                                                              | اردو                          |
|------------------------|------------------------------------------------------------------------------------------------------|-------------------------------|
| Computational thinking | A systematic approach to solving problems using decomposition, patterns, abstraction and algorithms. | کمپیوٹیشنل سوچ                |
| Algorithm              | A step-by-step logical procedure to solve a problem, independent of any language.                    | الگورتھم، مرحلہ وار طریقہ حل  |
| Pseudocode             | A human-readable, structured but non-executable representation of an algorithm.                      | سوڈو کوڈ، فرضی کوڈ            |
| Decomposition          | Breaking a complex problem into smaller, manageable parts.                                           | تقسیم کاری                    |
| Sub-task               | One small part of a decomposed problem.                                                              | ذیلی کام                      |
| Pattern recognition    | Identifying regularities, repetitions or rules in data.                                              | نمونے کی پہچان                |
| Abstraction            | Hiding unnecessary details and focusing on essentials.                                               | تجريد، غير ضروری تفصيل چھپانا |
| Noise                  | Details that do not affect the solution and can be ignored.                                          | فالتو تفصیلات                 |
| Sorting                | Arranging items in ascending or descending order.                                                    | ترتیب دینا                    |
| Searching              | Locating an element in a list to find its position or existence.                                     | تلاش کرنا                     |
| Ascending order        | Smallest to largest (0–9, A–Z).                                                                      | صعودی ترتیب                   |
| Descending order       | Largest to smallest (9–0, Z–A).                                                                      | نزولی ترتیب                   |
| Bubble Sort            | Sort that repeatedly compares and swaps adjacent elements.                                           | ببل سارٹ                      |
| Selection Sort         | Sort that repeatedly selects the smallest element and puts it at the front.                          | سلیکشن سارٹ                   |
| Pass                   | One full run of the inner loop through the list.                                                     | ایک چکر                       |
| Swap                   | Exchanging the values of two positions, usually with a temp variable.                                | جگہ کا تبادلہ                 |
| In-place sorting       | Sorting without needing extra memory for another list.                                               | اضافی میموری کے بغیر ترتیب    |

| Term                       | Meaning                                                                                 | اردو                     |
|----------------------------|-----------------------------------------------------------------------------------------|--------------------------|
| Stable sort                | A sort that keeps equal items in their original order.                                  | مستحکم ترتیب             |
| Linear (sequential) search | Checking each element one by one until the target is found.                             | ترتیب وار تلاش           |
| Binary search              | Search that halves a sorted list at each step.                                          | دو حصی تلاش              |
| BEG / MID / END            | First, middle and last positions of the part being searched in Binary Search.           | آغاز / درمیان / اختتام   |
| Flag                       | A variable (like ValueFound) that records whether something has happened.               | نشان دہی کرنے والا متغیر |
| Correctness                | Giving expected output, handling edge cases and always finishing.                       | درستگی                   |
| Efficiency                 | How well an algorithm uses time and memory as input grows.                              | کارکردگی                 |
| Clarity                    | How easy an algorithm is to read, understand and modify.                                | وضاحت                    |
| Trace table                | A table recording values step by step during a dry run.                                 | ٹریس ٹیبل، جانچ جدول     |
| Edge case                  | An unusual or extreme input used to test an algorithm.                                  | انتہائی صورت             |
| Time complexity (Big O)    | A measure of how the number of steps grows with input size, e.g. $O(N)$ , $O(\log N)$ . | وقتی پیچیدگی             |

## Practice: multiple choice

1. An algorithm is best defined as:
  - a. a programming language
  - b. a step-by-step logical procedure to solve a problem
  - c. a type of computer hardware
  - d. compiled machine code
2. Pseudocode is:
  - a. executable machine code
  - b. a symbol used in flowcharts
  - c. a Python library
  - d. a human-readable representation of an algorithm using plain English and keywords
3. Breaking a complex problem into smaller, manageable parts is called:
  - a. decomposition
  - b. abstraction
  - c. pattern recognition
  - d. compilation
4. In the "making tea" example, ignoring the brand of the kettle is an example of:
  - a. decomposition
  - b. sorting
  - c. abstraction
  - d. searching
5. Pattern recognition forms the foundation of:
  - a. variables
  - b. loops
  - c. comments
  - d. input devices
6. Bubble Sort works by comparing:
  - a. the first and last elements
  - b. every element with the middle element
  - c. adjacent elements
  - d. random pairs of elements
7. After the first pass of Bubble Sort (ascending) on 8, 4, 1, 9, 3, the list is:
  - a. 1, 3, 4, 8, 9
  - b. 4, 8, 1, 9, 3
  - c. 1, 8, 4, 9, 3
  - d. 4, 1, 8, 3, 9
8. Binary Search works only on a list that is:
  - a. sorted
  - b. unsorted
  - c. stored in a linked list
  - d. of strings only
9. Linear Search is also known as:
  - a. binary search
  - b. sequential search
  - c. bubble search
  - d. random search
10. How many passes does Bubble Sort need to sort N elements?
  - a. N
  - b. N + 1
  - c. N - 1
  - d. 2N
11. In Selection Sort pseudocode, the inner loop starts at:
  - a. 0
  - b. N - 1
  - c. outer - 1
  - d. outer + 1
12. The worst-case time complexity of Binary Search is:
  - a.  $O(\log N)$
  - b.  $O(N)$
  - c.  $O(N^2)$
  - d.  $O(1)$
13. About how many steps does Binary Search need, at most, to find an item among 1 million sorted elements?
  - a. 2
  - b. 20
  - c. 1,000
  - d. 1,000,000
14. Which sorting algorithm is NOT stable?
  - a. Bubble Sort
  - b. Both are stable
  - c. Selection Sort
  - d. Neither sorts data
15. Which of the following is NOT an evaluation criterion of an algorithm?
  - a. Correctness
  - b. Efficiency
  - c. Clarity
  - d. Popularity

## Practice: short questions

1. Define computational thinking and name its four components.
2. Write any four points on the importance of algorithms.
3. Give three differences between an algorithm and pseudocode.
4. What is decomposition? Explain it with the example of learning a new language.
5. What three questions should you ask during pattern recognition? Apply them to the triangle of stars.
6. Define abstraction. In the "making tea" example, name two variables and two noise items.
7. Differentiate between sorting and searching.
8. Why does Binary Search require a sorted list? State two disadvantages of Binary Search.
9. Give two differences between Bubble Sort and Selection Sort.
10. Name the three evaluation criteria of an algorithm. What is a trace table used for?

## Practice: long / problem questions

1. Sort the list **5, 2, 7, 1, 6** in ascending order using Bubble Sort. Show every comparison of every pass in a table, and state the total number of comparisons and swaps. Then write the Python code for Bubble Sort.
2. Sort the list **6, 2, 9, 5, 1** in ascending order using Selection Sort (textbook method). Show every comparison of every pass and count the swaps. Also state why Selection Sort is called "blind" to an already sorted list.
3. The sorted list is **3, 8, 12, 17, 23, 31, 40, 52, 66** (Python indexes 0–8). Using Binary Search, show a low / mid / high table to search for (a) 40 and (b) 20. For each, state how many comparisons Linear Search would need.
4. Write the algorithm and pseudocode for Linear Search. Dry-run it on **8, 3, 6, 1, 7, 2, 4, 5** for FindValue = 2 and for FindValue = 9, and give two advantages and two disadvantages of Linear Search.
5. A school wants a program that reads the marks of 5 subjects for every student and prints each student's total, percentage and grade. Apply decomposition, pattern recognition and abstraction to this problem, and write a short algorithm.
6. A library has 50,000 book IDs stored in sorted order, and the librarian searches it hundreds of times a day. (a) Which search algorithm should be used and why? (b) What is the maximum number of comparisons for Linear and for Binary Search? (c) Explain the three evaluation criteria you would use to judge the chosen program.

## Answer key

---

### Multiple choice

1. b
2. d
3. a
4. c
5. b
6. c
7. d
8. a
9. b
10. c
11. d
12. a
13. b
14. c
15. d



## Short questions — model answers

1. Computational thinking is a systematic approach to solving problems. Its four components are decomposition (breaking down), pattern recognition (finding repetitions), abstraction (hiding unnecessary details) and algorithms (writing step-by-step solutions).
2. Algorithms (1) provide a structured approach that makes complex tasks manageable; (2) optimise time and memory usage, crucial for large applications; (3) can be reused for similar problems; (4) ensure accuracy and consistency, reducing human error; (5) act as a blueprint before coding. (Any four.)
3. (1) An algorithm is a step-by-step logical procedure; pseudocode is a human-readable representation of it using English and keywords. (2) An algorithm is conceptual and language-independent; pseudocode is semi-formal and structured but not executable. (3) An algorithm has no fixed format and is for anyone; pseudocode uses IF/FOR/WHILE, follows conventions and is mainly for programmers.
4. Decomposition is breaking a complex task into smaller, manageable parts. Learning a language is decomposed into: vocabulary (words like "I", "eat", "apple"), grammar rules (Subject → Verb → Object), tenses ("I eat" vs "I ate") and sentence building (combining words with rules).
5. Ask: What is repeating? What changes in a predictable way? Can I generalise this? For the star triangle: stars repeat in every row; each row has one more star than the previous; generalisation: Row  $N = N$  stars, so one loop can print it.
6. Abstraction means hiding unnecessary features and focusing on the essential ones. Variables: type of tea (black, green, milk), additions (sugar, lemon), tool (stove or kettle) — any two. Noise: brand of kettle, cup colour or design, where tea is made — any two.
7. Sorting arranges a collection of items in ascending or descending order to organise data (e.g., Bubble Sort, Selection Sort). Searching locates a specific element in a list to find its position or whether it exists (e.g., Linear Search, Binary Search).
8. Binary Search decides to go left or right by comparing with the middle value; this decision is only valid if all smaller values are on the left and larger on the right, i.e., the list is sorted. Disadvantages: needs sorted data; needs random access (arrays, not linked lists); code is more complex and prone to off-by-one errors. (Any two.)
9. (1) Bubble Sort compares adjacent elements; Selection Sort compares a fixed position with all later elements. (2) Bubble Sort moves the largest to the end each pass; Selection Sort moves the smallest to the front. Also: Bubble Sort is stable, Selection Sort is not; Selection Sort makes fewer swaps.
10. Correctness, efficiency (time and space) and clarity (readability). A trace table records the values of variables step by step during a dry run, so we can check that the output matches the expected result and so prove correctness.

## Long questions — model answers

### 1. Bubble Sort on 5, 2, 7, 1, 6

| Pass | Pair   | Left > Right? | List after    |
|------|--------|---------------|---------------|
| 1    | [5, 2] | Yes, swap     | 2, 5, 7, 1, 6 |
| 1    | [5, 7] | No            | 2, 5, 7, 1, 6 |
| 1    | [7, 1] | Yes, swap     | 2, 5, 1, 7, 6 |
| 1    | [7, 6] | Yes, swap     | 2, 5, 1, 6, 7 |
| 2    | [2, 5] | No            | 2, 5, 1, 6, 7 |
| 2    | [5, 1] | Yes, swap     | 2, 1, 5, 6, 7 |
| 2    | [5, 6] | No            | 2, 1, 5, 6, 7 |
| 3    | [2, 1] | Yes, swap     | 1, 2, 5, 6, 7 |
| 3    | [2, 5] | No            | 1, 2, 5, 6, 7 |
| 4    | [1, 2] | No            | 1, 2, 5, 6, 7 |

Total comparisons = 4 + 3 + 2 + 1 = 10; total swaps = 3 + 1 + 1 + 0 = 5. Sorted list: 1, 2, 5, 6, 7.

```
data = [5, 2, 7, 1, 6]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
print(data)
```

#### OUTPUT

```
[1, 2, 5, 6, 7]
```

### 2. Selection Sort (textbook method) on 6, 2, 9, 5, 1

| Pass | Compare      | Values | Action | List after    |
|------|--------------|--------|--------|---------------|
| 1    | Pos1 vs Pos2 | 6 > 2  | Swap   | 2, 6, 9, 5, 1 |
| 1    | Pos1 vs Pos3 | 2 > 9  | No     | 2, 6, 9, 5, 1 |
| 1    | Pos1 vs Pos4 | 2 > 5  | No     | 2, 6, 9, 5, 1 |
| 1    | Pos1 vs Pos5 | 2 > 1  | Swap   | 1, 6, 9, 5, 2 |
| 2    | Pos2 vs Pos3 | 6 > 9  | No     | 1, 6, 9, 5, 2 |
| 2    | Pos2 vs Pos4 | 6 > 5  | Swap   | 1, 5, 9, 6, 2 |
| 2    | Pos2 vs Pos5 | 5 > 2  | Swap   | 1, 2, 9, 6, 5 |
| 3    | Pos3 vs Pos4 | 9 > 6  | Swap   | 1, 2, 6, 9, 5 |
| 3    | Pos3 vs Pos5 | 6 > 5  | Swap   | 1, 2, 5, 9, 6 |
| 4    | Pos4 vs Pos5 | 9 > 6  | Swap   | 1, 2, 5, 6, 9 |

10 comparisons, 7 swaps. Sorted list: 1, 2, 5, 6, 9. (With the classic one-swap-per-pass method the passes give 1, 2, 9, 5, 6 → 1, 2, 9, 5, 6 → 1, 2, 5, 9, 6 → 1, 2, 5, 6, 9.)

Selection Sort is "blind" to a sorted list because its loops have fixed limits: it always makes all  $N(N-1)/2$  comparisons, even if the list is already in order. It has no way to notice that no swaps were needed and stop early.

### 3. (a) Search for 40

| Step | low | high | mid            | data[mid] | Compare   | Action           |
|------|-----|------|----------------|-----------|-----------|------------------|
| 1    | 0   | 8    | $(0+8)//2 = 4$ | 23        | $40 > 23$ | low = 5          |
| 2    | 5   | 8    | $(5+8)//2 = 6$ | 40        | $40 = 40$ | Found at index 6 |

Binary: 2 comparisons. Linear Search: 40 is the 7th item, so 7 comparisons.

### (b) Search for 20

| Step | low | high | mid                                             | data[mid] | Compare   | Action   |
|------|-----|------|-------------------------------------------------|-----------|-----------|----------|
| 1    | 0   | 8    | 4                                               | 23        | $20 < 23$ | high = 3 |
| 2    | 0   | 3    | $(0+3)//2 = 1$                                  | 8         | $20 > 8$  | low = 2  |
| 3    | 2   | 3    | $(2+3)//2 = 2$                                  | 12        | $20 > 12$ | low = 3  |
| 4    | 3   | 3    | 3                                               | 17        | $20 > 17$ | low = 4  |
| —    | 4   | 3    | low > high, so the loop stops: <b>Not Found</b> |           |           |          |

Binary: 4 comparisons. Linear Search must check all 9 items before saying "Not Found".

4. Algorithm and pseudocode: as in section 2.4.2 (Start; DATA, N, FindValue, ValueFound := 0; repeat for PTR = 1 to N: if DATA[PTR] = FindValue then ValueFound = 1 and exit; after the loop print DATA[PTR] if ValueFound = 1, else "Not Found"; End).

**FindValue = 2:**

| Step    | 1  | 2  | 3  | 4  | 5  | 6          |
|---------|----|----|----|----|----|------------|
| Element | 8  | 3  | 6  | 1  | 7  | 2          |
| == 2?   | No | No | No | No | No | Yes — exit |

Found at position 6 after 6 comparisons; ValueFound = 1; prints 2.

**FindValue = 9:** 8, 3, 6, 1, 7, 2, 4, 5 are all checked (8 comparisons), none equals 9, ValueFound stays 0, so "Data Not Found" is printed.

Advantages: works on sorted and unsorted data; very simple code; fast for small lists; no extra space. Disadvantages: up to N checks (1 million for 1 million items); time grows linearly; inefficient if the same list is searched thousands of times. (Any two of each.)

5. **Decomposition:** (1) input a student's 5 marks; (2) calculate total; (3) calculate percentage =  $\text{total} \div 500 \times 100$ ; (4) decide grade; (5) print result; (6) repeat for the next student.

**Pattern recognition:** the same steps repeat for every student, and the same "add a mark" step repeats for every subject — so use a loop over students and a loop over subjects.

**Abstraction:** keep only roll number, marks, total, percentage and grade; ignore noise such as the student's address, uniform or the colour of the result card. The number of students can be a variable.

**Algorithm:** Step 1: Start. Step 2: Read number of students S. Step 3: Repeat Steps 4–8 for each student. Step 4: TOTAL = 0. Step 5: Repeat 5 times: read MARK, TOTAL = TOTAL + MARK. Step 6: PER = TOTAL / 500 × 100. Step 7: If PER ≥ 80 grade "A1", else if PER ≥ 70 "A", else if PER ≥ 60 "B", else if PER ≥ 50 "C", else "Fail" (school's own grading scale). Step 8: Print TOTAL, PER, GRADE. Step 9: End.

6. (a) **Binary Search**, because the data is already sorted, it is large (50,000 items) and it is searched many times a day, so the time saved per search adds up; Linear Search would be far slower.

(b) Linear Search worst case: 50,000 comparisons. Binary Search worst case: 16 comparisons, because  $2^{15} = 32,768 < 50,000 \leq 2^{16} = 65,536$ , so 16 halvings are enough to reduce 50,000 items to one.

(c) **Correctness:** it must return the right position for IDs that exist, say "Not Found" for missing IDs (edge cases: first ID, last ID, missing ID), and always stop. **Efficiency:** time  $O(\log N)$  and no extra memory ( $O(1)$ ). **Clarity:** meaningful names like low, mid, high, clear steps and good indentation so another programmer can maintain it.

### Self-assessment: I can...

- ☐ define computational thinking and name its four parts.
- ☐ draw a table comparing algorithm and pseudocode on at least five points.
- ☐ find a pattern in data and turn it into a loop.
- ☐ dry-run Bubble Sort pass by pass and count comparisons and swaps.
- ☐ trace Linear Search with a step table, including the "not found" case.
- ☐ state best and worst time complexity for all four algorithms.
- ☐ define an algorithm and give five reasons it is important.
- ☐ decompose a real-life problem into sub-tasks and explain three benefits.
- ☐ separate necessary steps, variables and noise using abstraction.
- ☐ dry-run Selection Sort pass by pass and explain why it is not stable.
- ☐ trace Binary Search with a BEG / MID / END table and explain why it needs sorted data.
- ☐ evaluate an algorithm for correctness, efficiency and clarity, and choose the right algorithm for a situation.

## CHAPTER 3

# Programming Fundamentals

In this chapter you start writing real programs in Python. You will learn how programs store data, take input, make decisions, repeat work, use ready-made libraries, and how to find and fix your own mistakes.

اس باب میں آپ پائتھن (Python) میں حقیقی پروگرام لکھنا سیکھیں گے۔ آپ جانیں گے کہ پروگرام ڈیٹا کیسے محفوظ کرتا ہے، فیصلے کیسے کرتا ہے، کام کو کیسے دہراتا ہے، اور غلطیاں کیسے تلاش کی جاتی ہیں۔

### What you will learn

- Define a program and a programming language, and compare low-, mid- and high-level languages.
- Describe the features of an IDE and the main parts of the VS Code window.
- Take input with `input()`, convert it with `int()` / `float()`, and display output with `print()` and f-strings.
- Use sequence, selection (`if`, `if-else`, `if-elif-else`, nested `if`) and repetition (`for`, `while`, `range()`, `break`, `continue`, nested loops).
- Identify syntax, runtime and logical errors and read a traceback to fix a bug.
- Name career areas where Python is used.
- Write Python programs that use comments, correct indentation, valid variable names and the main data types.
- Evaluate expressions with arithmetic, assignment, relational, logical, membership and bitwise operators, including binary working.
- Import and use built-in libraries (`math`, `random`, `datetime`) and install third-party libraries with `pip`.

## 3.1 Programming

### LEARN IT

Computers are "dumb" machines. They are very fast, but they cannot think or do anything on their own. They need exact, step-by-step **instructions**.

- A set of instructions given to a computer is called a **program**.
- The language used to write these instructions is called a **programming language**.
- Writing programs is called **programming**, and the person who writes them is a **programmer**.

Think of a recipe. A cook follows the recipe line by line: "boil water, add rice, wait 15 minutes". A computer follows a program in the same way, one instruction at a time, without guessing what you meant.

#### WORKED EXAMPLE

**A tiny program.** Here is a complete Python program with one instruction. It tells the computer to show a message on the screen.

```
print("Hello")
```

OUTPUT  
Hello

The word `print` is the instruction; `"Hello"` is the data it works on.

#### REMEMBER IT

**Program = recipe, programmer = recipe writer, computer = cook who follows every line exactly.** If the recipe is wrong, the food is wrong. The cook does not fix it for you.

#### COMMON MISTAKE

Students often think the computer "understands" what they want. It does not. If you type `Print("Hello")` with a capital P, Python stops with an error, because it only knows `print`. Be exact.

#### اردو میں سمجھیں

کمپیوٹر خود سے کچھ نہیں سوچ سکتا۔ اسے کام کرنے کے لیے ہدایات چاہئیں۔ ہدایات کے مجموعے کو پروگرام (Program) کہتے ہیں، اور جس زبان میں یہ ہدایات لکھی جاتی ہیں اسے پروگرامنگ لینگویج (Programming Language) کہتے ہیں۔

#### CHECK YOURSELF

1. What is a program?
2. Why do we say computers are "dumb" machines?

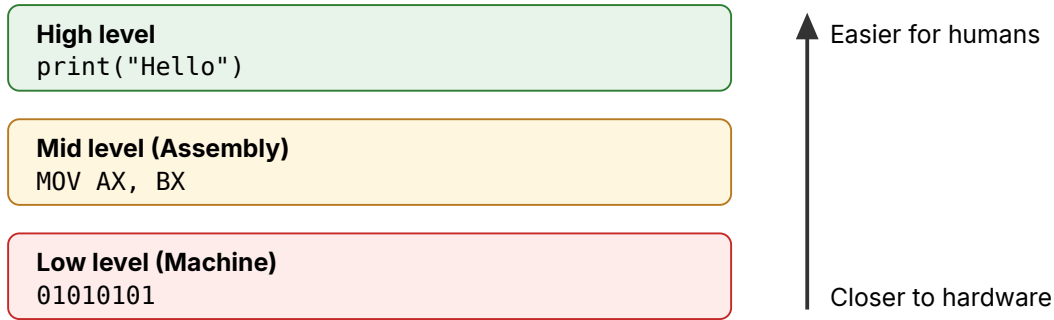
Answers: 1. A set of instructions given to a computer to perform a task. 2. Because they cannot think or do anything without instructions; they only follow exactly what they are told.

## 3.1.1 Programming Language Types

#### LEARN IT

Programming languages are classified into **three main types**, depending on how close they are to the computer's own language (binary 0s and 1s) or to human language.

| Type       | Description                                                                                                                                                                | Examples                  |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Low level  | Closest to the native language of the computer (binary 0s and 1s). Very difficult for humans to read and write.                                                            | Machine language (binary) |
| Mid level  | Bridges the gap. It can access hardware but is easier to read than binary because it uses short words called <b>mnemonics</b> (e.g. <code>MOV</code> , <code>ADD</code> ). | Assembly language         |
| High level | Designed to be easy for humans to read and write. Uses English-like words.                                                                                                 | C++, Java, Python         |



The three levels of programming languages, with a sample line from each.

## Common high-level languages

| Language | Key idea                         | Used for                                                                         |
|----------|----------------------------------|----------------------------------------------------------------------------------|
| C++      | Powerful and closer to hardware. | System software, operating systems, high-performance computer games.             |
| Java     | "Write once, run anywhere."      | Large enterprise applications, Android mobile apps, server programming.          |
| Python   | Simple, readable syntax.         | Data Science, Artificial Intelligence, Web Development, and learning to program. |

### WORKED EXAMPLE

**Same task, three levels.** Suppose you want to copy a value.

- Machine language: a pattern like `01010101` — you would need a chart to know what it means.
- Assembly: `MOV AX, BX` — "move the value in register BX into register AX". Shorter words, but you still need to know the hardware.
- Python: `a = b` — you simply say "a gets the value of b".

The higher the level, the less you need to know about the hardware and the faster you can write the program.

### REMEMBER IT

**"Low = Like the machine, High = Human."** Mid level sits in the Middle and uses Mnemonics (both start with M).

### COMMON MISTAKE

"Low level" does not mean "bad" or "weak". Low-level code runs very fast and controls hardware directly. It is just harder for humans to write.

### اردو میں سمجھیں

پروگرامنگ زبانوں کی تین اقسام ہیں: لو لیول (Low Level) جو بائری 0 اور 1 میں ہوتی ہے، مڈ لیول (Mid Level) یعنی اسمبلی جو مختصر الفاظ (Mnemonics) استعمال کرتی ہے، اور ہائی لیول (High Level) جیسے پائتھن، جاوا اور سی پلس پلس جو انگریزی جیسے الفاظ استعمال کرتی ہیں اور انسانوں کے لیے آسان ہیں۔

### CHECK YOURSELF

1. Which level is `MOV AX, BX`?
2. Which high-level language is famous for "write once, run anywhere"?
3. Give one use of C++.

Answers: 1. Mid level (assembly language). 2. Java. 3. Any one: system software, operating systems, high-performance games.

## 3.2 Career Growth with Python

### LEARN IT

Python is one of the most in-demand languages in the world. Learning it opens many career paths:

- **Software Development:** build desktop and web applications.
- **Data Science and Data Analysis:** analyse data, create reports and make predictions.
- **Artificial Intelligence and Machine Learning:** develop intelligent systems, chatbots and recommendation algorithms (like "videos you may like").
- **Web Development:** use frameworks such as **Django** and **Flask** to build websites.
- **Automation and Scripting:** automate repetitive office or lab tasks (renaming 500 files, filling spreadsheets).
- **Cybersecurity and Networking:** write security tools and analyse network data.

**Why Python?** Python lets you focus on **problem-solving and logic** instead of memorising complex syntax rules. Compare printing a message in Python with C++:

```
# Python
print("Hello")
```

```
// C++
#include <iostream>
int main() {
    std::cout << "Hello";
    return 0;
}
```

### WORKED EXAMPLE

**Automation example.** A school clerk must write "Roll No. 1" to "Roll No. 5" on a list. Python does it in two lines:

```
for n in range(1, 6):
    print("Roll No.", n)
```

#### OUTPUT

```
Roll No. 1
Roll No. 2
Roll No. 3
Roll No. 4
Roll No. 5
```

Change `6` to `501` and it writes 500 lines just as fast. (You will learn loops in 3.5.3.)

### REMEMBER IT

Six careers — remember **"S-D-A-W-A-C"**: **S**oftware, **D**ata, **A**I, **W**eb, **A**utomation, **C**ybersecurity.

#### COMMON MISTAKE

Django and Flask are **web frameworks** (for websites), not AI tools. AI/ML uses libraries such as TensorFlow and PyTorch (see 3.6.2).

#### اردو میں سمجھیں

پائٹھن سیکھ کر آپ سافٹ ویئر ڈیولپمنٹ، ڈیٹا سائنس، مصنوعی ذہانت (AI)، ویب ڈیولپمنٹ، آٹومیشن اور سائبر سیکیورٹی جیسے شعبوں میں کیریئر بنا سکتے ہیں۔ پائٹھن کا آسان انداز آپ کو مشکل اصول یاد کرنے کے بجائے مسئلہ حل کرنے پر توجہ دینے دیتا ہے۔

#### CHECK YOURSELF

1. Name two Python web frameworks.
2. Which career area builds chatbots and recommendation systems?

Answers: 1. Django and Flask. 2. Artificial Intelligence and Machine Learning.

### 3.3 Integrated Development Environment (IDE)

#### LEARN IT

An **IDE (Integrated Development Environment)** is a software tool that helps programmers **write, run and manage code in one place**. It is a complete workspace for programming. "Integrated" means many tools are joined together in one window.

#### Key features of an IDE:

- **Code Editor:** where you write and edit code.
- **Syntax Highlighting:** colours different parts of the code (keywords, strings, numbers) to make it easier to read.
- **Auto-completion:** suggests code while you type (type `pri` and it offers `print`).
- **Run/Execute Tool:** runs the program with one click.
- **Debugger:** helps you find and fix errors, for example by pausing the program at a chosen line.

#### WORKED EXAMPLE

**Without vs with an IDE.** Without an IDE you would write code in Notepad, save it, open a terminal and type `python file.py`, then go back to Notepad to fix mistakes. With an IDE you write, run (press the ► Run button) and debug in the same window, and errors are underlined while you type.

#### REMEMBER IT

IDE features — "**E-S-A-R-D**": Editor, Syntax highlighting, Auto-completion, Run, Debugger.

#### COMMON MISTAKE

An IDE does not write the program for you and does not catch logical mistakes. It only makes writing, running and debugging easier.

#### اردو میں سمجھیں

آئی ڈی ای (IDE) ایک ایسا سافٹ ویئر ہے جس میں کوڈ لکھنا، چلانا اور غلطیاں درست کرنا ایک ہی جگہ ممکن ہوتا ہے۔ اس کی اہم خصوصیات میں کوڈ ایڈیٹر، رنگین کوڈ (Syntax Highlighting)، خودکار تجاویز (Auto completion)، رن بٹن اور ڈیبگر شامل ہیں۔



#### CHECK YOURSELF

1. What does "integrated" mean in IDE?
2. Which IDE feature suggests code while you type?

Answers: 1. Many programming tools (editor, runner, debugger) are combined in one program. 2. Auto-completion.

### 3.3.1 Visual Studio Code (VS Code)

#### LEARN IT

**VS Code** is a **free, lightweight and powerful code editor** made by Microsoft and widely used for Python. By default it is **not a full IDE**, but it becomes one when you install **extensions** (add-ons), such as the **Python extension by Microsoft**.

#### VS Code interface elements

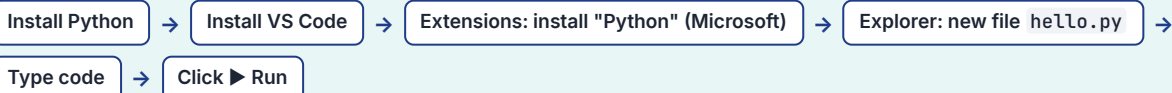
| Element           | Description                                                                                            |
|-------------------|--------------------------------------------------------------------------------------------------------|
| 1. Top Menu Bar   | Control over files, editing, navigation and workspace (File, Edit, Selection, View, Go, Run).          |
| 2. Side Bar       | Contains the core tools: Explorer, Search, Source Control, Run & Debug and Extensions.                 |
| 3. Explorer       | Shows project files and folders. Helps you create, open and manage your Python files.                  |
| 4. Search         | Finds and replaces text in the open file or across the whole project.                                  |
| 5. Source Control | Connects with Git (a version control system) to track changes, undo mistakes and collaborate.          |
| 6. Run & Debug    | Runs code, finds errors, pauses at specific lines and lets you inspect variables.                      |
| 7. Extensions     | Like an App Store for VS Code. Add features such as the Python extension, Prettier or Python Debugger. |



A simplified VS Code window: menu bar, side bar, Explorer and the code editor.

#### WORKED EXAMPLE

##### Setting up and running your first program in VS Code:



```
print("Hello, World!")
```

**OUTPUT**  
Hello, World!

The output appears in the **Terminal** panel at the bottom of the window.

#### REMEMBER IT

Extensions are the **"App Store"** of VS Code: just as apps give your phone new abilities, extensions give VS Code new abilities.

#### COMMON MISTAKE

Always save Python files with the `.py` extension (e.g. `hello.py`). Without it, VS Code does not know the file is Python, so there is no syntax highlighting and no Run button.

#### اردو میں سمجھیں

وی ایس کوڈ (VS Code) ایک مفت اور ہلکا کوڈ ایڈیٹر ہے۔ یہ اپنے آپ میں مکمل آئی ڈی ای نہیں، لیکن ایکسٹینشنز (Extensions) لگانے سے آئی ڈی ای بن جاتا ہے۔ اس کے اہم حصے مینو بار، سائیڈ بار، ایکسپلورر، سرچ، سورس کنٹرول، رن اینڈ ڈیبگ اور ایکسٹینشنز ہیں۔

#### CHECK YOURSELF

1. Which VS Code element connects to Git?
2. Is VS Code a full IDE by default? How does it become one?
3. Where do you see and manage your project's files?

Answers: 1. Source Control. 2. No; it becomes an IDE by installing extensions (e.g. the Python extension). 3. Explorer.

## 3.4 Fundamentals of Python Programming ★ Golden

#### LEARN IT

The basic structure of a Python program is simple and clear. A program takes data, **processes it using statements**, and finally **shows output**. Python runs the statements **top to bottom, one after another** (sequentially).

A **statement** is one complete instruction, usually written on one line, such as `print("Hi")` or `x = 5`.

1. **Comments (#)** — Comments are notes written in the code **for humans**. They start with the `#` symbol. The Python interpreter completely **ignores** them. Use them to explain what your code does.
2. **Indentation (spaces)** — C++ and Java use curly brackets `{ }` to mark a **block** of code (a group of statements that belong together). Python instead uses **indentation**: spaces at the beginning of a line. Indentation is **mandatory** in Python. The standard is **4 spaces** (or 1 tab).

```
# This is a comment. It explains the code below.
print("Hello, World!")
if 5 > 2:
    # The space before 'print' is the indentation
    print("Five is greater than two!")
```

#### OUTPUT

```
Hello, World!
Five is greater than two!
```

### WORKED EXAMPLE

#### Reading the program line by line:

1. Line 1 starts with `#`, so Python skips it.
2. Line 2 prints `Hello, World!`.
3. Line 3 asks a question: is 5 greater than 2? The colon `:` says "a block follows".
4. Line 4 is a comment inside the block (ignored).
5. Line 5 is indented by 4 spaces, so it belongs to the `if` block. Because `5 > 2` is true, it runs.

#### What happens if you forget the indentation?

```
if 5 > 2:
print("Five is greater than two!")
```

#### OUTPUT

```
File "indent.py", line 2
  print("Five is greater than two!")
  ^
IndentationError: expected an indented block after 'if' statement on line 1
```

Python refuses to run the program and points to line 2. Add 4 spaces before `print` to fix it.

### REMEMBER IT

Think of indentation like the points and sub-points in your notebook. Sub-points are written a little to the right to show they "belong" to the main point. A colon `:` at the end of a line always means "**indent the next line**".

### COMMON MISTAKE

**IndentationError?** Check your spaces. **Mixing tabs and spaces** can break a Python program even when the code looks lined up. Choose 4 spaces and use them everywhere (VS Code inserts 4 spaces when you press Tab in a `.py` file).

### اردو میں سمجھیں

پائتھن پروگرام اوپر سے نیچے ترتیب سے چلتا ہے۔ `#` سے شروع ہونے والی لائن تبصرہ (Comment) ہوتی ہے جسے پائتھن نظرانداز کر دیتا ہے۔ پائتھن میں کوڈ کے بلاک کو بریکٹ کے بجائے انڈینٹیشن (Indentation) یعنی لائن کے شروع میں چار خالی جگہوں سے ظاہر کیا جاتا ہے، اور یہ لازمی ہے۔

### EXAM TIP

Common questions: "What is the purpose of comments?", "Why is indentation important in Python?", "How is Python different from C++/Java in defining blocks?" Write: comments start with `#` and are ignored by the interpreter; Python uses indentation (4 spaces) instead of `{ }`; wrong indentation gives an `IndentationError`. Always give a 3–4 line code example.

### CHECK YOURSELF

#### 1. Predict the output:

```
# print("A")
print("B")
if 3 > 7:
    print("C")
print("D")
```

#### 2. What is the standard amount of indentation in Python?

Answers: 1. B then D on separate lines (line 1 is a comment; `3 > 7` is false so C is skipped). 2. 4 spaces (or 1 tab).

## 3.4.1 Variables ★ Golden

### LEARN IT

A **variable** is a **named space in computer memory used to store data**. Think of it as a labelled container (a box with a name written on it). You put a value in it with the **assignment operator** `=`, which means "store", not "equals".

In Python you **do not need to declare the type** of a variable. Python figures it out automatically from the value. This is called **dynamic typing**: the same variable can later hold a different type.

#### Rules for naming variables:

- Can contain **letters, numbers and underscores** (`_`) only — no spaces or symbols like `-`, `@`, `$`.
- **Must start with a letter or underscore**, NOT a number (`1name` is invalid).
- Are **case-sensitive**: `age` and `Age` are two completely different variables.
- Cannot be a Python **keyword** (a reserved word such as `if`, `for`, `while`, `else`, `True`).

#### Valid names ✓

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
```

#### Invalid names ✗

```
1name = "Ali" # starts with a number
my height = 5.7 # contains a space
if = 100 # keyword used
```

### WORKED EXAMPLE

#### Using variables and case-sensitivity:

```
name = "Ali"
age = 16
my_height = 5.7
score1 = 100
print(name, age, my_height, score1)

Age = 17 # a different variable from age
print(age, Age)
```

#### OUTPUT

```
Ali 16 5.7 100
16 17
```

**Dynamic typing** — the type follows the value. `type()` tells you the type:

```
x = 5
print(x, type(x))
x = "five"
print(x, type(x))
```

#### OUTPUT

```
5 <class 'int'>
five <class 'str'>
```

#### What Python says about an invalid name:

```
1name = "Ali"
```

#### OUTPUT

```
File "names.py", line 1
  1name = "Ali"
    ^
SyntaxError: invalid decimal literal
```

#### REMEMBER IT

Choose **meaningful names**: `total_marks` is far clearer than `tm` or `x`. Use lowercase words joined by underscores (this style is called **snake\_case**).

#### COMMON MISTAKE

`=` is **not** "equals" in the maths sense. `x = x + 1` is not a false equation; it means "take the value of `x`, add 1, and store the result back in `x`". To **compare** two values you use `==` (see 3.4.3).

#### اردو میں سمجھیں

ویری ایبل (Variable) میموری میں ایک نام والی جگہ ہے جس میں ڈیٹا رکھا جاتا ہے، بالکل ایک لیبل لگے ڈیے کی طرح۔ پائتھن میں ٹائپ بتانے کی ضرورت نہیں ہوتی۔ نام حرف یا انڈر سکور سے شروع ہونا چاہیے، اس میں خالی جگہ نہیں ہو سکتی، یہ کیس سینسیٹو (Case-sensitive) ہوتا ہے اور کوئی کی ورڈ (Keyword) نہیں ہو سکتا۔

#### EXAM TIP

A favourite question: "Which of these variable names are valid? Give reasons." Check each name against all four rules and state the rule broken, e.g. "`2nd_place` — invalid, starts with a digit". Remember that `While` (capital W) is **valid** because keywords are lowercase and Python is case-sensitive.

#### CHECK YOURSELF

1. Valid or invalid? `_total`, `roll-no`, `class`, `marks2026`, `2marks`
2. Predict the output:

```
city = "Karachi"
City = "Hyderabad"
print(city)
```

Answers: 1. `_total` valid; `roll-no` invalid (hyphen); `class` invalid (keyword); `marks2026` valid; `2marks` invalid (starts with a digit). 2. `Karachi` — `city` and `City` are different variables.

### 3.4.1 Data Types in Python ★ Golden

#### LEARN IT

A **data type** specifies the **kind of data** a variable holds — a whole number, a decimal, text, True/False, or a collection of items. The type decides what you can do with the data (you can divide numbers, but not names). Python has several built-in data types:

| Name    | Type             | Description                                                  | Example               |
|---------|------------------|--------------------------------------------------------------|-----------------------|
| Integer | <code>int</code> | Whole numbers without decimals (positive, negative or zero). | <code>age = 16</code> |

| Name           | Type               | Description                                            | Example                                                 |
|----------------|--------------------|--------------------------------------------------------|---------------------------------------------------------|
| Floating point | <code>float</code> | Numbers with a decimal point.                          | <code>height = 5.7</code>                               |
| String         | <code>str</code>   | Text data, enclosed in quotes ( " " or ' ' ).          | <code>name = "Sammy" or<br/>year = "2026"</code>        |
| Boolean        | <code>bool</code>  | Logical value: True or False (capital T / F).          | <code>is_student = True</code>                          |
| List           | <code>list</code>  | Ordered, changeable collection in square brackets [ ]. | <code>marks = [85, 90, 75]</code>                       |
| Tuple          | <code>tuple</code> | Ordered, unchangeable sequence in parentheses ( ).     | <code>coords = (10, 20)</code>                          |
| Dictionary     | <code>dict</code>  | Key : value pairs in curly braces { }.                 | <code>student = {"name":<br/>"Ahmed", "age": 16}</code> |
| Set            | <code>set</code>   | Unordered collection of unique items (no duplicates).  | <code>vowels = {'a', 'e',<br/>'i'}</code>               |

Note: the slide abbreviates tuple as "tup"; the real Python type name is `tuple`, as the program below shows. The slides describe dictionaries as unordered; since Python 3.7 a dictionary remembers the order in which keys were added, but you always look items up **by key**, not by position.

#### WORKED EXAMPLE

##### Checking every type with `type()` :

```
age = 16
height = 5.7
name = "Sammy"
year = "2026"
is_student = True
marks = [85, 90, 75]
coords = (10, 20)
student = {"name": "Ahmed", "age": 16}
vowels = {'a', 'e', 'i'}

print(type(age))
print(type(height))
print(type(name), type(year))
print(type(is_student))
print(type(marks))
print(type(coords))
print(type(student))
print(type(vowels))
```

**OUTPUT**

```
<class 'int'>
<class 'float'>
<class 'str'> <class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Notice that `year = "2026"` is a **string**, not a number, because it is inside quotes.

**Working with the collection types:**

```
marks = [85, 90, 75]
marks[0] = 88          # lists are changeable
marks.append(60)
print(marks)

coords = (10, 20)
print(coords[0])       # read item at position 0

student = {"name": "Ahmed", "age": 16}
print(student["name"]) # look up by key

nums = {1, 2, 2, 3, 3, 3} # duplicates are removed
print(nums)
print(len(nums))
```

**OUTPUT**

```
[88, 90, 75, 60]
10
Ahmed
{1, 2, 3}
3
```

Positions (called **indexes**) start at **0**, so `coords[0]` is the first item. Trying to change a tuple causes an error:

```
coords = (10, 20)
coords[0] = 50
```

**OUTPUT**

```
Traceback (most recent call last):
  File "tuple_test.py", line 2, in <module>
    coords[0] = 50
    ~~~~~^
TypeError: 'tuple' object does not support item assignment
```

**REMEMBER IT**

**Brackets decide the collection:** `[]` List (changeable), `()` Tuple (locked), `{ }` with **key:value** = Dictionary, `{ }` with single items = Set (unique). Think: a **list** is a shopping list you can edit; a **tuple** is your date of birth — it never changes.

#### COMMON MISTAKE

True and False must start with a capital letter. true is not a Boolean; Python treats it as an unknown variable name and gives a NameError. Also, "16" (string) and 16 (int) are different: "16" + "1" gives "161", while 16 + 1 gives 17.

#### اردو میں سمجھیں

ڈیٹا ٹائپ (Data Type) بتاتی ہے کہ ویری ایبل میں کس قسم کا ڈیٹا ہے۔ int مکمل اعداد، float اعشاریہ والے اعداد، str متن، اور bool صرف True یا False کے لیے ہے۔ لسٹ (List) تبدیل ہو سکتی ہے، ٹپل (Tuple) تبدیل نہیں ہو سکتا، ڈکشنری (Dictionary) کی اور ویلیو کے جوڑے ہوتے ہیں، اور سیٹ (Set) میں کوئی چیز دہرائی نہیں جاتی۔

#### EXAM TIP

Expect: "Name any four data types with examples" or "Differentiate list and tuple". For a difference question, use a two-column table: list — [ ], changeable, e.g. marks = [85, 90]; tuple — ( ), unchangeable, e.g. coords = (10, 20).

#### CHECK YOURSELF

1. What is the data type of each? 3.0, "3", 3, False, [3]
2. Predict the output: print(len({5, 5, 6, 6, 6}))
3. Which type should store a student's CNIC/B-Form number such as "42101-1234567-1"? Why?

Answers: 1. float, str, int, bool, list. 2. 2 (the set keeps only 5 and 6). 3. str, because it contains hyphens and you never do arithmetic on it.

## 3.4.2 Input and Output Handling

#### LEARN IT

**Output — the print() function.** A **function** is a named, ready-made piece of code that does a job when you call it with brackets. print() displays information on the screen.

- Items separated by commas are printed with one space between them.
- An **f-string** (a string with f before the quote) lets you put variables directly inside { }. It is the fastest and most readable way to format output.

**Input — the input() function.** It shows a message (a **prompt**), **pauses the program** and waits for the user to type something and press Enter. Whatever the user types is **always a string** by default.

**Type casting** means converting a value from one type to another: int() converts to an integer, float() to a decimal number, str() to a string.

```
# 1. Print only a string
print("Hello Class!")

# 2. Print a string and a variable (comma separates them)
name = "Ali"
print("Welcome,", name)

# 3. f-strings (fastest and most readable way to format)
age = 16
print(f"My name is {name} and I am {age} years old.")
```



**OUTPUT**

```
Hello Class!
Welcome, Ali
My name is Ali and I am 16 years old.
```

Two useful extras: `sep` changes the separator between items, and `end` changes what is printed at the end (normally a new line).

```
print("10", "09", "2026", sep="-")
print("Loading", end="...")
print("done")
print(f"Total: {7 * 3}")
```

**OUTPUT**

```
10-09-2026
Loading...done
Total: 21
```

**WORKED EXAMPLE****Taking input with type casting.**

```
# Taking text input
user_name = input("Enter your name: ")
# Type casting: converting string input to an integer (int)
age = int(input("Enter your age: "))
# Type casting: converting string input to a decimal (float)
height = float(input("Enter your height: "))
print(f"Hello {user_name}! Next year you will be {age + 1}.")
print("Height:", height, "feet")
```

Sample run (the user types Sara, 16 and 5.4):

**OUTPUT**

```
Enter your name: Sara
Enter your age: 16
Enter your height: 5.4
Hello Sara! Next year you will be 17.
Height: 5.4 feet
```

**Why type casting matters.** Without `int()`, the numbers stay as text and `+` joins them:

```
a = input("First number: ")
b = input("Second number: ")
print(a + b)
print(int(a) + int(b))
```

**OUTPUT**

```
First number: 5
Second number: 3
53
8
```

"5" + "3" joins two strings into "53" (called **concatenation**). 5 + 3 adds numbers to give 8.

**REMEMBER IT**

**"input() always gives a string."** If you plan to do maths, wrap it: `int(input(...))` for whole numbers, `float(input(...))` for decimals. Read it from the inside out: first `input`, then convert.

#### COMMON MISTAKE

If the user types letters where a number is expected, `int()` crashes with `ValueError: invalid literal for int() with base 10` (see 3.7). Also, `int("5.4")` fails — use `float()` for decimal input.

#### اردو میں سمجھیں

`print()` اسکرین پر معلومات دکھاتا ہے اور `input()` صارف سے معلومات لیتا ہے۔ یاد رکھیں کہ `input()` ہمیشہ اسٹرنگ (String) دیتا ہے، اس لیے حساب کے لیے اسے `int()` یا `float()` سے تبدیل کرنا ضروری ہے۔ اسے ٹائپ کاسٹنگ (Type Casting) کہتے ہیں۔ ایف اسٹرنگ (f-string) سے ویری ایبلز کو جملے کے اندر آسانی سے لکھا جا سکتا ہے۔

#### CHECK YOURSELF

1. Predict the output:

```
a = "3"
b = 4
print(a * b)
print(int(a) * b)
```

2. Find the error: the user types 21 but the output is 2121, not 42.

```
num = input("Enter a number: ")
print(num * 2)
```

Answers: 1. 3333 then 12 (a string times a number repeats the string). 2. `input()` returns the string "21", and a string times 2 repeats it. Fix: `num = int(input("Enter a number: "))`.

### 3.4.3 Operators and Operands: Arithmetic and Assignment ★ Golden

#### LEARN IT

An **operand** is the value or variable being worked on. An **operator** is the symbol that performs the action. Together they form an **expression**, which Python evaluates to a single value.

In `10 + 20`: **10** is an operand, **+** is the operator, **20** is an operand, and the result is **30**.

1. **Arithmetic operators** do maths:

| Operator | Purpose                                 | Example | Result |
|----------|-----------------------------------------|---------|--------|
| +        | Addition                                | 5 + 3   | 8      |
| -        | Subtraction                             | 10 - 4  | 6      |
| *        | Multiplication                          | 4 * 2   | 8      |
| /        | Division (result is always a float)     | 9 / 2   | 4.5    |
| //       | Floor division (drops the decimal part) | 9 // 2  | 4      |
| %        | Modulus (remainder)                     | 9 % 2   | 1      |
| **       | Exponentiation (power)                  | 2 ** 3  | 8      |

```

print(5 + 3)
print(10 - 4)
print(4 * 2)
print(9 / 2)
print(9 // 2)
print(9 % 2)
print(2 ** 3)
print(8 / 2)

```

#### OUTPUT

```

8
6
8
4.5
4
1
8
4.0

```

Notice the last line: `8 / 2` gives `4.0`, not `4`, because `/` always gives a float.

**Order of operations (precedence).** Python follows the same order as maths (like BODMAS): brackets first, then `**`, then `*` / `//` / `%` (left to right), then `+` / `-` (left to right). `**` is the exception: it works right to left.

```

print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(-9 // 2)
print(17 % 5, 17 // 5)

```

#### OUTPUT

```

14
20
512
3
-5
2 3

```

`2 ** 3 ** 2 = 2 ** 9 = 512` (right to left). `-9 // 2` is `-5` because floor division always rounds **down** to the smaller whole number (`-4.5 → -5`).

**2. Assignment operators** store a value in a variable, often updating it. The short forms are called **compound assignment**.

| Operator                                                                | Example             | Equivalent to                      |
|-------------------------------------------------------------------------|---------------------|------------------------------------|
| <code>=</code>                                                          | <code>x = 5</code>  | Assign 5 to x                      |
| <code>+=</code>                                                         | <code>x += 2</code> | <code>x = x + 2</code>             |
| <code>-=</code>                                                         | <code>x -= 1</code> | <code>x = x - 1</code>             |
| <code>*=</code>                                                         | <code>x *= 3</code> | <code>x = x * 3</code>             |
| <code>/=</code> , <code>/!=</code> , <code>%=</code> , <code>**=</code> | <code>x /= 2</code> | <code>x = x / 2</code> (and so on) |

### WORKED EXAMPLE

**Tracing assignment operators.** Follow the value of `x` line by line:

```
x = 5
x += 2      # x = 5 + 2
print(x)
x -= 1      # x = 7 - 1
print(x)
x *= 3      # x = 6 * 3
print(x)
x /= 2      # x = 18 / 2
print(x)
```

#### OUTPUT

```
7
6
18
9.0
```

**Using % and // to split digits.** `% 10` gives the last digit; `// 10` removes it.

```
n = 357
last = n % 10      # remainder after dividing by 10
rest = n // 10     # drop the last digit
print("Last digit:", last)
print("Remaining:", rest)
```

#### OUTPUT

```
Last digit: 7
Remaining: 35
```

$357 \div 10 = 35$  remainder 7. So `357 // 10 = 35` and `357 % 10 = 7`.

### REMEMBER IT

**Three kinds of division:** `/` = exact answer (float), `//` = quotient only ("floor"), `%` = remainder only. A number is **even** when `n % 2 == 0`.

### COMMON MISTAKE

`^` is **not** power in Python — it is bitwise XOR (see bitwise operators). `2 ^ 3` gives 1, not 8. For power, always use `**`.

### اردو میں سمجھیں

آپرینڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔ `/` ہمیشہ اعشاریہ والا جواب دیتا ہے، `//` صرف حاصل تقسیم دیتا ہے، `%` باقی دیتا ہے اور `**` طاقت (Power) نکالتا ہے۔ `x += 2` کا مطلب ہے `x = x + 2`۔

- 2

### EXAM TIP

You will be asked to evaluate expressions such as `17 // 5`, `17 % 5`, `2 ** 4` or to trace compound assignment. Show each step and write the final value with its type (e.g. `9.0` is a float, so write the `.0`).

### CHECK YOURSELF

1. Predict the output:

```
x = 7
y = 2
print(x // y, x % y, x / y)
print(x ** y)
```

2. Predict the output:

```
x = 10
x += 5
x /= 4
print(x)
```

Answers: 1. 3 1 3.5 then 49 . 2. 3 (10 + 5 = 15; 15 // 4 = 3).

## 3.4.3 Relational, Logical and Membership Operators ★ Golden

### LEARN IT

**3. Relational (comparison) operators** compare two values and return a **Boolean** ( True or False ). They are used in conditions.

| Op | Description              | Example (a = 10, b = 5) | Result |
|----|--------------------------|-------------------------|--------|
| =  | Equal to                 | a == b                  | False  |
| ≠  | Not equal to             | a != b                  | True   |
| >  | Greater than             | a > b                   | True   |
| <  | Less than                | a < b                   | False  |
| ≥  | Greater than or equal to | a ≥ 10                  | True   |
| ≤  | Less than or equal to    | a ≤ b                   | False  |

```
a = 10
b = 5
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a ≥ 10)
print(a ≤ b)
```

### OUTPUT

```
False
True
True
False
True
False
```

**4. Logical operators** combine several conditions:

| A     | B     | A and B | A or B | not A |
|-------|-------|---------|--------|-------|
| False | False | False   | False  | True  |
| False | True  | False   | True   | True  |
| True  | False | False   | True   | False |
| True  | True  | True    | True   | False |

- `and` — True **only if both** conditions are true.
- `or` — True if **at least one** condition is true.
- `not` — **reverses** the result (True becomes False, False becomes True).

```
a = 10
b = 5
print(a > 5 and b > 5)
print(a > 5 or b > 5)
print(not a > 5)
```

#### OUTPUT

```
False
True
False
```

**5. Membership operators** check whether a value exists inside a sequence (such as a string or list): `in` and `not in`.

```
name = "Python"
print("P" in name)
print("z" not in name)
print("p" in name)
marks = [85, 90, 75]
print(90 in marks)
```

#### OUTPUT

```
True
True
False
True
```

"p" in name is False because strings are case-sensitive: the word has a capital P.

#### WORKED EXAMPLE

**Real-life conditions.** A person can apply for a CNIC if they are 18 or older **and** have the documents. Marks are invalid if they are below 0 **or** above 100.

```
age = 20
has_cnic = True
print(age ≥ 18 and has_cnic)
marks = 45
print(marks < 0 or marks > 100)
```

#### OUTPUT

```
True
False
```

Step by step:  $20 \geq 18$  is True, `has_cnic` is True, True and True = **True**. For marks:  $45 < 0$  is False,  $45 > 100$  is False, False or False = **False** (the marks are valid).

#### REMEMBER IT

**AND is strict, OR is generous.** AND is like a teacher who needs both homework and attendance; OR is like a shop that accepts cash **or** card. Order of evaluation: **not** first, then **and**, then **or**.

#### COMMON MISTAKE

Do not confuse `=` (assignment, "store") with `==` (comparison, "is it equal?"). Writing `if x = 5:` is a **syntax error**; the correct form is `if x == 5:`.

#### اردو میں سمجھیں

ریشنل آپریٹرز (Relational Operators) دو ویلیوز کا موازنہ کر کے True یا False دیتے ہیں۔ لاجیکل آپریٹرز میں **and** تب True دیتا ہے جب دونوں شرطیں درست ہوں، **or** تب جب کم از کم ایک درست ہو، اور **not** نتیجہ الٹ دیتا ہے۔ **in** اور **not in** دیکھتے ہیں کہ کوئی چیز اسٹرنگ یا لسٹ میں موجود ہے یا نہیں۔

#### EXAM TIP

Typical: "Evaluate `a > b` and `b > 3` for `a = 10`, `b = 5`" or "Write the truth table of **and** / **or**". Write each sub-condition's value first, then combine. Remember: `==` compares, `=` assigns.

#### CHECK YOURSELF

1. Predict the output:

```
print(True and False or True)
print(not (5 > 3) or 2 == 2.0)
```

2. Find the error:

```
x = 5
if x = 5:
    print("five")
```

Answers: 1. True and True. (True and False = False; False or True = True. not True = False; 2 == 2.0 is True; False or True = True.) 2. = is assignment; a comparison needs ==. Python reports SyntaxError: invalid syntax. Maybe you meant '=' or ':=' instead of '='? Fix: `if x == 5:`

### 3.4.3 Bitwise Operators ★ Golden

#### LEARN IT

**6. Bitwise operators** treat numbers as **binary digits** (bits: 0s and 1s) and work on them **bit by bit**, column by column. Think of each bit as a switch that is ON (1) or OFF (0). Python's `bin()` function shows the binary form of a number (it starts with `0b`).

- **AND ( & )** — gives 1 **only if both** bits are 1, otherwise 0.
- **OR ( | )** — gives 1 if **at least one** bit is 1.
- **XOR ( ^ )** — gives 1 if the bits are **different**, 0 if they are the **same**.
- **NOT ( ~ )** — inverts all bits ( $0 \rightarrow 1$ ,  $1 \rightarrow 0$ ). In Python the result is the number made negative, minus 1:  $\sim x = -(x + 1)$ . So  $\sim 10$  gives  $-11$ .
- **Left shift ( << )** — moves bits left and adds 0s on the right. Each shift **multiplies by 2**.
- **Right shift ( >> )** — moves bits right and removes the rightmost bits. Each shift **divides by 2** (floor division).

| Bit A | Bit B | A & B | A   B | A ^ B | ~A |
|-------|-------|-------|-------|-------|----|
| 0     | 0     | 0     | 0     | 0     | 1  |
| 0     | 1     | 0     | 1     | 1     | 1  |
| 1     | 0     | 0     | 1     | 1     | 0  |
| 1     | 1     | 1     | 1     | 0     | 0  |

#### WORKED EXAMPLE

**Worked example: a = 10, b = 6.** First convert to binary using place values 8, 4, 2, 1: 10 = 8 + 2 = **1010**; 6 = 4 + 2 = **0110**. Then work column by column.

|        | 8 | 4 | 2 | 1 | Decimal |
|--------|---|---|---|---|---------|
| a = 10 | 1 | 0 | 1 | 0 | 10      |
| b = 6  | 0 | 1 | 1 | 0 | 6       |
| a & b  | 0 | 0 | 1 | 0 | 2       |
| a   b  | 1 | 1 | 1 | 0 | 14      |
| a ^ b  | 1 | 1 | 0 | 0 | 12      |

- AND: only the "2" column has 1 in both → 0010 = **2**.
- OR: columns 8, 4 and 2 have at least one 1 → 1110 = 8 + 4 + 2 = **14**.
- XOR: columns 8 and 4 are different; column 2 is the same (both 1) → 1100 = 8 + 4 = **12**.
- NOT:  $\sim 10 = -(10 + 1) = \mathbf{-11}$ .

```
a = 10
b = 6
print(bin(a), bin(b))
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
```

#### OUTPUT

```
0b1010 0b110
2
14
12
-11
```

**Worked example: shifts.** 5 in 8 bits is **0000 0101**.

| Expression | Binary    | Decimal | What happened                     |
|------------|-----------|---------|-----------------------------------|
| 5          | 0000 0101 | 5       | start                             |
| 5 << 1     | 0000 1010 | 10      | add one 0 on the right (5 × 2)    |
| 5 << 2     | 0001 0100 | 20      | add 00 on the right (5 × 4)       |
| 20 >> 1    | 0000 1010 | 10      | remove the rightmost bit (20 ÷ 2) |
| 20 >> 2    | 0000 0101 | 5       | remove two bits (20 ÷ 4)          |



```

a = 5          # Binary: 0000 0101
print(a << 1)   # 0000 1010
print(a << 2)   # 0001 0100
b = 20         # Binary: 0001 0100
print(b >> 1)   # 0000 1010
print(b >> 2)   # 0000 0101
print(bin(20 >> 2))

```

#### OUTPUT

```

10
20
10
5
0b101

```

#### REMEMBER IT

**Shortcuts:**  $x \ll n = x \times 2^n$ ,  $x \gg n = x // 2^n$ ,  $\sim x = -x - 1$ . For AND/OR/XOR always write both numbers in binary, one under the other, with the same number of bits.

#### COMMON MISTAKE

Do not mix up the logical and bitwise operators: `and` / `or` / `not` work on True/False conditions, while `&` / `|` / `~` work on the bits of integers. Also remember `bin(6)` prints `0b110`, without the leading zero — add zeros yourself when lining up columns.

#### اردو میں سمجھیں

بٹ وائز آپریٹرز (Bitwise Operators) اعداد کو بائری میں بدل کر ہر بٹ پر الگ الگ کام کرتے ہیں۔ `&` تب 1 دیتا ہے جب دونوں بٹ 1 ہوں، `|` تب جب کوئی ایک بھی 1 ہو، اور `^` تب جب دونوں بٹ مختلف ہوں۔ `~` تمام بٹ الٹ دیتا ہے، اس لیے `10~` کا جواب 11- ہے۔ لیفٹ شفٹ عدد کو 2 سے ضرب دیتا ہے اور رائٹ شفٹ 2 سے تقسیم کرتا ہے۔

#### EXAM TIP

Exams ask you to "evaluate `12 & 10`, `12 | 10`, `12 ^ 10`, `~5`, `7 >> 1` showing binary working". Marks are given for the binary conversion and column working, not just the final number. Always draw the column table.

#### CHECK YOURSELF

1. Predict the output (show the binary working):

```

print(12 & 10, 12 | 10, 12 ^ 10)
print(~5, 7 >> 1, 3 << 3)

```

2. What does `bin(13)` print?

Answers: 1. 8 14 6 and -6 3 24.  $12 = 1100$ ,  $10 = 1010$ : AND = 1000 = 8, OR = 1110 = 14, XOR = 0110 = 6.  $\sim 5 = -(5 + 1) = -6$ .  $7 = 111 \rightarrow 11 = 3$ .  $3 = 11 \rightarrow 11000 = 24$  ( $3 \times 8$ ). 2. `0b1101` ( $8 + 4 + 1$ ).

## 3.5 Control Structures ★ Golden

#### LEARN IT

**Control structures** decide the **order in which statements are executed** in a program (the "flow of control"). Almost every program is built from just three basic structures:

1. **Sequence** — do the steps one after another, top to bottom.
2. **Selection** — choose a path depending on a condition (`if`, `else`).
3. **Repetition** — repeat steps (loops: `for`, `while`).

#### WORKED EXAMPLE

##### Morning routine as control structures:

- Sequence: wake up → brush teeth → have breakfast.
- Selection: **if** it is raining, take an umbrella; **else** take a cap.
- Repetition: **while** the bus has not come, keep waiting.

#### REMEMBER IT

Remember "**S-S-R**": Sequence, Selection, Repetition — "Step, Select, Repeat".

#### COMMON MISTAKE

Selection and repetition both use conditions, but they are different: selection runs a block **at most once**; repetition can run a block **many times**.

#### اردو میں سمجھیں

کنٹرول اسٹرکچرز (Control Structures) طے کرتے ہیں کہ پروگرام کی ہدایات کس ترتیب سے چلیں گی۔ تین بنیادی اسٹرکچر ہیں: سیکوئنس (ترتیب وار)، سلیکشن (شرط کی بنیاد پر انتخاب) اور ریپیٹیشن (دہرانا یعنی لوپ)۔

#### EXAM TIP

"Name and explain the three basic control structures" is a very common short question. Give a one-line definition and a two-line Python example for each.

#### CHECK YOURSELF

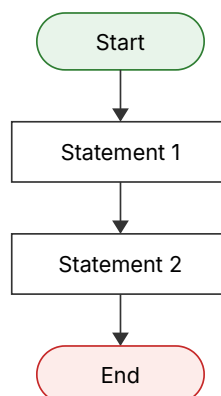
1. Name the three basic control structures.
2. Which structure is used for "keep asking for the password until it is correct"?

Answers: 1. Sequence, selection, repetition. 2. Repetition (a `while` loop).

## 3.5.1 Sequence

#### LEARN IT

**Sequence** means statements are executed **one after another, strictly from top to bottom**, in the exact order they are written. This is the **default behaviour** of Python. No statement is skipped or repeated.



Sequence: control flows straight down from one statement to the next.

### WORKED EXAMPLE

Input → Process → Output in sequence.

```
print("Enter two numbers")
a = int(input("Enter first number: ")) # Step 1: Input
b = int(input("Enter second number: ")) # Step 2: Input
total = a + b # Step 3: Process
print("Sum =", total) # Step 4: Output
```

Sample run (user types 7 and 5):

#### OUTPUT

```
Enter two numbers
Enter first number: 7
Enter second number: 5
Sum = 12
```

Each line runs exactly once, in order. If you moved line 4 ( `total = a + b` ) above the input lines, Python would stop with a `NameError` because `a` and `b` would not exist yet.

### REMEMBER IT

Sequence is like reading a page: line 1, then line 2, then line 3. Most programs follow the pattern **Input → Process → Output**.

### COMMON MISTAKE

Order matters. A variable must be given a value **before** it is used. Using it first causes `NameError: name '...' is not defined`.

### اردو میں سمجھیں

سیکوانس (Sequence) کا مطلب ہے کہ ہدایات بالکل اسی ترتیب سے ایک کے بعد ایک اوپر سے نیچے چلتی ہیں جس ترتیب سے لکھی گئی ہیں۔ یہ پائنتھن کا عام طریقہ ہے۔ کسی ویری ایبل کو استعمال کرنے سے پہلے اس میں ویلیو رکھنا ضروری ہے۔

### CHECK YOURSELF

1. Predict the output:

```
x = 4
y = x * 2
x = 10
print(x, y)
```

2. What is the default control structure in Python?

Answers: 1. 10 8 — `y` was calculated when `x` was still 4; changing `x` later does not change `y`. 2. Sequence.

## 3.5.2 Selection Statements: if and if-else ★ Golden

### LEARN IT

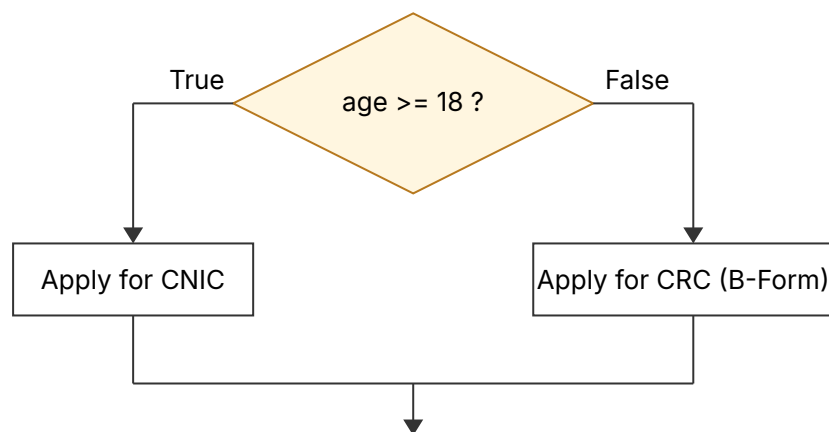
**Selection** changes the flow of a program based on a **condition** — an expression that is either True or False. It helps a program respond intelligently to different situations.

**1. The if statement** executes a block of code **only if** the condition is True. If the condition is False, the block is completely skipped.

```
if condition:
    statement(s)    # runs only when condition is True
```

**2. The if-else statement** provides **two paths**: if the condition is true, do this; **else** (otherwise), do that. Exactly one of the two blocks always runs.

```
if condition:
    block A        # condition True
else:
    block B        # condition False
```



An if-else statement: one diamond (condition), two paths, then the paths join again.

### WORKED EXAMPLE

**The if statement.**

```
age = int(input("Enter your age: "))
if age >= 18:
    print("You are an Adult.")    # runs only if age >= 18
print("Program continues...")    # runs every time
```

Run 1 (user types 20):

```
OUTPUT
Enter your age: 20
You are an Adult.
Program continues...
```

Run 2 (user types 15):

```
OUTPUT
Enter your age: 15
Program continues...
```

The last line is not indented, so it is outside the `if` block and runs every time.

### The if-else statement.

```
age = int(input("Enter your age: "))
if age ≥ 18:
    print("Apply for CNIC.")
else:
    print("Apply for CRC (B-Form).")
```

Sample run (user types 16):

```
OUTPUT
Enter your age: 16
Apply for CRC (B-Form).
```

Even or odd — a classic if-else program:

```
n = 7
if n % 2 == 0:
    print(n, "is even")
else:
    print(n, "is odd")
```

```
OUTPUT
7 is odd
```

7 % 2 = 1, and 1 == 0 is False, so the `else` block runs.

#### REMEMBER IT

**Formatting rule:** put a **colon ( : )** after `if` and `else`, and **indent** the code inside the blocks. `else` never has a condition of its own.

#### COMMON MISTAKE

Common mistakes: forgetting the colon ( `SyntaxError: expected ':'` ), writing a condition after `else` ( `else age < 18: is wrong` ), and not indenting `else` at the same level as its `if`.

#### اردو میں سمجھیں

سلیکشن (Selection) میں پروگرام شرط کی بنیاد پر فیصلہ کرتا ہے۔ `if` کا بلاک صرف تب چلتا ہے جب شرط درست (True) ہو۔ `if-else` میں دو راستے ہوتے ہیں: شرط درست ہو تو پہلا بلاک، ورنہ `else` والا بلاک چلتا ہے۔ `if` اور `else` کے بعد کولن (:) اور اندر کے کوڈ کی انڈینٹیشن ضروری ہے۔

#### EXAM TIP

Asked as: "Write a program that inputs a number and prints whether it is even or odd / positive or negative / pass or fail." Write the full program with `int(input())`, the condition, both blocks, and a sample run.

### CHECK YOURSELF

1. Predict the output when `marks = 32`:

```
marks = 32
if marks ≥ 33:
    print("Pass")
else:
    print("Fail")
print("Done")
```

2. Find the two errors:

```
n = 5
if n > 0
    print("Positive")
else n ≤ 0:
    print("Not positive")
```

Answers: 1. Fail then Done . 2. Missing colon after `if n > 0`; `else` must not have a condition — write `else: .`

## 3.5.2 Selection Statements: if-elif-else and Nested if ★ Golden

### LEARN IT

3. The **if-elif-else** statement handles **many conditions**. (`elif` is short for "else if".) Python checks the conditions **from top to bottom**. As soon as it finds a True condition, it runs that block and **ignores all the rest**. If none is True, the `else` block runs.

4. **Nested if** is an `if` inside another `if`. It is used when a decision needs to pass **several levels of checks**: the inner check happens only if the outer check passed.

### WORKED EXAMPLE

Grading with if-elif-else.

```
marks = int(input("Enter marks: "))
if marks ≥ 80:
    print("A-1 Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 60:
    print("B Grade")
else:
    print("Fail or below B")
```

Sample run (user types 75):

#### OUTPUT

```
Enter marks: 75
A Grade
```

Trace for different marks:

| marks | >= 80? | >= 70?        | >= 60?        | Output          |
|-------|--------|---------------|---------------|-----------------|
| 92    | True   | (not checked) | (not checked) | A-1 Grade       |
| 75    | False  | True          | (not checked) | A Grade         |
| 60    | False  | False         | True          | B Grade         |
| 41    | False  | False         | False         | Fail or below B |

### Loan approval with nested if.

```
age = 20
salary = 35000
if age >= 18:
    # passed first check
    if salary >= 30000:
        # passed second check
        print("Loan APPROVED!")
    else:
        print("Salary too low for loan.")
else:
    print("Too young for loan.")
```

#### OUTPUT

Loan APPROVED!

| age | salary | Outer check (age<br>>= 18) | Inner check<br>(salary >= 30000) | Output                   |
|-----|--------|----------------------------|----------------------------------|--------------------------|
| 20  | 35000  | True                       | True                             | Loan APPROVED!           |
| 20  | 25000  | True                       | False                            | Salary too low for loan. |
| 16  | 35000  | False                      | (never checked)                  | Too young for loan.      |

When you only need a yes/no answer, two levels can often be joined with `and` — but then you lose the separate messages:

```
age = 20
salary = 25000
if age >= 18 and salary >= 30000:
    print("Loan APPROVED!")
else:
    print("Loan not approved.")
```

#### OUTPUT

Loan not approved.

### REMEMBER IT

In an `if-elif-else` ladder, put the **strictest condition first** (highest marks first). Think of a ladder: you step off at the first rung that fits.

### COMMON MISTAKE

**Wrong order gives wrong answers.** Here 85 marks gets "B Grade", because  $85 \geq 60$  is True first and the other conditions are ignored:

```
marks = 85
if marks ≥ 60:
    print("B Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 80:
    print("A-1 Grade")
```

**OUTPUT**  
B Grade

This is a **logical error**: no crash, just a wrong result. Fix it by checking  $\geq 80$  first.

### اردو میں سمجھیں

if-elif-else میں کئی شرطیں اوپر سے نیچے ترتیب سے دیکھی جاتی ہیں، اور جو شرط سب سے پہلے درست ہو اسی کا بلاک چلتا ہے، باقی سب چھوڑ دی جاتی ہیں۔ نیسٹڈ ایف (Nested if) میں ایک if دوسرے if کے اندر ہوتا ہے، اور اندر والی شرط تب ہی دیکھی جاتی ہے جب باہر والی شرط درست ہو۔

### EXAM TIP

Grade-calculator programs (if-elif-else) and "largest of three numbers" are very common long questions. Check conditions in the right order, indent nested blocks carefully (8 spaces for the inner block), and give a sample run. Tracing tables like the ones above earn full marks in "predict the output" questions.

### CHECK YOURSELF

1. Predict the output:

```
temp = 30
if temp > 35:
    print("Hot")
elif temp > 25:
    print("Warm")
elif temp > 15:
    print("Pleasant")
else:
    print("Cold")
```

2. Predict the output:

```
x = 15
if x > 10:
    if x > 20:
        print("A")
    else:
        print("B")
else:
    print("C")
```

Answers: 1. Warm (30 > 35 is False; 30 > 25 is True, so the rest are skipped even though 30 > 15 is also True). 2. B (outer check 15 > 10 passes; inner check 15 > 20 fails).



### 3.5.3 Repetition Statements: the for Loop and range() ★ Golden

#### LEARN IT

A **repetition statement (loop)** lets a block of code run **again and again**. This greatly reduces the need to write the same code many times. Each single run of the loop body is called an **iteration**.

**1. The for loop** is used when the **exact number of iterations is known**. It takes each value from a sequence one by one and stores it in the **loop variable** (e.g. `i`).

The `range()` function generates the sequence of numbers for the loop:

| Syntax                                | How it works                                   | Example                     | Values produced |
|---------------------------------------|------------------------------------------------|-----------------------------|-----------------|
| <code>range(stop)</code>              | From 0 up to, but <b>not including</b> , stop. | <code>range(3)</code>       | 0 1 2           |
| <code>range(start, stop)</code>       | From start up to, but not including, stop.     | <code>range(2, 5)</code>    | 2 3 4           |
| <code>range(start, stop, step)</code> | From start towards stop, jumping by step.      | <code>range(1, 6, 2)</code> | 1 3 5           |

```
for i in range(3):  
    print(i)
```

#### OUTPUT

```
0  
1  
2
```

```
for i in range(1, 6, 2):  
    print(i)
```

#### OUTPUT

```
1  
3  
5
```

```
for i in range(2, 5):  
    print(i)
```

#### OUTPUT

```
2  
3  
4
```

```
for i in range(0, 6, 2):  
    print("Even:", i)
```

#### OUTPUT

```
Even: 0  
Even: 2  
Even: 4
```

You can see all the values of a range at once by converting it to a list. A negative step counts **down**; if start equals stop, the range is empty:

```
print(list(range(5)))  
print(list(range(2, 5)))  
print(list(range(1, 6, 2)))  
print(list(range(10, 0, -2)))  
print(list(range(5, 5)))
```

#### OUTPUT

```
[0, 1, 2, 3, 4]  
[2, 3, 4]  
[1, 3, 5]  
[10, 8, 6, 4, 2]  
[]
```

### WORKED EXAMPLE

#### Multiplication table of 7:

```
for i in range(1, 6):  
    print(f"7 x {i} = {7 * i}")
```

```
OUTPUT  
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
7 x 5 = 35
```

**Adding numbers with an accumulator.** An **accumulator** is a variable that starts at 0 and collects a running total.

```
total = 0  
for i in range(1, 11):  
    total += i  
print("Sum of 1 to 10 =", total)
```

```
OUTPUT  
Sum of 1 to 10 = 55
```

| i           | 1 | 2 | 3 | 4  | 5  | ... | 10 |
|-------------|---|---|---|----|----|-----|----|
| total after | 1 | 3 | 6 | 10 | 15 | ... | 55 |

**Looping over a string or list.** A `for` loop can also take items directly from any sequence:

```
for ch in "Sindh":  
    print(ch, end=" ")  
print()  
for m in [85, 90, 75]:  
    print(m + 5)
```

```
OUTPUT  
S i n d h  
90  
95  
80
```

### REMEMBER IT

**range stops one short:** `range(1, 11)` gives 1 to 10. To include a number `n`, write `range(..., n + 1)`. Number of values in `range(a, b) = b - a`.

### COMMON MISTAKE

Writing `range(1, 10)` to print 1 to 10 misses 10 — an "off-by-one" logical error. Also, the loop body must be indented; a `for` line without an indented block gives `IndentationError: expected an indented block after 'for' statement`.

### اردو میں سمجھیں

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔ `for` لوپ تب استعمال ہوتا ہے جب پہلے سے معلوم ہو کہ کتنی بار دہرانا ہے۔ `range(stop)` صفر سے شروع ہوتا ہے، `range(start, stop)` سے، اور `range(start, stop, step)` ہر بار `step` کے برابر چھلانگ لگاتا ہے۔ آخری عدد (`stop`) کبھی شامل نہیں ہوتا۔

### EXAM TIP

Expect "predict the output" questions on `range()` with a step, and programs such as "print a table", "sum of first n numbers", "even numbers from 1 to 20". Always state what range produces before tracing.

### CHECK YOURSELF

1. Predict the output:

```
for i in range(10, 0, -3):  
    print(i, end=" ")
```

2. Predict the output:

```
s = 0  
for k in range(2, 11, 2):  
    s += k  
print(s)
```

3. Find the error:

```
for i in range(3):  
print(i)
```

Answers: 1. 10 7 4 1. 2. 30 (2 + 4 + 6 + 8 + 10). 3. The body is not indented: `IndentationError`. Indent `print(i)` by 4 spaces.

## 3.5.3 The while Loop, break and continue ★ Golden

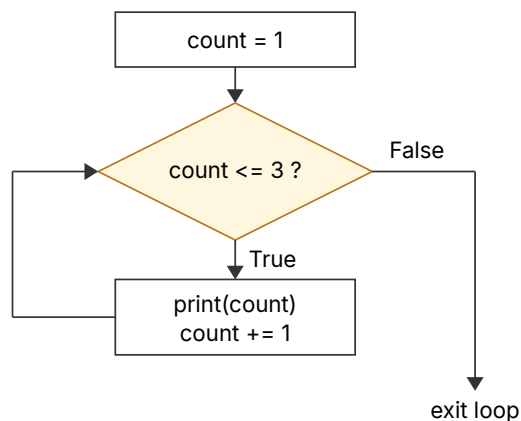
### LEARN IT

**2. The while loop** repeats a block of code **as long as a condition remains True**. It is used when the exact number of iterations is **not known in advance** (for example, "keep asking until the password is correct").

A while loop needs three parts: (1) **initialise** a variable before the loop, (2) **test** the condition, (3) **update** the variable inside the loop. If you forget the update, the condition never becomes False and the loop never stops — an **infinite loop**.

**Loop control statements** change the normal flow of a loop:

- **break** — exits (stops) the loop **entirely and immediately**.
- **continue** — **skips the rest of the current iteration** and jumps to the next one.



A while loop: test the condition, run the body, go back and test again.

```
count = 1
while count ≤ 3:
    print(count)
    count += 1 # VERY IMPORTANT: update the condition!
```

#### OUTPUT

```
1
2
3
```

#### WORKED EXAMPLE

**Trace of the while loop above:**

| count | count ≤ 3? | Printed | count after update |
|-------|------------|---------|--------------------|
| 1     | True       | 1       | 2                  |
| 2     | True       | 2       | 3                  |
| 3     | True       | 3       | 4                  |
| 4     | False      | —       | loop ends          |

**Sum of digits** — we do not know how many digits the number has, so `while` fits well:

```
n = 4072
total = 0
while n > 0:
    digit = n % 10
    total += digit
    n = n // 10
print("Sum of digits =", total)
```

#### OUTPUT

```
Sum of digits = 13
```

Digits taken: 2, 7, 0, 4 → 2 + 7 + 0 + 4 = 13. `n` becomes 407, 40, 4, 0, and then the loop stops.

**Password check** — repeat until the user gets it right:

```
password = input("Enter password:")
while password ≠ "sindh123":
    print("Wrong password, try again.")
    password = input("Enter password: ")
print("Welcome!")
```

Sample run (user types `abc`, `sindh`, `sindh123`):

#### OUTPUT

```
Enter password: abc
Wrong password, try again.
Enter password: sindh
Wrong password, try again.
Enter password: sindh123
Welcome!
```

### break and continue side by side:

```
for i in range(5):
    if i == 3:
        break
    print(i)
print("Loop ended")
```

```
OUTPUT
0
1
2
Loop ended
```

At  $i = 3$  the loop stops completely, so 3 and 4 are never printed.

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
OUTPUT
0
1
2
4
```

At  $i = 3$  only that iteration is skipped; the loop carries on with 4.

**while True with break** — a loop that runs "forever" until `break` stops it:

```
n = 1
while True:
    if n * n > 50:
        break
    n += 1
print("First number whose square is more than 50:", n)
```

```
OUTPUT
First number whose square is more than 50: 8
```

### REMEMBER IT

**break** = "exit the room"; **continue** = "skip this turn". For every `while` loop, check the **I-T-U** rule: Initialise, Test, Update.

### COMMON MISTAKE

**Infinite loop — find the error:**

```
count = 1
while count ≤ 5:
    print(count)
```

`count` is never updated, so `count ≤ 5` stays True and the program prints 1 forever. Fix: add `count += 1` inside the loop. (To stop a running program in the terminal, press **Ctrl + C**.)

### اردو میں سمجھیں

`while` لوپ اس وقت تک چلتا رہتا ہے جب تک شرط درست (True) رہے، اور یہ تب استعمال ہوتا ہے جب پہلے سے معلوم نہ ہو کہ کتنی بار دہرانا ہے۔ لوپ کے اندر ویری ایبل کو اپڈیٹ کرنا لازمی ہے ورنہ لوپ کبھی ختم نہیں ہوگا (Infinite Loop)۔ `break` لوپ کو فوراً مکمل طور پر روک دیتا ہے، جبکہ `continue` صرف موجودہ چکر چھوڑ کر اگلے چکر پر چلا جاتا ہے۔

### EXAM TIP

"Differentiate `break` and `continue` with examples" and "What is an infinite loop?" are frequent. Use the two side-by-side programs above with their outputs — examiners like seeing the difference in the output (0 1 2 vs 0 1 2 4).

### CHECK YOURSELF

1. Predict the output:

```
n = 1
while n < 20:
    n = n * 3
print(n)
```

2. Predict the output:

```
for i in range(1, 8):
    if i % 3 == 0:
        continue
    if i == 7:
        break
    print(i, end=" ")
```

Answers: 1. 27 (n: 1 → 3 → 9 → 27; 27 < 20 is False, so the loop stops). 2. 1 2 4 5 (3 and 6 are skipped by `continue`; at 7 the loop breaks).

## 3.5.3 Nested Loops and for vs while ★ Golden

### LEARN IT

A **nested loop** is a loop inside another loop. The **inner loop runs completely, from start to finish, for every single iteration of the outer loop**. If the outer loop runs 2 times and the inner loop runs 3 times, the inner body runs  $2 \times 3 = 6$  times.

```
for outer in range(1, 3):          # runs 2 times (1, 2)
    for inner in range(1, 4):      # runs 3 times per outer (1, 2, 3)
        print(f"Outer: {outer}, Inner: {inner}")
```

### OUTPUT

```
Outer: 1, Inner: 1
Outer: 1, Inner: 2
Outer: 1, Inner: 3
Outer: 2, Inner: 1
Outer: 2, Inner: 2
Outer: 2, Inner: 3
```

### WORKED EXAMPLE

**A star triangle.** The outer loop chooses the row; the inner loop prints that many stars on the same line (`end=""`), and the empty `print()` moves to the next line.

```
for row in range(1, 5):
    for col in range(row):
        print("*", end="")
    print()
```

### OUTPUT

```
*
**
***
****
```

Row 1: inner loop runs once → \*. Row 2: twice → \*\*. And so on up to row 4.

## for loop vs while loop

| for loop                                             | while loop                                                                    |
|------------------------------------------------------|-------------------------------------------------------------------------------|
| Used when the number of iterations is <b>known</b> . | Used when the number of iterations is <b>unknown</b> (condition-based).       |
| Automatically updates the loop variable.             | Requires a manual update of the counter (risk of an infinite loop).           |
| Cleaner and shorter for counting tasks.              | Better for dynamic situations (like "keep playing until the user says Stop"). |
| <code>for i in range(1, 4): print(i)</code>          | <code>i = 1 / while i ≤ 3: / print(i) / i += 1</code>                         |

### REMEMBER IT

Ask yourself: "Do I know **how many times**?" Yes → `for`. "Do I only know **when to stop**?" → `while`. Nested loops are like a clock: the minute hand (inner loop) goes all the way round for each step of the hour hand (outer loop).

### COMMON MISTAKE

Indentation decides which loop a line belongs to. In the triangle program, if `print()` were indented under the inner loop, every star would go on its own line.

### اردو میں سمجھیں

نیسٹڈ لوپ (Nested Loop) میں ایک لوپ دوسرے لوپ کے اندر ہوتا ہے، اور باہر والے لوپ کے ہر چکر میں اندر والا لوپ شروع سے آخر تک مکمل چلتا ہے۔ جب دہرائے کی تعداد معلوم ہو تو `for` لوپ، اور جب صرف رکنے کی شرط معلوم ہو تو `while` لوپ بہتر ہے۔

### EXAM TIP

"Differentiate between for and while loops" (give three points and an example of each) and "How many times will the inner statement run?" (multiply the counts) are typical. For pattern programs, explain what the outer and inner loops each control.

### CHECK YOURSELF

1. Predict the output:

```
count = 0
for i in range(3):
    for j in range(2):
        count += 1
    print(count)
```

2. Which loop would you use to keep reading numbers until the user enters 0?

Answers: 1. 6 (3 × 2). 2. A `while` loop, because the number of inputs is not known in advance.

## Practical Activity: Shopping Bill Calculator

### LEARN IT

This program combines all three control structures: **sequence** (steps in order), **selection** (checking "done", checking the price, checking for a discount) and **repetition** (a `while` loop that keeps adding items). It uses a **flag** — a Boolean variable ( `shopping` ) that controls when the loop stops. `item.lower()` converts the text to lowercase so that "done", "Done" and "DONE" all work.

```

print("==== SHOPPING BILL ====")
total = 0.0
count = 0
shopping = True

# REPETITION: keep adding items
while shopping:
    item = input("Enter Item Name (or 'done' to stop): ")
    # SELECTION: check if user wants to stop
    if item.lower() == "done":
        shopping = False
    else:
        price = float(input(f"Enter Price of {item}: "))
        # SELECTION: validate price
        if price <= 0:
            print("Invalid price!")
        else:
            total += price
            count += 1

# Summary and discount
if total >= 5000:
    discount = total * 0.20 # 20% off
else:
    discount = 0

final = total - discount
print(f"Total Items: {count}")
print(f"Total Bill: {total}")
print(f"Final to Pay: {final}")

```

#### WORKED EXAMPLE

**Sample run.** The user buys Rice for 3000, types a wrong price -50 for Soap, buys Oil for 2500, then types DONE :

##### OUTPUT

```

==== SHOPPING BILL ====
Enter Item Name (or 'done' to stop): Rice
Enter Price of Rice: 3000
Enter Item Name (or 'done' to stop): Soap
Enter Price of Soap: -50
Invalid price!
Enter Item Name (or 'done' to stop): Oil
Enter Price of Oil: 2500
Enter Item Name (or 'done' to stop): DONE
Total Items: 2
Total Bill: 5500.0
Final to Pay: 4400.0

```

| Step | item                    | price                      | total  | count |
|------|-------------------------|----------------------------|--------|-------|
| 1    | Rice                    | 3000.0                     | 3000.0 | 1     |
| 2    | Soap                    | -50.0 (invalid, not added) | 3000.0 | 1     |
| 3    | Oil                     | 2500.0                     | 5500.0 | 2     |
| 4    | DONE → shopping = False | —                          | 5500.0 | 2     |

After the loop:  $5500.0 \geq 5000$ , so  $\text{discount} = 5500.0 \times 0.20 = 1100.0$ , and  $\text{final} = 5500.0 - 1100.0 = \mathbf{4400.0}$ .



#### REMEMBER IT

Label the parts of any program with comments, as here: `# REPETITION`, `# SELECTION`. It helps you, your teacher and the examiner see the structure.

#### COMMON MISTAKE

`total` and `count` must be set to 0 **before** the loop. If you put `total = 0.0` inside the loop, it resets on every item and the bill is wrong.

#### اردو میں سمجھیں

یہ شاپنگ بل پروگرام تینوں کنٹرول اسٹرکچرز کو ملا کر بنایا گیا ہے: `while` لوپ سے اشیاء بار بار شامل ہوتی ہیں، `if-else` سے غلط قیمت روکی جاتی ہے اور 5000 یا زیادہ کے بل پر 20 فیصد رعایت دی جاتی ہے۔

#### CHECK YOURSELF

1. What will "Final to Pay" be if the user buys one item for 4000 and then types `done` ?
2. What is the purpose of the variable `shopping` ?

Answers: 1. Final to Pay: 4000.0 (4000 is less than 5000, so no discount). 2. It is a flag that keeps the `while` loop running; setting it to `False` stops the loop.

## 3.6 Libraries in Python

#### LEARN IT

A **library** (also called a **module**) is a collection of pre-written code that offers ready-made functions. Instead of writing a square-root algorithm from scratch, you import `math` and call `math.sqrt()`. Libraries save time, reduce bugs and let you stand on other programmers' work.

There are five common ways to import a library. Method 3 (import only what you need) is the best habit: it is clearer and uses less memory than `from math import *`.

| Method               | Syntax                                 | How you call the function              |
|----------------------|----------------------------------------|----------------------------------------|
| 1. Entire module     | <code>import math</code>               | <code>math.sqrt(25)</code>             |
| 2. Alias (nickname)  | <code>import math as m</code>          | <code>m.sqrt(16)</code>                |
| 3. Specific function | <code>from math import sqrt</code>     | <code>sqrt(36)</code>                  |
| 4. Several functions | <code>from math import sqrt, pi</code> | <code>sqrt(9)</code> , <code>pi</code> |
| 5. Everything (*)    | <code>from math import *</code>        | <code>sqrt(64)</code> — avoid this     |

#### WORKED EXAMPLE

Five imports, one result each:

```
import math
print(math.sqrt(25))
import math as m
print(m.sqrt(16))
from math import sqrt
print(sqrt(36))
from math import sqrt, pi
print(pi)
from math import *
print(sqrt(64))
```

#### OUTPUT

```
5.0
4.0
6.0
3.141592653589793
8.0
```

#### COMMON MISTAKE

`from math import *` dumps every name into your program. If you later write a function called `sqrt`, you silently overwrite the library one. Prefer methods 1–4.

#### اردو میں سمجھیں

لائبریری پہلے سے لکھا ہوا کوڈ ہے جسے `import` سے پروگرام میں لایا جاتا ہے۔ سب سے اچھا طریقہ یہ ہے کہ صرف وہ فنکشن درآمد کریں جس کی ضرورت ہو، ستارہ `*` سے سب کچھ نہ کھینچیں۔

#### CHECK YOURSELF

1. Write the line that lets you call `ceil` without writing `math.` in front.
2. Why is an alias such as `import pandas as pd` useful?

Answers: 1. `from math import ceil`. 2. The library name is long; a short nickname keeps later lines short and is a community convention.

### 3.6.1 Built-in Libraries

#### LEARN IT

**Built-in libraries** come pre-installed with Python. You only import them; you do not run `pip`. Three that every Class XI student must know are `math`, `random` and `datetime`.

#### math

```
import math
print(math.pi)
print(math.ceil(4.3))
print(math.floor(4.9))
print(math.pow(2, 4))
print(math.sqrt(81))
```

#### OUTPUT

```
3.141592653589793
5
4
16.0
9.0
```

#### random and datetime

```
import random
print(random.randint(1, 1))
print(0.0 ≤ random.random() < 1.0)

import datetime
now = datetime.datetime(2026, 9, 27, 11, 59, 40)
print(now.strftime("%d-%m-%Y"))
```

#### OUTPUT

```
1
True
27-09-2026
```

`randint(a, b)` includes both ends. `random()` is a float in `[0.0, 1.0)`. `strftime` formats a date; `%d-%m-%Y` is day-month-year.

#### REMEMBER IT

`ceil` always rounds **up** (ceiling); `floor` always rounds **down**. `round()` (built-in, not in `math`) goes to the nearest integer and is a different function.

#### اردو میں سمجھیں

`math` حساب کے لیے، `random` بے ترتیب عدد کے لیے اور `datetime` تاریخ وقت کے لیے بنی ہوئی لائبریریاں ہیں۔ انہیں انسٹال نہیں کرنا پڑتا، صرف `import` کافی ہے۔

#### CHECK YOURSELF

1. What does `math.ceil(4.0)` return?
2. Write a line that prints a random integer from 1 to 6 (a dice).

Answers: 1. 4 (already an integer). 2. `print(random.randint(1, 6))` after `import random`.

## 3.6.2 Third-Party Libraries

#### LEARN IT

**Third-party libraries** are written by the community. They are **not** bundled with Python, so you install them from a terminal with `pip install library_name` and then import them as usual.

| Library              | Primary use                                  |
|----------------------|----------------------------------------------|
| NumPy                | Heavy numerical computing and large arrays   |
| Pandas               | Data analysis; table-like data (CSV / Excel) |
| Matplotlib           | Data visualisation — graphs and charts       |
| TensorFlow / PyTorch | Artificial intelligence and machine learning |

#### WORKED EXAMPLE

Installing and importing Pandas (you will use this in Class XII):

```
pip install pandas
import pandas as pd
```

#### COMMON MISTAKE

`pip` is a **terminal** command, not a Python statement. Do not type `pip install pandas` inside a `.py` file. In VS Code, open the Terminal panel.

اردو میں سمجھیں

تھرڈ پارٹی لائبریریاں پائتھن کے ساتھ نہیں آتیں، انہیں `pip install` سے انسٹال کیا جاتا ہے۔ نمپائی، پینڈاز، میٹپلاٹ لیب اور ٹینسر فلو اسی قسم کی ہیں۔

#### CHECK YOURSELF

1. Which library would you choose to draw a bar chart of class marks?
2. Is `math` a third-party library?

Answers: 1. Matplotlib. 2. No — `math` is built-in.

## 3.7 Debugging Code in Python

#### LEARN IT

**Debugging** is the process of identifying, tracing and fixing bugs so a program works correctly. Every program you write will have bugs. The skill is not "never make a mistake"; it is "find the mistake quickly".

Python errors fall into three families:

| Type                      | When it appears                 | What you notice             | Example                                                      |
|---------------------------|---------------------------------|-----------------------------|--------------------------------------------------------------|
| Syntax error              | Before the program even starts  | Python refuses to run       | <code>print("Hello" — missing )</code>                       |
| Runtime error (exception) | While the program is running    | It crashes with a traceback | <code>x = 10 / 0 → ZeroDivisionError</code>                  |
| Logical error             | Program finishes "successfully" | The output is wrong         | <code>area = length + width</code> instead of <code>*</code> |

#### WORKED EXAMPLE

**Reading a traceback.** When Python crashes it prints a roadmap: the file, the line number, and the error type.

```
x = int("abc")
```

#### OUTPUT

```
Traceback (most recent call last):
  File "debug_program.py", line 1, in <module>
    x = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

Diagnosis: on **line 1** we tried to convert the letters "abc" into an integer, which raised a `ValueError`.

#### EXAM TIP

Examiners love "identify the error type" questions. Ask: (1) Does it refuse to start? → syntax. (2) Does it crash mid-run? → runtime. (3) Does it run but give a wrong answer? → logical. Write the name of the error **and** how to fix it.

#### REMEMBER IT

Fix syntax errors first (the program cannot start). Then run it. Then check the output against a hand-worked example to catch logical errors.

#### COMMON MISTAKE

A logical error is the most dangerous because Python reports nothing. Always test with values whose answer you already know (for a rectangle 3 by 4, area must be 12, not 7).

#### اردو میں سمجھیں

ڈیبگنگ یعنی غلطی ڈھونڈ کر ٹھیک کرنا۔ نحوی غلطی پر پروگرام چلتا ہی نہیں، رن ٹائم پر چلتے چلتے رک جاتا ہے، منطقی غلطی پر چل تو جاتا ہے مگر جواب غلط دیتا ہے — یہ سب سے خطرناک ہے۔

#### CHECK YOURSELF

- `print("Hi" — which error type?`
- A program that should add two numbers multiplies them instead. Which error type?
- `int("12.5")` raises which named exception?

Answers: 1. Syntax error. 2. Logical error. 3. `ValueError` (runtime).

## Chapter 3 Review

---

### Chapter summary

- A program is a set of instructions; programming languages are low-level (close to hardware) or high-level (close to English). Python is a high-level, interpreted, readable language.
- Python is widely used in web development, data science, AI and automation — skills that grow a career in Pakistan and abroad.
- An IDE (such as VS Code plus the Python extension) combines an editor, a terminal, a debugger and syntax highlighting.
- Variables store values; Python types are dynamic. Core types: `int`, `float`, `str`, `bool`, `list`, `dict`.
- `print()` displays output; `input()` always returns a **string** — convert with `int()` or `float()` before arithmetic.
- Operators: arithmetic (`+` `-` `*` `/` `//` `%` `**`), assignment (`=` `+=` `...`), relational, logical (`and` `or` `not`), membership (`in`), bitwise (`&` `|` `^` `~` `<<` `>>`).
- Control structures: sequence (top to bottom), selection (`if` / `if-else` / `if-elif-else` / nested), repetition (`for` / `while`).
- `range(stop)` goes 0 ... stop-1; `range(start, stop, step)` jumps by step. `break` leaves a loop; `continue` skips one iteration.
- Use `for` when the count is known; use `while` when you wait for a condition (and always update that condition).
- Libraries extend Python. Built-in: `math`, `random`, `datetime`. Third-party (need `pip`): NumPy, Pandas, Matplotlib, TensorFlow.
- Errors: syntax (grammar, will not run), runtime (crash, traceback), logical (runs, wrong answer).

## Key terms

| Term                | Meaning                                                         | اردو                          |
|---------------------|-----------------------------------------------------------------|-------------------------------|
| Program             | A set of instructions a computer can follow.                    | پروگرام                       |
| High-level language | A language close to English (Python, Java, C++).                | اعلیٰ سطحی زبان               |
| Interpreter         | Runs a program line by line without a separate compile step.    | انٹریپرٹر                     |
| IDE                 | Software that combines editor, terminal and debugger.           | انٹیگریٹڈ ڈویلپمنٹ انوائرنمنٹ |
| Variable            | A named box that holds a value.                                 | متغیر                         |
| Data type           | The kind of value: int, float, str, bool, list, dict.           | ڈیٹا کی قسم                   |
| Operator            | A symbol that performs an operation on operands.                | عامل                          |
| Operand             | A value an operator works on.                                   | معمول                         |
| Sequence            | Instructions executed from top to bottom.                       | تسلسل                         |
| Selection           | Choosing a path with <code>if / else</code> .                   | انتخاب                        |
| Repetition / loop   | Repeating a block with <code>for</code> or <code>while</code> . | تکرار / لوپ                   |
| Nested              | A structure placed inside another of the same kind.             | تو در تو                      |
| Library / module    | A collection of ready-made functions.                           | لائبریری / ماڈیول             |
| pip                 | The tool that installs third-party Python libraries.            | پپ                            |
| Syntax error        | A grammar mistake; the program will not start.                  | نحوی غلطی                     |
| Runtime error       | A crash while the program is running (exception).               | رن ٹائم غلطی                  |
| Logical error       | The program runs but the result is wrong.                       | منطقی غلطی                    |
| Traceback           | Python's report of where a crash happened.                      | ٹریس بیک                      |
| Flag                | A Boolean variable that controls a loop or decision.            | فلیگ                          |
| Integer division    | <code>//</code> division that drops the fraction.               | صحیح تقسیم                    |

## Practice: multiple choice

1. Python is best described as a:
  - a. low-level compiled language
  - b. high-level interpreted language
  - c. machine language
  - d. hardware description language
2. `input("Age: ")` always returns a:
  - a. int
  - b. float
  - c. str
  - d. bool
3. The value of `17 // 5` is:
  - a. 3.4
  - b. 2
  - c. 3
  - d. 3.0
4. The value of `17 % 5` is:
  - a. 2
  - b. 3
  - c. 3.4
  - d. 0
5. `2 ** 3` equals:
  - a. 6
  - b. 8
  - c. 9
  - d. 5
6. Which operator is bitwise AND?
  - a. `and`
  - b. `&&`
  - c. `&`
  - d. `^`
7. `list(range(2, 5))` is:
  - a. `[2, 3, 4, 5]`
  - b. `[2, 3, 4]`
  - c. `[2, 5]`
  - d. `[0, 1, 2, 3, 4]`
8. A `while` loop is preferred when:
  - a. you know the exact number of repeats
  - b. you must wait for a condition
  - c. you never need a counter
  - d. the body must run exactly once
9. `break` inside a loop:
  - a. skips one iteration
  - b. restarts the loop
  - c. leaves the loop immediately
  - d. pauses for input
10. `from math import sqrt` lets you write:
  - a. `math.sqrt(9)` only
  - b. `sqrt(9)`
  - c. `import.sqrt(9)`
  - d. `m.sqrt(9)`
11. Which library is built-in?
  - a. Pandas
  - b. NumPy
  - c. random
  - d. Matplotlib
12. `print("Hi")` is a:
  - a. syntax error
  - b. runtime error
  - c. logical error
  - d. warning only
13. `10 / 0` is a:
  - a. syntax error
  - b. runtime error
  - c. logical error
  - d. style error
14. `area = length + width` when you meant to multiply is a:
  - a. syntax error
  - b. runtime error
  - c. logical error
  - d. import error
15. `math.ceil(4.3)` is:
  - a. 4
  - b. 4.0
  - c. 5
  - d. 4.3

## Practice: short questions

1. Differentiate between a compiler and an interpreter.
2. Why must the result of `input()` often be converted with `int()`?
3. State the difference between `/` and `//` with one example each.
4. Write the truth table of the logical operator `and` for two Boolean values.
5. Explain `if-elif-else`. Why is the order of conditions important?
6. Differentiate between a `for` loop and a `while` loop.
7. What do `break` and `continue` do?
8. Name two built-in and two third-party Python libraries and their uses.
9. Differentiate between syntax, runtime and logical errors with one example each.
10. What three pieces of information does a Python traceback give you?

## Practice: long / problem questions

1. Write a complete Python program that reads a student's marks (0–100) and prints the grade: 80+ A-1, 70–79 A, 60–69 B, otherwise Fail. Include input conversion and a sample run.
2. Write a program that prints all even numbers from 0 to 10 using `range`, then rewrite it with a `while` loop. Dry-run both.
3. Trace this nested loop and write every line of output:

```
for o in range(1, 3):  
    for i in range(1, 4):  
        print(o, i)
```
4. A shop gives 20% discount if the bill is 5000 or more. Write a program that keeps reading item prices until the user types 0, prints the total, the discount and the amount to pay. Show a sample run with prices 3000, 2500, 0.
5. Classify each bug and write the corrected line:
  - (a) `print("Total is", total`
  - (b) `avg = total / 0` when there are no items
  - (c) `discount = total + 0.20` when 20% off was intended.
6. Explain how you would use `math`, `random` and `datetime` in a small "lucky student of the day" program for a class of 40. Write the code.

## Answer key

---

### Multiple choice

1. b
2. c
3. c
4. a
5. b
6. c
7. b
8. b
9. c
10. b
11. c
12. a
13. b
14. c
15. c



## Short questions — model answers

1. A compiler translates the whole program into machine code before it runs (C, C++). An interpreter translates and runs line by line (Python). Compiled programs often run faster; interpreted ones are easier to test.
2. `input()` always returns a string. Adding strings concatenates them ( `"2"+"3"` is `"23"` ), so numeric work needs `int()` or `float()`.
3. `/` is true division: `17/5` is `3.4`. `//` is integer (floor) division: `17//5` is `3`.
4.  $F \text{ and } F \rightarrow F$ ;  $F \text{ and } T \rightarrow F$ ;  $T \text{ and } F \rightarrow F$ ;  $T \text{ and } T \rightarrow T$ . Output is True only when both operands are True.
5. Python tests conditions from the top. The first True block runs and the rest are skipped. If you put `marks >= 60` above `marks >= 80`, an 85 would match the weaker condition first and get the wrong grade.
6. `for`: known count, counter updates automatically, good with `range`. `while`: unknown count, you must update the condition yourself or you get an infinite loop.
7. `break` leaves the loop immediately. `continue` skips the rest of this iteration and starts the next one.
8. Built-in: `math` (roots, ceil/floor), `random` (random integers). Third-party: Pandas (tables), Matplotlib (charts). (NumPy and TensorFlow also accepted.)
9. Syntax: missing bracket, will not run. Runtime: `10/0`, crashes. Logical: `area = l + w`, runs but answer is wrong.
10. The error type (e.g. `ValueError`), the file name, and the line number that caused the crash.

## Long questions — model answers

```
1. marks = int(input("Enter marks: "))
if marks ≥ 80:
    print("A-1 Grade")
elif marks ≥ 70:
    print("A Grade")
elif marks ≥ 60:
    print("B Grade")
else:
    print("Fail or below B")
```

Sample: input 85 → A-1 Grade; input 72 → A Grade; input 40 → Fail or below B.

```
2. for i in range(0, 11, 2):
    print(i)
n = 0
while n ≤ 10:
    print(n)
    n += 2
```

Both print 0, 2, 4, 6, 8, 10.

3. Output (six lines): 1 1 / 1 2 / 1 3 / 2 1 / 2 2 / 2 3. The inner loop runs fully for each value of the outer loop.

```
4. total = 0.0
while True:
    p = float(input("Price (0 to stop): "))
    if p == 0:
        break
    if p > 0:
        total += p
    discount = total * 0.20 if total ≥ 5000 else 0
    print("Total", total)
    print("Discount", discount)
    print("Pay", total - discount)
```

Sample: 3000 + 2500 = 5500; discount 1100; pay 4400.

5. (a) Syntax — add `)`: `print("Total is", total)`. (b) Runtime `ZeroDivisionError` — only divide when the count of items is not 0. (c) Logical — 20% of total is `total * 0.20`, not `total + 0.20`.

```
6. import random, datetime, math
n = random.randint(1, 40)
today = datetime.datetime.now().strftime("%d-%m-%Y")
print("Lucky roll", n, "on", today)
print("Root of roll", math.sqrt(n))
```

`random` picks the student number, `datetime` stamps the date, `math` shows a sample extra calculation.

## Self-assessment: I can...

- |                                                                                                            |                                                                                                      |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> classify a language as high-level or low-level and explain why Python is popular. | <input type="checkbox"/> declare variables and identify int, float, str, bool, list and dict values. |
| <input type="checkbox"/> read input, convert types, and print formatted output.                            | <input type="checkbox"/> evaluate arithmetic, relational, logical and bitwise expressions by hand.   |
| <input type="checkbox"/> write if, if-else, if-elif-else and nested selections.                            | <input type="checkbox"/> write for and while loops, including range, break and continue.             |
| <input type="checkbox"/> import built-in libraries (math, random, datetime) in more than one style.        | <input type="checkbox"/> name pip and at least two third-party libraries.                            |
| <input type="checkbox"/> classify a bug as syntax, runtime or logical and read a traceback.                | <input type="checkbox"/> combine sequence, selection and repetition in a small complete program.     |

## CHAPTER 4

# Data and Analysis

This chapter teaches you how raw facts become organised information in a database, how tables are linked with keys, how to draw an ER diagram, and how to build tables, forms, queries and charts in Microsoft Access. Banks, NADRA, college libraries and fee offices all run on these ideas.

اس باب میں آپ سیکھیں گے کہ خام حقائق کو ڈیٹا بیس میں کیسے منظم کیا جاتا ہے، ٹیبلز کو کیسے جوڑا جاتا ہے، ای آر خاکہ کیسے بنتا ہے، اور مائیکروسافٹ ایکسیس میں ٹیبل، فارم، کوئری اور چارٹ کیسے بنائے جاتے ہیں۔

### What you will learn

- Distinguish data, information and a database, and explain what a DBMS does.
- Choose candidate, primary, alternate and foreign keys and state integrity rules.
- Explain referential integrity, Cascade Update and Cascade Delete with before/after tables.
- Develop the college-library case study from requirements to the final ER schema.
- Create tables in Access (Datasheet and Design view) and pick data types.
- Identify tables, records and fields in a given example.
- Draw ER diagrams (entities, attributes, relationships) and give cardinality 1:1, 1:M, M:N.
- Write a relational schema and resolve an M:N relationship with a junction table.
- State the job of tables, forms, queries and reports.
- Build a form with the Form Wizard and a SELECT query with criteria, AND/OR, totals and charts.

### 4.1.1 Data, Information and Database

#### LEARN IT

**Data** are unprocessed facts, figures or observations. They can be **quantitative** (numbers: 80, 75, 90) or **qualitative** (descriptions: the colours of cars in a parking lot). Data on its own is not very useful — a list of marks does not yet tell you whether the class is strong.

When you organise, process and interpret data, it becomes **information**. Information has meaning and supports a decision. Example: "The class average is 80%, so we can move to the next chapter."

A **database** is an organised collection of **related** data, stored in a structured way so it can be accessed, managed and updated easily. Paper registers and loose files make searching painful; a database is built for search, update and sharing.

Data → Processing → Information → Decision

| Aspect     | Data                  | Information                |
|------------|-----------------------|----------------------------|
| Definition | Raw facts and figures | Processed, meaningful data |
| Example    | 85, 90, 75            | Average marks = 83.3%      |
| Usefulness | Little, on its own    | Useful for decisions       |

#### WORKED EXAMPLE

Five CS XI marks: 80, 75, 90, 70, 85. Sum = 400. Average =  $400 \div 5 = 80\%$ . The numbers are data; "the class average is 80%" is information; a **Student** table holding every student's marks is a database.

#### REMEMBER IT

Exam line: *Data is raw; information is processed data used for decision-making.* Always add one pair of examples.

#### COMMON MISTAKE

Do not call a jumble of WhatsApp numbers a database. Related + structured + easy to update is what makes it a database.

#### اردو میں سمجھیں

ڈیٹا خام حقائق ہیں، معلومات وہی حقائق ہیں جب انہیں ترتیب دے کر معنی دے دیا جائے۔ ڈیٹا بیس متعلقہ ڈیٹا کا منظم ذخیرہ ہے جسے آسانی سے تلاش اور تبدیل کیا جا سکتا ہے۔

#### CHECK YOURSELF

1. Is "Nadia, Class 11, 72 marks" data or information?
2. Why is a stack of unmarked answer sheets not a database?

Answers: 1. Data (raw facts about one student). 2. It is not organised in a structured, searchable, updatable collection of related records.

## 4.1.2 Database Management System (DBMS)

#### LEARN IT

A **DBMS** is software that lets you create, manage and interact with databases. It sits between the user and the stored files. You never edit raw data files by hand — you talk to the DBMS (Access, MySQL, Oracle, SQLite) and it handles storage, security and consistency.

Advantages you must be able to list:

- **Less redundancy.** The department name is stored once, not in every student row.
- **Consistency.** Change a phone number once; every screen that uses it shows the new number.
- **Security.** Only authorised users see sensitive fields (an admin sees marks; office staff see names).
- **Integrity.** Rules stop duplicate roll numbers and empty primary keys.

#### WORKED EXAMPLE

In a paper register, Ali's address is written on the admission page, the fee page and the sports page. If he moves house, two of the three pages stay wrong. In a DBMS, Address lives in one **Student** table. Every form and report reads that single field.

#### اردو میں سمجھیں

ڈی بی ایم ایس وہ سافٹ ویئر ہے جو صارف اور ڈیٹا بیس کے درمیان پل کا کام کرتا ہے۔ اس سے تکرار کم ہوتی ہے، ڈیٹا ایک جیسا رہتا ہے، صرف مجاز افراد دیکھ سکتے ہیں، اور غلط اندراج رک جاتی ہے۔

#### CHECK YOURSELF

1. Name two DBMS programs.
2. Which advantage stops two students sharing the same roll number?

Answers: 1. MS Access, MySQL (Oracle, SQLite also). 2. Integrity (unique / primary-key constraint).

## 4.2 Database Components (Tables, Records, Fields)

### LEARN IT

The basic structure is a **table** (also called a relation): rows and columns, like a spreadsheet, holding facts about one kind of thing (Students, Teachers, Books).

A **record** (row, tuple) is one complete instance — everything about Ali. A **field** (column, attribute) is one kind of fact — the whole Name column, or the whole Grade column.

| Roll No | Name  | Course | Grade |
|---------|-------|--------|-------|
| 101     | Ali   | CS     | A     |
| 102     | Sara  | IT     | B+    |
| 103     | Ahmed | CS     | A     |

The row 101 | Ali | CS | A is one **record**. The Name column is one **field**.

### REMEMBER IT

Memory: **row = record = one person. column = field = one question on the admission form.**

### COMMON MISTAKE

Do not mix two entities in one table (students and library books together). Each entity gets its own table; keys will link them later.

اردو میں سمجھیں

ٹیبل قطاروں اور کالموں میں ڈیٹا رکھتا ہے۔ ایک قطار ایک ریکارڈ (مثلاً علی کی پوری معلومات) ہے، ایک کالم ایک فیلڈ (مثلاً صرف نام) ہے۔

### CHECK YOURSELF

1. How many records and how many fields are in the table above?
2. Is "CS" a record or a field value?

Answers: 1. 3 records, 4 fields. 2. A value stored in the Course field.

## 4.3.1 Keys and Integrity Constraints ★ Golden

### LEARN IT

A **key** is a field (or a set of fields) that uniquely identifies records and links tables. Integrity constraints are the rules that keep those keys honest.

**Candidate key.** Any field (or combination) that *could* uniquely identify a row. A table may have several. In an Employee table: EmpID, EmpLicense# and EmpPassport are all candidate keys.

**Primary key (PK).** The one candidate you actually choose. Rules: unique, never NULL, only one PK per table (it may be a **composite** of two columns). Example: EmpID.

**Alternate key.** The leftover candidate keys. After EmpID is chosen, EmpLicense# and EmpPassport become alternate keys — still unique, just not "the" identifier.

**Foreign key (FK).** A field in this table that refers to the PK of another table. It *may* contain duplicates (many employees in one department) and, in some designs, NULL.

#### WORKED EXAMPLE

**Classroom analogy.** Roll Number, B-Form number and phone number can all identify a student (candidate keys). The school picks Roll Number as the primary key. B-Form and phone become alternate keys. In a **Result** table, RollNo is a foreign key pointing back to Student.

| Table   | Field           | Role                             |
|---------|-----------------|----------------------------------|
| Student | RollNo          | Primary key                      |
| Student | BFormNo         | Alternate key                    |
| Result  | RollNo          | Foreign key → Student.RollNo     |
| Result  | ExamID + RollNo | Could be a composite primary key |

#### EXAM TIP

Board questions: “Differentiate primary and foreign key” (table of 4 points) and “Why can a PK not be NULL?” Because a NULL would mean “this row has no identity”, so the DBMS could not tell rows apart.

#### COMMON MISTAKE

A phone number is a bad PK: people change numbers, two siblings may share one, and some students have none (NULL). Roll numbers are issued by the school and stay unique.

#### اردو میں سمجھیں

کینڈیڈٹ کی وہ فیلڈ ہے جو ریکارڈ کی شناخت کر سکے۔ پرائمری کی منتخب شدہ شناخت ہے — منفرد اور خالی نہیں۔ جو بچ جائیں وہ آلٹرنیٹ کیز ہیں۔ فارن کی دوسرے ٹیبل کی پرائمری کی طرف اشارہ کرتی ہے اور ٹیبلز کو جوڑتی ہے۔

#### CHECK YOURSELF

1. Can a table have two primary keys?
2. Can a foreign key have duplicate values?

Answers: 1. No — only one PK (it may be composite). 2. Yes — many child rows may point to the same parent.

## 4.3 Relational Database Management System (RDBMS)

#### LEARN IT

An **RDBMS** stores data in several tables and uses primary and foreign keys to keep relationships. Banks, universities and NADRA use RDBMS software. Extra strengths on top of a plain DBMS:

- Reduces redundancy by splitting data into related tables.
- Supports many users at once, with permissions.
- Is queried with **SQL** (Structured Query Language) — a standard language of SELECT, INSERT, UPDATE, DELETE.

#### WORKED EXAMPLE

Instead of one giant spreadsheet of students-and-departments, an RDBMS has Department(DeptID, DeptName) and Student(RollNo, Name, DeptID FK). CS is stored once. SQL: SELECT Name FROM Student WHERE DeptID = 1;

آر ڈی بی ایم ایس کئی ٹیبلز میں ڈیٹا رکھتا ہے اور کیڑ سے ان کے تعلقات برقرار رکھتا ہے۔ ایس کیو ایل سے سوال پوچھے جاتے ہیں۔ اس سے تکرار کم اور کئی صارفین محفوظ طریقے سے کام کر سکتے ہیں۔

**CHECK YOURSELF**

1. What language do you use to query an RDBMS?
2. Why split Department out of the Student table?

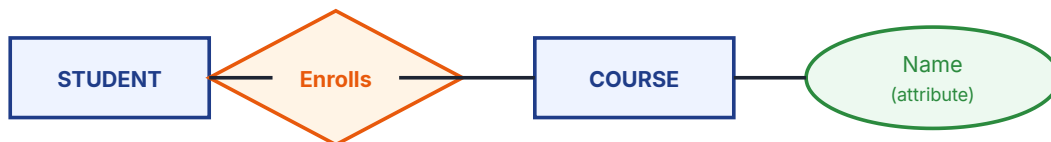
Answers: 1. SQL. 2. To store the department name once (less redundancy) and update it in one place.

## 4.4 Entity Relationship Model (ER-Model) ★ Golden

**LEARN IT**

The **ER model** is the architect's blueprint: you draw what will be stored and how it connects *before* you create tables in Access. Three building blocks:

| Component    | Meaning                                   | Symbol    | Example                  |
|--------------|-------------------------------------------|-----------|--------------------------|
| Entity       | A real-world object we store data about   | Rectangle | STUDENT, TEACHER, COURSE |
| Attribute    | A property of an entity. PK is underlined | Oval      | StudentID, Name, Class   |
| Relationship | How two entities are connected            | Diamond   | Enrolls, Teaches, Has    |



Rectangle = entity, diamond = relationship, oval = attribute. Underline the primary-key attribute.

### Cardinality (types of relationship)

| Type               | Meaning                                                    | Example                              |
|--------------------|------------------------------------------------------------|--------------------------------------|
| One-to-One (1:1)   | One record ↔ exactly one record. Rare; often for security. | One student has one ID card.         |
| One-to-Many (1:M)  | One record in A ↔ many in B. The most common type.         | One teacher teaches many students.   |
| Many-to-Many (M:N) | Many in A ↔ many in B.                                     | Many students enrol in many classes. |

**COMMON MISTAKE**

**The M:N rule.** A relational database cannot store M:N directly — it duplicates data. Break it into two 1:M relationships with a **junction table** (also called a bridge or associative table). Example: Enrollment(StudentID, ClassID, Date).

#### EXAM TIP

Always state cardinality on the diagram (1 and M at the ends of the line). For M:N, the long question is “how do you resolve it?” — draw the junction entity with two foreign keys.

#### اردو میں سمجھیں

ای آر ماڈل ڈیٹا بیس کا خاکہ ہے۔ مستطیل ہستی، بیضوی صفت، لوزی تعلق۔ تعلقات 1:1، M:1 یا M:N ہوتے ہیں۔ M:N کو دو M:1 میں توڑنے کے لیے جنکشن ٹیبل بنانا پڑتا ہے۔

#### CHECK YOURSELF

1. What symbol is used for a relationship?
2. A hospital: one patient, many appointments. Cardinality?

Answers: 1. Diamond. 2. 1:M (Patient 1 — has — M Appointment).

## 4.5 Referential Integrity ★ Golden

#### LEARN IT

**Referential integrity** means every foreign-key value in a child table must match an existing primary-key value in the parent — or be NULL if the design allows it. It stops **orphan records**.

If Result.StudentID is a FK to Student.StudentID, you cannot type Result for student 999 if 999 is not in Student. You also cannot delete student 101 while results for 101 still exist — unless you first handle those child rows.

Parent — STUDENT

| ID  | Name |
|-----|------|
| 101 | Ali  |
| 102 | Sara |

Child — RESULT

| ID  | Grade        |
|-----|--------------|
| 101 | A            |
| 999 | F — rejected |

### Cascade operations

**Cascade Update.** Change the PK in the parent, and every matching FK in the child changes automatically. Ali's ID 101 → 201 updates Enrollments too. Nothing is left pointing at a vanished 101.

**Cascade Delete.** Delete the parent, and related child rows are deleted too. If Sara (102) leaves school, her Enrolment and Result rows disappear. That prevents “ghost” marks for a student who is gone.

#### WORKED EXAMPLE

##### Before Cascade Update

| Student.ID | Enrol.StudentID |
|------------|-----------------|
| 101        | 101             |

##### After changing 101 → 201 with Cascade Update ON

| Student.ID | Enrol.StudentID |
|------------|-----------------|
| 201        | 201             |

If Cascade Update were OFF, Access would refuse the change, or (in a sloppy file) leave Enrol stuck on 101 — an orphan.



### COMMON MISTAKE

Cascade Delete is powerful and dangerous. Deleting one parent can wipe years of child history. In a real school, you more often **archive** a student (a Status field) than delete them.

### EXAM TIP

Write: definition, why it matters (no orphans), then one numbered example of Cascade Update and one of Cascade Delete. Mention parent vs child.

### اردو میں سمجھیں

ریفرنشل انٹیگریٹی کا مطلب ہے بچہ ٹیبل کی فارن کی ہمیشہ والدین کی موجود پرائمری کی طرف اشارہ کرے۔ کیسکیڈ اپ ڈیٹ والدین کی کی بدلنے پر بچوں کو خود بدل دیتی ہے؛ کیسکیڈ ڈیلیٹ والدین مٹانے پر متعلقہ بچے ریکارڈ بھی مٹا دیتی ہے۔

### CHECK YOURSELF

1. What is an orphan record?
2. Cascade Delete: parent Teacher is removed. What happens to that teacher's Class rows?

Answers: 1. A child row whose FK value does not exist in the parent. 2. They are deleted automatically (if Cascade Delete is on).

## 4.6 Relational Schema Development

### LEARN IT

A **relational schema** is the formal written design of the database: table names, fields, primary keys, foreign keys and how tables relate. You write it *before* clicking around in Access.

It defines (1) how data is split into tables, (2) which columns each table has, (3) which fields are PK/FK, (4) the relationships. Benefits: less redundancy, integrity planned in advance, easier to grow later.

### WORKED EXAMPLE

Written schema for a tiny college:

```
Department (DeptID PK, DeptName)
Student    (RollNo PK, Name, Class, DeptID FK → Department)
Course     (CourseID PK, Title, Credit, DeptID FK → Department)
```

### REMEMBER IT

Skipping the schema is like building a house without a drawing. You get messy, broken structures that are painful to repair.

### اردو میں سمجھیں

ریلیشنل اسکیم ڈیٹا بیس کا لکھا ہوا نقشہ ہے: کون سے ٹیبل، کون سے فیلڈ، کون سی کیز، کون سے تعلقات۔ سافٹ ویئر کھولنے سے پہلے یہ بنانا چاہیے۔

### CHECK YOURSELF

1. Name two things a schema must show.
2. Give one reason to write a schema before opening Access.

Answers: 1. Tables/fields and keys/relationships. 2. It reduces redundancy and mistakes (any valid benefit).

## 4.7 Case Study — Library Management System

### LEARN IT

A college library needs a database. Walk the seven steps examiners expect.

**Step 1 — Requirements.** A book has ID, title, author, publisher. A member (student/teacher) has ID, name, contact. A member can borrow many books; a book can be borrowed by different members over time. Record Issue Date and Return Date.

**Step 2 — Entities.** Book, Member. (Borrowing will need a third entity in step 6.)

**Step 3 — Attributes.** Book: BookID (PK), Title, Author, Publisher. Member: MemberID (PK), Name, ContactInfo.

**Step 4 — Relationship.** Member *borrow*s Book.

**Step 5 — Cardinality.** One member borrows many books over time; one book is borrowed by many members over time → **M:N**.

**Step 6 — Resolve M:N.** Add junction entity **BorrowRecord**: BorrowID (PK), MemberID (FK), BookID (FK), IssueDate, ReturnDate. Now Member 1—M BorrowRecord M—1 Book (two 1:M links).

**Step 7 — Final schema.**

```
Member      (MemberID PK, Name, ContactInfo)
Book        (BookID PK, Title, Author, Publisher)
BorrowRecord (BorrowID PK, MemberID FK, BookID FK, IssueDate, ReturnDate)
```



Final library ER: two 1:M relationships through the BorrowRecord junction table.

### WORKED EXAMPLE

Sample rows after Sara borrows “Khushwant” on 5 Oct 2026:

| BorrowID | MemberID | BookID | IssueDate  | ReturnDate |
|----------|----------|--------|------------|------------|
| 5001     | 102      | B-17   | 2026-10-05 |            |

The same book can later be borrowed by Ali as BorrowID 5002 — no duplicated book title, no duplicated member address.

لائبریری میں ممبر اور کتاب کا تعلق بہت-بہت-سے ہے، اس لیے بورو ریکارڈ نامی پل ٹیبل بناتے ہیں جس میں دونوں کی فارن کیز اور اجراء/واپسی کی تاریخ ہوتی ہے۔

**CHECK YOURSELF**

1. Why is BorrowRecord needed?
2. Where do IssueDate and ReturnDate belong, and why not on Book?

Answers: 1. To break M:N into two 1:M links. 2. On BorrowRecord — they belong to a *transaction*, not to the book itself (a book is issued many times).

## 4.8 Database Objects (Tables, Forms, Queries, Reports)

**LEARN IT**

Four objects you will meet in Access. Learn the one-line jobs cold:

| Object | Job                                                              | You ...                                      |
|--------|------------------------------------------------------------------|----------------------------------------------|
| Table  | Stores the data (foundation; every other object depends on it)   | keep the actual records here                 |
| Form   | Friendly screen for enter / edit / view (text boxes, drop-downs) | type data in; fewer mistakes than a raw grid |
| Query  | Ask a question: filter, sort, calculate                          | e.g. "students above 80%"                    |
| Report | Formatted output for printing or sharing                         | read only — never type into a report         |

**REMEMBER IT**

Store → Enter → Search → Print. Tables, Forms, Queries, Reports.

**COMMON MISTAKE**

A query does not copy the table into a second storage place. It is a saved question. Change the table, re-run the query, and the answer updates.

ٹیبل ڈیٹا رکھتی ہے، فارم سے اندراج ہوتا ہے، کوئری سوال پوچھ کر چھانتی ہے، رپورٹ چھپائی کے لیے سنواری ہوئی پیشکش ہے — اس میں ڈیٹا بھرا نہیں جاتا۔

**CHECK YOURSELF**

1. Which object would you use for a printed fee slip?
2. Which object actually holds the fee amount?

Answers: 1. Report. 2. Table.

## 4.9 Creation of Tables in Microsoft Access

### LEARN IT

**Plan first** (4.9.1): purpose, field list, data type of each field, which field is the PK.

**Datasheet View** (4.9.2) — fast, spreadsheet-like, good for beginners, weak on rules.

1. Create tab → Table.
2. A blank grid appears.
3. Double-click Field1, Field2, ... to rename.
4. Type data; Access guesses the data type.

Limitation: you cannot easily say "Marks cannot exceed 100". Professionals use Design View.

**Design View** (4.9.3) — full control before any data is typed.

1. Create tab → Table Design.
2. Type each Field Name (StudentID, Name, ...).
3. Pick the Data Type from the dropdown.
4. Right-click the unique field → Primary Key (key icon).
5. Save the table.

| Data type  | Use it for                                                                            |
|------------|---------------------------------------------------------------------------------------|
| Short Text | Names, addresses, phone numbers (you do not add phone numbers). Up to 255 characters. |
| Number     | Values you calculate with: Age, Marks, Quantity.                                      |
| Date/Time  | Date of birth, admission date, appointment time.                                      |
| Currency   | Fees, salary.                                                                         |
| AutoNumber | Access fills a unique sequential number — excellent PK.                               |

### COMMON MISTAKE

Phone numbers look numeric but are Short Text. Leading zeros (0333...) would vanish in a Number field, and you never sum two phone numbers.

### اردو میں سمجھیں

پہلے ٹیبل کا منصوبہ بنائیں۔ ڈیٹا شیٹ ویو تیز لیکن کمزور ہے؛ ڈیزائن ویو میں فیلڈ، ڈیٹا ٹائپ اور پرائمری کی خود منتخب کرتے ہیں۔ فون نمبر شارٹ ٹیکسٹ رکھیں۔

### CHECK YOURSELF

1. Best data type for StudentID if you want Access to generate it?
2. Best data type for FeeAmount?

Answers: 1. AutoNumber. 2. Currency.

## 4.10 Designing Forms for Data Manipulation

### LEARN IT

A form is the screen people actually type into. It cuts errors and can show validation messages. Every form **must** have a data source — a table or a query. A form with no source has nowhere to save.

**Form Wizard** (the exam method):

1. Create tab → Form Wizard.
2. Select the table (e.g. Student).
3. Move fields with > (one) or >> (all).
4. Choose layout. **Columnar** = one record at a time, best for data entry.
5. Name it ("Student Registration") → Finish.

### WORKED EXAMPLE

A Student Registration form shows Student ID (AutoNumber, locked), Full Name, Class, Department as labelled boxes, plus Save. The clerk never sees the raw datasheet and cannot accidentally overwrite the next row.

اردو میں سمجھیں

فارم اندراج کی سادہ اسکرین ہے۔ ہر فارم کسی ٹیبل یا کوئری سے جڑا ہوتا ہے۔ فارم وزرڈ سے ٹیبل چنائیں، فیلڈز منتخب کریں، کالم نما لے آؤٹ بہتر ہے، نام دے کر ختم کریں۔

### CHECK YOURSELF

1. Can a form exist without a table or query?
2. Which layout is best for entering one student at a time?

Answers: 1. No — it needs a data source. 2. Columnar.

## 4.11 Creating Queries in MS Access ★ Golden

### LEARN IT

A query retrieves, filters and analyses data **without changing the original tables**. A simple SELECT query:

1. Create tab → Query Design.
2. Show Table → add Student → Close.
3. Double-click the fields you want; they drop into the grid.
4. Red Run (!) button executes it.

**Criteria** restrict rows. Type a value in the Criteria row under a field.

| Goal                  | Where to type                  | Example                                      |
|-----------------------|--------------------------------|----------------------------------------------|
| Single criterion      | Criteria row, one column       | Department → "Computer Science"              |
| AND (both must match) | Same Criteria row, two columns | Name "Ali" and Department "Computer Science" |
| OR (either may match) | Criteria row and the or: row   | Department "IT" or "Computer Science"        |

Wildcards in Access: \* = any characters, ? = one character. Like "A\*" finds Ali, Ahmed, Ayesha.

#### WORKED EXAMPLE

Equivalent SQL (what Access writes behind the grid):

```
SELECT StudentName, Department
FROM Student
WHERE Department = "Computer Science";

SELECT StudentName, Department
FROM Student
WHERE StudentName = "Ali" AND Department = "Computer Science";

SELECT StudentName, Department
FROM Student
WHERE Department = "IT" OR Department = "Computer Science";
```

#### EXAM TIP

Remember: AND = same row in the grid; OR = different rows. Examiners draw a blank grid and ask you to fill Criteria and or:.

#### COMMON MISTAKE

Text criteria need quotes: "IT". Numbers do not:  $\geq 80$ . Mixing these is a common cause of empty results.

#### اردو میں سمجھیں

کوئری ٹیبل کو بدلے بغیر سوال پوچھتی ہے۔ ایک شرط Criteria قطار میں، AND ایک ہی قطار کی دو کالموں میں، OR الگ قطار (or): میں۔ وائلڈ کارڈ \* کئی حروف، ? ایک حرف۔

#### CHECK YOURSELF

1. Criteria for marks at least 80?
2. You want CS or IT. Same row or two rows?

Answers: 1.  $\geq 80$  under Marks. 2. Two rows (OR).

## 4.11.4 Data Summarization and Statistical Analysis

#### LEARN IT

**Grouping** combines rows that share a value (all CS students together) and then an **aggregate** calculates a number for the group. In Access click Totals ( $\Sigma$ ) to reveal the Total row in the grid.

| Function      | What it calculates                       |
|---------------|------------------------------------------|
| COUNT()       | How many records (students in IT)        |
| SUM()         | Total of a number field (fees collected) |
| AVG()         | Mean                                     |
| MAX() / MIN() | Highest / lowest in the group            |

### WORKED EXAMPLE

Average marks per department: add Department and Marks; Totals Σ; Department → Group By; Marks → Avg; Run.

| Department       | Avg of Marks |
|------------------|--------------|
| Computer Science | 82.5         |
| IT               | 76.0         |

Check: CS marks 80, 85 → average  $(80+85)/2 = 82.5$ . IT 70, 82 → 76.0.

```
SELECT Department, AVG(Marks) AS AvgOfMarks
FROM Student
GROUP BY Department;
```

### اردو میں سمجھیں

گروپ بائی ایک جیسے ریکارڈ جوڑتا ہے، پھر COUNT، SUM، AVG، MAX، MIN سے خلاصہ نکلتا ہے۔ ایکسیس میں Σ بٹن Total قطار کھولتا ہے۔

### CHECK YOURSELF

1. Which Total setting goes under Department?
2. Which function counts how many books each member has borrowed?

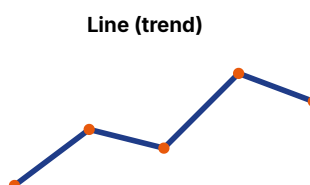
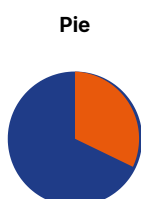
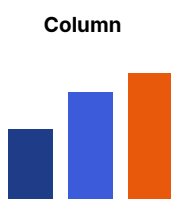
Answers: 1. Group By. 2. COUNT (of BorrowID or BookID) grouped by MemberID.

## 4.11.6 Data Visualization in MS Access (Charts)

### LEARN IT

Charts turn numbers into a picture so a pattern jumps out. Pick the chart that matches the question:

| Chart             | Shape              | Best for                                                     |
|-------------------|--------------------|--------------------------------------------------------------|
| Bar (horizontal)  | Bars sideways      | Categories with long names (English, Mathematics, ...)       |
| Column (vertical) | Bars standing      | Side-by-side comparison (2024 / 2025 / 2026 admissions)      |
| Pie               | Slices of a circle | Parts of one whole (percent of CS vs IT vs Eng). Few slices. |
| Line              | Dots joined        | A trend over time (visitors per month)                       |



Column = compare categories. Pie = share of a total. Line = change over time.

#### EXAM TIP

"Which chart?" — line for time, pie for composition, bar/column for comparing categories. Mention one reason.

#### اردو میں سمجھیں

بار لمبے ناموں والی اقسام، کالم سادہ موازنہ، پائی حصوں کا تناسب، لائن وقت کے ساتھ رجحان۔ چارٹ کوئری یا ٹیبل کے اعداد کو تصویر بناتا ہے۔

#### CHECK YOURSELF

1. Website visitors each month — which chart?
2. Share of boys vs girls in one class — which chart?

Answers: 1. Line. 2. Pie (two parts of a whole).

## Chapter 4 Review

### Chapter summary

- Data = raw facts; information = processed data used for decisions; a database stores related data in a structured, updatable way.
- A DBMS is the software layer that provides less redundancy, consistency, security and integrity.
- Table = rows (records) and columns (fields). One table, one entity.
- Candidate keys could identify a row; the primary key is the one you choose (unique, not NULL); leftover candidates are alternate keys; a foreign key points at another table's PK.
- An RDBMS keeps several related tables and is queried with SQL.
- ER symbols: rectangle entity, oval attribute, diamond relationship. Cardinality 1:1, 1:M, M:N. M:N needs a junction table.
- Referential integrity forbids orphan FKs. Cascade Update copies a changed PK into children; Cascade Delete removes children with the parent.
- Write a relational schema before building. Library case: Member and Book are M:N, resolved by BorrowRecord.
- Objects: tables store, forms enter, queries search, reports print.
- Access Design View sets field names, data types and the PK. Phone numbers are Short Text; money is Currency; auto IDs are AutoNumber.
- Forms need a table/query source; Wizard + columnar layout for data entry.
- Query criteria: one value; AND on the same row; OR on two rows. Totals  $\Sigma$  for Group By / Count / Sum / Avg / Max / Min.
- Bar/column compare categories, pie shows parts of a whole, line shows a trend over time.



## Key terms

| Term                  | Meaning                                           | اردو                  |
|-----------------------|---------------------------------------------------|-----------------------|
| Data                  | Raw facts and figures.                            | ڈیٹا / خام حقائق      |
| Information           | Processed data that supports a decision.          | معلومات               |
| Database              | Organised collection of related data.             | ڈیٹا بیس              |
| DBMS                  | Software that creates and manages databases.      | ڈیٹا بیس مینجمنٹ سسٹم |
| Table                 | Rows and columns about one entity.                | ٹیبل                  |
| Record                | One row — one instance.                           | ریکارڈ                |
| Field                 | One column — one attribute.                       | فیلڈ                  |
| Primary key           | Chosen unique, non-empty identifier.              | پرائمری کی            |
| Foreign key           | Field that refers to another table's PK.          | فارن کی               |
| Candidate key         | Any field(s) that could be the PK.                | کینڈیڈیٹ کی           |
| Alternate key         | A candidate not chosen as PK.                     | آلٹرنیٹ کی            |
| RDBMS                 | DBMS that stores related tables linked by keys.   | ریلیشنل ڈی بی ایم ایس |
| SQL                   | Language to query relational databases.           | ساختمہ استفساری زبان  |
| Entity                | Object we store data about (ER rectangle).        | ہستی                  |
| Attribute             | Property of an entity (ER oval).                  | صفت                   |
| Relationship          | Link between entities (ER diamond).               | تعلق                  |
| Cardinality           | 1:1, 1:M or M:N.                                  | عددییت                |
| Junction table        | Bridge table that splits an M:N into two 1:M.     | جنکشن ٹیبل            |
| Referential integrity | Every FK matches a live PK (no orphans).          | حوالہ جاتی درستگی     |
| Cascade Update        | Parent PK change copied to child FKs.             | کیسکیڈ اپ ڈیٹ         |
| Cascade Delete        | Deleting the parent deletes related children.     | کیسکیڈ ڈیلیٹ          |
| Schema                | Written blueprint of tables, keys, relationships. | اسکیما / نقشہ         |
| Form                  | Screen for entering data.                         | فارم                  |
| Query                 | Saved question that filters/calculates data.      | کوئری                 |
| Report                | Formatted printable output.                       | رپورٹ                 |
| AutoNumber            | Access-generated unique integer, often the PK.    | آٹو نمبر              |

## Practice: multiple choice

1. Processed data used for a decision is called:
  - a. a field
  - b. information
  - c. a query
  - d. redundancy
2. A complete row about one student is a:
  - a. field
  - b. report
  - c. record
  - d. schema
3. A primary key cannot contain:
  - a. numbers
  - b. text
  - c. NULL
  - d. an AutoNumber
4. EmpLicense# after EmpID is chosen as PK is an:
  - a. foreign key
  - b. alternate key
  - c. orphan
  - d. report
5. A foreign key:
  - a. must be unique in the child table
  - b. may contain duplicates
  - c. cannot exist in an RDBMS
  - d. is always AutoNumber
6. The ER symbol for a relationship is a:
  - a. rectangle
  - b. oval
  - c. diamond
  - d. circle
7. The most common cardinality is:
  - a. 1:1
  - b. 1:M
  - c. M:N stored as-is
  - d. 0:0
8. M:N is stored in an RDBMS by:
  - a. duplicating both tables
  - b. a junction table
  - c. deleting one entity
  - d. using a pie chart
9. A child row whose StudentID is not in Student is:
  - a. a candidate key
  - b. an orphan record
  - c. a form
  - d. a primary key
10. Changing parent ID 101 → 201 and seeing the child follow is:
  - a. Cascade Delete
  - b. Cascade Update
  - c. a SELECT query
  - d. a pie chart
11. Which Access object is for printing a report card?
  - a. Table
  - b. Form
  - c. Query
  - d. Report
12. Phone numbers should be stored as:
  - a. Number
  - b. Currency
  - c. Short Text
  - d. Date/Time
13. AND criteria go:
  - a. on two different rows of the grid
  - b. on the same Criteria row
  - c. in a report header
  - d. only in SQL Server
14. To average marks per department you set Department to:
  - a. Sum
  - b. Where
  - c. Group By
  - d. Expression
15. The best chart for admissions in 2024, 2025, 2026 is a:
  - a. pie chart
  - b. column chart
  - c. junction table
  - d. foreign key

## Practice: short questions

1. Differentiate data and information with one example each.
2. List four advantages of a DBMS.
3. Define record and field with reference to a Student table.
4. Differentiate primary key and foreign key (four points).
5. Why can a relational database not store M:N directly?
6. What is referential integrity? Give one rejected-insert example.
7. Differentiate Cascade Update and Cascade Delete.
8. State the four Access objects and the job of each in one line.
9. Why are phone numbers Short Text, not Number?
10. How do you express OR in Query Design?

## Practice: long / problem questions

1. Draw a complete ER diagram for Student–Course enrolment (M:N). Resolve it with a junction entity. Write the relational schema with keys marked.
2. A Result table uses StudentID as a foreign key. Explain referential integrity, then show before/after tables for (a) Cascade Update of ID 101 → 201 and (b) Cascade Delete of student 102.
3. Work through the library case study: requirements, entities, attributes, relationship, cardinality, junction table, final schema. Include two sample BorrowRecord rows.
4. Write step-by-step how to create a Student table in Access Design View (fields RollNo Number PK, Name Short Text, Marks Number, Fee Currency, AdmDate Date/Time) and then a columnar form with the Wizard.
5. Write the Query Design settings and the SQL to list CS students with Marks  $\geq 80$ , sorted by Name. Then write a totals query that shows COUNT of students and AVG(Marks) for each Department.
6. A principal wants to see (i) fee collected per class, (ii) the share of each class in total enrolment, (iii) enrolment over five years. Which chart for each, and why?

## Answer key

---

### Multiple choice

1. b
2. c
3. c
4. b
5. b
6. c
7. b
8. b
9. b
10. b
11. d
12. c
13. b
14. c
15. b

## Short questions — model answers

1. Data = raw facts (80, 75, 90). Information = processed meaning ("average is 81.7%, the class is ready").
2. Less redundancy, consistency, security (authorised access), integrity (valid, unique keys). (Sharing / backup also accepted.)
3. Record = one student's full row (101, Ali, CS, A). Field = one column such as Name.
4. PK uniquely identifies a row, cannot be NULL, one per table, lives in the parent. FK refers to a PK, may duplicate, may be NULL, lives in the child, creates the relationship.
5. It would duplicate attributes of both entities. Split into two 1:M links with a junction table that holds both FKs plus extra attributes (dates).
6. Every FK must match an existing parent PK. Inserting Result.StudentID = 999 is rejected if 999 is not in Student — that would have been an orphan.
7. Cascade Update copies a changed parent PK into child FKs. Cascade Delete removes child rows when the parent is deleted.
8. Table stores; form enters; query searches/filters; report prints.
9. They are identifiers, not quantities; leading zeros would vanish; you never add two numbers of a phone.
10. Put one value on the Criteria row and the other on the or: row of the same field (or of different fields if that is the question).

## Long questions — model answers

1. Entities STUDENT and COURSE, relationship Enrolls (M:N). Junction ENROLLMENT(EnrollID PK, StudentID FK, CourseID FK, Date). Schema: Student(StudentID PK, Name, Class); Course(CourseID PK, Title); Enrollment as above. Diagram: 1 — M — M — 1 through the junction.
2. (a) Parent 101 Ali → 201 Ali; child Result rows that said 101 now say 201. (b) Delete parent 102 Sara; all Result and Enrolment rows with 102 disappear. Integrity definition: FK must match a live PK; this stops orphans and keeps reports true.
3. Follow steps 1–7 in the chapter. Sample: (5001, 102, B-17, 2026-10-05, NULL); (5002, 101, B-17, 2026-10-12, 2026-10-20) — same book, two members, no duplicated titles.
4. Create → Table Design; enter the five fields and types; right-click RollNo → Primary Key; Save as Student. Create → Form Wizard → Student → >> all fields → Columnar → name Student Registration → Finish.
5. Grid: Name, Department, Marks. Criteria under Department: "CS" (or "Computer Science"); Criteria under Marks:  $\geq 80$  (same row = AND); Sort under Name: Ascending. SQL: `SELECT Name, Department, Marks FROM Student WHERE Department="CS" AND Marks  $\geq 80$  ORDER BY Name`; Totals query: Department Group By, StudentID Count, Marks Avg.
6. (i) Column or bar — compare categories (classes). (ii) Pie — each class is a part of one whole enrolment. (iii) Line — a trend over years.

## Self-assessment: I can...

- |                                                                                      |                                                                                                   |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> explain data vs information vs database vs DBMS.            | <input type="checkbox"/> point at records and fields in a table.                                  |
| <input type="checkbox"/> choose PK / FK / candidate / alternate keys correctly.      | <input type="checkbox"/> draw ER symbols and label 1:1, 1:M, M:N.                                 |
| <input type="checkbox"/> resolve M:N with a junction table.                          | <input type="checkbox"/> explain referential integrity and both cascade operations with examples. |
| <input type="checkbox"/> write a relational schema including the library case study. | <input type="checkbox"/> match tables, forms, queries and reports to a task.                      |
| <input type="checkbox"/> create a table in Design View and pick data types.          | <input type="checkbox"/> build a Wizard form and a criteria query (AND, OR, totals).              |
| <input type="checkbox"/> choose bar, column, pie or line for a given question.       |                                                                                                   |

# Application and Impacts of Computing

This chapter shows you how computing technologies such as Artificial Intelligence, the Internet of Things and Data Analytics are changing life in Pakistan — in cities, hospitals, farms and classrooms. You will also learn how to judge information sources, weigh the good and bad effects of computing, and see how assistive technologies give everyone an equal chance.

اس باب میں آپ سیکھیں گے کہ مصنوعی ذہانت (AI)، انٹرنیٹ آف تھنگز (IoT) اور ڈیٹا اینالیٹکس ہماری روزمرہ زندگی کو کیسے بدل رہے ہیں۔ آپ معلومات کے ذرائع کو پرکھنا، کمپیوٹنگ کے مثبت اور منفی اثرات سمجھنا، اور معذور افراد کی مدد کرنے والی ٹیکنالوجی کے بارے میں بھی جانیں گے۔

## What you will learn

- Explain what Artificial Intelligence (AI) is and give everyday examples of it.
- Name and explain the six components of an IoT system, with examples of each.
- Compare IoT, AI and Data Analytics by definition, purpose, type of work and output.
- Discuss the uses, benefits and challenges of AI in education in Pakistan.
- Analyse positive and negative impacts of computing in five fields, and practise responsible use.
- Describe how IoT, Data Analytics and AI work together in a smart system.
- List the five components of Data Analytics in the correct order.
- Describe uses of IoT in smart cities, industry, healthcare, education, homes and farming.
- Classify information sources as primary, secondary or tertiary, and judge when a source is unreliable.
- Explain the four categories of assistive technologies, their importance, and related careers.

## 5.1 Introduction to Computing Impacts

### LEARN IT

**Computing** means using computers and computer-based devices to store, process and share information. Today computing is an important part of your daily life. You use computers, mobile phones, the internet and smart devices almost everywhere — at home, in college and on the road.

In Pakistan, computing is becoming **vital** (very necessary) in:

- **Schools and colleges** — online admissions, e-books, online results of the Sindh boards.
- **Hospitals** — computerised patient records, digital X-rays, online appointment systems.
- **Banks** — ATMs, mobile banking apps, Raast instant payments, JazzCash and Easypaisa wallets.
- **Farms** — weather forecasts on mobile phones, soil sensors, tube-well control.
- **Industries** — automatic machines in textile mills in Karachi, Hyderabad and Faisalabad.

An **impact** is the effect something has on people and society. Computing has **positive impacts** (speed, convenience, new jobs) and **negative impacts** (privacy risks, addiction, job loss). In this chapter you will study three key technologies first — AI, IoT and Data Analytics — and then their uses and impacts.

#### WORKED EXAMPLE

##### A day in Sara's life in Karachi — spot the computing:

**Step 1.** 7:00 am — Her phone unlocks by recognising her face (AI).

**Step 2.** 7:30 am — She books a ride on a ride-hailing app; GPS shows the car's location (internet + location data).

**Step 3.** 9:00 am — College attendance is marked by a biometric (fingerprint) machine (IoT).

**Step 4.** 2:00 pm — She pays her fee through a mobile banking app (computing in business).

**Step 5.** 8:00 pm — YouTube suggests a physics lecture based on what she watched yesterday (AI).

Conclusion: in a single day, Sara uses computing in at least five different fields without even thinking about it.

#### اردو میں سمجھیں

کمپیوٹنگ آج ہماری روزمرہ زندگی کا اہم حصہ بن چکی ہے۔ پاکستان میں اسکول، اسپتال، بینک، کھیت اور صنعتیں سب کمپیوٹر اور انٹرنیٹ پر انحصار کرنے لگے ہیں۔ کمپیوٹنگ کے فائدے بھی ہیں اور نقصانات بھی، جنہیں ہم اس باب میں سمجھیں گے۔

#### CHECK YOURSELF

1. Name four places in Pakistan where computing has become vital.
2. What is meant by the "impact" of computing?

Answers: 1. Any four of: schools, hospitals, banks, farms, industries. 2. The effect computing has on people and society — it can be positive or negative.

## 5.1.1 Artificial Intelligence (AI) ★ Golden

#### LEARN IT

**Artificial Intelligence (AI)** is the ability of a computer system to perform tasks that normally need human thinking, such as recognising faces, understanding language and making decisions.

Understand the name in two parts:

- **Artificial** — it is *created by humans*; it is not natural.
- **Intelligence** — it can perform tasks that *require thinking*.

AI systems **use data to make decisions**. The more good data an AI system sees, the better it becomes at recognising patterns (repeating features) and predicting what to do.

#### Examples from the source:

1. **Face recognition** on a mobile phone to unlock the screen.
2. **YouTube suggesting videos** based on your interests.
3. **Chatbots** answering questions on a website (for example, a bank's help chat).

More Pakistani examples: Google Maps predicting traffic on Shahrah-e-Faisal, Urdu voice typing on your phone, banks flagging a suspicious card transaction, and AI tools that translate English into Urdu.

### AI in smart systems

AI rarely works alone. It often acts as the **"brain"** of a system, working together with the **Internet of Things (IoT)** and **Data Analytics**:

- **IoT devices gather data** — they are the "eyes" (sensors).
- **Analytics sorts the data** — it organises and studies it.
- **AI makes the final decision** — it is the "brain".
- An **action** follows — something happens in the real world.

IoT (Eyes) — collects data



Analytics — sorts data



AI (Brain) — decides



Action

#### WORKED EXAMPLE

**Smart air-conditioner in a hot Hyderabad room — trace the system:**

**Step 1 (IoT – eyes).** A temperature sensor reads the room temperature: 36 °C.

**Step 2 (Analytics).** The readings of the last 10 minutes are organised: 34, 35, 35, 36 °C — the room is getting hotter.

**Step 3 (AI – brain).** The AI compares this with the comfortable limit (say 26 °C) and with the fact that a person is in the room (motion sensor). Decision: "Turn the AC on."

**Step 4 (Action).** A switch turns the AC on automatically.

No human pressed a button — the system sensed, thought and acted.

#### REMEMBER IT

**Eyes – Sort – Brain – Do.** IoT is the *eyes*, Analytics *sorts*, AI is the *brain*, and the actuator *does* the action. Think of a cricket fielder: his eyes see the ball (IoT), he judges speed and direction (analytics), decides to dive (AI), and his body moves (action).

#### COMMON MISTAKE

**Mistake:** "AI means robots." **Fix:** Most AI is just *software* — YouTube suggestions, spam filters and face unlock have no robot body. A robot may use AI, but AI itself is the thinking software.

#### اردو میں سمجھیں

مصنوعی ذہانت (AI) کو "مصنوعی" اس لیے کہتے ہیں کہ اسے انسانوں نے بنایا ہے، اور "ذہانت" اس لیے کہ یہ ایسے کام کر سکتی ہے جن کے لیے سوچنا پڑتا ہے۔ AI ڈیٹا کی مدد سے فیصلے کرتی ہے، جیسے چہرہ پہچان کر فون کھولنا یا یوٹیوب پر ویڈیوز تجویز کرنا۔ ایک اسمارٹ سسٹم میں IoT آنکھوں کا کام کرتا ہے اور AI دماغ کا۔

#### EXAM TIP

Common questions: "Define AI and give two examples" (2–3 marks), and "Explain how AI, IoT and Data Analytics work together" (4–5 marks).

To score full marks: (1) write the two-part meaning of the name — *artificial* = *made by humans*, *intelligence* = *can think/decide*; (2) say "AI uses data to make decisions"; (3) give at least two examples; (4) for the "together" question, draw the flow IoT → Analytics → AI → Action and explain it with the room-AC example.

#### CHECK YOURSELF

1. Why is AI called "artificial"?
2. In a smart system, which technology is the "brain" and which is the "eyes"?
3. Is a spam filter in Gmail an example of AI? Why?

Answers: 1. Because it is created by humans. 2. AI is the brain; IoT is the eyes. 3. Yes — it uses data (past spam emails) to decide whether a new email is spam, a task that needs "thinking".

## 5.1.2 Internet of Things (IoT) & Components ★ Golden

### LEARN IT

The **Internet of Things (IoT)** is a system in which **physical devices are connected to the Internet** and can **collect and share data** to perform tasks **automatically**.

The "things" are ordinary objects — watches, cameras, cars, fans, water pumps — that have been given sensors and an internet connection so they can "talk" to each other and to us.

**Source example:** A **smartwatch** collects your heart-rate data and sends it to your mobile phone, where it is displayed.

### Components of IoT systems

IoT relies on **six foundational components** to work properly.

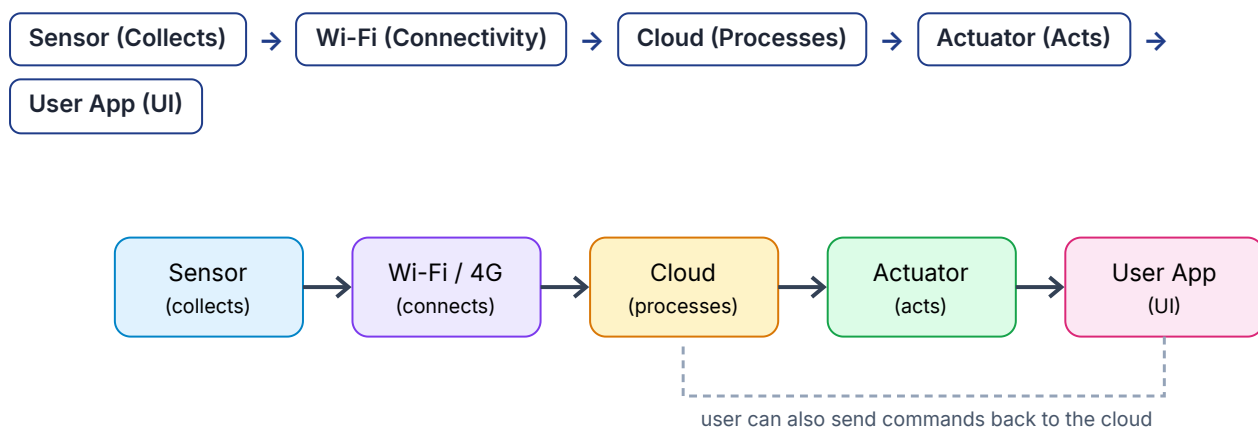
| # | Component                  | What it does                                                                                                                      | Examples                                                                                  |
|---|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1 | <b>Sensors</b>             | Devices that <b>collect information</b> from the physical world. They are the "eyes and ears" of the system.                      | Temperature sensor, motion sensor, fingerprint sensor, smoke sensor, soil moisture sensor |
| 2 | <b>Actuators</b>           | Devices that <b>perform physical actions</b> in response to system commands. They convert digital signals into physical movement. | Electric motor, switch, automatic door opener, water valve, fan controller                |
| 3 | <b>Connectivity</b>        | The connection that allows devices to <b>send and receive data</b> . Without it, sensors can't talk to the system.                | Wi-Fi, Bluetooth, ZigBee, 4G/5G                                                           |
| 4 | <b>Data Processing</b>     | <b>Changing raw data into useful information.</b><br>Steps: collecting, organising, analysing and deciding.                       | Checking if the room temperature is too hot to trigger the AC                             |
| 5 | <b>Cloud Computing</b>     | <b>Storing data and using services over the internet</b> instead of local hard drives. Handles massive IoT data.                  | Google Drive, iCloud, online LMS                                                          |
| 6 | <b>User Interface (UI)</b> | The component that enables <b>interaction between the user and the device</b> . Helps users see information and give commands.    | Mobile apps, touch screens, voice commands                                                |



## Understanding each component

- **Sensors** turn something in the real world (heat, movement, a fingerprint, smoke, wetness of soil) into data. A smoke sensor in a Karachi shopping mall "smells" smoke and reports it.
- **Actuators** do the opposite: they take a digital command and make something *move* or *change*. The water valve that opens, the motor that turns, the door that slides open at a bank entrance.
- **Connectivity** is the "road" for data. Short distances use Bluetooth (watch to phone); a house uses Wi-Fi; low-power sensor networks use **ZigBee**; devices far away in fields use mobile networks (4G/5G from Jazz, Zong, Telenor/Ufone).
- **Data processing** is where raw numbers ("36") become meaning ("too hot — turn on AC"). Its four steps are *collect* → *organise* → *analyse* → *decide*.
- **Cloud computing** stores and processes the huge amount of data that thousands of sensors produce every second — far more than a single small device can hold.
- **User Interface** is how *you* see the data and give orders — an app on your phone, a touch screen on a smart fridge, or saying "Hey Google, turn off the lights".

## How the components connect



The six IoT components working as one system (data processing happens inside the cloud).

### WORKED EXAMPLE

#### Smart fire alarm in a Karachi factory — identify all six components:

**Step 1 — Sensor:** A smoke sensor detects smoke near the machines.

**Step 2 — Connectivity:** It sends the reading over the factory Wi-Fi.

**Step 3 — Cloud computing:** The reading is stored on an online server along with the history of all sensors.

**Step 4 — Data processing:** The software checks: "Is the smoke level above the safe limit?" Yes → decide "Fire risk!"

**Step 5 — Actuator:** Water valves of the sprinklers open and the alarm bell switch turns on.

**Step 6 — User interface:** The manager's mobile app shows a red alert: "Smoke detected – Hall 3".

### REMEMBER IT

**Sensor senses, Actuator acts.** Both words give you the answer: **sensor** = **see**/sense (input), **actuator** = **act** (output). Remember the six components with "**Some Ants Carry Data Carefully Up**" — Sensors, Actuators, Connectivity, Data processing, Cloud, UI.

#### COMMON MISTAKE

**Mistake:** Calling a fan controller or water valve a "sensor". **Fix:** Ask "does it *measure* something or *do* something?" A valve opens and closes — it *does*, so it is an **actuator**. A soil moisture sensor only *measures*, so it is a **sensor**.

#### اردو میں سمجھیں

انٹرنیٹ آف تھنگز (IoT) ایک ایسا نظام ہے جس میں مختلف آلات (جیسے گھڑیاں، کیمرے، گاڑیاں) انٹرنیٹ سے جڑے ہوتے ہیں اور خودکار طریقے سے ڈیٹا جمع اور شیئر کرتے ہیں۔ اس کے چھ بنیادی اجزاء ہیں: سینسرز (معلومات جمع کرتے ہیں)، ایکچویٹرز (جسمانی کام انجام دیتے ہیں جیسے موٹر چلانا یا دروازہ کھولنا)، کنیکٹیویٹی (وائی فائی، بلوٹوتھ)، ڈیٹا پروسیسنگ، کلاؤڈ کمپیوٹنگ، اور یوزر انٹرفیس (موبائل ایپ)۔

#### EXAM TIP

"Define IoT and explain its components" is one of the most frequent long questions. Write the definition with the three key ideas — *physical devices, connected to the internet, collect and share data automatically*. Then list all **six** components, each with a one-line meaning and one example. A labelled flow diagram (Sensor → Wi-Fi → Cloud → Actuator → App) earns extra credit.

Short questions often ask: "Differentiate between a sensor and an actuator" — sensor collects data (input, eyes and ears), actuator performs physical action (output, converts digital signals into movement).

#### CHECK YOURSELF

1. Classify as sensor or actuator: (a) fingerprint scanner, (b) automatic door opener, (c) motion detector, (d) electric motor.
2. Name four types of connectivity used in IoT.
3. Why is cloud computing needed in IoT?

Answers: 1. (a) sensor, (b) actuator, (c) sensor, (d) actuator. 2. Wi-Fi, Bluetooth, ZigBee, 4G/5G. 3. Because IoT devices produce massive amounts of data; the cloud stores and processes it over the internet instead of on small local hard drives.

## 5.1.3 Data Analytics & Components

#### LEARN IT

**Data Analytics** is the process of **collecting, organising, cleaning, studying and presenting data** to find useful information. It helps people make better decisions.

**Data** means raw facts and figures (for example, "72, 45, 88"). **Information** is data that has been processed so it has meaning ("Ali scored 72%; he passed"). Data Analytics is the bridge from data to information.

**Source example:** A college collects exam marks, calculates percentages, and identifies which students need extra help. This entire process is Data Analytics.

### Components of Data Analytics

Data Analytics comprises **five key components**, which happen in order like a pipeline:

1. Collection → 2. Storage → 3. Cleaning → 4. Analysis → 5. Visualization

- 1. Data Collection** — Gathering raw information from multiple sources: websites, surveys, sensors, hospital records, bank transactions.
- 2. Data Storage** — Storing data safely and properly after collection. It can be stored in files, databases, spreadsheets or the cloud.
- 3. Data Cleaning** — Identifying and correcting errors in data. Raw data may contain:
  - **Duplication** of records (the same student entered twice),
  - **Incorrect values** (typos, e.g. marks "870" instead of "87"),
  - **Missing** files or records (a student's marks not entered),
  - **Improper formats** (dates written as 05/10/2026 in one place and "5 Oct 26" in another).
- 4. Data Analysis** — Studying the clean data to identify patterns, trends and relevant information for problem-solving (e.g. finding out which subjects students failed the most).
- 5. Data Visualization** — Creating visual representations of data (plots, charts, graphs) to provide valuable insights at a glance (e.g. a pie chart showing pass vs fail percentage).

#### WORKED EXAMPLE

A college in Sukkur analyses Physics test results (out of 50) for 6 students.

**Step 1 — Collection:** Marks are gathered from the teacher's register.

| Roll No | Name   | Marks (raw) |
|---------|--------|-------------|
| 1       | Ayesha | 42          |
| 2       | Bilal  | 18          |
| 3       | Hina   | 35          |
| 3       | Hina   | 35          |
| 4       | Kashif | 250         |
| 5       | Sana   | (missing)   |

**Step 2 — Storage:** The marks are typed into a spreadsheet saved on Google Drive (cloud).

**Step 3 — Cleaning:** Hina's record is a *duplicate* → delete one copy. Kashif's 250 is an *incorrect value* (more than 50) → the register shows 25, so correct it. Sana's marks are *missing* → ask the teacher; she was absent, so mark her "absent".

**Step 4 — Analysis:** Clean marks: 42, 18, 35, 25. Pass mark is 17 (33% of 50 = 16.5, rounded up). All 4 passed. Percentages: Ayesha 84%, Bilal 36%, Hina 70%, Kashif 50%. Average =  $(42 + 18 + 35 + 25) \div 4 = 120 \div 4 = 30$  marks = 60%. Bilal is closest to failing → needs extra help.

**Step 5 — Visualization:** A bar chart shows each student's percentage; Bilal's short bar makes the problem visible at a glance.

#### REMEMBER IT

**"Cows Sit Calmly And Vote"** — Collection, Storage, Cleaning, Analysis, Visualization. Or think of cooking daal: buy the lentils (collect), keep them in a jar (store), pick out stones (clean), cook (analyse), serve beautifully on a plate (visualise).

#### COMMON MISTAKE

**Mistake:** Skipping cleaning and going straight to analysis. **Fix:** Dirty data gives wrong answers. In the example, Kashif's wrong "250" would have made the average  $(42 + 18 + 35 + 250) \div 4 = 86.25$  — more than 50, which is impossible. Always clean first.

ڈیٹا اینالیٹکس ڈیٹا کو جمع کرنے، ترتیب دینے، صاف کرنے، اس کا مطالعہ کرنے اور پیش کرنے کا عمل ہے تاکہ مفید معلومات حاصل ہوں اور بہتر فیصلے کیے جا سکیں۔ اس کے پانچ اجزاء ہیں: ڈیٹا جمع کرنا، محفوظ کرنا، صاف کرنا (غلطیاں اور دہرائے گئے ریکارڈ ہٹانا)، تجزیہ کرنا، اور چارٹ یا گراف کی صورت میں دکھانا۔

#### CHECK YOURSELF

1. List the five components of Data Analytics in order.
2. Name four kinds of errors removed during data cleaning.
3. A pie chart of pass/fail percentage belongs to which component?

Answers: 1. Collection, Storage, Cleaning, Analysis, Visualization. 2. Duplicate records, incorrect values (typos), missing records, improper formats. 3. Data Visualization.

## Comparison: IoT vs AI vs Data Analytics

#### LEARN IT

These three technologies are often used together, so students mix them up. The table below, from the source, compares them on four aspects.

| Aspect                        | IoT                                                    | Artificial Intelligence (AI)                               | Data Analytics                                              |
|-------------------------------|--------------------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------|
| <b>Definition</b>             | Network of physical devices connected to the internet. | Technology enabling machines to think and decide.          | Process of examining data to find patterns.                 |
| <b>Main purpose</b>           | Data collection and automation.                        | Intelligent decision making and predictions.               | Extracting meaningful insights.                             |
| <b>Type of work</b>           | Works with physical devices (hardware).                | Works with smart software systems.                         | Works with data (numbers, records).                         |
| <b>Output</b>                 | Raw data from the environment.                         | Smart decisions or actions.                                | Reports, graphs, insights.                                  |
| <b>Role in a smart system</b> | Eyes and hands                                         | Brain                                                      | Organiser / reporter                                        |
| <b>Pakistani example</b>      | Safe City cameras in Lahore/Karachi capturing traffic  | Software recognising a car's number plate for an e-challan | Monthly report showing which roads have the most violations |

#### WORKED EXAMPLE

##### One problem, three technologies — a K-Electric smart meter:

**IoT:** The smart electricity meter measures units used every 15 minutes and sends them over the mobile network. Output: raw readings.

**Data Analytics:** The company cleans and studies a month of readings and draws a graph showing that most electricity is used between 7 pm and 11 pm. Output: report/graph.

**AI:** Software learns each area's pattern, predicts tomorrow's demand, and decides how to plan load management. Output: smart decision.

#### REMEMBER IT

**Ask "What is the output?"** Raw readings → IoT. Charts and reports → Data Analytics. A decision or prediction → AI.

#### COMMON MISTAKE

**Mistake:** Writing that IoT "makes decisions". **Fix:** In the comparison table IoT's main purpose is *data collection and automation*; the *intelligent decision* belongs to AI.

#### اردو میں سمجھیں

IoT جسمانی آلات (ہارڈویئر) کا نیٹ ورک ہے جو ماحول سے خام ڈیٹا جمع کرتا ہے۔ AI ذہین سافٹ ویئر ہے جو فیصلے اور پیش گوئیاں کرتا ہے۔ ڈیٹا اینالیٹکس ڈیٹا کا جائزہ لے کر رپورٹس اور گراف بناتا ہے۔ تینوں مل کر ایک اسمارٹ نظام بناتے ہیں۔

#### CHECK YOURSELF

1. Which of the three works mainly with hardware?
2. What is the output of Data Analytics?

Answers: 1. IoT. 2. Reports, graphs and insights.

## 5.2 Uses of IoT in Different Areas

#### LEARN IT

IoT **improves efficiency** (doing more with less effort), **saves time**, and **enhances the overall quality of life**. It plays a significant role in developing modern **infrastructure** (the basic systems of a country — roads, power, water, hospitals). The source lists six areas.

### 1. Smart Cities

- **Traffic management:** Sensors adjust traffic signals automatically to reduce congestion (crowding). Example: signals on a busy Karachi road stay green longer for the direction with more cars.
- **Street lighting:** Lights turn ON/OFF based on daylight or movement, saving electricity.
- **Parking:** Sensors guide drivers to empty parking spaces, e.g. in a large shopping mall car park.
- **Waste management:** Smart bins send alerts when full, so trucks collect only full bins instead of driving to every bin.

### 2. Industrial Automation

- Sensors check machine **temperature and vibration** to prevent breakdowns (called predictive maintenance).
- Automated machines perform repeated tasks precisely.
- Quality-control sensors check products during manufacturing — for example, in the **textile industries of Sindh**, cameras and sensors spot faults in cloth as it is woven.

### 3. Healthcare (Smart Health)

- **Smartwatches** continuously measure heart rate and vital signs (pulse, oxygen level).
- **Remote monitoring devices** send patient data directly to doctors — vital for **rural areas of Pakistan**, where a specialist may be hundreds of kilometres away (e.g. a heart patient in Tharparkar monitored by a doctor in Karachi).
- **Emergency alert devices** inform hospitals instantly, for example when an elderly person falls.

## 4. Education (Smart Learning)

- **Smart classrooms:** digital boards and connected projectors.
- **Attendance systems:** biometric automated attendance for students and teachers (used in many Sindh government schools and colleges).
- **Resource management:** automated classroom lighting and AC control — off when the room is empty.

## 5. Home Automation

- **Smart lighting:** controlled remotely via mobile apps.
- **Security:** real-time monitoring via Wi-Fi cameras — you can watch your house gate on your phone.
- **Voice control:** using assistants (like Alexa or Google Assistant) to control TVs and appliances.

## 6. Smart Farming

Smart farming is **crucial for agriculture in Pakistan** (especially Sindh and Punjab) to **save water** and **improve yields** (the amount of crop produced).

- **Soil moisture sensors:** check if crops actually need water.
- **Automated irrigation:** turns water pumps ON/OFF based on soil data.
- **Drones:** monitor crop health from above, spotting disease or dry patches.

Smart farming process:



### WORKED EXAMPLE

**A cotton farmer near Nawabshah uses smart irrigation:**

**Step 1.** A soil moisture sensor in the field reads 18% moisture.

**Step 2.** It sends the reading over the 4G network to a cloud service.

**Step 3.** The cloud AI checks: cotton needs at least 25% moisture, and the weather forecast shows no rain → decision "Needs water: YES".

**Step 4.** The actuator (pump switch) turns the tube-well ON.

**Step 5.** When moisture reaches 30%, the AI decides "enough" and the pump turns OFF.

Result: water is used only when needed — important because Sindh faces water shortages — and the farmer sees everything on his phone app.

### REMEMBER IT

Remember the six areas with "**CHIEF H**": **C**ities, **H**ealth, **I**ndustry, **E**ducation, **F**arming, **H**omes.

### COMMON MISTAKE

**Mistake:** Giving only the name of the area ("smart cities") in an exam answer. **Fix:** Always add *what* IoT does there — e.g. "smart bins send alerts when full" — and, if possible, a Pakistani example.

### اردو میں سمجھیں

IoT کارکردگی بہتر بناتا ہے، وقت بچاتا ہے اور زندگی کا معیار بلند کرتا ہے۔ اسمارٹ شہروں میں ٹریفک سگنل اور اسٹریٹ لائٹس خود کار ہوتی ہیں، اسپتالوں میں مریضوں کا ڈیٹا براہ راست ڈاکٹر تک پہنچتا ہے، اور کھیتوں میں مٹی کی نمی کے سینسر بتاتے ہیں کہ فصل کو پانی چاہیے یا نہیں، جس سے سندھ اور پنجاب میں پانی کی بچت ہوتی ہے۔

### CHECK YOURSELF

1. How do smart bins help waste management?
2. Why is remote health monitoring especially useful in Pakistan?
3. In smart farming, which device is the actuator?

Answers: 1. They send alerts when full, so only full bins are emptied. 2. Rural areas lack doctors; devices send patient data directly to doctors far away. 3. The water pump (its switch).

## 5.2.1 Scope of AI in Education in Pakistan

### LEARN IT

Artificial Intelligence is gradually becoming an important part of Pakistan's education system, improving **teaching, learning** and **administration** (managing the school: admissions, records, timetables).

#### Practical uses:

- **Learning Management Systems (LMS)** — online platforms (e.g. Google Classroom, Moodle) where teachers post lessons and assignments; AI features can recommend lessons to each student.
- **Automated grading and digital testing** — MCQ tests marked instantly by computer.
- **AI-based career counselling** — software that studies a student's marks and interests and suggests suitable fields.

#### Benefits:

- Helps students learn **at their own speed**.
- **Identifies students who need extra help**.
- Supports students with disabilities (**Assistive AI**), e.g. text-to-speech for a visually impaired student.

#### Major challenges in Pakistan:

- Limited **internet access in rural areas**.
- Lack of **computers and digital devices** for all students.
- Need for proper **teacher training** to use AI tools.
- **Data privacy and security** issues — students' personal data must be protected.

| Practical uses                      | Benefits                            | Challenges                |
|-------------------------------------|-------------------------------------|---------------------------|
| LMS                                 | Learn at own speed                  | Limited rural internet    |
| Automated grading & digital testing | Identifies students needing help    | Lack of computers/devices |
| AI career counselling               | Supports students with disabilities | Teacher training needed   |
| —                                   | —                                   | Data privacy & security   |

### WORKED EXAMPLE

#### Scenario — Fatima in Larkana and an AI tutor app:

**Step 1.** Fatima takes a 10-question online maths quiz on the college LMS. It is graded instantly (automated grading).

**Step 2.** The AI notices she got all 3 trigonometry questions wrong.

**Step 3.** It recommends two short videos and five easy practice questions on trigonometry — she learns at her own speed.

**Step 4.** Her teacher's dashboard flags "Fatima – needs help in trigonometry" (identifies students who need help).

**Challenge:** At home Fatima's village has weak 3G signals, so the videos buffer — showing the rural internet challenge.

#### COMMON MISTAKE

**Mistake:** Writing that AI will replace teachers. **Fix:** The source says AI *improves* teaching and needs *teacher training* — teachers use AI as a tool; they are not replaced.

#### اردو میں سمجھیں

پاکستان کے تعلیمی نظام میں مصنوعی ذہانت آہستہ آہستہ اہم کردار ادا کر رہی ہے، جیسے لرننگ مینجمنٹ سسٹم (LMS)، خودکار جانچ اور کیریئر کونسلنگ۔ اس سے طلبہ اپنی رفتار سے سیکھ سکتے ہیں اور کمزور طلبہ کی نشاندہی ہوتی ہے۔ لیکن دیہی علاقوں میں انٹرنیٹ کی کمی، کمپیوٹرز کی قلت، اساتذہ کی تربیت اور ڈیٹا کی حفاظت بڑے چیلنج ہیں۔

#### CHECK YOURSELF

1. Give three practical uses of AI in education.
2. State any two challenges of AI in education in Pakistan.

Answers: 1. LMS, automated grading and digital testing, AI-based career counselling. 2. Any two: limited rural internet, lack of devices, need for teacher training, data privacy and security.

## 5.2.2 Career Connection in IoT and AI

#### LEARN IT

Learning computing skills opens doors to many employers in Pakistan: **software houses** (e.g. in Karachi, Lahore and Islamabad technology parks), **telecom companies** (Jazz, Telenor), **banks**, and **government IT departments** (e.g. NADRA, provincial IT boards).

- **IoT Engineer:** designs smart systems and connected devices (e.g. a smart irrigation kit).
- **Data Analyst:** analyses the data generated by IoT systems and produces reports.

#### Key skills needed:

- **Programming** — Python, C++.
- **Networking** — Wi-Fi, protocols (rules for how devices communicate).
- **Electronics** — sensors and circuits.
- **Problem-solving.**

#### REMEMBER IT

Skills = "**PNEP**": **P**rogramming, **N**etworking, **E**lectronics, **P**roblem-solving. You start building the first one this year when you learn Python!

#### اردو میں سمجھیں

کمپیوٹنگ کی مہارتیں سیکھ کر آپ سافٹ ویئر ہاؤسز، ٹیلی کام کمپنیوں (جیسے جاز، ٹیلی نا)، بینکوں اور سرکاری آئی ٹی محکموں میں کام کر سکتے ہیں۔ IoT انجینئر اسمارٹ آلات بناتا ہے اور ڈیٹا اینالسٹ ان سے پیدا ہونے والے ڈیٹا کا تجزیہ کرتا ہے۔ اس کے لیے پروگرامنگ، نیٹ ورکنگ، الیکٹرانکس اور مسئلہ حل کرنے کی صلاحیت ضروری ہے۔

#### CHECK YOURSELF

1. What does an IoT Engineer do?
2. Name the four key skills for IoT/AI careers.

Answers: 1. Designs smart systems and connected devices. 2. Programming (Python, C++), networking, electronics, problem-solving.



## 5.3 Information Sources (Primary, Secondary, Tertiary) ★ Golden

### LEARN IT

**Information sources** are ways to find knowledge and data. The internet has vast information, but **not all of it is accurate**. Sources are divided into three main categories.

**1. Primary sources** give **original, firsthand information** straight from people, events or official records. They are **highly authentic** (genuine).

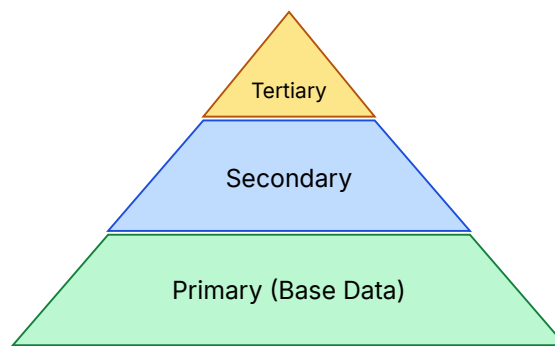
Examples: interviews, photographs, diaries, NADRA official records.

**2. Secondary sources** contain information that has **already been collected and explained by someone else**. They **summarise or analyse primary data**.

Examples: textbooks, news reports, research papers, Pakistan Bureau of Statistics reports.

**3. Tertiary sources** provide **highly summarised and organised information** derived from both primary and secondary sources. They are used for **quick reference**.

Examples: encyclopedias, Wikipedia, dictionaries, handbooks.



The information pyramid: primary data is the base; each higher level is built from the levels below it and is more summarised.

### Many examples, sorted

| Primary (firsthand)                         | Secondary (explained by others)                            | Tertiary (summarised, quick reference)  |
|---------------------------------------------|------------------------------------------------------------|-----------------------------------------|
| Interview with a flood victim in Dadu       | Newspaper report on the 2022 floods                        | Encyclopedia entry "Floods in Pakistan" |
| Photograph taken at an event                | Textbook chapter explaining the event                      | Wikipedia article                       |
| Personal diary or letter                    | Biography written by a historian                           | Dictionary                              |
| NADRA official records (CNIC data)          | Pakistan Bureau of Statistics report analysing census data | Handbook / almanac of facts             |
| Original survey responses                   | Research paper analysing the survey                        | Index or list of research papers        |
| Sensor readings from an IoT weather station | TV news analysis of the weather                            | A school "fact sheet" on climate        |
| Quaid-e-Azam's original speech recording    | Article discussing the speech                              | Encyclopedia summary of his life        |

### When do sources become unreliable?

In the digital age you must check facts carefully. A source is **unreliable** (cannot be trusted) if:

| Source type | Reasons for unreliability                                                                                |
|-------------|----------------------------------------------------------------------------------------------------------|
| Primary     | Information is incomplete, data is recorded incorrectly, or false information was deliberately provided. |
| Secondary   | The information is outdated, lacks proper references/citations, or has not been verified by an expert.   |
| Tertiary    | Comes from an unknown/unverified website, or the information is oversimplified and misses key facts.     |

#### REMEMBER IT

**Rule of thumb (Author – Date – References):** If an online article does not state the **author's name**, the **date of publication**, or list its **references**, it should not be trusted for academic work. Remember **"ADR"** — like checking a bus has a driver, a route and a ticket before you board.

#### WORKED EXAMPLE

**Classify these — worked solutions:**

**(a) Your grandfather's handwritten diary from 1947.** Written by a person who lived through the event → firsthand → **Primary**.

**(b) A Dawn newspaper article analysing the budget.** A journalist collected facts and explained them → **Secondary**.

**(c) An Oxford English–Urdu dictionary.** Organised, summarised reference built from many sources → **Tertiary**.

**(d) A Bureau of Statistics report on literacy in Sindh.** It analyses census (primary) data → **Secondary**.

**(e) A photo you took of a road accident.** Original record of the event → **Primary**.

Method: Ask (1) "Was the creator *there* / is it the original record?" → Primary. (2) "Is someone *explaining or analysing* original data?" → Secondary. (3) "Is it a *quick-reference summary* of many sources?" → Tertiary.

#### CHECK YOURSELF

**Exercise — classify these as Primary (P), Secondary (S) or Tertiary (T):**

1. A video interview with a cricketer after a match.
2. Your Computer Science textbook.
3. Wikipedia page on "Mohenjo-daro".
4. A birth certificate issued by the union council.
5. A research paper reviewing ten studies on smog in Lahore.
6. A handbook of physics formulas.
7. Original exam answer sheets of students.
8. A TV news report summarising a court decision.

Answers: 1. P 2. S 3. T 4. P 5. S 6. T 7. P 8. S

#### COMMON MISTAKE

**Mistake:** Thinking "primary" means "most important" or "tertiary" means "useless". **Fix:** The names describe *how far the information is from the original event*. Primary = straight from the source; tertiary = summarised summary. Each is useful for a different job. Also, Wikipedia is **tertiary** — not primary — because it summarises other sources.

#### اردو میں سمجھیں

معلومات کے ذرائع علم اور ڈیٹا تلاش کرنے کے طریقے ہیں، جنہیں تین اقسام میں تقسیم کیا جاتا ہے۔ پرائمری ذرائع براہ راست اور اصل معلومات دیتے ہیں (جیسے انٹرویو، تصاویر، نادرا کا ریکارڈ)۔ سیکنڈری ذرائع وہ ہیں جن میں کسی اور نے پرائمری معلومات کی وضاحت یا تجزیہ کیا ہو (جیسے درسی کتابیں، خبریں)۔ ٹریشیری ذرائع خلاصہ شدہ معلومات دیتے ہیں جیسے انسائیکلوپیڈیا، ویکی پیڈیا اور لغت۔ جس آن لائن مضمون میں مصنف کا نام، تاریخ یا حوالہ جات نہ ہوں اس پر بھروسہ نہ کریں۔

#### EXAM TIP

Typical questions: "Differentiate between primary, secondary and tertiary sources with two examples each" (long/short), "Classify the following sources..." (MCQ/short), and "When does a source become unreliable?"

Write the definition key phrase for each — *firsthand/original, collected and explained by someone else, highly summarised for quick reference* — plus two examples each. Draw the pyramid with primary at the base. For unreliability, give one reason for each type and quote the author–date–references rule.

## 5.4 Impacts of Computing in Various Fields

#### LEARN IT

Computing has drastically changed society. It makes life **faster and more connected**, but it also brings challenges like **privacy risks** and **technology addiction**. For every field, think of both sides.

| Field         | Positive impact (+)                                                                                                                 | Negative impact (–)                                                                                      |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Education     | Online classes, e-books and learning apps make studying highly flexible and accessible.                                             | Excessive screen time reduces attention spans and causes physical health issues (eye strain, back pain). |
| Communication | Emails and video calls allow instant, global communication at almost zero cost (e.g. a WhatsApp video call to a relative in Dubai). | People interact less in person, leading to social isolation and reduced face-to-face skills.             |
| Business      | Online shopping, digital banking and automation dramatically improve speed and efficiency.                                          | Traditional manual jobs are lost to machines; risk of severe cyber-attacks on banks.                     |
| Healthcare    | Digital patient records and telemedicine allow doctors to treat remote patients accurately.                                         | Patient data privacy is at risk if hospital servers are hacked.                                          |
| Agriculture   | Smart irrigation and weather forecasting massively increase crop yields.                                                            | Poor internet access in rural areas means many farmers cannot use these tools.                           |

## Smart and responsible use

- **Protect personal information** and use **strong passwords** (long, with letters, numbers and symbols; never share your OTP or CNIC details with callers).
- **Do not share fake news** or unverified WhatsApp forwards.
- **Balance screen time** with physical sports and activities.

### WORKED EXAMPLE

#### Scenario analysis — a bank in Karachi moves to mobile banking:

**Step 1 — Identify the field:** Business (banking).

**Step 2 — Positive:** Customers transfer money in seconds from home; no queue at the branch; the bank processes more transactions with fewer staff (speed and efficiency).

**Step 3 — Negative:** Some cashier jobs are no longer needed (job loss); hackers may attempt cyber-attacks or phishing SMS to steal accounts.

**Step 4 — Responsible use:** Customers should use strong passwords, never share OTPs, and report suspicious messages.

### REMEMBER IT

For the five fields, remember "**E-C-B-H-A**" — **E**ducation, **C**ommunication, **B**usiness, **H**ealthcare, **A**griculture — and always write one + and one – for each.

### COMMON MISTAKE

**Mistake:** Writing only positive impacts in a "discuss the impacts" question. **Fix:** "Impact" means *both* sides. Examiners expect a balanced answer: advantages, disadvantages, and how to use technology responsibly.

### اردو میں سمجھیں

کمپیوٹنگ نے معاشرے کو بہت بدل دیا ہے۔ تعلیم، مواصلات، کاروبار، صحت اور زراعت میں اس نے کام تیز اور آسان بنائے ہیں، لیکن اس کے ساتھ اسکرین کی لت، سماجی تنہائی، ملازمتوں کا خاتمہ، ہیکنگ اور پرائیویسی کے خطرات بھی پیدا ہوئے ہیں۔ ذمہ دارانہ استعمال کے لیے مضبوط پاس ورڈ رکھیں، جعلی خبریں نہ پھیلائیں، اور کھیل کود کے ساتھ اسکرین ٹائم میں توازن رکھیں۔

### CHECK YOURSELF

1. Give one positive and one negative impact of computing in healthcare.
2. Why can't many farmers in Pakistan benefit from smart agriculture tools?
3. State three rules of smart and responsible use.

Answers: 1. + Digital records and telemedicine help treat remote patients; – patient data privacy is at risk if servers are hacked. 2. Poor internet access in rural areas. 3. Protect personal information/use strong passwords; do not share fake news; balance screen time with physical activity.

## 5.5 Assistive Technologies ★ Golden

### LEARN IT

**Assistive technologies** are **devices, software and systems** that help people with **disabilities or functional limitations** (difficulty in seeing, hearing, moving, speaking, remembering, etc.). They allow more people to **participate equally** in school, work and society.

There are **four categories**:

| Category                            | Purpose                                                            | Examples                                                   |
|-------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------|
| 1. <b>Mobility aids</b>             | Help people move from one place to another.                        | Smart wheelchairs, stair lifts                             |
| 2. <b>Sensory aids</b>              | Support vision and hearing to improve interaction.                 | Hearing aids, screen readers (VoiceOver), Braille displays |
| 3. <b>Communication aids</b>        | Assist individuals with speech difficulties to express themselves. | Speech-generating devices, text-to-speech apps             |
| 4. <b>Cognitive / learning aids</b> | Help people with memory, reading and understanding.                | Audiobooks, spell-checkers, digital reminder apps          |

Mobility

Sensory

Communication

Cognitive

- A **screen reader** is software that reads aloud the text on a screen, so a blind person can use a phone or computer. **VoiceOver** is Apple's screen reader; Android has TalkBack.
- A **Braille display** is a device with small pins that rise and fall to form Braille dots, so a blind user can read screen text with their fingers.
- A **speech-generating device** speaks out words that a person types or selects on a screen.
- A **smart wheelchair** can be controlled by a joystick, voice or head movement and may avoid obstacles with sensors.

#### WORKED EXAMPLE

##### Scenario — choose the right assistive technology:

(a) Ahmed, a student in Hyderabad, is blind and wants to read his e-textbook. → **Sensory aid**: a screen reader or Braille display.

(b) Mrs Rashida cannot climb the stairs of her house after a knee operation. → **Mobility aid**: stair lift (or smart wheelchair).

(c) Zain cannot speak clearly due to a medical condition but can type. → **Communication aid**: text-to-speech app or speech-generating device.

(d) Nani often forgets to take her medicine. → **Cognitive aid**: digital reminder app.

(e) Usman has dyslexia (difficulty reading). → **Cognitive/learning aid**: audiobooks and spell-checkers.

Method: identify the *limitation* (moving, seeing/hearing, speaking, memory/reading) → choose the matching category.

#### REMEMBER IT

Four categories = "**My Small Cat Cries**": **Mobility**, **Sensory**, **Communication**, **Cognitive**. Link each to a body function: *legs* (mobility), *eyes/ears* (sensory), *mouth* (communication), *mind* (cognitive).

#### COMMON MISTAKE

**Mistake**: Putting text-to-speech and screen readers in the same category every time. **Fix**: In the source, the **screen reader** is a *sensory* aid (supports vision), while a **text-to-speech app** is listed as a *communication* aid (helps a person with speech difficulty express themselves). Decide by *whose problem* it solves.

#### اردو میں سمجھیں

معاون ٹیکنالوجیز (Assistive Technologies) وہ آلات، سافٹ ویئر اور نظام ہیں جو معذور افراد کی مدد کرتے ہیں تاکہ وہ اسکول، کام اور معاشرے میں برابر حصہ لے سکیں۔ ان کی چار اقسام ہیں: نقل و حرکت میں مدد (اسمارٹ ویبل چیئر)، حسی مدد (سماعتی آلہ، اسکرین ریڈر، بریل ڈسپلے)، گفتگو میں مدد (اسپیج جنریٹنگ ڈیوائس) اور ذہنی و تعلیمی مدد (آڈیو بکس، اسپیل چیکر، یاد دہانی ایپس)۔

#### EXAM TIP

Expect: "What are assistive technologies? Explain their categories with examples" (long), "Name two sensory aids" (short), or "A screen reader is an example of \_\_\_" (MCQ).

For full marks write the definition (devices, software and systems; help people with disabilities; equal participation), then all **four** categories each with purpose + two examples. Finish with one line on importance (equal access, independence, inclusion).

#### CHECK YOURSELF

1. Which category does a hearing aid belong to?
2. Give two examples of cognitive/learning aids.
3. Define assistive technologies in one sentence.

Answers: 1. Sensory aids. 2. Any two: audiobooks, spell-checkers, digital reminder apps. 3. Devices, software and systems that help people with disabilities or functional limitations participate equally in school, work and society.

## 5.5.1 Importance of Assistive Technologies

#### LEARN IT

Assistive technologies are important for three main reasons:

1. **Equal access to education:** Screen readers allow blind students to read digital textbooks equally with their peers (classmates).
2. **Independence in daily life:** Mobility aids reduce the need for constant human assistance — a person in a smart wheelchair can go to the market without someone pushing.
3. **Social inclusion:** They break down barriers, allowing individuals to work, study and socialise confidently.

Equal Access → Independence → Inclusion

#### WORKED EXAMPLE

**How one tool changes a life:** A visually impaired student in Karachi uses a screen reader on her laptop. **Equal access** — she reads the same e-books as her class. **Independence** — she no longer needs someone to read to her. **Inclusion** — she joins the class WhatsApp group, takes online tests and later gets a job, contributing to society.

#### REMEMBER IT

Remember **"All"**: Access, Independence, Inclusion.

#### اردو میں سمجھیں

معاون ٹیکنالوجیز کی اہمیت تین باتوں میں ہے: تعلیم تک برابر رسائی (ناہینا طلبہ اسکرین ریڈر سے کتابیں پڑھ سکتے ہیں)، روزمرہ زندگی میں خود مختاری (کسی کی مسلسل مدد کی ضرورت کم ہوتی ہے)، اور سماجی شمولیت (لوگ اعتماد کے ساتھ پڑھ، کام اور میل جول کر سکتے ہیں)۔

#### CHECK YOURSELF

1. State three reasons why assistive technologies are important.
2. How do mobility aids support independence?

Answers: 1. Equal access to education, independence in daily life, social inclusion. 2. They reduce the need for constant human assistance.

## 5.5.2 Career Connection in Assistive Tech

### LEARN IT

Designing assistive technology is a **highly rewarding career path** — you earn a living while improving people's lives.

| Career                              | What they do                                                                                                                                 |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Assistive Software Developer</b> | Creates tools such as speech-to-text software (e.g. Urdu voice typing for people who cannot use a keyboard).                                 |
| <b>Biomedical Engineer</b>          | Designs artificial limbs and hearing aids.                                                                                                   |
| <b>UX/UI Accessibility Expert</b>   | Ensures websites are easy to navigate for visually impaired users (e.g. adding text descriptions to images so screen readers can read them). |

**UX** means *user experience* (how easy and pleasant a product is to use); **UI** means *user interface* (the screens and buttons you interact with). **Accessibility** means designing so that *everyone*, including people with disabilities, can use it.

### اردو میں سمجھیں

معاون ٹیکنالوجی بنانا ایک قابلِ قدر پیشہ ہے۔ اسسٹو سافٹ ویئر ڈویلپر اسپیشل ٹو ٹیکسٹ جیسے ٹولز بناتا ہے، ہائیو میڈیکل انجینئر مصنوعی اعضاء اور سماعتی آلات تیار کرتا ہے، اور UX/UI ایکسیسبیلٹی ماہر یقینی بناتا ہے کہ ویب سائٹس نابینا افراد کے لیے بھی آسانی سے استعمال ہو سکیں۔

### CHECK YOURSELF

1. Who designs artificial limbs and hearing aids?
2. What does a UX/UI accessibility expert ensure?

Answers: 1. A biomedical engineer. 2. That websites are easy to navigate for visually impaired users.

## Chapter 5 Review

## Chapter summary

- Computing is vital in Pakistan's schools, hospitals, banks, farms and industries; it has both positive and negative impacts.
- **AI** is "artificial" (made by humans) and "intelligent" (does tasks needing thinking); it uses data to learn, recognise patterns and make decisions.
- In smart systems, IoT is the "eyes" that gather data, Analytics sorts it, and AI is the "brain" that decides the action.
- **IoT** is a system of physical devices connected to the internet that collect and share data to perform tasks automatically.
- The six IoT components are sensors (collect), actuators (act), connectivity, data processing, cloud computing and user interface.
- **Data Analytics** is a pipeline: collection → storage → cleaning → analysis → visualization.
- Data cleaning removes duplicates, incorrect values, missing records and improper formats.
- IoT outputs raw data (hardware); AI outputs smart decisions (software); Data Analytics outputs reports, graphs and insights (data).
- IoT is used in smart cities, industrial automation, healthcare, education, home automation and smart farming.
- Smart farming (soil sensor → cloud AI → water pump) saves water and improves yields in Sindh and Punjab.
- AI in Pakistani education: LMS, automated grading, career counselling; challenges are rural internet, lack of devices, teacher training and data privacy.
- IoT/AI careers (IoT engineer, data analyst) need programming, networking, electronics and problem-solving.
- Information sources: primary (firsthand/original), secondary (collected and explained by others), tertiary (summarised for quick reference).
- Do not trust an online article that lacks an author, a publication date or references.
- Computing helps education, communication, business, healthcare and agriculture but risks screen addiction, isolation, job loss, cyber-attacks and privacy loss.
- **Assistive technologies** — mobility, sensory, communication and cognitive aids — give equal access, independence and social inclusion to people with disabilities.



## Key terms

| Term                             | Meaning                                                                                       | اردو                               |
|----------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------|
| Computing                        | Using computers and digital devices to store, process and share information.                  | کمپیوٹنگ                           |
| Artificial Intelligence (AI)     | Human-made technology that lets machines think, learn and decide using data.                  | مصنوعی ذہانت                       |
| Internet of Things (IoT)         | Physical devices connected to the internet that collect and share data to work automatically. | انٹرنیٹ آف تھنگز، اشیاء کا انٹرنیٹ |
| Sensor                           | Device that collects information from the physical world.                                     | حساس آلہ (سینسر)                   |
| Actuator                         | Device that performs a physical action in response to a command.                              | عمل کار آلہ (ایکچوٹر)              |
| Connectivity                     | The link (Wi-Fi, Bluetooth, ZigBee, 4G/5G) that lets devices send and receive data.           | رابطہ کاری                         |
| Data processing                  | Changing raw data into useful information.                                                    | ڈیٹا کی پروسیسنگ                   |
| Cloud computing                  | Storing data and using services over the internet instead of local drives.                    | کلاؤڈ کمپیوٹنگ                     |
| User interface (UI)              | The part that lets a user see information and give commands to a device.                      | صارف انٹرفیس                       |
| Data Analytics                   | Collecting, storing, cleaning, analysing and presenting data to find useful information.      | ڈیٹا کا تجزیہ                      |
| Data cleaning                    | Finding and correcting errors such as duplicates and missing values.                          | ڈیٹا کی صفائی                      |
| Data visualization               | Showing data as charts, plots or graphs.                                                      | ڈیٹا کی بصری پیشکش                 |
| Smart city                       | City using IoT for traffic, lighting, parking and waste.                                      | اسمارٹ شہر                         |
| Smart farming                    | Using sensors, automated irrigation and drones in agriculture.                                | اسمارٹ کاشتکاری                    |
| Learning Management System (LMS) | Online platform for lessons, assignments and tests.                                           | آن لائن تعلیمی نظام                |
| Telemedicine                     | Treating patients remotely using computing and communication.                                 | دور دراز علاج                      |
| Primary source                   | Original, firsthand information.                                                              | بنیادی ذریعہ                       |

| Term                 | Meaning                                                                       | اردو              |
|----------------------|-------------------------------------------------------------------------------|-------------------|
| Secondary source     | Information collected and explained by someone else.                          | ثانوی ذریعہ       |
| Tertiary source      | Highly summarised information for quick reference.                            | ثالثی ذریعہ       |
| Reliability          | How far a source can be trusted.                                              | قابلِ اعتماد ہونا |
| Privacy              | Keeping personal information safe from others.                                | رازداری           |
| Cyber-attack         | An attempt by hackers to damage or steal from computer systems.               | سائبر حملہ        |
| Assistive technology | Devices, software and systems that help people with disabilities.             | معاون ٹیکنالوجی   |
| Screen reader        | Software that reads screen text aloud for blind users.                        | اسکرین ریڈر       |
| Braille display      | Device with raised pins showing text as Braille dots.                         | بریل ڈسپلے        |
| Accessibility        | Designing so everyone, including people with disabilities, can use a product. | رسائی پذیری       |

## Practice: multiple choice

1. AI is called "artificial" because it:
  - a. runs only on electricity
  - b. is created by humans
  - c. is always wrong
  - d. works without data
2. Which of the following is an actuator?
  - a. Temperature sensor
  - b. Motion sensor
  - c. Water valve
  - d. Wi-Fi router
3. ZigBee is an example of which IoT component?
  - a. Connectivity
  - b. Sensor
  - c. User interface
  - d. Actuator
4. Google Drive and iCloud are examples of:
  - a. Sensors
  - b. Data cleaning
  - c. Actuators
  - d. Cloud computing
5. Removing duplicate student records belongs to which component of Data Analytics?
  - a. Data collection
  - b. Data storage
  - c. Data cleaning
  - d. Data visualization
6. A pie chart showing pass vs fail percentage is an example of:
  - a. Data collection
  - b. Data storage
  - c. Data cleaning
  - d. Data visualization
7. According to the comparison table, the output of IoT is:
  - a. Raw data from the environment
  - b. Smart decisions
  - c. Reports and graphs
  - d. Printed documents
8. In a smart system, AI acts as the:
  - a. eyes
  - b. brain
  - c. hands
  - d. wires
9. NADRA's official records are a:
  - a. tertiary source
  - b. secondary source
  - c. primary source
  - d. unreliable source
10. Wikipedia is classified as a:
  - a. primary source
  - b. tertiary source
  - c. secondary source
  - d. firsthand record
11. Smart bins that send alerts when full are a use of IoT in:
  - a. smart farming
  - b. healthcare
  - c. home automation
  - d. smart cities
12. A system in which a soil sensor tells the cloud AI to switch on a water pump is used in:
  - a. smart farming
  - b. industrial automation
  - c. smart classrooms
  - d. traffic management
13. A screen reader such as VoiceOver is a:
  - a. mobility aid
  - b. communication aid
  - c. sensory aid
  - d. cognitive aid
14. A speech-generating device is a:
  - a. mobility aid
  - b. cognitive aid
  - c. sensory aid
  - d. communication aid
15. An online article should NOT be trusted for academic work if it lacks:
  - a. colourful pictures
  - b. the author's name, publication date or references
  - c. a long title
  - d. advertisements

### Practice: short questions

1. Define Artificial Intelligence and give two examples from daily life.
2. Define the Internet of Things (IoT) with one example.
3. Differentiate between a sensor and an actuator with two examples of each.
4. What is the role of cloud computing in an IoT system?
5. List the five components of Data Analytics. What kinds of errors are removed in data cleaning?
6. Give two uses of IoT in smart cities and two in healthcare.
7. State two benefits and two challenges of AI in education in Pakistan.
8. Differentiate between primary and secondary sources with two examples each.
9. When does a secondary source become unreliable?
10. Give one positive and one negative impact of computing on (a) communication and (b) business.

### Practice: long / problem questions

1. Define IoT. Explain its six components with an example of each. Draw a diagram showing how they connect.
2. Compare IoT, AI and Data Analytics in a table (definition, purpose, type of work, output). Then explain, with an example, how the three work together in a smart system.
3. A college in Mirpurkhas wants to find out why many students are failing Mathematics. Explain how it would use each of the five components of Data Analytics to solve this problem.
4. Explain the three types of information sources with examples. Classify the following: (i) a school textbook, (ii) a photograph of an event, (iii) an encyclopedia, (iv) an interview, (v) a Bureau of Statistics report, (vi) a dictionary. When does each type become unreliable?
5. What are assistive technologies? Explain their four categories with examples, state their importance, and name two related careers.
6. "Computing has both positive and negative impacts on society." Discuss with reference to education, healthcare and agriculture, and suggest three rules for smart and responsible use.

## Answer key

---

### Multiple choice

1. b
2. c
3. a
4. d
5. c
6. d
7. a
8. b
9. c
10. b
11. d
12. a
13. c
14. d
15. b

## Short questions — model answers

1. AI is human-made ("artificial") technology that can perform tasks requiring thinking ("intelligence"); it uses data to make decisions. Examples: face recognition to unlock a phone; YouTube suggesting videos; chatbots answering questions.
2. IoT is a system in which physical devices are connected to the internet and can collect and share data to perform tasks automatically. Example: a smartwatch collects heart-rate data and sends it to a mobile phone for display.
3. A sensor collects information from the physical world (input; the "eyes and ears"), e.g. temperature sensor, motion sensor. An actuator performs a physical action in response to a command, converting digital signals into movement (output), e.g. electric motor, automatic door opener.
4. Cloud computing stores IoT data and provides processing services over the internet instead of local hard drives, so it can handle the massive amounts of data produced by many devices. Examples: Google Drive, iCloud, online LMS.
5. Collection, storage, cleaning, analysis, visualization. Cleaning removes duplicated records, incorrect values (typos), missing files/records, and improper formats.
6. Smart cities: sensors adjust traffic signals to reduce congestion; smart bins send alerts when full (also street lighting, parking). Healthcare: smartwatches measure heart rate and vitals; remote monitoring devices send patient data to doctors (also emergency alerts).
7. Benefits: students learn at their own speed; AI identifies students who need extra help (also supports students with disabilities). Challenges: limited internet access in rural areas; lack of computers/devices (also teacher training, data privacy).
8. Primary sources give original, firsthand information from people, events or official records (e.g. interviews, NADRA records). Secondary sources contain information already collected and explained by someone else, summarising or analysing primary data (e.g. textbooks, news reports).
9. When the information is outdated, lacks proper references/citations, or has not been verified by an expert.
10. (a) Communication: + emails and video calls give instant, global communication at almost no cost; – less in-person interaction causes social isolation. (b) Business: + online shopping, digital banking and automation improve speed and efficiency; – manual jobs are lost to machines and banks face cyber-attacks.

## Long questions — model answers

1. **Definition:** IoT is a system in which physical devices are connected to the internet and collect and share data to perform tasks automatically. **Components:** (1) Sensors — collect information from the physical world, e.g. temperature, motion, fingerprint, smoke, soil moisture sensors. (2) Actuators — perform physical actions in response to commands, converting digital signals to movement, e.g. motor, switch, door opener, water valve, fan controller. (3) Connectivity — lets devices send/receive data, e.g. Wi-Fi, Bluetooth, ZigBee, 4G/5G. (4) Data processing — changes raw data into useful information through collecting, organising, analysing and deciding, e.g. checking if a room is too hot to switch on the AC. (5) Cloud computing — stores data and provides services over the internet, handles massive data, e.g. Google Drive, iCloud, LMS. (6) User interface — lets users see information and give commands, e.g. mobile apps, touch screens, voice commands. **Diagram:** Sensor (collects) → Wi-Fi → Cloud (processes) → Actuator (acts) → User App (UI).
2. **Table:** IoT — network of physical devices connected to the internet; purpose: data collection and automation; works with hardware; output: raw data from environment. AI — technology enabling machines to think and decide; purpose: intelligent decisions and predictions; works with smart software; output: smart decisions or actions. Data Analytics — process of examining data to find patterns; purpose: extracting meaningful insights; works with data (numbers, records); output: reports, graphs, insights. **Working together:** IoT is the eyes that gather data, Analytics sorts it, AI is the brain that decides. Example: a temperature sensor reads 36 °C (IoT) → recent readings are organised showing the room is heating up (Analytics) → AI decides the room is too hot (AI) → a switch turns on the AC (action).
3. (1) **Collection:** gather marks of all Maths tests, attendance and survey responses from students. (2) **Storage:** keep them safely in a spreadsheet or database, e.g. on the cloud. (3) **Cleaning:** remove duplicate records, correct typing errors (e.g. 120 out of 100), fill or mark missing records, and make formats (dates, roll numbers) consistent. (4) **Analysis:** study the clean data to find patterns — e.g. which chapters most students got wrong, whether low attendance links to failing. (5) **Visualization:** present results in bar charts or a pie chart of pass vs fail so the principal sees the problem at a glance and plans extra classes for weak chapters.
4. **Primary:** original, firsthand, highly authentic (interviews, photographs, diaries, NADRA records). **Secondary:** collected and explained by someone else, summarises/analyses primary data (textbooks, news reports, research papers, Bureau of Statistics reports). **Tertiary:** highly summarised and organised from primary and secondary sources for quick reference (encyclopedias, Wikipedia, dictionaries, handbooks). **Classification:** (i) secondary, (ii) primary, (iii) tertiary, (iv) primary, (v) secondary, (vi) tertiary. **Unreliability:** primary — incomplete, recorded incorrectly, or deliberately false; secondary — outdated, no references/citations, not verified by an expert; tertiary — unknown/unverified website, or oversimplified and missing key facts. Rule: do not trust an article without author, date or references.
5. **Definition:** devices, software and systems that help people with disabilities or functional limitations participate equally in school, work and society. **Categories:** (1) Mobility aids — help people move; smart wheelchairs, stair lifts. (2) Sensory aids — support vision and hearing; hearing aids, screen readers (VoiceOver), Braille displays. (3) Communication aids — help people with speech difficulties express themselves; speech-generating devices, text-to-speech apps. (4) Cognitive/learning aids — help with memory, reading, understanding; audiobooks, spell-checkers, reminder apps. **Importance:** equal access to education (screen readers let blind students read digital textbooks), independence in daily life (mobility aids reduce need for constant help), social inclusion (people work, study and socialise confidently). **Careers:** any two of assistive software developer (speech-to-text tools), biomedical engineer (artificial limbs, hearing aids), UX/UI accessibility expert (websites usable by visually impaired users).
6. **Introduction:** computing makes life faster and more connected but brings privacy risks and addiction. **Education:** + online classes, e-books and learning apps make study flexible and accessible; – excessive screen time reduces attention span and harms physical health. **Healthcare:** + digital patient records and telemedicine let doctors treat remote patients accurately (e.g. rural Sindh); – patient data privacy is at risk if hospital servers are hacked. **Agriculture:** + smart irrigation and weather forecasting greatly increase yields; – poor rural internet means many farmers cannot use these tools. **Responsible use:** protect personal information and use strong passwords; do not share fake news or unverified forwards; balance screen time with sports and physical activity. **Conclusion:** benefits are large if technology is used wisely.

## Self-assessment: I can...

- ☐ define AI and explain why it is called "artificial intelligence".
- ☐ define IoT and name its six components with examples.
- ☐ list the five components of Data Analytics in order and describe data cleaning.
- ☐ describe IoT uses in smart cities, industry, healthcare, education, homes and farms.
- ☐ classify any source as primary, secondary or tertiary and judge its reliability.
- ☐ explain the four categories of assistive technologies and their importance.
- ☐ explain how IoT, Data Analytics and AI work together (eyes → sort → brain → action).
- ☐ tell the difference between a sensor and an actuator.
- ☐ compare IoT, AI and Data Analytics in a table.
- ☐ state uses, benefits and challenges of AI in education in Pakistan.
- ☐ discuss positive and negative impacts of computing in five fields.
- ☐ name careers linked to IoT, AI and assistive technology and the skills they need.

## CHAPTER 6

# Digital Literacy

This chapter teaches you how to find, collect, organise and present information using digital tools, safely and responsibly. You will learn about types of data, ways to collect it, and how to turn it into a useful report, presentation or infographic. At the end you will follow a complete research project from the first question to the final product.

اس باب میں آپ سیکھیں گے کہ ڈیجیٹل آلات کی مدد سے معلومات کو محفوظ اور ذمہ دارانہ طریقے سے کیسے تلاش کیا جائے، جمع کیا جائے، ترتیب دیا جائے اور پیش کیا جائے۔ آخر میں آپ ایک مکمل تحقیقی منصوبہ (Digital Inquiry Project) شروع سے آخر تک دیکھیں گے۔

## What you will learn

- Define digital literacy and explain why it matters in the Information Age.
- Describe five data-collection strategies: interviews, surveys, prototypes, observation and simulations.
- Compare primary and secondary data by source, examples, purpose and control.
- Use spreadsheet formulas such as `=SUM()`, `=AVERAGE()` and `=COUNTIF()` to analyse data.
- Use advanced search operators ( " " , `filetype:` , `site:` ) and evaluate the reliability of a source.
- Distinguish between qualitative and quantitative data and give examples of each.
- Write clear, unbiased survey questions and recognise badly written ones.
- List the eight steps for designing a data-collection approach, including ethics and consent.
- Choose a suitable digital tool (spreadsheet, presentation, infographic, report, dashboard) to present findings.
- Carry out all 12 steps of a digital inquiry project and produce a final digital artefact.

## 6.1 Introduction to Digital Literacy

### LEARN IT

**Digital literacy** is the ability to understand and use digital devices, the internet, and online tools in a **safe, effective,** and **responsible** manner.

Look at the three key words:

- **Safe** – you protect yourself and others online (for example, you do not share passwords or private details).
- **Effective** – you get the job done well (for example, you find the right information quickly).
- **Responsible** – you use information honestly and fairly (for example, you do not copy someone else's work or spread false news).

In our increasingly digital world, digital literacy is essential for **everyone**, not only for computer experts. It enables you to:

- **find** information,
- **communicate** with others,
- **produce** digital content, and
- **solve problems** through technology.

**Did you know?** Digital literacy is not only *using devices*; it is *using digital information safely and wisely*. A digitally literate student can:

Locate information → Organise it properly → Turn it into something useful → Share it responsibly

### WORKED EXAMPLE

**Is this student digitally literate?**

Ali can type fast and plays games on his phone all day. When his teacher asks him to find the population of Karachi, he copies the first number he sees on a random website and does not check where it came from.

**Step 1 – Can he use devices?** Yes. He has basic skills.

**Step 2 – Can he locate and judge information?** No. He did not check whether the website was reliable.

**Step 3 – Conclusion:** Ali has *computer skills* but is not yet fully *digitally literate*. A digitally literate student would compare two or three trusted sources (for example, the Pakistan Bureau of Statistics website) before using the number.

### REMEMBER IT

Remember the four verbs of a digitally literate student: **Locate – Organise – Create – Share** ("**LOCS**": think of a *lock* that keeps your digital life safe).

### COMMON MISTAKE

**Mistake:** "Digital literacy means knowing how to use a computer." **Fix:** Using a computer is only the start. Digital literacy also means finding, judging, creating and sharing information safely and responsibly.

### اردو میں سمجھیں

ڈیجیٹل خواندگی (Digital Literacy) سے مراد ڈیجیٹل آلات، انٹرنیٹ اور آن لائن ٹولز کو محفوظ، مؤثر اور ذمہ دارانہ طریقے سے سمجھنا اور استعمال کرنا ہے۔ ایک ڈیجیٹل خواندہ طالب علم معلومات تلاش کر سکتا ہے، انہیں ترتیب دے سکتا ہے، ان سے کوئی مفید چیز بنا سکتا ہے اور انہیں ذمہ داری سے دوسروں تک پہنچا سکتا ہے۔



### CHECK YOURSELF

1. Write the definition of digital literacy in one sentence.
2. Name the four things digital literacy enables you to do.

Answers: 1. The ability to understand and use digital devices, the internet and online tools in a safe, effective and responsible manner.  
2. Find information, communicate, produce digital content, and solve problems through technology.

## 6.2 Digital Literacy in the Information Age

### LEARN IT

The **Information Age** is the period we live in now, where vast amounts of data are **produced**, **shared** and **accessed instantly** through the internet, mobile devices and digital platforms (websites, apps, social media).

Simply **having access** to technology is **not enough**. Owning a smartphone does not make you good at using information. You must develop the skills to **understand**, **manage** and **use** information wisely.

Digital literacy goes beyond basic computer skills. It involves three kinds of ability:

| Ability              | What it means                                       | Example                                                     |
|----------------------|-----------------------------------------------------|-------------------------------------------------------------|
| Technical            | Knowing how to operate devices and software.        | Creating a Google Form; using a spreadsheet.                |
| Cognitive (thinking) | Thinking critically about information.              | Checking whether a news story is true before believing it.  |
| Social               | Communicating and behaving well with others online. | Being polite in a class WhatsApp group; respecting privacy. |

Together these abilities help you function successfully in a **digital society**.

### WORKED EXAMPLE

**Applying the three abilities.** A video claims "Drinking hot water cures dengue." What does a digitally literate student do?

**Technical:** Uses a search engine to search `dengue treatment site:.gov`.

**Cognitive:** Notices the video has no doctor or source, and that official health websites do not support the claim. Concludes it is false.

**Social:** Does not forward the video, and politely tells the family group that it is not reliable.

### REMEMBER IT

Three abilities = **"Tech, Think, Talk"**: Technical, Cognitive (think), Social (talk with others).

### COMMON MISTAKE

**Mistake:** Thinking that "more information" automatically means "better information". **Fix:** In the Information Age the problem is not finding *some* information; it is choosing the *right, reliable* information from too much.

### اردو میں سمجھیں

معلوماتی دور (Information Age) میں انٹرنیٹ اور موبائل کے ذریعے بے پناہ معلومات فوراً تیار اور شیئر ہوتی ہیں۔ صرف ٹیکنالوجی تک رسائی کافی نہیں؛ معلومات کو سمجھنے، سنبھالنے اور دانشمندی سے استعمال کرنے کی مہارت ضروری ہے۔ ڈیجیٹل خواندگی میں تکنیکی، ذہنی (Cognitive) اور سماجی صلاحیتیں شامل ہیں۔

### CHECK YOURSELF

1. What is the Information Age?
2. Name the three types of ability that digital literacy involves.

Answers: 1. A period in which vast amounts of data are produced, shared and accessed instantly through the internet, mobile devices and digital platforms. 2. Technical, cognitive and social abilities.

## 6.3 Types of Data ★ Golden

### LEARN IT

**Data** means facts, numbers, words or observations that we collect. The data gathered through a survey, an investigation or a classroom project can be described in two ways: **qualitatively** or **quantitatively**. Knowing the difference is critical, because each type is analysed in a different way.

### 6.3.1 Qualitative Data

**Qualitative data** describes what people **think, feel, notice or experience**. It is **non-numerical** and **descriptive**: it is made of words, not numbers. The word comes from *quality* (what something is like).

**Examples:** students' opinions about online classes; descriptions of study habits; observation notes ("The students looked bored after 20 minutes").

### 6.3.2 Quantitative Data

**Quantitative data** deals with **numbers**. The word comes from *quantity* (how much or how many). It is useful when you need to **count, measure, compare, calculate averages**, or **display results in charts**.

**Examples:** number of students using the internet daily; marks obtained in a test; time spent online in hours.

### 6.3.3 Comparing Qualitative and Quantitative Data

| Type of data        | Nature                             | Examples                           | Best used for                                             |
|---------------------|------------------------------------|------------------------------------|-----------------------------------------------------------|
| <b>Qualitative</b>  | Descriptive, non-numerical         | Opinions, reasons, behaviour       | Understanding <b>why</b> and <b>how</b> something happens |
| <b>Quantitative</b> | Numerical, countable or measurable | Hours, scores, percentages, totals | <b>Comparing</b> results and making <b>graphs/charts</b>  |

### LEARN IT

**Going a little further (background).** Quantitative data is sometimes split into two kinds:

- **Discrete** data can only take separate, countable values, e.g. number of students in a class (35, 36 – never 35.5).
- **Continuous** data can take any value in a range and is *measured*, e.g. height (162.4 cm) or time online (2.75 hours).

Also note: a question with fixed categories (e.g. "Which activity do you use your phone for most? Lectures / Notes / Social media / Games") gives **categories** that you can **count**. In this chapter's case study such answers are treated as **quantitative (categories)**, because you count how many students chose each option.

### WORKED EXAMPLE

Classify each piece of data.

| Data                                                    | Type         | Reason                              |
|---------------------------------------------------------|--------------|-------------------------------------|
| "I like online classes because I can replay lectures."  | Qualitative  | An opinion with a reason, in words. |
| Ayesha scored 78 out of 100.                            | Quantitative | A score – a number you can compare. |
| 28 out of 40 students use the internet daily.           | Quantitative | A count (and gives 70%).            |
| Observer notes: "Students chat during the last period." | Qualitative  | A description of behaviour.         |
| Bilal spends 3.5 hours online per day.                  | Quantitative | A measurement of time.              |

**Quick test:** ask "Can I calculate an average of this?" If yes, it is quantitative. If the answer is words, it is qualitative.

### REMEMBER IT

**QUAL**itative = **QUAL**ity = words and feelings. **QUANT**itative = **QUANT**ity = numbers and counts. Qualitative answers **why/how**; quantitative answers **how many/how much**.

### COMMON MISTAKE

**Mistake:** Thinking that any data containing a number is quantitative. A phone number or a roll number is a *label*, not a quantity – averaging roll numbers is meaningless. **Fix:** Data is quantitative only if the number measures or counts something.

### اردو میں سمجھیں

کیفیتی ڈیٹا (Qualitative Data) الفاظ پر مشتمل ہوتا ہے اور بتاتا ہے کہ لوگ کیا سوچتے، محسوس کرتے یا تجربہ کرتے ہیں، جیسے آن لائن کلاسز کے بارے میں رائے۔ مقداری ڈیٹا (Quantitative Data) اعداد پر مشتمل ہوتا ہے، جیسے ٹیسٹ کے نمبر یا آن لائن گزاریے گئے گھنٹے۔ کیفیتی ڈیٹا "کیوں اور کیسے" سمجھنے کے لیے، جبکہ مقداری ڈیٹا موازنہ کرنے اور چارٹ بنانے کے لیے بہترین ہے۔

### EXAM TIP

Common questions: "Differentiate between qualitative and quantitative data" (4–5 marks) or "Classify the following data". Write the answer as a **table** with four rows: **nature, examples, best used for**, plus a one-line definition of each. Always give **two examples** of each type. The source's table (Nature / Examples / Best used for) is exactly what examiners expect.

### CHECK YOURSELF

1. Is "time spent online in hours" qualitative or quantitative?
2. Which type of data is best for understanding *why* something happens?
3. Classify: "The canteen food is too oily."

Answers: 1. Quantitative (measurable number). 2. Qualitative. 3. Qualitative (an opinion in words).

## 6.4 Data-Collection Strategies ★ Golden

### LEARN IT

Digital literacy is not limited to finding information that **already exists** on the internet. You should also learn how to gather **original** (new) information yourself. A **data-collection strategy** is a planned method for gathering this information.

The five common strategies are:

Interviews

• Surveys

• Prototypes

• Observation

• Simulations

| Strategy    | How it works (one line)                               | Usual data type                              | Best when...                                      |
|-------------|-------------------------------------------------------|----------------------------------------------|---------------------------------------------------|
| Interview   | Planned conversation with a person or small group     | Mostly qualitative                           | You need deep, detailed answers from a few people |
| Survey      | Same questions asked to many people                   | Quantitative and qualitative                 | You need answers from many people quickly         |
| Prototype   | Users test an early version and give feedback         | Both                                         | You want to improve a product or solution         |
| Observation | Watch and record; no questions asked                  | Notes (qualitative) or counts (quantitative) | You want to see what people actually do           |
| Simulation  | Computer model; change variables and see what happens | Mostly quantitative                          | Real experiments are costly, risky or difficult   |

### REMEMBER IT

Mnemonic for the five strategies: "**I S P O S**" – "**I** Saw **P**eople **O**n **S**creens": Interviews, Surveys, Prototypes, Observation, Simulations.

### اردو میں سمجھیں

ڈیجیٹل خواندگی صرف موجود معلومات تلاش کرنے تک محدود نہیں، بلکہ نئی معلومات خود جمع کرنا بھی اس کا حصہ ہے۔ ڈیٹا جمع کرنے کے پانچ عام طریقے یہ ہیں: انٹرویو، سروے، پروٹوٹائپ، مشاہدہ (Observation) اور سمیولیشن (Simulation)۔

### EXAM TIP

"List and explain the data-collection strategies" is a frequent long question. Name all **five**, give a one-sentence definition and **one example** for each, and say what type of data each gives. Detailed notes on each follow in the next two topics.

### CHECK YOURSELF

1. Name the five data-collection strategies.

2. What does "original information" mean?

Answers: 1. Interviews, surveys, prototypes, observation, simulations. 2. New information that you gather yourself, rather than information that already exists.

## 6.4.1 – 6.4.2 Interviews and Surveys ★ Golden

### 6.4.1 Interviews

#### LEARN IT

An **interview** is a **planned conversation** used to gather information **directly from a person**.

- It may be **one-to-one** (you and one person) or in a **small group**.
- Questions are **planned in advance**, so you do not forget anything important.
- Answers are usually **qualitative** (opinions, reasons, experiences).

**Strength:** you get deep, detailed answers and can ask follow-up questions ("Why do you say that?"). **Weakness:** it takes a lot of time, so you can only interview a few people.

### 6.4.2 Surveys / Questionnaires

#### LEARN IT

A **survey** (or **questionnaire**) is useful when the **same set of questions** needs to be asked to **many people**.

- It may be on **paper** or created with **digital tools** such as **Google Forms** or **Microsoft Forms**.
- **Note:** online survey tools **automatically collect responses** and **display basic charts**, which makes data analysis much easier.

#### Types of survey and interview questions

| Question type                  | What it looks like                                                                       | Data it gives                              |
|--------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------|
| <b>Closed: Yes/No</b>          | Do you study with your phone nearby? (Yes / No)                                          | Countable (quantitative)                   |
| <b>Closed: Multiple choice</b> | Which activity do you use your phone for most? (Lectures / Notes / Social media / Games) | Categories you can count                   |
| <b>Number / scale</b>          | How many hours do you use your phone daily? ____                                         | Quantitative                               |
| <b>Rating scale</b>            | "My phone helps me study." 1 = Strongly disagree ... 5 = Strongly agree                  | Quantitative (you can average the ratings) |
| <b>Open-ended</b>              | How does your phone help you in studies?                                                 | Qualitative (free text)                    |

#### Good questions vs bad questions

A good question is **clear, short, unbiased** (it does not push the person towards one answer) and asks about **one thing at a time**.

| Problem                                               | Bad question                                          | Better question                                                                                                 |
|-------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Leading / biased</b> – suggests the "right" answer | Don't you agree that phones waste students' time?     | How do phones affect your study time? (Helps a lot / Helps a little / No effect / Harms a little / Harms a lot) |
| <b>Double-barrelled</b> – two questions in one        | Do you use your phone for lectures and games?         | Split into two: "Do you use your phone for lectures?" and "Do you use your phone for games?"                    |
| <b>Vague</b> – unclear words                          | Do you use your phone a lot?                          | How many hours do you use your phone on a normal school day?                                                    |
| <b>Overlapping options</b>                            | 1–3 hrs / 3–5 hrs / 5+ hrs (where does exactly 3 go?) | Less than 3 hrs / 3 to less than 5 hrs / 5 hrs or more                                                          |
| <b>Sensitive / too personal</b>                       | What is your father's monthly income?                 | Leave it out unless it is truly needed for the study.                                                           |

### WORKED EXAMPLE

**Building a short Google Form, step by step.** Research question: "How do Class XI students travel to college?"

**Step 1:** Open Google Forms → Blank form → Title: "Travel to College Survey". Add a description: "This survey is anonymous and for a class project. Do you agree to take part? (Yes/No)" – this is your **consent** question.

**Step 2:** Q1 (multiple choice): "How do you usually travel to college?" Walk / Bus / Rickshaw / Motorbike / Car / Other.

**Step 3:** Q2 (number): "How many minutes does your journey take?"

**Step 4:** Q3 (open-ended): "What is the biggest problem you face while travelling?"

**Step 5:** Click *Send* and share the link in the class group. Responses appear in the *Responses* tab with automatic pie/bar charts, and can be opened in Google Sheets.

**Result:** Q1 and Q2 give quantitative data; Q3 gives qualitative data.

### REMEMBER IT

**Interview = In-depth with few. Survey = Same questions to many.** Think of a doctor talking to one patient (interview) vs a census form sent to every house (survey).

### COMMON MISTAKE

**Mistake:** Writing only open-ended questions in a survey for 100 people. You will get 100 paragraphs that are very hard to analyse. **Fix:** Use mostly closed or number questions in a large survey, and only one or two open-ended questions.

### اردو میں سمجھیں

انٹرویو ایک منصوبہ بند گفتگو ہے جس میں کسی ایک فرد یا چھوٹے گروہ سے براہ راست معلومات حاصل کی جاتی ہیں؛ اس کے جوابات عموماً کیفیتی ہوتے ہیں۔ سروے میں ایک ہی سوالات بہت سے لوگوں سے پوچھے جاتے ہیں، کاغذ پر یا گوگل فارمز (Google Forms) جیسے ڈیجیٹل ٹولز کے ذریعے، جو جوابات خودکار طور پر جمع کر کے چارٹ بھی دکھاتے ہیں۔ اچھا سوال واضح، مختصر، غیر جانبدار اور ایک وقت میں ایک ہی بات کے بارے میں ہوتا ہے۔

### EXAM TIP

You may be asked to "differentiate between an interview and a survey" or to "identify what is wrong with this survey question and rewrite it". For rewriting, name the problem (leading, double-barrelled, vague, overlapping) and give the corrected version.

### CHECK YOURSELF

1. Name two digital tools for creating surveys.
2. What is wrong with: "Do you enjoy maths and physics?"
3. Are interview answers usually qualitative or quantitative?

Answers: 1. Google Forms and Microsoft Forms. 2. It is double-barrelled (two subjects in one question); ask about each subject separately. 3. Usually qualitative.

## 6.4.3 – 6.4.5 Prototypes, Observation and Simulations ★ Golden

### 6.4.3 Prototypes

#### LEARN IT

A **prototype** is a **simple early version** of a product, model or solution. It allows an idea to be **tested before** time and effort are spent on the final version. Users **try** the prototype and give **feedback**, and this feedback becomes useful **data for improvement**.

**Example:** Before building a full school app, you draw the screens on paper (or make a few clickable screens in a design tool) and ask five classmates to try "finding the timetable". You note where they get confused.

### 6.4.4 Observation

#### LEARN IT

In **observation**, the researcher **watches** what people do or what happens in a situation and **records the results carefully**. **No direct questions are asked**.

The data may be **qualitative** (written notes, e.g. "students push in the queue") or **quantitative** (counts, e.g. "42 students bought samosas in 10 minutes").

**Why use it?** People sometimes say one thing and do another. Observation shows what really happens.

### 6.4.5 Simulations

#### LEARN IT

A **simulation** is a **computer-based model of a real situation**. **Variables** (the things that can change, like speed, temperature or number of people) can be changed to see what happens.

Simulations are highly useful when **real experiments are costly, risky or difficult** to perform.

**Examples:** a flight simulator for training pilots (real crashes are dangerous); a traffic simulation to test what happens if a new signal is added (changing real roads is costly); a weather model to predict a cyclone (you cannot experiment with real weather).

#### WORKED EXAMPLE

**Choosing the right method.** Which method would you use?

(a) "How many cars cross an intersection at 8 AM?" → **Observation**. Stand at the corner from 8:00 to 8:15 and tally each car. Asking drivers would be impossible and unnecessary; you just count.

(b) "How do students feel about the new cafeteria food?" → **Survey or interview**. Feelings are inside people's heads, so you must ask them. Use a survey for many students, or interviews for deeper reasons.

(c) "Will a new college website menu be easy to use?" → **Prototype**. Build a rough version and let users try it.

(d) "What happens to traffic if the bridge is closed for repairs?" → **Simulation**. Closing a real bridge to test is costly and disruptive.

#### REMEMBER IT

**Watch** it → Observation. **Ask** it → Survey/Interview. **Try** it → Prototype. **Model** it → Simulation.

#### COMMON MISTAKE

**Mistake:** Saying that observation involves asking people questions. **Fix:** In observation, **no direct questions** are asked – the researcher only watches and records. Also, a prototype is *not* the finished product; it is an early test version.

#### اردو میں سمجھیں

پروٹوٹائپ کسی پروڈکٹ یا حل کا ابتدائی سادہ نمونہ ہے جسے صارفین آزما کر رائے دیتے ہیں تاکہ اسے بہتر بنایا جاسکے۔ مشاہدے (Observation) میں محقق کوئی سوال نہیں پوچھتا بلکہ صرف دیکھ کر نتائج درج کرتا ہے۔ سمیولیشن کسی حقیقی صورتحال کا کمپیوٹر ماڈل ہے، جو اس وقت مفید ہے جب حقیقی تجربہ مہنگا، خطرناک یا مشکل ہو۔

#### EXAM TIP

Scenario questions are popular: "Which data-collection method is most suitable for ... ? Give a reason." Always name the method **and** justify it using its key feature (e.g. "Simulation, because a real experiment would be risky").

#### CHECK YOURSELF

1. Why would an engineer use a simulation instead of a real test?
2. Can observation produce quantitative data? Give an example.
3. What becomes the "data" when using a prototype?

Answers: 1. Because the real experiment may be costly, risky or difficult. 2. Yes – for example, counting the number of cars passing a point. 3. The feedback users give after trying the prototype.

## 6.5 Primary and Secondary Data ★ Golden

#### LEARN IT

Whenever you use data, it is important to know **where it came from**. The source affects how much **control** the researcher had over collecting it, and how **confidently** the results can be used.

### 6.5.1 Primary Data

**Primary data** is collected **first-hand** by the researcher for a **particular purpose**. It is gathered **directly** from people, events or situations, rather than taken from an already published source.

**Sources:** interviews, surveys, observations, experiments.

### 6.5.2 Secondary Data

**Secondary data** has **already been collected by someone else** and is being used again for a **new purpose**. It is taken from existing archives (stored collections of information).

**Sources:** books, research articles, official reports, web statistics, online databases.

#### Primary Data

Fresh, direct collection  
(You do the work)

Surveys · Interviews · Observation

#### Secondary Data

Existing archives  
(Someone else did it)

Books · Reports · Databases

Primary data is collected by you; secondary data was collected by others.



### 6.5.3 Difference Between Primary and Secondary Data

| Aspect          | Primary Data                                        | Secondary Data                                   |
|-----------------|-----------------------------------------------------|--------------------------------------------------|
| <b>Source</b>   | Collected directly by the researcher.               | Already collected by others.                     |
| <b>Examples</b> | Survey responses, interviews, observations.         | Reports, articles, websites, databases.          |
| <b>Purpose</b>  | Prepared for a specific study.                      | Reused for a new purpose.                        |
| <b>Control</b>  | High control over questions and collection process. | Limited control because the data already exists. |

#### WORKED EXAMPLE

**Primary or secondary?** Hira is researching "Internet use among students in Hyderabad".

(a) She sends a Google Form to 50 students in her college. → **Primary** (she collected it herself, for this study).

(b) She uses a PTA (Pakistan Telecommunication Authority) report on the number of broadband users in Pakistan. → **Secondary** (PTA collected it for its own purpose; Hira reuses it).

(c) She counts how many students use phones in the library during one hour. → **Primary** (her own observation).

(d) She quotes a statistic from a research article. → **Secondary**.

**Why use both?** Her primary data tells her about *her own* college; the secondary data lets her **compare** it with the national picture.

#### REMEMBER IT

**Primary** = "1st hand" (you collect it first). **Secondary** = "2nd hand" (like a second-hand book – someone else used it first).

#### COMMON MISTAKE

**Mistake:** Thinking data from a website is always primary because "I found it myself". **Fix:** What matters is *who collected* the data. If someone else collected it and you only found it, it is secondary.

#### اردو میں سمجھیں

بنیادی ڈیٹا (Primary Data) محقق خود کسی خاص مقصد کے لیے براہ راست جمع کرتا ہے، مثلاً سروے، انٹرویو یا مشاہدہ۔ ثانوی ڈیٹا (Secondary Data) پہلے ہی کسی اور نے جمع کیا ہوتا ہے اور اسے نئے مقصد کے لیے دوبارہ استعمال کیا جاتا ہے، جیسے کتابیں، رپورٹس اور آن لائن ڈیٹا بیس۔ بنیادی ڈیٹا پر محقق کا کنٹرول زیادہ اور ثانوی ڈیٹا پر کم ہوتا ہے۔

#### EXAM TIP

"Differentiate between primary and secondary data" is one of the most-asked questions in this chapter. Draw the four-row table: **Source, Examples, Purpose, Control**. Learn it exactly. Add one example from each side to secure full marks.

#### CHECK YOURSELF

1. Which type of data gives the researcher high control?
2. Is a government census report primary or secondary data for your project?
3. Give two sources of primary data.

Answers: 1. Primary data. 2. Secondary (someone else collected it). 3. Any two: interviews, surveys, observations, experiments.

## 6.6 Designing a Data-Collection Approach

### LEARN IT

Before collecting data, a researcher must have a **plan**. A plan makes sure that the information gathered is **accurate**, **relevant** (related to the question) and **useful**. Follow these eight steps in order:

1. **Define the research question clearly.** What exactly do you want to find out?
2. **Decide the type of data needed:** quantitative, qualitative, or both.
3. **Select suitable methods** (survey, interview, observation).
4. **Choose the sample or target group** – the people who will give you data.
5. **Prepare the tools** such as forms, questions and observation sheets.
6. **Collect the data** carefully and accurately.
7. **Store the data digitally** in a spreadsheet and **keep a backup**.
8. **Follow ethical practice and respect privacy. Consent is mandatory** (you must get permission).

Question → Data type → Method → Sample → Tools → Collect → Store + backup → Ethics

### WORKED EXAMPLE

**Applying the eight steps.** Topic: water wastage in your college.

1. **Question:** "How much water is wasted at the college taps during break time, and why?"
2. **Data type:** Both – quantitative (number of taps left running, minutes) and qualitative (reasons students give).
3. **Methods:** Observation (count running taps) + a short survey (ask about habits).
4. **Sample:** All 12 taps in the college; 30 students from different classes for the survey.
5. **Tools:** An observation sheet with columns *Tap no. / Time / Left running? (Y/N) / Minutes*; a Google Form with 5 questions.
6. **Collect:** Observe during break for five days; send the form link.
7. **Store:** Enter the observation counts into Google Sheets; download a copy to a USB drive as backup.
8. **Ethics:** Ask the principal's permission; tell students the survey is anonymous and voluntary; do not record names.

### REMEMBER IT

Remember the steps with: "**Quiet Teachers Make Students Think, Check, Save Ethically**" → Question, Type, Method, Sample, Tools, Collect, Store, Ethics.

### COMMON MISTAKE

**Mistake:** Starting to collect data before writing a clear research question. You then collect lots of data that does not answer anything. **Fix:** Always write the question first. Another mistake: keeping data in only one place – if the file is lost, the project is lost. Always keep a backup.

### اردو میں سمجھیں

ڈیٹا جمع کرنے سے پہلے منصوبہ بندی ضروری ہے: تحقیقی سوال واضح کریں، ڈیٹا کی قسم طے کریں، مناسب طریقہ چنیں، نمونہ (Sample) منتخب کریں، فارم اور سوالات تیار کریں، احتیاط سے ڈیٹا جمع کریں، اسے اسپریڈشیٹ میں محفوظ کر کے بیک اپ رکھیں، اور اخلاقیات کا خیال رکھیں۔ اجازت (Consent) لینا لازمی ہے۔

### CHECK YOURSELF

1. What is the first step in designing a data-collection approach?
2. Which step says "consent is mandatory"?
3. Where should collected data be stored?

Answers: 1. Define the research question clearly. 2. Step 8 – follow ethical practice and respect privacy. 3. Digitally, in a spreadsheet, with a backup copy.

## 6.7 Presenting Data Using Digital Tools: 6.7.1 Spreadsheets and 6.7.2 Presentations

### LEARN IT

Collecting data is only **one part** of the work. You must also **present** your findings so that others can understand them **quickly and easily**. Digital tools make this much easier. The four main tools are spreadsheets, presentations, infographics and reports.

### 6.7.1 Spreadsheets

#### LEARN IT

**Spreadsheets** such as **MS Excel**, **Google Sheets** and **Apple Numbers** arrange information in **rows** (horizontal, numbered 1, 2, 3...) and **columns** (vertical, lettered A, B, C...). The box where a row and column meet is a **cell**, named by its column and row, e.g. **B2**. With a spreadsheet you can:

- **Enter data** systematically (one record per row).
- **Calculate** totals, averages and percentages.
- **Sort** (arrange in order) and **filter** (show only some rows) records.
- **Create charts and graphs** automatically.

A **formula** always starts with = . A **range** such as B2:B4 means "cells B2 to B4".

| Formula                   | What it does                                       |
|---------------------------|----------------------------------------------------|
| =SUM(B2:B4)               | Adds all values in B2 to B4.                       |
| =AVERAGE(B2:B4)           | Mean: sum ÷ number of values.                      |
| =MAX(B2:B4) / =MIN(B2:B4) | Largest / smallest value.                          |
| =MEDIAN(B2:B4)            | Middle value when sorted.                          |
| =COUNT(B2:B4)             | Counts cells containing numbers.                   |
| =COUNTA(C2:C4)            | Counts cells that are not empty (text or numbers). |
| =COUNTIF(C2:C4, "Yes")    | Counts cells that match a condition.               |

### WORKED EXAMPLE

**Analysing the case-study spreadsheet.** The data from the 6.8 case study (Step 8) is entered like this:

|   | A       | B     | C         | D             | E           |
|---|---------|-------|-----------|---------------|-------------|
| 1 | Student | Hours | Study Use | Main Activity | Distracted? |
| 2 | 1       | 3     | Yes       | Lectures      | Sometimes   |
| 3 | 2       | 5     | No        | Social Media  | Yes         |
| 4 | 3       | 2     | Yes       | Notes         | No          |

**Step 1 – Total hours:** =SUM(B2:B4) → 3 + 5 + 2 = **10**.

**Step 2 – Average hours:** =AVERAGE(B2:B4) → 10 ÷ 3 = **3.33** hours (to 2 decimal places).

**Step 3 – Most and least:** =MAX(B2:B4) → **5**; =MIN(B2:B4) → **2**.

**Step 4 – Median:** sorted values 2, 3, 5 → middle = **3**. =MEDIAN(B2:B4) gives 3.

**Step 5 – How many use the phone for study?** =COUNTIF(C2:C4, "Yes") → **2**.

**Step 6 – Percentage:** =COUNTIF(C2:C4, "Yes")/COUNTA(C2:C4) → 2 ÷ 3 = 0.667; format the cell as % → **66.7%**.

**Step 7 – How many said "Yes" to distracted?** =COUNTIF(E2:E4, "Yes") → **1** (only "Yes" matches; "Sometimes" does not).

**Step 8 – Sort:** select the data → Data → Sort by column B (largest to smallest) → order becomes Student 2 (5), Student 1 (3), Student 3 (2).

## 6.7.2 Presentations

### LEARN IT

Presentation software such as **PowerPoint**, **Google Slides** and **Canva** helps you explain findings to an **audience** in an organised way. Good practice:

- Begin with a clear **title** and the **research question**.
- Use **short bullet points** instead of long paragraphs.
- Include **tables, charts and key findings**.
- Keep the design **simple and readable** (large font, few colours).

Spreadsheets and presentations work together: you make a chart from the spreadsheet data and insert it into the slides.

Spreadsheet data → Chart / Graph → Insert → Slides

### REMEMBER IT

Presentation rule of thumb: **one idea per slide**, and no more than about **six short bullet points**. If you are reading long sentences off the slide, the slide has too much text.

### COMMON MISTAKE

**Mistake:** Typing a result by hand (e.g. writing "3.33" in a cell) instead of using a formula. If the data changes, the typed result becomes wrong. **Fix:** Always use formulas like =AVERAGE(B2:B4) so results update automatically. Also remember every formula must begin with =.

### اردو میں سمجھیں

ڈیٹا جمع کرنے کے بعد اسے پیش کرنا بھی ضروری ہے۔ اسپریڈشیت (جیسے ایکسل اور گوگل شیٹس) میں ڈیٹا قطاروں اور کالموں میں ترتیب دیا جاتا ہے، اور =SUM() اور =AVERAGE() جیسے فارمولوں سے مجموعہ، اوسط اور فیصد نکالے جاتے ہیں۔ پریزنٹیشن سافٹ ویئر (جیسے پاور پوائنٹ) میں مختصر نکات، چارٹس اور سادہ ڈیزائن کے ساتھ نتائج سامعین کے سامنے پیش کیے جاتے ہیں۔

#### CHECK YOURSELF

1. Write the formula to find the average of cells D2 to D20.
2. Name three spreadsheet programs.
3. Give two rules for a good presentation.

Answers: 1. `=AVERAGE(D2:D20)` . 2. MS Excel, Google Sheets, Apple Numbers. 3. Any two: start with a clear title and research question; use short bullet points; include tables/charts/key findings; keep design simple and readable.

## 6.7.3 – 6.7.4 Infographics and Reports

### 6.7.3 Infographics

#### LEARN IT

An **infographic** (information + graphic) is a **visual way** of presenting information, facts or data so that people can understand the main idea **quickly and easily**. Tools such as **Canva**, **Power BI** or **Tableau** can be used. A good infographic should:

- **Simplify** complex concepts.
- **Highlight** the most important findings and results.
- Use **clear visuals**, **limited text** and **accurate** information.

### 6.7.4 Reports

#### LEARN IT

**Reports** are **structured documents** used to present collected data, findings, analysis and conclusions. Tools like **MS Word** or **Google Docs** are standard. A good report should:

- Start with a clear **title**, **introduction** and **purpose**.
- Present the collected data using **tables or charts**.
- Explain findings in **simple language**.
- Include **conclusions and recommendations** based on the data.

**Optional:** you may use AI tools carefully to improve wording, but **always review for accuracy**. AI can make mistakes or invent facts, and you are responsible for what you submit.

#### LEARN IT

**Visual rule of thumb:** a good chart or infographic can explain complex data **faster** than a long paragraph. However, visuals must always be **accurate** and based strictly on the **real data collected** – never exaggerated.

#### WORKED EXAMPLE

Which tool should you choose?

| Situation                                                           | Best tool    | Why                                 |
|---------------------------------------------------------------------|--------------|-------------------------------------|
| You must calculate the average marks of 40 students.                | Spreadsheet  | Formulas calculate instantly.       |
| You will explain results to the class in assembly.                  | Presentation | Designed for a live audience.       |
| You want a one-page poster for the notice board.                    | Infographic  | Main result understood at a glance. |
| The principal wants the full study with method and recommendations. | Report       | Detailed, structured document.      |

#### REMEMBER IT

Match tool to job: **Spreadsheet = Calculate, Presentation = Tell, Infographic = Show quickly, Report = Explain fully.**

#### COMMON MISTAKE

**Mistake:** Making a chart look more dramatic than the data – for example, starting the bar-chart axis at 18 instead of 0 so that 20 looks twice as big as 19. **Fix:** Start bar-chart axes at 0, label axes, and show real numbers only.

#### اردو میں سمجھیں

انفوگرافک معلومات کو تصویری انداز میں پیش کرنے کا طریقہ ہے تاکہ اصل بات جلدی سمجھ آ جائے؛ اس کے لیے کینوا (Canva) اور پاور بی آئی (Power BI) جیسے ٹولز استعمال ہوتے ہیں۔ رپورٹ ایک منظم دستاویز ہے جس میں عنوان، تعارف، ڈیٹا، نتائج، اختتامیہ اور سفارشات شامل ہوتی ہیں۔ تصاویر اور چارٹس ہمیشہ اصل ڈیٹا کے مطابق اور درست ہونے چاہئیں، مبالغہ آمیز نہیں۔

#### CHECK YOURSELF

1. Name two tools for making infographics.
2. What must you always do if you use AI to improve your report's wording?
3. What should a report end with?

Answers: 1. Any two: Canva, Power BI, Tableau. 2. Review it carefully for accuracy. 3. Conclusions and recommendations based on the data.

## 6.8 Case Study: Digital Inquiry Project ★ Golden

#### LEARN IT

This case study shows how to apply **all** the ideas of this chapter to solve a real-world problem from start to finish. An **inquiry** is an investigation to find the answer to a question.

**Research question:** "How does daily mobile phone use affect the study habits of students?"

**Scenario:** Students notice classmates using phones during free periods and while studying. Some say phones **help** them learn; others say phones **waste time**. The class must investigate whether phone use helps or harms study habits.

### The digital inquiry workflow

1 Question → 2 Search → 3 Collect → 4 Analyse → 5 Create

The project is carried out in **12 steps** that fit into this five-stage workflow:

| Workflow stage    | Steps                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------|
| Question & Search | 1 Advanced searches · 2 Identify data types · 3 Select methods                                       |
| Collect           | 4 Prepare the survey · 5 Choose the sample · 6 Collect primary data ethically · 7 Use secondary data |
| Analyse           | 8 Organise in a spreadsheet · 9 Create charts · 10 Analyse results                                   |
| Create            | 11 Write the conclusion · 12 Recommendations → final digital artefact                                |

#### REMEMBER IT

Workflow in five words: **Ask → Search → Collect → Analyse → Create**. Every research project you ever do will follow this pattern.

#### اردو میں سمجھیں

اس کیس اسٹڈی میں تحقیقی سوال یہ ہے: "روزانہ موبائل فون کا استعمال طلبہ کی پڑھائی کی عادات پر کیسے اثر ڈالتا ہے؟" تحقیق پانچ مراحل میں ہوتی ہے: سوال، تلاش، ڈیٹا جمع کرنا، تجزیہ اور آخری پروڈکٹ بنانا۔ یہ کام بارہ قدموں میں مکمل کیا جاتا ہے۔

#### EXAM TIP

Expect a long question: "Describe the steps of a digital inquiry project using the example of mobile phone use." Write the research question first, then the 12 steps with one or two lines each, and finish with the artefact. Learn the five-stage workflow as your skeleton.

#### CHECK YOURSELF

1. What is the research question in the case study?
2. Name the five stages of the digital inquiry workflow.

Answers: 1. "How does daily mobile phone use affect the study habits of students?" 2. Question, Search, Collect, Analyse, Create.

## 6.8 Steps 1 – 3: Advanced Search and Methodology ★ Golden

The **methodology** is the plan of methods a researcher uses to answer the question.

### Step 1: Perform advanced searches

#### LEARN IT

Students first search for **background information** using search engines (such as Google or Bing). They use **advanced search operators** – special symbols or words typed into the search box – to **narrow down** results and find **reliable secondary data**.

| Suggested query                            | Operator used | What it does                                                                                            |
|--------------------------------------------|---------------|---------------------------------------------------------------------------------------------------------|
| "mobile phone use" "study habits" students | " " (quotes)  | Finds the <b>exact phrases</b> "mobile phone use" and "study habits", not the words scattered randomly. |
| "academic performance" filetype:pdf        | filetype:     | Finds only <b>PDF documents</b> (often official reports and research papers).                           |
| digital distraction site:.edu              | site:         | Searches only <b>educational websites</b> (addresses ending in .edu).                                   |

**Other useful operators (background):** - (minus) excludes a word, e.g. phone addiction -game; OR finds either word, e.g. smartphone OR mobile; site:.gov.pk searches only Pakistani government websites.

#### LEARN IT

**Source evaluation checklist.** Before trusting a source, ask:

- |                                                                                       |                                                                                                                                          |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> Is the <b>author/organisation</b> clearly mentioned?         | <input type="checkbox"/> Is the information <b>recent</b> ?                                                                              |
| <input type="checkbox"/> Is the website <b>reliable</b> (.edu, .gov, research-based)? | <input type="checkbox"/> Does the source provide <b>facts/references</b> , or is it <b>biased</b> (one-sided, trying to sell something)? |

#### WORKED EXAMPLE

**Building a search query step by step.** You want official PDF reports about screen time of students on Pakistani government websites.

**Step 1 – Plain search:** screen time students → millions of results, many blogs and adverts.

**Step 2 – Exact phrase:** "screen time" students → results must contain the exact phrase.

**Step 3 – Only PDFs:** "screen time" students filetype:pdf → mostly reports and papers.

**Step 4 – Only government sites:** "screen time" students filetype:pdf site:.gov.pk → a small number of highly relevant, reliable results.

**Step 5 – Evaluate:** open a result and check the checklist: author named? year recent? references given?

### Step 2: Identify data types

#### LEARN IT

Students decide what kind of **primary data** they will collect:

- **Quantitative:** number of hours used daily; percentage who feel distracted.
- **Qualitative:** opinions on why they get distracted; suggestions for better use.

### Step 3: Select data-collection methods

#### LEARN IT

Because they need to ask the **same questions to many students**, a **survey/questionnaire** (via Google Forms or on paper) is the **most efficient** method. Interviews would take too long for so many people, and observation could not reveal opinions.

#### REMEMBER IT

Three operators, three jobs: " " = **exact**, filetype: = **format**, site: = **where**. Note: there is **no space** after the colon (site:.edu, not site: .edu).

#### COMMON MISTAKE

**Mistake:** Assuming the first search result is the most reliable. Search results can be adverts or popular blogs. **Fix:** Always apply the source evaluation checklist. A website ending in .edu or .gov is usually more reliable, but you should still check the date and author.

#### اردو میں سمجھیں

پہلے قدم میں طلبہ سرچ انجن پر جدید سرچ آپریٹرز (Advanced Search Operators) استعمال کرتے ہیں: کوئیشن مارکس (" ") بالکل وہی جملہ تلاش کرتے ہیں، filetype:pdf صرف پی ڈی ایف فائلیں اور site:.edu صرف تعلیمی ویب سائٹس دکھاتا ہے۔ ہر ذریعے کو مصنف، تاریخ، قابل اعتماد ویب سائٹ اور حوالوں کی بنیاد پر پرکھا جاتا ہے۔ پھر ڈیٹا کی اقسام طے کی جاتی ہیں اور بہت سے طلبہ سے سوال کرنے کے لیے سروے کو بہترین طریقہ چنا جاتا ہے۔



### EXAM TIP

"Explain any three advanced search operators with examples" is a likely short question. For each, write the operator, an example query, and what it does. Also learn the four-point source evaluation checklist.

### CHECK YOURSELF

1. Write a query that finds the exact phrase "online learning" only on educational websites.
2. Why was a survey chosen in Step 3?
3. Give one quantitative and one qualitative data item from Step 2.

Answers: 1. "online learning" site:.edu . 2. Because the same questions had to be asked to many students, and a survey is the most efficient way to do that. 3. Quantitative: hours used daily (or % who feel distracted). Qualitative: opinions on why they get distracted (or suggestions for better use).

## 6.8 Steps 4 – 6: Survey Preparation and Primary Data Collection ★ Golden

### Step 4: Prepare the survey

#### LEARN IT

Design **clear, unbiased** questions. Each question is linked to the type of data it produces:

| Question                                           | Type of data              |
|----------------------------------------------------|---------------------------|
| How many hours do you use your phone daily?        | Quantitative              |
| Which activity do you use your phone for the most? | Quantitative (categories) |
| How does your phone help you in studies?           | Qualitative               |
| Suggest one way to use phones responsibly.         | Qualitative               |

### Step 5: Choose the sample

#### LEARN IT

The **sample** is the group of people who provide the data. It is usually impossible to ask everyone (the whole **population**), so a smaller group is chosen. The students decide to survey **40 Class XI students from different sections** to get a **representative** mix of habits. "Representative" means the sample looks like the whole group – it includes different kinds of students, not just your friends.

### Step 6: Collect primary data (ethically)

#### LEARN IT

**Ethics** means doing what is morally right. When conducting the survey, students must follow strict ethical guidelines:

- Ask **permission** before collecting responses (**consent**).
- Avoid collecting **sensitive personal information**.
- Do **not record names** unless absolutely necessary (**anonymity**).
- Keep the responses **private**.
- Use the data **only for educational purposes**.

#### WORKED EXAMPLE

**Why "different sections" matters.** Suppose the group surveyed only the 40 students of the Pre-Engineering section who sit in the front rows.

**Problem:** these students might study more than average, so the results would show less phone distraction than is true for the whole class.

**Fix:** pick students from every section (e.g. 10 each from Pre-Medical, Pre-Engineering, Computer Science and Commerce). Now the sample better represents all Class XI students.

**Ethical opening text for the form:** "This survey is for a class project on phone use and study habits. It is anonymous – do not write your name. Your answers will be kept private and used only for this project. Do you agree to take part? O Yes O No"

#### REMEMBER IT

Ethics in five words: **Consent, No-Sensitive, Anonymous, Private, Educational – "C-NAPE"**. Think: "Cover the person's nape (neck) – protect them."

#### COMMON MISTAKE

**Mistake:** Thinking a bigger sample is always enough. A sample of 200 students all from one section is still biased. **Fix:** A sample must be **representative**, not just large.

#### اردو میں سمجھیں

چوتھے قدم میں واضح اور غیر جانبدار سوالات تیار کیے جاتے ہیں۔ پانچویں قدم میں نمونہ (Sample) چنا جاتا ہے: مختلف سیکشنز کے چالیس طلبہ تاکہ نتائج پوری جماعت کی نمائندگی کریں۔ چھٹے قدم میں اخلاقی اصولوں کے ساتھ ڈیٹا جمع کیا جاتا ہے: اجازت لیں، حساس معلومات نہ لیں، نام نہ لکھیں (گمنامی)، جوابات کو راز میں رکھیں اور صرف تعلیمی مقصد کے لیے استعمال کریں۔

#### EXAM TIP

Questions often ask: "What ethical guidelines should be followed when collecting data?" List all five points. Use the words **consent** and **anonymity** – examiners look for them. You may also be asked to classify the four survey questions by data type.

#### CHECK YOURSELF

1. What is a sample?
2. What does anonymity mean in a survey?
3. What type of data does "Suggest one way to use phones responsibly" give?

Answers: 1. The group of people who provide the data. 2. Not recording the names of participants (unless absolutely necessary). 3. Qualitative.

## 6.8 Steps 7 – 9: Secondary Data and Spreadsheet Organisation ★ Golden

### Step 7: Use secondary data

#### LEARN IT

Students include a reliable online article stating that "**excessive screen time lowers concentration.**" They will **compare** this secondary data with their own primary survey results. If both agree, the conclusion is stronger. If they disagree, the students must think about why (for example, a different age group or country).

## Step 8: Organise data in a spreadsheet

### LEARN IT

Responses are entered into **MS Excel** or **Google Sheets** to **sort**, **calculate totals** and **find averages**. Each row is one student; each column is one question.

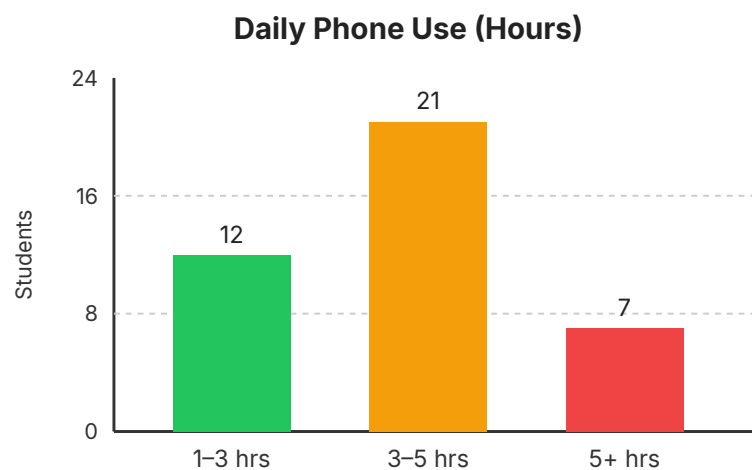
| Student | Hours | Study Use | Main Activity | Distracted? |
|---------|-------|-----------|---------------|-------------|
| 1       | 3     | Yes       | Lectures      | Sometimes   |
| 2       | 5     | No        | Social Media  | Yes         |
| 3       | 2     | Yes       | Notes         | No          |

(The full sheet has 40 rows, one per student. The formulas for this table are worked out in topic 6.7.1.)

## Step 9: Create charts and graphs

### LEARN IT

The spreadsheet data is converted into **visuals** to make it easy to digest. In the source, the "Daily Phone Use (Hours)" bar chart groups the 40 students into three bars: 1–3 hrs, 3–5 hrs and 5+ hrs, with the 3–5 hrs bar the tallest.



Bar chart of daily phone use for 40 students. The counts (12, 21, 7) are estimated from the bar heights in the source chart.

### WORKED EXAMPLE

#### Making the chart in Google Sheets.

**Step 1 – Count each group.** If the hours are in B2:B41, use `=COUNTIF(B2:B41,"<3")` for under 3 hrs, `=COUNTIFS(B2:B41,"≥3",B2:B41,"<5")` for 3 to under 5 hrs, and `=COUNTIF(B2:B41,"≥5")` for 5 hrs or more. (This gives non-overlapping groups.)

**Step 2 – Check the total.**  $12 + 21 + 7 = 40$  ✓ – every student is counted exactly once.

**Step 3 – Percentages.**  $12 \div 40 = 30\%$ ;  $21 \div 40 = 52.5\%$ ;  $7 \div 40 = 17.5\%$ . Total = 100% ✓.

**Step 4 – Insert chart.** Select the group labels and counts → Insert → Chart → choose *Column chart*. Add the title "Daily Phone Use (Hours)" and label the axes.

**Step 5 – Choose the right chart.** Bar/column chart to compare groups; pie chart to show parts of a whole (e.g. % in each group); line chart to show change over time.

#### REMEMBER IT

Chart chooser: **Bar = Compare, Pie = Parts of a whole, Line = Change over time.**

#### COMMON MISTAKE

**Mistake:** Using overlapping groups like "1–3" and "3–5": a student who uses the phone for exactly 3 hours could be counted twice or put in the wrong bar. **Fix:** Define groups clearly, e.g. "under 3", "3 to under 5", "5 or more", and check that the group counts add up to the sample size.

#### اردو میں سمجھیں

ساتویں قدم میں ایک قابلِ اعتماد آن لائن مضمون کا ثانوی ڈیٹا ("زیادہ اسکرین ٹائم توجہ کم کرتا ہے") اپنے سروے کے بنیادی ڈیٹا سے موازنہ کرنے کے لیے لیا جاتا ہے۔ آٹھویں قدم میں جوابات ایکسل یا گوگل شیٹس میں درج کر کے ترتیب، مجموعہ اور اوسط نکالی جاتی ہے۔ نویں قدم میں اسپریڈشیٹ کے ڈیٹا سے چارٹ اور گراف بنائے جاتے ہیں تاکہ نتائج آسانی سے سمجھ آئیں۔

#### EXAM TIP

You may get the spreadsheet table and be asked to "write formulas to find the total and average hours" or "count students who use the phone for study". Write the full formula with `=`, the correct range, and the result.

#### CHECK YOURSELF

1. What is the secondary data used in Step 7?
2. Which formula counts how many students answered "Yes" in column C, rows 2 to 41?
3. Which type of chart is best for comparing the three hour-groups?

Answers: 1. A reliable online article stating "excessive screen time lowers concentration". 2. `=COUNTIF(C2:C41, "Yes")`. 3. A bar (column) chart.

## 6.8 Steps 10 – 12: Data Analysis, Conclusion and Recommendations ★ Golden

### Step 10: Analyse the results

#### LEARN IT

To **analyse** means to study the data carefully to find **patterns** and **trends**. By looking at the spreadsheet and charts, the group identifies:

- **Most students** (the majority) use phones for **3–5 hours daily**.
- **Half** use it for **educational purposes** (lectures/notes).
- **A clear trend:** students who use phones mostly for **entertainment** report much **higher distraction** levels than those who use them for study.

### Step 11: Write the conclusion

#### LEARN IT

A **conclusion** answers the research question using the evidence:

*"The research shows mobile phones have both positive and negative effects. They provide access to learning materials. However, excessive use of social media and gaming reduces concentration. Therefore, phones should not be banned, but managed wisely."*

## Step 12: Recommendations

### LEARN IT

**Recommendations** are practical suggestions based on the data. The students recommend:

- Turn off **social media notifications** while studying.
- Set **daily screen-time limits**.
- Take **physical breaks** from screens to protect health.

### WORKED EXAMPLE

**From numbers to a finding.** Using the estimated chart counts (12, 21, 7 out of 40):

**Step 1:**  $21 \div 40 = 52.5\%$ . Since  $52.5\% > 50\%$ , the statement "most students use phones for 3–5 hours" is supported.

**Step 2:** "Half use it for educational purposes" means about 20 of the 40 students chose Lectures or Notes as their main activity ( $20 \div 40 = 50\%$ ). Formula:  $=\text{COUNTIF}(D2:D41, \text{"Lectures"}) + \text{COUNTIF}(D2:D41, \text{"Notes"})$ .

**Step 3 – Compare groups:** filter the sheet to show only Social Media/Games users and count how many said "Yes" to distracted; then do the same for Lectures/Notes users. If the entertainment group has a much higher share of "Yes", that is the trend.

**Step 4 – Link to secondary data:** the article said excessive screen time lowers concentration; the survey agrees, so the conclusion is stronger.

### REMEMBER IT

Conclusion formula: "**The data shows ... However ... Therefore ...**" – state the main finding, mention the other side, then give the overall answer.

### COMMON MISTAKE

**Mistake:** Writing a conclusion that is not based on the data, e.g. "Phones should be banned" when the data showed phones also help learning. **Fix:** Every sentence in the conclusion and recommendations must be supported by your findings. Also, a trend between two things (entertainment use and distraction) shows a *link*; be careful before claiming one definitely *causes* the other.

### اردو میں سمجھیں

دسویں قدم میں چارٹس اور اسپریڈشیٹ سے رجحانات تلاش کیے جاتے ہیں: زیادہ تر طلبہ روزانہ تین سے پانچ گھنٹے فون استعمال کرتے ہیں، نصف اسے تعلیم کے لیے استعمال کرتے ہیں، اور تفریح کے لیے استعمال کرنے والے زیادہ توجہ بننے کی شکایت کرتے ہیں۔ گیارہویں قدم میں نتیجہ لکھا جاتا ہے کہ فون پر پابندی نہیں بلکہ اس کا دانشمندانہ استعمال ہونا چاہیے۔ بارہویں قدم میں سفارشات دی جاتی ہیں: پڑھائی کے دوران نوٹیفیکیشن بند کریں، اسکرین ٹائم کی حد مقرر کریں اور وقفے لیں۔

### EXAM TIP

You may be asked to "write a conclusion and two recommendations" for given data. Use the "shows ... however ... therefore" pattern and make sure each recommendation follows from a finding.

### CHECK YOURSELF

1. According to Step 10, which group reported higher distraction?
2. Should phones be banned according to the conclusion?
3. Give two of the students' recommendations.

Answers: 1. Students who use phones mostly for entertainment. 2. No – they should not be banned but managed wisely. 3. Any two: turn off social media notifications while studying; set daily screen-time limits; take physical breaks from screens.

## 6.8 The Final Digital Artefact ★ Golden

### LEARN IT

A **digital artefact** is a final product created using digital technologies. Students must create **one final artefact** that answers the original research question. It is **not just a decorated file**; it must present **evidence** and **conclusions**.

| Artefact option | Suitable digital tools    | Main purpose                                            |
|-----------------|---------------------------|---------------------------------------------------------|
| Digital Report  | MS Word, Google Docs      | Explain the full study in a structured text document.   |
| Presentation    | PowerPoint, Canva, Slides | Share key findings with a live audience.                |
| Infographic     | Canva, Adobe Express      | Show the main results visually and quickly on one page. |
| Dashboard       | Power BI, Tableau         | Explore data through interactive charts and filters.    |

A **dashboard** is an interactive screen of charts; the viewer can click filters (e.g. "show only Commerce students") and the charts update.

### Standard artefact structure

1. Title
2. Research Question
3. Introduction
4. Method of Data Collection
5. Primary Data Findings
6. Secondary Data Summary
7. Charts and Graphs
8. Conclusion
9. Recommendations
10. References

### WORKED EXAMPLE

**Planning a 10-slide presentation artefact for the phone study** (one slide per structure item):

**Slide 1** Title: "Phones and Study Habits – Class XI Survey". **Slide 2** Research question. **Slide 3** Introduction: why the class investigated this. **Slide 4** Method: Google Form survey, 40 students, different sections, anonymous with consent. **Slide 5** Primary findings: 52.5% use phones 3–5 hours; half use for study. **Slide 6** Secondary data: article on screen time and concentration. **Slide 7** The bar chart of daily phone use. **Slide 8** Conclusion: "manage wisely, don't ban". **Slide 9** Three recommendations. **Slide 10** References: the article's title, author, website and date.

### REMEMBER IT

The structure follows the inquiry itself: **What** (title, question, intro) → **How** (method) → **Found** (primary, secondary, charts) → **So what** (conclusion, recommendations) → **Credit** (references).

### COMMON MISTAKE

**Mistake:** Spending all your time on colours and animations and forgetting evidence. **Fix:** An artefact is judged on whether it answers the research question with evidence. Also never skip **References** – listing your secondary sources is honest and avoids plagiarism.

ڈیجیٹل آرٹیفیکٹ (Digital Artefact) ڈیجیٹل ٹیکنالوجی سے بنائی گئی آخری پروڈکٹ ہے جو تحقیقی سوال کا جواب ثبوت اور نتائج کے ساتھ پیش کرتی ہے؛ یہ صرف سچی ہوئی فائل نہیں۔ اس کی چار صورتیں ہیں: رپورٹ، پریزنٹیشن، انفوگرافک اور ڈیش بورڈ۔ معیاری ساخت میں عنوان، تحقیقی سوال، تعارف، طریقہ کار، بنیادی و ثانوی ڈیٹا، چارٹس، نتیجہ، سفارشات اور حوالہ جات شامل ہیں۔

**EXAM TIP**

Learn the four artefact options with tools and purpose (a table question), and the ten-part structure in order. "What is a digital artefact?" – stress that it must present **evidence and conclusions**, not just decoration.

**CHECK YOURSELF**

1. Which artefact lets viewers explore data with interactive filters?
2. What is the last item in the standard artefact structure?
3. Name two tools for making an infographic artefact.

Answers: 1. A dashboard (Power BI, Tableau). 2. References. 3. Canva and Adobe Express.

## Careers in Digital Research

**LEARN IT**

The skills in this chapter lead to real jobs:

| Career                                 | What they do                                                                        | Chapter skill used                  |
|----------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------|
| <b>Market Research Analyst</b>         | Designs surveys to collect primary data to guide business decisions.                | Surveys, primary data               |
| <b>UX (User Experience) Researcher</b> | Gathers qualitative and quantitative data to improve app and website designs.       | Prototypes, interviews, observation |
| <b>Data Journalist</b>                 | Uses advanced search skills to research stories and presents them via infographics. | Advanced search, infographics       |
| <b>Digital Marketing Specialist</b>    | Collects audience data and presents results through dashboards and reports.         | Dashboards, reports                 |

**WORKED EXAMPLE**

**A day for a UX researcher.** A Karachi food-delivery app wants to redesign its checkout. The UX researcher (1) builds a clickable **prototype**, (2) **observes** 8 users trying it and counts how many seconds each takes (quantitative), (3) **interviews** them about what confused them (qualitative), and (4) presents a short **report** with recommendations to the design team.

#### REMEMBER IT

Link each career to one tool: Market Researcher → **Survey**; UX Researcher → **Prototype**; Data Journalist → **Search + Infographic**; Digital Marketer → **Dashboard**.

#### اردو میں سمجھیں

اس باب کی مہارتوں سے کئی کیریئر بنتے ہیں: مارکیٹ ریسرچ اینالسٹ کاروباری فیصلوں کے لیے سروے بناتا ہے، یو ایکس (UX) ریسرچر ایپس اور ویب سائٹس بہتر بنانے کے لیے ڈیٹا جمع کرتا ہے، ڈیٹا جرنلسٹ جدید سرچ اور انفوگرافکس استعمال کرتا ہے، اور ڈیجیٹل مارکیٹنگ اسپیشلسٹ ڈیش بورڈز اور رپورٹس کے ذریعے نتائج پیش کرتا ہے۔

#### CHECK YOURSELF

1. Which career designs surveys to guide business decisions?
2. What does UX stand for?

Answers: 1. Market Research Analyst. 2. User Experience.

## Chapter 6 Review

### Chapter summary

- **Digital literacy** is the ability to understand and use digital devices, the internet and online tools safely, effectively and responsibly – not just using devices, but finding, evaluating and creating information.
- In the **Information Age**, access to technology is not enough; digital literacy needs technical, cognitive and social abilities.
- **Qualitative data** is descriptive and non-numerical (opinions, reasons, behaviour) and explains why/how.
- **Quantitative data** is numerical, countable or measurable (hours, scores, percentages) and is used for comparing and charting.
- The five **data-collection strategies** are interviews, surveys, prototypes, observation and simulations.
- **Interviews** are planned conversations giving mostly qualitative data; **surveys** ask the same questions to many people, on paper or with Google/Microsoft Forms.
- Good survey questions are clear, short, unbiased and ask about one thing; avoid leading, double-barrelled, vague and overlapping questions.
- A **prototype** is an early version tested for feedback; **observation** records what happens without asking questions; a **simulation** is a computer model used when real experiments are costly, risky or difficult.
- **Primary data** is collected first-hand (high control); **secondary data** was collected by others and reused (limited control).
- Designing a data-collection approach has eight steps: question, data type, method, sample, tools, collect, store with backup, and ethics (consent is mandatory).
- **Spreadsheets** organise and calculate (e.g. =SUM(), =AVERAGE(), =COUNTIF()); **presentations** deliver to an audience; **infographics** give quick visual summaries; **reports** give detailed structure.
- Visuals must always be accurate and based strictly on real data, never exaggerated.
- **Advanced search operators** such as " ", filetype:pdf and site:.edu help find reliable sources efficiently; sources must be evaluated for author, date, reliability and bias.
- **Ethics**: get consent, avoid sensitive information, keep anonymity, keep responses private, and use data only for educational purposes.
- The **digital inquiry project** follows Question → Search → Collect → Analyse → Create in 12 steps.
- A **digital artefact** (report, presentation, infographic or dashboard) answers the research question using evidence and visuals, following a ten-part structure ending in references.



## Key terms

| Term                   | Meaning                                                                                                           | اردو                     |
|------------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------|
| Digital literacy       | Ability to understand and use digital devices, the internet and online tools safely, effectively and responsibly. | ڈیجیٹل خواندگی           |
| Information Age        | Period in which vast amounts of data are produced, shared and accessed instantly.                                 | معلوماتی دور             |
| Data                   | Facts, numbers, words or observations collected for study.                                                        | مواد / ڈیٹا              |
| Qualitative data       | Descriptive, non-numerical data about thoughts, feelings and experiences.                                         | کیفیتی ڈیٹا              |
| Quantitative data      | Numerical data that can be counted or measured.                                                                   | مقداری ڈیٹا              |
| Interview              | Planned conversation to gather information directly from a person or small group.                                 | انٹرویو / بالمشافہ گفتگو |
| Survey / Questionnaire | Same set of questions asked to many people.                                                                       | سروے / سوالنامہ          |
| Prototype              | Simple early version of a product used to test an idea and get feedback.                                          | ابتدائی نمونہ            |
| Observation            | Watching and recording what happens without asking questions.                                                     | مشاہدہ                   |
| Simulation             | Computer-based model of a real situation in which variables can be changed.                                       | نقلی ماڈل (سمیولیشن)     |
| Variable               | A factor that can be changed or measured.                                                                         | متغیر                    |
| Primary data           | Data collected first-hand by the researcher for a specific purpose.                                               | بنیادی ڈیٹا              |
| Secondary data         | Data already collected by someone else and reused for a new purpose.                                              | ثانوی ڈیٹا               |
| Research question      | The clear question an investigation tries to answer.                                                              | تحقیقی سوال              |
| Sample                 | The group of people who provide the data.                                                                         | نمونہ                    |
| Representative sample  | A sample that reflects the mix of the whole group.                                                                | نمائندہ نمونہ            |
| Consent                | Permission given by a person before their data is collected.                                                      | رضامندی / اجازت          |

| Term                     | Meaning                                                                                      | اردو                           |
|--------------------------|----------------------------------------------------------------------------------------------|--------------------------------|
| Anonymity                | Not recording people's names so they cannot be identified.                                   | گمنامی                         |
| Ethics                   | Rules of right and fair behaviour when collecting and using data.                            | اخلاقیات                       |
| Spreadsheet              | Software that arranges data in rows and columns and performs calculations.                   | اسپریڈشیٹ / حسابی جدول         |
| Formula                  | A spreadsheet instruction starting with = that calculates a result.                          | فارمولا                        |
| Infographic              | A visual presentation of information so the main idea is understood quickly.                 | تصویری معلومات (انفوگرافک)     |
| Report                   | Structured document presenting data, findings, analysis and conclusions.                     | رپورٹ                          |
| Dashboard                | Interactive screen of charts and filters for exploring data.                                 | ڈیش بورڈ                       |
| Advanced search operator | Special symbol or word (e.g. " ", filetype:, site:) that narrows search results.             | جدید تلاش آپریٹر               |
| Methodology              | The plan of methods used to answer a research question.                                      | طریقہ کار                      |
| Digital artefact         | Final product made with digital technology that answers the research question with evidence. | ڈیجیٹل آرٹیفیکٹ / ڈیجیٹل تخلیق |
| Recommendation           | A practical suggestion based on the findings.                                                | سفارش                          |

## Practice: multiple choice

1. Digital literacy is best described as the ability to:
  - a. type quickly on a keyboard
  - b. understand and use digital devices, the internet and online tools safely, effectively and responsibly
  - c. repair computer hardware
  - d. write computer programs
2. Which of the following is qualitative data?
  - a. Marks obtained in a test
  - b. Number of hours spent online
  - c. Students' opinions about online classes
  - d. Percentage of students using the internet
3. Which method is most suitable when the same questions must be asked to many people?
  - a. Survey
  - b. Interview
  - c. Prototype
  - d. Simulation
4. Simulations are especially useful when:
  - a. you want people's opinions
  - b. the data already exists in a report
  - c. you want to watch people without asking questions
  - d. real experiments are costly, risky or difficult
5. To find out how many cars cross an intersection at 8 AM, the best method is:
  - a. Interview
  - b. Observation
  - c. Survey
  - d. Prototype
6. Which of these is primary data?
  - a. Responses to a Google Form you created for your project
  - b. An official government report
  - c. A published research article
  - d. Web statistics from a website
7. A researcher has limited control over:
  - a. their own interviews
  - b. their own survey questions
  - c. their own observations
  - d. secondary data
8. The first step in designing a data-collection approach is to:
  - a. collect the data
  - b. choose the sample
  - c. define the research question clearly
  - d. store the data with a backup
9. Which spreadsheet formula finds the mean of cells B2 to B6?
  - a. `=AVERAGE(B2:B6)`
  - b. `=SUM(B2:B6)`
  - c. `=COUNT(B2:B6)`
  - d. `=MAX(B2:B6)`
10. The search query `digital distraction site:.edu` will:
  - a. find only PDF files
  - b. search only educational websites
  - c. find the exact phrase "digital distraction" only
  - d. exclude educational websites
11. The operator `filetype:pdf` is used to:
  - a. search for an exact phrase
  - b. search only government websites
  - c. convert a web page into PDF
  - d. find only PDF documents
12. "Do not record names unless absolutely necessary" describes the principle of:
  - a. consent
  - b. accuracy
  - c. anonymity
  - d. sampling
13. In the case study, how many Class XI students were surveyed?
  - a. 20
  - b. 40
  - c. 50
  - d. 100
14. Which tools are suggested for a dashboard artefact?
  - a. Power BI, Tableau
  - b. MS Word, Google Docs
  - c. PowerPoint, Google Slides
  - d. Google Forms, Microsoft Forms
15. A prototype is:
  - a. the final finished product
  - b. a published report
  - c. a type of survey question
  - d. a simple early version of a product used to test an idea

## Practice: short questions

1. Define digital literacy. Name the three kinds of ability it involves.
2. Why is simply having access to technology not enough in the Information Age?
3. Differentiate between qualitative and quantitative data, giving two examples of each.
4. How is an interview different from a survey?
5. What is a prototype, and how does it produce useful data?
6. When is a simulation used? Give one example.
7. Compare primary and secondary data in terms of purpose and control.
8. List any four ethical guidelines to follow when collecting primary data.
9. Explain three advanced search operators with an example of each.
10. What is a digital artefact? Name its four options with one suitable tool each.

## Practice: long / problem questions

1. Explain the eight steps of designing a data-collection approach. Apply each step to the research question: "How much water is wasted at the college taps during break time, and why?"
2. **Design a survey for** finding out how Class XI students in your college travel to college and how long the journey takes. Write a consent statement and at least six questions (mixing closed, number, rating and open-ended types), state the data type of each question, describe your sample, and explain how you will keep the data ethical.
3. Ten students recorded their daily phone use (hours) in cells B2 to B11: 2, 4, 3, 5, 6, 4, 3, 4, 1, 4. (a) Write the spreadsheet formulas and give the results for the total, mean, median, mode, maximum and minimum. (b) Write a formula to count students using the phone for more than 4 hours, and find the percentage. (c) Group the data into "1–2 hrs", "3–4 hrs" and "5+ hrs" and name a suitable chart.
4. Describe all 12 steps of the Digital Inquiry Project on the question "How does daily mobile phone use affect the study habits of students?", including the final digital artefact and its structure.
5. **Design a survey for** investigating "How do Class XI students use AI tools for homework?" Then explain which secondary sources you would search for (with advanced search queries), and which artefact you would create to present your findings to the principal, with reasons.

## Answer key

---

### Multiple choice

1. b
2. c
3. a
4. d
5. b
6. a
7. d
8. c
9. a
10. b
11. d
12. c
13. b
14. a
15. d

## Short questions — model answers

1. Digital literacy is the ability to understand and use digital devices, the internet and online tools in a safe, effective and responsible manner. It involves technical, cognitive and social abilities.
2. In the Information Age vast amounts of data are produced and shared instantly. Owning a device does not mean you can judge or use this information well. Users must develop skills to understand, manage and use information wisely, and to tell reliable information from unreliable.
3. Qualitative data is descriptive and non-numerical; it explains why and how (e.g. students' opinions about online classes; observation notes). Quantitative data is numerical, countable or measurable; it is used for comparing and making charts (e.g. marks in a test; hours spent online).
4. An interview is a planned conversation with one person or a small group, giving mostly qualitative, in-depth answers. A survey asks the same set of questions to many people (on paper or via Google/Microsoft Forms) and is efficient for large groups; online surveys also collect responses and make charts automatically.
5. A prototype is a simple early version of a product, model or solution. Users try it and give feedback; this feedback is data used to improve the product before time and effort are spent on the final version.
6. A simulation is used when real experiments are costly, risky or difficult. Example: a flight simulator to train pilots, or a traffic simulation to test the effect of closing a road.
7. Purpose: primary data is prepared for a specific study; secondary data is reused for a new purpose. Control: the researcher has high control over primary data (questions and process) and limited control over secondary data because it already exists.
8. Any four: ask permission (consent); avoid sensitive personal information; do not record names unless necessary (anonymity); keep responses private; use data only for educational purposes.
9. Quotes " " find an exact phrase, e.g. "study habits" students. filetype: finds a file format, e.g. "academic performance" filetype:pdf gives only PDFs. site: limits to a type of website, e.g. digital distraction site:.edu searches only educational sites.
10. A digital artefact is a final product created with digital technology that answers the research question with evidence and conclusions. Options: digital report (MS Word/Google Docs), presentation (PowerPoint/Canva/Slides), infographic (Canva/Adobe Express), dashboard (Power BI/Tableau).

## Long questions — model answers

- 1. Eight steps applied to water wastage.** (1) **Define the question:** "How much water is wasted at the college taps during break time, and why?" (2) **Data type:** both – quantitative (number of taps left running, minutes running) and qualitative (students' reasons). (3) **Methods:** observation to count running taps; a short survey to learn habits and reasons. (4) **Sample:** all 12 taps observed over 5 break times; 30 students from different classes surveyed. (5) **Tools:** an observation sheet (Tap no., Time, Left running Y/N, Minutes) and a Google Form. (6) **Collect** carefully, at the same time each day, recording accurately. (7) **Store** data in Google Sheets and keep a backup copy (download or USB). (8) **Ethics:** get the principal's permission; the survey is voluntary and anonymous with a consent question; data used only for the project.
- 2. Survey on travel to college (model).** *Consent:* "This anonymous survey is for a Class XI project on travel to college. Do not write your name. Answers will be kept private and used only for this project. Do you agree to take part? Yes / No." *Questions:* Q1 How do you usually travel to college? Walk / Bus / Rickshaw / Motorbike / Car / Other – quantitative (categories). Q2 How many minutes does your journey usually take? \_\_\_\_ – quantitative. Q3 How many days a week are you late because of travel? 0 / 1 / 2 / 3 or more – quantitative. Q4 "My journey to college is safe." 1 (strongly disagree) to 5 (strongly agree) – quantitative (rating). Q5 Do you travel alone? Yes / No – quantitative (count). Q6 What is the biggest problem you face while travelling? – qualitative. Q7 Suggest one way the college could make travel easier. – qualitative. Questions are unbiased, ask one thing each, and options do not overlap. *Sample:* about 40 Class XI students, roughly 10 from each section, to be representative. *Ethics:* consent, no sensitive information (no home address), no names (anonymity), private storage, educational use only. Data is collected through Google Forms and exported to Google Sheets for analysis.
- 3. Spreadsheet problem.** (a)  $\text{=SUM(B2:B11)} \rightarrow 2+4+3+5+6+4+3+4+1+4 = 36$ .  $\text{=AVERAGE(B2:B11)} \rightarrow 36 \div 10 = 3.6$  hours. Sorted: 1, 2, 3, 3, 4, 4, 4, 4, 5, 6;  $\text{=MEDIAN(B2:B11)} \rightarrow (4 + 4) \div 2 = 4$ .  $\text{=MODE(B2:B11)} \rightarrow 4$  (appears 4 times).  $\text{=MAX(B2:B11)} \rightarrow 6$ ;  $\text{=MIN(B2:B11)} \rightarrow 1$ . (b)  $\text{=COUNTIF(B2:B11, ">4")}$  → values 5 and 6 → 2 students; percentage  $\text{=COUNTIF(B2:B11, ">4")}/\text{COUNT(B2:B11)} \rightarrow 2 \div 10 = 20\%$ . (c) 1–2 hrs: 2 students (2, 1); 3–4 hrs: 6 students (4, 3, 4, 3, 4, 4); 5+ hrs: 2 students (5, 6). Check:  $2 + 6 + 2 = 10 \checkmark$ . A **bar/column chart** is suitable for comparing the groups (or a pie chart to show 20%, 60%, 20%).
- 4. Twelve steps.** (1) Advanced searches with operators such as "mobile phone use" "study habits" students, "academic performance" filetype:pdf, digital distraction site:.edu; evaluate sources (author, recent, reliable site, facts/bias). (2) Identify data types: quantitative (hours daily, % distracted) and qualitative (reasons, suggestions). (3) Select method: survey via Google Forms/paper, because the same questions go to many students. (4) Prepare the survey: hours daily (quantitative), main activity (quantitative categories), how the phone helps (qualitative), one responsible-use suggestion (qualitative). (5) Sample: 40 Class XI students from different sections. (6) Collect ethically: consent, no sensitive data, anonymity, privacy, educational use only. (7) Secondary data: an article saying excessive screen time lowers concentration, to compare with results. (8) Organise in Excel/Google Sheets (Student, Hours, Study Use, Main Activity, Distracted?), sort, total, average. (9) Charts: bar chart of daily phone use (1–3, 3–5, 5+ hrs). (10) Analyse: majority use 3–5 hours; half use for education; entertainment users report more distraction. (11) Conclusion: phones have positive and negative effects; should be managed wisely, not banned. (12) Recommendations: turn off social media notifications, set screen-time limits, take physical breaks. **Artefact:** one of report, presentation, infographic or dashboard, structured as Title, Research Question, Introduction, Method, Primary Findings, Secondary Summary, Charts, Conclusion, Recommendations, References.
- 5. AI tools for homework (model).** *Survey:* consent statement (anonymous, voluntary, educational use only). Q1 Have you used an AI tool (e.g. ChatGPT, Gemini) for homework? Yes / No – quantitative. Q2 How many times a week? 0 / 1–2 / 3–5 / 6 or more – quantitative (non-overlapping categories). Q3 What do you use it for most? Explaining topics / Checking answers / Writing assignments / Translation / Other – quantitative (categories). Q4 "AI tools help me understand my subjects better." 1–5 rating – quantitative. Q5 Do you check AI answers against your textbook? Always / Sometimes / Never – quantitative. Q6 Describe one problem you have faced with AI answers. – qualitative. *Sample:* about 40 Class XI students from all sections. *Secondary sources:* "generative AI" students homework filetype:pdf; "AI in education" site:.edu; "artificial intelligence" policy site:.gov.pk; evaluate each for author, date, reliability and bias. *Artefact:* a **digital report** (MS Word/Google Docs) because the principal needs the full study – method, findings, charts, conclusion, recommendations and references – in a structured document; a one-page infographic could be attached as a quick summary. Charts would be made in Google Sheets from the form responses, and every visual must match the real data.

## Self-assessment: I can...

- ☐ define digital literacy and explain why it matters in the Information Age.
- ☐ name and explain the five data-collection strategies and choose the right one for a scenario.
- ☐ compare primary and secondary data by source, examples, purpose and control.
- ☐ follow ethical rules: consent, anonymity, privacy and educational use.
- ☐ use advanced search operators and evaluate whether a source is reliable.
- ☐ describe the 12 steps of the digital inquiry project.
- ☐ classify data as qualitative or quantitative and explain the difference in a table.
- ☐ write clear, unbiased survey questions and fix leading, double-barrelled or vague ones.
- ☐ list the eight steps for designing a data-collection approach.
- ☐ use spreadsheet formulas such as `=SUM()`, `=AVERAGE()` and `=COUNTIF()` and calculate results correctly.
- ☐ choose a suitable tool (spreadsheet, presentation, infographic, report, dashboard) to present findings.
- ☐ plan a digital artefact using the ten-part standard structure.

## FINAL REVISION

# One-page recaps and a mock paper

Use these pages in the last week before the exam. Read one recap, close the booklet, and say it aloud. Then sit the mock paper in 2 hours 30 minutes with no notes.

امتحان سے ایک ہفتہ پہلے یہ خلاصے زبانی دہرائیں، پھر ماک پیپر بغیر نوٹس کے حل کریں۔

## Chapter 1 recap — Computer Systems

- Discrete = counted (stairs). Continuous = measured (ramp). Digital systems use bits 0 and 1; they are reliable, accurate, easy to design and programmable.
- Analog signal = smooth sine wave. Digital signal = square wave HIGH/LOW. Digital data resists noise.
- AND · output 1 only if all inputs 1. OR + output 1 if any input 1. NOT ' inverts. NAND = NOT-AND, NOR = NOT-OR (universal). XOR 1 when inputs differ. XNOR 1 when inputs same.
- Truth table rows =  $2^n$ . SOP = sum of minterms where output is 1. POS = product of maxterms where output is 0.
- K-map: group 1, 2, 4, 8 adjacent 1s (power of 2, wrap-around allowed). Each group drops the variables that change.
- SDLC: requirement → design → implementation → testing → deployment → maintenance. Waterfall = sequential; Agile = short sprints and feedback.
- OSI 7: Physical, Data Link, Network, Transport, Session, Presentation, Application. Mnemonic: Please Do Not Throw Sausage Pizza Away. TCP/IP 4: Network Access, Internet, Transport, Application.

## Chapter 2 recap — Computational Thinking & Algorithm

- CT pillars: decomposition, pattern recognition, abstraction, algorithms.
- Algorithm = finite, ordered, unambiguous steps. Pseudocode = structured English, not a programming language.
- Bubble Sort: compare neighbours, swap if out of order; largest "bubbles" to the end each pass.  $O(n^2)$ .
- Selection Sort: find the minimum in the unsorted part and swap it to the front.  $O(n^2)$ .
- Linear Search: check one by one. Works on unsorted data.  $O(n)$ .
- Binary Search: list MUST be sorted. Repeatedly test the middle.  $O(\log n)$ . Trace with low / mid / high.
- Judge an algorithm on correctness, efficiency (time and space) and clarity. Prefer binary search on large sorted lists; linear search on small or unsorted lists.

## Chapter 3 recap — Programming Fundamentals

---

- Python: high-level, interpreted. Variables are names; types int, float, str, bool, list, dict. `input()` returns str.
- Arithmetic `+` `-` `*` `/` `//` `%` `**`. Relational. Logical `and` `or` `not`. Bitwise `&` `|` `^` `~` `<<` `>>`.
- Selection: `if` / `if-else` / `if-elif-else` / nested. Test from the top; first True wins.
- `for` = known count + range. `while` = unknown count; always update the condition. `break` leaves; `continue` skips one.
- Libraries: `import math` / `random` / `datetime`. Third-party via `pip`: NumPy, Pandas, Matplotlib.
- Syntax = will not run. Runtime = crash + traceback. Logical = runs, wrong answer.

## Chapter 4 recap — Data and Analysis

---

- Data = raw facts. Information = processed data used for decisions. Database = organised related data. DBMS = software that manages it (reduces redundancy, improves security, sharing, integrity).
- Table / relation; row / record / tuple; column / field / attribute. Primary key uniquely identifies a row. Foreign key refers to another table's primary key.
- ER model: entity (rectangle), attribute (oval), relationship (diamond), cardinality 1:1, 1:M, M:N.
- Referential integrity: you cannot orphan a child row. Cascade Update / Cascade Delete copy or remove matching child rows.
- Access objects: tables, forms, queries, reports. Queries filter with criteria, wildcards (`*`, `?`), AND/OR. Totals: Count, Sum, Avg, Group By. Charts visualise the result.

## Chapter 5 recap — Application and Impacts of Computing

---

- IoT = connected devices. Six parts: sensor, actuator, connectivity, processing, cloud, UI. AI = machines that learn and decide. Analytics = collect → store → clean → analyse → visualise.
- Together: IoT (eyes) → Analytics (sort) → AI (brain) → actuator (action).
- Sources: primary (firsthand), secondary (explained by others), tertiary (summaries). No author / date / references → do not trust.
- Assistive tech: mobility, sensory, communication, cognitive. Goal: equal access, independence, inclusion.

## Chapter 6 recap — Digital Literacy

---

- Digital literacy = find, evaluate, create and share information with digital tools, safely.
- Qualitative = words/descriptions. Quantitative = numbers. Strategies: interview, survey, prototype, observation, simulation.
- Primary data you collect; secondary data already exists. Good survey questions are clear, single-idea, unbiased.
- Present with spreadsheets (formulas such as `=AVERAGE()`), slides, infographics and reports. Cite sources. Follow the 12-step inquiry project from question to artefact.

## Mock paper — Computer Science XI

---

Time: 2 hours 30 minutes | Total: 75 marks | Answer all questions. Calculators allowed. Write Python with correct indentation.



## Section A — Multiple choice (15 × 1 = 15)

1. Which quantity is discrete?
  - a. Room temperature
  - b. Number of desks
  - c. Car speed
  - d. Time of day
2. NAND is called universal because:
  - a. it has two inputs
  - b. any gate can be built from it
  - c. its output is always 1
  - d. it is faster than AND
3. The Boolean expression for XOR is:
  - a.  $A \cdot B$
  - b.  $A + B$
  - c.  $A \oplus B$
  - d.  $(A \cdot B)'$
4. OSI layer that chooses the best path (IP addresses) is:
  - a. Transport
  - b. Network
  - c. Data Link
  - d. Session
5. Which SDLC model delivers working software in short sprints?
  - a. Waterfall
  - b. Agile
  - c. Spiral only
  - d. Big Bang
6. Binary Search requires the list to be:
  - a. unsorted
  - b. sorted
  - c. all unique and unsorted
  - d. of even length
7. Worst-case time of Bubble Sort is:
  - a.  $O(n)$
  - b.  $O(\log n)$
  - c.  $O(n^2)$
  - d.  $O(1)$
8. `input()` in Python returns a:
  - a. int
  - b. str
  - c. float
  - d. bool
9. `17 // 5` equals:
  - a. 3.4
  - b. 3
  - c. 2
  - d. 3.0
10. A program that runs but prints the wrong total has a:
  - a. syntax error
  - b. runtime error
  - c. logical error
  - d. hardware fault
11. A field that uniquely identifies each record is a:
  - a. foreign key
  - b. primary key
  - c. composite description
  - d. query
12. An actuator in IoT:
  - a. collects temperature
  - b. opens a water valve
  - c. stores cloud files
  - d. draws a chart
13. Wikipedia is a:
  - a. primary source
  - b. secondary source
  - c. tertiary source
  - d. sensor
14. Marks out of 100 are:
  - a. qualitative data
  - b. quantitative data
  - c. tertiary data
  - d. metadata only
15. Which Access object is best for printing a formatted class list?
  - a. Table
  - b. Form
  - c. Query
  - d. Report

## Section B — Short questions (10 × 3 = 30)

1. Differentiate analog and digital signals with one example of each.
2. Draw the symbol and truth table of a 2-input NOR gate.
3. List the four pillars of computational thinking in one sentence each.
4. Why can you not use Binary Search on an unsorted attendance list?
5. Write a Python `if-elif-else` that prints Pass if marks  $\geq 40$ , else Fail.
6. Define primary key and foreign key.
7. Name the six components of IoT.
8. Classify: (a) a photograph you took, (b) a textbook, (c) an encyclopedia.
9. Give two good and two bad survey questions about mobile-phone use in college.
10. State three advantages of a DBMS over paper files.

## Section C — Long questions (3 × 10 = 30). Attempt any three.

1. Simplify  $F(A,B,C) = \Sigma(1,3,5,7)$  using a 3-variable K-map. Show the grouping and the final expression. Draw the logic diagram.
2. Dry-run Bubble Sort on the list [5, 1, 4, 2]. Show every pass. Then search for 4 using Binary Search on the sorted result, with a low/mid/high table.
3. Write a complete Python program that reads n subject marks, prints the average, and prints "A" if average  $\geq 80$ , "B" if  $\geq 70$ , "C" if  $\geq 60$ , else "D". Include a sample run of four marks: 80, 70, 90, 60.
4. Design an ER diagram and relational schema for a school library (Student, Book, Issue). Mark keys. Write one Access/SQL query that lists all books issued to roll number 101.
5. Explain how IoT, Data Analytics and AI work together on a Sindh farm. Then discuss two positive and two negative impacts of computing on education in Pakistan.

## Mock paper — answers

---

### Section A

1. b
2. b
3. c
4. b
5. b
6. b
7. c
8. b
9. b
10. c
11. b
12. b
13. c
14. b
15. d

## Section B

1. Analog: continuous, sine wave, e.g. human voice. Digital: two levels 0/1, square wave, e.g. computer data. Digital resists noise.
2. NOR symbol: OR-gate shape with a bubble on the output.  $Y = (A+B)'$ . Truth: 00→1, 01→0, 10→0, 11→0.
3. Decomposition = split a problem. Pattern recognition = reuse similarities. Abstraction = ignore extra detail. Algorithms = write ordered steps.
4. Binary Search repeatedly halves a sorted range. If the list is unsorted, the middle value does not tell you which half to keep, so the method is invalid.
5. `m = int(input("Marks: ")) then if m ≥ 40: print("Pass") else print("Fail").`
6. Primary key: unique identifier of a row (e.g. StudentID). Foreign key: a field that refers to the primary key of another table (e.g. StudentID in Issuance).
7. Sensors, actuators, connectivity, data processing, cloud computing, user interface.
8. (a) primary (b) secondary (c) tertiary.
9. Good: "How many hours per day do you use a phone for study?" / "Which apps do you use for class notes?" Bad: "Don't you agree phones are destroying students?" (leading) / "Do you like phones and also use laptops?" (double-barrelled).
10. Less redundancy, easier search and update, better security and multi-user access, integrity constraints. (Any three.)

## Section C — outline

1. K-map 1s at m1, m3, m5, m7 form one group of 4 covering B=1 (or C=1 depending on labelling — with standard ABC order the 1s are all cells where C=1).  $F = C$ . Diagram: a single wire C to output (or a buffer). Show the map with the group circled.
2. Pass 1: [1,4,2,5]; Pass 2: [1,2,4,5]; Pass 3: already sorted. Sorted [1,2,4,5]. Search 4: low=0 high=3 mid=1 (2<4) → low=2; mid=2 (4=4) found at index 2.
3. Read n, loop n times adding `float(input())`, avg = total/n, then if-elif for A/B/C/D. Sample: total 300, avg 75.0, grade B.
4. Entities Student(StudentID PK, Name, Class), Book(BookID PK, Title, Author), Issue(IssueID PK, StudentID FK, BookID FK, Date). Query: `SELECT Book.Title FROM Issue INNER JOIN Book ON Issue.BookID = Book.BookID WHERE StudentID = 101;`
5. Soil sensor (IoT) → cloud spreadsheet of moisture (Analytics) → model decides "too dry" (AI) → pump (actuator). Education +: LMS, remote classes. Education -: screen addiction, urban/rural device gap.

# Computer Science XII

Learn every chapter on your own — explanations, worked examples, memory tricks, practice questions and full answer keys.

Bilingual support: English + اردو

کمپیوٹر سائنس — بارہویں جماعت: خود سیکھیں، خود پرکھیں

01 Computer Systems (HCI)

02 Computational Thinking & Algorithms

03 Programming Fundamentals

04 Data and Analysis

05 Application and Impacts of Computing

06 Entrepreneurship in the Digital Age

XII

# Contents

## Chapter 1: Computer Systems (HCI)

- 1.1 Fundamentals of Human-Computer Interaction
    - 1.1.1 Traditional Interaction and 1.1.2 Natural Interaction
  - 1.2 Applications of HCI (Domains of Life)
  - Types of User Interaction
  - 1.4 Importance of HCI
  - 1.6 Need Analysis for Interface Design
  - 1.8 Methods for Improving HCI
  - 1.9.3–1.9.5 Wireframing, Figma and Prototypes
  - 1.11.2 Testing Methods (Usability, A/B, Automated)
  - 1.3 Components of HCI
  - Interface Types, Environment and Feedback
  - 1.5 Accessibility Principles
  - 1.7 Human-Computer Interaction Problems
  - 1.9 User Interface Design (UI vs UX)
  - 1.11 Test and Evaluate HCI (Evaluation Methods)
  - Career Opportunities in UI/UX
- 

## Chapter 2: Computational Thinking & Algorithms

- 2.1 Analyze algorithms for correctness
    - 2.1.1 Trace Tables
    - 2.1.2 Stepwise Reasoning
    - 2.1.3 Comparing Trace Tables and Stepwise Reasoning
  - 2.2 Evaluating the Clarity of an Algorithm
    - 2.2.1 Modularity
  - 2.2.2 Readability
  - 2.3.1 Number of Steps (Big O Notation)
  - 2.3.2 Number of Conditions
  - 2.3.3 Number of Repetitions
  - 2.3.4 The Efficiency Evaluation Framework
  - 2.4 Refinements to improve clarity and efficiency
  - 2.5 Concept of Data Structure
  - 2.5.1 Linear Data Structures: Array
    - Linear Data Structures: Linked List
  - Linear Data Structures: Stack
    - Linear Data Structures: Queue
  - 2.5.2 Non-Linear Data Structures: Tree
    - Non-Linear Data Structures: Graph
  - 2.5.3 Operations on Data Structures
  - 2.6 Identify data structures in problem scenarios
  - 2.7 Trace data retrieval in Array, List and Queue
  - Career Opportunities in Algorithms
- 

## Chapter 3: Programming Fundamentals

- 3.1 Data Structures in Python
    - 3.1.1 Lists (Ordered, Mutable, Methods)
    - 3.1.2 Tuples (Ordered, Immutable)
    - 3.1.3 Sets (Unordered, Unique, Math Operations)
    - 3.1.4 Dictionaries (Key-Value Pairs)
    - 3.1.5 Common Built-in Functions (min, max, len, sum)
    - 3.1.6 Activity: Student Attendance System
    - 3.1.7 Activity: Word Frequency Counter
  - 3.2 Functions in Python
    - 3.2.1 Types of Functions (Built-in vs User-defined)
    - 3.2.2 Functions with Return Values
    - 3.2.3 Activity: Arithmetic Calculator Using Functions
    - 3.2.4 Scope of Local and Global Variables
  - 3.3 & 3.4 File Handling and Operations (Modes r, w, a)
  - 3.6 The with Statement (Context Manager)
  - 3.8 Errors and Exceptions (Try-Except)
  - 3.9 Mini File Handling Project: Grade Tracker
  - Career Opportunities
-

## Chapter 4: Data and Analysis

- 4.1 Concept of Data Analysis
- 4.2 Creating a SQLite Database in Python
- 4.3.3 Handling Missing Values (NaN)
- 4.5 Data Visualization and Types of Graphs
- Career Opportunities
- 4.1.1 – 4.1.2 Data Sources and Database Connection
- 4.3 Pandas and DataFrames
- 4.4 Data Organization and Representation
- 4.6 Descriptive Statistics

## Chapter 5: Application and Impacts of Computing

- 5.1 Machine Learning, Neural Networks and Deep Learning
- 5.1.2 Neural Networks (NN)
- 5.1.4 Components of Neural Networks
- Future of AI, NN and DL in Pakistan
- 5.3.1 Secure Collaboration (Authentication, MFA, Access Control)
- 5.4.1 Common Security Threats (Malware, Phishing, DoS)
- 5.4.3 Apply Mitigation Techniques
- 5.5.1 Functions of Equity and Equal Access
- Career Opportunities
- 5.1.1 Machine Learning (ML)
- 5.1.3 Deep Learning (DL)
- 5.1.5 Components of Deep Learning
- 5.2 Applications of Neural Networks and Deep Learning
- 5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall)
- 5.4.2 Identifying Security Threats
- 5.5 Equity and Equal Access in Digital Collaboration
- 5.5.2 Collaboration Tools

## Chapter 6: Entrepreneurship in the Digital Age

- 6.1 Entrepreneur and Entrepreneurship
- Local Examples of Digital Entrepreneurship (Table 6.1)
- 6.4 Prototype
- 6.6 Prototype Development Cycle
- 6.8 Minimum Viable Product (MVP)
- 6.10 Identifying the Riskiest Assumption
- 6.12 Case Study: MVP for a Canteen Pre-Order System
- 6.14 Beachhead Market
- 6.16 Ethical and Safe Use of Digital Tools
- 6.2 Entrepreneurship in the Digital Age
- 6.3 From Problem to Business Idea
- 6.5 Types of Prototypes
- 6.7 Class Activity: Create a Prototype
- 6.9 Difference between Prototype and MVP
- 6.11 Developing an MVP
- 6.13 Testing an MVP
- 6.15 Complete Student Project
- Career Opportunities

**Final Revision:** one-page summaries, mock paper and answers

## How to use this booklet

This booklet is written so that you can learn every chapter **on your own**, without a teacher sitting next to you. Each topic explains the idea in simple English, shows a worked example, gives you a memory trick, warns you about common mistakes, explains it again in Urdu, and ends with quick questions so you can check that you really understood.

یہ کتابچہ اس طرح لکھا گیا ہے کہ آپ استاد کے بغیر خود ہر باب سیکھ سکیں۔ ہر موضوع پہلے آسان انگریزی میں سمجھایا گیا ہے، پھر حل شدہ مثال، یاد رکھنے کا طریقہ، عام غلطیاں، اردو میں وضاحت اور آخر میں خود کو پرکھنے کے سوالات دیے گئے ہیں۔

## What the coloured boxes mean

### LEARN IT

The main explanation. Read it slowly and make sure every sentence makes sense.

### WORKED EXAMPLE

A solved example, step by step. Cover the solution and try it yourself first.

### REMEMBER IT

A short trick, analogy or mnemonic that helps you recall the idea in the exam.

### COMMON MISTAKE

Mistakes students often make. Read these before every test.

اردو میں سمجھیں

اسی موضوع کی اردو میں آسان وضاحت۔

### EXAM TIP

How Golden topics are asked in board exams and what examiners look for.

### CHECK YOURSELF

Two or three quick questions. Answer them before looking at the answers underneath.

### GOLDEN TOPIC

★ **Golden** marks a high-yield topic that appears often in examinations. Give these topics extra time.

## The 5-step self-study method

1. **Preview (5 minutes).** Read the chapter opener and the "What you will learn" list. Turn each objective into a question you want answered.
2. **Learn one topic at a time.** Read the explanation, then work through the example with a pen and paper. Do not just read the solution — reproduce it.
3. **Check yourself straight away.** Answer the "Check yourself" questions from memory. If you get one wrong, re-read that topic before moving on.
4. **Practise at the end of the chapter.** Do the multiple-choice, short and long questions in exam conditions (no notes), then mark your work using the answer key.
5. **Review and repeat.** Tick the "I can..." checklist. Revisit anything you could not tick after 1 day, 3 days and 7 days. Spaced repetition makes knowledge stick.

## Tips for board exam answers

- **Define, explain, example.** A strong short answer gives a clear definition, one or two lines of explanation and a real-life example.
- **Use tables for "differentiate" questions.** Compare at least four points side by side.
- **Draw and label.** Logic gates, diagrams, charts and data structures earn marks when drawn neatly and labelled.
- **Show every step.** In K-maps, sorting traces, trace tables, statistics and code output questions, examiners give marks for the working, not only the final answer.
- **Write correct code.** Indent Python carefully, use meaningful variable names, and write the expected output under your program.

### CHAPTER 1

## Computer Systems (HCI)

This chapter teaches you Human-Computer Interaction (HCI): how people and computers talk to each other, how to design screens that are easy and fair to use, and how to test those designs before they go live. You will apply every idea to things you already use in Pakistan — a bank ATM, JazzCash, and a school portal.

اس باب میں آپ ہیومن کمپیوٹر انٹرایکشن (HCI) سیکھیں گے: انسان اور کمپیوٹر آپس میں کیسے بات کرتے ہیں، اسکرینیں کیسے آسان اور منصفانہ بنائی جاتی ہیں، اور انہیں عوام تک پہنچانے سے پہلے کیسے جانچا جاتا ہے۔

## What you will learn

- Define HCI (also called CHI and MMI) and name the five sensory channels of interaction.
- Describe HCI applications in health care, banking, education, and networking.
- Explain environment and the two kinds of feedback.
- Carry out a need analysis before designing an interface.
- Differentiate UI from UX, and compare wireframes, Figma work and prototypes.
- Differentiate traditional interaction from natural interaction with local examples.
- Name the components of HCI and classify types of user interaction and interface.
- State why HCI is important and apply five accessibility principles.
- Identify common HCI problems and methods for improving HCI.
- Choose evaluation and testing methods (SUS, NPS, usability, A/B, automated audits).

## 1.1 Fundamentals of Human-Computer Interaction

### LEARN IT

**Human-Computer Interaction (HCI)** is the study and design of technology so that it **serves people**. An HCI system is easy to use, efficient, and responsive to human needs. HCI is also known as **CHI** (Computer-Human Interface) or **MMI** (Man-Machine Interaction). The three names mean the same idea: the meeting point of a person and a machine.

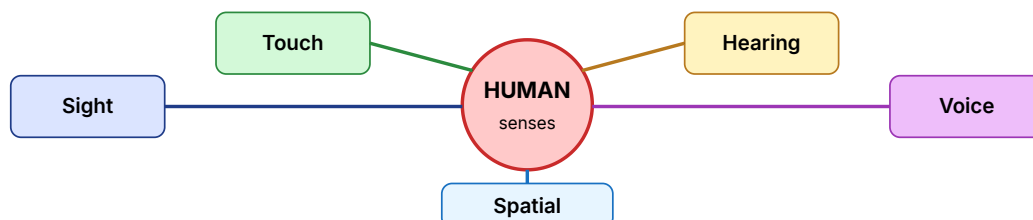
HCI is not only "making a pretty screen". It asks: Can a student in Hyderabad find results on the school portal in 30 seconds? Can a grandmother send JazzCash without help? Can a person standing at an outdoor ATM in bright sunlight read the buttons?

### Sensory channels of interaction

Humans and computers communicate through **sensory channels**. A sensory channel is a path that carries information to (or from) one of the human senses. Computers must send and receive signals on the same channels, or the conversation breaks.

| Channel                | How humans use it                      | How computers respond                                  |
|------------------------|----------------------------------------|--------------------------------------------------------|
| <b>Sight</b> (بصارت)   | Look at screens, graphics or videos.   | Displays visuals on monitors and screens.              |
| <b>Touch</b> (چھونا)   | Tap, type or swipe.                    | Responds with clicks or vibrations ( <b>haptics</b> ). |
| <b>Hearing</b> (سننا)  | Listen to sounds or voices.            | Provides audio feedback via speakers.                  |
| <b>Voice</b> (آواز)    | Speak commands (for example, "Alexa"). | Processes speech and replies.                          |
| <b>Spatial</b> (مکانی) | Move in physical space.                | Tracks position or body movement (for example, VR).    |

**Haptics** means physical feedback you can feel, such as a phone vibrating when a JazzCash payment succeeds. **Spatial** interaction means the system notices where your body is, as in a VR headset or a Kinect camera.



Five sensory channels meet at the human. HCI design must support the channels the user actually needs.



#### WORKED EXAMPLE

**Worked example: map a JazzCash payment to channels.** Sara in Sukkur sends Rs 500 to her brother.

1. **Sight:** she looks at the JazzCash home screen and the amount field.
2. **Touch:** she taps "Send Money" and types the mobile number.
3. **Voice (optional):** she could say "JazzCash, send five hundred rupees" if voice commands are on.
4. **Hearing + haptics:** a beep and a short vibration confirm "Payment successful".

If the app showed no tick, no sound and no vibration, Sara would not know whether the money left. That is an HCI failure, not a banking failure.

#### REMEMBER IT

**Remember the three names:** HCI = CHI = MMI. Think "Human meets Machine." Five channels: **S-T-H-V-S** — Sight, Touch, Hearing, Voice, Spatial. "See The Handy Voice in Space."

#### COMMON MISTAKE

**Mistake:** Writing "HCI is only about computer hardware" or mixing up Voice with Hearing.

**Fix:** HCI is about the *design of the conversation* between people and machines. Hearing is what you *listen to* (speakers). Voice is what you *speak* (microphone + speech software).

#### اردو میں سمجھیں

ہیومن کمپیوٹر انٹرایکشن (HCI) کا مطلب ہے ٹیکنالوجی کو اس طرح ڈیزائن کرنا کہ وہ انسانوں کے لیے فائدہ مند، آسان اور مؤثر ہو۔ اسے CHI یا MMI بھی کہتے ہیں۔ انسان اور کمپیوٹر پانچ حسّی راستوں سے بات کرتے ہیں: بصارت، چھونا، سنا، آواز اور مکانی حرکت۔ ہیپٹکس (Haptics) وہ ارتعاش ہے جو آپ کو چھو کر محسوس ہوتا ہے۔

#### CHECK YOURSELF

1. Give two other names for HCI.
2. Which channel tracks body movement in VR?
3. Is a phone vibration sight, touch (haptics), or voice?

Answers: 1. CHI (Computer-Human Interface) and MMI (Man-Machine Interaction). 2. Spatial. 3. Touch / haptics.

## 1.1.1 Traditional Interaction and 1.1.2 Natural Interaction

#### LEARN IT

**1.1.1 Traditional interaction** is the basic way people use everyday devices. It relies on standard input and output: a mouse, a keyboard, a keypad, a monitor, speakers, or a simple touchscreen tap and swipe.

- **Computer:** moving a mouse and typing on a keyboard; output via monitor and speakers.
- **Smartphone (traditional use):** tapping and swiping on a touchscreen; output via screen lighting and vibrations.

**1.1.2 Natural interaction** is advanced, smart technology that lets people interact **without a mouse or keyboard**. It feels more human — you speak, move, or show your face, and the system understands.

- **Voice:** speaking commands into a device (for example, "Alexa, play a song").
- **Body and gesture:** facial recognition to unlock a phone, or playing VR games by waving hands.

| Feature        | Traditional                                              | Natural                                          |
|----------------|----------------------------------------------------------|--------------------------------------------------|
| Main input     | Mouse, keyboard, keypad, simple tap/swipe                | Voice, face, hand/body gesture, movement         |
| Feels like     | Operating a machine                                      | Talking or moving as you do with people          |
| Examples       | TV remote, ATM keypad, school-portal login typed on a PC | FaceID, Siri, Kinect games, "Hey Google"         |
| Typical output | Screen, speakers, vibration                              | Spoken reply, unlock, tracking of the body in VR |

#### WORKED EXAMPLE

**Worked example: sort today's tools.** List three traditional and three natural interfaces you might use in one day in Sindh.

**Traditional:** (1) HBL ATM keypad to withdraw cash. (2) College computer lab: mouse + keyboard to open the school portal. (3) Television remote at home.

**Natural:** (1) Unlocking a phone with FaceID. (2) Asking Google Assistant "Call Aslam". (3) A VR science demo in which you pick a molecule by waving your hand.

Notice: the *same smartphone* can be traditional (you type a PIN) or natural (the camera reads your face). The classification depends on **how you interact**, not on the brand of the device.

#### REMEMBER IT

**Test sentence:** If you must press keys or tap labelled buttons, it is traditional. If the machine understands speech, face or body movement, it is natural.

#### COMMON MISTAKE

**Mistake:** "Touchscreens are always natural interaction."

**Fix:** Ordinary tapping and swiping on a phone is still listed as traditional in this chapter. Natural interaction starts when the system understands voice, face, or full-body gesture without a mouse or keyboard.

#### اردو میں سمجھیں

روایتی تعامل (Traditional Interaction) میں ماؤس، کی بورڈ، کی پیڈ یا عام ٹیپ اور سوائپ استعمال ہوتے ہیں، جیسے اے ٹی ایم کی پیڈ۔ قدرتی تعامل (Natural Interaction) میں آواز، چہرہ یا جسم کی حرکت سے کام ہوتا ہے، جیسے فیس آئی ڈی (FaceID) یا "الیکسا، گانا چلاؤ"۔ درجہ بندی آلے کی نہیں، تعامل کے طریقے کی ہوتی ہے۔

#### CHECK YOURSELF

1. Is an ATM keypad traditional or natural?
2. Give one voice example of natural interaction from the source.
3. Why can one smartphone be both traditional and natural?

Answers: 1. Traditional. 2. "Alexa, play a song" (or any spoken command). 3. Because typing a PIN is traditional, while FaceID or a voice assistant on the same phone is natural — it depends on the method, not the device.

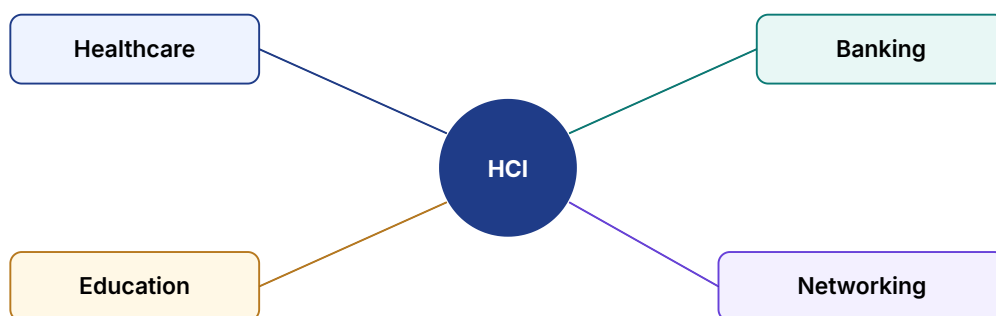
## 1.2 Applications of HCI (Domains of Life) ★ Golden

### LEARN IT

HCI is used to design and develop interactive systems such as **desktop applications, websites and mobile apps**. These are **human-centred solutions** built to meet user needs across many domains of life. A domain is an area of daily activity — health, money, school, talking to people.

When you design for a domain, you design for that domain's **users, tasks and risks**. A mistake on a games app is annoying. A mistake on a medical or banking screen can harm a person or lose money. That is why examiners treat this topic as golden.

| Domain                       | Application examples                                       | User benefit                                               |
|------------------------------|------------------------------------------------------------|------------------------------------------------------------|
| Health care                  | Patient portals, telemedicine apps, assistive technologies | Improves access to medical services remotely               |
| Banking                      | Mobile banking apps, secure ATM interfaces                 | Enables quick transactions and secure financial management |
| Education                    | E-learning platforms (LMS), online exam portals            | Enhances teaching, learning and student engagement         |
| Networking and communication | Social media, messaging apps (WhatsApp), Zoom              | Connects people globally through collaboration tools       |



HCI sits at the centre of four everyday domains: healthcare, banking, education and networking.

### WORKED EXAMPLE

**Worked example: the same HCI idea in three Pakistani systems.**

**Step 1 — ATM (banking).** You insert a card, the screen shows large buttons (Fast Cash, Balance, Other), you enter a PIN on the keypad, and you take cash. HCI benefit: a quick, secure transaction without a human teller.

**Step 2 — JazzCash (banking + communication).** You log in with a PIN or fingerprint, choose Send Money, type a number and amount, confirm, and see "Successful" with a JazzCash SMS. HCI benefit: money moves in seconds from a shop in Larkana to a student in Karachi.

**Step 3 — School portal (education).** A Grade XII student opens the college LMS, taps "Download date sheet", and submits a leave form. HCI benefit: learning and admin tasks happen without standing in a queue at the office.

**Step 4 — Compare the user benefit.** Banking HCI saves time and keeps money safe. Education HCI raises engagement. Health HCI (a Sehat/telemedicine app) brings a doctor to a village phone. Networking HCI (WhatsApp, Zoom) keeps a family in Dubai in touch with parents in Sukkur.

#### REMEMBER IT

**Exam mnemonic H-BEN:** Health, Banking, Education, Networking. For each, write: one application + one user benefit. That structure scores full marks on "applications of HCI".

#### COMMON MISTAKE

**Mistake:** Listing brand names only ("WhatsApp, JazzCash, Zoom") without saying the *domain* and the *user benefit*.

**Fix:** Always write three parts: domain → example → what the user gains. "Banking: JazzCash mobile app — enables quick, secure transfers."

#### اردو میں سمجھیں

ایچ سی آئی کی ایپلیکیشنز صحت، بینکنگ، تعلیم اور نیٹ ورکنگ کے شعبوں میں انسانی ضروریات پوری کرتی ہیں۔ صحت میں مریض پورٹل اور ٹیلی میڈیسن دور سے علاج دیتے ہیں، بینکنگ میں موبائل ایپ اور اے ٹی ایم تیز اور محفوظ لین دین دیتے ہیں، تعلیم میں ای لرننگ اور آن لائن امتحان مشغولیت بڑھاتے ہیں، اور واٹس ایپ یا زوم لوگوں کو جوڑتے ہیں۔

#### EXAM TIP

Very common 5–8 mark question: "Explain applications of HCI in different domains of life with examples." Draw a four-row table (Health care, Banking, Education, Networking & communication). Each row: one or two application names + the user benefit from the source. Add a local line (ATM / JazzCash / school portal) to show you can apply the idea. Do not write a story about only one app.

#### CHECK YOURSELF

1. Name the four domains of HCI given in the source.
2. What user benefit does HCI bring in banking?
3. Give one education application and one networking application.

Answers: 1. Health care, banking, education, networking and communication. 2. Quick transactions and secure financial management. 3. Education: LMS or online exam portal. Networking: social media, WhatsApp or Zoom.

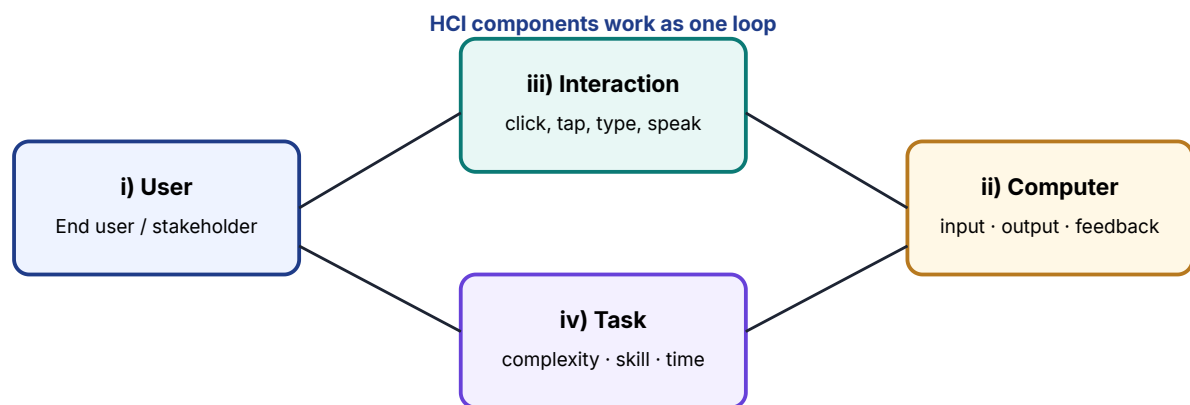
## 1.3 Components of HCI

#### LEARN IT

HCI relies on several primary components that work together. If one component is weak, the whole interaction feels broken. The first four are the people, the machine, the conversation, and the job to be done.

|   | Component   | Meaning                                                                                                                                                                                                                                                                                      |
|---|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| i | User (صارف) | Any individual or group that interacts with a computer system to achieve defined goals. Two categories: <b>end users</b> (people who directly operate the system, from beginners to experts) and <b>stakeholders</b> (people with an interest in the outcome, such as managers and clients). |

|     | Component                               | Meaning                                                                                                                                                                                                                                                                                                      |
|-----|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ii  | <b>Computer system</b> (کمپیوٹر سسٹم)   | The platform where humans and computers communicate through input, output and feedback using an interface. It connects human actions with machine responses.                                                                                                                                                 |
| iii | <b>User interaction</b> (صارف کا تعامل) | The process through which a user communicates with a system by performing actions (clicking, tapping, typing, speaking). It transforms user intent into system responses.                                                                                                                                    |
| iv  | <b>Task</b> (کام)                       | The specific action users perform (for example, booking a ticket). It has three characteristics: <b>task complexity</b> (filling a form vs designing a 3D model), <b>user knowledge</b> (basic typing vs advanced programming), and <b>time requirement</b> (ATM withdrawal vs online passport application). |



The user performs a task through interaction with a computer system. All four parts must fit together.

#### WORKED EXAMPLE

##### Worked example: label the four components of an HBL ATM withdrawal.

1. **User:** end user = the account holder standing at the machine; stakeholder = the bank that owns the ATM and must keep it safe and working.
2. **Computer system:** the ATM hardware plus the bank's software; it takes PIN input, shows a screen, and dispenses notes (output and feedback).
3. **User interaction:** insert card → type PIN → choose "Cash" → type amount → confirm. Intent ("I want Rs 5,000") becomes a machine response (cash + receipt).
4. **Task:** withdraw cash. Complexity: low (a few buttons). User knowledge: basic (PIN and amounts). Time: about one minute — much shorter than an online passport application.

#### REMEMBER IT

**U-C-I-T:** User, Computer system, Interaction, Task. "U Can Interact with Tasks." Then add three task questions: How hard? What skill? How long?

#### COMMON MISTAKE

**Mistake:** Calling the bank manager an "end user" of the ATM, or saying a task has only one property (difficulty).

**Fix:** The manager is usually a **stakeholder**. The person pressing the keys is the **end user**. A task always has complexity, knowledge and time — mention all three in long answers.

#### اردو میں سمجھیں

ایچ سی آئی کے چار بنیادی اجزاء ہیں: صارف (اینڈ یوزر جو خود سسٹم چلاتا ہے اور اسٹیک ہولڈر جس کا نتیجے میں مفاد ہے)، کمپیوٹر سسٹم (ان پٹ، آؤٹ پٹ اور فیڈبیک کا پلیٹ فارم)، صارف کا تعامل (کلک، ٹیپ، ٹائپ، بولنا) اور کام (ٹاسک) جس کی پیچیدگی، مطلوبہ مہارت اور وقت مختلف ہوتا ہے۔ اے ٹی ایم سے رقم نکالنا آسان اور مختصر کام ہے؛ پاسپورٹ کی آن لائن درخواست لمبا کام ہے۔

#### CHECK YOURSELF

1. What is the difference between an end user and a stakeholder?
2. Name the three characteristics of a task.
3. In a school portal, who is a stakeholder?

Answers: 1. End users directly operate the system; stakeholders have an interest in the outcome (managers, clients) but may not click the buttons. 2. Task complexity, user knowledge, time requirement. 3. The principal, IT admin or board office — they care about results even if students are the end users.

## Types of User Interaction

#### LEARN IT

User interaction can happen in many different ways depending on the technology used. Learn every type with one clean example — board questions often ask you to "list and explain" five or more.

| Interaction type    | Description                                                              | Examples                                                         |
|---------------------|--------------------------------------------------------------------------|------------------------------------------------------------------|
| Direct manipulation | Handling digital objects on a screen exactly like physical objects.      | Dragging files, zooming images with fingers                      |
| Command-based       | Giving precise instructions through typed text commands.                 | MS-DOS, Linux terminal ( <code>dir</code> , <code>cd ..</code> ) |
| Menu-driven         | Selecting from predefined lists or options instead of typing.            | Clicking "Start Test" or "Download" on a portal                  |
| Form fill           | Entering structured data into blank text fields.                         | Sign-up forms, ticket booking screens                            |
| Conversational      | Communicating with systems using natural human language (text or voice). | Chatbots, ChatGPT, Google Assistant                              |
| Gesture-based       | Using body or hand movements to control systems.                         | Swiping touchscreens, VR hand gestures                           |
| Voice interaction   | Speaking commands directly to the system.                                | "Call Aslam" on a smartphone                                     |
| Context-aware       | Systems sense the environment and act automatically without a prompt.    | Smart lighting turning on when you enter a room                  |
| Biometric           | Using biological features for interaction and authentication.            | Fingerprint unlock, FaceID                                       |
| Haptic and tactile  | Providing physical feedback to confirm actions.                          | Phone vibration when typing                                      |
| Multimodal          | Combining two or more interaction types together.                        | Voice + touch (speak a search, then tap)                         |

### WORKED EXAMPLE

**Worked example: a full JazzCash "Send Money" session, labelled by type.**

1. Open the app with a fingerprint → **biometric**.
2. Home screen shows Send Money, Bill Payment, Load → you tap Send Money → **menu-driven**.
3. Type the receiver's mobile number and the amount → **form fill**.
4. The keypad vibrates on each tap → **haptic**.
5. Optional: say "Send five hundred to Ali" then tap Confirm → **multimodal** (voice + touch).

Same idea on a **school portal**: menu-driven ("Date Sheet", "Fee Challan"), form fill (leave application), conversational (a chatbot "When is the chemistry practical?").

### REMEMBER IT

**Group them:** hands on screen (direct, gesture, haptic) · words (command, conversational, voice) · lists and blanks (menu, form) · automatic/body (context-aware, biometric) · mix (multimodal).

### COMMON MISTAKE

**Mistake:** Treating "voice interaction" and "conversational" as identical, or calling a dropdown menu "command-based".

**Fix:** Voice is spoken commands only. Conversational includes typed chat (ChatGPT) as well as voice. Command-based means you type precise codes such as `dir`, not clicking a menu.

### اردو میں سمجھیں

تعامل کی اقسام میں براہ راست ہیرا پھیری، کمانڈ، مینو، فارم، گفتگو (چیٹ بوٹ)، اشارہ، آواز، سیاق و سباق سے آگاہ نظام، بائیومیٹرک، بیپٹک اور ملٹی موڈل شامل ہیں۔ جاز کیش میں فنگر پرنٹ بائیومیٹرک ہے، "رقم بھیجیں" مینو ہے، نمبر لکھنا فارم ہے، اور آواز کے ساتھ ٹیپ ملٹی موڈل ہے۔

### CHECK YOURSELF

1. Which type is a Linux `cd ..` command?
2. Smart lighting that turns on when you enter a room is which type?
3. Why is speaking a search and then tapping a result called multimodal?

Answers: 1. Command-based. 2. Context-aware. 3. Because it combines two or more types (voice + touch) in one task.

# Interface Types, Environment and Feedback

## LEARN IT

Components v–vii complete the HCI picture: the look of the controls, the place where you use them, and the signals that tell you "it worked" or "try again".

### v) User interface (UI)

The **user interface** is the medium through which humans interact with computer systems.

- **Graphical User Interface (GUI):** visual elements such as windows, icons and menus (Windows OS, Android).
- **Command-Line Interface (CLI):** text-based typing (MS-DOS, Linux terminal).
- **Touch interface:** tapping, swiping, pinching (smartphones, tablets).
- **Voice User Interface (VUI):** spoken commands (smart speakers).
- **Natural User Interface (NUI):** gestures, body movements or immersive environments (VR headsets, Microsoft Kinect).

### vi) Environment

The **environment** is the real-world setting where a person uses a computer system. It is shaped by:

- **Physical conditions** — an ATM outdoors in sunlight, heat, noise, rain.
- **Social conditions** — a quiet classroom, a crowded bazaar, privacy at a PIN pad.
- **Technical hardware** — slow college Wi-Fi, a cracked phone screen, an old browser.

### vii) Feedback

**Feedback** is the signal or response given to confirm or guide actions.

#### System feedback

Response from the computer: a "Loading..." bar, a "Message Sent!" tick, or an error sound.

#### User feedback

Input from the user to the system: tapping a "Submit" button, giving a 5-star rating.

## WORKED EXAMPLE

### Worked example: environment and feedback at a roadside ATM in Hyderabad.

**Physical:** noon sunlight washes out a dim screen — the GUI must use high contrast.

**Social:** people stand in a queue; the user needs a privacy shield around the PIN pad.

**Technical:** the printer may be out of paper, so on-screen "Take your cash" plus a beep still has to confirm the transaction.

**System feedback:** "Please wait", cash coming out, a printed or SMS receipt.

**User feedback:** the user presses Confirm, or later rates the bank's app 4 stars.

On a **school portal** during board-exam week, hundreds of students on weak Wi-Fi need a visible "Loading results..." bar. Silence after clicking "View marks" is poor system feedback.

## REMEMBER IT

**GUI vs CLI vs NUI:** GUI = pictures you click. CLI = words you type. NUI = body you move. VUI = voice you speak. Touch = fingers on glass.

Feedback rule: **system talks back; user talks in.**



#### COMMON MISTAKE

**Mistake:** Mixing up system feedback with user feedback, or saying "environment" means only the software theme (dark mode).

**Fix:** A tick mark is system feedback. A 5-star rating is user feedback. Environment is the *real world* around the user — sun, noise, hardware — not the colour of the app (though a good UI will adapt to that world).

#### اردو میں سمجھیں

یوزر انٹرفیس وہ واسطہ ہے جس سے انسان سسٹم سے بات کرتا ہے: GUI تصاویر اور مینو، CLI لکھی کمانڈ، ٹچ، آواز (VUI) اور قدرتی اشارے (NUI). ماحول وہ حقیقی جگہ ہے جہاں سسٹم استعمال ہوتا ہے — دھوپ میں اے ٹی ایم، خاموش کلاس روم، کمزور وائی فائی۔ سسٹم فیڈبیک مشین کا جواب ہے ("بھیج دیا گیا")؛ صارف کا فیڈبیک صارف کا ان پٹ ہے (سمٹ بٹن یا پانچ ستارے)۔

#### CHECK YOURSELF

1. Name five types of user interface given in the source.
2. Give one physical and one social example of environment.
3. Is a "Message Sent" tick system feedback or user feedback?

Answers: 1. GUI, CLI, Touch, VUI, NUI. 2. Physical: ATM in sunlight. Social: a quiet classroom. 3. System feedback.

## 1.4 Importance of HCI ★ Golden

#### LEARN IT

HCI ensures that machines are approachable and usable, turning complex technology into everyday tools. **Good design** makes technology simple and enjoyable (touchscreens). **Poor design** causes confusion and risks (unclear buttons on medical devices).

This is why HCI is not optional decoration. A confusing JazzCash confirm button can send money to the wrong number. A confusing school-portal "Submit" can lock a student out of an online exam. Importance questions in the board exam expect the five implications below, each with a sentence of explanation.

| Implication               | Description                                                                                                                          |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 1. Enhance productivity   | Well-designed interfaces help users work faster, make fewer mistakes, and achieve goals efficiently.                                 |
| 2. Promotes inclusivity   | Accessibility features (screen readers, voice commands) empower marginalized groups and ensure equal use of technology for everyone. |
| 3. Drives innovation      | Emerging tech such as AR/VR, AI and IoT rely strictly on solid HCI principles for people to actually adopt and use them.             |
| 4. Economic impact        | User-friendly products gain higher adoption rates, customer loyalty and market success, driving profits.                             |
| 5. Ethical responsibility | Responsible design ensures technology respects user privacy, reduces bias and supports human well-being.                             |

#### WORKED EXAMPLE

##### Worked example: good vs poor HCI, same task.

**Task:** send Rs 2,000 through JazzCash to a parent.

**Good HCI:** large "Send Money" button, amount shown in words and numbers, last 4 digits of the number repeated, fingerprint confirm, green "Successful" plus SMS. Result: fast, few mistakes, the user trusts the app (productivity + economic loyalty + ethics of clear consent).

**Poor HCI:** tiny grey buttons, no amount-in-words, no confirm screen, no tick. Result: wrong number, no idea if it worked, user deletes the app and goes back to a bank queue.

**Medical contrast from the source:** unclear buttons on a medical device are not just ugly — they are dangerous. That is why ethical responsibility sits in this table.

#### REMEMBER IT

**P-I-I-E-E:** Productivity, Inclusivity, Innovation, Economic impact, Ethics. Sentence: "Please Include Innovative, Ethical Earnings."

#### COMMON MISTAKE

**Mistake:** Writing only "HCI is important because it makes apps look nice."

**Fix:** Looks are a tiny part of UI. Importance answers must name productivity, inclusivity, innovation, money and ethics. Quote the medical-device warning if you have time.

#### اردو میں سمجھیں

اچھا ایچ سی آئی پیچیدہ ٹیکنالوجی کو روزمرہ کا آسان اوزار بناتا ہے؛ برا ڈیزائن الجھن اور خطرہ پیدا کرتا ہے، جیسے طبی آلے پر غیر واضح بٹن۔ پانچ مضمرات: پیداوری بڑھتی ہے، شمولیت آتی ہے، جدت اپنائی جاتی ہے، معیشت مضبوط ہوتی ہے، اور رازداری و انصاف کی اخلاقی ذمہ داری پوری ہوتی ہے۔

#### EXAM TIP

Typical wording: "Why is HCI important? Discuss its implications." (8 marks) Write a short opening (good design vs poor design) then five numbered points with one example each. Local examples that earn marks: ATM PIN privacy (ethics), screen reader on a school portal (inclusivity), JazzCash one-tap send (productivity), a bank's app beating a rival because it is easier (economic), AR classroom app that students actually use (innovation). Do not skip ethics — examiners look for privacy and well-being.

#### CHECK YOURSELF

1. Give one example of poor design mentioned in the source.
2. Which implication is about screen readers and equal use?
3. How does HCI create economic impact?

Answers: 1. Unclear buttons on medical devices. 2. Promotes inclusivity. 3. User-friendly products get higher adoption, loyalty and profits.

## 1.5 Accessibility Principles

#### LEARN IT

**Accessibility principles** are HCI guidelines that ensure digital systems are usable by **all** individuals, including people with diverse physical, sensory and cognitive abilities. They remove barriers to create equal access.

Accessibility is not a "special extra". It is how you make sure a student with low vision, a shopkeeper who cannot use a mouse, or a colour-blind driver can still complete the task.

| Principle                                        | What it means                                                          | Example                                                                                    |
|--------------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1. High contrast (واضح رنگ)                      | Make text and pictures stand out clearly against the background.       | Black text on a white background in a railway app, readable in bright sunlight.            |
| 2. Readable fonts (پڑھنے میں آسان تحریر)         | Clear, simple fonts with proper size and spacing.                      | A school website using large Calibri so homework instructions are easy to read.            |
| 3. Colour-blind friendly design (رنگوں کی پہچان) | Do not depend <i>only</i> on colours; also use symbols or text labels. | A traffic app showing a red circle with a "STOP" label, not just a red colour.             |
| 4. Inclusive navigation (آسان نیویگیشن)          | Give users different ways to move around a system.                     | A website that lets you use the Tab key to jump between buttons if you cannot use a mouse. |
| 5. Multimodal detection (متعدد حسی ذرائع)        | Present information in different formats (sight, sound, touch).        | A video with text captions, or a phone vibrating to alert a user.                          |

#### WORKED EXAMPLE

##### Worked example: audit a school portal against the five principles.

1. **Contrast:** are result PDFs pale grey on white? Fail — use dark text.
2. **Fonts:** is the fee-challan note in 9-point decorative script? Fail — large Calibri (or a similar simple font), as the source recommends.
3. **Colour:** is "fail" only a red row with no word "Fail"? Fail — add the word, for colour-blind students.
4. **Navigation:** can a student who cannot use a mouse reach "Download date sheet" with Tab + Enter? If not, fail.
5. **Multimodal:** does the principal's video assembly have captions, and does a fee-due alert vibrate as well as flash? If not, deaf or busy students miss it.

The same five checks apply to an outdoor ATM (sunlight contrast) and to JazzCash (large numerals, Urdu labels, vibration on success).

#### REMEMBER IT

**C-F-C-N-M:** Contrast, Fonts, Colour-blind, Navigation, Multimodal. "Can Friends Catch New Messages?"

#### COMMON MISTAKE

**Mistake:** "Accessibility is only for people with disabilities, so our class portal does not need it."

**Fix:** The source says usable by **all** individuals. High contrast also helps in sunlight; captions help in a noisy bus; large fonts help tired eyes during board exams.

#### اردو میں سمجھیں

رسائی پذیری (Accessibility) کے اصول یقینی بناتے ہیں کہ ہر انسان سسٹم استعمال کر سکے، بشمول مختلف جسمانی، حسی اور ذہنی صلاحیتوں والے افراد۔ پانچ اصول: اونچا تضاد (کنٹراسٹ)، پڑھنے میں آسان فونٹ، صرف رنگ پر انحصار نہ کرنا، کی بورڈ سمیت متبادل نیویگیشن، اور معلومات کو نظر، آواز اور لمس کئی شکلوں میں دینا۔

### CHECK YOURSELF

1. Why is a red-only "STOP" sign a problem?
2. Which principle does the Tab key support?
3. Give the source's school-website font example.

Answers: 1. Colour-blind users may not see the meaning if there is no symbol or "STOP" text. 2. Inclusive navigation. 3. Large Calibri on a school website for homework instructions.

## 1.6 Need Analysis for Interface Design

### LEARN IT

**Need analysis** is the process of identifying the gap between the **existing situation** and the **desired outcome**. In HCI it is done **before** designing a UI, so that you understand users' goals, tasks, abilities and expectations.

If you skip need analysis, you guess. Guessing is how a school portal ends up with 40 menus and no "Download date sheet" button on the home screen.

### Steps in need analysis:



1. **Identify the users:** who exactly will use the system? (elderly people, teenagers, ...)
2. **Information collection:** gather data using interviews, questionnaires or observations.
3. **Analyse user needs:** identify the core problems, goals and requirements.
4. **Prioritise needs:** decide which features are the most critical to build first.
5. **Develop solutions:** design a system that meets those verified needs.

### WORKED EXAMPLE

**Case study from the source: mobile banking app.** Before building the app, the team interviews customers and discovers:

- Users want secure login via Fingerprint / FaceID.
- Elderly users need large buttons and simple navigation.
- Users want support for both English and local languages.

**Result:** the UI is designed around these proven needs, which makes success more likely.

### Apply the five steps to a college portal in Sindh.

1. Users: Grade XI–XII students, parents, clerks, teachers. Not "everyone in Pakistan".
2. Collect: interview 20 students; questionnaire to parents; watch a queue at the office on fee day.
3. Analyse: students cannot find date sheets; parents cannot read English-only fee notices; clerks re-type the same form.
4. Prioritise: (1) date sheet on home screen, (2) Urdu/Sindhi fee notice, (3) online leave form. Fancy animations wait.
5. Solution: a simple portal with those three features, large buttons, bilingual labels.

### REMEMBER IT

**Need analysis = gap finding.** Write: Existing = what hurts now. Desired = what "done" looks like. The UI must close that gap, not decorate it.

#### COMMON MISTAKE

**Mistake:** Starting in Figma on day one, then "asking users later".

**Fix:** Need analysis comes **before** UI design. Features you did not hear from users are guesses. Guessed features waste time and fail in exams when the question is "why do we analyse needs?"

#### اردو میں سمجھیں

نیڈ اینالیسس موجودہ صورتحال اور مطلوبہ نتیجے کے فرق کو یو آئی بنانے سے پہلے تلاش کرتا ہے۔ پانچ قدم: صارفین کی پہچان، انٹرویو یا مشاہدے سے معلومات، ضروریات کا تجزیہ، اہم خصوصیات کو ترجیح، پھر تصدیق شدہ ضرورتوں پر حل۔ موبائل بینکنگ کی مثال: فنگر پرنٹ لاگ ان، بڑے بٹن، اور انگریزی کے ساتھ مقامی زبان۔

#### CHECK YOURSELF

1. When in the design process is need analysis done?
2. List the five steps in order.
3. Name three needs found in the mobile-banking case study.

Answers: 1. Before designing the UI. 2. Identify users; collect information; analyse needs; prioritise; develop solutions. 3. Fingerprint/FaceID login; large buttons and simple navigation for elderly users; English plus local languages.

## 1.7 Human-Computer Interaction Problems ★ Golden

#### LEARN IT

**HCI problems** are common difficulties that stop users from using technology smoothly. They happen when systems fail to meet human needs or cognitive limits. **Cognitive limits** means the mind can only hold and process so much at once — if an app asks you to remember three SMS codes at the same time, it has gone past that limit.

Board answers score well when you name the problem, explain it in one line, and give a concrete example (the source examples are safe; a matching local example is even better).

| Problem              | Explanation and example                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interface complexity | Too many options or a confusing layout. Example: a shopping app with so many menus you cannot find the checkout button.                                |
| Poor feedback        | The system does not show if an action worked or failed. Example: clicking Submit and the screen freezes with no confirmation.                          |
| Inconsistency        | Different parts of the app look or act differently. Example: a magnifying-glass icon for Search on one page, a binocular icon on another.              |
| Accessibility gaps   | Design ignores people with disabilities. Example: a video tutorial platform without captions for deaf users.                                           |
| Compatibility issues | Interface fails on different devices or browsers. Example: a website that works on a laptop but crashes on a mobile browser.                           |
| Contextual neglect   | The system does not adjust to the user's situation. Example: a map app with no dark mode, blinding the driver at night.                                |
| High cognitive load  | Users must memorize too much or follow overly complex steps. Example: a registration system forcing you to remember three different SMS codes at once. |
| Trust deficiency     | The system feels unsafe or hides security cues. Example: an online store asking for credit cards but lacking an SSL/HTTPS lock icon.                   |

### WORKED EXAMPLE

#### Worked example: diagnose three local systems.

**ATM:** six tiny unmarked buttons beside the screen and no "Cash is coming" message → interface complexity + poor feedback. A user walks away while cash is still coming out.

**JazzCash:** "Send" is green on the home screen but orange on the confirm screen, and there is no padlock on the payment page → inconsistency + trust deficiency.

**School portal:** works in Chrome on a lab PC, blank on a student's Android browser; exam instructions are a caption-free video; login needs a portal password, an email OTP and an SMS OTP on the same screen → compatibility, accessibility gap, high cognitive load.

Once you can name the problem, Section 1.8 tells you how to fix it.

### REMEMBER IT

**Eight problems in two families:** (1) Screen problems — complexity, inconsistency, accessibility, compatibility, context. (2) Mind-and-trust problems — poor feedback, cognitive load, trust. If the user is lost, look at the screen family. If the user is scared or exhausted, look at the mind family.

### COMMON MISTAKE

**Mistake:** Writing "the app is bad" as an HCI problem, or mixing up cognitive load with compatibility.

**Fix:** Use the exact names from the table. Cognitive load is "too much to remember". Compatibility is "breaks on another device or browser".

### اردو میں سمجھیں

ایچ سی آئی مسائل وہ رکاوٹیں ہیں جو سسٹم کو انسانی ضرورت یا ذہنی حد کے مطابق نہ ہونے پر پیدا ہوتی ہیں۔ آٹھ مسائل: انٹرفیس کی پیچیدگی، کمزور فیڈبیک، عدم مطابقت (ایک جیسا نہ ہونا)، رسائی کی کمی، آلہ/براؤزر سے ناسازگاری، سیاق کو نظرانداز کرنا، زیادہ ذہنی بوجھ، اور اعتماد کی کمی (جیسے HTTPS تالے کے بغیر کارڈ مانگنا)۔

### EXAM TIP

Common 8-mark question: "Describe problems of HCI with examples." Aim for at least six of the eight, each in the pattern Name → one-line meaning → example. Memorise the source examples (checkout buried in menus, frozen Submit, binocular vs magnifying glass, no captions, laptop-only site, no dark mode, three SMS codes, missing HTTPS lock). Adding ATM / JazzCash / portal examples shows application and often gains the last marks.

### CHECK YOURSELF

1. Which problem is a missing HTTPS lock on a payment page?
2. Why are three SMS codes at once an HCI problem?
3. A site works on a laptop but crashes on a phone. Name the problem.

Answers: 1. Trust deficiency. 2. High cognitive load — users must memorize too much. 3. Compatibility issues.

## 1.8 Methods for Improving HCI

### LEARN IT

Once you can name an HCI problem, you fix it with a matching method. The source lists eight methods. Learn each with its tiny example — they pair almost one-to-one with Section 1.7.

- **Enhance usability:** simplify navigation (a clear "Buy Now" button).
- **Strengthen accessibility:** support disabilities (captions on videos).
- **Ensure consistency:** keep layouts uniform (the same "Save" button everywhere).
- **Provide clear feedback:** show action results ("Payment Successful" popup).
- **Improve adaptability:** responsive design (the website scales to mobile screens).
- **Apply user-centred design (UCD):** focus on real user goals and pilot-test.
- **Use multimodal interaction:** allow touch, voice or gesture inputs combined.
- **Continuous testing:** regularly update based on user feedback.

**User-centred design (UCD)** means the real user's goals drive every decision, and you trial (pilot-test) the design with those users instead of guessing in an office.

**Responsive design** means the layout rearranges itself for phone, tablet and PC, instead of staying a laptop-only page.

### WORKED EXAMPLE

**Worked example: match each JazzCash / portal problem to a fix.**

| Problem (1.7)                          | Method (1.8)                             |
|----------------------------------------|------------------------------------------|
| Cannot find "Send Money"               | Enhance usability — one clear button     |
| No captions on help video              | Strengthen accessibility                 |
| Search icon changes from page to page  | Ensure consistency                       |
| Submit freezes with no message         | Clear feedback — "Payment Successful"    |
| Portal crashes on mobile               | Improve adaptability — responsive layout |
| Built features nobody asked for        | UCD — interview, then pilot-test         |
| Elderly user cannot type a long number | Multimodal — voice + touch               |
| Complaints ignored after launch        | Continuous testing and updates           |

### REMEMBER IT

Think "problem → method" as a pair. Complexity ↔ usability. No captions ↔ accessibility. Different icons ↔ consistency. Freeze ↔ feedback. Phone crash ↔ adaptability. Guessing ↔ UCD. One input only ↔ multimodal. Launch and forget ↔ continuous testing.

### COMMON MISTAKE

**Mistake:** Naming methods without saying *what they change* ("we will improve HCI" is not an answer).

**Fix:** Always attach the source example: "Provide clear feedback, for example a Payment Successful popup."

### اردو میں سمجھیں

ایچ سی آئی بہتر بنانے کے طریقے: نیویگیشن آسان کرنا، رسائی بڑھانا (کیپشن)، ایک جیسا لے آؤٹ، واضح فیڈبیک ("ادائیگی کامیاب")، ریسپانسیو ڈیزائن، صارف مرکز ڈیزائن (UCD) اور پائلٹ ٹیسٹ، ملٹی موڈل ان پٹ، اور مسلسل جانچ۔ ہر مسئلے کا ایک جوڑی دار حل یاد رکھیں۔

### CHECK YOURSELF

1. What does UCD stand for, and what extra step does it include?
2. Which method stops a website from failing on a phone?
3. Give the source's example of clear feedback.

Answers: 1. User-Centred Design; focus on real user goals and pilot-test. 2. Improve adaptability (responsive design). 3. A "Payment Successful" popup.

## 1.9 User Interface Design (UI vs UX) ★ Golden

### LEARN IT

**UI design** is the process of creating and organizing the **visual elements** of a software application so users can interact with it easily and efficiently.

### 1.9.1 Components of a user interface

| Component                  | Role                                                                                                    |
|----------------------------|---------------------------------------------------------------------------------------------------------|
| Buttons                    | Clickable elements that perform actions (Submit, Cancel).                                               |
| Menus                      | Lists of options for navigating the system.                                                             |
| Icons                      | Small symbols representing tools or actions.                                                            |
| Text fields                | Boxes where users type information (passwords, search).                                                 |
| Checkboxes / radio buttons | Selecting options in a form. Checkboxes allow several choices; radio buttons allow only one in a group. |
| Sliders                    | Adjusting values along a bar (volume, brightness).                                                      |

### 1.9.2 User experience (UX)

**UX** is the overall experience a user has while interacting with a system — how smooth, comfortable and satisfying it feels.

**UI is what you see; UX is how it feels.**

### WORKED EXAMPLE

**Source example: food-delivery app.** The UI is the nice pictures of burgers and the red buttons. The UX is that the app loads fast, tracking works, and the food arrives without the user getting frustrated.

**Local walk-through: JazzCash.**

- **UI:** green Send button, JazzCash logo, amount text field, fingerprint icon, menu of Bill / Load / Send.
- **UX:** login in two seconds, no crash in a weak-signal bazaar, money arrives, SMS comes, the user feels safe and finished — or, if it hangs with no tick, the UX is bad even if the buttons look pretty.

**School portal:** UI = logo, "Login" button, date-sheet menu icon. UX = a student finds the chemistry practical date in under a minute on a phone, without calling the clerk.

Pretty UI + slow, confusing flow = poor UX. Plain UI + fast, clear flow = good UX.



#### REMEMBER IT

**Clothes vs day out.** UI is the outfit (colours, buttons, icons). UX is the whole day (Did you arrive on time? Did you get wet? Were you stressed?). Examiners love the line: *UI is what you see; UX is how it feels.*

#### COMMON MISTAKE

**Mistake:** Writing "UI and UX are the same" or "UX is just colours".

**Fix:** UI = visual components (buttons, menus, icons, fields, checkboxes, sliders). UX = overall smoothness, comfort and satisfaction, including speed and whether the task succeeds.

#### اردو میں سمجھیں

یو آئی ڈیزائن بصری عناصر — بٹن، مینو، آئیکن، ٹیکسٹ فیلڈ، چیک باکس/ریڈیو، سلائیڈر — کو ترتیب دینا ہے۔ یو ایکس پورا تجربہ ہے: کتنا ہموار، آرام دہ اور اطمینان بخش لگا۔ یو آئی وہ ہے جو آپ دیکھتے ہیں؛ یو ایکس وہ ہے جو آپ محسوس کرتے ہیں۔ کھانے کی ڈیلیوری ایپ میں برگر کی خوبصورت تصویر یو آئی ہے، تیز لوڈنگ اور وقت پر کھانا یو ایکس ہے۔

#### EXAM TIP

Highest-yield short question in the chapter: "Differentiate between UI and UX with an example." Write: (1) the two definitions, (2) the sentence "UI is what you see; UX is how it feels", (3) the food-delivery example from the source, (4) optionally JazzCash or the school portal. For 5 marks also list four UI components. Do not spend the answer describing Figma — that is the next topic.

#### CHECK YOURSELF

1. List six UI components from the source.
2. In the food-delivery example, what is UX?
3. Complete: "UI is what you see; UX is \_\_\_\_."

Answers: 1. Buttons, menus, icons, text fields, checkboxes/radio buttons, sliders. 2. The app loads fast, tracking works, food arrives without frustration. 3. how it feels.

## 1.9.3–1.9.5 Wireframing, Figma and Prototypes

#### LEARN IT

**1.9.3 Wireframing** is creating a basic visual outline or blueprint of a user interface **before** the final design. It plans the structure without focusing on colours or detailed graphics.

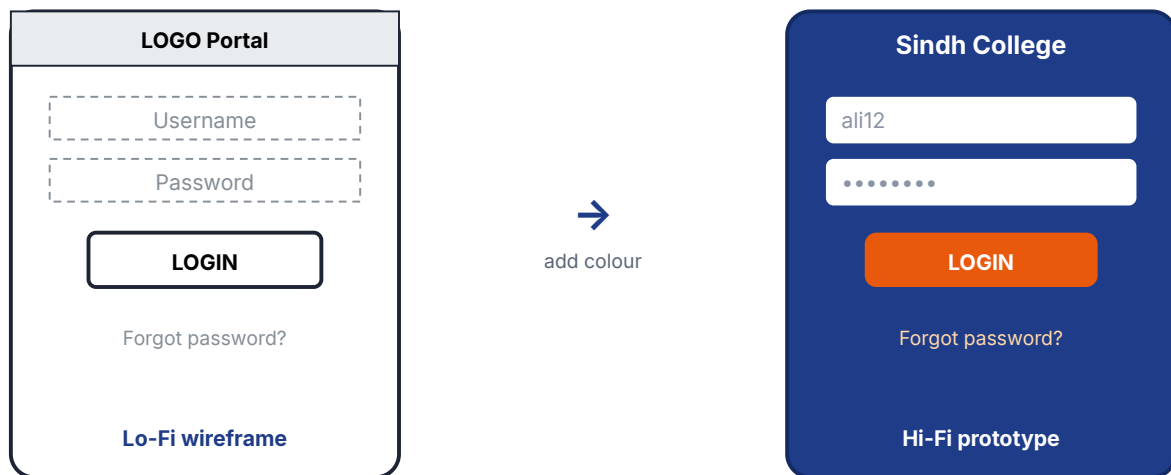
##### Low-fidelity wireframes (Lo-Fi)

A simple black-and-white sketch showing basic layout (boxes and lines). Used early to test and share ideas quickly. Focuses purely on structure and flow.

##### High-fidelity prototypes (Hi-Fi)

A detailed version that looks almost exactly like the final product. Includes real colours, fonts, images and interactive clickable buttons. Used for final user testing.

Lo-Fi sketch → Add details → Hi-Fi prototype



A school-portal login: Lo-Fi boxes and labels on the left; a coloured, realistic Hi-Fi prototype on the right.

## 1.10 What is Figma?

**Figma** is a popular, free, web-based tool used to design and collaborate on UI and prototypes. It works directly in a web browser — you do not have to install a heavy program on the college lab PCs.

### Creating a low-fidelity design in Figma

1. **Sign up:** access the Figma workspace online.
2. **Create frames:** select the Frame tool (for example an iPhone-sized frame) to act as your screen canvas.
3. **Add shapes:** draw basic rectangles for buttons and circles for avatars.
4. **Add text:** use the text tool to label buttons (for example, "Sign Up").
5. **Duplicate and edit:** copy the frame to create the next screen (for example, the Home Screen).

### 1.9.5 Prototype design

A **prototype** is an early interactive model used to test design ideas **before writing actual code**. In Figma you use the Prototype tab to link screens together.

- Set a **Trigger** (for example, "On Tap").
- Set an **Action** (for example, "Navigate to Home Screen").

Real users can then click through the fake app to see if the navigation makes sense.

#### WORKED EXAMPLE

##### Worked example: JazzCash-style "Send Money" in Figma, without writing code.

1. Lo-Fi on paper or in Figma: three frames — Home (boxes: Send, Bills, Load), Amount (text field + Next), Success (tick + Done).
2. Show the sketch to two classmates. If they cannot find Send, move that box to the top. This is the point of Lo-Fi: cheap changes.
3. In Figma, add a phone frame, rectangles, and labels. Duplicate the frame for Home, Amount and Success.
4. Prototype tab: Trigger = On Tap of the Send rectangle; Action = Navigate to Amount. On Tap of Next → Success.
5. Hand the laptop to a student. If they tap Bills by mistake, the navigation is still wrong — fix it before any programmer writes Java or Kotlin.

#### REMEMBER IT

**Lo-Fi = boxes. Hi-Fi = almost the real app. Prototype = clickable.** Figma can hold all three stages. Trigger = what the user does. Action = where the fake app goes.

#### COMMON MISTAKE

**Mistake:** Colouring a Lo-Fi wireframe, or thinking a Figma prototype is the finished coded app.

**Fix:** Lo-Fi stays black-and-white on purpose so people discuss structure, not taste in colours. A prototype tests ideas before code; developers still have to build the real system afterwards.

#### اردو میں سمجھیں

وائر فریم یو آئی کا بنیادی خاکہ ہے، رنگوں سے پہلے۔ لو فائی سیاہ و سفید ڈبے ہیں؛ ہائی فائی تقریباً حتمی شکل ہے، رنگ، فونٹ اور کلک ہونے والے بٹن کے ساتھ۔ فگما (Figma) مفت ویب ٹول ہے: فریم بناؤ، شکل اور متن ڈالو، فریم کاپی کرو۔ پروٹوٹائپ کوڈ سے پہلے کا چلنے والا نمونہ ہے جس میں ٹرگر (On Tap) اور ایکشن (اگلی اسکرین) جوڑے جاتے ہیں۔

#### CHECK YOURSELF

1. What does a Lo-Fi wireframe deliberately leave out?
2. Where does Figma run?
3. In Figma prototyping, what is a trigger? Give the source example.

Answers: 1. Colours and detailed graphics (it is structure only). 2. Directly in a web browser. 3. The user event that starts a move between screens, e.g. "On Tap".

## 1.11 Test and Evaluate HCI (Evaluation Methods)

#### LEARN IT

**Testing** checks if the product works properly (finding bugs). **Evaluation** determines whether it meets users' needs (usability). Together they improve the product before release.

A school portal can be "bug-free" (testing passed) and still unusable if students cannot find the date sheet (evaluation failed). You need both.

#### 1.11.1 Evaluation methods

| Method                       | Description                                                                                   |
|------------------------------|-----------------------------------------------------------------------------------------------|
| SUS (System Usability Scale) | A standard 10-item questionnaire used to calculate a score for how usable a system is.        |
| NPS (Net Promoter Score)     | Asks users: "How likely are you to recommend this app to a friend?" Measures user loyalty.    |
| Formative evaluation         | Done <b>during</b> development to collect early feedback and improve the design continuously. |
| Summative evaluation         | Done at the <b>end</b> of development to measure overall success against the original goals.  |
| Expert reviews               | Usability specialists review the app against established design standards (heuristics).       |

**Heuristics** are agreed rules of good design (for example "show system status", "be consistent"). An expert review checks the product against those rules without waiting for a huge user survey.

#### WORKED EXAMPLE

##### Worked example: when to use each method on a school portal.

1. Week 2 of building: show the Lo-Fi to five students, change the menu. That is **formative** evaluation.
2. A teacher who has studied HCI checks the screens against standard rules (Is feedback visible? Are labels consistent?). That is an **expert review**.
3. After the portal is "finished", 30 students fill the 10 SUS questions. A low score means it is still hard to use, even if there are no crashes.
4. Ask: "On a scale of 0–10, would you recommend this portal to a friend in another college?" That is **NPS** (loyalty).
5. Compare the finished portal with the original goals (date sheet in one tap, bilingual fee notice). That is **summative** evaluation.

#### REMEMBER IT

**Formative = forming it (during). Summative = summing it up (end).** SUS = 10 questions on usability. NPS = "Would you recommend it?" Expert = specialist vs heuristics.

#### COMMON MISTAKE

**Mistake:** Saying testing and evaluation are the same, or mixing formative with summative.

**Fix:** Testing = bugs. Evaluation = does it meet human needs. Formative is during; summative is at the end against original goals.

#### اردو میں سمجھیں

ٹیسٹنگ یہ دیکھتی ہے کہ پروڈکٹ ٹھیک چلتا ہے (بگز)؛ ایویلیوایشن یہ دیکھتی ہے کہ صارف کی ضرورت پوری ہوئی۔ SUS دس سوالوں والا استعمال کا اسکور ہے، NPS پوچھتا ہے کہ آپ یہ ایپ دوست کو کتنی تجویز کریں گے۔ فارمیٹو جانچ ترقی کے دوران ہوتی ہے، سمیٹو آخر میں اہداف کے خلاف۔ ماہر جائزہ بیورسٹکس (Heuristics) پر ہوتا ہے۔

#### CHECK YOURSELF

1. What is the difference between testing and evaluation?
2. How many items are on the SUS questionnaire?
3. When is summative evaluation done?

Answers: 1. Testing finds bugs / checks the product works; evaluation checks it meets users' needs (usability). 2. 10. 3. At the end of development, against the original goals.

## 1.11.2 Testing Methods (Usability, A/B, Automated) ★ Golden

### LEARN IT

Testing involves observing how the system behaves under specific conditions. Learn all seven methods. The three named in the heading — usability, A/B and automated — are the ones examiners ask most, but a complete answer also names the others.

1. **Usability testing:** observing real users while they perform realistic tasks, to measure efficiency and satisfaction. If the user gets stuck, designers know where to fix the UI.
2. **Task-based testing:** assigning a specific task (for example, "Find a product and add it to cart") and strictly measuring **completion time**, **success rate** and **number of errors**.
3. **Automated audits:** software bots scan websites to detect technical issues such as broken links, slow loading speeds or low colour contrast.
4. **A/B testing:** comparing two versions of an interface (A and B). Half the users see A, half see B. The one with more clicks or sign-ups wins.
5. **Compatibility testing:** verifying the system on different browsers (Chrome, Safari), devices (mobiles, tablets, PCs) and operating systems.
6. **Performance and load testing:** simulating high traffic (for example 10,000 users logging in at once) to see if speed drops or the server crashes.
7. **Keyboard and screen-reader walkthroughs:** navigating using *only* the keyboard (Tab/Enter) or assistive technologies, to check accessibility for users with disabilities.

#### Task-based

"Add to cart"  
Time: 12 s  
Success: Yes

#### A/B test

Version A | Version B  
half / half users  
more clicks wins

#### Automated audit

Bot scanning...  
3 broken links  
Low contrast

Three golden testing pictures: timed tasks, A versus B, and a bot that finds broken links and weak contrast.

### WORKED EXAMPLE

**Worked example: test JazzCash "Send Money" and a school portal with numbers.**

**Usability testing.** Sit beside five students. Task: "Send Rs 100 to this test number." Do not help. Note where they pause. If three people look for Confirm at the bottom and it is at the top, that is your UI fix.

**Task-based testing (numbers).** Same task for 10 students.

- Success: 8 of 10 finish = success rate  $(8 \div 10) \times 100 = 80\%$ .
- Times: 12, 14, 11, 40, 13, 12, 15, 50, 14, 12 seconds. The two slow times (40, 50) are the students who got stuck — they also made errors.
- Errors: extra taps, wrong field. Count them. High errors + high time = poor HCI, even if most people eventually succeed.

**A/B testing.** Version A: green "Send" at the top. Version B: large orange "Send" under the amount. 200 users each. If B gets 140 completed payments and A gets 90, B wins. You keep B.

**Automated audit.** A bot reports 3 broken "Fee challan" links and low contrast on the portal's grey-on-grey marks table. Fix those before the board-exam rush.

**Load testing.** On result day, 10,000 students log in at 9:00 a.m. If the server crashes, performance testing failed — the UI never even appears.

**Keyboard walkthrough.** Unplug the mouse. Can a student Tab to "Download date sheet" and press Enter? If focus jumps randomly, accessibility testing failed.

#### REMEMBER IT

**Task-based three numbers:** Time, Success, Errors. **A/B:** two versions, split users, the higher clicks/sign-ups win. **Bot:** links, speed, contrast. **Load:** many users at once. **Keyboard:** Tab and Enter only.

#### COMMON MISTAKE

**Mistake:** Calling A/B testing "showing both designs to the same user and asking which is prettier".

**Fix:** Half the users see only A, half see only B. You measure behaviour (clicks, sign-ups), not a taste vote. Also: an automated audit does not replace watching real users — bots cannot see that a student is confused.

#### اردو میں سمجھیں

سات ٹیسٹنگ طریقے: یوزیبلٹی (حقیقی صارف کو کام کرتے دیکھنا)، ٹاسک بیسڈ (وقت، کامیابی، غلطیاں ناپنا)، خودکار آڈٹ (بوت ٹوٹے لنک، سست رفتار، کم کنٹراسٹ ڈھونڈنے)، اے/بی ٹیسٹ (آدھا صارف اے، آدھا بی؛ زیادہ کلک والا جیتے)، کمپیٹیبلٹی (براؤزر اور آلہ)، لوڈ ٹیسٹ (مثلاً دس ہزار لاگ ان)، اور صرف کی بورڈ یا اسکرین ریڈر سے چل کر رسائی کی جانچ۔

#### EXAM TIP

Golden 8-mark question: "Explain testing methods used in HCI." Write seven short paragraphs or a seven-row table. For full marks, define usability testing (observe real users on realistic tasks), give the three task-based measures, describe automated audits with the three bot findings (broken links, slow load, low contrast), explain A/B with the half-and-half rule, then add compatibility, load (10,000 users), and keyboard/screen-reader walkthroughs. If the paper asks only "usability, A/B and automated", go deep on those three and still mention the rest in one closing line.

#### CHECK YOURSELF

1. Name the three measurements in task-based testing.
2. In A/B testing, who sees version A?
3. Give two issues an automated audit can detect.

Answers: 1. Completion time, success rate, number of errors. 2. Half of the users (the other half see B). 3. Any two of: broken links, slow loading, low colour contrast.

## Career Opportunities in UI/UX

#### LEARN IT

The same HCI skills you practised on ATMs, JazzCash and school portals are real jobs. Each role uses a different slice of this chapter.

| Career                   | What the person does                                                                |
|--------------------------|-------------------------------------------------------------------------------------|
| UX designer              | Researches user needs to ensure the app is intuitive and satisfying.                |
| UI designer              | Designs the visual layout (colours, typography, buttons) using tools such as Figma. |
| Interaction designer     | Focuses on the flow between screens, animations and gestures.                       |
| Usability researcher     | Runs SUS/NPS tests and A/B tests with real users to find flaws.                     |
| Accessibility specialist | Ensures products meet inclusive standards for people with disabilities.             |

#### WORKED EXAMPLE

##### Worked example: who does which JazzCash task?

- "Interview shopkeepers about sending money" → **UX designer**.
- "Pick the green, the typeface and the Send button in Figma" → **UI designer**.
- "On Tap Send, slide to Confirm, then Success" → **interaction designer**.
- "Run SUS and an A/B test of two Send buttons" → **usability researcher**.
- "Check contrast, captions and Tab order" → **accessibility specialist**.

#### REMEMBER IT

UI looks; UX asks; interaction connects screens; research measures; accessibility includes everyone.

#### COMMON MISTAKE

**Mistake:** "UI designer and UX designer are one job."

**Fix:** In small teams one person may wear both hats, but the source lists them separately: UX researches needs and satisfaction; UI draws the visual layout in tools like Figma.

#### اردو میں سمجھیں

اس باب سے ملازمتیں: یو ایکس ڈیزائنر صارف کی ضرورت تحقیق کرتا ہے، یو آئی ڈیزائنر فگما میں رنگ اور بٹن بناتا ہے، انٹریکشن ڈیزائنر اسکرینوں کا بہاؤ سنبھالتا ہے، یوزیبیلٹی ریسرچر SUS، NPS اور اے/بی ٹیسٹ چلاتا ہے، اور ایکسیسیبلٹی ماہر سب کے لیے رسائی یقینی بناتا ہے۔

#### CHECK YOURSELF

1. Who runs SUS, NPS and A/B tests?
2. Which career uses Figma for colours, type and buttons?

Answers: 1. Usability researcher. 2. UI designer.

## Chapter 1 Review

## Chapter summary

- **HCI** (also CHI, MMI) designs technology to be easy, efficient and responsive to people, using five sensory channels: sight, touch (haptics), hearing, voice and spatial.
- **Traditional interaction** uses mouse, keyboard, keypad or simple tap/swipe; **natural interaction** uses voice, face or body gesture without a mouse or keyboard.
- HCI applications cover **health care, banking, education, and networking/communication**, each with a clear user benefit (remote care, quick secure money, better learning, global connection).
- Four core components: **user** (end user vs stakeholder), **computer system**, **user interaction**, and **task** (complexity, knowledge, time).
- Interaction types include direct manipulation, command-based, menu-driven, form fill, conversational, gesture, voice, context-aware, biometric, haptic and multimodal.
- Interfaces: GUI, CLI, touch, VUI, NUI. **Environment** is physical, social and technical. **System feedback** is the machine's reply; **user feedback** is what the person sends in.
- HCI matters for productivity, inclusivity, innovation, economic impact and ethical responsibility. Poor design (unclear medical buttons) creates risk.
- Five accessibility principles: high contrast, readable fonts, colour-blind friendly labels, inclusive navigation, multimodal presentation.
- **Need analysis** finds the gap before UI design: identify users, collect data, analyse, prioritise, develop solutions (banking case: FaceID, large buttons, local language).
- Eight HCI problems: complexity, poor feedback, inconsistency, accessibility gaps, compatibility, contextual neglect, high cognitive load, trust deficiency.
- Improve HCI with usability, accessibility, consistency, clear feedback, adaptability (responsive design), UCD and pilot tests, multimodal input, and continuous testing.
- **UI** is the visual components (buttons, menus, icons, fields, checkboxes, sliders); **UX** is how the whole experience feels. UI is what you see; UX is how it feels.
- Lo-Fi wireframes plan structure; Hi-Fi prototypes look real; **Figma** is a free browser tool; prototypes use Trigger (On Tap) and Action (navigate) before code.
- Evaluation (SUS, NPS, formative, summative, expert/heuristic reviews) checks needs; testing (usability, task-based, automated, A/B, compatibility, load, keyboard/screen-reader) checks behaviour under conditions.



## Key terms

| Term                    | Meaning                                                                                                                    | اردو                             |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| HCI                     | Human-Computer Interaction; designing technology that is easy, efficient and responsive to people. Also called CHI or MMI. | انسان اور کمپیوٹر کا باہمی تعامل |
| Haptics                 | Physical feedback you can feel, such as a vibration.                                                                       | لمسی / ارتعاشی جواب              |
| Traditional interaction | Using standard input/output such as mouse, keyboard, keypad or simple tap and swipe.                                       | روایتی تعامل                     |
| Natural interaction     | Interacting without mouse or keyboard, using voice, face or body gesture.                                                  | قدرتی تعامل                      |
| End user                | A person who directly operates the system.                                                                                 | حتمی صارف                        |
| Stakeholder             | A person or group with an interest in the system's outcome (manager, client).                                              | فریق مفاد                        |
| GUI                     | Graphical User Interface using windows, icons and menus.                                                                   | تصویری صارف انٹرفیس              |
| CLI                     | Command-Line Interface using typed text commands.                                                                          | کمانڈ لائن انٹرفیس               |
| NUI                     | Natural User Interface using gestures, body movement or immersive environments (VR, Kinect).                               | قدرتی صارف انٹرفیس               |
| System feedback         | The computer's response that confirms or guides an action.                                                                 | نظام کا جواب                     |
| User feedback           | Input the user sends to the system (Submit click, star rating).                                                            | صارف کی رائے / ان پٹ             |
| Accessibility           | Making digital systems usable by all, including people with diverse abilities.                                             | رسائی پذیری                      |
| Need analysis           | Finding the gap between the current situation and the desired outcome before designing a UI.                               | ضروریات کا تجزیہ                 |
| Cognitive load          | How much the user must remember or process at once.                                                                        | ذہنی بوجھ                        |
| UCD                     | User-Centred Design: focus on real user goals and pilot-test.                                                              | صارف مرکز ڈیزائن                 |
| UI                      | The visual elements of an application (buttons, menus, icons, fields, sliders).                                            | صارف انٹرفیس                     |

| Term                 | Meaning                                                                                                  | اردو                            |
|----------------------|----------------------------------------------------------------------------------------------------------|---------------------------------|
| UX                   | The overall smoothness, comfort and satisfaction of using a system.                                      | صارف کا تجربہ                   |
| Wireframe            | A basic outline of a UI that plans structure without colours or detailed graphics.                       | انٹرفیس کا خاکہ                 |
| Lo-Fi                | Low-fidelity sketch of boxes and lines used early to test ideas.                                         | کم تفصیل والا خاکہ              |
| Hi-Fi                | High-fidelity prototype that looks almost like the final product.                                        | زیادہ تفصیل والا نمونہ          |
| Prototype            | An early interactive model used to test design ideas before writing code.                                | ابتدائی چلنے والا نمونہ         |
| Figma                | A free, web-based tool for UI design and prototypes.                                                     | فگما (ویب پر یو آئی ٹول)        |
| SUS                  | System Usability Scale: a 10-item questionnaire that scores how usable a system is.                      | نظام کی استعمال پذیری کا پیمانہ |
| NPS                  | Net Promoter Score: how likely a user is to recommend the app to a friend.                               | تجویز کا درجہ (وفاداری)         |
| Formative evaluation | Evaluation during development to improve the design continuously.                                        | دورانِ تعمیر جانچ               |
| Summative evaluation | Evaluation at the end, measured against original goals.                                                  | اختتامی جانچ                    |
| A/B testing          | Showing version A to half the users and version B to the other half; the better-performing version wins. | اے/بی موازنہ ٹیسٹ               |
| Automated audit      | Bots scanning for broken links, slow pages or low contrast.                                              | خودکار جانچ                     |
| Heuristic            | An established design standard used in expert reviews.                                                   | مستند ڈیزائن اصول               |

## Practice: multiple choice

1. HCI is also known as:
  - a. CPU and ALU
  - b. CHI or MMI
  - c. HTTP or HTML
  - d. GUI only
2. Which sensory channel tracks body movement in VR?
  - a. Sight
  - b. Hearing
  - c. Spatial
  - d. Voice
3. Unlocking a phone with FaceID is an example of:
  - a. natural interaction
  - b. command-based interaction only
  - c. traditional interaction
  - d. CLI
4. Which domain of HCI includes patient portals and telemedicine?
  - a. Banking
  - b. Education
  - c. Networking
  - d. Health care
5. A bank manager who never uses the ATM but cares that it works is a:
  - a. stakeholder
  - b. end user
  - c. haptic device
  - d. wireframe
6. Typing `dir` in MS-DOS is:
  - a. menu-driven
  - b. command-based
  - c. context-aware
  - d. biometric
7. A "Message Sent!" tick mark is:
  - a. user feedback
  - b. a stakeholder
  - c. need analysis
  - d. system feedback
8. Unclear buttons on a medical device mainly show that poor HCI:
  - a. only affects game apps
  - b. is a colour preference
  - c. causes confusion and risk
  - d. improves load testing
9. Not depending only on colour, but adding a "STOP" label, is:
  - a. colour-blind friendly design
  - b. load testing
  - c. CLI design
  - d. A/B testing
10. Need analysis for a UI should be done:
  - a. after the product is sold
  - b. before designing the UI
  - c. only by programmers
  - d. instead of testing
11. An online store that asks for a credit card but shows no HTTPS lock has:
  - a. high contrast
  - b. natural interaction
  - c. summative evaluation
  - d. trust deficiency
12. "UI is what you see; UX is how it feels" means:
  - a. UI and UX are identical
  - b. UX is only the logo colour
  - c. UI is code and UX is hardware
  - d. UI is visual elements; UX is the overall experience
13. A Lo-Fi wireframe should focus on:
  - a. final photographs and brand colours
  - b. structure and flow using boxes and lines
  - c. server load of 10,000 users
  - d. writing the finished Python code
14. The System Usability Scale (SUS) is:
  - a. a 10-item questionnaire that scores usability
  - b. a bot that finds broken links
  - c. a type of ATM keypad
  - d. a Figma trigger
15. In A/B testing:
  - a. every user sees both versions at once and votes for the prettier one
  - b. only experts review heuristics
  - c. half the users see A, half see B, and the version with more clicks or sign-ups wins
  - d. the server is crashed on purpose

## Practice: short questions

1. Define HCI. Give two other names for it and list the five sensory channels.
2. Differentiate traditional interaction from natural interaction with one example of each from daily Pakistani use.
3. State four domains of HCI. For each, give one application and one user benefit.
4. Name the four primary HCI components. Distinguish end user from stakeholder, and list the three characteristics of a task.
5. Define GUI, CLI, VUI and NUI, with one example of each.
6. Write five implications of the importance of HCI.
7. List the five accessibility principles with one example each.
8. What is need analysis? Write its five steps and three findings from the mobile-banking case study.
9. Differentiate UI from UX using the food-delivery example. Name six UI components.
10. What is A/B testing? How is it different from an automated audit?

## Practice: long / problem questions

1. Explain the components of HCI (user, computer system, user interaction, task, interface types, environment, feedback). Illustrate every part with a labelled walk-through of a person withdrawing cash from an outdoor ATM in Hyderabad.
2. Describe eight HCI problems with examples. Then match each problem to a method of improving HCI. Use JazzCash or a school portal wherever it makes the example clearer.
3. "UI is what you see; UX is how it feels." Explain UI design, its six components, and UX. Then describe Lo-Fi wireframes, Hi-Fi prototypes, the steps of making a low-fidelity design in Figma, and how a Figma prototype uses a trigger and an action. Sketch a Lo-Fi school-portal login.
4. Differentiate testing from evaluation. Explain SUS, NPS, formative evaluation, summative evaluation and expert reviews. Then explain usability testing, task-based testing (including the three measurements), automated audits, A/B testing, compatibility testing, performance/load testing, and keyboard/screen-reader walkthroughs. Apply three of these methods to a college portal on result day.
5. A team is designing a JazzCash-style "Send Money" screen for elderly users in Sukkur who speak Sindhi, stand in bright sunlight, and have never used a CLI. Carry out a need analysis (five steps), list accessibility principles you will apply, choose interaction types, and say which evaluation and testing methods you will use before launch and why.

## Answer key

---

### Multiple choice

1. b
2. c
3. a
4. d
5. a
6. b
7. d
8. c
9. a
10. b
11. d
12. d
13. b
14. a
15. c

## Short questions — model answers

1. HCI is the design of technology to serve people; systems that are easy to use, efficient and responsive. Also called CHI (Computer-Human Interface) and MMI (Man-Machine Interaction). Channels: sight, touch, hearing, voice, spatial.
2. Traditional uses standard I/O (mouse, keyboard, keypad, simple tap/swipe), e.g. an ATM keypad or typing on a school portal. Natural lets people interact without mouse or keyboard, using voice, face or gesture, e.g. FaceID or "Alexa, play a song".
3. Health care: patient portals / telemedicine — remote access to medical services. Banking: mobile banking and ATMs — quick, secure transactions. Education: LMS and exam portals — better teaching, learning and engagement. Networking: WhatsApp, Zoom, social media — global connection and collaboration.
4. User, computer system, user interaction, task. End users directly operate the system; stakeholders (managers, clients) have an interest in the outcome. Task characteristics: complexity, user knowledge, time requirement.
5. GUI: windows, icons, menus (Windows, Android). CLI: typed commands (MS-DOS, Linux). VUI: spoken commands (smart speakers). NUI: gestures, body, immersive VR/Kinect.
6. Enhance productivity; promote inclusivity; drive innovation (AR/VR, AI, IoT need solid HCI); economic impact (adoption, loyalty, profit); ethical responsibility (privacy, less bias, well-being).
7. High contrast (black text on white in a railway app); readable fonts (large Calibri on a school site); colour-blind friendly (STOP label, not colour alone); inclusive navigation (Tab key); multimodal detection (captions or vibration).
8. Need analysis finds the gap between the existing situation and the desired outcome, before UI design. Steps: identify users; collect information (interviews, questionnaires, observation); analyse needs; prioritise; develop solutions. Banking case: Fingerprint/FaceID login; large buttons and simple navigation for elderly users; English and local languages.
9. UI is the visual elements so people can interact easily; UX is the overall smoothness, comfort and satisfaction. Food app: UI = burger pictures and red buttons; UX = fast load, working tracking, food arrives without frustration. Components: buttons, menus, icons, text fields, checkboxes/radio buttons, sliders.
10. A/B testing shows version A to half the users and version B to the other half; the version with more clicks or sign-ups wins. An automated audit uses bots to scan for technical issues such as broken links, slow loading or low contrast; it does not split users between two designs.

## Long questions — model answers

- 1. Components through an outdoor ATM.** User: end user = account holder; stakeholder = bank. Computer system: ATM hardware and bank software linking PIN input to cash output. Interaction: insert card, type PIN, choose Cash, confirm — intent becomes a response. Task: withdraw cash; low complexity, basic knowledge, about one minute (contrast with a long passport application). Interface: GUI on screen + keypad (traditional); possibly fingerprint (biometric). Environment: physical sunlight so contrast must be high; social queue so a privacy shield is needed; technical printer or network may fail. System feedback: "Please wait", cash, receipt/SMS. User feedback: pressing Confirm. If any part is weak (glare, no "cash coming" message), HCI fails even if the bank account is correct.
- 2. Problems and methods.** Interface complexity (buried checkout / hidden Send Money) → enhance usability (clear Buy Now / Send). Poor feedback (frozen Submit) → Payment Successful popup. Inconsistency (magnifying glass vs binoculars, or Send changing colour) → same Save/Send control everywhere. Accessibility gaps (no captions) → strengthen accessibility. Compatibility (laptop-only portal) → responsive/adaptability. Contextual neglect (map with no night mode; ATM unreadable in sun) → design for environment. High cognitive load (three SMS codes) → simplify steps. Trust deficiency (no HTTPS lock on JazzCash-style payment) → show security cues. Also apply UCD with a pilot test, multimodal input (voice + touch for elderly users), and continuous testing after launch.
- 3. UI, UX, wireframe, Figma, prototype.** UI design organises visual elements. Six components: buttons (Submit/Cancel), menus, icons, text fields, checkboxes/radio, sliders. UX = overall experience; UI is what you see, UX is how it feels; food-delivery pictures vs fast delivery. Lo-Fi: black-and-white boxes for structure; Hi-Fi: real colour, type, images, clickable controls for final tests. Figma: sign up in the browser; Frame tool for a phone canvas; rectangles and circles; text labels such as Sign Up; duplicate frames for the next screen. Prototype: before code, Prototype tab, Trigger = On Tap, Action = Navigate to Home. Sketch: header "LOGO / Portal", dashed Username and Password boxes, a LOGIN rectangle, "Forgot password?" — the Lo-Fi login in this chapter.
- 4. Test vs evaluate, then methods.** Testing finds bugs / whether it works; evaluation checks whether it meets user needs. SUS: 10-item usability score. NPS: "recommend to a friend?" (loyalty). Formative: during development. Summative: at the end vs original goals. Expert reviews: specialists vs heuristics. Usability testing: watch real users on realistic tasks. Task-based: time, success rate, errors (e.g. Add to cart in 12 s, success yes). Automated audit: bots find broken links, slow load, low contrast. A/B: half/half, more clicks win. Compatibility: Chrome/Safari, phone/tablet/PC, OS. Load: 10,000 logins at once. Keyboard/screen-reader: Tab/Enter and assistive tech. Result-day portal: load-test the 9:00 a.m. rush; usability-test "find chemistry marks"; automated audit for broken result links and grey-on-grey contrast; optional A/B of two home-screen layouts.
- 5. Send Money for elderly users in Sukkur.** Need analysis: (1) users = elderly Sindhi speakers, not teenagers; (2) collect: interviews and watch them at a JazzCash shop agent; (3) analyse: fear of wrong number, cannot read small English, glare on the phone; (4) prioritise: huge Sindhi "رقم بھیجیں", amount in words, confirm screen, high contrast; (5) solution: that UI, not a CLI. Accessibility: high contrast for sunlight, large simple fonts, not colour-only status, Tab/large targets, vibration + spoken confirm (multimodal). Interaction: menu-driven + form fill + biometric + haptic; optional voice. Evaluate formatively on paper then in Figma; expert review against heuristics; SUS and NPS before launch. Test: usability and task-based ("send Rs 100") measuring time/success/errors; A/B two button sizes; compatibility on cheap Androids; keyboard if a kiosk is used; do not skip a confirm screen — trust and feedback are the main risks.

### Self-assessment: I can...

- ☐ define HCI, CHI and MMI and name the five sensory channels.
- ☐ describe HCI in health, banking, education and networking.
- ☐ name the types of user interaction and give an example of each.
- ☐ apply the five accessibility principles and carry out a need analysis.
- ☐ differentiate UI from UX and list six UI components.
- ☐ choose evaluation methods (SUS, NPS, formative, summative, expert).
- ☐ tell traditional interaction from natural interaction with local examples.
- ☐ label user, computer system, interaction, task, UI type, environment and feedback on an ATM, JazzCash or portal.
- ☐ explain why HCI is important using the five implications.
- ☐ name eight HCI problems and the methods that fix them.
- ☐ compare Lo-Fi, Hi-Fi, Figma steps and prototype triggers/actions.
- ☐ explain usability, task-based, automated, A/B, compatibility, load and keyboard tests.

## CHAPTER 2

# Computational Thinking & Algorithms

Class XI taught you how to write algorithms. This chapter teaches you how to **judge** them: is the algorithm correct, clear and efficient? You will then learn the main **data structures** — array, linked list, stack, queue, tree and graph — so you can store data the smart way and trace how it is retrieved.

گیارہویں جماعت میں آپ نے الگورتھم لکھنا سیکھا۔ اس باب میں آپ سیکھیں گے کہ الگورتھم درست، واضح اور تیز ہے یا نہیں، اور ایرے، لنکڈ لسٹ، سٹیک، کیو، ٹری اور گراف جیسے ڈیٹا سٹرکچرز میں ڈیٹا کیسے رکھا اور نکالا جاتا ہے۔

### What you will learn

- Define **correctness** and prove it with a complete **trace table**.
- Evaluate **clarity** using modularity (SRP, coupling, abstraction) and readability (names, flow, consistency).
- Explain how the number of **conditions** and **repetitions** changes efficiency, and apply the three-question efficiency framework.
- Define a **data structure** and classify it as linear or non-linear.
- Draw a tree and a graph, and name the five operations on data structures.
- Apply **stepwise reasoning** and compare it with a trace table on at least four points.
- Define **efficiency** in time and space, and state Big O for  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$  and  $O(n^2)$  with examples.
- Refine an algorithm for clearer names, modules, comments and fewer wasted steps.
- Draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- Choose a data structure for a real scenario and trace retrieval in an array, a linked list and a queue.

## 2.1 Analyze algorithms for correctness

### LEARN IT

An algorithm is **correct** if its **output matches the expected results** for a given input. Analysing correctness means showing that the algorithm produces the intended output for **all valid inputs**, not just one lucky example.

Two methods are used in this course:

You already met the idea of a dry run in Class XI. Here you learn to do it formally, and to choose the method that fits the size of the algorithm.

#### WORKED EXAMPLE

**Worked example: what “correct” means.** An algorithm is supposed to count how many of the heights 1.91, 2.01, 1.87, 1.57, 1.48 lie between 1.57 m and 1.91 m inclusive.

Expected output: **3** (1.91, 1.87 and 1.57 qualify; 2.01 is too tall; 1.48 is too short).

If a program prints 4, it **runs** but it is **not correct**. Correctness is about the right answer, not about “no crash”.

#### REMEMBER IT

Remember: **runs ≠ correct**. A logical error still produces output — just the wrong output. That is why we analyse correctness on paper before we trust the code.

#### COMMON MISTAKE

**Mistake:** testing only one “normal” input and calling the algorithm correct.

**Fix:** also try edge cases (empty list,  $N = 0$ , the exact boundary values 1.57 and 1.91). The definition says **all valid inputs**.

#### اردو میں سمجھیں

الگورتھم تب درست (Correct) سمجھا جاتا ہے جب دیے گئے ان پٹ پر اس کا آؤٹ پٹ وہی ہو جو ہونا چاہیے۔ صرف چل جانا کافی نہیں؛ منطقی غلطی سے غلط جواب بھی نکل سکتا ہے۔ درستگی جانچنے کے دو طریقے ہیں: ٹریس ٹیبل (Trace Table) اور مرحلہ وار استدلال (Stepwise Reasoning)۔

#### CHECK YOURSELF

1. When is an algorithm considered correct?
2. Name the two methods used to analyse correctness in this chapter.
3. Why is “the program ran without an error” not enough?

Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.

## 2.1.1 Trace Tables ★ Golden

#### LEARN IT

A **trace table** is a technique used to monitor how the values of variables change during the execution of an algorithm. It is a **paper execution** of the code — a manual walkthrough, also called a **dry run**. You act as the computer and fill in one row after another.

#### Purpose of trace tables

- **Dry running:** execute the algorithm line by line by hand.
- **Debugging:** find logical errors where the algorithm runs but the output is wrong.
- **Validation:** check that the algorithm does what it is intended to do.

#### Components of a trace table

- **Line / step number** — which line is being executed.
- **Variables** — one column for every variable.
- **Input / output** — user inputs and printed results.
- **Condition check** — True/False for IF, WHILE, FOR, and similar tests.



### WORKED EXAMPLE

**Worked example: soldiers who meet the height rule.** To qualify for training, a soldier's height must be between **1.57 and 1.91 metres, inclusive**.

Inputs: all soldiers' heights. Output: the count of eligible soldiers.

Step 1: Start

Step 2: SET Total = 0

Step 3: Set AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]

Step 4: FOR EACH height FROM AllHeights DO

Step 5: IF height  $\geq$  1.57 AND height  $\leq$  1.91 THEN

Step 6: SET Total = Total + 1

Step 7: END IF

Step 8: END FOR EACH

Step 9: Print Total

Step 10: End

Inclusive means the two boundary values **count**: 1.57 and 1.91 are eligible.

| Step | Total | Height | AllHeights (input)           | Height between 1.57 and 1.91? | Output |
|------|-------|--------|------------------------------|-------------------------------|--------|
| 2    | 0     |        |                              |                               |        |
| 3    | 0     |        | 1.91, 2.01, 1.87, 1.57, 1.48 |                               |        |
| 4    | 0     | 1.91   |                              |                               |        |
| 5    | 0     | 1.91   |                              | True                          |        |
| 6    | 1     | 1.91   |                              |                               |        |
| 4    | 1     | 2.01   |                              |                               |        |
| 5    | 1     | 2.01   |                              | False (skip Step 6)           |        |
| 4    | 1     | 1.87   |                              |                               |        |
| 5    | 1     | 1.87   |                              | True                          |        |
| 6    | 2     | 1.87   |                              |                               |        |
| 4    | 2     | 1.57   |                              |                               |        |
| 5    | 2     | 1.57   |                              | True (boundary)               |        |
| 6    | 3     | 1.57   |                              |                               |        |
| 4    | 3     | 1.48   |                              |                               |        |
| 5    | 3     | 1.48   |                              | False (skip Step 6)           |        |
| 9    | 3     |        |                              |                               | 3      |

Eligible: 1.91, 1.87, 1.57. Not eligible: 2.01 (above 1.91) and 1.48 (below 1.57). Printed Total = **3**.

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height  $\geq$  1.57 and height  $\leq$  1.91:
        Total = Total + 1
print(Total)
```

OUTPUT

3

#### REMEMBER IT

**How to fill a trace table without getting lost.** Write a new row every time a variable **changes** or a condition is **tested**. Copy unchanged values into the new row so you never guess. Circle the final output.

Mnemonic: **V-I-C-O** — Variables, Input, Condition, Output. If a column is missing, the table is incomplete.

#### COMMON MISTAKE

**Mistake 1:** treating 1.57 or 1.91 as out of range. The words “inclusive” and the operators  $\geq$  and  $\leq$  mean the boundaries **are allowed**.

**Mistake 2:** updating Total on a False condition. If the IF is False, skip the SET line and leave Total unchanged.

**Mistake 3:** writing only the last Total. Examiners award marks for the **changing values in the table**, not only for the printed 3.

#### اردو میں سمجھیں

ٹریس ٹیبل الگورتھم کا کاغذی اجرا (Dry Run) ہے۔ ہر قطار میں لکیر نمبر، متغیرات کی نئی قیمتیں، ان پٹ/آؤٹ پٹ اور شرط کا نتیجہ (True/False) لکھا جاتا ہے۔ فوجیوں کی مثال میں قد 1.57 سے 1.91 میٹر تک ہونا چاہیے، حدود سمیت؛ پانچ قدوں میں سے تین اہل ہیں اس لیے آؤٹ پٹ 3 ہے۔

#### EXAM TIP

Very common long question: “Dry-run the given algorithm using a trace table and state the output.” Draw columns for **every** variable plus a condition column. Show the loop once per item. Boundary values ( $\geq$ ,  $\leq$ , empty list) are favourite traps. Short questions ask: purpose of a trace table (dry run / debug / validate) and its components (line, variables, I/O, condition).

#### CHECK YOURSELF

1. List the four usual columns / components of a trace table.
2. In the soldier example, why is height 1.57 counted but 1.48 not counted?
3. What is printed, and how many times is Step 6 executed?

Answers: 1. Line/step number, variables, input/output, condition check. 2. 1.57 is the lower boundary (inclusive, so True); 1.48 is below 1.57 (False). 3. 3 is printed; Step 6 runs three times (for 1.91, 1.87 and 1.57).

## 2.1.2 Stepwise Reasoning

#### LEARN IT

**Stepwise reasoning** (also called **logic verification**) breaks an algorithm into logical sections and checks the **state** of the algorithm at each stage. A **state** is a snapshot of what is true about the variables at that moment (for example, “Sum currently holds the total of the numbers seen so far”).

You do **not** fill a row for every line. You think through the logic:

1. Analyse each step of the algorithm.
2. Check: does it **start in a valid state**?
3. Does it **handle all possible cases**?
4. Are the **conditions correct**?
5. If the loop is correct at the **start** of an iteration, does it remain correct at the **end**?

### WORKED EXAMPLE

#### Worked example: sum of the numbers from 1 to N.

Step 1: Start  
Step 2: Input N  
Step 3: SET Sum = 0  
Step 4: For i = 1 to N  
Step 5: Sum = Sum + i  
Step 6: END For  
Step 7: Print Sum  
Step 8: End

#### Stepwise reasoning breakdown

| Check            | What you verify                                                                |
|------------------|--------------------------------------------------------------------------------|
| Initial state    | Sum = 0 before the loop. Valid: nothing has been added yet.                    |
| Loop boundary    | The loop runs exactly from i = 1 to i = N. No missing number, no extra number. |
| Iteration        | Each i is added once: after k iterations, Sum = 1+2+...+k.                     |
| Final state      | When the loop ends, Sum holds 1+2+...+N, which is the required total.          |
| Edge case N = 0  | The loop body never runs, Sum stays 0. Correct (the sum of no numbers is 0).   |
| Check case N = 5 | Sum becomes 15, which matches $5 \times 6 / 2 = 15$ .                          |

```
N = 5
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT  
15

```
N = 0
Sum = 0
for i in range(1, N + 1):
    Sum = Sum + i
print(Sum)
```

OUTPUT  
0

### REMEMBER IT

Think of stepwise reasoning as a **teacher checking a proof**, not as a cashier counting coins one by one. You ask “is this block always true?” instead of “what is Sum when i = 17?”.

#### COMMON MISTAKE

**Mistake:** rewriting the whole trace table and calling it stepwise reasoning.

**Fix:** write short comments about **state** (start, loop, finish, edges). If you listed every  $i$  from 1 to 1000, you used the wrong method.

#### اردو میں سمجھیں

مرحلہ وار استدلال میں الگورتھم کو منطقی حصوں میں توڑ کر ہر مرحلے پر اس کی حالت (State) دیکھی جاتی ہے: کیا شروع درست ہے، کیا تمام صورتیں سنبھلی ہیں، کیا شرط صحیح ہے، اور کیا لوپ کے ہر چکر کے بعد بھی نتیجہ درست رہتا ہے۔ ایک سے  $N$  تک جمع کی مثال میں Sum شروع میں 0 ہے، لوپ 1 سے  $N$  تک چلتا ہے، اور  $N=0$  پر جواب 0 ہی رہتا ہے۔

#### CHECK YOURSELF

1. What other name is given to stepwise reasoning?
2. For the sum algorithm, why is  $\text{Sum} = 0$  a valid start?
3. What happens when  $N = 0$ , and is that correct?

Answers: 1. Logic verification. 2. Because no numbers have been added yet, so the running total should be zero. 3. The loop does not run and Sum remains 0, which is the correct sum of no numbers.

## 2.1.3 Comparing Trace Tables and Stepwise Reasoning

★ Golden

#### LEARN IT

Both methods verify correctness, but they fit different situations depending on the **complexity and length** of the algorithm.

| Feature         | Trace table                                                                       | Stepwise reasoning                                                    |
|-----------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Best for        | Small algorithm blocks, complex loops, and beginners learning to track variables. | Large systems, architectural logic, and abstract mathematical proofs. |
| Effort required | High. Very tedious and impractical if a loop runs 1,000 times.                    | Moderate. Needs strong logical deduction and abstract thinking.       |
| Primary goal    | Catching syntax misinterpretations or specific arithmetic errors line by line.    | Catching conceptual or structural flaws in the overall design.        |
| Level           | <b>Micro-level</b> (line by line). Checks exact values.                           | <b>Macro-level</b> (logic blocks). Checks overall state.              |

#### Trace table

Micro-level  
Line by line  
Exact values

#### Stepwise reasoning

Macro-level  
Logic blocks  
Overall state

Trace tables zoom in on values; stepwise reasoning zooms out on design.

### WORKED EXAMPLE

#### Worked example: which method would you pick?

1. **A 12-line loop that adds five heights.** Use a **trace table**. The loop is short, you are a beginner, and you need the exact Total.
2. **A billing system with 40 functions.** Use **stepwise reasoning**. You cannot fill 10,000 rows. You check each module's start state, end state and edge cases.
3. **A loop from  $i = 1$  to 1,000 that should produce  $\text{Sum} = 500500$ .** Do **not** write 1,000 rows. Argue: "Sum starts at 0; each  $i$  is added once; final  $\text{Sum} = n(n+1)/2 = 1000 \times 1001/2 = 500500$ ." That is stepwise reasoning. Then dry-run a tiny  $N$  (say 5) with a trace table if you want extra confidence.

### REMEMBER IT

**Micro vs macro.** Trace table = microscope (every cell). Stepwise reasoning = map (whole city). Beginners start with the microscope; designers need the map.

### COMMON MISTAKE

**Mistake:** saying one method is "better" in every question.

**Fix:** they have different jobs. For a short dry-run in the exam, draw the table. For "explain why the algorithm is correct for any  $N$ ", write stepwise reasoning.

### اردو میں سمجھیں

ٹریس ٹیبل خرد بینی (Micro) سطح پر کام کرتا ہے: ہر لائن اور ہر قیمت۔ مرحلہ وار استدلال بڑی (Macro) سطح پر ڈیزائن اور حالت چیک کرتا ہے۔ چھوٹے لوپ اور مبتدیوں کے لیے ٹیبل بہتر ہے؛ ہزار بار چلنے والے لوپ یا بڑے سسٹم کے لیے استدلال بہتر ہے۔

### EXAM TIP

Differentiate questions are almost guaranteed: "Compare a trace table with stepwise reasoning." Write a **four-row table** (best for, effort, primary goal, micro/macro). Add one sentence of advice: table for small loops and exact values; reasoning for large systems and proofs. Do not write "trace table is always better".

### CHECK YOURSELF

1. Which method is micro-level and checks exact values?
2. Why is a trace table a poor choice if a loop runs 1,000 times?
3. Which method is better for catching a structural flaw in a large design?

Answers: 1. Trace table. 2. It is very tedious and impractical (too many rows). 3. Stepwise reasoning (macro-level, conceptual / structural flaws).

## 2.2 Evaluating the Clarity of an Algorithm

### LEARN IT

**Clarity** means an algorithm is easy to **read, understand and modify**. In many cases, evaluating clarity is even more important than making the algorithm slightly faster. A clear algorithm is easier to **maintain, debug and extend** over time.

The two key aspects of clarity are:

Clarity



Modularity (parts)



Readability (text and flow)

#### WORKED EXAMPLE

**Worked example: speed versus clarity.** Algorithm A finds the class average in 8 confusing lines with variables  $a$ ,  $b$ ,  $c$ . Algorithm B uses two named functions and takes the same number of additions. Both are  $O(n)$ . Algorithm B is the better school solution because the next person (or you, in three months) can change it without breaking it.

#### REMEMBER IT

Ask two questions: **"Can I split this into parts?"** (modularity) and **"Can another student read it aloud and understand it?"** (readability).

#### COMMON MISTAKE

**Mistake:** treating a clever one-line trick as "better" than a clear five-line solution.

**Fix:** the source says clarity is often more important than speed. Board answers should praise meaningful names and modules, not compacted code.

#### اردو میں سمجھیں

وضاحت (Clarity) کا مطلب ہے الگورتھم پڑھنا، سمجھنا اور بدلنا آسان ہو۔ بہت سی صورتوں میں وضاحت رفتار سے بھی اہم ہے۔ اس کے دو حصے ہیں: ماڈیولریٹی (Modularity) یعنی کام کو حصوں میں بانٹنا، اور پڑھائی پن (Readability) یعنی متن اور بہاؤ صاف ہونا۔

#### CHECK YOURSELF

1. What does clarity mean?
2. Name the two key aspects of clarity.
3. Why can clarity matter more than a small speed gain?

Answers: 1. The algorithm is easy to read, understand and modify. 2. Modularity and readability. 3. Because a clear algorithm is easier to maintain, debug and extend over time.

## 2.2.1 Modularity

#### LEARN IT

**Modularity** means breaking an algorithm into smaller, independent parts such as **functions, classes or modules**. This makes the algorithm more organised and easier to work with.

#### How to evaluate modularity

1. **Single Responsibility Principle (SRP).** Each module should focus on **one** specific task. If a function calculates a sum, prints a report and saves a file, it violates SRP.
2. **Coupling.** Modules should be reusable in different contexts. Good modularity means **loose coupling**: a change in one part does not smash the others. Tight coupling is when two functions share so many hidden details that you cannot reuse one without the other.
3. **Abstraction.** A well-designed algorithm hides complex implementation details. You can call a sorting function without knowing whether it uses Bubble Sort or Selection Sort inside.

## WORKED EXAMPLE

### Worked example: average of an array.

#### Poor modularity (monoblock)

All logic sits in one block, so you cannot reuse the sum.

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Set total = 0
Step 5: Set avg = 0
Step 6: For I = 1 to n
Step 7:  total = total + A[I]
Step 8: End For
Step 9: avg = total / n
Step 10: Print avg
Step 11: End
```

#### Good modularity (functions)

Two clear functions: `CalculateSum()` and `FindAverage()`.

```
Step 1: Start
Step 2: Set A = [1, 2, 3, 4, 5]
Step 3: Set n = 5
Step 4: Print FindAverage(A, n)
Step 5: End

Function FindAverage(A, n)
    Sum = CalculateSum(A, n)
    Avg = Sum / n
    Return Avg
End Function

Function CalculateSum(A, n)
    Set Total = 0
    For I = 1 to n
        Total = Total + A[I]
    End For
    Return Total
End Function
```

SRP check: `CalculateSum` only adds; `FindAverage` only divides. Loose coupling: you can reuse `CalculateSum` to find a total without computing an average. Abstraction: the main algorithm just calls `FindAverage` and does not see the loop.

The source pseudocode numbers array positions from 1. Python numbers them from 0, so the loop is `for i in range(n)`.

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

A = [1, 2, 3, 4, 5]
n = 5
print(find_average(A, n))
```

#### OUTPUT

3.0

## REMEMBER IT

SRP in one line: **one function, one job**. If you cannot name the function with a short verb-phrase (`CalculateSum`, `FindAverage`), it is doing too much.

### COMMON MISTAKE

**Mistake:** copying the 1-based loop `for I = 1 to n: total = total + A[I]` into Python. Then `A[1]` skips the first item and `A[5]` crashes.

**Fix:** in Python use `for i in range(n)` so the indices are 0, 1, 2, 3, 4.

### اردو میں سمجھیں

ماڈیولریتی کا مطلب الگورتھم کو آزاد حصوں (فنکشن، کلاس، ماڈیول) میں توڑنا ہے۔ جانچ کے تین اصول: واحد ذمہ داری (SRP) یعنی ایک فنکشن ایک کام؛ ڈھیلی جڑت (Loose Coupling) تاکہ ایک حصہ بدلنے سے دوسرے نہ ٹوٹیں؛ اور تجرید (Abstraction) یعنی اندرونی تفصیل چھپا کر صرف کام منگوانا، جیسے سورٹ فنکشن بلائے بغیر یہ جانے کہ ببل ہے یا سلیکشن۔

### CHECK YOURSELF

1. What does SRP say each module should do?
2. In the good average algorithm, which function hides the loop?
3. Why is the monoblock harder to reuse?

Answers: 1. One specific task. 2. CalculateSum (FindAverage only divides; the main program only prints). 3. All logic is in one block, so you cannot call "just the sum" from another algorithm.

## 2.2.2 Readability

### LEARN IT

**Readability** is how easily another person can understand the purpose and logic of an algorithm **without running it**. It is about how clear the text and the flow are.

#### How to evaluate readability

1. **Naming conventions** — consistent rules for names. Good names make an algorithm easier to search and understand.  
Bad: `v = d / t`  
Good: `velocity = distance / time`
2. **Self-documentation** — the algorithm explains itself, so you need fewer comments. Meaningful names and clean formatting make the purpose obvious.
3. **Control-flow simplicity** — keep the logic straightforward. Avoid deep nesting (IF inside IF inside IF), limit extra conditions, and keep each function on one task.
4. **Consistency** — one style throughout: indentation, spacing, and casing (camelCase or snake\_case). Then the logic stands out instead of the mess.

Names → Self-doc → Simple flow → Consistent



### WORKED EXAMPLE

Worked example: the same formula, two styles.

```
# Hard to read
d = 100
t = 4
s = 20
v = d / t
if v > s:
    if t > 0:
        print(v)
```

OUTPUT  
25.0

```
# Readable
distance = 100
time = 4
velocity = distance / time
speed_limit = 20
if time > 0 and velocity > speed_limit:
    print(velocity)
```

OUTPUT  
25.0

The second version uses names, avoids an IF nested inside an IF, and keeps one consistent snake\_case style. You can understand it without running it. Both print 25.0, but only the second is clear.

### REMEMBER IT

Remember readability as **N-S-S-C**: **N**ames, **S**elf-documentation, **S**imple flow, **C**onsistency.

### COMMON MISTAKE

**Mistake:** dumping a comment on every line (`total = total + 1 # add one to total`).

**Fix:** if the name already says it, skip the comment. Comments should explain **why**, not repeat **what**.

اردو میں سمجھیں

پڑھائی پن یہ ہے کہ دوسرا شخص الگورتھم چلائے بغیر اس کا مقصد سمجھ لے۔ اچھے نام (`velocity = distance / time`)، خود بیان (کم تبصرے)، سادہ کنٹرول فلو (گہری IF کے بغیر) اور ایک جیسا انداز (`casing` اور `indentation`) اسے آسان بناتے ہیں۔

### CHECK YOURSELF

1. Rewrite `v = d / t` with readable names.
2. What is deep nesting, and why is it a readability problem?
3. Name two style habits that support consistency.

Answers: 1. `velocity = distance / time`. 2. IF inside IF inside IF (or similar); it makes the flow hard to follow. 3. Any two of: consistent indentation, spacing, and casing (`camelCase` or `snake_case`).

## 2.3 Assess Algorithm Efficiency

### LEARN IT

**Algorithm efficiency** is measured in terms of **time** and **space** — how fast it runs and how much memory it uses.

The key considerations for assessing efficiency are:

1. **Number of steps** (described with **Big O notation**).
2. **Number of conditions** (IF decisions).
3. **Number of repetitions** (loops).

Time (speed) + Space (memory) → Efficiency

### WORKED EXAMPLE

**Worked example: two ways to find a name in 1,000,000 sorted records.**

Linear search checks about 1,000,000 names in the worst case (time grows with  $n$ ). Binary search checks about 20 names (it halves the list each time). Both can be correct and clear. They are not equally efficient.

### REMEMBER IT

Efficiency is always “as the input grows”. An algorithm that feels instant on 10 items can freeze on 10 million items if it is  $O(n^2)$ .

### COMMON MISTAKE

**Mistake:** judging efficiency by how short the code looks.

**Fix:** a five-line nested loop can do  $n^2$  work. Count steps, conditions and repetitions — not lines of text.

اردو میں سمجھیں

کارکردگی (Efficiency) وقت (رفتار) اور جگہ (میموری) سے ناپی جاتی ہے۔ تین چیزیں دیکھی جاتی ہیں: قدموں کی تعداد (بگ او)، شرطوں کی تعداد، اور دہرائی (لوپ) کی تعداد۔ چھوٹے ان پٹ پر تیز لگنے والا الگورتھم بڑے ان پٹ پر سست پڑ سکتا ہے۔

### CHECK YOURSELF

1. Efficiency is measured in terms of which two resources?
2. List the three key considerations named in the source.

Answers: 1. Time (speed) and space (memory). 2. Number of steps (Big O), number of conditions, number of repetitions.

### 2.3.1 Number of Steps (Big O Notation) ★ Golden

### LEARN IT

When we discuss efficiency we ask how performance changes as the **input size  $n$**  increases. **Big O notation** is a mathematical method used to describe how the running time (or memory) **grows** as  $n$  grows. We ignore small constants and keep the most important term: if an algorithm does  $3n + 5$  steps we write  $O(n)$ .

In computer science, **log  $n$**  almost always means log base 2 (how many times you can halve  $n$  until you reach 1).

## Five common classes

| Class         | Name         | What happens when n grows                                                    | Typical example                                        | Grade       |
|---------------|--------------|------------------------------------------------------------------------------|--------------------------------------------------------|-------------|
| $O(1)$        | Constant     | Time stays the same for 10 items or 10 million.                              | Access <code>arr[0]</code> (or any known index).       | Excellent   |
| $O(\log n)$   | Logarithmic  | Time grows very slowly; the problem is halved each step.                     | Binary search on a sorted list.                        | Great       |
| $O(n)$        | Linear       | If n doubles, time roughly doubles.                                          | Linear search of an unsorted list.                     | Fair / Good |
| $O(n \log n)$ | Linearithmic | A little worse than linear; much better than $n^2$ . Common in good sorting. | Merge Sort (split in half, then merge).                | Good        |
| $O(n^2)$      | Quadratic    | Time grows with the square of n. Nested loops. Painful for large n.          | Bubble Sort; compare every item with every other item. | Poor        |

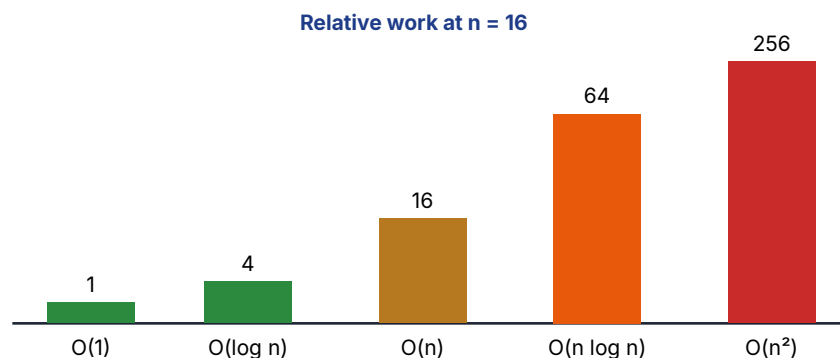
The source stresses  $O(1)$ ,  $O(\log n)$ ,  $O(n)$  and  $O(n^2)$ .  $O(n \log n)$  is the class of efficient general-purpose sorts, so you should recognise it when a question says "better than Bubble Sort but more than linear".

## Growth table (log is $\log_2$ )

Each cell is the approximate number of operations for that n. Read down a column to see how fast that class explodes.

| n         | $O(1)$ | $O(\log n)$ | $O(n)$    | $O(n \log n)$ | $O(n^2)$          |
|-----------|--------|-------------|-----------|---------------|-------------------|
| 2         | 1      | 1           | 2         | 2             | 4                 |
| 4         | 1      | 2           | 4         | 8             | 16                |
| 8         | 1      | 3           | 8         | 24            | 64                |
| 16        | 1      | 4           | 16        | 64            | 256               |
| 32        | 1      | 5           | 32        | 160           | 1,024             |
| 1,024     | 1      | 10          | 1,024     | 10,240        | 1,048,576         |
| 1,000,000 | 1      | ~20         | 1,000,000 | ~20,000,000   | 1,000,000,000,000 |

From  $n = 8$  to  $n = 1,000,000$ ,  $O(1)$  does not move,  $O(\log n)$  only goes from 3 to about 20,  $O(n)$  becomes a million, and  $O(n^2)$  becomes a million million — which is why nested loops "fail for large datasets".



Same  $n = 16$ . Quadratic work is already four times  $n \log n$  and sixteen times linear.

#### WORKED EXAMPLE

**Worked example A —  $O(1)$ : first element of an array.** Whether the array holds 10 or 10 million values, `arr[0]` is one jump to a known address.

```
arr = [23, 76, 12, 90, 34, 68]
print(arr[0])
```

```
OUTPUT
23
```

**Worked example B —  $O(n)$ : linear search.** Each extra item adds one extra check.

```
def linear_search(items, key):
    for i, value in enumerate(items):
        if value == key:
            return i
    return -1

print(linear_search([10, 50, 20, 70, 90, 30], 20))
print(linear_search([10, 50, 20, 70, 90, 30], 99))
```

```
OUTPUT
2
-1
```

**Worked example C —  $O(\log n)$ : binary search on a sorted list.** Each step throws away half of the remaining items. For 8 sorted numbers the worst case is about 4 comparisons ( $\text{floor}(\log_2 8) + 1$ ).

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps = steps + 1
        if arr[mid] == key:
            return mid, steps
        if arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, steps

data = [3, 8, 12, 17, 23, 31, 40, 52]
print(binary_search(data, 40))
print(binary_search(data, 3))
```

```
OUTPUT
(6, 3)
(0, 3)
```

Key 40 is found at index 6 after 3 comparisons. Key 3 is found at index 0 after 3 comparisons. Never more than a handful of steps, even if the list later grows to a million (about 20 steps).

**Worked example D —  $O(n \log n)$ : Merge Sort idea.** To sort 8 items, Merge Sort makes  $\log_2 8 = 3$  split-levels, and each level walks all 8 items once, so work  $\approx 8 \times 3 = 24$ , which matches the growth table. Bubble Sort on 8 items is on the order of  $8^2 = 64$  comparisons. That gap becomes enormous at  $n = 1,000,000$ .

**Worked example E —  $O(n^2)$ : nested loops.**

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

```
OUTPUT
16
```

Inner body runs  $n \times n = 16$  times. That is the pattern of Bubble Sort and of “compare every item with every other item”.

#### REMEMBER IT

**Order from fastest to slowest (for large n):**  $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$ .

Mnemonic: **"Constant Logs Never Need Squares"** — Constant, Logarithmic, N, N-log-N, Squares.

#### COMMON MISTAKE

**Mistake 1:** writing  $O(n)$  for binary search because "there is a loop". The loop runs about  $\log n$  times, not  $n$  times.

**Mistake 2:** writing  $O(2n)$  or  $O(n+5)$ . Big O drops constants: both are  $O(n)$ .

**Mistake 3:** using binary search on an **unsorted** list. Binary search is  $O(\log n)$  only on sorted data.

#### اردو میں سمجھیں

بگ او (Big O) بتاتا ہے کہ ان پٹ  $n$  بڑھنے پر وقت یا میموری کیسے بڑھتی ہے۔  $O(1)$  مستقل (ایرے کا  $arr[0]$ )،  $O(\log n)$  ہر قدم پر آدھا (بائنری سرچ)،  $O(n)$  لکیری (لینیئر سرچ)،  $O(n \log n)$  اچھے سارٹ جیسے مرچ سارٹ، اور  $O(n^2)$  مربع (بیل سارٹ، اندرونی لوپ)۔ بڑے  $n$  پر  $O(n^2)$  ناقابل استعمال ہو جاتا ہے۔

#### EXAM TIP

Memorise the five rows of the class table (name, growth, one example, grade). Typical MCQ: "Accessing  $arr[0]$  is ...?"  $\rightarrow O(1)$ . "Binary search is ...?"  $\rightarrow O(\log n)$ . "Nested loops / Bubble Sort ...?"  $\rightarrow O(n^2)$ . Long questions ask you to fill or interpret a growth table, or to say what happens if  $n$  becomes 10 times larger (linear  $\rightarrow 10\times$  time; quadratic  $\rightarrow 100\times$  time). Always mention that  $\log n$  is "halving" and  $n^2$  is "nested loops".

#### CHECK YOURSELF

1. Give one example each of  $O(1)$ ,  $O(\log n)$ ,  $O(n)$  and  $O(n^2)$ .
2. From the growth table, how many operations does  $O(n^2)$  need when  $n = 32$ ?
3. Why is Merge Sort described as  $O(n \log n)$ ?

Answers: 1.  $O(1)$   $arr[0]$ ;  $O(\log n)$  binary search;  $O(n)$  linear search;  $O(n^2)$  Bubble Sort / nested loops. 2. 1,024. 3. It makes about  $\log n$  split-levels and does about  $n$  work on each level, so  $n \times \log n$ .

## 2.3.2 Number of Conditions

#### LEARN IT

**Conditions** (IF statements) are decision points. Every condition forces the CPU to evaluate True or False, so excessive branching reduces performance.

- **Computational cost.** Each IF is extra work, even if the rest of the algorithm is simple.
- **Nested conditions.** IF inside IF often means a brute-force approach. It hurts both readability and speed.
- **Short-circuiting** (the optimisation goal). Stop evaluating as soon as the result is known. In a logical AND, if the first part is False the second part is skipped. In a logical OR, if the first part is True the second part is skipped.

#### WORKED EXAMPLE

**Worked example: cheap test first.** You only divide if the denominator is not zero. Place the cheap, safe test on the left so Python never evaluates the dangerous part.

```
x = 0
y = 10
if x != 0 and (y / x) > 2:
    print("ratio is large")
else:
    print("skipped safely")
```

#### OUTPUT

skipped safely

$x \neq 0$  is False, so AND short-circuits and  $y / x$  is never attempted. If you wrote the division first, the program would crash. Same idea in the soldier algorithm: a False height test skips Step 6.

#### REMEMBER IT

Put the **cheapest / most likely False** test first in an AND, and the **most likely True** test first in an OR. That is short-circuiting used on purpose.

#### COMMON MISTAKE

**Mistake:** writing a chain of nested IFs when one compound condition would do (as in the velocity example in 2.2.2).

**Fix:** flatten with AND/OR, and rely on short-circuiting instead of extra layers.

#### اردو میں سمجھیں

IF پروسیسر سے True/False پوچھتی ہے اس لیے زیادہ شاخیں کارکردگی گھٹاتی ہیں۔ اندرونی IF اکثر brute-force ہوتی ہے۔ شارٹ سرکٹ (Short-circuit) کا مطلب ہے جیسے ہی جواب معلوم ہو جانچ روک دی جائے، مثلاً AND میں پہلی شرط False ہو تو دوسری نہ چلے۔

#### CHECK YOURSELF

1. Why do extra IF statements reduce performance?
2. What does nested IF often indicate?
3. In  $x \neq 0$  and  $(y / x) > 2$  with  $x = 0$ , why is there no division-by-zero?

Answers: 1. Each condition must be evaluated True/False by the CPU. 2. A brute-force approach (and it reduces readability and speed). 3. AND short-circuits: the first part is False, so the division is never evaluated.

## 2.3.3 Number of Repetitions

#### LEARN IT

**Repetitions** (loops) are the most significant factor in algorithm efficiency. The **structure of the loop** decides how time grows with  $n$ .

| Type of repetition     | Conceptual impact                                                                   | Efficiency grade        |
|------------------------|-------------------------------------------------------------------------------------|-------------------------|
| Linear loop            | Processes each element exactly once. Time grows in proportion to size.              | Good — $O(n)$           |
| Logarithmic repetition | Reduces the problem size by half each iteration (binary search). Huge time savings. | Excellent — $O(\log n)$ |
| Nested loops           | For each element, process all other elements. Operations explode.                   | Poor — $O(n^2)$         |

#### WORKED EXAMPLE

**Worked example: count the inner-body runs.**

```
n = 5
linear = 0
for i in range(n):
    linear = linear + 1

nested = 0
for i in range(n):
    for j in range(n):
        nested = nested + 1

print(linear, nested)
```

**OUTPUT**  
5 25

One loop: 5. Nested loops:  $25 = 5^2$ . For  $n = 1,000$  that would be 1,000 versus 1,000,000.

#### REMEMBER IT

Find the **deepest loop**. One loop  $\approx O(n)$ . Loop inside a loop  $\approx O(n^2)$ . A loop that halves  $n \approx O(\log n)$ .

#### COMMON MISTAKE

**Mistake:** counting the word `for` and ignoring how the index moves.

**Fix:** a loop `while n > 1: n = n // 2` is logarithmic, even though it is still “a loop”.

اردو میں سمجھیں

دہرائی (لوپ) کارکردگی کا سب سے بڑا عامل ہے۔ ایک بار ہر عنصر پر چلنے والا لوپ  $O(n)$  (اچھا)، ہر بار مسئلہ آدھا کرنے والا  $O(\log n)$  (بہترین)، اور اندرونی لوپ  $O(n^2)$  (خراب) ہے۔ اندرونی جسم کے چلنے کی گنتی سے یہ فرق صاف نظر آتا ہے۔

#### CHECK YOURSELF

1. What grade does the source give nested loops?
2. Why is binary search's loop logarithmic rather than linear?
3. If  $n = 5$ , how many times does a fully nested  $n$ -by- $n$  pair of loops run?

Answers: 1. Poor —  $O(n^2)$ . 2. Each iteration halves the remaining problem, so the number of iterations is about  $\log n$ . 3. 25.

## 2.3.4 The Efficiency Evaluation Framework

### LEARN IT

To evaluate an algorithm at a conceptual level, ask three questions:

1. **Does it scale?** If  $n$  grows 10 times, does time jump 10 times (linear) or 100 times (quadratic)? Quadratic scaling fails for large datasets.
2. **What is the bottleneck?** Focus on the most time-consuming section (usually the deepest loop) and try to improve that part.
3. **Is the cost worth it?** A theoretically faster  $O(\log n)$  algorithm can be too complex to code and maintain. For a list of just 50 items, a simple  $O(n)$  linear search is practically better because it is simple.

Does it scale?



Where is the bottleneck?



Is the extra complexity worth it?

### WORKED EXAMPLE

Worked example: 50 names versus 1 million names.

| Situation                                          | Scale                                                                              | Bottleneck           | Worth the cost?                                                      | Choice                    |
|----------------------------------------------------|------------------------------------------------------------------------------------|----------------------|----------------------------------------------------------------------|---------------------------|
| 50 unsorted class names                            | Tiny $n$                                                                           | A single linear scan | Binary search needs a sort first, which is extra work and extra code | Linear search $O(n)$      |
| 1,000,000 sorted account numbers, searched all day | $n \times 10$ compared with 100,000 is already huge; quadratic would be impossible | The search loop      | Yes — binary search is standard and well understood                  | Binary search $O(\log n)$ |

### REMEMBER IT

Framework mnemonic: **S-B-W** — Scale, Bottleneck, Worth it.

### COMMON MISTAKE

**Mistake:** always choosing the theoretically fastest Big O.

**Fix:** question 3 exists for a reason. For  $n = 50$ , simple  $O(n)$  beats a fragile  $O(\log n)$  implementation.

اردو میں سمجھیں

کارکردگی کے تین سوال: کیا الگورتھم بڑے  $n$  پر چل سکتا ہے (Scale)؟ سب سے سست جگہ کون سی ہے (Bottleneck، عموماً گہرا لوپ)؟ کیا تیز تر طریقے کی پیچیدگی اس چھوٹے فائدے کے لائق ہے؟ پچاس ناموں کے لیے سادہ  $O(n)$  تلاش عملی طور پر بہتر ہو سکتی ہے۔

### CHECK YOURSELF

1. If  $n$  grows  $10\times$ , how much slower is a quadratic algorithm?
2. Where do you usually look for the bottleneck?
3. Why might linear search beat binary search for 50 items?

Answers: 1. About  $100\times$  slower ( $10^2$ ). 2. The most time-consuming section, usually the deepest loop. 3. Binary search is more complex to code and maintain, and for  $n = 50$  the simplicity of  $O(n)$  is practically better.



## 2.4 Refinements to improve clarity and efficiency

### LEARN IT

Improving an algorithm means reviewing it step by step: simplifying logic, removing redundant work, and picking smart data structures. A clear algorithm is easier to follow; an efficient one wastes fewer steps.

### 2.4.1 Refinements for clarity

1. **Use meaningful names.** Replace vague variables with descriptive ones. Bad: `a = b / c`. Good: `average = totalSum / totalItems`.
2. **Break into modules.** Divide a massive block into functions, for example separating `calculateSum()` from the printing logic.
3. **Improve structure and indentation.** Packed loops and IF statements confuse the reader. Spacing reduces mental effort.
4. **Add comments.** Keep them short. Explain **why** something is done, not just what is happening, especially for tricky logic.

### 2.4.2 Refinements for efficiency

1. **Reduce unnecessary steps.** In Bubble Sort, after the first pass the largest value has settled at the end. Ignore that sorted tail on the next pass by shrinking the inner loop, and you save comparisons.
2. **Use better algorithms.** Searching 1 million **sorted** records with Binary Search instead of Linear Search changes the time from seconds to milliseconds.
3. **Avoid re-computing values** (memoization / caching). If a value never changes, compute it once outside the loop, store it, and reuse it.
4. **Use appropriate data structures.** Direct access → array. Hierarchical data → tree. Do not force one structure everywhere.

### WORKED EXAMPLE

**Worked example 1 — shrinking Bubble Sort's inner loop (n = 5).** A naive inner loop always runs  $n-1$  = 4 times, for 4 passes: 16 comparisons. The refined inner loop runs 4, then 3, then 2, then 1: 10 comparisons.

```
n = 5
naive = 0
for i in range(n - 1):
    for j in range(n - 1):
        naive = naive + 1

refined = 0
for i in range(n - 1):
    for j in range(n - 1 - i):
        refined = refined + 1

print(naive, refined)
```

**OUTPUT**

16 10

**Worked example 2 — do not re-compute a constant inside the loop.**

```
A = 3
B = 4
N = 5

# BAD: C is recalculated every time
for i in range(1, N + 1):
    C = A + B
    print(C * i)
```

**OUTPUT**

7  
14  
21  
28  
35

```
A = 3
B = 4
N = 5
# GOOD: compute C once
C = A + B
for i in range(1, N + 1):
    print(C * i)
```

**OUTPUT**

7  
14  
21  
28  
35

Same output, fewer additions. That is caching: compute once, reuse N times.

**REMEMBER IT**

Clarity: **N-M-I-C** (Names, Modules, Indentation, Comments). Efficiency: **S-A-C-D** (Shrink steps, Algorithm choice, Cache, Data structure).

**COMMON MISTAKE**

**Mistake:** "optimising" by renaming variables and thinking that is an efficiency refinement.

**Fix:** meaningful names improve **clarity**. Efficiency refinements change the **amount of work** (fewer loop iterations, a better algorithm, no re-compute, a better structure).

**اردو میں سمجھیں**

وضاحت بہتر کرنے کے لیے معنی خیز نام، ماڈیول، درست indentation اور مختصر تبصرے استعمال کریں۔ کارکردگی کے لیے فضول قدم ہٹائیں (بیل سارٹ میں ترتیب شدہ دم نظر انداز کریں)، بہتر الگورتھم چنیں (لینینئر کی جگہ بائنری سرچ)، لوپ کے اندر وہی حساب بار بار نہ کریں (Caching/Memoization)، اور مناسب ڈیٹا سٹرکچر رکھیں۔

### CHECK YOURSELF

1. Rewrite  $a = b / c$  with meaningful names for an average.
2. After Bubble Sort's first pass, what should the next pass ignore?
3. In the A+B example, where should  $C = A + B$  be written?

Answers: 1.  $\text{average} = \text{totalSum} / \text{totalItems}$ . 2. The already-sorted largest value at the end (shrink the inner loop). 3. Once, outside the loop, then reuse C inside.

## 2.5 Concept of Data Structure ★ Golden

### LEARN IT

A **data structure** is a systematic way of **organising, storing and managing** data so that an algorithm can access, modify and process it efficiently.

**Real-life analogy.** Books on a shelf are arranged in a set order, so you can find a title quickly. Files in folders are grouped on purpose. Data structures do the same job inside computer memory.

### 1. Linear data structures

Elements stand in a strict sequence, one after another. Each element (except the first and last) has exactly **one predecessor** and **one successor**.

Examples: Array, Linked List, Stack, Queue.

### 2. Non-linear data structures

Elements are connected in a hierarchy or a network. One element can connect to **several** others at the same time.

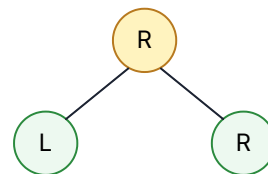
Examples: Tree, Graph.

Linear (sequence)



one after another

Non-linear (branch / net)



Linear: a line of neighbours. Non-linear: a node with more than one connection.

### WORKED EXAMPLE

**Worked example: classify each everyday picture.**

1. Students standing in a single assembly line → **linear** (each has one in front and one behind).
2. A family tree of grandparents, parents and children → **non-linear** (one parent, several children).
3. Roads linking Karachi, Hyderabad, Sukkur and Larkana in many ways → **non-linear** (a network).

### REMEMBER IT

Ask: "Does each item have at most one next item?" Yes → linear. "Can one item link to many, or can links form a net?" → non-linear.

### COMMON MISTAKE

**Mistake:** calling a tree "linear" because you can walk it from top to bottom.

**Fix:** walking is traversal. The **shape** is hierarchical: one parent, many children. That is non-linear.

ڈیٹا سٹرکچر ڈیٹا کو منظم طریقے سے رکھنے کا نظام ہے تاکہ الگورتھم اسے جلدی استعمال کر سکے۔ لکیری ڈھانچوں میں عناصر قطار میں ہوتے ہیں (ایرے، لنکڈ لسٹ، سٹیک، کیو)۔ غیر لکیری میں ایک عنصر کئی سے جڑ سکتا ہے (ٹری، گراف)۔ الماری کی کتابیں اور فولڈرز کی مثال یاد رکھیں۔

#### EXAM TIP

Define data structure in full (organise, store, manage; efficient access and processing). Then split linear vs non-linear with the "one predecessor and one successor" test, and list the four linear and two non-linear examples. The shelf / folder analogy earns a mark. MCQs: "Which is non-linear?" → Tree or Graph.

#### CHECK YOURSELF

1. Define data structure.
2. Name four linear and two non-linear examples.
3. What is true of each middle element in a linear structure?

Answers: 1. A systematic way of organising, storing and managing data for efficient access, modification and processing. 2. Linear: array, linked list, stack, queue. Non-linear: tree, graph. 3. It has exactly one predecessor and one successor.

## 2.5.1 Linear Data Structures: Array

#### LEARN IT

An **array** holds a collection of **similar items** (the same data type) placed next to each other in **contiguous memory** (one continuous block).

Each item has a position number called an **index**, usually starting from **0**. Because the address of any cell can be calculated from the index, you can jump straight to it. That is **direct access** (also called random access), and it is  $O(1)$ .

Array name: A

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 23 | 76 | 12 | 90 | 34 | 68 |
| 0  | 1  | 2  | 3  | 4  | 5  |

**A[3] = 90 (index starts at 0)**

Six integers in contiguous cells. Direct access: A[3] jumps to 90 without walking 23, 76 and 12.

#### WORKED EXAMPLE

**Worked example: read by index.**

```
A = [23, 76, 12, 90, 34, 68]
print(A[3])
print(A[0])
```

#### OUTPUT

```
90
23
```

### REMEMBER IT

Index 0 is the first shelf on the row. The third item that humans call “number 4” is index 3. Formula in your head: **position from the left, minus one.**

### COMMON MISTAKE

**Limitation.** Arrays are fast to read by index but **inflexible** to grow in the middle. Inserting a new value between 12 and 90 means shifting 90, 34 and 68 one cell to the right. That shift is  $O(n)$ , so arrays are a poor fit when you insert and delete in the middle all the time.

### اردو میں سمجھیں

ایرے ایک جیسے عناصر کو میموری کے لگاتار بلاک میں رکھتا ہے۔ ہر عنصر کا انڈیکس (عموماً 0 سے) ہوتا ہے جس سے براہ راست رسائی  $O(1)$  میں مل جاتی ہے، جیسے  $A[3] = 90$ ۔ درمیان میں نیا عنصر ڈالنے کے لیے دائیں طرف شفٹ کرنا پڑتا ہے جو سست ہے۔

### CHECK YOURSELF

1. What does “contiguous memory” mean?
2. In the diagram, what is  $A[2]$ ?
3. Why is inserting in the middle of an array inefficient?

Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.

## Linear Data Structures: Linked List

### LEARN IT

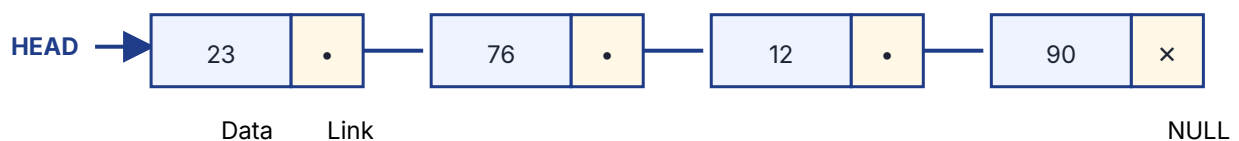
A **linked list** stores data as a chain. Unlike an array, the items are **not** one continuous block. Each separate piece is a **node** sitting somewhere in memory.

Each node has two parts:

1. the actual **data**;
2. a **link** (pointer) holding the memory address of the **next** node.

The first node is pointed to by **HEAD**. The last node's link is **NULL** (nowhere). To find an item you start at HEAD and follow links one by one — **sequential access**,  $O(n)$ .

**Real-life example:** browser history. Clicking Back follows the pointer to the previous page-node.



Follow HEAD → 23 → 76 → 12 → 90 → NULL

Four nodes. The last link is NULL, which ends the chain.

### WORKED EXAMPLE

Worked example: walk from HEAD to print every data field.

```
nodes = [23, 76, 12, 90]
for data in nodes:
    print(data, end=" → ")
print("NULL")
```

#### OUTPUT

23 → 76 → 12 → 90 → NULL

There is no `nodes[2]` jump in a real linked list. The loop above models following each link once. Inserting a new node between 76 and 12 only means changing two pointers — you do not shift the rest of a big block, which is why lists beat arrays for frequent insert/delete.

### REMEMBER IT

Array = row of lockers with numbers on the doors. Linked list = treasure hunt: each clue (link) tells you where the next box is.

### COMMON MISTAKE

**Mistake:** writing `list[3]` and calling it linked-list access.

**Fix:** a true linked list has no index. Python's `list` type is an array-like structure. In this chapter, "linked list" means HEAD-and-pointers.

### اردو میں سمجھیں

لنکڈ لسٹ زنجیر کی طرح نوڈز پر مشتمل ہے جو میموری میں بکھرے ہو سکتے ہیں۔ ہر نوڈ میں ڈیٹا اور اگلے نوڈ کا پتہ (Link) ہوتا ہے۔ پہلا نوڈ HEAD سے ملتا ہے، آخری کا لنک NULL ہوتا ہے۔ براؤزر کا Back بٹن پچھلے نوڈ کے پوائنٹر پر چلتا ہے۔ رسائی ترتیب وار ہے، انڈیکس سے چھلانگ نہیں۔

### CHECK YOURSELF

1. Name the two parts of a node.
2. What does the last node's link store?
3. Why is browser history a fair analogy?

Answers: 1. Data, and a link/pointer to the next node. 2. NULL (no next node). 3. Back follows the pointer to the previous page-node instead of jumping by index.

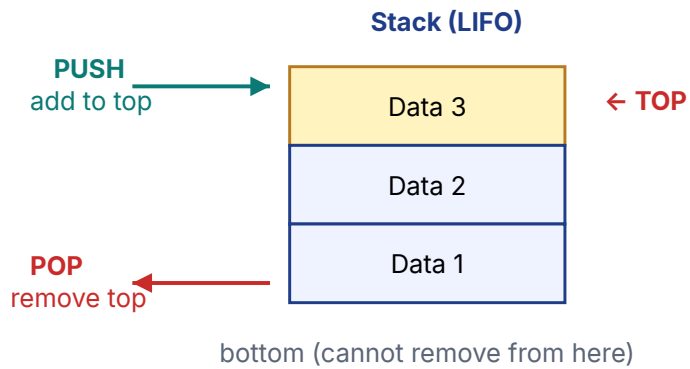
## Linear Data Structures: Stack ★ Golden

### LEARN IT

A **stack** is a linear data structure that works on the **LIFO** principle: **Last-In, First-Out**. The last element added is the first one removed. You cannot pull an item from the bottom or the middle.

- **PUSH** — add an item to the **top**.
- **POP** — remove the item from the **top**.

A **TOP** pointer always names the current top item. **Real-life example:** Undo in a text editor. Every action is piled on the stack; Undo POPs the most recent action first.



PUSH Data1, Data2, Data3 — first POP is Data3

Three PUSH operations. The TOP is Data 3, so the first POP returns Data 3.

#### WORKED EXAMPLE

**Worked example: PUSH / POP trace.** Start with an empty stack. Then PUSH Data1, PUSH Data2, PUSH Data3, POP, POP.

| Step | Operation  | Stack (bottom → top)  | TOP   | Popped value |
|------|------------|-----------------------|-------|--------------|
| 0    | (empty)    | []                    | none  |              |
| 1    | PUSH Data1 | [Data1]               | Data1 |              |
| 2    | PUSH Data2 | [Data1, Data2]        | Data2 |              |
| 3    | PUSH Data3 | [Data1, Data2, Data3] | Data3 |              |
| 4    | POP        | [Data1, Data2]        | Data2 | Data3        |
| 5    | POP        | [Data1]               | Data1 | Data2        |

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print("after 3 PUSHes:", stack)
print("POP:", stack.pop())
print("POP:", stack.pop())
print("left:", stack)
```

#### OUTPUT

```
after 3 PUSHes: ['Data1', 'Data2', 'Data3']
POP: Data3
POP: Data2
left: ['Data1']
```

Python's `list.append` is PUSH and `list.pop()` (no index) is POP from the top.

#### REMEMBER IT

**LIFO = Last In First Out = "last plate washed is first plate used"** on a canteen pile. Undo, Redo, Back button and function calls are all stacks.

### COMMON MISTAKE

**Mistake:** POPing from the bottom, or mixing LIFO with FIFO.

**Fix:** stack = plates (top only). Queue = ticket line (front leaves, rear joins). If the first POP is Data1, you drew a queue, not a stack.

### اردو میں سمجھیں

سٹیک LIFO پر چلتا ہے: جو سب سے بعد میں آیا وہ سب سے پہلے نکلے گا۔ اوپر رکھنا PUSH، اوپر سے ہٹانا POP، اور TOP پوائنٹر موجودہ سب سے اوپر عنصر بتاتا ہے۔ انڈو (Undo) کی مثال: ہر عمل ڈھیر پر پڑتا ہے، انڈو سب سے تازہ عمل ہٹاتا ہے۔

### EXAM TIP

Define stack + LIFO, define PUSH and POP, mention TOP, give the Undo example, and draw the vertical boxes with TOP labelled. Trace questions: "Show the stack after PUSH A, PUSH B, POP, PUSH C." Work left to right; the first POP must be B. Never remove from the middle.

### CHECK YOURSELF

1. Expand LIFO.
2. After PUSH Data1, Data2, Data3, what does the first POP return?
3. Why is Undo modelled as a stack?

Answers: 1. Last-In, First-Out. 2. Data3. 3. Each action is PUSHed; Undo POPs the most recent action first.

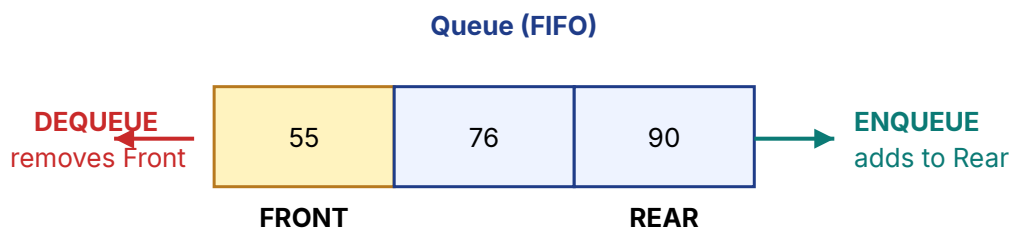
## Linear Data Structures: Queue ★ Golden

### LEARN IT

A **queue** organises data on the **FIFO** principle: **First-In, First-Out**. Whoever arrives first is served first. Order truly matters.

- **ENQUEUE** — add an element at the **Rear** (back).
- **DEQUEUE** — remove an element from the **Front**.

**Real-life example:** several documents sent to a printer. The printer prints the first document received, then the next.



Next ENQUEUE joins after 90. Next DEQUEUE removes 55.

Three jobs waiting. New jobs join at the rear; service always takes the front.



### WORKED EXAMPLE

**Worked example: ENQUEUE / DEQUEUE trace.** Start empty. ENQUEUE 55, 76, 90, then ENQUEUE D, then DEQUEUE.

| Step | Operation  | Queue (Front → Rear) | Front | Rear | Removed |
|------|------------|----------------------|-------|------|---------|
| 0    | (empty)    | []                   | —     | —    |         |
| 1    | ENQUEUE 55 | [55]                 | 55    | 55   |         |
| 2    | ENQUEUE 76 | [55, 76]             | 55    | 76   |         |
| 3    | ENQUEUE 90 | [55, 76, 90]         | 55    | 90   |         |
| 4    | ENQUEUE D  | [55, 76, 90, D]      | 55    | D    |         |
| 5    | DEQUEUE    | [76, 90, D]          | 76    | D    | 55      |

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("left:", list(q))
```

#### OUTPUT

```
after ENQUEUE D: [55, 76, 90, 'D']
DEQUEUE: 55
left: [76, 90, 'D']
```

append is ENQUEUE at the rear; popleft is DEQUEUE from the front. The first DEQUEUE is 55 (the oldest), never D.

### REMEMBER IT

**FIFO = First In First Out = a canteen line.** Front = the person being served. Rear = the person who just joined.

### COMMON MISTAKE

**Mistake:** DEQUEUEing from the rear, or ENQUEUEing at the front.

**Fix:** new items only at Rear; removal only at Front. If your first removal is D, you treated the queue as a stack.

### اردو میں سمجھیں

کیو FIFO پر چلتی ہے: جو پہلے آیا وہ پہلے نکلے۔ پیچھے (Rear) شامل کرنا ENQUEUE، آگے (Front) سے ہٹانا DEQUEUE ہے۔ پرنٹر کی قطار: جو فائل پہلے بھیجی گئی وہ پہلے چھپے گی۔ پہلی DEQUEUE ہمیشہ سب سے پرانا عنصر نکالتی ہے۔

### EXAM TIP

Define queue + FIFO, define ENQUEUE (rear) and DEQUEUE (front), give the printer example, draw the labelled Front/Rear diagram, and trace a short sequence. Examiners mix stack and queue on the same paper: if they say "first come, first served" write Queue; if they say "Undo" write Stack.

### CHECK YOURSELF

1. Expand FIFO. Where is an item added, and where is it removed?
2. After ENQUEUE 55, 76, 90, what does DEQUEUE return?
3. Why is a printer a queue, not a stack?

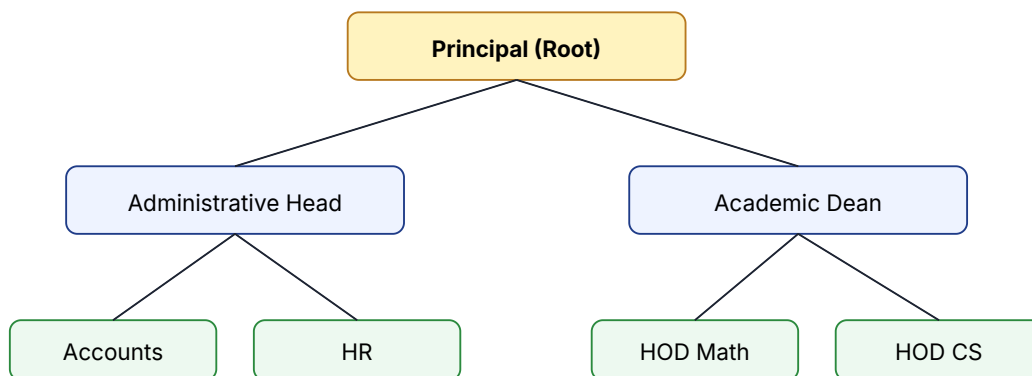
Answers: 1. First-In, First-Out; added at Rear, removed from Front. 2. 55. 3. Documents must print in the order they arrived (first received, first printed), not last-received first.

## 2.5.2 Non-Linear Data Structures: Tree

### LEARN IT

A **tree** organises data in a **hierarchy**. Everything branches out from a single starting point called the **root**. It creates a strict **one-to-many (parent-child)** relationship. That is an excellent way to show how roles flow in an organisation.

**Real-life examples:** a family tree (grandparents → parents → children) and folders inside folders on a drive.



College tree: one root (Principal), then parent-child branches. No child has two parents in this model.

### WORKED EXAMPLE

**Worked example: name the relationships.**

- Root: Principal.
- Children of the root: Administrative Head, Academic Dean.
- Children of Academic Dean: HOD Math, HOD CS.
- HOD CS has one parent (Academic Dean), not two — that is the tree rule. If HOD CS also reported sideways to Accounts as an equal link, the picture would become a **graph**.

### REMEMBER IT

Tree = family / folders / organogram. One root. Branches down. Parent-child, not a web of equals.

### COMMON MISTAKE

**Mistake:** drawing a tree with two roots, or with an arrow going back up to the Principal from HOD CS (a cycle).

**Fix:** one root, links only parent → child, no loops. Cycles belong to graphs.

ٹری ڈیٹا کو درجہ بندی (Hierarchy) میں رکھتی ہے جو ایک جڑ (Root) سے پھوٹتی ہے۔ تعلق والدین-بچہ (ایک سے کئی) ہوتا ہے، جیسے پرنسپل → ڈین → ایچ او ڈی، خاندانی شجرہ، یا فولڈر کے اندر فولڈر۔ دو جڑیں یا چکر ٹری کو گراف بنا دیتے ہیں۔

## CHECK YOURSELF

1. What is the starting node of a tree called?
2. What kind of relationship does a tree model?
3. Give two real-life tree examples from the source.

Answers: 1. The root. 2. One-to-many parent-child. 3. A family tree, and folders inside folders (or a college organogram).

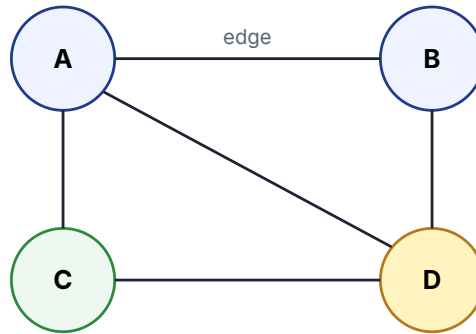
## Non-Linear Data Structures: Graph

## LEARN IT

A **graph** organises data when things connect in a flexible, complex way. Unlike a tree, there is **no strict hierarchy**: any node can connect to any other node.

- **Nodes (vertices)** — the items (cities, people).
- **Edges** — the connections (roads, friendships).

**Examples:** Facebook friends; GPS mapping (the fastest route between two cities).



Four vertices A, B, C, D. Edges can form a cycle (A-B-D-C-A). A tree cannot do that.

## WORKED EXAMPLE

**Worked example: Karachi – Hyderabad – Sukkur – Larkana.** Cities are vertices. Roads are edges. You can travel Karachi → Hyderabad → Sukkur or Karachi → Larkana → Sukkur. There is no single “root city”. That is a graph, and GPS uses it to pick a fast route.

## REMEMBER IT

Tree  $\subset$  Graph in shape-talk: a tree is a special graph with one root and no cycles. If you need cycles or many-to-many links, say **graph**.

## COMMON MISTAKE

**Mistake:** using a tree for GPS because “Hyderabad comes under Sindh”.

**Fix:** administration can be a tree; **roads** are a graph because you can loop and take alternative paths.

گراف لچکدار روابط کے لیے ہے: کوئی سخت درجہ بندی نہیں، کوئی بھی نوڈ کسی سے جڑ سکتا ہے۔ نوڈز (Vertices) چیزیں ہیں، کنارے (Edges) تعلق۔ فیس بک دوست اور GPS نقشہ کلاسیکی مثالیں ہیں۔ چکر (Cycle) اور کئی راستے گراف کو ٹری سے الگ کرتے ہیں۔

**CHECK YOURSELF**

1. What are the two parts of a graph?
2. Give two source examples of graphs.
3. State one difference between a tree and a graph.

Answers: 1. Nodes/vertices and edges. 2. Social networks (Facebook friends) and GPS mapping. 3. A tree has a hierarchy and one root; a graph has no strict hierarchy and any node may connect to any other (cycles allowed).

## 2.5.3 Operations on Data Structures

**LEARN IT**

Whatever the shape, you keep meeting the same five operations:

| Operation | Meaning                                                      | Quick picture                                                  |
|-----------|--------------------------------------------------------------|----------------------------------------------------------------|
| Insertion | Adding a new item into the structure.                        | PUSH on a stack; ENQUEUE on a queue; putting 40 into an array. |
| Deletion  | Removing an item that is no longer needed.                   | POP; DEQUEUE; deleting a node from a list.                     |
| Traversal | Going through each item one by one.                          | Walking HEAD → ... → NULL; visiting every tree node.           |
| Searching | Finding a specific item.                                     | Linear search in an array; following links until 87 is found.  |
| Sorting   | Arranging items in a proper order (ascending or descending). | Bubble Sort on an array of marks.                              |

**WORKED EXAMPLE**

**Worked example: name the operation.**

1. "Print every song from the start of the playlist." → **Traversal**.
2. "Add a print job at the rear." → **Insertion** (ENQUEUE).
3. "Undo the last typing action." → **Deletion** (POP).
4. "Is roll number 87 in the list?" → **Searching**.
5. "Arrange heights from shortest to tallest." → **Sorting**.

**REMEMBER IT**

Memorise **I-D-T-S-S**: Insert, Delete, Traverse, Search, Sort.

**COMMON MISTAKE**

**Mistake:** calling a PUSH "sorting" because the stack looks tidy.

**Fix:** PUSH is insertion. Sorting means putting values in ascending or descending order.

ہر ڈیٹا سٹرکچر پر پانچ عمل ہوتے ہیں: داخل کرنا (Insertion)، حذف کرنا (Deletion)، ایک ایک کر کے گزرنا (Traversal)، تلاش (Searching) اور ترتیب (Sorting، صعودی یا نزولی)۔

### CHECK YOURSELF

1. List the five operations.
2. Which operation is ENQUEUE?
3. Traversal vs searching — one difference?

Answers: 1. Insertion, deletion, traversal, searching, sorting. 2. Insertion (at the rear of a queue). 3. Traversal visits every item; searching stops when the target is found (or the structure ends).

## 2.6 Identify data structures in problem scenarios

### LEARN IT

To pick a structure, first see how the data is organised. If each item has a clear “before and after”, it is **linear**. If it branches or connects to many items, it is **non-linear**.

### 2.6.1 Linear scenarios

1. **Storing student grades for a class → Array**. You usually know how many students there are, and all grades are the same type. Indices give rapid lookup.
2. **Music playlist / browser history → Linked list**. Every song knows what comes next. When one finishes, follow the pointer. Frequent insert/delete is cheap.
3. **Undo or Redo → Stack (LIFO)**. Each typing or formatting action is PUSHed. Undo POPs the most recent action.
4. **Printer task scheduling → Queue (FIFO)**. Documents print in the exact order they were received. First-come, first-served.

### 2.6.2 Non-linear scenarios

1. **Organisational chart of a college → Tree**. Principal is the root, then deans, then HODs, then teachers. Clear parent-child links.
2. **GPS navigation and maps → Graph**. Cities are nodes, roads are edges. No strict top-down hierarchy. Several routes can reach the same destination.

| If the data...                                     | Then it is... | Recommended structure |
|----------------------------------------------------|---------------|-----------------------|
| Follows “next” or “previous” logic                 | Linear        | Array, Linked list    |
| Follows “parent” and “child” logic                 | Non-linear    | Tree                  |
| Follows “first-come, first-served” logic           | Linear        | Queue                 |
| Follows “connections / relationships” logic        | Non-linear    | Graph                 |
| Needs to be accessed in the reverse order of entry | Linear        | Stack                 |

### WORKED EXAMPLE

Worked example: match five school problems.

| Scenario                                                         | Structure   | Why (one line)                               |
|------------------------------------------------------------------|-------------|----------------------------------------------|
| Store 40 practical marks you already counted                     | Array       | Fixed size, same type, need A[i] lookup.     |
| A playlist where the teacher inserts a nasheed between two songs | Linked list | Insert by moving two pointers, no big shift. |
| Ctrl+Z after formatting a heading                                | Stack       | Last action must come off first.             |
| Students lining up for the photocopier                           | Queue       | First in line is served first.               |
| Friend links in a class WhatsApp "everyone knows everyone"       | Graph       | Any student can connect to any other.        |

### REMEMBER IT

Decision order: reverse-of-entry? → stack. First-come? → queue. Parent-child? → tree. Many-to-many routes? → graph. Known list with indexes? → array. Lots of insert/delete in a sequence? → linked list.

### COMMON MISTAKE

**Mistake:** choosing an array for Undo because "actions are stored in a list".

**Fix:** the **access rule** matters more than "it is a list". Undo is reverse order of entry → stack.

### اردو میں سمجھیں

اگر ہر عنصر کا واضح اگلا/پچھلا ہے تو ڈھانچہ لکیری ہے؛ اگر شاخیں یا کئی روابط ہیں تو غیر لکیری۔ نمبر: ایرے؛ پلے لسٹ/ہسٹری؛ لنکڈ لسٹ؛ انڈو: سٹیک؛ پرنٹر: کیو؛ کالج کا چارٹ: ٹری؛ GPS: گراف۔ فیصلے کی جدول یاد کر لیں۔

### CHECK YOURSELF

1. Which structure stores a class's grades, and why?
2. Printer jobs: stack or queue? Why?
3. "Needs reverse order of entry" maps to which structure?

Answers: 1. Array — known size, same type, fast index lookup. 2. Queue, because first-come, first-served. 3. Stack.

## 2.7 Trace data retrieval in Array, List and Queue

### LEARN IT

**Tracing data retrieval** means following the step-by-step path of how an algorithm finds or accesses data. Array, linked list and queue are all linear, but the **effort** to find an item is very different because of how they are organised inside.

### 2.7.1 Retrieval in an array

Every element has a fixed index from 0 to  $n-1$  pointing at a spot in memory. If the algorithm already knows the index, it does not search: it **jumps** (direct access,  $O(1)$ ).

Array = [10, 50, 20, 70, 90, 30]. Direct access: Array[2] = 20.

| Loop iteration | Index | Array access | Value |
|----------------|-------|--------------|-------|
| 1              | 0     | Array[0]     | 10    |
| 2              | 1     | Array[1]     | 50    |
| 3              | 2     | Array[2]     | 20    |

```
Array = [10, 50, 20, 70, 90, 30]
print("Direct Array[2] =", Array[2])
for i in range(3):
    print(i + 1, i, f"Array[{i}]", Array[i])
```

#### OUTPUT

```
Direct Array[2] = 20
1 0 Array[0] 10
2 1 Array[1] 50
3 2 Array[2] 20
```

### 2.7.2 Retrieval in a linked list

Linked lists have **no index**. The only way is to start at HEAD and follow links one node at a time (sequential access,  $O(n)$ ).

List: [12] → [98] → [23] → [87] → [56] → NULL. Search key: 87.

| Step | Node data | Compare with 87 | Action              |
|------|-----------|-----------------|---------------------|
| 1    | 12        | 12 != 87        | Move next           |
| 2    | 98        | 98 != 87        | Move next           |
| 3    | 23        | 23 != 87        | Move next           |
| 4    | 87        | 87 == 87        | FOUND — output True |

```
nodes = [12, 98, 23, 87, 56]
key = 87
found = False
for i, value in enumerate(nodes, start=1):
    print(f"Step {i}: Check Node {i} ({value}).")
    if value == key:
        found = True
        print("FOUND! Output: True")
        break
    print("Not equal. Move Next.")
print("Result:", found)
```

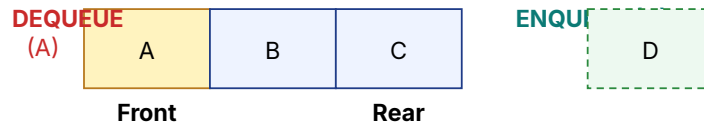
#### OUTPUT

```
Step 1: Check Node 1 (12).
Not equal. Move Next.
Step 2: Check Node 2 (98).
Not equal. Move Next.
Step 3: Check Node 3 (23).
Not equal. Move Next.
Step 4: Check Node 4 (87).
FOUND! Output: True
Result: True
```

Key 87 sits at the fourth node, so you paid four checks. An array that already knew index 3 would have paid one jump.

### 2.7.3 Retrieval in a queue

A queue is FIFO. You **cannot** retrieve from the middle or the rear. The oldest item at the Front is always DEQUEUEEd first.



You cannot pull B until A has left.

Queue A, B, C. ENQUEUE D joins the rear. DEQUEUE must remove A, not C or B.

```
from collections import deque
q = deque(["A", "B", "C"])
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("now:", list(q))
```

#### OUTPUT

```
after ENQUEUE D: ['A', 'B', 'C', 'D']
DEQUEUE: A
now: ['B', 'C', 'D']
```

#### WORKED EXAMPLE

**Worked example: three retrievals compared for "get 20 / get 87 / get A".**

| Structure                      | Target                       | Path                                     | Can you jump?              |
|--------------------------------|------------------------------|------------------------------------------|----------------------------|
| Array [10, 50, 20, ...]        | 20 at index 2                | Array[2] in one step, or scan 10, 50, 20 | Yes, if the index is known |
| Linked list<br>12→98→23→87→... | 87                           | Must visit 12, 98, 23, then 87           | No                         |
| Queue A-B-C                    | A (the only legal retrieval) | DEQUEUE Front                            | No — never the middle      |

#### REMEMBER IT

**Array jumps, list walks, queue waits.** If the exam asks "why is array retrieval different?", write **direct access by index** versus **sequential links** versus **Front only**.

#### COMMON MISTAKE

**Mistake:** DEQUEUEing C because "C is the interesting letter", or reading linked-list node 4 as `list[3]` without walking.

**Fix:** show every hop in the table. Marks are for the path, not only for FOUND.

#### اردو میں سمجھیں

ایرے میں اگر انڈیکس معلوم ہو تو سیدھی چھلانگ لگتی ہے، جیسے `Array[2] = 20`۔ لنکڈ لسٹ میں HEAD سے ایک ایک نوڈ دیکھنا پڑتا ہے؛ 87 چوتھے نوڈ پر ملا۔ کیوں کہ صرف Front نکلتا ہے، درمیان سے نہیں؛ A, B, C میں DEQUEUE ہمیشہ A ہے۔



### CHECK YOURSELF

1. What is `Array[2]` in `[10, 50, 20, 70, 90, 30]`?
2. How many nodes are checked before 87 is found in `12→98→23→87→56`?
3. In queue A, B, C, can you retrieve B without removing A?

Answers: 1. 20. 2. Four (12, 98, 23, 87). 3. No — a queue only DEQUEUEs the Front, so A must leave first.

## Career Opportunities in Algorithms

### LEARN IT

The skills in this chapter — proving correctness, writing clear modules, judging Big O, and choosing data structures — are the daily tools of several computing careers.

| Career                 | What the person does                                                                                           |
|------------------------|----------------------------------------------------------------------------------------------------------------|
| Software Engineer      | Builds and maintains software that relies on data structures and efficient algorithms to solve logic problems. |
| Backend Developer      | Builds server-side logic where choosing the right structure (array, queue) directly affects web performance.   |
| Algorithm Engineer     | Specialises in optimising algorithms for search engines, finance or logistics.                                 |
| Systems Analyst        | Evaluates and improves the efficiency and clarity of existing computer systems in an organisation.             |
| Competitive Programmer | Solves complex algorithmic problems under time limits, a common path into top tech roles.                      |

### WORKED EXAMPLE

**Worked example: match the task.**

1. "Replace an  $O(n^2)$  report with an  $O(n \log n)$  sort so the payroll finishes overnight." → **Algorithm Engineer** (or a performance-minded Software Engineer).
2. "Put incoming HTTP requests into a FIFO buffer so the first visitor is served first." → **Backend Developer** (queue).
3. "Read a college's old fee software, find the nested-IF bottleneck, and recommend modules." → **Systems Analyst**.

### REMEMBER IT

Every career in the table uses the same three tests you learned: **correct, clear, efficient** — plus the right data structure.

### COMMON MISTAKE

**Mistake:** thinking only "Algorithm Engineer" needs Big O.

**Fix:** a backend developer who puts a nested loop on every page request will take a site down. Efficiency is a shared skill.

اس باب کی مہارتیں سافٹ ویئر انجینئر، بیک اینڈ ڈیولپر، الگورتھم انجینئر، سسٹمز اینالسٹ اور کمپیٹیو پروگرامر کے کام میں روز استعمال ہوتی ہیں: درست منطق، صاف ماڈیول، بگ او اور درست ڈیٹا سٹرکچر۔

#### CHECK YOURSELF

1. Which career specialises in search-engine or logistics algorithms?
2. Why does a backend developer care about queues and arrays?

Answers: 1. Algorithm Engineer. 2. The choice of structure (array, queue) directly affects web performance.

## Chapter 2 Review

### Chapter summary

- **Correctness:** output matches expected results for valid inputs. Test with a trace table (manual dry run) or stepwise reasoning (logic verification).
- A **trace table** is micro-level: line, variables, I/O, conditions. Best for small loops. The soldier heights [1.91, 2.01, 1.87, 1.57, 1.48] print Total = 3.
- **Stepwise reasoning** is macro-level: valid start, all cases, correct conditions, loop stays correct. Sum 1..N starts at 0; N = 0 stays 0; N = 5 becomes 15.
- **Clarity** (read, understand, modify) often beats a tiny speed gain. It has two parts: modularity and readability.
- **Modularity:** SRP (one job), loose coupling, abstraction (call Sort without knowing Bubble vs Selection). Split CalculateSum and FindAverage.
- **Readability:** names (`velocity = distance / time`), self-documentation, simple flow, consistent style.
- **Efficiency** = time + space. Drivers: number of steps (Big O), conditions, repetitions.
- **Big O:** O(1) constant (`arr[0]`), O(log n) binary search, O(n) linear search, O(n log n) Merge Sort, O(n<sup>2</sup>) nested loops / Bubble Sort. Quadratic fails as n grows.
- **Framework:** Does it scale? What is the bottleneck (deepest loop)? Is the extra complexity worth it? For n = 50, simple O(n) can beat fragile O(log n).
- **Refinements:** meaningful names, modules, indentation, why-comments; shrink Bubble Sort's inner loop; better algorithms; cache constants; pick the right structure.
- **Data structure:** systematic organising of data. Linear (one after another): array, linked list, stack, queue. Non-linear: tree, graph.
- **Array:** contiguous, index from 0, direct access O(1), shifting on mid-insert. **Linked list:** nodes of data+link, HEAD to NULL, sequential access.
- **Stack LIFO:** PUSH/POP at TOP (Undo). **Queue FIFO:** ENQUEUE rear, DEQUEUE front (printer).
- **Tree:** one root, parent-child (college chart). **Graph:** vertices + edges, no strict hierarchy (GPS, friends).
- Five operations: insert, delete, traverse, search, sort. Choose by next/previous, parent/child, FIFO, relationships, or reverse-of-entry.
- **Retrieval:** array can jump to an index; a list walks from HEAD; a queue only removes the Front.

## Key terms

| Term                            | Meaning                                                                   | اردو                  |
|---------------------------------|---------------------------------------------------------------------------|-----------------------|
| Correctness                     | The algorithm's output matches the expected results for valid inputs.     | درستگی                |
| Trace table                     | A paper dry-run table of steps, variables, I/O and conditions.            | ٹریس ٹیبل             |
| Dry run                         | Executing an algorithm by hand, line by line.                             | دستی اجرا             |
| Stepwise reasoning              | Checking each logic block's state; also called logic verification.        | مرحلہ وار استدلال     |
| Clarity                         | How easy an algorithm is to read, understand and modify.                  | وضاحت                 |
| Modularity                      | Breaking work into independent functions, classes or modules.             | ماڈیولریتی            |
| Single Responsibility Principle | Each module should do one specific task.                                  | واحد ذمہ داری کا اصول |
| Coupling                        | How tightly modules depend on each other; loose coupling is the goal.     | ماڈیولز کی جڑت        |
| Abstraction                     | Hiding implementation details so a caller need not know the inner method. | تجربید                |
| Readability                     | How easily another person understands the logic without running it.       | پڑھائی پن             |
| Efficiency                      | How well an algorithm uses time (speed) and space (memory).               | کارکردگی              |
| Big O notation                  | A description of how running time or memory grows as n grows.             | بگ او نوٹیشن          |
| $O(1)$                          | Constant time; e.g. accessing <code>arr[0]</code> .                       | مستقل وقت             |
| $O(\log n)$                     | Logarithmic time; the problem is halved each step (binary search).        | لا جارتھمک وقت        |
| $O(n)$                          | Linear time; work grows in proportion to n (linear search).               | لکیری وقت             |
| $O(n \log n)$                   | Work about n times log n (Merge Sort).                                    | این لاگ این وقت       |
| $O(n^2)$                        | Quadratic time; nested loops (Bubble Sort).                               | مربع وقت              |
| Bottleneck                      | The slowest, most time-consuming section, often the deepest loop.         | رکاوٹ، سب سے سست حصہ  |

| Term                  | Meaning                                                                 | اردو                          |
|-----------------------|-------------------------------------------------------------------------|-------------------------------|
| Short-circuiting      | Stopping a boolean evaluation as soon as the result is known.           | جلدی ختم ہونے والی شرط        |
| Memoization / caching | Computing a fixed value once and reusing it instead of recalculating.   | ایک بار حساب، بار بار استعمال |
| Data structure        | A systematic way to organise, store and manage data for efficient use.  | ڈیٹا سٹرکچر                   |
| Linear structure      | A sequence; each middle item has one predecessor and one successor.     | لکیری ڈھانچہ                  |
| Array                 | Same-type items in contiguous memory, accessed by index.                | ایرے                          |
| Linked list           | Nodes of data + pointer, starting at HEAD and ending at NULL.           | لنکڈ لسٹ                      |
| Stack                 | LIFO structure: PUSH and POP at the TOP.                                | سٹیک                          |
| Queue                 | FIFO structure: ENQUEUE at Rear, DEQUEUE at Front.                      | کیو                           |
| Tree                  | Hierarchical non-linear structure with one root and parent-child links. | ٹری                           |
| Graph                 | Nodes (vertices) joined by edges, with no strict hierarchy.             | گراف                          |
| Traversal             | Visiting each item in a structure one by one.                           | گزرنا، طواف                   |
| Direct access         | Jumping to an array index without walking earlier items.                | براہ راست رسائی               |

## Practice: multiple choice

1. An algorithm is correct when:
  - a. it compiles with no syntax error
  - b. its output matches the expected results for the given input
  - c. it uses Big O notation
  - d. it has no comments
2. A trace table is best described as:
  - a. a paper execution that tracks variables line by line
  - b. a diagram of a tree
  - c. a type of graph edge
  - d. an  $O(n^2)$  nested loop
3. Stepwise reasoning mainly checks:
  - a. printer ink levels
  - b. the colour of variable names
  - c. only the last printed line
  - d. the overall state of logic blocks
4. Which method is impractical if a loop runs 1,000 times?
  - a. Abstraction
  - b. A full trace table
  - c. Stepwise reasoning
  - d. Using a function
5. The Single Responsibility Principle says each module should:
  - a. focus on one specific task
  - b. print and save and calculate everything
  - c. avoid meaningful names
  - d. use nested IF only
6. Which assignment is more readable?
  - a. `v = d / t`
  - b. `x1 = a2 / b2`
  - c. `velocity = distance / time`
  - d. `a = b / c`
7. Accessing `arr[0]` is:
  - a.  $O(n^2)$
  - b.  $O(n \log n)$
  - c.  $O(n)$
  - d.  $O(1)$
8. Binary search on a sorted list is:
  - a.  $O(\log n)$
  - b.  $O(n^2)$
  - c.  $O(1)$  always for any n
  - d.  $O(n)$  because a loop exists
9. Bubble Sort and nested "compare every pair" loops are:
  - a.  $O(1)$
  - b.  $O(\log n)$
  - c.  $O(n)$
  - d.  $O(n^2)$
10. If n grows 10 times, a quadratic algorithm's time grows about:
  - a. 2 times
  - b. 10 times
  - c. 100 times
  - d. log 10 times
11. LIFO is the rule of a:
  - a. queue
  - b. graph of cities
  - c. tree root only
  - d. stack
12. ENQUEUE adds an item at the:
  - a. Front of a queue
  - b. TOP of a stack only
  - c. Rear of a queue
  - d. root of a tree
13. A college organogram with a Principal at the top is modelled as a:
  - a. tree
  - b. queue
  - c. stack
  - d. contiguous array of roads
14. GPS maps (cities and roads, many possible routes) need a:
  - a. stack of undo actions
  - b. graph
  - c. strict one-parent tree only
  - d. single array index
15. To retrieve 87 from `HEAD→12→98→23→87→56` you must:
  - a. jump with `Array[87]`
  - b. DEQUEUE the rear
  - c. follow four nodes from HEAD until 87 matches
  - d. POP from the bottom of a stack

## Practice: short questions

1. Define correctness of an algorithm. Name the two analysis methods used in this chapter.
2. List the components of a trace table. Why is a dry run useful for debugging?
3. Give four differences between a trace table and stepwise reasoning.
4. What is modularity? Explain SRP, coupling and abstraction in one sentence each.
5. State four ways to evaluate readability, with one example of a bad name and a good name.
6. Define Big O notation. Give one example each of  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$  and  $O(n^2)$ .
7. Write the three questions of the efficiency evaluation framework and answer them for searching 50 unsorted names.
8. List four refinements for clarity and four for efficiency.
9. Define data structure. Distinguish linear from non-linear with examples.
10. Define stack and queue. Include LIFO/FIFO, PUSH/POP, ENQUEUE/DEQUEUE, and one real-life example of each.

## Practice: long / problem questions

1. The soldier-eligibility algorithm uses AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48] and counts heights from 1.57 to 1.91 inclusive. (a) Draw a complete trace table. (b) State the printed Total. (c) Explain in stepwise-reasoning style why the boundaries 1.57 and 1.91 are counted. (d) Write equivalent Python and its output.
2. Copy the monoblock average algorithm for A = [1, 2, 3, 4, 5] and rewrite it with CalculateSum and FindAverage. (a) Show that SRP, loose coupling and abstraction are improved. (b) Dry-run both versions to the same average. (c) Give two readability improvements you would still make to the functions.
3. Using the growth table idea, (a) fill n = 8 and n = 32 for  $O(1)$ ,  $O(\log_2 n)$ ,  $O(n)$ ,  $O(n \log_2 n)$  and  $O(n^2)$ . (b) Explain what happens if n becomes 10 times larger for  $O(n)$  and for  $O(n^2)$ . (c) A library has 1,000,000 sorted IDs. Choose linear or binary search using the Scale / Bottleneck / Worth-it framework. (d) Show a 4-by-4 nested loop counts 16 operations in Python.
4. Draw a labelled stack and a labelled queue. Then trace: (a) PUSH Data1, PUSH Data2, PUSH Data3, POP, POP. (b) ENQUEUE 55, 76, 90, D then DEQUEUE. Include Python that matches both traces. (c) Why is Undo a stack and a printer a queue?
5. A college needs structures for: (i) 200 integer marks, (ii) a playlist with frequent insertions, (iii) Undo in the fee-office editor, (iv) photocopier jobs, (v) the Principal-Dean-HOD chart, (vi) roads between Sindh cities. Identify each structure using the decision guide, draw the tree and the graph, and then trace retrieval of 20 from Array = [10, 50, 20, 70, 90, 30] and of 87 from the linked list 12 → 98 → 23 → 87 → 56 → NULL.

## Answer key

---

### Multiple choice

1. b
2. a
3. d
4. b
5. a
6. c
7. d
8. a
9. d
10. c
11. d
12. c
13. a
14. b
15. c

## Short questions — model answers

1. An algorithm is correct if its output matches the expected results for a given input (and does so for all valid inputs). The two methods are trace tables and stepwise reasoning (logic verification).
2. Components: line/step number, a column per variable, input/output, and a condition-check column. A dry run acts as the computer, so you see logical errors where the program runs but the values go wrong, and you can validate that the intention is met.
3. Trace table: small blocks/complex loops/beginners; high effort (bad for 1,000 iterations); catches arithmetic/line mistakes; micro-level exact values. Stepwise reasoning: large systems and proofs; moderate effort; catches conceptual/structural flaws; macro-level overall state.
4. Modularity is breaking an algorithm into smaller independent parts (functions, classes, modules). SRP: each module does one task. Coupling: modules should be loosely coupled so a change in one does not break others. Abstraction: hide inner details (call Sort without knowing Bubble vs Selection).
5. Naming conventions, self-documentation, simple control flow, consistency of indentation/spacing/casing. Bad:  $v = d / t$ . Good: `velocity = distance / time`.
6. Big O describes how time or memory grows as n grows.  $O(1)$ : `arr[0]`;  $O(\log n)$ : binary search;  $O(n)$ : linear search;  $O(n \log n)$ : Merge Sort;  $O(n^2)$ : Bubble Sort / nested loops.
7. Does it scale?  $n = 50$  is tiny, even  $O(n)$  is fine. Bottleneck? A single scan of 50 names. Worth it? Binary search needs sorted data and more complex code, so linear search is practically better.
8. Clarity: meaningful names; modules; structure/indentation; short why-comments. Efficiency: reduce extra steps (shrink Bubble Sort's inner loop); better algorithms (binary vs linear on sorted millions); avoid re-computing (cache  $C = A+B$  outside the loop); appropriate data structures (array for direct access, tree for hierarchy).
9. A data structure is a systematic way of organising, storing and managing data for efficient access, modification and processing. Linear: sequence with one predecessor and one successor (array, linked list, stack, queue). Non-linear: hierarchical or network links (tree, graph).
10. Stack: LIFO; PUSH adds at TOP; POP removes from TOP; example Undo. Queue: FIFO; ENQUEUE adds at Rear; DEQUEUE removes from Front; example printer jobs.

## Long questions — model answers

1. (a) Columns: Step, Total, Height, AllHeights, condition, output. Step 2 sets Total = 0. Step 3 stores the five heights. Then five loop cycles:

- 1.91 → True → Total 1
- 2.01 → False → Total stays 1
- 1.87 → True → Total 2
- 1.57 → True (boundary) → Total 3
- 1.48 → False → Total stays 3

(b) Printed Total = **3**.

(c) The condition is height  $\geq 1.57$  AND height  $\leq 1.91$ , so the closed interval includes both ends. 1.57 and 1.91 satisfy the inequalities; 2.01 and 1.48 do not. Initial Total = 0 is valid; each True branch adds one; False skips the add; after five items Total is the count of eligible soldiers.

(d) Python:

```
AllHeights = [1.91, 2.01, 1.87, 1.57, 1.48]
Total = 0
for height in AllHeights:
    if height >= 1.57 and height <= 1.91:
        Total = Total + 1
print(Total)
```

OUTPUT  
3

2. (a) Monoblock: one sequence does the loop, the division and the print, so you cannot reuse the sum. Modular version: CalculateSum only adds; FindAverage only divides; main only prints. SRP holds. CalculateSum can be called from another program (loose coupling). Main does not see the loop (abstraction).

(b) Both compute total = 15 and avg =  $15/5 = 3.0$ .

```
def calculate_sum(A, n):
    total = 0
    for i in range(n):
        total = total + A[i]
    return total

def find_average(A, n):
    return calculate_sum(A, n) / n

print(find_average([1, 2, 3, 4, 5], 5))
```

OUTPUT  
3.0

(c) Keep snake\_case names; avoid 1-based I in Python; a one-line comment on why you divide by n (empty-list guard could be added later) is enough — do not comment every addition.



3. (a) Growth ( $\log_2$ ):

| n  | $O(1)$ | $O(\log n)$ | $O(n)$ | $O(n \log n)$ | $O(n^2)$ |
|----|--------|-------------|--------|---------------|----------|
| 8  | 1      | 3           | 8      | 24            | 64       |
| 32 | 1      | 5           | 32     | 160           | 1,024    |

(b)  $O(n)$ :  $10\times$  input  $\rightarrow$  about  $10\times$  time.  $O(n^2)$ :  $10\times$  input  $\rightarrow$  about  $100\times$  time, which fails for large sets.

(c) Scale: a million IDs; linear worst case is a million checks, binary about 20. Bottleneck: the search. Worth it: the IDs are already sorted and binary search is standard, so choose **binary search  $O(\log n)$** .

(d)

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

OUTPUT  
16

4. Draw the vertical stack with TOP on Data3 after three PUSHes, and the horizontal queue with FRONT on 55 and REAR on 90 (then D).

(a) Stack:  $[] \rightarrow [Data1] \rightarrow [Data1, Data2] \rightarrow [Data1, Data2, Data3] \rightarrow POP\ Data3 \rightarrow POP\ Data2 \rightarrow [Data1]$ .

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print(stack.pop())
print(stack.pop())
print(stack)
```

OUTPUT  
Data3  
Data2  
['Data1']

(b) Queue:  $[55] \rightarrow [55, 76] \rightarrow [55, 76, 90] \rightarrow [55, 76, 90, D] \rightarrow DEQUEUE\ 55 \rightarrow [76, 90, D]$ .

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print(q.popleft())
print(list(q))
```

OUTPUT  
55  
[76, 90, 'D']

(c) Undo must cancel the most recent action (LIFO  $\rightarrow$  stack). A printer must respect arrival order (FIFO  $\rightarrow$  queue).

5. (i) Array — known count, same type, index lookup. (ii) Linked list — next-pointer, cheap insert. (iii) Stack — reverse order of entry. (iv) Queue — first-come, first-served. (v) Tree — parent-child from the Principal root (Administrative Head / Academic Dean, then Accounts, HR, HOD Math, HOD CS). (vi) Graph — cities as vertices, roads as edges, alternative routes.

Array retrieval:  $Array[2] = 20$  in one direct access (or after visiting 10 then 50 then 20 if you scan).

List retrieval: Step1 12  $\neq$  87, Step2 98  $\neq$  87, Step3 23  $\neq$  87, Step4 87 = 87 FOUND True.

## Self-assessment: I can...

- ☐ prove correctness with a full trace table, including boundary values.
- ☐ judge modularity (SRP, coupling, abstraction) and readability (N-S-S-C).
- ☐ apply the Scale / Bottleneck / Worth-it framework, including the  $n = 50$  warning.
- ☐ define a data structure and classify linear vs non-linear.
- ☐ draw a tree and a graph and pick a structure from a scenario using the decision guide.
- ☐ write stepwise reasoning (start, loop, finish, edges) and compare it with a trace table.
- ☐ state  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n \log n)$  and  $O(n^2)$  with examples and a growth table.
- ☐ refine an algorithm for names, modules, fewer loop steps and cached values.
- ☐ draw and trace an array, linked list, stack (PUSH/POP) and queue (ENQUEUE/DEQUEUE).
- ☐ trace retrieval: array jump, list walk from HEAD, queue Front only.

## CHAPTER 3

# Programming Fundamentals

This chapter teaches you how Python stores collections of data, how you write reusable functions, and how you save information in files so it is still there after the program ends. You will build small programs — attendance, a word counter, a calculator, and a grade tracker — that you can type and run yourself.

اس باب میں آپ سیکھیں گے کہ پائتھن (Python) ڈیٹا کو فہرست، ٹیپل، سیٹ اور ڈکشنری میں کیسے رکھتا ہے، فنکشن کیسے لکھے جاتے ہیں، اور فائلوں میں ڈیٹا کیسے محفوظ کیا جاتا ہے تاکہ پروگرام بند ہونے کے بعد بھی وہ موجود رہے۔

## What you will learn

- Distinguish primitive types from abstract data structures (list, tuple, set, dictionary).
- Use tuples as ordered, immutable collections, including indexing and the two tuple methods.
- Store and retrieve key–value data with dictionaries, including safe access with `.get()`.
- Write user-defined functions with parameters and `return` values, and contrast them with built-in functions.
- Open text files in modes `r`, `w` and `a`, and close them safely with `with`.
- Create, index, slice, traverse and update lists using the main list methods.
- Build sets of unique items and compute union, intersection, difference and symmetric difference.
- Apply `min()`, `max()`, `len()`, `sum()` and `sorted()` across data structures.
- Explain local and global variable scope with short programs.
- Catch file errors with `try / except` and complete a grade-tracker file project.

## 3.1 Data Structures in Python

### LEARN IT

A **data structure** is a way of organising values in a program so you can store them, find them and change them. Every useful program is built on data structures. Python gives you two layers:

- **Primitive data structures** — the basic types that hold **one** value. Examples: integers ( `int` ), floating-point numbers ( `float` ), and booleans ( `bool` ).
- **Abstract data structures** — higher-level containers built from primitive types so you can hold **many** values and run richer operations. Examples: **lists, tuples, sets, dictionaries**.

Python has these four abstract structures built in. You do not install anything extra. That is why collecting marks, names or settings is simpler in Python than in many other languages.

| Category  | Description                                                     | Examples                          |
|-----------|-----------------------------------------------------------------|-----------------------------------|
| Primitive | Basic types that represent a single value.                      | int, float, bool                  |
| Abstract  | Higher-level structures for collections and complex operations. | Lists, tuples, sets, dictionaries |

#### WORKED EXAMPLE

**One value versus a collection.** A student's age is one integer. That student's three test marks need a list.

```
age = 17
height = 5.8
passed = True
marks = [80, 75, 90]
print(type(age))
print(type(height))
print(type(passed))
print(type(marks))
```

#### OUTPUT

```
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
```

type() tells you which structure a value has. age is a primitive integer. marks is an abstract list that holds three integers together.

#### REMEMBER IT

**Primitive = one piece. Abstract = a box of pieces.** Think of a single rupee coin (primitive) versus a wallet that can hold many coins (list, tuple, set or dictionary).

#### COMMON MISTAKE

Do not call a list a “variable type” and stop there. A list is a **structure**: it has an order, methods, and rules. The four abstract structures look similar at first (they all hold several values) but they follow different rules. Learn those rules one by one in 3.1.1–3.1.4.

#### اردو میں سمجھیں

ڈیٹا سٹرکچر (Data Structure) پروگرام میں ڈیٹا کو ترتیب دینے کا طریقہ ہے۔ پرائمیٹو اقسام جیسے عدد (int) صرف ایک قیمت رکھتی ہیں، جبکہ فہرست، ٹیپل، سیٹ اور ڈکشنری کئی قیمتیں ایک ساتھ سنبھالتی ہیں۔ پائٹھن میں یہ چاروں سٹرکچرز پہلے سے موجود ہیں، اس لیے ڈیٹا جمع کرنا دوسری زبانوں کے مقابلے میں آسان ہے۔

#### CHECK YOURSELF

1. Name two primitive types and two abstract data structures in Python.
2. Why is a list called an abstract data structure rather than a primitive type?

Answers: 1. Primitive: any two of int, float, bool. Abstract: any two of list, tuple, set, dictionary. 2. Because it is a higher-level container built from primitive values, holding many items and supporting operations such as append and slice.

### 3.1.1 Lists (Ordered, Mutable, Methods) ★ Golden

#### LEARN IT

A **list** is Python's most versatile built-in data structure. It stores an **ordered** collection of items in a single variable. Lists are **mutable** (you can change them after you create them) and they are written with **square brackets** `[]`.

**Why use a list?** One variable can hold thousands of values. You can mix types if you need to. You can add, remove and sort items as the program runs.

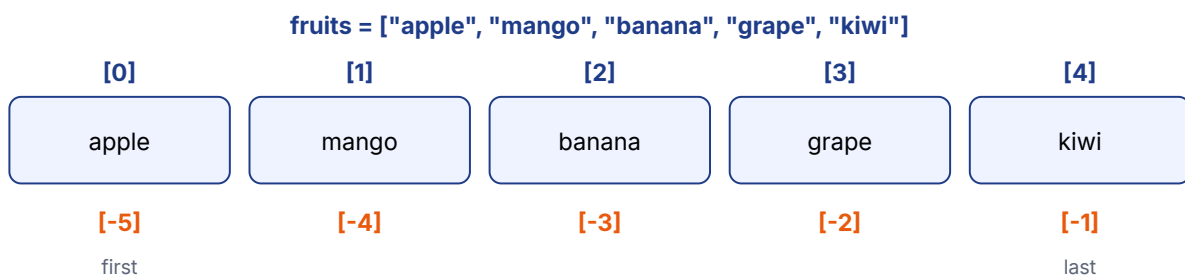
```
numbers = [10, 2, 15, 14, 3, 33]      # list of integers
names = ["Hassan", "Qasim", "Malika"]  # list of strings
mixed = [1, "Python", 3.14, True]     # mixed data types
print(numbers)
print(names)
print(mixed)
```

#### OUTPUT

```
[10, 2, 15, 14, 3, 33]
['Hassan', 'Qasim', 'Malika']
[1, 'Python', 3.14, True]
```

#### List indexing (positive and negative)

Each item sits in a numbered slot called an **index**. Numbering starts at **0** from the left. Python also supports **negative indexing** from the right: `-1` is the last item, `-2` is the second last, and so on.



Positive indices count left to right from 0. Negative indices count right to left from -1.

```
fruits = ["apple", "mango", "banana", "grape", "kiwi"]
print(fruits[0])    # first item
print(fruits[4])    # last item (when you know the length)
print(fruits[-1])   # last item (works even if the list grows)
print(fruits[-5])   # first item, counted from the right
```

#### OUTPUT

```
apple
kiwi
kiwi
apple
```

#### REMEMBER IT

**Always remember `[-1]`**. If a list has 10,000 items and you do not want to compute `len(list) - 1`, write `list[-1]`. It grabs the last item in one step.

## Slicing a list

**Slicing** copies a portion of a list into a **new** list. The original is not changed.

Syntax: `list[start:stop:step]`

- `start` — first index to include (default 0).
- `stop` — first index to **exclude**. The slice goes up to, but does **not** include, this index.
- `step` — jump size (default 1). A step of `-1` walks backwards.

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[2:6])
print(nums[:4])
print(nums[6:])
print(nums[::2])
print(nums[::-1])
```

### OUTPUT

```
[2, 3, 4, 5]
[0, 1, 2, 3]
[6, 7, 8, 9]
[0, 2, 4, 6, 8]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Trace of `nums[2:6]`: start at index 2 (value 2), take 3, 4, 5, and stop **before** index 6. Result `[2, 3, 4, 5]`. `nums[::-1]` is the standard way to reverse a list without changing the original (it returns a new reversed list).

## Traversal

**Traversal** means visiting each element, one by one, in order. Use a `for` loop (the usual Python style) or a `while` loop with an index.

```
numsList = [10, 20, 30]
for num in numsList:
    print(num)
```

### OUTPUT

```
10
20
30
```

```
numsList = [10, 20, 30]
index = 0
while index < len(numsList):
    print(numsList[index])
    index += 1
```

### OUTPUT

```
10
20
30
```

The `for` loop gives you each value directly. The `while` loop needs a counter, a test `index < len(numsList)`, and `index += 1` so the loop ends.

## Built-in list methods

Because a list is mutable, Python gives you methods that change it in place. `len()` is listed with them in many notes, but it is a **built-in function**: you write `len(fruits)`, not `fruits.len()`.

| Operation                          | What it does                                                                              |
|------------------------------------|-------------------------------------------------------------------------------------------|
| <code>len(list)</code>             | Returns the number of items.                                                              |
| <code>.append(value)</code>        | Adds an item at the <b>end</b> .                                                          |
| <code>.insert(index, value)</code> | Inserts an item at a chosen position (later items shift right).                           |
| <code>.remove(value)</code>        | Removes the <b>first</b> matching value. Raises <code>ValueError</code> if it is missing. |
| <code>.clear()</code>              | Removes every item. The list becomes <code>[]</code> .                                    |
| <code>.count(value)</code>         | How many times a value appears.                                                           |
| <code>.index(value)</code>         | Index of the first occurrence.                                                            |
| <code>.sort()</code>               | Sorts the list in place, ascending. Use <code>reverse=True</code> for descending.         |
| <code>.reverse()</code>            | Reverses the list in place (does not sort).                                               |

## WORKED EXAMPLE

Worked example: methods change the same list step by step.

```
fruits = ['apple', 'banana', 'apple']
print(fruits)
fruits.append('grape')
print(fruits)
fruits.remove('apple')
print(fruits)
fruits.sort()
print(fruits)
```

### OUTPUT

```
['apple', 'banana', 'apple']
['apple', 'banana', 'apple', 'grape']
['banana', 'apple', 'grape']
['apple', 'banana', 'grape']
```

| After this line  | List in memory                                                  |
|------------------|-----------------------------------------------------------------|
| create           | ['apple', 'banana', 'apple']                                    |
| .append('grape') | ['apple', 'banana', 'apple', 'grape']                           |
| .remove('apple') | ['banana', 'apple', 'grape'] — only the <b>first</b> apple goes |
| .sort()          | ['apple', 'banana', 'grape']                                    |

More methods on numbers:

```
nums = [10, 20, 30]
print(len(nums))
nums.insert(1, 15)
print(nums)
print(nums.count(20))
print(nums.index(30))
nums.reverse()
print(nums)
nums.clear()
print(nums)
```

### OUTPUT

```
3
[10, 15, 20, 30]
1
3
[30, 20, 15, 10]
[]
```

`insert(1, 15)` places 15 at index 1, so 20 and 30 shift right. After that, 30 sits at index 3. `reverse()` flips the current order; it does not sort. `clear()` leaves an empty list `[]`, not the value `None`.

You can also replace an item by index, because lists are mutable:

```
numbers = [10, 2, 15]
numbers[0] = 99
print(numbers)
```

### OUTPUT

```
[99, 2, 15]
```

## Advantages of lists

- **Mutable** — add, remove or replace items as the program runs.
- **Organised** — thousands of values live in one variable instead of `n1`, `n2`, `n3`...
- **Flexible** — integers, strings, booleans and even other lists can sit in the same list.

## Activity: searching an element in a list

This program combines a list, user input, a `for` loop, an `if` test and a **flag** (a Boolean that remembers whether the search succeeded).

```
numbers = [5, 10, 15, 20, 25]
search = int(input("Enter number to search: "))
found = False
for num in numbers:
    if num == search:
        found = True
        break
if found:
    print("Number found in the list.")
else:
    print("Number NOT found in the list.")
```

**Execution 1** — the user types 15, which is at index 2. The loop sets `found = True` and `break` stops the scan at once.

### OUTPUT

```
Enter number to search: 15
Number found in the list.
```

**Execution 2** — the user types 99. The loop finishes without a match, so the flag stays `False`.

### OUTPUT

```
Enter number to search: 99
Number NOT found in the list.
```

Why `break`? If the value is found at index 1 in a list of 10,000 items, there is no need to check the remaining 9,998. `break` makes linear search cheaper as soon as it succeeds.

### REMEMBER IT

**LIST** — Lots of items, Indexed from 0, Square brackets, Tweakable (mutable). Slice memory trick: "start is in, stop is out."

### COMMON MISTAKE

**Mistake 1:** `.remove('apple')` deletes every apple. **Fix:** it deletes only the first match. Call it again if you need the next one, or use a loop.

**Mistake 2:** `.sort()` and `sorted()` are the same. **Fix:** `.sort()` is a list method that changes the original and returns `None`. `sorted()` is a function that works on many structures and returns a **new** list (see 3.1.5).

**Mistake 3:** `fruits[5]` on a five-item list. Valid positive indices are 0 to 4. Use `[-1]` for the last item.

### اردو میں سمجھیں

فہرست (List) مربع بریکٹ [ ] میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable). انڈیکس صفر سے شروع ہوتا ہے، اور [1-] آخری عنصر دیتا ہے۔ `.append()` آخر میں جوڑتا ہے، `.remove()` پہلی مشابہ قیمت ہٹاتا ہے، اور `.sort()` اصل فہرست کو ترتیب دیتا ہے۔ تلاش کرتے وقت `break` مل جانے کے بعد لوپ روک دیتا ہے۔

#### EXAM TIP

Lists appear every year: "What is a list?", "List four methods with one line each", "Differentiate list and tuple", "Write a program to search an element", "Predict the output of a slice." Write: ordered, mutable, [], index from 0, [-1] for last. For a slice, always say the stop index is excluded. For .remove(), say "first occurrence only." Keep a short program ready that uses append, remove and sort with the printed list after each line.

#### CHECK YOURSELF

1. What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
2. After `fruits = ['apple', 'banana', 'apple']` then `fruits.remove('apple')`, what is `fruits`?
3. Why does the search program use `break`?

Answers: 1. `[2, 3, 4, 5]`. 2. `['banana', 'apple']` (only the first apple is removed). 3. To stop scanning once the value is found, so the rest of a long list is not checked.

### 3.1.2 Tuples (Ordered, Immutable) ★ Golden

#### LEARN IT

A **tuple** is an ordered collection that is **immutable** — once created, you cannot add, remove or replace items. Think of it as a **read-only list**. Tuples use **parentheses** `()`. Like lists, they allow duplicates, mixed types, and indexing from 0 (including negative indices).

#### List []

Can add / remove / replace

#### Tuple ()

LOCKED — read-only

A list is an open box you can refill. A tuple is a sealed box you may only look into.

```
t = (10, 20, 30, 40, 50)
print(t[0])
print(t[-1])
print(t[1:4])
print(t[::-1])
```

#### OUTPUT

```
10
50
(20, 30, 40)
(50, 40, 30, 20, 10)
```

Slicing a tuple returns a **new tuple**, not a list. Negative indexing and `[::-1]` work exactly as they do for lists.

#### Immutability

You cannot assign to an index of a tuple. Python raises `TypeError`.

```
t = (1, 2, 3)
t[0] = 99
```



#### OUTPUT

```
Traceback (most recent call last):
  File "tuple.py", line 2, in <module>
    t[0] = 99
TypeError: 'tuple' object does not support item assignment
```

**Exception:** if a tuple contains a **mutable object** (a list inside the tuple), that inner object's contents can still change. The tuple still holds the same two list objects; you changed a list, not the tuple's structure.

```
t = ([1, 2], [3, 4])
t[0].append(99)
print(t)
```

#### OUTPUT

```
([1, 2, 99], [3, 4])
```

`t[0] = [9, 9]` would still be illegal, because that would replace the tuple's first item. `t[0].append(99)` is legal because it mutates the list that already lives at index 0.

## Traversal and methods

Traverse a tuple with `for`, just as you do a list. You are only reading, which is the usual reason to choose a tuple.

```
student = ("Ali", 20, "Computer Science")
for item in student:
    print(item)
```

#### OUTPUT

```
Ali
20
Computer Science
```

Because a tuple cannot grow or shrink, it has only **two** methods:

| Method                     | Meaning                           |
|----------------------------|-----------------------------------|
| <code>.count(value)</code> | How many times the value appears. |
| <code>.index(value)</code> | Index of the first occurrence.    |

```
t = (10, 20, 10, 30, 10)
print(t.count(10))
print(t.index(20))
```

#### OUTPUT

```
3
1
```

## Advantages of tuples

- **Protection** — coordinates, identity-card numbers or settings cannot be changed by accident.
- **Speed** — a fixed-size tuple is processed slightly faster than a list.
- **Hashable keys** — a tuple of immutables can be used as a dictionary key; a list cannot. (You will use dictionaries in 3.1.4.)

### WORKED EXAMPLE

**When to choose a tuple.** A student's roll number, name and class form a record that should not be edited in place:

```
record = ("XII-CS-07", "Ayesha", "Computer Science")
print(record[0])
print(len(record))
```

#### OUTPUT

```
XII-CS-07
3
```

If later you need to change the class, you build a **new** tuple. You do not assign to `record[2]`.

### REMEMBER IT

**Tuple = locked lunchbox.** You can see what is inside; you cannot swap the sandwich. Memory: "Tuple is Tight (immutable), written with round brackets that look like a hug holding the data still."

### COMMON MISTAKE

**One-item trap:** `(5)` is just the integer 5 with extra parentheses. A one-item tuple needs a trailing comma: `(5,)`.

```
t1 = (5)
t2 = (5,)
print(type(t1), t1)
print(type(t2), t2)
```

#### OUTPUT

```
<class 'int'> 5
<class 'tuple'> (5,)
```

An empty tuple is `()`.

### اردو میں سمجھیں

ٹیپل (Tuple) گول بریکٹ `()` میں لکھا جاتا ہے، ترتیب رکھتا ہے مگر تبدیل نہیں ہو سکتا (Immutable). انڈیکس اور سلائس فہرست جیسی ہیں، لیکن `t[0] = 99` سے `TypeError` آتا ہے۔ صرف دو میتھڈز ہیں: `count()` اور `index()`۔ حساس ڈیٹا کے لیے ٹیپل محفوظ ہے۔ اگر ٹیپل کے اندر فہرست ہو تو اس فہرست کے اندر کی اشیاء بدل سکتی ہیں۔

### EXAM TIP

Expect "Differentiate list and tuple" (at least four points: brackets, mutability, methods, speed/protection), "Why use a tuple?", and "What happens if you write `t[0] = 1`?" Quote the `TypeError` wording. Mention the inner-list exception if the question is a 6-mark "explain with example." A one-item tuple `(5,)` is a favourite trick question.

### CHECK YOURSELF

1. Name the two tuple methods.
2. What error does `t[0] = 99` raise for a tuple?
3. After `t = ([1, 2], [3, 4]); t[0].append(99)`, what is `t`?

Answers: 1. `count` and `index`. 2. `TypeError: 'tuple' object does not support item assignment`. 3. `[[1, 2, 99], [3, 4]]`.

### 3.1.3 Sets (Unordered, Unique, Math Operations)

#### LEARN IT

A **set** stores an **unordered, unindexed** collection of **unique** elements. It is inspired by mathematical sets. You write a set with **curly braces** `{}`. Duplicates are removed as soon as the set is created. Because there is no index, `myset[0]` is illegal.

The set itself is **mutable** (you can add and remove items). Each item inside must be **immutable** (numbers, strings, tuples of immutables — not lists).

```
nums = {1, 4, 3, 3, 2, 2}
print(nums)
```

#### OUTPUT

```
{1, 2, 3, 4}
```

The two 3s and two 2s collapse to one each. Do not depend on the printed order: a set is unordered. CPython often prints small integers in a tidy way, but that is not a language promise.

#### Adding and removing

| Method                       | Description                                                       |
|------------------------------|-------------------------------------------------------------------|
| <code>.add(value)</code>     | Adds one element.                                                 |
| <code>.update(values)</code> | Adds many elements from a list, tuple or set.                     |
| <code>.remove(value)</code>  | Removes the value. Raises <code>KeyError</code> if it is missing. |
| <code>.discard(value)</code> | Removes the value. Does <b>not</b> crash if it is missing.        |
| <code>.clear()</code>        | Removes every element. Result is <code>set()</code> .             |

```
s = {1, 2, 3}
s.add(4)
print(s)
s.update([5, 6])
print(s)
s.discard(99)
print(s)
s.remove(1)
print(s)
s.clear()
print(s)
```

#### OUTPUT

```
{1, 2, 3, 4}
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
{2, 3, 4, 5, 6}
set()
```

`discard(99)` quietly does nothing. After `clear()` Python prints `set()`, not `{}`, because `{}` already means an empty **dictionary**.

Compare with `.remove(99)` on a set that does not contain 99:

```
s = {1, 2, 3}
s.remove(99)
```

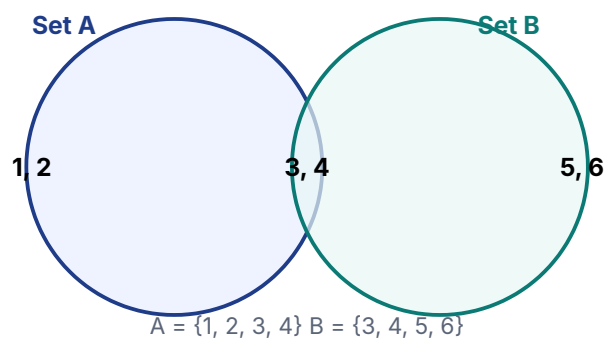
#### OUTPUT

```
Traceback (most recent call last):
  File "set_remove.py", line 2, in <module>
    s.remove(99)
KeyError: 99
```

Prefer `.discard()` when the item might not be there, so a missing value does not crash the program.

## Mathematical operations

Sets are built for union, intersection, difference and symmetric difference.



Overlap {3, 4} is the intersection. Left-only {1, 2} is A - B. Right-only {5, 6} is B - A.

| Operation            | Meaning                    | Syntax                                                 | Result for A, B above           |
|----------------------|----------------------------|--------------------------------------------------------|---------------------------------|
| Union                | All unique items from both | $A \mid B$ or <code>A.union(B)</code>                  | <code>{1, 2, 3, 4, 5, 6}</code> |
| Intersection         | Items in both              | $A \& B$ or <code>A.intersection(B)</code>             | <code>{3, 4}</code>             |
| Difference           | In A, not in B             | $A - B$ or <code>A.difference(B)</code>                | <code>{1, 2}</code>             |
| Symmetric difference | In either, but not both    | $A \wedge B$ or <code>A.symmetric_difference(B)</code> | <code>{1, 2, 5, 6}</code>       |

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)
print(A & B)
print(A - B)
print(A ^ B)
```

#### OUTPUT

```
{1, 2, 3, 4, 5, 6}
{3, 4}
{1, 2}
{1, 2, 5, 6}
```

B - A would be {5, 6} — difference is not symmetric. Symmetric difference  $A \wedge B$  is  $(A - B) \mid (B - A)$ .

## Set traversal

There is no index, so you loop with `for`. Print order is not guaranteed. Use `sorted()` when you need a stable alphabetical or numerical order (it returns a list).

```
colors = {"red", "green", "blue"}
for color in sorted(colors):
    print(color)
```

### OUTPUT

```
blue
green
red
```

### WORKED EXAMPLE

**Deduplication.** A teacher types roll numbers and some are repeated. A set keeps each roll once:

```
rolls = [12, 5, 12, 8, 5, 19]
unique = set(rolls)
print(unique)
print(sorted(unique))
```

### OUTPUT

```
{8, 19, 12, 5}
[5, 8, 12, 19]
```

`set(rolls)` drops duplicates. `sorted(unique)` gives a list in numeric order for a register.

### REMEMBER IT

**SET = Same-item Eaten Twice-never.** Empty set is `set()`, never `{}`. Safe delete = `discard`; angry delete = `remove`.

### COMMON MISTAKE

`myset[0]` raises `TypeError` because sets are unindexed. Convert with `list(myset)` only if you accept that the order is arbitrary. Do not put a list inside a set — lists are mutable, so they cannot be set elements.

### اردو میں سمجھیں

سیٹ (Set) منفرد عناصر کا بغیر ترتیب اور بغیر انڈیکس مجموعہ ہے، جو `{}` میں لکھا جاتا ہے۔ دہرائی ہوئی قیمتیں خود بخود ختم ہو جاتی ہیں۔ `add()` اور `update()` عناصر جوڑتے ہیں، `discard()` محفوظ طریقے سے ہٹاتا ہے جبکہ `remove()` غیر موجود عنصر پر `KeyError` دیتا ہے۔ اجتماع (Union) `|`، اشتراک (Intersection) `&`، فرق - اور متوازی فرق `^` ریاضی کی کارروائیاں ہیں۔

### CHECK YOURSELF

1. What is printed by `print({1, 4, 3, 3, 2, 2})`?
2. Why prefer `.discard()` over `.remove()`?
3. If  $A = \{1, 2, 3, 4\}$  and  $B = \{3, 4, 5, 6\}$ , what is  $A \cap B$ ?

Answers: 1. `{1, 2, 3, 4}` (duplicates removed; order may vary). 2. `discard` does not raise `KeyError` if the value is missing. 3. `{3, 4}`.

## 3.1.4 Dictionaries (Key-Value Pairs) ★ Golden

### LEARN IT

A **dictionary** stores data as **key–value pairs**. It works like a real dictionary: you look up a word (the **key**) to find its meaning (the **value**). Write a dictionary with curly braces, and put a **colon** between each key and value.

- **Ordered** (Python 3.7+): insertion order is kept.
- **Mutable**: you can add, change and delete pairs.
- **Keys are unique**: if you repeat a key, the last value silently overwrites the earlier one. Values may repeat.



You never search a dictionary by position. You search by key.

A school analogue: a **roll number** (key) can point to only one student (value). If you assign that roll number to someone else, the old link is gone.

### Adding, changing and deleting

```
d = {"name": "Alice", "age": 30}
print(d)
d["age"] = 31
print(d)
d["city"] = "NYC"
print(d)
del d["city"]
print(d)
```

#### OUTPUT

```
{'name': 'Alice', 'age': 30}
{'name': 'Alice', 'age': 31}
{'name': 'Alice', 'age': 31, 'city': 'NYC'}
{'name': 'Alice', 'age': 31}
```

Assigning to an existing key updates the value. Assigning to a new key adds a pair. `del d["city"]` removes that pair.

**The duplicate-keys rule.** Python does not crash. The last assignment wins.

```
d = {"a": 1, "a": 2, "a": 3}
print(d)
```

#### OUTPUT

```
{'a': 3}
```

### Accessing values safely

#### 1. Direct brackets (risky)

`dict[key]` works if the key exists. If it is missing, the program crashes with `KeyError`.

#### 2. `.get()` (safe)

`dict.get(key)` returns `None` if the key is missing. You may pass a default: `dict.get(key, 0)`.

```

person = {"name": "Alice", "age": 30}
print(person["name"])
print(person.get("salary"))
print(person.get("salary", 0))
print("name" in person)
print("salary" in person)

```

#### OUTPUT

```

Alice
None
0
True
False

```

The operator `in` tests **keys**, not values. `"Alice" in person` would be `False`.

```

person = {"name": "Alice", "age": 30}
print(person["salary"])

```

#### OUTPUT

```

Traceback (most recent call last):
  File "keyerror.py", line 2, in <module>
    print(person["salary"])
  KeyError: 'salary'

```

## Dictionary views and traversal

Three view methods return live objects. If the dictionary changes, the view reflects the change immediately.

| Method                 | Returns                    |
|------------------------|----------------------------|
| <code>.keys()</code>   | All keys.                  |
| <code>.values()</code> | All values.                |
| <code>.items()</code>  | Key–value pairs as tuples. |

```

d = {"a": 1}
keys = d.keys()
print(keys)
d["b"] = 2
print(keys)

```

#### OUTPUT

```

dict_keys(['a'])
dict_keys(['a', 'b'])

```

The variable `keys` was not rebuilt. The view updated itself.

```

inventory = {"apple": 50, "banana": 30, "cherry": 20}
print(list(inventory.keys()))
print(list(inventory.values()))
print(list(inventory.items()))
for key in inventory:
    print(key)
for val in inventory.values():
    print(val)
for key, val in inventory.items():
    print(f"{key} costs {val}")

```

**OUTPUT**

```
['apple', 'banana', 'cherry']
[50, 30, 20]
[('apple', 50), ('banana', 30), ('cherry', 20)]
apple
banana
cherry
50
30
20
apple costs 50
banana costs 30
cherry costs 20
```

A plain `for key in inventory` loops over keys (the default). `.items()` is the most useful form because you get both sides at once.

## Comparison: list, tuple, set, dictionary

| Feature      | List               | Tuple            | Set                           | Dictionary                                   |
|--------------|--------------------|------------------|-------------------------------|----------------------------------------------|
| Written with | [ ]                | ( )              | { }                           | {key: value}                                 |
| Ordered      | Yes                | Yes              | No                            | Yes (Python 3.7+)                            |
| Mutable      | Yes                | No               | Yes (items must be immutable) | Yes                                          |
| Duplicates   | Allowed            | Allowed          | Removed                       | Duplicate keys not allowed (last value wins) |
| Access by    | Index 0, 1, 2...   | Index 0, 1, 2... | No index                      | Key                                          |
| Empty value  | []                 | ()               | set()                         | {}                                           |
| Typical use  | Changing sequences | Fixed records    | Unique items, set maths       | Look-up tables                               |

**WORKED EXAMPLE**

**Same data, four structures.** Three subject marks:

- List `[80, 70, 90]` — easy to append a fourth test.
- Tuple `(80, 70, 90)` — a frozen result slip.
- Set `{80, 70, 90}` — unique scores only; two papers of 80 would become one 80.
- Dictionary `{"math": 80, "eng": 70, "cs": 90}` — you ask "What is CS?" and get 90 without remembering an index.

**REMEMBER IT**

**Keys are unique, like roll numbers.** Values are the student records and may repeat (two students can both score 80). Memory: "**get** is gentle, brackets can **break**."

**COMMON MISTAKE**

Older notes (and the chapter summary in some slides) still call dictionaries "unordered." From Python 3.7 onwards insertion order is preserved, so you may write "ordered since 3.7." Keys must be immutable: a list cannot be a key; a tuple of numbers can. Do not write `{ }` when you want an empty set.



ڈکشنری (Dictionary) کلید اور قدر (Key-Value) کے جوڑوں میں ڈیٹا رکھتی ہے۔ کلید منفرد ہوتی ہے؛ دہرائی ہوئی کلید پر آخری قدر رہ جاتی ہے۔ `d[key]` غیر موجود کلید پر `KeyError` دیتا ہے، جبکہ `d.get(key, default)` محفوظ ہے۔ `.values()`، `keys()` اور `.items()` سے گھما جا سکتا ہے۔ فہرست انڈیکس سے، ڈکشنری کلید سے پہچانی جاتی ہے۔

**EXAM TIP**

High-yield: definition of a dictionary, difference between `d[key]` and `d.get(key)`, "what if a key is repeated?", traversal with `.items()`, and a full comparison table of the four structures. Memorise the comparison table in this section — it is a complete 8-mark answer if you add one example row. Always mention that `in` checks keys.

**CHECK YOURSELF**

1. What does `{"a": 1, "a": 2, "a": 3}` print?
2. What does `person.get("salary", 0)` return if `salary` is missing?
3. Which structure has no index and no duplicate items?

Answers: 1. `{'a': 3}` . 2. `0` . 3. A set.

### 3.1.5 Common Built-in Functions (min, max, len, sum)

**LEARN IT**

Several built-in functions work across lists, tuples, sets and dictionaries. You do not call them with a dot on the object (except when a type also has its own method, such as `list.sort()`).

| Function              | Purpose                                               | Works on                                                                    |
|-----------------------|-------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>min()</code>    | Smallest element (numeric or alphabetical)            | List, tuple, set, <b>dictionary keys</b>                                    |
| <code>max()</code>    | Largest element                                       | List, tuple, set, <b>dictionary keys</b>                                    |
| <code>len()</code>    | Number of elements (for a dict: number of keys)       | List, tuple, set, dictionary                                                |
| <code>sum()</code>    | Adds numeric elements                                 | List, tuple, set, <b>dictionary values</b> via <code>sum(d.values())</code> |
| <code>sorted()</code> | Returns a <b>new sorted list</b> ; original unchanged | List, tuple, set, dictionary keys                                           |

```
s = {5, 1, 9, 3, 7}
print(len(s))
print(min(s))
print(max(s))
print(sum(s))
print(sorted(s))
```

**OUTPUT**

```
5
1
9
25
[1, 3, 5, 7, 9]
```

$5 + 1 + 9 + 3 + 7 = 25$ . `sorted(s)` always returns a **list**, even when the input is a set.

#### WORKED EXAMPLE

The same five functions on four structures.

```
nums = [8, 3, 5, 3]
t = (8, 3, 5, 3)
s = {8, 3, 5}
d = {"math": 80, "eng": 70, "cs": 90}
print(len(nums), min(nums), max(nums), sum(nums), sorted(nums))
print(len(t), min(t), max(t), sum(t), sorted(t))
print(len(s), min(s), max(s), sum(s), sorted(s))
print(len(d), min(d), max(d), sum(d.values()), sorted(d))
```

##### OUTPUT

```
4 3 8 19 [3, 3, 5, 8]
4 3 8 19 [3, 3, 5, 8]
3 3 8 16 [3, 5, 8]
3 cs math 240 ['cs', 'eng', 'math']
```

On the dictionary, `min(d)` and `max(d)` compare **keys as strings**: "cs" comes first alphabetically, "math" last. `sum(d)` would try to add the keys "math" + "eng" + ... and fail. Use `sum(d.values())` —  $80 + 70 + 90 = 240$ . The set dropped the duplicate 3, so `len(s)` is 3 and `sum(s)` is 16.

#### `sorted()` versus `.sort()`

```
original = [8, 3, 5]
print("before sort:", original)
print("sorted():", sorted(original))
print("after sorted():", original)
original.sort()
print("after .sort():", original)
```

##### OUTPUT

```
before sort: [8, 3, 5]
sorted(): [3, 5, 8]
after sorted(): [8, 3, 5]
after .sort(): [3, 5, 8]
```

`.sort()` exists only on lists and rewrites the original. `sorted()` works on any iterable and leaves the original alone.

#### REMEMBER IT

**LEN-MIN-MAX-SUM** — count, smallest, largest, total. For a dictionary: `len` / `min` / `max` / `sorted` use **keys**; `sum` needs `.values()`.

#### COMMON MISTAKE

`sum()` needs numbers. `sum(["a", "b"])` raises `TypeError`. Empty structures: `len([])` is 0, but `min([])` and `max([])` raise `ValueError` because there is no smallest of nothing.

#### اردو میں سمجھیں

`len()` عناصر گنتا ہے، `min()` سب سے چھوٹا، `max()` سب سے بڑا، `sum()` اعداد کا مجموعہ، اور `sorted()` نئی ترتیب شدہ فہرست دیتا ہے بغیر اصل کو بدلے ہیں۔ ڈکشنری پر `min` اور `max` کلیدوں پر چلتے ہیں؛ قدروں کا مجموعہ `sum(d.values())` سے نکالیں۔ `sort()` صرف فہرست کا میتھڈ ہے جو اصل فہرست بدل دیتا ہے۔

### CHECK YOURSELF

1. For `s = {5, 1, 9, 3, 7}`, what are `len(s)`, `min(s)`, `max(s)`, `sum(s)`?
2. Does `sorted(s)` return a set or a list?
3. How do you total the values of `{"math": 80, "eng": 70, "cs": 90}`?

Answers: 1. 5, 1, 9, 25. 2. A list: `[1, 3, 5, 7, 9]`. 3. `sum(d.values())` → 240.

## 3.1.6 Activity: Student Attendance System

### LEARN IT

Teachers need a quick way to mark who is in class. This program stores names in a **list**, asks about each student, counts how many said yes, and prints a summary with `len()`.

Store names in a list → Ask present? for each → Count yes → Print totals

```
students = ["Ali", "Sara", "Ayesha"]
present_count = 0
print("--- Attendance System ---\n")
for student in students:
    status = input(f"Is {student} present? (yes/no): ").lower()
    if status == "yes":
        present_count += 1
print("\n--- Attendance Summary ---")
print("Total Students:", len(students))
print("Students Present:", present_count)
print("Students Absent:", len(students) - present_count)
```

### WORKED EXAMPLE

**Sample run.** Ali yes, Sara no, Ayesha yes. `.lower()` means YES or Yes still counts as present.

#### OUTPUT

```
--- Attendance System ---

Is Ali present? (yes/no): yes
Is Sara present? (yes/no): no
Is Ayesha present? (yes/no): yes

--- Attendance Summary ---
Total Students: 3
Students Present: 2
Students Absent: 1
```

| Student | Typed answer | After <code>.lower()</code> | <code>present_count</code> |
|---------|--------------|-----------------------------|----------------------------|
| Ali     | yes          | yes → add 1                 | 1                          |
| Sara    | no           | not yes                     | 1                          |
| Ayesha  | yes          | yes → add 1                 | 2                          |

Absent = `len(students) - present_count` = 3 - 2 = 1. The counter must start at 0 **before** the loop. If you put `present_count = 0` inside the loop, the count would reset for every student.

#### REMEMBER IT

**Counter pattern:** start at 0, loop, add 1 when a condition is true. You will reuse this in the word-frequency program (add 1 per word) and the grade tracker (add each mark).

#### COMMON MISTAKE

Only the exact string "yes" after `.lower()` counts as present. "y" or "present" is treated as absent. In a longer program you could also accept "y"; this activity keeps the rule simple.

#### اردو میں سمجھیں

حاضری کا پروگرام طلبہ کے نام ایک فہرست میں رکھتا ہے، ہر نام کے لیے پوچھتا ہے، اور yes پر شمار کار (Counter) ایک بڑھاتا ہے۔ کل طلبہ `len(students)` سے، غیر حاضر کل - حاضر سے نکلتے ہیں۔ شمار کار لوپ سے پہلے صفر ہونا چاہیے۔

#### CHECK YOURSELF

1. If all three students type no, what is printed for present and absent?
2. Why is `.lower()` used on the answer?

Answers: 1. Present 0, Absent 3. 2. So that YES, Yes and yes all match the string "yes".

### 3.1.7 Activity: Word Frequency Counter

#### LEARN IT

Count how many times each word appears in a sentence. This is a classic use of a **dictionary** plus `.get()`: the word is the key, the count is the value.

```
sentence = "python is easy python is fun python is powerful"
words = sentence.split()
word_freq = {}
for word in words:
    word_freq[word] = word_freq.get(word, 0) + 1
print(f"Sentence: '{sentence}'\n")
print("Word Frequency Table:")
for word, count in word_freq.items():
    print(f"{word:<10} : {count}")
top_word = max(word_freq, key=word_freq.get)
print(f"\nMost Frequent: '{top_word}' ({word_freq[top_word]} times)")
```

#### WORKED EXAMPLE

**Logic, step by step.**

1. `sentence.split()` cuts on spaces and gives `['python', 'is', 'easy', 'python', 'is', 'fun', 'python', 'is', 'powerful']`.
2. For a new word, `word_freq.get(word, 0)` is 0, then you store  $0 + 1 = 1$ .
3. For a word already in the dictionary, `.get(word, 0)` returns the current count, then you add 1.
4. `{word:<10}` pads the word to 10 characters so the colons line up.
5. `max(word_freq, key=word_freq.get)` picks the key whose value is largest. If two words share the top count, `max` returns the first of those keys in insertion order.

#### OUTPUT

Sentence: 'python is easy python is fun python is powerful'

Word Frequency Table:

```
python      : 3
is          : 3
easy        : 1
fun         : 1
powerful    : 1
```

Most Frequent: 'python' (3 times)

| After seeing        | word_freq                                                  |
|---------------------|------------------------------------------------------------|
| python              | {'python': 1}                                              |
| is                  | {'python': 1, 'is': 1}                                     |
| easy                | {'python': 1, 'is': 1, 'easy': 1}                          |
| python (again)      | {'python': 2, 'is': 1, 'easy': 1}                          |
| ... end of sentence | {'python': 3, 'is': 3, 'easy': 1, 'fun': 1, 'powerful': 1} |

Here python and is both occur 3 times. max returns 'python' because it appears first in the dictionary.

#### REMEMBER IT

The heart of every counter is one line: `d[item] = d.get(item, 0) + 1`. You can count letters, roll numbers or votes with the same pattern.

#### COMMON MISTAKE

This program is case-sensitive and space-based. "Python" and "python" would be two keys. Punctuation such as `easy.` would not match `easy`. For a class exercise, keep the sentence lowercase with spaces only, as here. If you wrote `word_freq[word] += 1` without `.get` on a new word, you would get `KeyError`.

#### اردو میں سمجھیں

جملے کو `.split()` سے الفاظ کی فہرست بناتے ہیں، پھر ہر لفظ ڈکشنری میں کلید ہوتا ہے اور اس کی گنتی قدر۔ نئے لفظ کے لیے `get(word, 0) + 1` سے گنتی 1 سے شروع ہوتی ہے۔ `max(... , key=word_freq.get)` سب سے زیادہ بار آنے والا لفظ دیتا ہے۔

#### CHECK YOURSELF

1. How many times does "is" occur in the sample sentence?
2. Why use `.get(word, 0)` instead of `word_freq[word]` on the right-hand side?

Answers: 1. Three times. 2. Because a brand-new word is not yet a key; `.get` returns 0 instead of raising `KeyError`.

## 3.2 Functions in Python ★ Golden

### LEARN IT

When the same instructions are written again and again, that is **duplicate code**. Duplicate code makes a program longer, harder to read, and painful to update (you must fix every copy). A **function** lets you write the instructions **once** and run them whenever you call the function's name.

| Benefit          | Meaning                                         |
|------------------|-------------------------------------------------|
| Code reusability | Write once, use anywhere.                       |
| Modularity       | Break a big problem into small named tasks.     |
| Readability      | Shorter, cleaner files with meaningful names.   |
| Maintainability  | Fix a bug in one place and every call is fixed. |

```
def greet():  
    print("Hello, World!")  
  
greet()  
greet()
```

### OUTPUT

```
Hello, World!  
Hello, World!
```

The body is written once. Two calls run it twice. To change the message you edit one line inside `greet`.

### Key components of a function



- **Keyword** — Python uses `def` to start a definition.
- **Function name** — the identifier you call, e.g. `greet`.
- **Parameters** — input names inside the parentheses. When you call the function, the values you pass are called **arguments**.
- **Body** — the indented block that runs on each call.
- **Return statement** — sends a result back to the caller (3.2.2).

```
def add(a, b):  
    print(f"Adding {a} + {b}")  
    return a + b  
  
result = add(4, 6)  
print(result)
```

### OUTPUT

```
Adding 4 + 6  
10
```

Here `a` and `b` are parameters. `4` and `6` are arguments. `return a + b` sends 10 back, and that 10 is stored in `result`.

#### WORKED EXAMPLE

**Without a function** you would copy the print-and-add lines every time you needed a sum. With a function, a new sum is one call: `add(10, 20)`, `add(1.5, 2.5)`.

#### REMEMBER IT

**DRY — Don't Repeat Yourself.** If you are about to copy-paste a block, wrap it in `def`. Name functions with verbs: `greet`, `add`, `save_grades`.

#### COMMON MISTAKE

Defining is not calling. After `def greet()`: nothing prints until you write `greet()`. Forgetting the parentheses — writing `greet` alone — does not run the body. The body must be indented; a line at column 0 is outside the function.

#### اردو میں سمجھیں

فنکشن (Function) ہدایات کا وہ بلاک ہے جو ایک بار لکھا جاتا ہے اور بار بار بلایا جا سکتا ہے۔ اس سے کوڈ مختصر، ماڈیولر اور آسان اصلاح والا بنتا ہے۔ تعریف `def` سے شروع ہوتی ہے، نام اور پیرامیٹرز قوسین میں ہوتے ہیں، جسم انڈینٹ ہوتا ہے، اور `return` نتیجہ واپس بھیج سکتا ہے۔ تعریف کرنے سے فنکشن چلتا نہیں جب تک آپ اسے کال نہ کریں۔

#### EXAM TIP

Write the four benefits (reusability, modularity, readability, maintainability) as separate bullets — that is a standard short question. For long questions, define a function, label `def`, name, parameters, body and `return`, then show one call. "What is duplicate code and how do functions solve it?" is also common.

#### CHECK YOURSELF

1. What keyword starts a function definition?
2. How many times does `Hello, World!` print in the `greet` program above, and why?

Answers: 1. `def`. 2. Twice, because `greet()` is called twice.

### 3.2.1 Types of Functions (Built-in vs User-defined)

#### LEARN IT

There are two main types of functions in Python.

**a) Built-in functions** are already defined by Python. You just call them. Examples you have already used: `len()`, `range()`, `sum()`, `print()`, `input()`, `min()`, `max()`, `sorted()`, `int()`, `float()`, `type()`.

**b) User-defined functions** are functions **you** create with `def` for tasks that no built-in covers in one step — greeting a user, dividing with a zero check, writing a grade file.

|                | Built-in                               | User-defined                             |
|----------------|----------------------------------------|------------------------------------------|
| Who writes it? | The Python language                    | The programmer                           |
| How you start  | Just call the name                     | First write <code>def name(...):</code>  |
| Examples       | <code>len("Ali")</code> → 3            | <code>def add(a, b): return a + b</code> |
| Purpose        | Everyday operations needed by everyone | Your program's specific jobs             |

#### WORKED EXAMPLE

**Side by side.** Counting letters is built-in. Combining a greeting with a name is user-defined.

```
print(len("Sindh"))

def greet_student(name):
    print("Hello,", name)

greet_student("Hassan")
greet_student("Malika")
```

#### OUTPUT

```
5
Hello, Hassan
Hello, Malika
```

You did not write the code inside `len`. You did write the code inside `greet_student`, then reused it for two names.

#### REMEMBER IT

If Python already has it (`len`, `sum`, `max`), do not rewrite it. Write a user-defined function when you are packaging **your** steps.

#### COMMON MISTAKE

Do not name your function `list`, `print`, `sum` or `input`. You would hide the built-in name for the rest of that file. Choose names such as `print_bill` or `sum_marks`.

#### اردو میں سمجھیں

بلٹ ان فنکشن پائتھن میں پہلے سے موجود ہوتے ہیں، جیسے `len()` اور `print()`۔ یوزر ڈیفائنڈ فنکشن وہ ہیں جو پروگرامر `def` سے خود بناتا ہے تاکہ مخصوص کام بار بار کیا جاسکے۔ جو کام بلٹ ان پہلے کر دیتے ہیں انہیں دوبارہ نہ لکھیں۔

#### CHECK YOURSELF

1. Is `range()` built-in or user-defined?
2. Which keyword creates a user-defined function?

Answers: 1. Built-in. 2. `def`.

## 3.2.2 Functions with Return Values

#### LEARN IT

A function can either **print** a result (show it on the screen) or **return** a result (send it back to the caller). A returned value can be stored in a variable, printed later, or used in a further calculation. The call is replaced by the returned value, as if you had written that value in its place.

```
def multiply(a, b):
    return a * b

result = multiply(5, 3)
print("The answer is:", result)
```



#### OUTPUT

The answer is: 15

`multiply(5, 3)` evaluates to 15. That 15 is stored in `result`.

#### WORKED EXAMPLE

**Print versus return.** This is the difference examiners want.

```
def show_sum(a, b):  
    print(a + b)  
  
def give_sum(a, b):  
    return a + b  
  
x = show_sum(2, 3)  
print("stored from show_sum:", x)  
y = give_sum(2, 3)  
print("stored from give_sum:", y)  
print("double it:", y * 2)
```

#### OUTPUT

```
5  
stored from show_sum: None  
stored from give_sum: 5  
double it: 10
```

`show_sum` prints 5 inside the function, then returns `None` (the default when there is no `return`). You cannot double that answer later. `give_sum` hands 5 back, so `y * 2` is 10. Use `print` for messages; use `return` for results the rest of the program still needs.

#### REMEMBER IT

**return = send back; print = show now.** `return` also **exits** the function immediately. Lines after `return` in the same function do not run.

#### COMMON MISTAKE

Writing `print(a * b)` inside `multiply` and then `result = multiply(5, 3)` stores `None` in `result`, not 15. If you later write `result + 1` you get `TypeError`. Return the number; let the caller decide whether to print it.

#### اردو میں سمجھیں

`return` نتیجہ کال کرنے والے پروگرام کو واپس بھیجتا ہے تاکہ وہ متغیر میں محفوظ ہو سکے یا آگے حساب میں لگے۔ اگر `return` نہ ہو تو فنکشن `None` دیتا ہے۔ `print` صرف اسکرین پر دکھاتا ہے؛ اس سے حاصل شدہ قیمت دوبارہ استعمال نہیں ہوتی۔

#### CHECK YOURSELF

1. What is stored in `result` after `result = multiply(5, 3)` with `return a * b`?
2. What does a function return if it has no `return` statement?

Answers: 1. 15. 2. `None`.

### 3.2.3 Activity: Arithmetic Calculator Using Functions

#### LEARN IT

Each arithmetic operation is a **separate function**. That is modularity: addition is one small task, division is another (with a guard against dividing by zero). The main program only asks for two numbers and calls the functions.

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
print("Addition:", add(num1, num2))
print("Subtraction:", subtract(num1, num2))
print("Multiplication:", multiply(num1, num2))
print("Division:", divide(num1, num2))
```

#### WORKED EXAMPLE

**Sample run** with 12 and 4. `float` is used so decimals work; even whole numbers print with a decimal (12 becomes 12.0).

##### OUTPUT

```
Enter first number: 12
Enter second number: 4
Addition: 16.0
Subtraction: 8.0
Multiplication: 48.0
Division: 3.0
```

Check:  $12 + 4 = 16$ ,  $12 - 4 = 8$ ,  $12 \times 4 = 48$ ,  $12 \div 4 = 3$ . In Python 3, `/` always produces a float, so 3.0 not 3.

#### Zero divisor:

```
def divide(a, b):
    if b != 0:
        return a / b
    else:
        return "Cannot divide by zero"

print(divide(10, 0))
print(divide(10, 2))
```

##### OUTPUT

```
Cannot divide by zero
5.0
```

Without the `if b != 0` test, `10 / 0` would raise `ZeroDivisionError` and stop the program.

#### REMEMBER IT

Four functions, four names: `add`, `subtract`, `multiply`, `divide`. The main program should look like a contents page — a sequence of calls — not a pile of formulae.

#### COMMON MISTAKE

`input()` returns a string. If you skip `float()` (or `int()`), `add` would concatenate: `"12" + "4"` is `"124"`. Convert first. Also, `divide` returns either a number or a string; do not later do arithmetic on the error message.

#### اردو میں سمجھیں

کیلکولیٹر میں ہر عمل الگ فنکشن ہے: جمع، تفریق، ضرب، تقسیم۔ یہ ماڈیولریٹی ہے۔ تقسیم سے پہلے صفر کی جانچ ضروری ہے ورنہ پروگرام رک جاتا ہے۔ `input()` سے ملنے والی تار کو `float()` سے عدد بنانا لازمی ہے ورنہ جمع جوڑنے کے بجائے الفاظ جوڑ دے گی۔

#### CHECK YOURSELF

1. What does `divide(10, 0)` return in this activity?
2. Why are the numbers converted with `float()` ?

Answers: 1. The string "Cannot divide by zero". 2. Because `input()` returns text; `float()` turns it into a number so `+` `-` `*` `/` do arithmetic, including decimals.

## 3.2.4 Scope of Local and Global Variables

#### LEARN IT

**Scope** is the region of a program where a variable can be seen and used. Python has two scopes you must know for this chapter.

- i) Local scope.** A **local variable** is created inside a function. It exists only while that function is running. The main program cannot read it.
- ii) Global scope.** A **global variable** is created outside all functions. Any function may **read** it, and the main program can use it too.

#### WORKED EXAMPLE

**Local — the name is invisible outside.**

```
def calc():
    secret = 50
    print(secret)

calc()
print(secret)
```

#### OUTPUT

```
50
Traceback (most recent call last):
  File "local_scope.py", line 5, in <module>
    print(secret)
NameError: name 'secret' is not defined
```

First the function prints 50. Then the main program tries `print(secret)` and fails: `secret` was born inside `calc` and died when `calc` finished.

**Global — readable inside and outside.**

```
x = 10

def show():
    print("Inside function:", x)

show()
print("Outside function:", x)
```

#### OUTPUT

```
Inside function: 10
Outside function: 10
```

**Assignment inside a function creates a new local** with the same name. It does not change the global:

```
x = 10

def show():
    x = 99
    print("Inside:", x)

show()
print("Outside:", x)
```

#### OUTPUT

```
Inside: 99
Outside: 10
```

Inside, `x = 99` is a local. After the call, the global is still 10.

#### REMEMBER IT

**Local = inside the room; global = in the corridor.** Everyone can see the corridor. Things left in a room are gone when you leave. Prefer locals. Too many globals make bugs hard to find because any function might be using them.

#### COMMON MISTAKE

If you only **read** `x` inside the function, Python uses the global. If you **assign** to `x` anywhere in that function, Python treats `x` as local for the whole function. Mixing the two without care causes `UnboundLocalError`. Keep function inputs as parameters and outputs as `return` values instead of editing globals.

دائرہ کار (Scope) بتاتا ہے کہ متغیر کہاں دکھائی دیتا ہے۔ لوکل متغیر فنکشن کے اندر بنتا ہے اور فنکشن ختم ہوتے ہی غائب ہو جاتا ہے، باہر سے پڑھنے پر `NameError` آتا ہے۔ گلوبل متغیر تمام فنکشنز سے باہر بنتا ہے اور اندر سے پڑھا جا سکتا ہے۔ فنکشن کے اندر اسی نام کو نئی قدر دینے سے نیا لوکل بنتا ہے، گلوبل نہیں بدلتا۔

**CHECK YOURSELF**

1. Where is a local variable defined?
2. In the last example, what does the main program print for `x` after `show()` ?

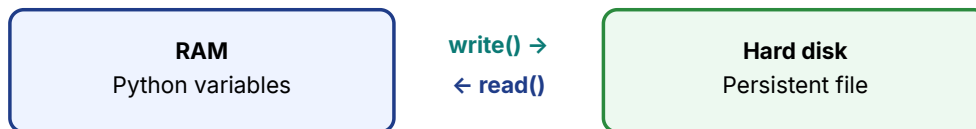
Answers: 1. Inside a function. 2. 10 (the global is unchanged).

**3.3 & 3.4 File Handling and Operations (Modes `r`, `w`, `a`)**

★ Golden

**LEARN IT**

**File handling** is a program's ability to create, read, write, update and delete files on disk. Variables live in **RAM** and vanish when the program ends. Files live on a **hard disk or SSD** and stay there — they are **persistent**.



Read copies data from disk into RAM. Write copies data from RAM onto disk.

**Types of files**

| Type        | What it stores                                               | Examples                                                  |
|-------------|--------------------------------------------------------------|-----------------------------------------------------------|
| Text file   | Human-readable characters. You can open it in a text editor. | <code>.txt</code> , <code>.csv</code> , <code>.py</code>  |
| Binary file | Raw bytes, not meant to be read as text.                     | <code>.jpg</code> , <code>.mp3</code> , <code>.dat</code> |

This chapter uses text files.

**The `open()` function and modes**

You must open a file before you use it: `open(filepath, mode)`.

| Mode                                  | Name   | Description                                                                                |
|---------------------------------------|--------|--------------------------------------------------------------------------------------------|
| <code>'r'</code>                      | Read   | Open for reading. <b>Fails</b> if the file does not exist.                                 |
| <code>'w'</code>                      | Write  | Open for writing. <b>Erases</b> existing content. Creates the file if it is missing.       |
| <code>'a'</code>                      | Append | Open for writing at the <b>end</b> . Keeps old content. Creates the file if it is missing. |
| <code>'rb'</code> / <code>'wb'</code> | Binary | Read or write bytes (images, audio).                                                       |

## File object methods

| Method                      | What it does                                                                                |
|-----------------------------|---------------------------------------------------------------------------------------------|
| <code>.read()</code>        | Whole file as one string.                                                                   |
| <code>.readlines()</code>   | All lines as a <b>list</b> of strings (each usually ends with <code>\n</code> ).            |
| <code>.write("text")</code> | Writes a string. You must add <code>\n</code> yourself for a new line.                      |
| <code>.close()</code>       | Saves to disk, closes the file, frees RAM. Essential if you did not use <code>with</code> . |

### WORKED EXAMPLE

#### Create a file in write mode.

```
new_file = open('myfile.txt', 'w')
new_file.write("The quick brown fox jumps over the lazy dog!!!\n")
new_file.close()
print("File has been created successfully.")
```

##### OUTPUT

```
File has been created successfully.
```

If `myfile.txt` did not exist, `'w'` created it. The sentence is now on disk. `.close()` flushes the buffer so the bytes actually reach the disk.

**Danger of `'w'`:** opening an existing file in `'w'` deletes everything first.

```
f = open('myfile.txt', 'w')
f.write("FIRST\n")
f.close()
f = open('myfile.txt', 'w')
f.write("SECOND\n")
f.close()
with open('myfile.txt', 'r') as f:
    print(f.read(), end="")
```

##### OUTPUT

```
SECOND
```

`FIRST` is gone. To keep old lines and add a new one, use `'a'`:

```
f = open('notes.txt', 'w')
f.write("Line 1\n")
f.close()
f = open('notes.txt', 'a')
f.write("Line 2\n")
f.close()
with open('notes.txt', 'r') as f:
    print(f.read(), end="")
```

##### OUTPUT

```
Line 1
Line 2
```

#### REMEMBER IT

**r = Read existing, w = Wipe then write, a = Add at end.** Remember: `write` does not add a newline unless you type `\n`.

#### COMMON MISTAKE

Forgetting `.close()` can leave data in a memory buffer so the file on disk looks empty or incomplete — or the file can stay locked. Section 3.6 shows `with`, which closes for you even after an error. Never use `'w'` when you only meant to add a line; that is the most common way students wipe their own data.

#### اردو میں سمجھیں

فائل ہینڈلنگ سے ڈیٹا ڈسک پر مستقل (Persistent) رہتا ہے، برخلاف RAM کے متغیرات کے جو پروگرام بند ہوتے ہی مٹ جاتے ہیں۔  
`open(path, mode)` سے فائل کھلتی ہے: `'r'` پڑھنے کے لیے (فائل ہونی چاہیے)، `'w'` لکھنے کے لیے (پرانے مواد مٹ جاتا ہے)، `'a'` آخر میں جوڑنے کے لیے۔ کام کے بعد `.close()` ضروری ہے اگر `with` استعمال نہ ہو۔

#### EXAM TIP

Define file handling, contrast RAM with disk, list modes `r / w / a` with one property each, and warn that `w` overwrites. Short programs that write one line and close the file are common. Mention `.read()`, `.readlines()`, `.write()` and `.close()` by name.

#### CHECK YOURSELF

1. Which mode overwrites an existing file?
2. Which mode fails if the file is missing?
3. Why must `.write` often include `\n`?

Answers: 1. `'w'` . 2. `'r'` . 3. Because `write()` does not start a new line by itself; without `\n` the next write continues on the same line.

## 3.6 The with Statement (Context Manager) ★ Golden

#### LEARN IT

It is easy to forget `.close()`. If the program crashes before `.close()` runs, the file may be incomplete or corrupted. Python's `with` statement is a **context manager**: it opens the file, runs the indented block, and **always closes** the file when the block ends — even if an error occurs inside.

with open(...) as f → use f → block ends → file closed automatically

```
with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")
# no .close() here – with already closed the file
```

After the indented block, `new_file` is closed. You do not write `new_file.close()`.

### 3.7 Activity: reading file content

`.read()` pulls the whole file into RAM. For a huge file that can waste memory. The professional pattern is to **iterate line by line**.

```

with open('myfile.txt', 'w') as new_file:
    new_file.write("Context managers are safe!\n")

with open('myfile.txt', 'r') as read_file:
    for line in read_file:
        print(line, end="")
print("\n\nFile read successfully.")

```

#### OUTPUT

```

Context managers are safe!

File read successfully.

```

`end=""` stops `print` from adding a second newline: each line from the file already ends with `\n`.

#### WORKED EXAMPLE

**Three ways to read** the same small file.

```

with open('sample.txt', 'w') as f:
    f.write("Ali 80\nSara 90\nAyesha 75\n")

with open('sample.txt', 'r') as f:
    print("≡ .read() ≡")
    print(repr(f.read()))

with open('sample.txt', 'r') as f:
    print("≡ .readlines() ≡")
    print(f.readlines())

with open('sample.txt', 'r') as f:
    print("≡ line by line ≡")
    for line in f:
        print(line, end="")

```

#### OUTPUT

```

≡ .read() ≡
'Ali 80\nSara 90\nAyesha 75\n'
≡ .readlines() ≡
['Ali 80\n', 'Sara 90\n', 'Ayesha 75\n']
≡ line by line ≡
Ali 80
Sara 90
Ayesha 75

```

`repr` shows the hidden `\n` characters. `.readlines()` is handy when you need a list (the grade tracker uses this). Line-by-line looping is best for large files because only one line sits in memory at a time.

#### REMEMBER IT

**with = “take, use, put back.”** Prefer `with open(...)` as `f`: in every new program. You still choose the mode: `'r'`, `'w'` or `'a'`.

#### COMMON MISTAKE

You cannot read from `f` after the `with` block — the file is already closed. Do all reading and writing inside the block. Also, `print(line)` without `end=""` looks double-spaced because both the line and `print` add a newline.



with بیان ایک کانٹیکسٹ مینیجر (Context Manager) ہے جو فائل کو بلاک کے اختتام پر خود بند کر دیتا ہے، چاہے اندر غلطی ہو جائے۔ بڑی فائلیں ایک ایک سطر کے لوپ سے پڑھنا بہتر ہے بجائے پورے مواد کو .read() سے میموری میں بھرنے کے۔  
print(line, end="") دگنی خالی سطر روکتا ہے۔

**EXAM TIP**

Define the with statement as a context manager that guarantees close. Contrast it with manual open / close. Be ready to write a four-line program that writes with with and a four-line program that reads line by line. Mention memory: iterating is safer for large files than .read().

**CHECK YOURSELF**

1. What does a context manager guarantee for a file?
2. Why pass end="" when printing each line from a file?

Answers: 1. That the file is closed when the with block ends, even if an error occurs. 2. Because each line already ends with \n; print would otherwise add another newline and the output would look double-spaced.

## 3.8 Errors and Exceptions (Try-Except)

**LEARN IT**

An **exception** is an error raised while the program is running — for example opening a file that is not there. Without handling, Python prints a traceback and **stops**. A try / except block catches the error and lets you show a clear message instead of crashing.

| Error type        | Meaning                                                        |
|-------------------|----------------------------------------------------------------|
| FileNotFoundError | The path does not exist.                                       |
| PermissionError   | The operating system will not let you read or write that file. |
| IOError           | A general input/output problem (disk full, disk disconnected). |

Put risky code in try. Put the recovery in except ErrorType. A final except Exception as e can catch anything else, using e as the error object.

```
try:
    with open('missing_data.txt', 'r') as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")
```

**OUTPUT**

Error: The file does not exist. Please check the name.

Without `try`, the same open raises:

#### OUTPUT

```
Traceback (most recent call last):
  File "missing.py", line 1, in <module>
    open('missing_data.txt', 'r')
FileNotFoundError: [Errno 2] No such file or directory: 'missing_data.txt'
```

#### WORKED EXAMPLE

When the file exists, the except blocks are skipped.

```
with open('ok.txt', 'w') as f:
    f.write("Present\n")

try:
    with open('ok.txt', 'r') as f:
        content = f.read()
        print(content, end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

#### OUTPUT

Present

Professional software does not dump a red traceback on a user. If a file is missing or a disk fails, the user should see a calm sentence such as "Please check the file name" or "Please try again later."

#### REMEMBER IT

**try = attempt; except = backup plan.** Catch the **specific** error first ( `FileNotFoundError` ), then a general `Exception` if you need it. Order matters: a bare `except Exception` first would swallow the specific error.

#### COMMON MISTAKE

Do not wrap your entire program in one silent `except:` that ignores every mistake. You would hide real bugs. Catch the errors you expect, and keep the message honest. `'w'` and `'a'` create a missing file, so they do not raise `FileNotFoundError`; `'r'` does.

#### اردو میں سمجھیں

ایکسپشن (Exception) چلتے پروگرام کی غلطی ہے، جیسے غیر موجود فائل کھولنا۔ `try` میں خطرناک کوڈ لکھیں اور `except` میں `FileNotFoundError` صاف پیغام دکھائیں تاکہ پروگرام کریش نہ ہو۔ پیشہ ورانہ سافٹ ویئر صارف کو سرخ ٹریس بیک کی بجائے سمجھ آنے والا جملہ دکھاتا ہے۔

#### CHECK YOURSELF

1. Which exception is raised when `open('missing_data.txt', 'r')` cannot find the file?
2. Does the sample program crash, or print a message, when the file is missing?

Answers: 1. `FileNotFoundError`. 2. It prints "Error: The file does not exist. Please check the name." and continues.

## 3.9 Mini File Handling Project: Grade Tracker

### LEARN IT

This project uses 'w' to create a grades file, 'a' to add a student later, and 'r' to read every line and compute an average. It combines file handling, string methods and arithmetic.

Write names and marks ('w') → Append a new student ('a') → Read lines ('r') → Split, total, average

```
with open("grades.txt", "w") as f:
    f.write("Alice: 88\n")
    f.write("Bob: 75\n")
    f.write("Carol: 92\n")
print("Grades file created.")

with open("grades.txt", "a") as f:
    f.write("David: 81\n")
print("New student appended.")

with open("grades.txt", "r") as f:
    lines = f.readlines()
    total = 0
    for line in lines:
        name, grade = line.strip().split(":")
        print(f"{name} scored {grade}")
        total += int(grade)
    average = total / len(lines)
    print(f"\nClass average: {average:.1f}")
```

### WORKED EXAMPLE

**What happens in memory and on disk.**

1. 'w' creates grades.txt (or wipes an old one) with three lines.
2. 'a' adds David: 81\n at the end. Alice, Bob and Carol stay.
3. .readlines() returns  
['Alice: 88\n', 'Bob: 75\n', 'Carol: 92\n', 'David: 81\n'].
4. .strip() removes the trailing \n. .split(":") turns "Alice: 88" into ["Alice", " 88"].
5. int(grade) accepts the extra space: int(" 88") is 88.
6. Total = 88 + 75 + 92 + 81 = 336. Average = 336 / 4 = 84.0. The format :.1f prints one decimal place.

#### OUTPUT

```
Grades file created.
New student appended.
Alice scored  88
Bob scored   75
Carol scored  92
David scored  81

Class average: 84.0
```

The extra space before each mark in the output comes from the space after the colon in the file ("Alice: 88"). int() ignores it. To tidy the printout you could use grade.strip() as well; the average would not change.

#### REMEMBER IT

Same file, three modes, three with blocks. Never open the same file in 'w' when you only meant to add David — that would delete Alice, Bob and Carol.

#### COMMON MISTAKE

If a line has no colon, `split(":")` will not produce two parts and the unpacking `name, grade = ...` will raise `ValueError`. Keep the `Name: mark` format. An empty file would make `len(lines)` zero and then `total / 0` would fail — a real system should check that, just as `divide` checked for zero.

#### اردو میں سمجھیں

گریڈ ٹریکر پہلے 'w' سے فائل بناتا ہے، پھر 'a' سے نیا طالب علم جوڑتا ہے، پھر 'r' سے تمام سطریں پڑھ کر اوسط نکالتا ہے۔ ہر سطر کو `strip` اور `split(":")` سے نام اور نمبر میں توڑا جاتا ہے۔ اوسط = کل نمبر ÷ طلبہ کی تعداد۔ 'w' دوبارہ استعمال کرنے سے پرانے ریکارڈ مٹ جائیں گے۔

#### CHECK YOURSELF

1. What is the class average in the sample, and how is it calculated?
2. Why is David written with mode "a" rather than "w" ?

Answers: 1. 84.0 because  $(88+75+92+81)/4 = 336/4 = 84.0$ . 2. Because "a" appends; "w" would erase the three existing students.

## Career Opportunities

#### LEARN IT

The skills in this chapter — data structures, functions and files — are the same skills used in many jobs.

- **Python / software developer** — builds applications, scripts and tools in Python, one of the most widely used languages in industry.
- **Data analyst** — uses lists and dictionaries, then file handling, to process, visualise and interpret data for decisions.
- **Automation engineer** — writes scripts that repeat file and testing tasks so humans do not have to.
- **Game developer** — uses functions, data structures and logic to build interactive games.

#### WORKED EXAMPLE

The word-frequency dictionary is a tiny version of what a data analyst does with a much larger file of customer comments. The grade tracker is a tiny version of a school information system. The attendance list is a tiny version of an HR register. Same structures, larger files.

#### REMEMBER IT

Four careers: **P-D-A-G** — **P**ython developer, **D**ata analyst, **A**utomation engineer, **G**ame developer.

#### COMMON MISTAKE

You do not need all four job titles for the exam. You do need one sentence each if the paper asks "List career opportunities related to this chapter."

اس باب کی مہارتیں پائٹھن ڈویلپر، ڈیٹا اینالسٹ، آٹومیشن انجینئر اور گیم ڈویلپر کے کام میں روز استعمال ہوتی ہیں۔ فہرستیں، ڈکشنریز، فنکشنز اور فائلیں چھوٹے اسکول پروگراموں سے لے کر بڑے صنعتی نظام تک ایک جیسی بنیاد ہیں۔

#### CHECK YOURSELF

1. Which career uses lists, dictionaries and files to interpret business data?
2. Name two careers listed in this chapter.

Answers: 1. Data analyst. 2. Any two of: Python/software developer, data analyst, automation engineer, game developer.

## Chapter 3 Review

### Chapter summary

- Python programs rest on primitive types (`int`, `float`, `bool`) and abstract structures: lists, tuples, sets and dictionaries.
- A **list** is ordered and mutable, written with `[]`. Index from 0; `[-1]` is last. Slice stop is exclusive. Methods include `append`, `insert`, `remove`, `sort`, `reverse`, `clear`.
- A **tuple** is ordered and immutable, written with `()`. Item assignment raises `TypeError`. Only `count` and `index` exist as methods.
- A **set** is unordered and unique, written with `{}`. Use `|` & `-` ^ for union, intersection, difference and symmetric difference. Prefer `discard` over `remove`.
- A **dictionary** maps unique keys to values. `d[key]` can raise `KeyError`; `d.get(key, default)` is safe. Duplicate keys keep the last value.
- `min`, `max`, `len`, `sum` and `sorted` work across structures. On a dict, `min/max/sorted` use keys; total values with `sum(d.values())`. `sorted()` returns a new list; `.sort()` changes a list in place.
- Functions written with `def` give reusability, modularity, readability and maintainability. Built-in functions already exist; user-defined functions solve your tasks.
- `return` sends a value to the caller. Without `return`, the result is `None`. Local variables live inside a function; global variables live outside all functions.
- Files persist on disk. Modes: `r` read (must exist), `w` write (overwrites), `a` append. Methods: `read`, `readlines`, `write`, `close`.
- The `with` statement is a context manager that closes the file automatically. Loop line by line to read large files safely.
- `try/except FileNotFoundError` stops a missing file from crashing the program.
- The grade tracker writes with `w`, appends with `a`, reads with `r`, splits each `Name: mark` line, and computes the mean.

## Key terms

| Term            | Meaning                                                                             | اردو            |
|-----------------|-------------------------------------------------------------------------------------|-----------------|
| Data structure  | A way of organising values so a program can store and use them.                     | ڈیٹا سٹرکچر     |
| Primitive type  | A basic single value: <code>int</code> , <code>float</code> , <code>bool</code> .   | بنیادی قسم      |
| List            | Ordered, mutable sequence written with <code>[]</code> .                            | فہرست           |
| Index           | The position of an item, starting at 0 (or -1 from the end).                        | انڈیکس          |
| Slicing         | Copying a portion of a sequence with <code>start:stop:step</code> .                 | سلائسنگ         |
| Traversal       | Visiting each element in turn, usually with a loop.                                 | گشت / تراورسل   |
| Mutable         | Can be changed after creation.                                                      | قابل تبدیلی     |
| Immutable       | Cannot be changed after creation.                                                   | ناقابل تبدیلی   |
| Tuple           | Ordered, immutable sequence written with <code>()</code> .                          | ٹیپل            |
| Set             | Unordered collection of unique items written with <code>{}</code> .                 | سیٹ             |
| Union           | All unique items from two sets ( <code> </code> ).                                  | اجتماع          |
| Intersection    | Items common to both sets ( <code>&amp;</code> ).                                   | اشتراک          |
| Dictionary      | Mutable map of unique keys to values.                                               | ڈکشنری          |
| Key             | The unique lookup name in a dictionary.                                             | کلید            |
| Value           | The data stored under a key.                                                        | قدر             |
| Function        | A named, reusable block of code.                                                    | فنکشن           |
| Parameter       | An input name in a function definition.                                             | پیرامیٹر        |
| Return value    | The result sent back with <code>return</code> .                                     | واپسی کی قدر    |
| Local variable  | A name that exists only inside a function.                                          | مقامی متغیر     |
| Global variable | A name defined outside all functions.                                               | عالمی متغیر     |
| File handling   | Creating, reading, writing and closing disk files.                                  | فائل ہینڈلنگ    |
| File mode       | How a file is opened: <code>r</code> , <code>w</code> , <code>a</code> (or binary). | فائل موڈ        |
| Context manager | A <code>with</code> block that sets up a resource and cleans it up.                 | کانٹیکسٹ مینیجر |
| Exception       | A runtime error object, such as <code>FileNotFoundError</code> .                    | ایکسپشن         |
| try / except    | A pair of blocks that attempt code and handle errors.                               | ٹرائی / ایکسپٹ  |

## Practice: multiple choice

- Which structure is ordered and mutable?
  - List
  - Tuple
  - Set
  - Frozen set
- What does `nums[2:6]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?
  - `[2, 3, 4, 5, 6]`
  - `[2, 3, 4, 5, 6, 7]`
  - `[2, 3, 4, 5]`
  - `[3, 4, 5, 6]`
- `t = (1, 2, 3)` then `t[0] = 99` raises
  - `KeyError`
  - `TypeError`
  - `ValueError`
  - `IndexError`
- A set is best described as
  - ordered with duplicates
  - indexed and mutable
  - key-value pairs
  - unordered with unique elements
- If `A = {1, 2, 3, 4}` and `B = {3, 4, 5, 6}`, then `A & B` is
  - `{3, 4}`
  - `{1, 2}`
  - `{1, 2, 3, 4, 5, 6}`
  - `{1, 2, 5, 6}`
- `{"a": 1, "a": 2, "a": 3}` prints
  - `{'a': 1}`
  - `{'a': 1, 'a': 2, 'a': 3}`
  - `{'a': 3}`
  - a `KeyError`
- The safe way to read a possibly missing dictionary key is
  - `d[key]`
  - `d.get(key)`
  - `d.keys(key)`
  - `d.index(key)`
- For `s = {5, 1, 9, 3, 7}`, `sum(s)` is
  - 5
  - 9
  - 7
  - 25
- Which statement is true?
  - `sorted(x)` returns a new list and leaves `x` unchanged
  - `sorted(x)` works only on lists
  - `.sort()` works on tuples
  - `.sort()` returns a new list
- A function is created with the keyword
  - `func`
  - `def`
  - `define`
  - `return`
- A function that has no `return` statement returns
  - 0
  - False
  - None
  - an empty string
- A variable defined inside a function is
  - local
  - global
  - persistent
  - public
- Opening a file with mode `'w'`
  - fails if the file already exists
  - adds new text at the end
  - is read-only
  - erases existing content (and creates the file if needed)
- The `with open(...) as f:` statement is used mainly because
  - it makes the file binary
  - it closes the file automatically
  - it is faster than lists
  - it prevents `KeyError`
- Reading a missing file in mode `'r'` without `try` raises
  - `TypeError`
  - `KeyError`
  - `FileNotFoundError`
  - `ValueError`

## Practice: short questions

1. Define a data structure. Distinguish primitive from abstract data structures with one example of each.
2. What is negative indexing? Give the expression that returns the last item of a list named `fruits`.
3. State four differences between a list and a tuple.
4. Differentiate `.remove()` and `.discard()` on a set.
5. Why can a dictionary not have two identical keys? What happens if you write the same key twice?
6. Contrast `sorted()` with list `.sort()`.
7. List four benefits of using functions.
8. Define local scope and global scope.
9. Write the meaning of file modes `'r'`, `'w'` and `'a'`.
10. What is a context manager? Why is `with` safer than calling `.close()` yourself?

## Practice: long / problem questions

1. Compare list, tuple, set and dictionary in a table (syntax, order, mutability, duplicates, how you access an item, one typical use). Then write two lines of Python that create one example of each holding the numbers 1, 2 and 2, and state what each structure actually stores.
2. Write a complete Python program that takes a sentence, counts the frequency of each word using a dictionary and `.get()`, prints a two-column table, and prints the most frequent word. Trace your program on `"python is easy python is fun python is powerful"`.
3. Write an arithmetic calculator with four user-defined functions `add`, `subtract`, `multiply` and `divide`. `divide` must not crash when the second number is 0. Explain how `return` is used and why `input` values are converted with `float`.
4. Explain file handling. Describe modes `r`, `w` and `a`, the `with` statement, and `try/except FileNotFoundError`. Include a short program that writes one line with `with` and another that reads a file that might be missing.
5. The grade tracker writes three students with `'w'`, appends David with `'a'`, then computes the class average. Write the full program, show the exact output for marks 88, 75, 92 and 81, and explain `strip`, `split(":")` and `average:.1f`.

## Answer key

---

### Multiple choice

1. a
2. c
3. b
4. d
5. a
6. c
7. b
8. d
9. a
10. b
11. c
12. a
13. d
14. b
15. c



## Short questions — model answers

1. A data structure organises values in a program. Primitive types hold one value (e.g. `int 17`). Abstract structures hold collections and richer operations (e.g. a list of marks).
2. Negative indexing counts from the right. `fruits[-1]` is the last item. `fruits[-2]` is the second last.
3. List: `[]`, mutable, many methods, slightly slower, use when data will change. Tuple: `()`, immutable, only `count / index`, slightly faster, use for protected records. Both are ordered and allow duplicates and indexing.
4. Both delete an element. `remove` raises `KeyError` if the value is absent. `discard` does nothing in that case, so it is the safer choice.
5. Each key must identify one value (like one roll number per student). A repeated key does not crash; the last assigned value overwrites the earlier one, e.g. `{"a": 3}`.
6. `sorted(x)` is a built-in function for any iterable; it returns a new list and does not change `x`. `.sort()` is a list method that sorts in place and returns `None`.
7. Reusability (write once, call many times), modularity (small named tasks), readability (shorter files), maintainability (fix one definition).
8. Local: defined inside a function; invisible outside; gone when the function ends. Global: defined outside all functions; readable throughout the program.
9. `'r'` read, error if missing. `'w'` write, create if missing, erase old content. `'a'` append at the end, create if missing, keep old content.
10. A context manager sets up a resource and cleans it up. `with open(...)` as `f` closes the file when the block ends, even after an exception, so you cannot forget `.close()` and you are safer if the program crashes mid-write.

## Long questions — model answers

1. Use the comparison table from 3.1.4 (brackets, ordered/mutable/duplicates/access/empty/use). Examples:

```
print([1, 2, 2])
print((1, 2, 2))
print({1, 2, 2})
print({"a": 1, "b": 2, "b": 2})
```

### OUTPUT

```
[1, 2, 2]
(1, 2, 2)
{1, 2}
{'a': 1, 'b': 2}
```

The list and tuple keep both 2s and an order. The set drops the extra 2. The dictionary keeps unique keys; the second "b" overwrites the first.

2. Use the program in 3.1.7. Trace: nine tokens; `python` and `is` end at 3; others at 1. Output matches the worked example, including Most Frequent: 'python' (3 times) because `max` returns the first key among ties. Heart of the loop:  
`word_freq[word] = word_freq.get(word, 0) + 1`.
3. Use the four functions from 3.2.3. `return` sends each result to `print` in the main program. `divide` returns the string "Cannot divide by zero" when `b == 0`. `float(input(...))` is required because `input` returns a string and `"12" + "4"` would concatenate. Sample 12 and 4: 16.0, 8.0, 48.0, 3.0.
4. File handling stores data on disk so it outlives the program. Modes: `r` read / must exist; `w` wipe-or-create; `a` add-or-create. `with` closes automatically. Try/except:

```
with open("note.txt", "w") as f:
    f.write("Saved\n")
try:
    with open("note.txt", "r") as f:
        print(f.read(), end="")
except FileNotFoundError:
    print("Error: The file does not exist. Please check the name.")
```

### OUTPUT

```
Saved
```

If the name were wrong, the except message would print instead of a traceback.

5. Full program as in 3.9. Disk after append:

### OUTPUT

```
Alice: 88
Bob: 75
Carol: 92
David: 81
```

Output of the program (exact):

### OUTPUT

```
Grades file created.
New student appended.
Alice scored 88
Bob scored 75
Carol scored 92
David scored 81

Class average: 84.0
```

`strip` removes `\n`; `split(":")` separates name and mark; `int(grade)` allows the extra space; `{average:.1f}` prints one decimal.  $336 \div 4 = 84.0$ .

## Self-assessment: I can...

- ☐ Choose list, tuple, set or dictionary for a given problem and justify the choice.
- ☐ Explain tuple immutability and the inner-list exception, and use `count / index`.
- ☐ Update a dictionary, access values with `.get()`, and loop with `.items()`.
- ☐ Write a user-defined function with parameters and a `return` value, and contrast it with a built-in.
- ☐ Open a text file in `r`, `w` or `a` using `with`, and read it line by line.
- ☐ Index, slice, traverse and update a list, including `[-1]` and the main methods.
- ☐ Build a set, drop duplicates, and compute union, intersection, difference and symmetric difference.
- ☐ Apply `min`, `max`, `len`, `sum` and `sorted` correctly, including on dictionaries.
- ☐ State whether a name is local or global and predict a `NameError`.
- ☐ Catch `FileNotFoundError` and complete the grade-tracker project with the correct average.

## CHAPTER 4

# Data and Analysis

This chapter shows you how to pull data from a SQLite database with Python, clean it in Pandas, draw the right chart, and summarise a dataset with mean, median, mode and standard deviation. These are the same steps a data analyst uses every day.

اس باب میں آپ سیکھیں گے کہ پائتھن سے ایس کیو لائٹ ڈیٹا بیس کیسے جوڑا جاتا ہے، پینڈاز میں ڈیٹا کیسے صاف ہوتا ہے، درست چارٹ کیسے بنتا ہے، اور اوسط، میڈین، موڈ اور معیاری انحراف سے خلاصہ کیسے نکلتا ہے۔

## What you will learn

- Define data analysis and say why Python is used for it.
- Create a SQLite database, table, insert, select, commit and close.
- Create calculated columns and handle NaN with `isnull`, `fillna` and `dropna`.
- Choose and code bar, line, pie, histogram, scatter and box plots.
- Name the five parts of a Python database connection in order.
- Build a Pandas DataFrame from a dictionary, CSV or Excel file.
- Organise data in tables and frequency distributions.
- Compute mean, median, mode, range, variance, standard deviation, min and max by hand, and know that pandas `.std()` uses `n-1`.

## 4.1 Concept of Data Analysis

### LEARN IT

**Data analysis** is examining, organising and interpreting data so you can extract useful information. Schools study results, shops study sales, hospitals study patient records — all to make a decision, spot a trend or solve a problem.

Python is popular for this work because it is readable, talks to files and databases, does statistics, and draws graphs. Five lines of Pandas often replace an hour of spreadsheet clicking.

### WORKED EXAMPLE

A Hyderabad college has 400 CS marks in a spreadsheet. Analysis answers: What is the average? How many failed? Is the second shift weaker than the first? The numbers are data; those answers are information.

### اردو میں سمجھیں

ڈیٹا کا تجزیہ یعنی اعداد کو ترتیب دے کر ان سے فیصلے کے قابل معلومات نکالنا۔ پائنتھن اس لیے منتخب کیا جاتا ہے کہ وہ فائلیں، ڈیٹا بیس، حساب اور گراف سب سنبھال لیتا ہے۔

### CHECK YOURSELF

1. Give one school use of data analysis.
2. Name two Python superpowers for analysis.

Answers: 1. Study results to improve teaching (or similar). 2. Any two: read CSV/Excel/databases, organise tables, statistics, charts.

## 4.1.1 – 4.1.2 Data Sources and Database Connection ★ Golden

### LEARN IT

**Data sources** are where information comes from: school records, websites, sensors, databases.

**Connectivity** is the conversation between your Python program and that source so you can read and update data.

Five components of a database connection, in the order they actually work:

| # | Component         | Role                                                                   |
|---|-------------------|------------------------------------------------------------------------|
| 1 | Database          | Organised store (SQLite = one .db file; MySQL = a server for big apps) |
| 2 | Driver            | Translator between Python and that DBMS (sqlite3 is built in)          |
| 3 | Connection object | Opens, manages and closes the pipeline                                 |
| 4 | Cursor object     | Runs SQL and walks through result rows                                 |
| 5 | SQL queries       | CREATE, INSERT, SELECT, UPDATE, DELETE                                 |

Python → Driver → Connection → Cursor → SQL → Database

### EXAM TIP

Write the five names in order and one line each. A common trap is swapping cursor and connection: the connection is the pipe; the cursor is the messenger that travels through it.

### اردو میں سمجھیں

خمسہ اجزاء: ڈیٹا بیس، ڈرائیور، کنکشن، کرسر، ایس کیو ایل۔ کنکشن پائپ ہے، کرسر اس پائپ میں چل کر حکم چلاتا ہے۔

### CHECK YOURSELF

1. Which object executes `SELECT *`?
2. Is SQLite a driver, a database, or both?

Answers: 1. The cursor. 2. SQLite is the database; the `sqlite3` module is the driver.

## 4.2 Creating a SQLite Database in Python

### LEARN IT

SQLite is built into Python. The whole database is one `.db` file next to your script — no password, no port, no server. Nine steps you must be able to write from memory:

```
import sqlite3
conn = sqlite3.connect("school.db") # 1-2 driver + connection
cur = conn.cursor()                 # 3 cursor
cur.execute("CREATE TABLE IF NOT EXISTS students (id INTEGER, name TEXT, marks INTEGER)")
cur.execute("INSERT INTO students VALUES (1, 'Ali', 85)")
conn.commit()                       # 6 SAVE or the row vanishes
cur.execute("SELECT * FROM students")
rows = cur.fetchall()
for r in rows:
    print(r)
conn.close()                        # 9 unlock the file
```

### OUTPUT

```
(1, 'Ali', 85)
```

| Step     | Why it matters                                     |
|----------|----------------------------------------------------|
| connect  | Creates the file if it does not exist              |
| cursor   | Vehicle that carries SQL                           |
| execute  | Runs one SQL command                               |
| commit   | Writes INSERT/UPDATE/DELETE to disk                |
| fetchall | Brings SELECT results into a Python list of tuples |
| close    | Frees memory and unlocks the file                  |

### COMMON MISTAKE

Forgetting `commit()` is the classic bug: the program prints nothing next time because the insert was never saved. Forgetting `close()` can lock `school.db` so the next run fails.

### REMEMBER IT

Strings inside SQL use single quotes: `'Ali'`. Python strings around the SQL use double quotes (or a triple-quoted string).

اردو میں سمجھیں

ایس کیو لائٹ پائٹھن کے ساتھ آتی ہے، پورا ڈیٹا بیس ایک فائل ہے۔ ترتیب: امپورٹ، کنکٹ، کرسر، EXECUTE، COMMIT، SELECT، FETCALL، CLOSE نہ ہو تو اندراج ضائع ہو جاتا ہے۔

#### CHECK YOURSELF

1. What does `*` mean in `SELECT *`?
2. Which call must follow `INSERT`?

Answers: 1. All columns. 2. `conn.commit()`.

## 4.3 Pandas and DataFrames ★ Golden

#### LEARN IT

**Pandas** is a third-party library for tables. Install once: `pip install pandas` in the terminal (not inside the `.py` file). Convention: `import pandas as pd`.

A **DataFrame** is a 2-D table of rows and columns, like Excel or an SQL table. You can filter, sort and calculate whole columns without writing a `for` loop.

```
import pandas as pd
data = {
    "Name": ["Ali", "Sara", "Ahmed"],
    "Math": [85, 90, 78],
    "Science": [80, 88, 70],
}
df = pd.DataFrame(data)
print(df)
```

#### OUTPUT

|   | Name  | Math | Science |
|---|-------|------|---------|
| 0 | Ali   | 85   | 80      |
| 1 | Sara  | 90   | 88      |
| 2 | Ahmed | 78   | 70      |

`read_csv('students.csv')` and `read_excel('students.xlsx', sheet_name='Sheet1')` load files. `df.head()` prints the first five rows — use it on large files so you do not flood the screen.

```
df["Total"] = df["Math"] + df["Science"]
df["Average"] = df["Total"] / 2
print(df)
```

#### OUTPUT

|   | Name  | Math | Science | Total | Average |
|---|-------|------|---------|-------|---------|
| 0 | Ali   | 85   | 80      | 165   | 82.5    |
| 1 | Sara  | 90   | 88      | 178   | 89.0    |
| 2 | Ahmed | 78   | 70      | 148   | 74.0    |

#### WORKED EXAMPLE

Check: Ali  $85+80=165$ ,  $165/2=82.5$ . Sara  $90+88=178$ ,  $89.0$ . Ahmed  $78+70=148$ ,  $74.0$ .

اردو میں سمجھیں

پینڈا ڈیٹا فریم دیتا ہے۔ ڈکشنری، سی ایس وی یا ایکسل سے لوڈ کریں۔ نئے کالم بغیر لوپ کے پوری قطار پر حساب کر کے بنتے ہیں۔ `head()` صرف پہلی پانچ قطاریں دکھاتا ہے۔

#### CHECK YOURSELF

1. What is `pd`?
2. Why not `print(df)` on a million-row file?

Answers: 1. The usual alias for pandas. 2. It would print everything; use `df.head()`.

### 4.3.3 Handling Missing Values (NaN) ★ Golden

#### LEARN IT

Missing cells are **NaN** ("Not a Number"). Statistics crash or lie if you ignore them. Three tools: `isnull()` (True where empty), `fillna(value)` (replace), `dropna()` (delete the row).

```
import pandas as pd
df = pd.DataFrame({
    "Name": ["Ali", "Sara", "Ahmed", "Ayesha"],
    "Marks": [85.0, None, 78.0, 92.0],
    "Grade": ["A", "A", None, "A"],
})
print(df.isnull().sum())
```

#### OUTPUT

```
Name      0
Marks      1
Grade      1
dtype: int64
```

#### WORKED EXAMPLE

Replace Marks with the mean of the known marks, and Grade with a label. Known marks 85, 78, 92 → mean =  $255/3 = 85.0$ .

```
avg = df["Marks"].mean()          # 85.0
df["Marks"] = df["Marks"].fillna(avg)
df["Grade"] = df["Grade"].fillna("Not Assigned")
print(df)
```

#### OUTPUT

|   | Name   | Marks | Grade        |
|---|--------|-------|--------------|
| 0 | Ali    | 85.0  | A            |
| 1 | Sara   | 85.0  | A            |
| 2 | Ahmed  | 78.0  | Not Assigned |
| 3 | Ayesha | 92.0  | A            |

`df = df.dropna()` would instead delete Sara's row and Ahmed's row (any row that still had a NaN).

#### EXAM TIP

State: identify with `isnull / sum`, fill numeric gaps with the mean, fill categories with a label, or drop rows if filling would invent a lie. Never analyse first and clean second.

#### COMMON MISTAKE

Filling a salary gap with the mean of a company that includes the CEO can hide poverty (the same outlier problem as the mean). Know *why* the value is missing before you fill.

خالی خانے NaN ہیں۔ isnull تلاش، fillna بھرنا (اکثر اوسط سے)، dropna قطار ہٹانا۔ پہلے صاف کرو، پھر تجزیہ۔

#### CHECK YOURSELF

1. What does `df.isnull().sum()` return?
2. Does `dropna()` delete a column or a row by default?

Answers: 1. Count of NaNs in each column. 2. Any **row** that contains at least one NaN.

## 4.4 Data Organization and Representation

#### LEARN IT

Organised data is easier to read. Three everyday forms:

- **Table** — exact values side by side (Ali 85 Math, 80 Science).
- **Frequency distribution** — how many items fall in each range (bin).
- **Graph** — a picture of a trend or comparison (next topics).

| Range of marks | Frequency (students) |
|----------------|----------------------|
| 50–59          | 5                    |
| 60–69          | 7                    |
| 70–79          | 9                    |
| 80–89          | 4                    |
| 90–100         | 2                    |

Most students scored 70–79. That pattern is hard to see in a raw list of 27 marks, and obvious in this table.

Matplotlib (`import matplotlib.pyplot as plt`) turns tables into graphs: `import` → `figure` → `plot` → `labels` → `title` → `show()`.

ٹیبل درست عدد دکھاتی ہے، تعدد کی جدول بتاتی ہے کتنے طلبہ کس رینج میں ہیں، گراف رجحان دکھاتا ہے۔ میٹپلاٹ لب گراف بناتی ہے۔

#### CHECK YOURSELF

1. How many students scored 80 or above in the table?
2. Name the library that draws graphs in Python.

Answers: 1.  $4 + 2 = 6$ . 2. Matplotlib (pyplot).

## 4.5 Data Visualization and Types of Graphs ★ Golden

#### LEARN IT

Pick the graph that matches the question. Then label axes and give a title — an unlabelled chart scores zero in an exam.



| Chart        | Shows                                               | Watch out                                                |
|--------------|-----------------------------------------------------|----------------------------------------------------------|
| Bar / column | Compare discrete categories (subjects, departments) | Column = vertical; bar often = horizontal                |
| Line         | Change over time (temperature, visitors)            | Needs an ordered x-axis (months, years)                  |
| Pie          | Parts of one whole as percentages                   | Bad with many or almost-equal slices                     |
| Histogram    | Distribution of one numeric variable in bins        | Bars <b>touch</b> (continuous range), unlike a bar chart |
| Scatter      | Relationship of two numeric variables               | Each dot is one pair (x, y)                              |
| Box plot     | Min, Q1, median, Q3, max, and outliers              | Best tool for spotting extreme values                    |

#### 4.5.1 Bar / column

```
import matplotlib.pyplot as plt
subjects = ["English", "Math", "Science", "History"]
marks = [64, 82, 71, 49]
plt.bar(subjects, marks, color="skyblue", edgecolor="black")
plt.title("Marks vs Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks out of 100")
```

#### 4.5.2 Line

```
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
temps = [12, 15, 20, 25, 30, 35]
plt.plot(months, temps, marker="o")
plt.title("Average Monthly Temperature")
plt.grid(True)
```

#### 4.5.3 Pie

```
subjects = ["Math", "Physics", "CS", "Biology"]
students = [25, 15, 30, 10] # total 80
plt.pie(students, labels=subjects, autopct="%1.1f%%", startangle=90)
```

Check: Math 25/80 = 31.25% → 31.2%. Physics 18.8%. CS 37.5%. Biology 12.5%.

#### 4.5.4 Histogram

```
scores = [45, 50, 55, 60, 62, 65, 68, 70, 72, 75, 78, 80, 82, 85, 90, 95]
plt.hist(scores, bins=[40, 50, 60, 70, 80, 90, 100], edgecolor="black")
```

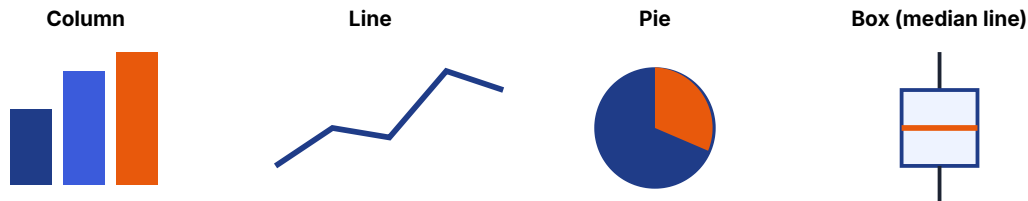
#### 4.5.5 Scatter

```
plt.scatter([10, 20, 30, 40, 50, 60, 70, 80],
            [2, 4, 5, 4, 7, 8, 9, 12], color="green", alpha=0.7)
plt.title("Rainfall vs Crop Yield") # positive correlation
```

### 4.5.6 Box plot

```
plt.boxplot([[55, 60, 65, 70, 75, 80, 85],  
            [40, 65, 70, 72, 75, 80, 95]],  
            tick_labels=["Class A", "Class B"])
```

The box is the middle 50% (Q1 to Q3). The line inside is the median. Whiskers reach min and max (or a fence, with outliers drawn as dots). Class B is more spread out than Class A.



Column compares; line shows a trend; pie shows share; the box summarises spread around the median.

#### EXAM TIP

"Which graph and why?" is a 3-mark gift. Write the name, what it shows, and one reason the others are worse.

اردو میں سمجھیں

بار/کالم اقسام کا موازنہ، لائن وقت کا رجحان، پائی حصے، ہسٹوگرام مسلسل تقسیم (بار آپس میں لگے)، اسکیترو دو متغیرات کا تعلق، باکس پلاٹ پانچ اعداد اور آؤٹ لائنر۔

#### CHECK YOURSELF

1. Why do histogram bars touch?
2. Rainfall vs yield — which chart?

Answers: 1. The x-axis is a continuous numeric range, not separate categories. 2. Scatter (two numeric variables, looking for correlation).

## 4.6 Descriptive Statistics ★ Golden

#### LEARN IT

**Descriptive statistics** squeeze a long list into a few honest numbers. **Central tendency** asks "what is typical?". **Spread** asks "how mixed is the list?".

### 4.6.1 Mean, median, mode

| Measure | How                                                                         | Use when                                                  |
|---------|-----------------------------------------------------------------------------|-----------------------------------------------------------|
| Mean    | Sum ÷ n. Uses every value.                                                  | Values sit close together. Example: ordinary test scores. |
| Median  | Middle after sorting. Odd n: centre. Even n: mean of the two centre values. | Outliers would wreck the mean (CEO salary).               |
| Mode    | Most frequent value. May be none, one, or several.                          | Most popular choice (shoe size).                          |

#### WORKED EXAMPLE

Marks 70, 80, 90. Mean =  $240/3 = 80$ .

Odd list 35, 10, 15, 70, 95. Sort: 10, 15, 35, 70, 95. Median = **35**.

Bottles sold 20, 25, 20, 30, 20, 25, 20. Mode = **20** (appears four times).

Outlier trap: nine people earn 10, one earns 1000. Mean =  $1090/10 = 109$  (looks rich). Median = 10 (tells the truth).

#### 4.6.2 Spread: range, variance, standard deviation, min, max

- **Min / max** — smallest and largest. Marks 70, 80, 90, 60, 75 → min 60, max 90.
- **Range** = max – min. Sales 120, 150, 180 → range 60.
- **Population variance**  $\sigma^2$  = (sum of squared deviations from the mean) ÷ n.
- **Sample variance**  $s^2$  uses ÷ (n-1). `pandas.Series.std()` uses the sample version.
- **Standard deviation** =  $\sqrt{\text{variance}}$ . Same units as the data (marks, not marks<sup>2</sup>). Low SD → tight, consistent. High SD → spread out, unpredictable.

#### WORKED EXAMPLE

**Full working** on 70, 80, 90. Mean  $\mu = 80$ . Deviations -10, 0, +10. Squares 100, 0, 100. Sum = 200.

Population variance  $\sigma^2 = 200/3 = 66.67$  (2 d.p.). Population SD  $\sigma = \sqrt{(200/3)} = 8.16$  (2 d.p.).

Sample variance  $s^2 = 200/2 = 100$ . Sample SD  $s = 10$ . That is what `pd.Series([70,80,90]).std()` returns.

#### EXAM TIP

Always state whether you used n or n-1. Show the sorted list for the median. For SD, show: mean → deviations → squares → divide → square root. A box plot pictures min, Q1, median, Q3, max of the same data.

#### COMMON MISTAKE

Do not report variance as if it were in "marks". Variance is marks<sup>2</sup>. Quote the standard deviation when you want a sentence in the original units.

#### اردو میں سمجھیں

اوسط سب کا مجموعہ تقسیم تعداد، میڈین ترتیب کے بعد بیچ، موڈ سب سے زیادہ بار آنے والا۔ رینج بڑا منفی چھوٹا۔ ویرینس اوسط سے فاصلوں کے مربع کی اوسط ہے، معیاری انحراف اس کا جذر ہے اور اصل اکائی میں ہوتا ہے۔ پینڈاز `std()` (نمونے والا  $n-1$ ) استعمال کرتا ہے۔

#### CHECK YOURSELF

1. Median of 4, 8, 6, 10, 2?
2. Range of 120, 150, 180?
3. Does `pandas .std()` divide by n or n-1?

Answers: 1. Sorted 2,4,6,8,10 → 6. 2. 60. 3. n-1 (sample).

## Career Opportunities

### LEARN IT

The skills in this chapter map onto real jobs: data analyst (Pandas, Excel), database administrator (SQLite/MySQL), data scientist (statistics + modelling), visualisation specialist (Matplotlib, Power BI), business-intelligence analyst (dashboards for managers), backend developer (the code that talks to the database).

اردو میں سمجھیں

یہ باب ڈیٹا اینالسٹ، ڈیٹا بیس ایڈمن، ڈیٹا سائنسدان، ویژولائزیشن اور بیک اینڈ ڈویلپر جیسے کیریئرز کی بنیاد ہے۔

## Chapter 4 Review

### Chapter summary

- Data analysis turns raw facts into decisions. Python reads files and databases, calculates and plots.
- Connection chain: database → driver → connection → cursor → SQL.
- SQLite: connect, cursor, execute, commit, fetchall, close. No commit means no save.
- A DataFrame is a 2-D table. Load with a dictionary, `read_csv` or `read_excel`. New columns are vectorised arithmetic.
- NaN: find (`isnull`), fill (`fillna`, often with the mean), or drop (`dropna`).
- Tables give exact values; frequency bins show how many in each range.
- Bar/column = categories; line = time; pie = parts of a whole; histogram = numeric distribution (touching bars); scatter = two variables; box = five-number summary + outliers.
- Mean uses every value and is sensitive to outliers; median is the middle; mode is the most common.
- Range = max–min. Variance = mean squared deviation ( $\div n$  or  $\div n-1$ ). SD =  $\sqrt{\text{variance}}$ . pandas `.std()` uses  $n-1$ .

## Key terms

| Term               | Meaning                                       | اردو                 |
|--------------------|-----------------------------------------------|----------------------|
| Data analysis      | Examining data to extract useful information. | ڈیٹا کا تجزیہ        |
| Connectivity       | Link between a program and a data source.     | رابطہ کاری           |
| Driver             | Translator between Python and a DBMS.         | ڈرائیور              |
| Connection         | Open pipeline to the database.                | کنکشن                |
| Cursor             | Object that executes SQL and fetches rows.    | کرسر                 |
| SQLite             | Built-in file-based database.                 | ایس کیو لائٹ         |
| commit             | Save INSERT/UPDATE/DELETE to disk.            | کمیٹ / محفوظ کرنا    |
| Pandas             | Library for table-like data.                  | پینڈا                |
| DataFrame          | 2-D labelled table of rows and columns.       | ڈیٹا فریم            |
| NaN                | Missing value ("Not a Number").               | خالی / غیر موجود قدر |
| Frequency          | How many items fall in a class/bin.           | تعدد                 |
| Histogram          | Distribution chart whose bars touch.          | ہسٹوگرام             |
| Scatter plot       | Dots showing two numeric variables.           | اسکیٹر پلاٹ          |
| Box plot           | Min, Q1, median, Q3, max (and outliers).      | باکس پلاٹ            |
| Mean               | Arithmetic average.                           | اوسط                 |
| Median             | Middle value of a sorted list.                | میڈین / درمیانی قدر  |
| Mode               | Most frequent value.                          | موڈ / کثرت           |
| Range              | Max – min.                                    | حد / رینج            |
| Variance           | Mean of squared deviations from the mean.     | ویریانس / تغیر       |
| Standard deviation | Square root of variance (original units).     | معیاری انحراف        |
| Outlier            | An extreme unusual value.                     | آؤٹ لائر             |

## Practice: multiple choice

1. The object that executes SQL is the:  
a. driver  
b. cursor  
c. DataFrame  
d. histogram
2. After INSERT you must:  
a. close then commit  
b. commit  
c. drop the table  
d. call head()
3. Pandas is installed with:  
a. import pandas  
b. pip install pandas  
c. sqlite3.connect  
d. plt.show()
4. `df.head()` shows:  
a. the last 5 rows  
b. the first 5 rows  
c. only NaNs  
d. the standard deviation
5. Missing Pandas values appear as:  
a. NULL in every chart  
b. NaN  
c. 0 always  
d. a pie slice
6. To delete every row that contains a gap:  
a. fillna  
b. dropna  
c. fetchall  
d. commit
7. A line graph is best for:  
a. parts of a whole  
b. a trend over months  
c. two unrelated categories  
d. a primary key
8. Histogram bars:  
a. must have gaps  
b. touch, because bins are a continuous range  
c. always show time  
d. replace the mean
9. Rainfall vs crop yield is a:  
a. pie chart  
b. scatter plot  
c. primary key  
d. mode
10. A box plot's inner line is the:  
a. mean  
b. mode  
c. median  
d. range
11. Mean of 70, 80, 90 is:  
a. 70  
b. 80  
c. 90  
d. 240
12. Median of 10, 15, 35, 70, 95 is:  
a. 10  
b. 35  
c. 70  
d. 45
13. Mode of 20, 25, 20, 30, 20, 25, 20 is:  
a. 25  
b. 30  
c. 20  
d. 7
14. Range of 120, 150, 180 is:  
a. 30  
b. 60  
c. 150  
d. 180
15. pandas `.std()` divides by:  
a. n  
b. n-1  
c. 2n  
d. zero

## Practice: short questions

1. List the five components of a Python database connection in order.
2. Why is `conn.commit()` required after INSERT?
3. What is a DataFrame? How do you load a CSV into one?
4. Differentiate `fillna` and `dropna`.
5. When is a pie chart a poor choice?
6. Differentiate a bar chart and a histogram.
7. Why is the median better than the mean for company salaries?
8. Define range, min and max with the list 70, 80, 90, 60, 75.
9. Why do we take the square root of variance?
10. State two careers that use this chapter's skills.

## Practice: long / problem questions

1. Write a complete Python program that creates `school.db`, a `students` table, inserts Ali 85 and Sara 90, commits, prints all rows, and closes. Number the five connection components beside the matching lines.
2. Starting from the dictionary of Ali/Sara/Ahmed marks, write Pandas code to build a DataFrame, add Total and Average columns, and print it. Show the exact output.
3. Given Marks [85, None, 78, 92] and Grade ["A","A", None, "A"], show `isnull().sum()`, then fill Marks with the mean and Grade with "Not Assigned". Compute the mean by hand.
4. For each goal pick a chart and justify in one sentence: (a) CS vs IT vs Eng enrolment, (b) website hits from Jan to Jun, (c) share of four clubs in one college, (d) distribution of 200 heights, (e) hours studied vs marks, (f) comparing spread of two classes.
5. For 4, 8, 6, 10, 2 compute mean, median, range, population variance and population SD. Show every step. Then say what pandas `.std()` would report instead.
6. Explain, with a numeric example, how an outlier distorts the mean but not the median. Link this to when you would draw a box plot.

## Answer key

---

### Multiple choice

1. b
2. b
3. b
4. b
5. b
6. b
7. b
8. b
9. b
10. c
11. b
12. b
13. c
14. b
15. b

## Short questions — model answers

1. Database, driver, connection object, cursor object, SQL queries.
2. Without commit the insert stays in memory and disappears when the program ends, so the next run cannot see the row.
3. A 2-D table of labelled rows and columns. `df = pd.read_csv("file.csv")`.
4. `fillna(v)` replaces NaN with v (the row stays). `dropna()` deletes any row that still contains a NaN.
5. When there are many slices, or the values are almost equal, so the eye cannot compare them.
6. Bar: separate categories, gaps between bars. Histogram: one numeric variable grouped into bins; bars touch.
7. A huge CEO salary pulls the mean up; the median stays with the typical worker.
8. Min 60, max 90, range 30.
9. Variance is in squared units. The SD brings the spread back to the original units (marks, rupees, °C).
10. Any two: data analyst, DBA, data scientist, visualisation specialist, BI analyst, backend developer.

## Long questions — model answers

1. Use the nine-step listing in 4.2 with two INSERTs. Label: `sqlite3` = driver, `connect` = connection, `cursor` = cursor, `execute strings` = SQL, `school.db` = database.
2. Output is the three-row table with Total 165, 178, 148 and Average 82.5, 89.0, 74.0 (see the worked example).
3. `isnull sum`: Marks 1, Grade 1. Mean of 85, 78, 92 =  $255/3 = 85.0$ . After fill: Sara Marks 85.0, Ahmed Grade "Not Assigned".
4. (a) column/bar — compare categories. (b) line — time. (c) pie — parts of a whole. (d) histogram — numeric distribution. (e) scatter — two numeric variables. (f) box — spread and outliers.
5. Sorted 2,4,6,8,10. Sum 30, mean 6, median 6, range 8. Deviations -4,-2,0,2,4. Squares 16,4,0,4,16 sum 40.  $\sigma^2 = 40/5 = 8$ .  $\sigma = \sqrt{8} = 2.828$  (3 d.p.). pandas `.std()` uses  $40/4 = 10$ ,  $\sqrt{10} \approx 3.162$ .
6. Nine salaries of 10 and one of 1000: mean 109, median 10. Draw a box plot — the 1000 appears as an outlier dot above the whisker, so the distortion is visible.

## Self-assessment: I can...

- |                                                                                          |                                                                                            |
|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <input type="checkbox"/> name the five connection components in order.                   | <input type="checkbox"/> write SQLite connect / create / insert / commit / select / close. |
| <input type="checkbox"/> build a DataFrame and add a calculated column.                  | <input type="checkbox"/> find, fill and drop missing values.                               |
| <input type="checkbox"/> choose bar, line, pie, histogram, scatter or box for a problem. | <input type="checkbox"/> compute mean, median, mode, range, variance and SD by hand.       |
| <input type="checkbox"/> state that pandas std uses $n-1$ .                              |                                                                                            |

## CHAPTER 5

# Application and Impacts of Computing

This chapter shows you how computers learn from data with machine learning, neural networks and deep learning, how to collaborate online without leaking secrets, how to protect data and stop cyber threats, and how to keep digital teamwork fair for every student in Sindh.

اس باب میں آپ سیکھیں گے کہ مشین لرننگ، نیورل نیٹ ورکس اور ڈیپ لرننگ کمپیوٹر کو ڈیٹا سے کیسے سکھاتے ہیں، آن لائن کام کو محفوظ کیسے رکھا جائے، ڈیٹا کی حفاظت اور سائبر خطرات کا مقابلہ کیسے کیا جائے، اور ڈیجیٹل تعاون کو سب کے لیے منصفانہ کیسے بنایا جائے۔



## What you will learn

- Define **machine learning (ML)** and describe the three steps: data input, learning, prediction.
- Define **deep learning (DL)** and draw the relationship **AI  $\supset$  ML  $\supset$  DL**.
- List the six components of deep learning and give applications of NN and DL in daily life and in Pakistan.
- Protect data using encryption (including a Caesar cipher), strong passwords, the 3-2-1 backup rule and a firewall.
- Distinguish **equal access** from **equity** and describe the functions of equity in digital collaboration.
- Explain a **neural network (NN)** as a brain-inspired model with an input layer, hidden layers and an output layer.
- Name the eight components of a neural network and compute a neuron's output from given numbers.
- Explain **secure collaboration**: authentication, multi-factor authentication (MFA) and access control.
- Identify common threats (malware, phishing, DoS) and apply mitigation techniques.
- Name common collaboration tools and use them fairly and safely.

## 5.1 Machine Learning, Neural Networks and Deep Learning

### LEARN IT

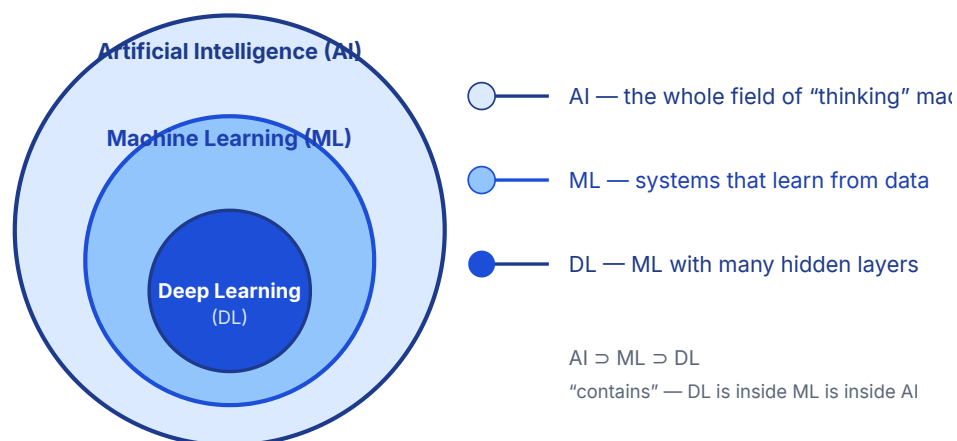
**Machine learning (ML)**, **neural networks (NN)** and **deep learning (DL)** are important areas of **artificial intelligence (AI)**. **AI** is the broad field in which computers perform tasks that normally need human thinking, such as recognising a face, understanding speech or making a decision.

These technologies help computers **learn from data** and make useful decisions **without being explicitly programmed for every single step**. You do not write a separate rule for every possible face or every Urdu accent. You show the computer many examples, and it finds the patterns itself.

They are used in many modern systems:

- **Face unlock** on smartphones.
- **Speech recognition** (for example Siri, Alexa, or Urdu voice typing on a phone).
- **Photo search** categorisation (typing "car" in a gallery and seeing car photos).
- **Medical diagnosis** from X-rays and scans.
- **Language translation** (English to Urdu or Sindhi).

Remember the nesting: **AI is the biggest set**. Inside AI sits **ML**. Inside ML sits **DL**. Every deep-learning system is a kind of machine learning, and every machine-learning system is a kind of AI — but not every AI system is deep learning.



The set relationship **AI  $\supset$  ML  $\supset$  DL**. Deep learning is a method of machine learning; machine learning is a part of artificial intelligence.

#### WORKED EXAMPLE

**Worked example: place each system in the correct set.** Sara in Karachi uses three features on her phone.

1. **A chess app that follows fixed if-then rules** written by a programmer ("if the queen is attacked, move it"). This is **AI** (the machine decides), but it is **not ML**, because it does not learn from data.
2. **A JazzCash fraud filter** that was trained on thousands of past transactions and then flags a new unusual payment. This is **ML** (and therefore also AI). If it uses only a simple model with few layers, it may **not** be DL.
3. **Face unlock** that studied millions of face images through many hidden layers. This is **DL** — and therefore also ML and AI.

Exam sentence you can write: *All DL is ML, all ML is AI, but not all AI is ML, and not all ML is DL.*

#### REMEMBER IT

**Russian-doll memory trick.** Picture three boxes. The outer box is **AI**. Inside it is a smaller box **ML**. Inside that is the smallest box **DL**. You cannot take DL out of ML, and you cannot take ML out of AI.

#### COMMON MISTAKE

**Mistake:** "AI, ML and DL are three different, separate subjects."

**Fix:** They nest.  $DL \subset ML \subset AI$ . A deep-learning face-unlock system is all three at once.

#### اردو میں سمجھیں

مصنوعی ذہانت (AI) سب سے بڑا میدان ہے۔ اس کے اندر مشین لرننگ (ML) ہے، اور مشین لرننگ کے اندر ڈیپ لرننگ (DL) ہے۔ یہ ٹیکنالوجیز کمپیوٹر کو ڈیٹا سے سکھاتی ہیں تاکہ ہر قدم الگ الگ پروگرام کیے بغیر فیصلے کیے جا سکیں، جیسے چہرے سے فون کھولنا یا آواز پہچاننا۔

#### CHECK YOURSELF

1. Which is the largest set: AI, ML or DL?
2. Is every neural-network face-unlock system an example of AI? Why?
3. Give one modern system from this section that uses these technologies.

Answers: 1. AI ( $AI \supset ML \supset DL$ ). 2. Yes — DL is inside ML, and ML is inside AI. 3. Any of: face unlock, speech recognition, photo search, medical diagnosis, language translation.

## 5.1.1 Machine Learning (ML)

#### LEARN IT

A **machine learning model** is the system that **learns from data and gives an output**. Think of it as the **trained brain** of a computer program.

In ordinary programming you write every rule yourself: *if marks  $\geq 40$  then "Pass"*. In machine learning you **do not list every rule**. You give the model many examples (data), it finds the patterns, and later it decides about new cases it has never seen.

**How it works — three steps:**

1. Data input → 2. Learning (find patterns) → 3. Prediction on new data

1. **Data input.** The model is given massive amounts of data — for example thousands of past BISE marksheets, or millions of crop photos from Sindh fields.
2. **Learning.** It studies the data and **learns patterns and rules automatically**. It might notice that students who scored low in Chapter 3 also failed the board exam, or that yellow spots on cotton leaves often mean a pest.
3. **Prediction.** After training, it can make **accurate predictions or decisions** when given **new, unseen data** — a new student's record, or a new leaf photograph.

**Note from the source:** there are many types of machine-learning models. A **neural network** is one important type. **Deep learning** is a highly advanced method of machine learning.

#### WORKED EXAMPLE

**Worked example: a wheat-yield model for a farm near Sukkur.**

**Step 1 — Data input.** For 8 years the farm recorded: rainfall (mm), temperature, fertiliser bags used, and the actual yield (maunds per acre).

**Step 2 — Learning.** The model studies those rows. It finds, for example, that yield rises with fertiliser up to a point, and falls when temperature is very high in April.

**Step 3 — Prediction.** This year the farmer types in this season's rainfall and temperature. The model has never seen this exact year before, but it predicts "about 32 maunds per acre." The farmer plans storage and a buyer in advance.

Nobody wrote a giant list of "if rain = 120 and temp = 38 then yield = ...". The model learned the pattern from data.

#### REMEMBER IT

**D-L-P.** Machine learning is **Data in, Learn patterns, Predict**. "Data Learns to Predict."

#### COMMON MISTAKE

**Mistake:** "The computer is intelligent on day one."

**Fix:** An ML model is useful only **after training** on enough good data. Garbage data → garbage predictions. Also, neural networks are *one* type of ML, not the only type; deep learning is a *method* that uses deep neural networks.

#### اردو میں سمجھیں

مشین لرننگ ماڈل وہ نظام ہے جو ڈیٹا سے سیکھ کر نتیجہ دیتا ہے — کمپیوٹر پروگرام کا تربیت یافتہ دماغ۔ پہلے بہت سا ڈیٹا دیا جاتا ہے، پھر ماڈل خود اصول اور پیٹرن سیکھتا ہے، اور تربیت کے بعد نئے ان دیکھے ڈیٹا پر پیش گوئی کرتا ہے۔ نیورل نیٹ ورک مشین لرننگ کی ایک اہم قسم ہے، اور ڈیپ لرننگ اس کا ایک اعلیٰ طریقہ ہے۔

#### CHECK YOURSELF

1. What are the three steps of how an ML model works?
2. What is a machine-learning model compared to in the source?
3. After training, what kind of data can the model handle?

Answers: 1. Data input, learning, prediction. 2. The trained brain of a computer program. 3. New, unseen data.

## 5.1.2 Neural Networks (NN) ★ Golden

### LEARN IT

A **neural network** is a computer model that **learns patterns from data**. It is **directly inspired by the human brain**, but it works as a **mathematical computer system** — not as living tissue.

Your brain has billions of nerve cells (**neurons**) that pass signals to each other. A neural network copies this idea with small processing units also called **neurons** or **nodes**. These nodes are **connected** and **pass information from one layer to another**, performing calculations at each step.

A typical neural network has three kinds of layer:

- **Input layer** — receives the raw data (pixels of a face, samples of a sound, marks of a student).
- **Hidden layer(s)** — the “thinking” in the middle. Nodes here combine the inputs and learn features (eye distance, nose shape, an edge in a photo).
- **Output layer** — gives the final answer (unlock / keep locked; “this is a cat”; “pass / fail”).

Input layer → Hidden layer(s) → Output layer

### WORKED EXAMPLE

**Practical example from the source: face unlock.**

**Step 1 — Input layer.** You look at your phone. The camera captures your face as numbers (brightness of each pixel). Those numbers enter the input layer.

**Step 2 — Hidden layers.** The system studies face features such as **eye distance** and **nose shape**. Early hidden nodes may detect edges and shadows; later hidden nodes combine those into “this looks like Sara’s face.” It compares them with stored data from when you first set up the phone.

**Step 3 — Output layer.** If the features match, the phone **unlocks**. If they do not (your brother holds the phone), it stays locked.

The same three-layer story works for an NADRA-style biometric gate at a college: camera (input) → feature matching (hidden) → open the turnstile or not (output).

### REMEMBER IT

**I-H-O.** Input receives, Hidden thinks, Output decides. “In Hidden Out.” A neural network is a **brain-inspired maths machine**, not a copy of a living brain.

### COMMON MISTAKE

**Mistake:** “A neural network is an actual human brain inside the computer.”

**Fix:** It is only **inspired by** the brain. It is mathematics: numbers, multiplications and additions running on ordinary hardware.

### اردو میں سمجھیں

نیورل نیٹ ورک ایک کمپیوٹر ماڈل ہے جو ڈیٹا سے پیٹرن سیکھتا ہے۔ یہ انسانی دماغ سے متاثر ہے مگر ایک ریاضیاتی نظام کے طور پر کام کرتا ہے۔ چھوٹی اکائیاں نیورون یا نوڈ کہلاتی ہیں، جو تھوں میں جڑی ہوتی ہیں: ان پٹ تہ ڈیٹا لیتی ہے، پوشیدہ تہ سوچتی ہے، اور آؤٹ پٹ تہ فیصلہ دیتی ہے — جیسے چہرے سے فون کھولنا۔

#### EXAM TIP

Very common questions: "What is a neural network?" (2–3 marks), "Name its three layers and explain each with the face-unlock example" (5 marks), and "How is a neural network related to the human brain?"

Full-mark recipe: (1) definition — computer model that learns patterns from data, inspired by the brain, mathematical; (2) neurons/nodes connected across layers; (3) Input / Hidden / Output, each in one sentence; (4) walk the camera → features (eye distance, nose shape) → unlock path. Draw the three boxes if the paper asks for a diagram.

#### CHECK YOURSELF

1. What are the small processing units of a neural network called?
2. In face unlock, which layer studies eye distance and nose shape?
3. Is a neural network a living brain? Explain in one sentence.

Answers: 1. Neurons or nodes. 2. Hidden layers. 3. No — it is inspired by the brain but works as a mathematical computer system.

### 5.1.3 Deep Learning (DL)

#### LEARN IT

**Deep learning** is a machine-learning method in which computers use **multi-layer neural networks** to learn **complex patterns** from data.

The word "**deep**" is used because data passes through **several hidden layers** before the system produces the final output. Each layer learns a **different level of detail**:

- **Early layers** learn simple features — edges, shapes, colours.
- **Deeper layers** combine those simple features to understand complex patterns — a whole face, or a specific car model (a Cultus versus a Corolla).

A network with only one or two hidden layers is a **shallow** neural network. Deep learning is always "deep": it needs **huge amounts of data** and **processing power**.

#### Neural network vs deep learning

|            | Neural network (the model)                                              | Deep learning (the method)                                                                    |
|------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| What it is | A system/model that has layers, nodes and connections to help it learn. | A specific method that trains neural networks containing <b>many hidden layers</b> .          |
| Depth      | Can be simple ( <b>shallow</b> ) with just one or two hidden layers.    | Always "deep," requiring huge amounts of data and processing power to learn complex patterns. |
| Role       | The architecture (the "machine").                                       | The way we train a deep version of that machine.                                              |

#### WORKED EXAMPLE

**Real-world example from the source: photo gallery search.**

A modern gallery app on a smartphone uses deep learning to recognise faces, objects or places in thousands of photos.

**Step 1.** During training (done by the company, using millions of images) early layers learned edges and colours; deeper layers learned "wheel", "petal", "minaret".

**Step 2.** The trained network has already scanned your phone's photos and tagged them.

**Step 3.** You type "**car**" or "**flower**" in the search bar. The app instantly shows related photos because its deep neural network has already learned visual patterns from millions of training images.

A student in Hyderabad can type "Ajrak" or "cricket" and jump to those pictures without scrolling for ten minutes.

#### REMEMBER IT

**Deep = many hidden layers.** Shallow NN: 1–2 hidden layers. Deep NN: many hidden layers, huge data, strong computers. Simple features first, complex meaning later.

#### COMMON MISTAKE

**Mistake:** "Deep learning and neural networks are two names for the same thing."

**Fix:** A neural network is the **model** (layers + nodes + connections). Deep learning is the **method** of training networks that have **many** hidden layers. A small NN with one hidden layer is not deep learning.

#### اردو میں سمجھیں

ڈیپ لرننگ مشین لرننگ کا وہ طریقہ ہے جس میں کئی پوشیدہ تہوں والے نیورل نیٹ ورک استعمال ہوتے ہیں۔ "ڈیپ" اس لیے کہ ڈیٹا کئی تہوں سے گزر کر پیچیدہ پیٹرن سیکھتا ہے: پہلی تہیں کنارے اور رنگ سیکھتی ہیں، گہری تہیں پورا چہرہ یا گاڑی کی قسم پہچانتی ہیں۔ یہ طریقہ بہت سے ڈیٹا اور طاقتور کمپیوٹنگ کا طالب ہے۔

#### CHECK YOURSELF

1. Why is the method called "deep" learning?
2. What do early layers learn, and what do deeper layers learn?
3. Can a neural network with one hidden layer be called deep learning?

Answers: 1. Because data passes through several hidden layers. 2. Early: simple features (edges, shapes, colours). Deeper: complex patterns (a whole face, a car model). 3. No — that is a shallow network; DL always uses many hidden layers.

## 5.1.4 Components of Neural Networks ★ Golden

#### LEARN IT

The components of a neural network can be understood through its **layered architecture**. Data **enters**, gets **processed**, and **exits**, with mathematical adjustments happening inside.

| # | Component              | Description                                                                            |
|---|------------------------|----------------------------------------------------------------------------------------|
| 1 | <b>Input layer</b>     | Receives the initial raw data (for example image pixels, sound waves, text).           |
| 2 | <b>Neurons / nodes</b> | Small processing units that receive, calculate, and pass information forward.          |
| 3 | <b>Hidden layer(s)</b> | The layers between input and output. They process the data and learn patterns from it. |

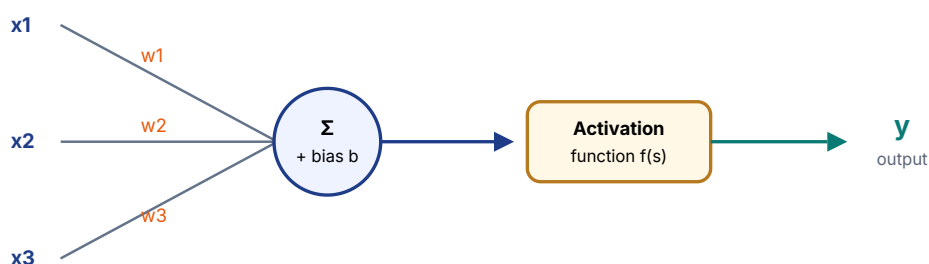
| # | Component                  | Description                                                                                                             |
|---|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 4 | <b>Weights</b>             | Numbers that show the <b>importance or strength</b> of connections between neurons.                                     |
| 5 | <b>Bias</b>                | An extra <b>constant</b> value added to help the model adjust its result and improve accuracy.                          |
| 6 | <b>Activation function</b> | A mathematical <b>gate</b> that decides whether the information is important enough to move to the next layer.          |
| 7 | <b>Output layer</b>        | Gives the final computed result, decision or prediction (for example "This is a cat").                                  |
| 8 | <b>Loss function</b>       | Checks the <b>error</b> (how much the model's answer is wrong compared with reality) so the network can correct itself. |

## What happens inside one neuron

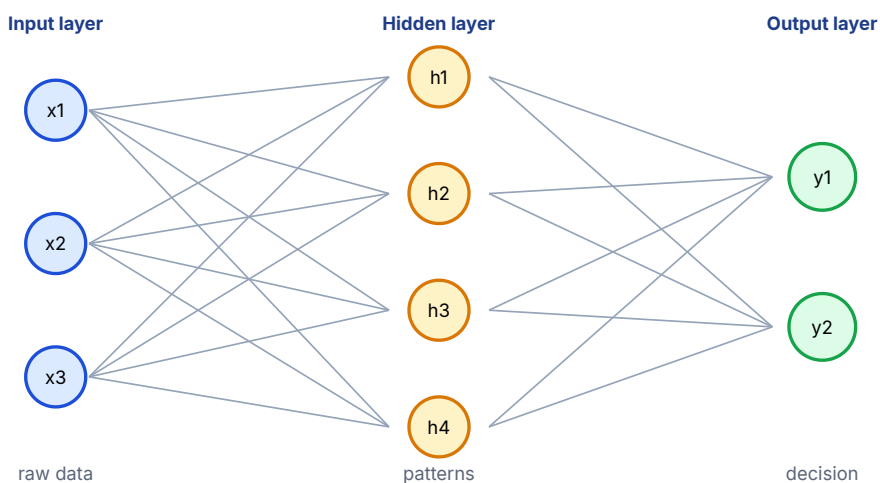
Each neuron does a short calculation:

1. Multiply each input  $x$  by its weight  $w$ .
2. Add those products together, then add the **bias**  $b$ . This total is the **weighted sum**  $s$ .
3. Pass  $s$  through an **activation function**. A simple exam-friendly gate is a **threshold (step) function**: if  $s$  is at least the threshold, the neuron **fires** (output = 1); otherwise output = 0.

In symbols:  $s = (x_1 \times w_1) + (x_2 \times w_2) + \dots + b$ , then  $y = 1$  if  $s \geq \text{threshold}$ , else  $y = 0$ .



One neuron: inputs  $\times$  weights are summed, bias is added, then an activation function decides the output  $y$ .



A small neural network: 3 input nodes, 4 hidden nodes, 2 output nodes. Each line is a weighted connection.

### WORKED EXAMPLE

**Worked example: neuron calculation with numbers.** A college in Larkana uses a tiny network to guess whether a student is ready for a test. One hidden neuron receives two inputs:

| Symbol    | Meaning                                 | Value |
|-----------|-----------------------------------------|-------|
| x1        | Hours studied yesterday (scaled 0 to 1) | 0.8   |
| x2        | Attendance this month (scaled 0 to 1)   | 0.5   |
| w1        | Weight of hours studied                 | 0.4   |
| w2        | Weight of attendance                    | 0.6   |
| b         | Bias                                    | 0.1   |
| threshold | Activation gate                         | 0.5   |

#### Step 1 — weighted products.

$$x1 \times w1 = 0.8 \times 0.4 = \mathbf{0.32}$$

$$x2 \times w2 = 0.5 \times 0.6 = \mathbf{0.30}$$

#### Step 2 — add bias to get the weighted sum s.

$$s = 0.32 + 0.30 + 0.1 = \mathbf{0.72}$$

**Step 3 — activation (threshold 0.5).** Is  $0.72 \geq 0.5$ ? **Yes**, so the neuron fires: **y = 1** (ready for the test).

**Step 4 — a second student** with  $x1 = 0.2$  (almost no study) and  $x2 = 0.3$ :

$$s = (0.2 \times 0.4) + (0.3 \times 0.6) + 0.1 = 0.08 + 0.18 + 0.1 = \mathbf{0.36}$$

0.36 is **less than** 0.5, so **y = 0** (not ready). The same neuron, different inputs, different decision.

You can check the first student in Python 3:

```
x1, x2 = 0.8, 0.5
w1, w2 = 0.4, 0.6
b = 0.1
s = x1 * w1 + x2 * w2 + b
y = 1 if s >= 0.5 else 0
print(round(s, 2))
print(y)
```

#### OUTPUT

```
0.72
1
```

**Loss function, briefly.** If the true answer for the first student was actually “not ready” (true  $y = 0$ ) but the model said 1, the loss is high. Training then slightly changes weights and bias so the next guess is closer to reality.

### REMEMBER IT

**Weights = importance. Bias = extra nudge. Activation = yes/no gate. Loss = how wrong.** A large weight means that input matters more. Bias shifts the sum up or down so the gate can open at the right place.

### COMMON MISTAKE

**Mistake:** Forgetting to add the bias, or multiplying instead of adding after the products.

**Fix:** Always **multiply first** (each  $x$  by its  $w$ ), **then add** the products, **then add  $b$** , **then** apply the activation. Write the four lines in the exam as we did above.



نیورل نیٹ ورک کے اجزاء ہیں: ان پٹ تہ، نیورون، پوشیدہ تہ، وزن (Weights)، بائیس (Bias)، ایکٹیویشن فنکشن، آؤٹ پٹ تہ اور لاس فنکشن۔ ہر نیورون ان پٹ کو وزن سے ضرب دے کر جمع کرتا ہے، بائیس جوڑتا ہے، پھر ایک گیٹ فیصلہ کرتا ہے کہ سگنل آگے جائے یا نہیں۔ وزن تعلق کی طاقت بتاتا ہے؛ لاس فنکشن غلطی ناپتا ہے تاکہ ماڈل خود درست ہو سکے۔

**EXAM TIP**

Golden questions: "List the components of a neural network" (often 8 names + one line each), "What are weights and bias?", and a **numerical neuron**: given  $x$ ,  $w$ ,  $b$  and a threshold, find  $s$  and  $y$ .

Show every arithmetic step. Underline the final  $y$ . If asked for a diagram, draw Input  $\rightarrow$  Hidden  $\rightarrow$  Output with circles and connecting lines, and label weights on the lines. Mention the loss function as the component that measures error so the network can correct itself.

**CHECK YOURSELF**

1. What do weights show?
2. Using  $x_1 = 1$ ,  $x_2 = 0$ ,  $w_1 = 0.5$ ,  $w_2 = 0.5$ ,  $b = 0$ , threshold = 0.4, what is  $y$ ?
3. What does the loss function check?

Answers: 1. The importance or strength of connections between neurons. 2.  $s = 1 \times 0.5 + 0 \times 0.5 + 0 = 0.5$ ;  $0.5 \geq 0.4$  so  $y = 1$ . 3. The error — how much the model's answer is wrong compared with reality.

## 5.1.5 Components of Deep Learning

**LEARN IT**

Deep learning relies on specific advanced components to achieve its high accuracy. These components **differentiate it from basic machine learning**.

| # | Component                         | What it means                                                                                                                               |
|---|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Large input data</b>           | Deep learning is " <b>data hungry</b> ." It needs massive datasets (thousands or millions of records) for better learning and accuracy.     |
| 2 | <b>Many hidden layers</b>         | Uses dozens or hundreds of hidden layers to learn deeper, more intricate patterns.                                                          |
| 3 | <b>Automatic feature learning</b> | Unlike older models where humans had to manually highlight important features, deep learning <b>extracts important features by itself</b> . |
| 4 | <b>Learning algorithm</b>         | Math routines (like <b>backpropagation</b> ) that help the system reduce mistakes and improve learning continuously.                        |
| 5 | <b>High computing power</b>       | Processing millions of data points across deep layers needs specialised, strong hardware, such as <b>GPUs</b> (Graphics Processing Units).  |
| 6 | <b>Training process</b>           | The model is trained again and again in loops called <b>epochs</b> to gradually improve its final performance.                              |

A **GPU** was originally made for video games' graphics, but it can do thousands of small multiplications in parallel — exactly what deep networks need. An **epoch** is one full pass through the training data. Ten epochs means the model has seen every training example ten times, adjusting weights a little each time.

**Backpropagation** means sending the error (from the loss function) **backwards** through the layers so each weight can be nudged in the direction that reduces the next error.

#### WORKED EXAMPLE

**Worked example: cotton-leaf disease detector for a farm in Mirpurkhas.** Match each DL component.

1. **Large input data.** Researchers collect 80,000 labelled photos: healthy leaves and leaves with whitefly damage.
2. **Many hidden layers.** The network has 20 hidden layers so it can go from "green edge" to "this exact pest pattern."
3. **Automatic feature learning.** No agronomist types in "look for yellow circles." The network finds the useful spots itself.
4. **Learning algorithm.** After a wrong guess ("healthy" when it was diseased), backpropagation lowers the weights that caused the mistake.
5. **High computing power.** Training runs on a GPU workstation at a university lab in Karachi; a school laptop would take weeks.
6. **Training process.** They train for 30 epochs. After epoch 1 accuracy is 62%; after epoch 30 it is 94% on photos it has not trained on.

#### REMEMBER IT

**D-L-A-L-G-T.** Data (lots), Layers (many), Automatic features, Learning algorithm, GPU power, Training epochs. "Deep Learners Always Love Good Training."

#### COMMON MISTAKE

**Mistake:** "Deep learning is just 'a neural network'."

**Fix:** Name the six extras: huge data, many hidden layers, automatic features, a learning algorithm (backpropagation), GPUs, and repeated epochs. A shallow NN with a small spreadsheet is ordinary ML, not DL.

#### اردو میں سمجھیں

ڈیپ لرننگ کو عام مشین لرننگ سے یہ چیزیں الگ کرتی ہیں: بہت بڑا ڈیٹا، بہت سی پوشیدہ تہیں، خود بخود خصوصیات نکالنا، سیکھنے کا الگورتھم جیسے بیک پروپیگیشن (Backpropagation)، طاقتور ہارڈویئر جیسا کہ جی پی یو (GPU)، اور بار بار تربیت جسے ایپوک (Epoch) کہتے ہیں۔

#### CHECK YOURSELF

1. Why is deep learning called "data hungry"?
2. What is an epoch?
3. Name the hardware often used for DL, and the learning algorithm named in the source.

Answers: 1. It needs massive datasets (thousands or millions of records) for accuracy. 2. One training loop — one full pass through the training data. 3. GPUs (Graphics Processing Units); backpropagation.

## Future of AI, NN and DL in Pakistan

### LEARN IT

Machine learning, neural networks and deep learning will play a key role in Pakistan's future. The **National AI Policy 2025** highlights how artificial intelligence is now part of national planning, focusing on **innovation, skill-building, ethical use and digital growth**.

| Sector               | Potential impact in Pakistan                                                                                                |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Education            | Smart learning systems, online tutoring, automatic assessments, and finding where students need specific help.              |
| Healthcare           | Disease detection, medical image analysis, and managing hospital data. Crucial for rural areas lacking specialist doctors.  |
| Agriculture          | Monitoring crops, predicting weather, managing irrigation, and estimating yields — highly important for Pakistan's economy. |
| Business and banking | Fraud detection, customer support (chatbots), market forecasting, and intelligent risk analysis.                            |
| Local languages      | Speech recognition, translation and text-to-speech tools for <b>Urdu, Sindhi</b> and other regional languages.              |

To benefit from this future, Pakistan needs skilled individuals. You should focus on **programming (such as Python), mathematics, data analysis, problem-solving** and the **ethical use of technology**.

### WORKED EXAMPLE

**Worked example: pick a sector and write a three-line "impact paragraph" for 5 marks.**

**Agriculture:** A cotton grower in Sanghar photographs leaves on a cheap phone. A deep-learning model trained on Pakistani crop images flags whitefly early. The farmer sprays only the affected block, saves pesticide cost, and protects yield — which matters because agriculture is a large part of Pakistan's economy.

### REMEMBER IT

**Five sectors: E-H-A-B-L.** Education, Healthcare, Agriculture, Business/banking, Local languages. Policy words: innovation, skills, ethics, digital growth.

### COMMON MISTAKE

**Mistake:** Writing only "AI will make Pakistan modern" with no sector or example.

**Fix:** Name the National AI Policy 2025, pick two sectors from the table, and give a local example (rural hospital without a specialist; Urdu/Sindhi speech tools; bank fraud).

### اردو میں سمجھیں

قومی اے آئی پالیسی 2025 کے مطابق مصنوعی ذہانت پاکستان کی منصوبہ بندی کا حصہ ہے: جدت، مہارت، اخلاقی استعمال اور ڈیجیٹل ترقی۔ تعلیم، صحت، زراعت، بینکنگ اور اردو و سندھی جیسی مقامی زبانوں میں اس کے اثرات ہوں گے۔ طلبہ کو پائنتھن، ریاضی، ڈیٹا کا تجزیہ، مسئلہ حل کرنا اور ٹیکنالوجی کا اخلاقی استعمال سیکھنا چاہیے۔

### CHECK YOURSELF

1. Name the national policy mentioned for AI in Pakistan.
2. Why is AI in healthcare especially important for rural areas?
3. List three skills students should learn to prepare for this future.

Answers: 1. National AI Policy 2025. 2. Those areas often lack specialist doctors; AI can help with disease detection and medical image analysis. 3. Any three of: programming (Python), mathematics, data analysis, problem-solving, ethical use of technology.

## 5.2 Applications of Neural Networks and Deep Learning

### LEARN IT

Neural networks and deep learning are widely used in modern life. Many intelligent systems use them in the background to **recognise information, make predictions and support users**.

| # | Application                              | What it does                                                                                                                                  | Example                                                                                                                                                         |
|---|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Image recognition</b>                 | Identifies objects, faces or patterns in images. Used in security cameras, photo search and biometric locks.                                  | Unlocking a smartphone using face recognition; a camera at a Karachi plaza matching a face with a staff list.                                                   |
| 2 | <b>Speech recognition</b>                | Converts spoken words into digital text or commands. Deep learning helps with accents, pronunciation and voice patterns.                      | A student speaks to Google Assistant or Alexa to set a morning alarm; Urdu voice search on a phone.                                                             |
| 3 | <b>Natural Language Processing (NLP)</b> | Helps computers understand, process and respond to human language grammatically and in context. Core of chatbots, translators and AI writers. | Using Google Translate to turn a difficult English paragraph into Urdu; a bank chatbot answering in Urdu.                                                       |
| 4 | <b>Healthcare systems</b>                | Analyse complex medical images (X-rays, CT scans, MRIs) to detect diseases early and accurately.                                              | A hospital system highlighting abnormal patterns in a lung X-ray that might indicate pneumonia or tumours — useful at a DHQ hospital that has few radiologists. |
| 5 | <b>Recommendation systems</b>            | Analyse past behaviour and preferences to suggest content, products or services.                                                              | A student watches computer-science videos on YouTube; the platform recommends more programming tutorials.                                                       |

| # | Application                   | What it does                                                                                                              | Example                                                                                               |
|---|-------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| 6 | <b>Autonomous vehicles</b>    | Self-driving cars use neural networks to process real-time camera and radar data: roads, signals, pedestrians, obstacles. | A self-driving car detects a pedestrian crossing and automatically applies the brakes.                |
| 7 | <b>Fraud detection</b>        | Banks compare current activity with previous spending patterns to catch unusual transactions.                             | An HBL or Meezan app flags a sudden Rs 80,000 payment from a new city at 3 a.m.                       |
| 8 | <b>Assistive technologies</b> | AI supports people with disabilities.                                                                                     | Speech-to-text tools for users who have difficulty typing; screen readers that speak Urdu text aloud. |

#### WORKED EXAMPLE

**Worked example: match the app to the application name — a day in Ahmed's life in Hyderabad.**

- 7:00 am — phone unlocks by face. → **Image recognition**.
- 7:05 am — "Ok Google, set alarm for Maghrib." → **Speech recognition**.
- 8:00 am — Google Translate helps with an English article. → **NLP**.
- 4:00 pm — YouTube suggests a Python tutorial after he watched Chapter 5 notes. → **Recommendation system**.
- 9:00 pm — JazzCash texts "We blocked a payment that does not match your pattern." → **Fraud detection**.

#### REMEMBER IT

**Eight applications: I-S-N-H-R-A-F-A.** Image, Speech, NLP, Healthcare, Recommendations, Autonomous vehicles, Fraud, Assistive tech. "I See New Hospitals Recommend Autonomous Fraud-Assist."

#### COMMON MISTAKE

**Mistake:** Mixing up speech recognition and NLP.

**Fix:** Speech recognition turns **sound into text** (or a command). NLP understands the **meaning of language** (grammar, context, translation, chat replies). You can have NLP on typed text with no microphone at all.

#### اردو میں سمجھیں

نیورل نیٹ ورکس اور ڈیپ لرننگ کی مشہور ایپلی کیشنز یہ ہیں: تصویر کی پہچان، آواز کی پہچان، قدرتی زبان کی پروسیسنگ (NLP)، صحت کے نظام، تجویز کرنے والے نظام، خود چلنے والی گاڑیاں، فراڈ کی نشاندہی اور معاون ٹیکنالوجیز۔ روزمرہ میں چہرے سے فون کھولنا، یوٹیوب کی تجاویز اور بینک کا مشکوک لین دین پکڑنا سب اسی خاندان سے ہیں۔

#### CHECK YOURSELF

1. Which application converts spoken words into text or commands?
2. Give the source example of a recommendation system.
3. How do self-driving cars use neural networks?

Answers: 1. Speech recognition. 2. YouTube recommending more programming tutorials after a student watches computer-science videos. 3. They process real-time camera and radar data to understand roads, traffic signals, pedestrians and obstacles (for example braking for a pedestrian).

### 5.3.1 Secure Collaboration (Authentication, MFA, Access Control)

#### ★ Golden

#### LEARN IT

Digital tools allow people to **share files, attend online classes, edit documents and work together from different locations**. Collaboration is useful only when **data is protected** and **communication remains safe**.

**Secure collaboration** means using digital tools to work together in a **safe, controlled and responsible** way. It depends on several key elements.

#### i) Authentication / identity verification

**Authentication** is **confirming the identity of a user before allowing access**. Common methods:

- **Username and password**
- **PIN** (a short number code)
- **OTP — One-Time Password**, a code that works only once, usually sent by SMS or an authenticator app
- **Biometrics** — fingerprint or face

Source example: a **banking app sending an OTP to your phone before login**.

#### Multi-factor authentication (MFA) and two-factor authentication (2FA)

A **factor** is a category of proof:

- **Something you know** — password or PIN.
- **Something you have** — your phone, an OTP, a bank SMS.
- **Something you are** — fingerprint, face (biometrics).

**Multi-factor authentication (MFA)** means the system asks for **two or more different factors** before it trusts you. **Two-factor authentication (2FA)** is the common case of MFA that uses **exactly two** factors — usually password + OTP.

If a thief in another city steals only your password, MFA still blocks them because they do not have your phone (or your fingerprint).

#### ii) Access control

**Access control** decides **what an authenticated user is allowed to do**: View, Edit, Comment, Download, Share.

Authentication answers **"Who are you?"** Access control answers **"What may you do?"**

Source example: a teacher shares a Google Doc with students as **"view only"**, but gives group leaders **"edit"** permissions.

### iii) Accountability and tracking

User actions are linked to **verified accounts**. You can see who opened, edited, deleted or shared a file. Example: a school **LMS** (Learning Management System) tracks which teacher uploaded an assignment and which student submitted it.

### iv) Data privacy

**Data privacy** means protecting personal and sensitive information and using it **only for the purpose it was collected**. It involves:

- **Protection** — keeping names, numbers, marks and passwords secure.
- **Controlled sharing** — sharing only with the right people.
- **Confidentiality** — data stays private during storage and transmission.
- **Compliance** — following legal and ethical rules for data handling.

### v) Safe communication / transmission

Exchanging information through digital platforms **securely**, protecting messages from unauthorised interception or wiretapping. Example: a teacher shares class instructions through an **official, encrypted LMS** rather than an open public forum or an unknown WhatsApp broadcast.

#### WORKED EXAMPLE

**Worked example: a Class XII Google Classroom in Sukkur.**

**Step 1 — Authentication.** Each student signs in with a school email and password. The college turns on MFA: after the password, Google sends an OTP to the student's registered Jazz/Zong number.

**Step 2 — Access control.** All 40 students can **view** the assignment sheet. Only the four group leaders can **edit** the shared report. Nobody except the teacher can **delete** the class materials. A student who left the college is removed from the class (access revoked).

**Step 3 — Accountability.** Version history shows that Ali edited paragraph 2 at 8:14 pm and that Hina submitted the PDF at 9:02 pm.

**Step 4 — Privacy and safe transmission.** Marks stay inside Classroom; the teacher does not post CNIC numbers or phone numbers in a public Facebook group. Traffic to Google is encrypted, unlike an open public forum.

If Ali shares his password with a cousin, authentication is broken. If the teacher gives every student "edit" on the marksheet, access control is broken even though login still works.

#### REMEMBER IT

**A-A-A-P-S.** Authentication, Access control, Accountability, Privacy, Safe transmission. MFA = at least two different factor types (know / have / are). 2FA is MFA with two factors.

#### COMMON MISTAKE

**Mistake:** Treating authentication and access control as the same thing.

**Fix:** Login only proves identity. After login, access control still limits View / Edit / Comment / Download / Share. A student who is authenticated must not automatically get teacher rights.

محفوظ تعاون کا مطلب ہے ڈیجیٹل ٹولز سے محفوظ، کنٹرول شدہ اور ذمہ دارانہ انداز میں ساتھ کام کرنا۔ تصدیق (Authentication) یہ جانچتی ہے کہ صارف کون ہے؛ رسائی کنٹرول (Access Control) یہ طے کرتا ہے کہ وہ کیا کر سکتا ہے (صرف دیکھنا، ترمیم، تبصرہ)۔ کثیر عنصری تصدیق (MFA) دو یا زیادہ مختلف ثبوت مانگتی ہے، جیسے پاس ورڈ اور او ٹی پی (OTP)۔ حساب دہی، ڈیٹا پرائیویسی اور محفوظ ترسیل بھی ضروری ہیں۔

#### EXAM TIP

Typical items: "Define secure collaboration", "Differentiate authentication and access control", "What is MFA/2FA?", "Explain five elements of secure collaboration with examples."

Write the five headings (authentication, access control, accountability, privacy, safe transmission). For MFA, name the three factor types and say that 2FA is password + OTP in a banking app. Use the Google Doc "view only vs edit" example — examiners look for it.

#### CHECK YOURSELF

1. What question does authentication answer, and what question does access control answer?
2. Password + OTP is an example of which idea?
3. Give the source example of access control in a Google Doc.

Answers: 1. Authentication: who are you? Access control: what may you do? 2. Two-factor / multi-factor authentication (2FA/MFA). 3. Teacher shares the document as view-only with students but gives group leaders edit permission.

## 5.3.2 Data Protection (Encryption, Passwords, Backup, Firewall)

#### LEARN IT

**Data protection** means keeping digital information safe from **unauthorised access, misuse, loss, damage or accidental deletion**.

### 1. Data encryption

**Encryption** changes readable data (**plaintext**) into a coded form called **ciphertext**. Unauthorised users cannot understand ciphertext without the key.

- **Symmetric encryption** — the **same key** is used to encrypt and to decrypt.
- **Asymmetric encryption** — **two keys**: a **public key** to encrypt, a **private key** to decrypt.

A simple exam cipher is the **Caesar cipher** (a symmetric substitution cipher). You shift every letter a fixed number of places along the alphabet. Shift 3 is the classic key: A → D, B → E, ..., Z → C (the alphabet wraps around).

### 2. Strong passwords

A strong password includes **uppercase, lowercase, numbers and symbols**. It resists **brute-force** guessing (trying every possible combination). Source example: `Zk@2026#Pass`.

Weak: `ahmed123`, `pakistan`, `sindh`, your date of birth. Never reuse the same password on the college LMS and on JazzCash.



### 3. Backup and recovery — the 3-2-1 rule

A **backup** is a copy of data in a safe location (cloud or external drive) so you can **restore** it after hardware damage, ransomware or accidental deletion.

The **3-2-1 backup rule** is a practical standard:

- **3** copies of the data (the original plus two backups).
- **2** different types of storage media (for example laptop disk + USB, or disk + cloud).
- **1** copy **off-site** (a different building or the cloud), so a fire or theft at home does not destroy every copy.

### 4. Firewall and antivirus software

A **firewall** is a security **barrier** that **monitors network traffic** and **blocks suspicious internet access**. Think of it as a checkpoint on the road between your device (or college lab) and the internet.

**Antivirus** software detects and removes harmful software (viruses, spyware and other malware) from the **local machine**.

Firewall  $\approx$  guards the **door to the network**. Antivirus  $\approx$  searches **inside the house** for intruders already present.

#### WORKED EXAMPLE

**Worked example A — Caesar cipher, shift = 3.** Encrypt the plaintext **SINDH**, then decrypt the result.

Write the alphabet and count three letters forward. Wrap after Z back to A.

| Plain | S | I | N | D | H |
|-------|---|---|---|---|---|
| +3    | V | L | Q | G | K |

Ciphertext = **VLQGK**.

**Decrypt:** shift each letter **back** by 3: V  $\rightarrow$  S, L  $\rightarrow$  I, Q  $\rightarrow$  N, G  $\rightarrow$  D, K  $\rightarrow$  H. You recover **SINDH**.

Spaces and punctuation are often left unchanged. The same key (3) is used both ways, so Caesar is **symmetric**.

Second check: **CAT** + 3  $\rightarrow$  C  $\rightarrow$  F, A  $\rightarrow$  D, T  $\rightarrow$  W  $\rightarrow$  **FDW**.

```
def caesar(text, shift):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord("A") if ch.isupper() else ord("a")
            result += chr((ord(ch) - base + shift) % 26 + base)
        else:
            result += ch
    return result

print(caesar("SINDH", 3))
print(caesar("VLQGK", -3))
print(caesar("CAT", 3))
```

#### OUTPUT

```
VLQGK
SINDH
FDW
```

**Worked example B — 3-2-1 for Fatima's Class XII project in Hyderabad.**

1. **Copy 1 (original):** project folder on her laptop hard disk.
2. **Copy 2 (different media):** the same folder on a USB drive she keeps in her bag.
3. **Copy 3 (off-site):** uploaded to Google Drive (cloud servers are not in her house).

That is 3 copies, 2 media types (internal disk + USB, with cloud as the off-site copy), 1 off-site. When her brother deletes the laptop folder, she restores from USB or Drive.

**Password check:** Zk@2026#Pass has uppercase Z, lowercase letters, digits 2026, and symbols @ and #. fatima has none of those extras — reject it.

3 copies



2 media types



1 copy off-site

**REMEMBER IT**

**Cipher** = lock the message. **Password** = lock the door. **3-2-1** = spare keys in three places. **Firewall** = gate. **Antivirus** = indoor search. Caesar: shift letters; same key both ways (symmetric).

**COMMON MISTAKE**

**Mistake 1:** Shifting the wrong way, or forgetting wrap-around (X, Y, Z + 3 → A, B, C).

**Mistake 2:** Two copies on the same laptop (Documents and Desktop) and calling it 3-2-1. That is still one disk in one house.

**Fix:** Count letters on a written alphabet. For backups, different media and one copy that would survive a house fire.

**اردو میں سمجھیں**

ڈیٹا پروٹیکشن کا مطلب ہے معلومات کو غیر مجاز رسائی، غلط استعمال، نقصان یا حادثاتی حذف سے بچانا۔ انکریپشن پڑھے جانے والے متن کو سائفر ٹیکسٹ بنا دیتی ہے؛ سیزر سائفر میں ہر حرف کو مقررہ جگہوں سے آگے سرکایا جاتا ہے۔ مضبوط پاس ورڈ میں بڑے، چھوٹے حروف، عدد اور علامات ہوں۔ 1-2-3 بیک اپ: تین کاپیاں، دو قسم کے میڈیا، ایک کاپی باہر (آف سائٹ)۔ فائر وال نیٹ ورک ٹریفک روکتی ہے؛ اینٹی وائرس مقامی نقصان دہ سافٹ ویئر ہٹاتا ہے۔

**CHECK YOURSELF**

1. Encrypt PAK with a Caesar shift of 3.
2. What do the 3, the 2 and the 1 stand for in 3-2-1 backup?
3. In asymmetric encryption, which key encrypts and which key decrypts?

Answers: 1. SDN (P→S, A→D, K→N). 2. 3 copies, 2 different media, 1 off-site copy. 3. Public key encrypts; private key decrypts.

**5.4.1 Common Security Threats (Malware, Phishing, DoS)**

★ Golden

**LEARN IT**

A **security threat** is any action, event or weakness that can **harm a computer system or its data**.

**Mitigation techniques** are protective measures used to prevent threats or reduce their impact. This section names the common threats; later sections show how to spot them and how to reduce the damage.

| # | Threat         | What it is                                                                                                                                                        | Example                                                                                                                               |
|---|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Malware</b> | Harmful software designed to damage a computer, steal data, or gain unauthorised access. Includes <b>viruses, worms, spyware, ransomware</b> and <b>Trojans</b> . | Installing cracked software that secretly corrupts files, or a "free game" USB that encrypts a college lab's coursework (ransomware). |

| # | Threat                             | What it is                                                                                                                                                                                                 | Example                                                                                                     |
|---|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 2 | <b>Phishing</b>                    | A <b>social-engineering</b> trick that deceives users into sharing personal information (passwords, bank details, OTPs) using fake emails or websites that look genuine.                                   | An SMS that looks like JazzCash or HBL asking you to "verify" by typing your OTP on a fake page.            |
| 3 | <b>Unauthorised access</b>         | Entering a system, account or file without permission. Often caused by weak passwords, shared passwords, or leaving devices unlocked.                                                                      | A classmate using an unlocked lab PC to open someone else's Google Classroom.                               |
| 4 | <b>Data theft / identity theft</b> | <b>Data theft</b> is stealing or copying data without permission. <b>Identity theft</b> is using another person's stolen details (name, CNIC, bank details) to pretend to be them online and commit fraud. | A leaked list of CNICs used to open a SIM or a wallet in someone else's name.                               |
| 5 | <b>Denial of Service (DoS)</b>     | A website or system is flooded with thousands of fake requests, causing it to slow down or crash, so real users cannot access the service.                                                                 | Hackers flooding an online exam portal so students cannot log in on BISE result day or during an e-test.    |
| 6 | <b>Human error</b>                 | Mistakes by legitimate users: accidentally deleting files, sending confidential data to the wrong email, or falling for a simple scam.                                                                     | A teacher emailing the whole marksheet to the wrong parent; a student forwarding an OTP "to help the bank." |

**Malware types in one line each:** a **virus** attaches to files and spreads when you share them; a **worm** spreads by itself across a network; **spyware** secretly watches you; **ransomware** locks your files and demands money; a **Trojan** pretends to be useful software but hides harm.

**Social engineering** means tricking a **person**, not breaking a lock with maths. Phishing is the most common exam example.

#### Phishing

Fake bank email / SMS  
steals password or OTP

#### DoS attack

Flood of fake requests  
server overload / crash

#### Identity theft

Stolen CNIC / details  
fraud in your name

Three high-yield threats: phishing (fake messages), DoS (server overload), identity theft (stolen CNIC or bank details).

### WORKED EXAMPLE

#### Worked example: classify the incident.

1. Bilal downloads a “cracked” Photoshop. Next day his project files are unreadable and a note asks for Bitcoin. → **Malware** (ransomware), started by cracked software.
2. Ayesha gets an email “Your Meezan account will close in 2 hours. Click here and send OTP.” The link is meezn-pk.tk, not the real bank. → **Phishing**.
3. On e-test morning the college portal spins forever; the admin graph shows 200,000 requests from one pattern of fake traffic. → **DoS**.
4. Someone opens a JazzCash wallet using Ahmed’s CNIC copy from a leaked form. → **Identity theft** (after data theft).
5. The lab assistant emails the password list to the whole staff by choosing the wrong group. → **Human error**.

### REMEMBER IT

**Malware = bad program. Phishing = fake message. DoS = flood. Identity theft = stolen ‘you’. Human error = honest mistake.** Cracked software and unknown USBs are the usual door for malware in Pakistani colleges.

### COMMON MISTAKE

**Mistake:** Calling every cyber problem a “virus”, or mixing phishing with malware.

**Fix:** A virus is only one kind of malware. Phishing is a **trick against the user**; it may not install any file at all. DoS does not steal your password — it makes the service unusable for everyone.

### اردو میں سمجھیں

سیکیورٹی خطرہ وہ عمل یا کمزوری ہے جو نظام یا ڈیٹا کو نقصان پہنچائے۔ میلویئر نقصان دہ سافٹ ویئر ہے (وائرس، ورم، سپائی ویئر، رینسم ویئر، ٹروجن)۔ فٹنگ جعلی ای میل یا ویب سائٹ سے پاس ورڈ اور او ٹی پی چھینتی ہے۔ ڈو ایس (DoS) جعلی درخواستوں کی بارش سے سائٹ کریش کر دیتا ہے۔ دیگر خطرات: غیر مجاز رسائی، ڈیٹا/شناخت کی چوری، اور انسانی غلطی۔

### EXAM TIP

Expect “Define a security threat”, “Explain malware / phishing / DoS with examples”, and “Differentiate data theft and identity theft.” List all six threats if the question says “common security threats.” For DoS, write: flood of fake requests → slowdown or crash → real users blocked, and quote the online exam portal. For phishing, write social engineering + fake genuine-looking email/site + passwords/OTPs.

### CHECK YOURSELF

1. Name five types of malware listed in the source.
2. What does a DoS attack do to a website?
3. How is identity theft different from data theft?

Answers: 1. Viruses, worms, spyware, ransomware, Trojans. 2. Floods it with thousands of fake requests so it slows down or crashes and real users cannot access it. 3. Data theft is stealing/copying data; identity theft is using stolen personal details (name, CNIC, bank) to pretend to be that person and commit fraud.

## 5.4.2 Identifying Security Threats

### LEARN IT

**Identifying security threats** means **noticing signs** that data or systems may be at risk. Early identification helps you act before serious damage occurs.

| Sign / indicator                                                                          | What it likely means                                |
|-------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Slow performance, missing files, or random pop-ups.                                       | <b>Malware</b> or background unauthorised activity. |
| Failed login attempts or strange login locations.                                         | Attempted <b>unauthorised access</b> .              |
| Emails urging quick action, asking for an OTP, or containing bad grammar / strange links. | Classic <b>phishing</b> .                           |
| Unexpected deletion, copying or movement of records.                                      | <b>Data theft</b> or insider misuse.                |
| Sudden massive spike in server requests or a very slow website.                           | An ongoing <b>DoS</b> attack.                       |

The first line of defence is also simple behaviour: avoid **unsafe downloads, cracked software, unverified websites and infected USB drives**.

### WORKED EXAMPLE

**Worked example: read the signs in a college computer lab in Khairpur.**

1. Three PCs show pop-ups and Word files have vanished after students used a "game" USB. → Treat as **malware**. Disconnect from the network; run antivirus; stop USB use.
2. Google Classroom emails: "Your account will be deleted in 30 minutes. Confirm password here." Link spelling is go0gle. → **Phishing**. Do not click; report to the teacher.
3. The e-portal that was fine at 8:00 am is unreachable at 8:05; the technician sees a sudden spike in requests. → Suspect **DoS**.
4. Nadia's Gmail security page shows a login from another country at 4 a.m. → **Unauthorised access** attempt. Change password; turn on MFA.

### REMEMBER IT

**Slow/pop-ups → malware. Odd logins → unauthorised access. Urgent OTP email → phishing. Files moving → theft. Traffic spike → DoS.**

### COMMON MISTAKE

**Mistake:** Ignoring early signs ("the PC is just old") until the ransomware note appears.

**Fix:** One unusual login or one OTP email is enough to investigate. Early action is cheaper than recovery.

### اردو میں سمجھیں

خطرے کی جلد پہچان نقصان سے پہلے اقدام ممکن بناتی ہے۔ سست روی، غائب فائلیں یا پاپ اپ میلویئر کی علامت ہیں؛ عجیب لاگ ان غیر مجاز رسائی کی؛ فوری او ٹی پی والی ای میل فشنگ کی؛ ریکارڈ کا اچانک منتقل ہونا ڈیٹا چوری کی؛ سرور پر درخواستوں کا دھماکہ ڈو ایس کی۔ کریک سافٹ ویئر، نامعلوم یو ایس بی اور غیر تصدیق شدہ ویب سائٹوں سے بچیں۔

#### CHECK YOURSELF

1. Random pop-ups and missing files most likely indicate what?
2. An email with bad grammar that demands your OTP is a sign of what?
3. Name four unsafe habits that invite these threats.

Answers: 1. Malware (or background unauthorised activity). 2. Phishing. 3. Unsafe downloads, cracked software, unverified websites, infected USB drives.

### 5.4.3 Apply Mitigation Techniques

#### LEARN IT

**Mitigation techniques** are actions taken to **prevent** security threats or **reduce their harmful effects** once identified. Technology helps, but it cannot stop all threats if the user is careless.

| # | Technique                                | How it helps                                                                                                                                                      |
|---|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Antivirus and firewalls</b>           | Antivirus scans and removes harmful software (for example from USB drives). A firewall monitors network traffic and blocks suspicious access from the internet.   |
| 2 | <b>Strong passwords and 2FA</b>          | Complex passwords slow brute-force guessing. 2FA adds an extra layer: an OTP on your phone as well as the password.                                               |
| 3 | <b>Data encryption</b>                   | Converts data into unreadable code during transmission. Protects passwords and sensitive data if they are intercepted.                                            |
| 4 | <b>Regular backups</b>                   | Copies on cloud or external drives allow quick recovery after a crash, human error or ransomware.                                                                 |
| 5 | <b>User awareness (the human factor)</b> | Training users to avoid unknown links, suspicious attachments and password sharing. The human is often the weakest link — and the strongest defence when trained. |

#### WORKED EXAMPLE

**Worked example: match each threat to a mitigation — BISE e-test week at a college in Larkana.**

| Threat seen                                       | Mitigation to apply                                                                                   |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| USB games in the lab, pop-ups starting.           | Scan with <b>antivirus</b> ; block unknown USBs; enable the lab <b>firewall</b> .                     |
| Phishing SMS about "update your portal password." | <b>User awareness</b> : do not click; verify on the official site typed by hand. Turn on <b>2FA</b> . |
| Fear that intercepted Wi-Fi could show passwords. | Use sites with <b>encryption</b> (HTTPS / encrypted LMS); never send passwords in plain WhatsApp.     |
| Ransomware risk to project files.                 | <b>Regular 3-2-1 backups</b> so files can be restored without paying.                                 |
| Shared passwords on a sticky note on the monitor. | <b>Strong unique passwords</b> + 2FA; remove the note; awareness session.                             |

#### REMEMBER IT

**Five shields: A-F, P-2, E, B, U.** Antivirus+Firewall, Passwords+2FA, Encryption, Backups, User awareness. Ransomware is why backups matter: if files are locked, a clean copy brings them back.

#### COMMON MISTAKE

**Mistake:** "If I installed antivirus I am 100% safe."

**Fix:** Antivirus does not stop you from typing an OTP into a fake page. Mitigation is a **set** of techniques plus human caution.

#### اردو میں سمجھیں

تحفظ کی تدابیر خطرات کو روکتی یا ان کا نقصان کم کرتی ہیں: اینٹی وائرس اور فائر وال، مضبوط پاس ورڈ اور دو عنصری تصدیق (2FA)، انکرپشن، باقاعدہ بیک اپ، اور صارف کی آگاہی۔ ٹیکنالوجی تب ناکام ہو جاتی ہے جب صارف نامعلوم لنک کھولے یا پاس ورڈ بانٹے۔

#### CHECK YOURSELF

1. How does 2FA add a layer on top of a password?
2. Why do regular backups help after ransomware?
3. What three careless behaviours should awareness training stop?

Answers: 1. It also requires an OTP sent to your phone (a second factor). 2. You can restore a clean copy instead of losing the files (or paying). 3. Unknown links, suspicious attachments, and password sharing.

## 5.5 Equity and Equal Access in Digital Collaboration

#### LEARN IT

Collaboration tools are vital for online learning and teamwork. To use these tools fairly, all users must participate **without barriers**. You must understand the difference between **equal access** and **equity**.

#### Equal access

Giving everyone the **exact same** opportunity, tools and resources.

Example: providing every student with the **same login ID** to access a Zoom class.

#### Equity

Giving **additional, specific support** to users who need it, so they can reach the **same outcome**.

Example: turning on live **subtitles/captions** in Zoom specifically for a student who has hearing difficulties.

Equal access is the same starting kit. Equity is extra help so a person who faces a barrier can still finish the race with everyone else. Fair collaboration needs **both**: the same Zoom link for all (equal access) **and** captions, recorded lectures or a low-data app where needed (equity).

#### WORKED EXAMPLE

**Worked example: one class, two ideas.** A teacher in Thatta starts online classes.

**Equal access:** every student receives the same Zoom ID and password. That is fair in the sense of "same tool," but a student with hearing loss still cannot follow the spoken lecture, and a student in a village with a weak 2G signal cannot join video.

**Equity:** the teacher turns on captions, posts a low-size audio recording, and accepts the assignment as a Google Doc in the browser (not only through a heavy app). Now those two students can reach the **same learning outcome**.

#### REMEMBER IT

**Equal = same tools. Equity = extra support for the same outcome.** Same shoes for all vs a ramp for the wheelchair user.

#### COMMON MISTAKE

**Mistake:** "We gave everyone the same login, so the class is fair."

**Fix:** Same login is equal access only. If some students still cannot hear, see, afford a smartphone, or get electricity, you still need equity.

#### اردو میں سمجھیں

مساوی رسائی (Equal Access) سب کو ایک جیسے اوزار دیتی ہے، جیسے ایک ہی زوم لاگ ان۔ انصاف (Equity) ضرورت مند کو اضافی مدد دیتی ہے تاکہ وہی نتیجہ نکل سکے، جیسے سماعت کی مشکل والے طالب علم کے لیے زندہ ذیلی عنوانات (Captions)۔ منصفانہ تعاون کے لیے دونوں چاہییں۔

#### CHECK YOURSELF

1. Define equal access in one sentence.
2. Define equity in one sentence.
3. Why are Zoom captions an example of equity, not merely equal access?

Answers: 1. Giving everyone the exact same opportunity, tools and resources. 2. Giving additional, specific support so users who need it can reach the same outcome. 3. Because they are extra support for a student with hearing difficulties, not the same identical tool given to all.

### 5.5.1 Functions of Equity and Equal Access ★ Golden

#### LEARN IT

Equity and equal access are not slogans. They **do work** in a digital classroom. The source lists four functions.

| Function                                 | What it does                                         | Pakistani example                                                                                                                  |
|------------------------------------------|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1. Providing need-based support (equity) | Help tailored to individuals.                        | Audio descriptions for visually impaired students; a screen reader so a blind student in Karachi can hear a digital textbook.      |
| 2. Removing barriers                     | Help students in remote areas attend online classes. | Mobile-friendly, <b>low-data</b> apps so a student in Tharparkar or the katcha areas of Sindh can join without a fibre connection. |
| 3. Enabling active participation         | Students who miss live class can still take part.    | <b>Recorded lectures</b> so load-shedding or an internet outage in Hyderabad does not make a student fall behind.                  |
| 4. Supporting different learning needs   | Offer more than one format.                          | Videos, text notes and audio of the same Chapter 5 topic so each student learns in the way that suits them best.                   |



### WORKED EXAMPLE

**Worked example: design four functions for one class WhatsApp + Classroom group in Nawabshah.**

1. **Need-based support:** Ali is visually impaired — share materials as selectable text (not only photos) so a screen reader can speak them; add a short audio description of each diagram.
2. **Remove barriers:** post a 5 MB audio summary instead of a 200 MB video for students on expensive mobile data.
3. **Active participation:** upload the Zoom recording within an hour; students whose lights went out at 7 pm watch it at 11 pm.
4. **Different learning needs:** the same lesson as (a) a 12-minute video, (b) a one-page PDF, (c) an Urdu audio note.

**Discussion from the source:** if a teacher **only** accepts assignments via an app that needs a **high-end smartphone**, is that equal access? **No**. It creates a **hardware barrier** for lower-income students. Web-based or light apps give better access.

### REMEMBER IT

**Four functions: Support, Remove, Participate, Formats.** "Some Rural Pupils Fail" without them — Support (need-based), Remove barriers, Participate via recordings, Formats (video/text/audio).

### COMMON MISTAKE

**Mistake:** Describing only captions and stopping.

**Fix:** Examiners want all four functions. Include remote-area low-data apps, recorded lectures for power cuts, and multiple formats — these are in the Sindh source on purpose.

### اردو میں سمجھیں

انصاف اور مساوی رسائی کے چار کام ہیں: ضرورت کے مطابق مدد (جیسے نابینا طالب علم کے لیے آڈیو وضاحت)، رکاوٹیں ہٹانا (دور دراز علاقوں کے لیے کم ڈیٹا والی ایپس)، فعال شرکت ممکن بنانا (لوڈ شیڈنگ کے لیے ریکارڈ شدہ لیکچر)، اور مختلف اندازِ تعلیم کی حمایت (ویڈیو، متن، آڈیو)۔ مہنگے فون والی ایپ پر صرف اسی طرح اسائنمنٹ لینا مساوی رسائی نہیں۔

### EXAM TIP

Golden question: "Explain the functions of equity and equal access in digital collaboration" (5–8 marks). Write the four headings in order, one example each, and add the hardware-barrier point (high-end app vs light/web app). Another favourite: "Differentiate equal access and equity with examples" — same login vs captions.

### CHECK YOURSELF

1. List the four functions of equity and equal access.
2. How do recorded lectures enable active participation in Pakistan?
3. Why is a high-end-smartphone-only assignment unfair?

Answers: 1. Need-based support; removing barriers; enabling active participation; supporting different learning needs. 2. Students who face power or internet outages can watch later and not fall behind. 3. It creates a hardware barrier for lower-income students; web-based or light apps give better access.

## 5.5.2 Collaboration Tools

### LEARN IT

**Digital collaboration tools** are online platforms that support **communication, file sharing, learning and teamwork** across different locations.

| Tool                   | Primary function and educational use                                                                                             |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Google Docs            | Write, edit, comment and share documents in real time. Excellent for group assignments.                                          |
| Zoom                   | Online meetings, virtual classes, screen sharing and video-based communication.                                                  |
| Microsoft Teams        | A unified workspace for online meetings, file sharing, class groups and structured teamwork.                                     |
| Google Classroom / LMS | Learning Management Systems help teachers manage lectures, distribute assignments, conduct quizzes and grade student activities. |
| WhatsApp               | Quick class announcements, urgent reminders and instant group communication. Must be used responsibly.                           |
| Email                  | Formal communication, official notices, submitting large attachments, and keeping a professional written record.                 |

**Responsibly** on WhatsApp means: no leaking of classmates' marks or photos, no fake forwards, and no forcing a student who has no smartphone to miss official work that was never posted on the LMS.

### WORKED EXAMPLE

**Worked example: choose the right tool for each job in a Hyderabad college group project.**

1. Write the report together after school, from different houses. → **Google Docs** (real-time edit and comments).
2. Sunday 7 pm teacher explanation with slides. → **Zoom** or **Teams** (video + screen share). Turn on captions (equity).
3. Collect homework, quiz and grades in one place. → **Google Classroom / LMS**.
4. "Tomorrow's test is postponed — tell everyone in the next ten minutes." → **WhatsApp** class group (then repeat on Classroom so nobody is left out).
5. Send the principal a formal complaint with a 8 MB scanned letter. → **Email** (written record + attachment).

### REMEMBER IT

**Docs = write together. Zoom/Teams = meet. Classroom = official class hub. WhatsApp = fast but informal. Email = formal record.**

### COMMON MISTAKE

**Mistake:** Running the whole course only on WhatsApp.

**Fix:** WhatsApp is for quick reminders. Assignments, marks and lectures belong on an LMS so there is access control, a record, and a path that does not depend on one person's phone.

ڈیجیٹل تعاون کے اوزار مختلف مقامات سے بات چیت، فائل شیئرنگ اور ٹیم ورک چلاتے ہیں: گوگل ڈاکس مشترکہ تحریر کے لیے، زوم اور مائیکروسافٹ ٹیمز میٹنگ کے لیے، گوگل کلاس روم یا ایل ایم ایس باقاعدہ پڑھائی کے لیے، واٹس ایپ فوری اطلاع کے لیے (ذمہ داری سے)، اور ای میل رسمی ریکارڈ کے لیے۔

#### CHECK YOURSELF

1. Which tool is excellent for real-time group writing?
2. What four jobs does an LMS / Google Classroom help a teacher do?
3. What is email especially good for?

Answers: 1. Google Docs. 2. Manage lectures, distribute assignments, conduct quizzes, grade activities. 3. Formal communication, official notices, large attachments, and a professional written record.

## Career Opportunities

#### LEARN IT

This unit covers neural networks, deep learning, secure collaboration, data protection and digital equity. Mastery of these ideas opens high-demand careers in **Artificial Intelligence** and **Cybersecurity** — including in Pakistan's banks, software houses, universities and public projects linked to the National AI Policy 2025.

| Career                           | What the person does                                                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Machine Learning Engineer</b> | Builds and trains machine-learning and neural-network models used in face recognition, NLP and recommendation systems.                           |
| <b>Cybersecurity Analyst</b>     | Monitors systems for threats such as malware and phishing, and applies mitigation (firewalls, 2FA) to protect organisational data.               |
| <b>AI Researcher</b>             | Studies and develops new techniques in artificial intelligence and deep learning, often in university labs or high-tech R&D centres.             |
| <b>Data Privacy Officer</b>      | Ensures the organisation's data handling complies with legal privacy principles, protecting sensitive user information and secure collaboration. |
| <b>Ethical Hacker</b>            | Legally tests systems and networks for vulnerabilities (penetration testing) before malicious attackers can exploit them via DoS or malware.     |

#### WORKED EXAMPLE

**Worked example: match the job.**

1. "Train a model that tags Urdu news headlines." → **Machine Learning Engineer.**
2. "Watch the bank's logs for phishing domains and turn on 2FA." → **Cybersecurity Analyst.**
3. "With written permission, try to break the college portal before criminals do." → **Ethical Hacker.**

#### REMEMBER IT

AI careers lean on Python, maths and ethics. Cyber careers lean on threats, 2FA, firewalls and careful testing. Both need clear communication.

#### COMMON MISTAKE

**Mistake:** Confusing an ethical hacker with a criminal hacker.

**Fix:** An ethical hacker has **permission** and reports weaknesses so they can be fixed. Attacking without permission is a crime.

#### اردو میں سمجھیں

اس باب سے جڑے پیشے ہیں: مشین لرننگ انجینئر، سائبر سیکیورٹی اینالسٹ، اے آئی محقق، ڈیٹا پرائیویسی آفیسر اور اخلاقی ہیکر۔ اخلاقی ہیکر اجازت کے ساتھ کمزوریاں ڈھونڈتا ہے تاکہ مجرمانہ حملے سے پہلے انہیں ٹھیک کیا جا سکے۔

#### CHECK YOURSELF

1. Who legally tests a network for weaknesses before attackers do?
2. Which career focuses on compliance and protecting sensitive user information?

Answers: 1. Ethical hacker. 2. Data Privacy Officer.

## Chapter 5 Review

### Chapter summary

- **ML** is a model (a trained brain) that learns from data in three steps: input, learning, prediction on new unseen data — without a hand-written rule for every case.
- **AI**  $\supset$  **ML**  $\supset$  **DL**: deep learning sits inside machine learning, which sits inside artificial intelligence.
- A **neural network** is a brain-inspired mathematical model of connected neurons in an input layer, hidden layer(s) and output layer (face unlock: camera  $\rightarrow$  features  $\rightarrow$  unlock).
- **Deep learning** trains neural networks with many hidden layers; early layers learn simple features, deeper layers learn complex patterns. It needs huge data and computing power.
- NN components: input layer, neurons, hidden layers, weights, bias, activation function, output layer, loss function. A neuron computes  $s = \sum(x \times w) + b$ , then an activation gate.
- DL components: large data, many hidden layers, automatic feature learning, learning algorithm (backpropagation), GPUs, training epochs. Pakistan's National AI Policy 2025 covers education, health, agriculture, banking and local languages.
- Applications: image and speech recognition, NLP, healthcare imaging, recommendations, autonomous vehicles, fraud detection, assistive tech.
- **Secure collaboration**: authentication (who), MFA/2FA (two or more factors), access control (what you may do), accountability, privacy, safe transmission.
- **Data protection**: encryption (plaintext  $\rightarrow$  ciphertext; symmetric vs asymmetric; Caesar shift), strong passwords, 3-2-1 backup, firewall and antivirus.
- Threats: malware, phishing, unauthorised access, data/identity theft, DoS, human error. Spot them by pop-ups, odd logins, urgent OTP emails, missing files, traffic spikes.
- Mitigation: antivirus + firewall, strong passwords + 2FA, encryption, regular backups, user awareness.
- **Equal access** = same tools for all; **equity** = extra support for the same outcome. Functions: need-based support, removing barriers, recorded participation, multiple formats. Tools: Docs, Zoom, Teams, Classroom/LMS, WhatsApp, email.

## Key terms

| Term                         | Meaning                                                                             | اردو                   |
|------------------------------|-------------------------------------------------------------------------------------|------------------------|
| Artificial intelligence (AI) | The broad field in which machines perform tasks that normally need human thinking.  | مصنوعی ذہانت           |
| Machine learning (ML)        | A model that learns patterns from data and then predicts on new data.               | مشین لرننگ             |
| Neural network (NN)          | A brain-inspired mathematical model of connected neurons in layers.                 | نیورل نیٹ ورک          |
| Deep learning (DL)           | An ML method that trains neural networks with many hidden layers.                   | ڈیپ لرننگ              |
| Weight                       | A number showing the strength or importance of a connection between neurons.        | وزن (وزنِ تعلق)        |
| Bias                         | An extra constant added to a neuron's sum to improve accuracy.                      | بائیس                  |
| Activation function          | A gate that decides whether a signal is important enough to pass on.                | ایکٹیویشن فنکشن        |
| Loss function                | A measure of how wrong the model's answer is compared with reality.                 | لاس فنکشن              |
| Epoch                        | One full training pass through the dataset.                                         | ایپوک (تربیتی چکر)     |
| GPU                          | Graphics Processing Unit — specialised hardware used to train deep networks.        | جی پی یو               |
| NLP                          | Natural Language Processing — computers understanding and producing human language. | قدرتی زبان کی پروسیسنگ |
| Authentication               | Confirming a user's identity before access (password, PIN, OTP, biometrics).        | تصدیقِ شناخت           |
| MFA / 2FA                    | Multi-/two-factor authentication: two or more different proofs of identity.         | کثیر / دو عنصری تصدیق  |
| Access control               | Rules for what an authenticated user may do (view, edit, share, ...).               | رسائی پر قابو          |
| Encryption                   | Turning plaintext into ciphertext so unauthorised people cannot read it.            | رمز کاری (انکرپشن)     |
| Caesar cipher                | A symmetric cipher that shifts letters by a fixed number of places.                 | سیزر سائفر             |

| Term                    | Meaning                                                                              | اردو               |
|-------------------------|--------------------------------------------------------------------------------------|--------------------|
| 3-2-1 backup            | Three copies, two media types, one copy stored off-site.                             | 3-2-1 بیک اپ       |
| Firewall                | A barrier that monitors network traffic and blocks suspicious access.                | فائر وال           |
| Malware                 | Harmful software such as viruses, worms, spyware, ransomware and Trojans.            | میلویئر            |
| Phishing                | Fake messages or sites that trick people into giving passwords or OTPs.              | فشنگ               |
| Denial of Service (DoS) | Flooding a service with fake requests so real users cannot use it.                   | سروس سے انکار      |
| Equal access            | Giving everyone the same tools and opportunity.                                      | مساوی رسائی        |
| Equity                  | Giving extra, specific support so people facing barriers reach the same outcome.     | انصاف (ایکویٹی)    |
| LMS                     | Learning Management System — platform for lectures, assignments, quizzes and grades. | تعلیمی نظم کا نظام |

## Practice: multiple choice

1. A machine-learning model works in which order?
  - a. Prediction, then data input, then learning
  - b. Data input, learning, then prediction on new data
  - c. Only prediction, with no data
  - d. Writing a separate rule for every case
2. A neural network is best described as:
  - a. a living human brain placed in a computer
  - b. a backup disk
  - c. a brain-inspired mathematical model of connected neurons
  - d. a type of firewall
3. The word "deep" in deep learning refers to:
  - a. several hidden layers through which data passes
  - b. very loud speakers
  - c. using only one neuron
  - d. storing data underground
4. In the relationship  $AI \supset ML \supset DL$ , the innermost set is:
  - a. AI
  - b. ML
  - c. equal access
  - d. DL
5. Weights in a neural network show the:
  - a. colour of the screen
  - b. importance or strength of connections between neurons
  - c. size of the hard disk
  - d. student's roll number
6. For  $x_1 = 1$ ,  $x_2 = 0$ ,  $w_1 = 0.5$ ,  $w_2 = 0.5$ ,  $b = 0$  and threshold 0.4, the neuron output  $y$  is:
  - a. 1
  - b. 0
  - c. 0.4
  - d. 2
7. Which of the following is a component of deep learning (not of a tiny shallow network)?
  - a. Exactly one hidden layer and ten rows of data
  - b. Manual listing of every feature by a human, always
  - c. Many hidden layers, huge datasets and GPUs
  - d. A Caesar cipher
8. Unlocking a phone by matching facial features is mainly an example of:
  - a. a DoS attack
  - b. 3-2-1 backup
  - c. equal access
  - d. image recognition
9. Authentication means:
  - a. choosing who may edit a file
  - b. confirming a user's identity before allowing access
  - c. shifting letters in a cipher
  - d. flooding a server with requests
10. Password plus an OTP sent to a phone is an example of:
  - a. MFA / 2FA
  - b. a worm
  - c. a shallow neural network
  - d. equity captions

11. With a Caesar cipher of shift 3, the plaintext CAT becomes:
  - a. DBU
  - b. CAW
  - c. FDW
  - d. ZXQ
12. The 3-2-1 backup rule requires:
  - a. three passwords only
  - b. one copy on the same laptop twice
  - c. no off-site copy
  - d. three copies, two media types, one copy off-site
13. Phishing is:
  - a. a social-engineering trick using fake emails or sites to steal passwords or OTPs
  - b. a type of GPU
  - c. an activation function
  - d. a collaboration tool like Zoom
14. A Denial of Service (DoS) attack:
  - a. always steals a CNIC
  - b. floods a system with fake requests so it slows or crashes
  - c. is the same as a strong password
  - d. is a hidden layer
15. Turning on live captions in Zoom for a student with hearing difficulties is an example of:
  - a. malware
  - b. a DoS attack
  - c. equal access only, with no extra support
  - d. equity

### Practice: short questions

1. Define a machine-learning model and state the three steps of how it works.
2. What is a neural network? Name its three kinds of layer and link them to face unlock.
3. Differentiate a neural network (the model) from deep learning (the method).
4. State the relationship  $AI \supset ML \supset DL$  in words.
5. List eight components of a neural network. What do weights and bias do?
6. Name six components of deep learning.
7. Differentiate authentication, MFA and access control with one example each.
8. Encrypt SINDH using a Caesar cipher with shift 3. State whether this cipher is symmetric or asymmetric.
9. Define malware, phishing and DoS. Give one Pakistani-style example of each.
10. Differentiate equal access and equity. List four functions of equity and equal access.

### Practice: long / problem questions

1. Explain machine learning, neural networks and deep learning. Draw and label the set diagram  $AI \supset ML \supset DL$ . Include the face-unlock walk-through and the photo-gallery example.
2. A neuron has inputs  $x_1 = 0.8$ ,  $x_2 = 0.5$ , weights  $w_1 = 0.4$ ,  $w_2 = 0.6$ , bias  $b = 0.1$  and threshold 0.5. (a) Compute the weighted sum  $s$  and the output  $y$ . (b) Repeat for  $x_1 = 0.2$ ,  $x_2 = 0.3$ . (c) Name all eight NN components and sketch a 3-4-2 network. (d) List the six DL components and map them onto a cotton-leaf disease detector in Sindh.
3. Describe eight applications of neural networks and deep learning with examples. Then discuss the future of AI in Pakistan with reference to the National AI Policy 2025 and five sectors.
4. What is secure collaboration? Explain authentication, MFA/2FA, access control, accountability, data privacy and safe transmission, using a Google Classroom / banking-app scenario from Pakistan.
5. Define data protection. Explain encryption (symmetric vs asymmetric) with a full Caesar-cipher working of SINDH (shift 3). Describe strong passwords, the 3-2-1 backup rule, firewall and antivirus.
6. Define a security threat. Explain malware, phishing, unauthorised access, data/identity theft, DoS and human error. Give a table of identifying signs and a matching set of mitigation techniques. End with equity versus equal access and a fair choice of collaboration tools.



## Answer key

---

### Multiple choice

1. b
2. c
3. a
4. d
5. b
6. a
7. c
8. d
9. b
10. a
11. c
12. d
13. a
14. b
15. d

### Short questions — model answers

1. An ML model is the system that learns from data and gives an output — the trained brain of a program. Steps: (1) data input — massive data is given; (2) learning — it studies data and finds patterns/rules automatically; (3) prediction — after training it decides on new, unseen data.
2. A neural network is a computer model that learns patterns from data; it is inspired by the human brain but works as a mathematical system of connected neurons/nodes. Input layer receives the camera image; hidden layers study features such as eye distance and nose shape; output layer unlocks the phone if they match stored data.
3. A neural network is the model: layers, nodes and connections; it can be shallow (one or two hidden layers). Deep learning is the method of training networks that have many hidden layers, needing huge data and processing power for complex patterns.
4. Artificial intelligence is the largest set; machine learning is a part of AI; deep learning is a part of ML. All DL is ML and all ML is AI, but not all AI is ML and not all ML is DL.
5. Input layer; neurons/nodes; hidden layer(s); weights; bias; activation function; output layer; loss function. Weights show connection strength/importance. Bias is an extra constant that helps adjust the result and improve accuracy.
6. Large input data; many hidden layers; automatic feature learning; learning algorithm (e.g. backpropagation); high computing power (GPUs); training process (epochs).
7. Authentication confirms identity before access (password, PIN, OTP, biometrics — e.g. bank OTP). MFA/2FA requires two or more factor types, typically password + OTP. Access control decides what that user may do (view/edit/comment/download/share — e.g. Google Doc view-only for the class, edit for group leaders).
8.  $S \rightarrow V$ ,  $I \rightarrow L$ ,  $N \rightarrow Q$ ,  $D \rightarrow G$ ,  $H \rightarrow K$  so ciphertext VLQ GK. Caesar uses the same shift key to encrypt and decrypt, so it is symmetric.
9. Malware: harmful software (viruses, worms, spyware, ransomware, Trojans) — e.g. cracked software that corrupts files. Phishing: fake genuine-looking email/SMS/site stealing passwords or OTPs — e.g. fake JazzCash OTP page. DoS: flood of fake requests that slows or crashes a service — e.g. online exam portal students cannot enter.
10. Equal access: same tools/opportunity for everyone (same Zoom login). Equity: extra specific support so people with barriers reach the same outcome (captions for hearing difficulties). Functions: need-based support; removing barriers (low-data apps for remote areas); enabling participation (recorded lectures during load-shedding); supporting different learning needs (video, text, audio).

## Long questions — model answers

1. **ML**: trained-brain model; data → learn patterns → predict on unseen data; used in face unlock, speech, photo search, diagnosis, translation. **NN**: brain-inspired maths model; neurons pass information through Input, Hidden, Output; face unlock: camera (input), eye distance/nose shape (hidden), unlock (output). **DL**: ML method with multi-layer NNs; “deep” = several hidden layers; early layers: edges/shapes/colours; deeper: whole face or car model; gallery search example. **Diagram**: three nested sets labelled AI, ML, DL with AI outermost. Sentence:  $DL \subset ML \subset AI$ .
2. **(a)**  $s = 0.8 \times 0.4 + 0.5 \times 0.6 + 0.1 = 0.32 + 0.30 + 0.1 = 0.72$ ;  $0.72 \geq 0.5$  so  $y = 1$ . **(b)**  $s = 0.2 \times 0.4 + 0.3 \times 0.6 + 0.1 = 0.08 + 0.18 + 0.1 = 0.36$ ;  $0.36 < 0.5$  so  $y = 0$ . **(c)** Eight components as in the table; sketch three input circles, four hidden, two output, labelled layers. **(d)** Huge labelled leaf photos; many hidden layers; features learned automatically; backpropagation; GPU training; many epochs until accuracy rises — cotton pest detector for a Sindh farm.
3. Cover all eight: image recognition (face lock, cameras); speech (assistant alarm, Urdu voice); NLP (Translate, chatbots); healthcare (X-ray/CT/MRI, pneumonia); recommendations (YouTube CS videos); autonomous vehicles (brake for pedestrian); fraud (unusual bank transaction); assistive (speech-to-text). Then Policy 2025: innovation, skills, ethics, digital growth. Sectors: education (tutoring, assessments); healthcare (rural diagnosis); agriculture (crops, irrigation, yield); business/banking (fraud, chatbots); local languages (Urdu, Sindhi). Students need Python, maths, data analysis, problem-solving, ethics.
4. Secure collaboration = working together with digital tools in a safe, controlled, responsible way. Authentication: prove identity (password/PIN/OTP/biometrics); bank OTP example. MFA: two or more of know/have/are; 2FA = password + phone OTP. Access control: view/edit/comment/download/share; teacher view-only vs group-leader edit. Accountability: LMS tracks who uploaded/submitted. Privacy: protect names, marks, passwords; share only with the right people; confidentiality; legal/ethical compliance. Safe transmission: encrypted official LMS, not an open public forum. Classroom scenario: school email + MFA, role permissions, version history, no CNICs on public WhatsApp.
5. Data protection = keeping digital information safe from unauthorised access, misuse, loss, damage or accidental deletion. Encryption: plaintext → ciphertext; symmetric = same key both ways; asymmetric = public encrypt, private decrypt. Caesar shift 3: SINDH → VLQKG; decrypt by -3; wrap-around at Z. Strong password: upper, lower, number, symbol (e.g. Zk@2026#Pass) against brute force. 3-2-1: three copies, two media, one off-site (laptop + USB + Google Drive). Firewall: barrier monitoring/blocking suspicious network traffic. Antivirus: detects/removes malware on the local machine.
6. A security threat is any action, event or weakness that can harm a system or its data. Malware (cracked software); phishing (fake bank mail); unauthorised access (unlocked PC, weak/shared passwords); data theft vs identity theft (stolen CNIC used for fraud); DoS (exam portal flood); human error (wrong email, deleting files, falling for a scam). Signs table: pop-ups/missing files → malware; odd logins → unauthorised access; urgent OTP/bad grammar → phishing; records moved → theft; traffic spike → DoS. Mitigation: antivirus+firewall; strong passwords+2FA; encryption in transit; regular backups; user awareness (no unknown links, attachments, or shared passwords). Equal access vs equity; do not require a high-end phone-only app. Tools: Docs, Zoom, Teams, Classroom/LMS, WhatsApp (responsible), email (formal record).

## Self-assessment: I can...

- |                                                                                                      |                                                                                                        |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> define ML and explain data input → learning → prediction.                   | <input type="checkbox"/> draw $AI \supset ML \supset DL$ and give a system that belongs in each layer. |
| <input type="checkbox"/> describe a neural network and the face-unlock I-H-O path.                   | <input type="checkbox"/> compute a neuron's weighted sum and threshold output from numbers.            |
| <input type="checkbox"/> list eight NN components and six DL components.                             | <input type="checkbox"/> give eight applications of NN/DL and Pakistan's AI policy sectors.            |
| <input type="checkbox"/> explain authentication, MFA/2FA and access control with examples.           | <input type="checkbox"/> encrypt a short word with a Caesar cipher and apply the 3-2-1 backup rule.    |
| <input type="checkbox"/> identify malware, phishing and DoS from signs and name mitigations.         | <input type="checkbox"/> distinguish equal access from equity and state four functions of equity.      |
| <input type="checkbox"/> choose a fair collaboration tool (Docs, Zoom, Teams, LMS, WhatsApp, email). |                                                                                                        |

# Entrepreneurship in the Digital Age

This chapter shows you how a real problem in your home, college or bazaar can become a business idea, and how you can test that idea cheaply with paper sketches, prototypes and a Minimum Viable Product (MVP) before you spend a lot of money. You will also learn how to choose your first users, the beachhead market, and how to collect feedback honestly and safely.

اس باب میں آپ سیکھیں گے کہ روزمرہ زندگی کے کسی حقیقی مسئلے کو کاروباری خیال میں کیسے بدلا جائے، اور پروٹوٹائپ (Prototype) اور ایم وی پی (MVP) کے ذریعے کم خرچ میں اس خیال کو کیسے آزمایا جائے۔

## What you will learn

- Define **entrepreneur** and **entrepreneurship** and describe the habits of an entrepreneur.
- Turn an everyday problem into a business idea using the steps Observe, Ask Users, Select One Problem, Suggest Solution, Test Early.
- Draw and explain the prototype development cycle: Design, Build, Test, Iterate.
- Identify the **riskiest assumption** of a business idea and design a small test for it.
- Choose a focused **beachhead market** for a new idea.
- Explain how digital tools (WhatsApp, Google Forms, Facebook pages, simple websites) make it cheap and quick to start and test a business.
- Define a **prototype**, list its uses, and compare low-, mid- and high-fidelity prototypes.
- Define a **Minimum Viable Product (MVP)** and state at least five differences between a prototype and an MVP.
- Prepare an MVP plan using the MVP planning checklist, and test the MVP with number-based and comment-based feedback.
- Complete a small group project and use digital tools ethically, safely and fairly.

## 6.1 Entrepreneur and Entrepreneurship

### LEARN IT

An **entrepreneur** is a person who **identifies a problem, thinks of a solution, and tries to change that solution into a product or service.**

An entrepreneur does not always begin with a large company or a big office. Many begin with a small idea, limited resources, and a strong wish to solve a problem. Think of a young person in Karachi who starts by delivering home-cooked lunch boxes to a few offices, or a student in Sukkur who repairs mobile phones for neighbours. They started small.

Entrepreneurs usually show three habits:

- **They observe people and listen to their needs.** They notice what annoys people again and again.
- **They take calculated risks.** A *calculated risk* is a risk you take after thinking about what you could lose and how to keep that loss small. It is not gambling.
- **They improve their idea step by step.** They do not expect the first attempt to be perfect.

## What is entrepreneurship?

**Entrepreneurship** is the **process of planning, starting, testing, and improving** a business idea. It involves **creativity, problem-solving, teamwork, communication, and calculated risk.**

Notice the difference: the *entrepreneur* is the **person**; *entrepreneurship* is the **process** that person goes through.

#### REMEMBER IT

**Simple meaning.** An entrepreneur asks: "*What problem can I solve?*" Entrepreneurship is the process of testing and improving that solution until it becomes useful for people.

Remember the three-step journey:

1. Problem → 2. Solution → 3. Product

#### WORKED EXAMPLE

**Worked example: spotting the three steps.** Ayesha lives in Larkana. Her mother and other women in the street make beautiful hand-embroidered *rillis*, but they can only sell them to visitors.

**Step 1, Problem:** skilled women have products but very few buyers.

**Step 2, Solution:** show the rillis online so buyers from other cities can see them.

**Step 3, Product/Service:** a Facebook page with photos, prices and a "message to order" button.

Ayesha is the **entrepreneur**. Planning the page, posting photos, seeing which designs sell, and changing prices is **entrepreneurship**.

#### COMMON MISTAKE

**Mistake:** "Only rich people with big offices can be entrepreneurs."

**Fix:** The source clearly says many entrepreneurs begin with a small idea and limited resources. What matters is a real problem, a useful solution, and the willingness to improve it.

#### اردو میں سمجھیں

کاروباری شخص (Entrepreneur) وہ ہوتا ہے جو کسی مسئلے کو پہچانتا ہے، اس کا حل سوچتا ہے اور اس حل کو کسی پراڈکٹ یا سروس میں بدلنے کی کوشش کرتا ہے۔ کاروبار شروع کرنے، آزمانے اور بہتر بنانے کے پورے عمل کو انٹرپرائیورشپ (Entrepreneurship) کہتے ہیں۔ ایسا شخص لوگوں کی بات سنتا ہے، سوچ سمجھ کر خطرہ مول لیتا ہے اور اپنے خیال کو قدم بہ قدم بہتر بناتا ہے۔

#### CHECK YOURSELF

1. Is "entrepreneurship" a person or a process?
2. Name the three habits of an entrepreneur given in the source.
3. What is a calculated risk?

Answers: 1. A process (planning, starting, testing and improving a business idea). 2. Observing people and listening to their needs; taking calculated risks; improving the idea step by step. 3. A risk taken after thinking carefully about the possible loss and keeping that loss small.

## 6.2 Entrepreneurship in the Digital Age ★ Golden

#### LEARN IT

**Entrepreneurship in the digital age** means using **digital tools** to **start, test, improve, and grow** a business idea.

These tools may include **mobile phones, social media, websites, online forms, digital payments, maps, videos, and simple apps**. In Pakistan you already know many of them: WhatsApp, Facebook, Instagram, TikTok, YouTube, Google Forms, Google Maps, and digital payments such as JazzCash, Easypaisa or Raast.

## The power of digital tools

A digital entrepreneur **does not always need to create a complex mobile app**. Sometimes a **WhatsApp group**, a **Google Form**, a **Facebook page**, or a **simple website** is enough to test an idea.

### In the past

- Large investment needed.
- Offices and shops had to be rented first.
- Long planning before anyone saw the product.
- A failed idea meant a big loss.

### Today (digital age)

- A small idea can be tested **quickly**.
- Free tools: WhatsApp, Google Forms, Facebook.
- Feedback arrives in hours or days.
- If users do not like the idea, the team can **change it before spending too much money**.

#### WORKED EXAMPLE

##### Real-world relevance (from the source).

**Example 1:** A student selling **handmade cards through Instagram**. She posts photos of her Eid cards, people message her, and she delivers. No website, no app.

**Example 2:** A shopkeeper creating a **WhatsApp catalogue for Ajrak**. WhatsApp Business lets him list each Ajrak with a photo and price. Customers in Hyderabad or Karachi browse and order through chat.

Both are digital entrepreneurship **without writing a single line of code**.

##### Walk-through: how the Ajrak seller tests a new design cheaply.

1. He photographs 3 new Ajrak designs and adds them to the catalogue (cost: almost zero).
2. He sends the catalogue link to his customer broadcast list.
3. After one week he counts the orders per design.
4. He prints more of the design that sold most and stops the one nobody ordered.

He learned what customers want **before** printing a large stock.

#### REMEMBER IT

**Remember "S-T-I-G":** digital tools help you **Start**, **Test**, **Improve** and **Grow**. Think: "Start The Idea Growing."

#### COMMON MISTAKE

**Mistake:** "Digital entrepreneurship means building a mobile app."

**Fix:** An app is only one option. A WhatsApp group, Google Form, Facebook page or simple website is often enough, and much cheaper, to test whether people want your idea.

#### اردو میں سمجھیں

ڈیجیٹل دور میں انٹرپرائیورسپ کا مطلب ہے موبائل فون، سوشل میڈیا، ویب سائٹ، آن لائن فارم اور ڈیجیٹل ادائیگی جیسے ٹولز کے ذریعے کاروبار شروع کرنا، آزمانا اور بڑھانا۔ ہر بار پیچیدہ ایپ بنانا ضروری نہیں؛ وائس ایپ گروپ یا گوگل فارم بھی کافی ہو سکتا ہے۔ اگر لوگوں کو خیال پسند نہ آئے تو زیادہ پیسہ خرچ ہونے سے پہلے اسے بدلا جا سکتا ہے۔

#### EXAM TIP

Common questions: "Define entrepreneurship in the digital age", "How have digital tools changed entrepreneurship?" and "Give examples of digital entrepreneurship without coding." Write the definition (start, test, improve, grow using digital tools), list 4–5 tools, give the past-versus-today contrast (big investment versus quick, cheap testing), and quote the Instagram cards and the WhatsApp Ajrak catalogue examples.

### CHECK YOURSELF

1. List four digital tools that the source mentions.
2. Why is it easier to change a business idea today than in the past?
3. Give one example of digital entrepreneurship that needs no coding.

Answers: 1. Any four of: mobile phones, social media, websites, online forms, digital payments, maps, videos, simple apps. 2. Because a small idea can be tested quickly and cheaply, so if users do not like it the team can change it before spending too much money. 3. Selling handmade cards through Instagram, or a WhatsApp catalogue for Ajrak.

## Local Examples of Digital Entrepreneurship (Table 6.1)

### LEARN IT

Digital entrepreneurship can begin from **ordinary problems**. The examples below are close to the context of Sindh and Pakistan, where many students, parents, and small service providers already use mobile phones.

| Problem                                  | Digital Idea                                              | Possible First Users            | Urdu Context                                         |
|------------------------------------------|-----------------------------------------------------------|---------------------------------|------------------------------------------------------|
| Parents miss college notices.            | WhatsApp notice and reminder service.                     | Parents of first-year students. | والدین تک کالج کے نوٹس پہنچانے کے لیے واٹس ایپ سروس۔ |
| Small embroidery shop has few customers. | Simple catalogue with order form on Facebook or WhatsApp. | Customers in the same city.     | کڑھائی کی دکان کے لیے واٹس ایپ کیٹلاگ۔               |
| Students forget test dates.              | Google Calendar guide or simple reminder app.             | Grade XII students.             | ٹیسٹ کی تاریخیں یاد دلانے والی ایپ یا کیلنڈر۔        |
| Canteen has long queues during break.    | Pre-order form for snacks.                                | Students and teachers.          | کینٹین کے لیے پری آرڈر فارم۔                         |

Read each row from left to right: a **problem** → a **simple digital idea** → a **small group of first users**. You will see these same examples again later in the chapter (the canteen pre-order system becomes a full case study).

### WORKED EXAMPLE

**Worked example: adding your own row.** Follow the same pattern for a new local problem.

| Problem                                                                                            | Digital Idea                                                                                | Possible First Users     |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------|
| Students in a Mirpurkhas college lose water bottles, calculators and ID cards and never find them. | Lost-and-found Google Form plus a WhatsApp group where found items are posted with a photo. | Students of one college. |

Check: Is it a **real** problem? Yes, it happens every week. Is the idea **simple**? Yes, a form and a group. Are the first users **reachable**? Yes, they are in the same college.

#### COMMON MISTAKE

**Important:** A digital business idea should **not** be selected only because it sounds modern. It should solve a **real problem for real users**. If the problem is weak, the business idea will also be weak.

For example, "an AI app that chooses your shirt colour" sounds modern, but if nobody actually struggles with that, nobody will use it.

#### REMEMBER IT

Think "**Problem first, tool second.**" Start with the pain, not with the technology.

#### اردو میں سمجھیں

جدول 6.1 بتاتی ہے کہ ڈیجیٹل کاروبار عام مسائل سے شروع ہو سکتا ہے، جیسے والدین کا کالج نوٹس نہ دیکھ پانا، کڑھائی کی دکان پر کم گاہک، ٹیسٹ کی تاریخ بھول جانا یا کینٹین پر لمبی قطار۔ کوئی خیال صرف اس لیے نہ چنیں کہ وہ جدید لگتا ہے؛ اسے حقیقی لوگوں کا حقیقی مسئلہ حل کرنا چاہیے۔

#### CHECK YOURSELF

1. In Table 6.1, what digital idea is suggested for a small embroidery shop?
2. Who are the possible first users of the canteen pre-order form?
3. Why should an idea not be chosen only because it sounds modern?

Answers: 1. A simple catalogue with an order form on Facebook or WhatsApp. 2. Students and teachers. 3. Because it must solve a real problem for real users; a weak problem gives a weak business idea.

## 6.3 From Problem to Business Idea

#### LEARN IT

A good business idea usually starts from **observing daily life**. Look around your home, college, market, village, or city and ask these simple questions:

1. **What problem do people face?**
2. **Who faces this problem?**
3. **How often does it happen?**
4. **What do people currently do to solve it?**
5. **Can technology help in a simple way?**

A short way to remember them: **What? Who? How often? Current fix? Tech help?**

#### WORKED EXAMPLE

**Example scenario (from the source): study material before exams.**

Students cannot find study material before exams. The quick answer is "students are careless", but that is not the real issue.

**Apply the five questions:**

1. **What problem?** Students cannot find notes before exams.
2. **Who?** Students of a class or college.
3. **How often?** Before every test and exam.
4. **Current fix?** Notes are shared in many places: WhatsApp, email, photocopies, and class groups. Students search all of them.
5. **Tech help?** Yes: a simple **subject-wise notes organizer** where all files are placed in one location (for example, one shared Google Drive folder per subject).

The real problem is **scattered notes**, and the idea solves exactly that.

## Scope of the problem

Avoid choosing **very large problems** at the start, such as "improving the whole education system". A better problem is **smaller and clear**, such as helping Grade XII students track **practical file submission dates**. Smaller problems are **easier to test and improve**.

### Table 6.2: From problem to idea

The journey from observing a problem to creating an idea follows a structured path.

| Step               | What Students Do                                          | Example                                          |
|--------------------|-----------------------------------------------------------|--------------------------------------------------|
| Observe            | Notice repeated problems around home, college, or market. | Students forget practical file dates.            |
| Ask Users          | Talk to people who face the problem.                      | Ask five classmates how they remember deadlines. |
| Select One Problem | Choose one clear problem only.                            | Practical file reminder.                         |
| Suggest Solution   | Think of a simple digital or paper-based solution.        | Reminder form or app screen.                     |
| Test Early         | Show the idea to users before final development.          | Ask classmates if they will use it.              |

After this, the idea moves along the full path that the rest of this chapter follows:



#### REMEMBER IT

**Mnemonic for Table 6.2: "Only Ask Simple Solutions Today":** Observe, Ask users, Select one problem, Suggest solution, Test early.

#### COMMON MISTAKE

**Mistake 1:** Choosing a huge problem ("fix traffic in Karachi"). **Fix:** Narrow it ("help students of one college know when their van will arrive").

**Mistake 2:** Skipping "Ask Users" and guessing what people need. **Fix:** Talk to at least five real people who face the problem.

#### اردو میں سمجھیں

اچھا کاروباری خیال روزمرہ زندگی کے مشاہدے سے شروع ہوتا ہے۔ پوچھیں: مسئلہ کیا ہے، کس کو ہے، کتنی بار ہوتا ہے، لوگ ابھی کیا کرتے ہیں، اور کیا ٹیکنالوجی آسان طریقے سے مدد کر سکتی ہے؟ شروع میں چھوٹا اور واضح مسئلہ چنیں، کیونکہ اسے آزمانا اور بہتر بنانا آسان ہوتا ہے۔

#### CHECK YOURSELF

1. Write the five steps of Table 6.2 in order.
2. Why is "improving the whole education system" a poor first problem?
3. In the study-material example, what is the real problem?

Answers: 1. Observe, Ask Users, Select One Problem, Suggest Solution, Test Early. 2. It is too large and unclear; small, clear problems are easier to test and improve. 3. Notes are scattered in many places (WhatsApp, email, photocopies, class groups), not that students are careless.



## 6.4 Prototype ★ Golden

### LEARN IT

A **prototype** is an **early model** of a product, service, website, or app. It shows **how the idea may look or work**. It is **NOT the final product**. It is made to **test the idea and collect feedback**.

Think of the clay model an architect makes before building a house, or the sample *kameez* a tailor stitches before making 50 for a shop. The model lets people say "make this bigger" or "move that here" before real money is spent.

### Why is a prototype useful?

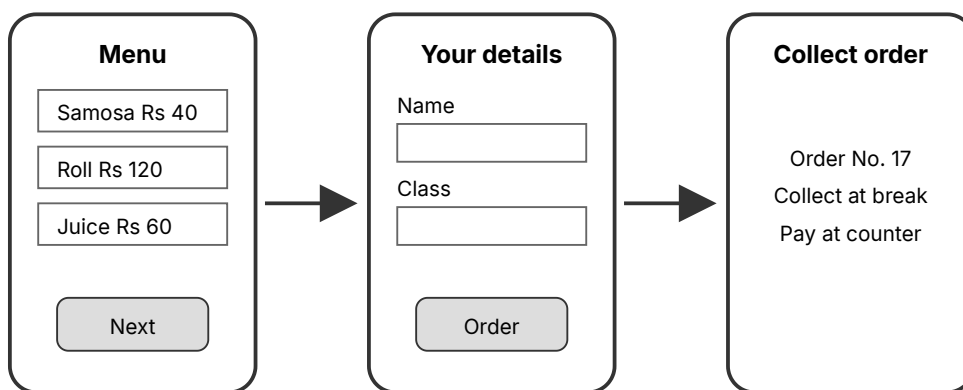
A prototype helps students and entrepreneurs **explain their idea clearly**, because users can **see something** instead of only listening to words.

- **It makes the idea visible.**
- **It helps the team find mistakes early.** Users can point out if a button is hard to find, if a screen has too much text, or if a feature is missing.
- **It saves time and money** before final coding begins. Fixing a pencil sketch takes a minute; fixing finished code can take days.

### What form can a prototype take?

A prototype may be a **paper sketch**, a **cardboard model**, a **clickable PowerPoint**, a **Canva design**, a **Figma wireframe**, a **simple website layout**, or a **sample service process chart**. A *wireframe* is a simple outline drawing of a screen that shows where things go (boxes, buttons, text) without colours or pictures.

For example, for a **canteen pre-order system**, students can just **draw the screens on paper**.



A paper prototype of a canteen pre-order system: three hand-drawn screens joined by arrows (select item → enter name → collect order).

### WORKED EXAMPLE

**Worked example: how a paper prototype finds a mistake early.** Bilal's team in a Nawabshah college draws the three screens above on A4 paper.

1. They hand screen 1 to a classmate and say, "Order a samosa."
2. The classmate looks for a quantity box and cannot find one. She asks, "What if I want two?"
3. The team notes: **feature missing: quantity**.
4. They add a small "Qty: [ ]" box with a pencil in 30 seconds.

If they had already coded an app, adding quantity would have meant changing the database, the screen and the order list. The prototype **saved time and money**.

#### REMEMBER IT

**Prototype = "practice model".** It is like a rough draft of an essay: you show it, get corrections, and rewrite it before the final copy.

#### COMMON MISTAKE

**Mistake:** Thinking a prototype must work or must look beautiful.

**Fix:** A prototype only has to make the idea **visible** so users can react. A pencil sketch is a valid prototype.

#### اردو میں سمجھیں

پروٹوٹائپ (Prototype) کسی پراڈکٹ، سروس، ویب سائٹ یا ایپ کا ابتدائی نمونہ ہوتا ہے جو دکھاتا ہے کہ خیال کیسا نظر آئے گا یا کیسے کام کرے گا۔ یہ حتمی پراڈکٹ نہیں ہوتا بلکہ خیال کو آزمانے اور رائے (Feedback) لینے کے لیے بنایا جاتا ہے۔ یہ کاغذ کا خاکہ، گتے کا ماڈل، پاورپوائنٹ یا فگما (Figma) ڈیزائن بھی ہو سکتا ہے، اور اس سے غلطیاں جلد پکڑی جاتی ہیں اور وقت اور پیسہ بچتا ہے۔

#### EXAM TIP

Expect "Define prototype and write its benefits" (3–5 marks). Write: (1) definition with the words *early model, not the final product, test the idea and collect feedback*; (2) three benefits: makes the idea visible, finds mistakes early, saves time and money; (3) forms it can take (paper sketch, cardboard model, clickable PowerPoint, Canva, Figma wireframe); (4) the canteen paper-screens example.

#### CHECK YOURSELF

1. Is a prototype the final product?
2. Name four forms a prototype can take.
3. Give two kinds of problems users can point out by looking at a prototype.

Answers: 1. No. It is an early model made to test the idea and collect feedback. 2. Any four: paper sketch, cardboard model, clickable PowerPoint, Canva design, Figma wireframe, simple website layout, sample service process chart. 3. Any two: a button is hard to find, a screen has too much text, a feature is missing.

## 6.5 Types of Prototypes

#### LEARN IT

Prototypes are commonly divided into **three types** according to their level of detail, called **fidelity**. **Fidelity means how close the prototype is to the final product.** Low fidelity = far from final; high fidelity = very close to final.

### 6.5.1 Low-Fidelity Prototype

A **quick and rough** model made at the **very beginning**. It is **drawn on paper**. It does **not focus on colours or fonts**. It shows the **main idea and flow** (which screen comes after which). It is **very suitable for students** because it is **cheap, fast, and easy to change**.

### 6.5.2 Mid-Fidelity Prototype

**Clearer than a paper sketch.** It is created in **PowerPoint, Canva, or Figma**. It shows screens and **navigation** in a more organised way, but **without final graphics**. It is useful to **test screen-to-screen navigation**, for example by linking slides so that clicking "Next" opens the next screen.

### 6.5.3 High-Fidelity Prototype

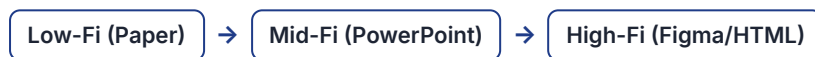
Looks **very close to the final product**. It includes **realistic colours, buttons, sample data, and working features**. It **takes more time** and should **only be created after the basic idea is tested**.

#### Comparison of prototype types

For most school and college projects, you should **begin with a low-fidelity prototype**. A common mistake is to start building a full app before knowing whether users need it. A rough prototype protects you from this mistake.

**Table 6.3: Types of prototypes**

| Type          | Meaning                                      | Best Use                      | Example                                       |
|---------------|----------------------------------------------|-------------------------------|-----------------------------------------------|
| Low-Fidelity  | A quick and rough model of the idea.         | Very early testing.           | Hand-drawn app screens for homework reminder. |
| Mid-Fidelity  | A clearer model that shows layout and flow.  | Testing navigation.           | Clickable screens in PowerPoint or Figma.     |
| High-Fidelity | A detailed model close to the final product. | Testing realistic experience. | Working demo of a small order form website.   |



Extra comparison to help you revise:

| Feature           | Low-Fidelity                 | Mid-Fidelity             | High-Fidelity                       |
|-------------------|------------------------------|--------------------------|-------------------------------------|
| Tool              | Paper and pencil             | PowerPoint, Canva, Figma | Figma (detailed), HTML              |
| Colours and fonts | No                           | Not final                | Realistic                           |
| Clickable?        | No (you move papers by hand) | Yes, basic links         | Yes, with working features          |
| Cost and time     | Lowest, fastest              | Medium                   | Highest, slowest                    |
| Easy to change?   | Very easy                    | Fairly easy              | Harder                              |
| When              | Very beginning               | After the flow is agreed | Only after the basic idea is tested |

#### WORKED EXAMPLE

**Worked example: one idea, three fidelities.** Idea: a Homework Reminder for a college in Jacobabad.

1. **Low-fi:** Sana draws three screens on paper in 15 minutes: "Today", "Add homework", "Reminder". Classmates point at the drawings to "use" them.
2. **Mid-fi:** After feedback, she makes three PowerPoint slides with grey boxes and links them with hyperlinks, so tapping "Add" opens the Add slide. Now she tests whether people can move between screens easily.
3. **High-fi:** Once users clearly want it, she builds a small HTML page with real colours, a working form and sample homework data.

Each step costs more, so she only moves up when the previous step proves the idea is worth it.

#### REMEMBER IT

**Fidelity = "faithfulness" to the final product.** Remember **Paper → PowerPoint → Polished** (low → mid → high).

### COMMON MISTAKE

**Mistake:** Starting with a high-fidelity prototype or a full app because it "looks professional".

**Fix:** Begin with low fidelity. If users do not need the idea, you have lost only paper and an hour, not weeks of work.

### اردو میں سمجھیں

فڈیلیٹی (Fidelity) کا مطلب ہے کہ پروٹوٹائپ حتمی پراڈکٹ سے کتنا ملتا جلتا ہے۔ لو فڈیلیٹی کاغذ پر بنایا گیا سستا اور جلدی بننے والا خاکہ ہے؛ مڈ فڈیلیٹی پاورپوائنٹ، کینوا یا فگما میں بنتا ہے اور اسکرینوں کے درمیان آنا جانا (Navigation) دکھاتا ہے؛ ہائی فڈیلیٹی بالکل اصل پراڈکٹ جیسا لگتا ہے اور اسے بنیادی خیال آزمانے کے بعد ہی بنانا چاہیے۔

### CHECK YOURSELF

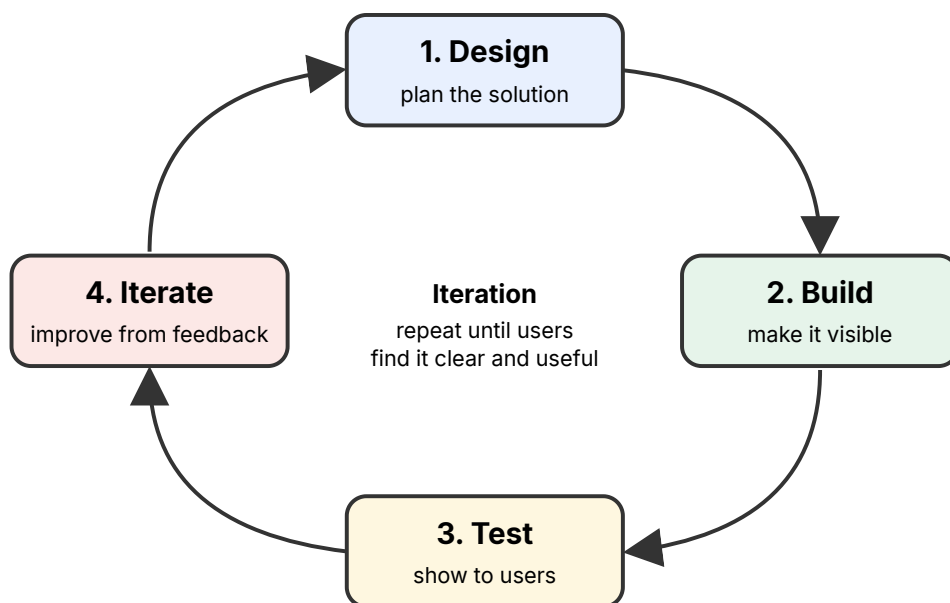
1. What does "fidelity" mean?
2. Which type of prototype is best for testing navigation?
3. Why should students begin with low fidelity?

Answers: 1. How close the prototype is to the final product. 2. Mid-fidelity (e.g. clickable screens in PowerPoint or Figma). 3. It is cheap, fast and easy to change, and it protects them from building a full app before knowing whether users need it.

## 6.6 Prototype Development Cycle ★ Golden

### LEARN IT

Prototype development is **not a one-time activity**. A prototype is **created, tested, improved, and tested again**. This repeating process is called **iteration**. The main stages are **Design, Build, Test, and Iterate**.



The prototype development cycle: Design → Build → Test → Iterate, then back to Design.

| Stage      | What happens                                                                                                      |
|------------|-------------------------------------------------------------------------------------------------------------------|
| 1. Design  | Understand the problem and plan the solution. Decide what the prototype should show (e.g. screen flow, features). |
| 2. Build   | Create the first version (physical or digital). The aim is not beauty; the aim is to make the idea visible.       |
| 3. Test    | Show the prototype to users. Observe what they understand and what they find difficult.                           |
| 4. Iterate | Improve the prototype based on user feedback. Remove unnecessary parts and add missing features.                  |

#### WORKED EXAMPLE

**Worked example: two rounds of the cycle.** A team in Thatta wants a WhatsApp catalogue for a local Ajrak shop.

##### Round 1

- **Design:** The problem is that customers do not know which designs and prices are available. Plan: one page per design with photo, price and an "Order" message.
- **Build:** They make a paper booklet with 6 hand-drawn "catalogue pages".
- **Test:** They show it to 5 customers. Three ask, "Is this single or double-sided Ajrak? What size?"
- **Iterate:** They add size and type to every page.

##### Round 2

- **Design:** Add the size and type fields, and a delivery note.
- **Build:** They make the catalogue in WhatsApp Business with real photos.
- **Test:** 5 new customers browse it. Now nobody asks about size; one says the delivery charge is unclear.
- **Iterate:** They add "Delivery in Thatta: Rs 100" to each item.

Each round produces a clearer version, because each change comes from real feedback.

#### REMEMBER IT

**Mnemonic: "Do Big Things Iteratively"** = Design, Build, Test, Iterate. And remember the arrow at the end goes **back to Design**: it is a **cycle**, not a line.

#### COMMON MISTAKE

**Mistake 1:** Drawing the stages as a straight line that ends at "Iterate". **Fix:** Always draw a loop with an arrow from Iterate back to Design.

**Mistake 2:** Making the Build stage beautiful. **Fix:** The source says "The aim is not beauty; the aim is to make the idea visible."

#### اردو میں سمجھیں

پروٹوٹائپ ایک ہی بار میں نہیں بنتا؛ اسے بنایا، آزمایا، بہتر کیا اور دوبارہ آزمایا جاتا ہے۔ اس بار بار دہرانے کے عمل کو **ٹریسٹیشن** (Iteration) کہتے ہیں۔ چار مراحل ہیں: ڈیزائن (مسئلہ سمجھ کر حل کی منصوبہ بندی)، **بیلڈ** (پہلا نمونہ بنانا)، ٹیسٹ (صارفین کو دکھانا) اور **ایٹریٹ** (رائے کی بنیاد پر بہتری)۔

### EXAM TIP

A favourite long question: "Explain the prototype development cycle with a diagram." Draw the four boxes in a **circle** with arrows, write one or two lines for each stage using the source's words, define **iteration**, and finish with a short example (e.g. the ClassBuddy activity in 6.7). Marks are often lost for drawing a straight line instead of a cycle.

### CHECK YOURSELF

1. What is iteration?
2. At which stage do you observe what users find difficult?
3. What is the aim of the Build stage?

Answers: 1. The repeating process of creating, testing, improving and testing a prototype again. 2. The Test stage. 3. To make the idea visible (not to make it beautiful).

## 6.7 Class Activity: Create a Prototype

### LEARN IT

**Business idea scenario.** Students often forget **homework deadlines, test dates, and required class items**. A group of students wants to design a simple "**Homework Helper**" or "**ClassBuddy**" reminder system.

The team will **not** immediately build a complete mobile app. They will first **understand the problem, draw a low-fidelity prototype, test it with classmates, and improve it**. This activity is the development cycle of 6.6 in action.

1. Understand → 2. Draw low-fi → 3. Test → 4. Analyse feedback → 5. Improve

### Step 1: Understand the problem

The team **interviews ten students**. They find that **7 out of 10** students (that is 70%) forget at least one task in a week. Students want **reminders before leaving home**, and some prefer a **very simple design**.

**Table 6.4: Understanding user needs**

| Finding                               | Meaning                     | Design Decision                     |
|---------------------------------------|-----------------------------|-------------------------------------|
| 7 out of 10 forget tasks.             | The problem is common.      | App should show a daily task list.  |
| Reminders needed before leaving home. | Reminder time is important. | Add morning notification option.    |
| Students want simple apps.            | Design must be simple.      | Use large buttons and clear labels. |

### Step 2: Draw a low-fidelity prototype

The team draws **three screens on paper**: (1) **Today's classes**, (2) **Add homework**, (3) **Show reminder**. **Arrows** show how a user moves from one screen to another.

### Step 3: Test the prototype

The team gives the paper screens to **five classmates**. The **observer watches silently** to see if users can understand the design by themselves. Users are given one task: "**add one homework task**".

## Step 4: Analyse feedback

The team looks for **repeated problems** in the feedback to identify what actually needs fixing.

**Table 6.5: Changing feedback into improvement**

| User Feedback                           | Problem Identified             | Improvement                                    |
|-----------------------------------------|--------------------------------|------------------------------------------------|
| "I cannot quickly find Add Homework."   | Button is not visible.         | Make Add Homework button larger and centre it. |
| "I want reminders by subject."          | Reminder is too general.       | Add subject name and due date fields.          |
| "I want to see all tasks for the week." | Only daily view is not enough. | Add a 'Weekly View' button.                    |

## Step 5: Improve the prototype

The team **redraws the home screen** with a **larger Add Homework button**, a **subject list**, and a **weekly view**. This second version is clearer because it is based on real user feedback.

**A prototype has not failed when users give negative feedback; it only fails if the team ignores useful feedback.**

### WORKED EXAMPLE

**Worked example: link the activity to the cycle.**

| Cycle stage | ClassBuddy activity step                                                                 |
|-------------|------------------------------------------------------------------------------------------|
| Design      | Step 1: interview 10 students, make design decisions (Table 6.4).                        |
| Build       | Step 2: draw three paper screens with arrows.                                            |
| Test        | Step 3: five classmates try "add one homework task" while the observer watches silently. |
| Iterate     | Steps 4 and 5: analyse feedback (Table 6.5) and redraw the home screen.                  |

**Why watch silently?** If the observer explains ("tap here"), the team never learns that the button is hard to find. Silence shows the real difficulty.

### REMEMBER IT

**Feedback → Problem → Improvement.** Every comment should be turned into a problem statement and then one clear change, exactly like the three columns of Table 6.5.

### COMMON MISTAKE

**Mistake:** Feeling that negative feedback means the prototype "failed", or changing the design for every single comment.

**Fix:** Negative feedback is useful information. Look for **repeated** problems and fix those first. The prototype only fails if you ignore useful feedback.

### اردو میں سمجھیں

اس سرگرمی میں طلبہ "کلاس بڈی" (ClassBuddy) نامی یاد دہانی نظام بناتے ہیں۔ پہلے دس طلبہ سے بات کر کے مسئلہ سمجھتے ہیں، پھر کاغذ پر تین اسکرینیں بناتے ہیں، پانچ ساتھیوں سے خاموشی سے آزمواتے ہیں، رائے کا تجزیہ کرتے ہیں اور ڈیزائن بہتر کرتے ہیں۔ منفی رائے سے پروٹوٹائپ ناکام نہیں ہوتا؛ ناکامی تب ہے جب مفید رائے کو نظرانداز کیا جائے۔

### CHECK YOURSELF

1. In Step 1, how many students forgot at least one task in a week, and what percentage is that?
2. Name the three paper screens drawn in Step 2.
3. What improvement was made for the feedback "I want to see all tasks for the week"?

Answers: 1. 7 out of 10, which is 70%. 2. Today's classes, Add homework, Show reminder. 3. A 'Weekly View' button was added.

## 6.8 Minimum Viable Product (MVP) ★ Golden

### LEARN IT

A **Minimum Viable Product (MVP)** is the **simplest working version** of a product. It includes **only the most important features** needed to **test the idea with real users**.

- **Minimum:** it has only the **essential** features. No extras.
- **Viable:** it is **usable enough** to actually solve the main problem. ("Viable" means able to work successfully.)

An MVP is **not the final product**. It is also **not only a drawing**. It is a **small working version** used to test whether users actually need the solution.

### WORKED EXAMPLE

Example from the source: canteen app.

| Complete canteen app (final product)                         | Canteen MVP                                                            |
|--------------------------------------------------------------|------------------------------------------------------------------------|
| Login, online payment, delivery tracking, discount offers... | A simple <b>Google Form</b> where students select snacks before break. |

**Step-by-step: how to cut a big idea down to an MVP.**

1. List all features you dream of: login, payment, delivery tracking, discounts.
2. Ask of each one: "Without this, can a student still pre-order a snack?"
3. Login? Not needed; the student can type name and class. Payment? Not needed; pay at the counter. Delivery tracking? Not needed; students collect. Discounts? Not needed.
4. What is left is the core: **choose snack + name + class**. A Google Form does this for free.

That form is **minimum** (no extras) and **viable** (students really can pre-order with it).

### REMEMBER IT

**Simple meaning.** An MVP answers: "**Will people actually use this solution?**" It tests **real use and behaviour**, not only the visual design.

Analogy: before opening a full restaurant, a cook in Hyderabad sells biryani from home to office workers for two weeks. If people keep ordering, the idea is worth growing. That home kitchen is the MVP.

### COMMON MISTAKE

**Mistake 1:** Calling a paper sketch an MVP. **Fix:** A sketch is a prototype. An MVP must **work** and be used by **real users**.

**Mistake 2:** Thinking MVP means a low-quality or broken product. **Fix:** It is small but **viable**; it must genuinely solve the main problem.



ایم وی پی (Minimum Viable Product) کسی پراڈکٹ کا سب سے سادہ مگر کام کرنے والا ورژن ہے جس میں صرف ضروری خصوصیات ہوتی ہیں۔ "منیمم" کا مطلب ہے صرف ضروری چیزیں، اور "وائیبل" کا مطلب ہے کہ وہ اصل مسئلہ حل کرنے کے قابل ہو۔ یہ حتمی پراڈکٹ نہیں بلکہ یہ جانچنے کا ذریعہ ہے کہ کیا لوگ واقعی اسے استعمال کریں گے، جیسے کینٹین کے لیے ایک سادہ گوگل فارم۔

**EXAM TIP**

Typical questions: "What is an MVP? Explain the words Minimum and Viable" and "Give an example of an MVP." Write the full form, the definition (simplest working version, only most important features, test with real users), explain both words, state that it is neither the final product nor only a drawing, and give the canteen Google Form example. Mention the key question: "Will people actually use this solution?"

**CHECK YOURSELF**

1. Write the full form of MVP.
2. What does "Viable" mean in MVP?
3. What question does an MVP answer?

Answers: 1. Minimum Viable Product. 2. Usable enough to actually solve the main problem. 3. "Will people actually use this solution?"

## 6.9 Difference between Prototype and MVP

**LEARN IT**

Prototype and MVP are **related, but not the same**. A **prototype** is mainly used to **show and test the design** of an idea. An **MVP** is used to **test real use and market demand**. *Market demand* means whether enough people want the product and will use (or pay for) it.

Usually a prototype comes first and an MVP comes after it: Problem → Idea → Prototype → Feedback → MVP.

**Table 6.6: Prototype and MVP comparison**

| Point          | Prototype                                 | MVP                                                      |
|----------------|-------------------------------------------|----------------------------------------------------------|
| Main purpose   | To show how the idea may look or work.    | To test whether users will actually use the solution.    |
| Working status | May not work; can be a sketch or model.   | Must work at least in a basic way.                       |
| Users          | Small group gives feedback on design.     | Early real users test the product in daily life.         |
| Cost           | Very low cost.                            | Low cost but may need some basic development.            |
| Example        | Paper screens for a canteen order system. | Google Form used for real canteen orders for three days. |

#### REMEMBER IT

##### Key takeaway

**Prototype** = "Does it **look right** and make sense?"

**MVP** = "Will people **actually use it** and find value in it?"

Memory hook: **Prototype** → **P**icture of the idea; **MVP** → **M**arket test of the idea.

#### WORKED EXAMPLE

**Worked example: classify each item.** Decide whether each is a prototype or an MVP and give the reason.

1. Hand-drawn screens of a tuition-finder app shown to 6 friends. → **Prototype**: it does not work; it tests design.
2. A WhatsApp broadcast group that really sends van arrival times to parents of one route for a week. → **MVP**: it works and real users use it daily.
3. Clickable PowerPoint screens of a notes organizer. → **Prototype** (mid-fidelity): links work, but nobody uses it for real notes.
4. A shared Google Drive folder per subject that a class actually uses before the monthly test. → **MVP**: real use in daily life.

Rule used: **Does it work, and is it used by real users in real life?** If yes → MVP. If it only shows the design → prototype.

#### COMMON MISTAKE

**Mistake:** Thinking "high-fidelity prototype" and "MVP" are the same because both look good.

**Fix:** The difference is **purpose and real use**, not appearance. A beautiful Figma design that nobody uses for real orders is still a prototype; a plain Google Form taking real orders is an MVP.

#### اردو میں سمجھیں

پروٹوٹائپ اور ایم وی پی آپس میں جڑے ہیں مگر ایک جیسے نہیں۔ پروٹوٹائپ خیال کا ڈیزائن دکھاتا اور آزماتا ہے اور ضروری نہیں کہ کام کرے، جبکہ ایم وی پی کم از کم بنیادی طور پر کام کرتا ہے اور اصل صارفین روزمرہ زندگی میں اسے استعمال کرتے ہیں۔ پروٹوٹائپ پوچھتا ہے "کیا یہ ٹھیک لگتا ہے؟" اور ایم وی پی پوچھتا ہے "کیا لوگ واقعی اسے استعمال کریں گے؟"

#### CHECK YOURSELF

1. Which one must work at least in a basic way?
2. Give the source's example of a prototype and of an MVP for the canteen.
3. Which question does a prototype answer?

Answers: 1. The MVP. 2. Prototype: paper screens for a canteen order system. MVP: a Google Form used for real canteen orders for three days. 3. "Does it look right and make sense?"

## 6.10 Identifying the Riskiest Assumption

#### LEARN IT

Every business idea is based on **assumptions**. An **assumption** is something we **believe to be true, but have not tested yet**.

The **riskiest assumption** is the belief that can make the **entire business fail** if it turns out to be wrong.

Entrepreneurs should **identify and test the riskiest assumption early**, because it **saves time and money**. If it is wrong, you find out before building everything else.

**Table 6.7: Testing risky assumptions**

| Business Idea             | Possible Risky Assumption                          | How to Test It (MVP)                                            |
|---------------------------|----------------------------------------------------|-----------------------------------------------------------------|
| Online tuition finder     | Students will pay for online tutoring.             | Ask 20 students and offer a trial class to see if they sign up. |
| Canteen pre-order form    | Students will order snacks before break.           | Test a Google Form for one week in real time.                   |
| Ajrak online catalogue    | Customers will place orders through WhatsApp.      | Share catalogue with 30 customers and record actual orders.     |
| College van update system | Parents and students will check van updates daily. | Create a trial WhatsApp broadcast group for one route.          |

Notice that every test measures **what people actually do** (sign up, order, check), not just what they say.

#### WORKED EXAMPLE

**Worked example: finding the riskiest assumption.** Idea: a home-tutor booking service for Grade IX–X students in Sukkur.

#### Step 1, list assumptions:

1. Parents want home tutors for their children.
2. Good tutors in Sukkur are willing to join.
3. Parents will **pay a booking fee** through our service instead of finding tutors through relatives.
4. Parents will like our logo.

**Step 2, ask of each: "If this is wrong, does the whole business fail?"** Logo (4): no, easily changed. Parents wanting tutors (1): very likely true; many already hire tutors. Tutors joining (2): important, but easy to check by phoning a few tutors. Paying a booking fee (3): if parents refuse to pay, the business earns nothing, so **this is the riskiest assumption**.

**Step 3, design a small MVP test:** post the offer in 3 parent WhatsApp groups for one week, offer a first booking at a small fee, and **count how many actually pay**.

#### REMEMBER IT

**Ask: "What would kill this idea?"** The answer is usually the riskiest assumption. Test it **first**, cheaply, with real behaviour.

#### COMMON MISTAKE

**Mistake:** Testing easy, safe assumptions (colours, names) first and leaving the dangerous one for later.

**Fix:** Test the assumption that could make the whole business fail first. Also, do not trust only "Yes, I would use it". Measure actual sign-ups or orders.

#### اردو میں سمجھیں

ہر کاروباری خیال کچھ مفروضوں (Assumptions) پر قائم ہوتا ہے، یعنی وہ باتیں جنہیں ہم سچ سمجھتے ہیں مگر ابھی آزمایا نہیں۔ سب سے خطرناک مفروضہ (Riskiest Assumption) وہ ہے جو غلط نکلے تو پورا کاروبار ناکام ہو جائے۔ اسے شروع میں ہی چھوٹے ایم وی پی سے آزمانا چاہیے تاکہ وقت اور پیسہ بچ سکے۔

#### CHECK YOURSELF

1. Define assumption.
2. What is the risky assumption for the Ajrak online catalogue, and how is it tested?
3. Why test the riskiest assumption early?

Answers: 1. Something we believe to be true but have not tested yet. 2. Customers will place orders through WhatsApp; share the catalogue with 30 customers and record actual orders. 3. Because if it is wrong the whole business fails, so testing it early saves time and money.

## 6.11 Developing an MVP

#### LEARN IT

To develop an MVP, the team must **select only the essential features**. Extra features should be avoided at the beginning. A common mistake is **trying to build everything at once**. This makes the product **difficult, costly, and slow**.

An **MVP plan** should clearly explain: the **problem, target users, main feature, basic design, estimated cost, marketing method, feedback method, and improvement plan**.

**Table 6.8: MVP planning checklist**

| MVP Element  | Question to Ask                         | Example Answer                     |
|--------------|-----------------------------------------|------------------------------------|
| Problem      | What problem are we solving?            | Students wait too long at canteen. |
| Target users | Who will use it first?                  | Grade XII students in one college. |
| Main feature | What is the ONE feature that solves it? | Pre-order snacks before break.     |
| Basic design | What steps are needed?                  | Menu, name, class, order time.     |
| Cost         | What free/low-cost tools to use?        | Google Form or printed slips.      |
| Marketing    | How will early users hear about it?     | WhatsApp message to class group.   |
| Feedback     | How will users respond?                 | Short feedback form after 3 days.  |

The eighth element, the **improvement plan**, answers "What will we change after feedback?" (you will see it in the case study in 6.12).

#### WORKED EXAMPLE

**Worked example: filling the checklist for a new idea.** Idea: a college notice reminder for parents in Hyderabad (from Table 6.1).

| MVP Element      | Answer                                                                        |
|------------------|-------------------------------------------------------------------------------|
| Problem          | Parents miss college notices (fee dates, PTMs, holidays).                     |
| Target users     | Parents of first-year (Class XI) students of one college.                     |
| Main feature     | Send each notice to parents' phones on the same day.                          |
| Basic design     | Photo of notice + 2-line summary in Urdu and Sindhi + date.                   |
| Cost             | Free: WhatsApp broadcast list from the college office phone.                  |
| Marketing        | Students give a printed slip to parents with the number to save.              |
| Feedback         | After 2 weeks, a 3-question voice-note or phone survey of 20 parents.         |
| Improvement plan | If parents say notices come too late, send a reminder 1 day before each date. |

Notice: only **one** main feature, a **free** tool, and a clear way to collect feedback.

#### REMEMBER IT

**Mnemonic for the 8 elements: "People Try My Bakery; Cheap Muffins Find Improvement":** Problem, Target users, Main feature, Basic design, Cost, Marketing, Feedback, Improvement plan.

#### COMMON MISTAKE

**Mistake:** Writing a list of ten "main features".

**Fix:** The checklist asks for the **ONE** feature that solves the problem. Everything else waits until the MVP proves people want it.

#### اردو میں سمجھیں

ایم وی پی بناتے وقت صرف ضروری خصوصیات چنیں؛ سب کچھ ایک ساتھ بنانے سے کام مشکل، مہنگا اور سست ہو جاتا ہے۔ ایم وی پی منصوبے میں مسئلہ، پہلے صارفین، ایک بنیادی خصوصیت، سادہ ڈیزائن، کم خرچ ٹولز، تشہیر کا طریقہ، رائے لینے کا طریقہ اور بہتری کا منصوبہ واضح ہونا چاہیے۔

#### CHECK YOURSELF

1. Why is building everything at once a mistake?
2. List the eight elements of an MVP plan.
3. In Table 6.8, what is the marketing method?

Answers: 1. It makes the product difficult, costly and slow. 2. Problem, target users, main feature, basic design, estimated cost, marketing method, feedback method, improvement plan. 3. A WhatsApp message to the class group.

## 6.12 Case Study: MVP for a Canteen Pre-Order System

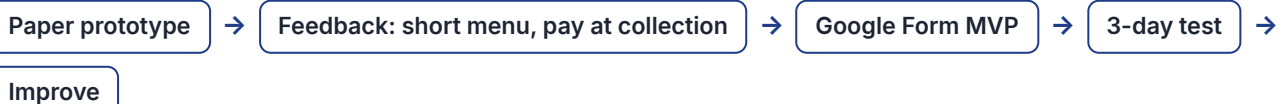
#### LEARN IT

**Problem:** In a college, students wait in long lines at the canteen. Sometimes snacks finish before students reach the counter. A group wants to test a canteen pre-order idea.

**Goal:** Their aim is **NOT** to create a complete app immediately. Their aim is to test **whether students will place orders before break** and **whether the canteen can manage these orders**.

### The approach

1. The team creates a **paper prototype** showing three steps: **select item, enter name, collect order**.
2. **Feedback** from the prototype suggests the **menu should be short** and **payment should be at collection time**.
3. For the actual MVP, the team uses a **Google Form**. Students select **one of five items**.
4. The canteen prepares these pre-orders. This MVP is tested for **three days only** to learn quickly.



**Table 6.9: MVP plan for canteen pre-order system**

| MVP Element     | Decision                                                 |
|-----------------|----------------------------------------------------------|
| Features        | Menu list, student name, class, item, and quantity.      |
| Cost & Tools    | Free Google Form or printed order slips.                 |
| Marketing       | Class announcement and WhatsApp group message.           |
| Testing period  | Three days.                                              |
| Feedback method | Ask students if they saved time and would use it again.  |
| Improvement     | Keep only 5 items; close orders 20 minutes before break. |

**Result:** Students liked saving time, but **canteen staff found too many items difficult**. The team improved the MVP by **keeping only five items** and creating a **separate collection line**. This improvement is based on real feedback.

#### WORKED EXAMPLE

**Worked example: trace the case study through the chapter's ideas.**

| Chapter idea                 | Where it appears in the case study                                       |
|------------------------------|--------------------------------------------------------------------------|
| Real problem (6.3)           | Long queues; snacks finish before students reach the counter.            |
| Low-fidelity prototype (6.5) | Paper screens: select item, enter name, collect order.                   |
| Iteration (6.6)              | Feedback led to a short menu and payment at collection.                  |
| MVP (6.8)                    | Google Form with one of five items, used for real orders.                |
| Riskiest assumption (6.10)   | Students will order before break, and the canteen can manage the orders. |
| Beachhead market (6.14)      | One college canteen during break.                                        |

**Why "close orders 20 minutes before break"?** Staff need time to prepare. If orders arrive at the last minute, the pre-order line becomes as slow as the normal line, and the main benefit disappears.

#### REMEMBER IT

Remember the case study as **"5 items, 3 days, 20 minutes"**: five menu items, a three-day test, orders closing twenty minutes before break.

#### COMMON MISTAKE

**Mistake:** Thinking only the students' opinion matters.

**Fix:** The case study tested **two** things: whether students will order **and** whether the canteen can manage the orders. The staff's feedback (too many items) caused the key improvement.

#### اردو میں سمجھیں

ایک کالج میں طلبہ کینٹین کی لمبی قطاروں میں انتظار کرتے تھے۔ ٹیم نے فوراً ایپ بنانے کے بجائے پہلے کاغذی پروٹوٹائپ بنایا، پھر تین دن کے لیے گوگل فارم کو ایم وی پی کے طور پر آزمایا جس میں پانچ اشیاء تھیں۔ طلبہ کو وقت کی بچت پسند آئی مگر عملے کو زیادہ اشیاء مشکل لگیں، اس لیے صرف پانچ اشیاء رکھی گئیں، الگ قطار بنائی گئی اور بریک سے بیس منٹ پہلے آرڈر بند کیے گئے۔

### CHECK YOURSELF

1. Which tool was used for the canteen MVP, and for how long was it tested?
2. What two things did the team want to test?
3. What two improvements were made after the result?

Answers: 1. A Google Form, tested for three days. 2. Whether students will place orders before break, and whether the canteen can manage these orders. 3. Keeping only five items and creating a separate collection line (orders also close 20 minutes before break).

## 6.13 Testing an MVP

### LEARN IT

**MVP testing** means giving the basic working version to **real users** and **studying their response**. The team should collect **both numbers and comments**.

- **Numbers** (quantitative data) show how many users tried the MVP and how many may use it again.
- **Comments** (qualitative data) explain what users liked, disliked, or found confusing.

Testing should be **planned before the MVP is launched**. Decide:

1. **How many users** will test it.
2. **How long** the test will continue.
3. **What information** will be collected.

**Table 6.10: Feedback during MVP testing**

| Type of Feedback | Question                         | Example Result                   |
|------------------|----------------------------------|----------------------------------|
| Number-based     | How many users tried it?         | 40 students used the form.       |
| Number-based     | How many would use it again?     | 30 out of 40 said yes.           |
| Comment-based    | What was confusing?              | Did not know order closing time. |
| Observation      | Where did users face difficulty? | Forgot to enter class name.      |

**Observation** means the team watches users and notes difficulties themselves, even ones users do not mention.

### WORKED EXAMPLE

**Worked example: studying the results in Table 6.10.**

1. **Repeat-use rate** = (users who would use it again ÷ users who tried it) × 100 = (30 ÷ 40) × 100 = **75%**.
2. **Did not want to use again** = 40 – 30 = **10 students** = (10 ÷ 40) × 100 = **25%**.
3. **Comment**: "Did not know order closing time" → **Fix**: write "Orders close 20 minutes before break" at the top of the form and in the WhatsApp message.
4. **Observation**: "Forgot to enter class name" → **Fix**: make the Class field **required** in the Google Form so it cannot be submitted empty.
5. **Decide what to fix first**: the closing-time problem stops people from ordering at all, so fix it first; the class-name problem only makes collection slower.

Three out of four users would come back, which is a promising sign; fixing the two problems could raise that number further.

#### REMEMBER IT

**Data skill link.** Feedback is useful only when it is studied: **Count** the responses → **Look** for repeated problems → **Decide** which improvement comes first. Remember "**C-L-D**: Count, Look, Decide."

#### COMMON MISTAKE

**Mistake:** Launching the MVP first and deciding later what to measure.

**Fix:** Plan the number of users, the test length and the questions **before** launch. Otherwise you may forget to record important data (for example, how many people opened the form but did not submit).

#### اردو میں سمجھیں

ایم وی پی کی جانچ کا مطلب ہے کہ بنیادی کام کرنے والا ورژن اصل صارفین کو دیا جائے اور ان کے ردعمل کا مطالعہ کیا جائے۔ اعداد بتاتے ہیں کتنے لوگوں نے استعمال کیا اور کتنے دوبارہ کریں گے، جبکہ تبصرے بتاتے ہیں کیا پسند آیا اور کیا الجھن تھی۔ جانچ کی منصوبہ بندی پہلے سے کریں، جوابات گتیں، بار بار آنے والے مسائل تلاش کریں اور پہلے سب سے اہم بہتری کریں۔

#### CHECK YOURSELF

1. Name the three types of feedback in Table 6.10.
2. If 50 people tried an MVP and 35 would use it again, what is the repeat-use rate?
3. Name the three things to decide before launching an MVP test.

Answers: 1. Number-based, comment-based, observation. 2.  $(35 \div 50) \times 100 = 70\%$ . 3. How many users will test it, how long the test will continue, and what information will be collected.

## 6.14 Beachhead Market ★ Golden

#### LEARN IT

A **beachhead market** is a **small and focused group of first users**. These users have a **clear need** and are **likely to try the product early**.

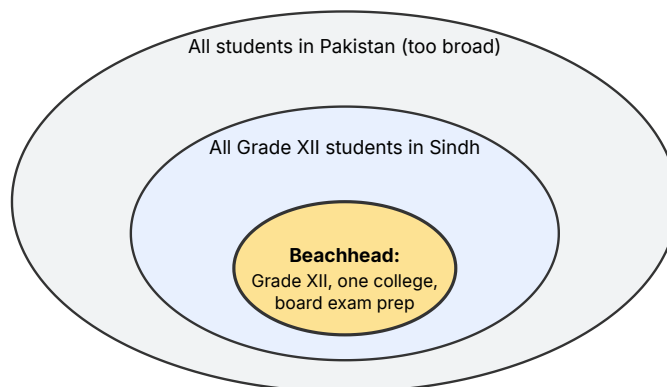
Instead of trying to serve everyone, entrepreneurs **first focus on one small group**. This helps them **test the MVP properly and improve it before expanding**.

The word comes from the army: a *beachhead* is the small piece of beach soldiers capture first when they land; from there they move inland. In business, your beachhead is the first small group you "win".

#### Characteristics of a good beachhead market

- It should be **specific and narrowly defined**.
- It should be **reachable** by the startup team.
- The users should have a **strong, immediate need** for the solution.

For example, targeting "**all students in Pakistan**" is too broad for a new study app. "**Grade XII students preparing for board exams in one college**" is much more focused and easier to reach.



Start with the small inner circle (beachhead), learn and improve, then expand outward.



**Table 6.11: Choosing a focused beachhead market**

| Business Idea     | Weak First Market         | Better Beachhead Market                                 |
|-------------------|---------------------------|---------------------------------------------------------|
| Study app         | All students in Pakistan. | Grade XII students preparing for exams in one college.  |
| Canteen pre-order | All canteens in the city. | One college canteen during lunch break.                 |
| Ajrak catalogue   | All online shoppers.      | Existing customers of one local shop.                   |
| Van update system | All transport users.      | Students and parents of ONE specific college van route. |

Choosing a small first market **does not mean the idea will remain small forever**. After learning from this group, the idea can be improved and expanded, even globally.

#### WORKED EXAMPLE

**Worked example: test a proposed beachhead against the three characteristics.** Idea: an app that reminds farmers about irrigation water turns (*warabandi*).

| Proposed market                                                                        | Specific? | Reachable?                              | Strong, immediate need?            | Verdict        |
|----------------------------------------------------------------------------------------|-----------|-----------------------------------------|------------------------------------|----------------|
| All farmers in Sindh                                                                   | No        | No (millions of people, many districts) | Varies                             | Weak           |
| Farmers on one watercourse near Badin whose sons/daughters study in the team's college | Yes       | Yes (through classmates)                | Yes (missing a turn damages crops) | Good beachhead |

Expansion path: one watercourse → nearby watercourses → the whole district → other districts.

#### REMEMBER IT

**Remember S-R-U: Specific, Reachable, Urgent need.** If you cannot meet your first users within a week, your market is too big.

#### COMMON MISTAKE

**Mistake:** Thinking a bigger first market means more success ("everyone can use my app").

**Fix:** A broad market is hard to reach and hard to learn from. Win a small group first, learn, improve, then expand.

#### اردو میں سمجھیں

بیج ہیڈ مارکیٹ (Beachhead Market) پہلے صارفین کا ایک چھوٹا اور مخصوص گروہ ہے جسے حل کی واضح اور فوری ضرورت ہو اور جو پراڈکٹ جلد آزمانے پر تیار ہو۔ سب کو ایک ساتھ خدمت دینے کے بجائے پہلے ایک چھوٹے گروہ پر توجہ دیں، جیسے "پاکستان کے تمام طلبہ" کے بجائے "ایک کالج کے بارہویں جماعت کے طلبہ"۔ وہاں سے سیکھ کر خیال کو بعد میں پھیلا دیا جا سکتا ہے۔

#### EXAM TIP

Asked as "Define beachhead market and give its characteristics" or "Suggest a beachhead market for ...". Write the definition (small, focused group of first users with a clear need, likely to try early), the three characteristics (specific, reachable, strong immediate need), one weak-versus-better pair from Table 6.11, and the line that a small first market does not mean the idea stays small.

#### CHECK YOURSELF

1. Give the three characteristics of a good beachhead market.
2. What is the better beachhead market for the van update system?
3. Does choosing a small first market mean the business will stay small?

Answers: 1. Specific and narrowly defined; reachable by the team; users have a strong, immediate need. 2. Students and parents of ONE specific college van route. 3. No. After learning from the first group, the idea can be improved and expanded.

## 6.15 Complete Student Project

#### LEARN IT

You will work in **groups** and complete one small entrepreneurship project. The project should be **simple, local, and possible in the classroom**.

The purpose is to practise the **complete journey**: problem → prototype → feedback → MVP plan → risky assumption → beachhead market.

**Suggested project ideas:** homework and test reminder system, college notice organizer, canteen pre-order system, small shop WhatsApp catalogue, college van update system, lost and found item reporting system, study notes organizer.

**Table 6.12: Student project sheet (template)**

| Item                                | Your group's answer |
|-------------------------------------|---------------------|
| Problem statement                   |                     |
| Target users                        |                     |
| Business idea                       |                     |
| Low-fidelity prototype description  |                     |
| Feedback from users                 |                     |
| Improvement made after feedback     |                     |
| MVP plan                            |                     |
| Riskiest assumption                 |                     |
| How to test the riskiest assumption |                     |
| Beachhead market                    |                     |
| Marketing method for first users    |                     |

### WORKED EXAMPLE

#### Worked example: a completed project sheet (Lost and Found system).

| Item                   | Answer                                                                                                                                                       |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem statement      | Students of our college in Mirpurkhas lose items (bottles, calculators, ID cards) and rarely get them back because there is no central place to report them. |
| Target users           | Students and staff of our college.                                                                                                                           |
| Business idea          | A Lost and Found WhatsApp group plus a Google Form to report lost or found items.                                                                            |
| Low-fidelity prototype | Two paper screens: "Report lost item" (item, colour, place, phone of class rep) and "Found items list" with sketches.                                        |
| Feedback from users    | 6 of 8 testers asked for photos; 3 did not want their own phone number shown.                                                                                |
| Improvement made       | Added a photo upload; replaced personal numbers with "collect from the library desk".                                                                        |
| MVP plan               | Google Form + one WhatsApp group run by two volunteers; free; tested for two weeks.                                                                          |
| Riskiest assumption    | Students who find items will take the trouble to report them.                                                                                                |
| How to test it         | Count how many "found" reports arrive in two weeks compared with "lost" reports.                                                                             |
| Beachhead market       | Grade XII students of our college.                                                                                                                           |
| Marketing method       | Announcement in morning assembly, a poster at the library and a message in each class group.                                                                 |

**Table 6.13: Rubric for project assessment**

A **rubric** is a marking guide that describes what excellent, satisfactory and weak work looks like for each part of a project.

| Criteria         | Excellent                                              | Satisfactory                                       | Needs Improvement                                   |
|------------------|--------------------------------------------------------|----------------------------------------------------|-----------------------------------------------------|
| Problem ID       | Clear, local, and important.                           | Understandable but needs detail.                   | Vague or too broad.                                 |
| Prototype        | Shows clear flow and features.                         | Shows the basic idea.                              | Incomplete or unclear.                              |
| Feedback & Fixes | Collected properly; changes clearly based on feedback. | Collected but analysis is weak; some changes made. | No clear feedback collected; changes not explained. |
| MVP Plan         | Essential features and clear testing plan.             | Basic features but needs testing clarity.          | Confused with the final product.                    |

### REMEMBER IT

Use the rubric **before** you submit: read your project sheet row by row and ask, "Is this Excellent yet?" For "Feedback & Fixes", always write **which** feedback caused **which** change.

#### COMMON MISTAKE

**Mistake:** Writing an MVP plan that lists every feature of the dream app. The rubric marks this as "Confused with the final product".

**Fix:** Keep the MVP to the essential feature(s) and add a clear testing plan (how many users, how long, what you will measure).

#### اردو میں سمجھیں

طلبہ گروپوں میں ایک چھوٹا، مقامی اور آسان منصوبہ مکمل کرتے ہیں جس میں مسئلے سے پروٹوٹائپ، رائے، ایم وی پی منصوبہ، سب سے خطرناک مفروضہ اور بیچ بیڈ مارکیٹ تک پورا سفر شامل ہوتا ہے۔ منصوبے کو جانچنے کے لیے روبرک (Rubric) استعمال ہوتا ہے جو مسئلے کی پہچان، پروٹوٹائپ، رائے و بہتری اور ایم وی پی منصوبے کو بہترین، تسلی بخش یا بہتری کی ضرورت کے درجوں میں پرکھتا ہے۔

#### CHECK YOURSELF

1. Name four suggested project ideas.
2. What does "Excellent" look like for Problem ID in the rubric?
3. Which rubric level says "Confused with the final product"?

Answers: 1. Any four: homework and test reminder system, college notice organizer, canteen pre-order system, small shop WhatsApp catalogue, college van update system, lost and found reporting system, study notes organizer. 2. Clear, local, and important. 3. Needs Improvement (for the MVP Plan).

## 6.16 Ethical and Safe Use of Digital Tools

#### LEARN IT

Digital entrepreneurship should be **responsible**. When collecting user feedback, you must respect:

- **Privacy:** people's personal information belongs to them.
- **Truthfulness:** report results honestly; do not make up numbers or hide bad feedback.
- **Fairness:** treat all users equally.

You should **not collect unnecessary personal information** or **share phone numbers or photos without permission**.

### Equity point

A good digital solution should consider users who have **weak internet**, **limited devices**, or **language difficulty**. A design with **large buttons**, **simple Urdu/Sindhi instructions**, and **low data usage** helps ensure **equal access**. *Equity* means making sure everyone, including people with fewer resources, can use the solution.

#### WORKED EXAMPLE

**Worked example: fix an unethical feedback form.** A team's canteen feedback form asks:

1. Full name, father's name, CNIC/B-form number, home address. → **Problem:** unnecessary personal data. **Fix:** ask only class (or nothing personal at all).
2. "Upload your selfie." → **Problem:** photos are not needed and could be misused. **Fix:** remove.
3. The team plans to post all respondents' phone numbers in the class group. → **Problem:** sharing numbers without permission. **Fix:** never share; keep responses private to the team.
4. The form is in English only and has a 5 MB video at the top. → **Problem:** excludes users with language difficulty or weak internet. **Fix:** add Urdu/Sindhi instructions and remove the heavy video.

When presenting, the team reports "30 out of 40 would use it again" exactly, without rounding it up to "almost everyone". That is truthfulness.

#### REMEMBER IT

**Ask yourself "P-T-F-E":** Is it **P**rivate, **T**ruthful, **F**air, and **E**qual-access?

#### COMMON MISTAKE

**Mistake:** "More data is always better, so let's collect everything."

**Fix:** Collect only what you need to answer your test questions. Extra personal data creates risk for users and gives you no benefit.

#### اردو میں سمجھیں

ڈیجیٹل کاروبار ذمہ داری سے کرنا چاہیے۔ رائے جمع کرتے وقت لوگوں کی رازداری (Privacy)، سچائی اور انصاف کا خیال رکھیں؛ غیر ضروری ذاتی معلومات نہ لیں اور اجازت کے بغیر فون نمبر یا تصاویر شیئر نہ کریں۔ اچھا حل کمزور انٹرنیٹ، محدود موبائل اور زبان کی مشکل والے لوگوں کا بھی خیال رکھتا ہے، جیسے بڑے بٹن، آسان اردو یا سندھی ہدایات اور کم ڈیٹا کا استعمال۔

#### CHECK YOURSELF

1. Name the three values students must respect when collecting feedback.
2. Give two things students must not do with users' information.
3. List three design features that support equal access.

Answers: 1. Privacy, truthfulness, fairness. 2. Collect unnecessary personal information; share phone numbers or photos without permission. 3. Large buttons, simple Urdu/Sindhi instructions, low data usage.

## Career Opportunities

#### LEARN IT

This unit covered entrepreneurship in the digital age, turning problems into business ideas, prototyping, building MVPs, and identifying beachhead markets. Mastery of these concepts opens the door to careers in **startups and digital innovation**. Pakistan has many incubators (for example, National Incubation Centres, NICs, in major cities) that support young founders.

| Career                              | What the person does                                                                                                                             |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Tech Entrepreneur / Startup Founder | Identifies a real-world problem and builds a digital product or service to solve it, guiding it from prototype through to MVP and market launch. |
| Product Manager                     | Guides a product's direction by identifying user needs, prioritising features, and testing ideas through prototypes and MVPs.                    |
| Business Analyst                    | Analyses business problems and processes to recommend digital solutions, often testing assumptions before full investment.                       |
| Innovation / Startup Consultant     | Advises businesses or founders on validating ideas, building MVPs, and identifying beachhead markets.                                            |
| Digital Marketing Specialist        | Plans how a new product or MVP will reach its first (beachhead) market and grow its user base.                                                   |
| Venture / Incubator Analyst         | Evaluates early-stage startup ideas and prototypes for potential investment or incubation support.                                               |

#### WORKED EXAMPLE

**Worked example: match the task to the career.**

1. "Decide whether the canteen app should add payments next or a weekly menu." → **Product Manager** (prioritising features).
2. "Run a WhatsApp and Instagram campaign to get the first 100 users in one college." → **Digital Marketing Specialist**.
3. "Review 20 student startup pitches and pick 5 for incubation." → **Venture / Incubator Analyst**.

#### REMEMBER IT

Every career in the table uses the same chapter skills: **understand users, test assumptions, start small**.

#### COMMON MISTAKE

**Mistake:** Thinking startup careers are only for programmers.

**Fix:** Product managers, analysts, consultants and marketers often write little or no code; they test ideas and understand users.

#### اردو میں سمجھیں

اس باب کی مہارتیں اسٹارٹ اپ اور ڈیجیٹل جدت کے شعبوں میں روزگار کے دروازے کھولتی ہیں، جیسے اسٹارٹ اپ بانی، پراڈکٹ مینیجر، بزنس اینالسٹ، اسٹارٹ اپ کنسلٹنٹ، ڈیجیٹل مارکیٹنگ ماہر اور انکیوبیٹر اینالسٹ۔ ان سب کاموں میں صارف کو سمجھنا، مفروضے آزمانا اور چھوٹے پیمانے سے شروع کرنا بنیادی ہے۔

#### CHECK YOURSELF

1. Which career plans how an MVP will reach its beachhead market?
2. What does a Business Analyst often do before full investment?

Answers: 1. Digital Marketing Specialist. 2. Tests assumptions (after analysing business problems and processes).

## Chapter 6 Review

## Chapter summary

- **Entrepreneur:** a person who identifies a problem, thinks of a solution, and turns it into a product or service; observes people, takes calculated risks, improves step by step.
- **Entrepreneurship:** the process of planning, starting, testing and improving a business idea until it becomes useful (Problem → Solution → Product).
- **Digital age:** entrepreneurs use accessible digital tools (social media, forms, websites, WhatsApp, digital payments) to start and test ideas quickly with low investment; often no coding is needed.
- A digital idea must solve a **real problem for real users**, not just sound modern (Table 6.1 examples: notices, embroidery shop, test dates, canteen queues).
- **Business ideas** come from observing daily life and talking directly to users: Observe → Ask Users → Select One Problem → Suggest Solution → Test Early. Start with a small, clear problem.
- **Prototype:** an early model used to show and test the design of an idea; not the final product; makes the idea visible, finds mistakes early, saves time and money.
- **Fidelity** is how close a prototype is to the final product: Low (paper), Mid (PowerPoint/Canva/Figma, tests navigation), High (realistic, Figma/HTML, only after the idea is tested).
- **Development cycle:** Design → Build → Test → Iterate, repeated (iteration). Feedback is crucial; a prototype fails only if useful feedback is ignored.
- **MVP:** the simplest working version with only essential features (Minimum) that still solves the main problem (Viable); it tests real use and market demand.
- **Prototype vs MVP:** prototype asks "Does it look right?"; MVP asks "Will people actually use it?" A prototype may not work; an MVP must.
- **Riskiest assumption:** the untested belief that, if wrong, will make the business fail; test it early with real behaviour.
- **MVP plan:** problem, target users, ONE main feature, basic design, cost, marketing, feedback, improvement plan. Do not build everything at once.
- **Canteen case study:** paper prototype → Google Form MVP with 5 items for 3 days → staff found too many items hard → kept 5 items, separate line, orders close 20 minutes before break.
- **MVP testing:** plan users, duration and data in advance; collect numbers (40 tried, 30 would return = 75%) and comments/observations; count, look for repeated problems, fix the most important first.
- **Beachhead market:** a small, focused, reachable group of first users with a strong, immediate need; win it first, then expand.
- **Ethics and equity:** respect privacy, truthfulness and fairness; collect no unnecessary data; share nothing without permission; design for weak internet, limited devices and Urdu/Sindhi speakers.

## Key terms

| Term                         | Meaning                                                                                                            | اردو                         |
|------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------|
| Entrepreneur                 | A person who identifies a problem, thinks of a solution, and turns it into a product or service.                   | کاروباری شخص                 |
| Entrepreneurship             | The process of planning, starting, testing and improving a business idea.                                          | کاروبار کاری                 |
| Calculated risk              | A risk taken after carefully thinking about the possible loss and keeping it small.                                | سوچا سمجھا خطرہ              |
| Digital entrepreneurship     | Using digital tools to start, test, improve and grow a business idea.                                              | ڈیجیٹل کاروبار کاری          |
| Business idea                | A proposed product or service that solves a real problem for real users.                                           | کاروباری خیال                |
| Target users                 | The people who will use the product first.                                                                         | ہدف صارفین                   |
| Prototype                    | An early model of a product, service, website or app made to test the idea and collect feedback.                   | ابتدائی نمونہ                |
| Fidelity                     | How close a prototype is to the final product.                                                                     | اصل سے مطابقت کا درجہ        |
| Low-fidelity prototype       | A quick, rough paper model showing the main idea and flow.                                                         | کم تفصیل والا نمونہ          |
| Mid-fidelity prototype       | A clearer model in PowerPoint, Canva or Figma showing layout and navigation without final graphics.                | درمیانی تفصیل والا نمونہ     |
| High-fidelity prototype      | A detailed model close to the final product with realistic colours, buttons, data and working features.            | زیادہ تفصیل والا نمونہ       |
| Wireframe                    | A simple outline drawing of a screen showing where items go, without colours or pictures.                          | اسکرین کا خاکہ               |
| Navigation                   | How a user moves from one screen to another.                                                                       | اسکرینوں کے درمیان آمد و رفت |
| Iteration                    | Repeating the cycle of creating, testing and improving.                                                            | بار بار دہرانا               |
| Feedback                     | Users' responses and comments about a product or idea.                                                             | رائے                         |
| Minimum Viable Product (MVP) | The simplest working version of a product with only the essential features, used to test the idea with real users. | کم از کم قابل عمل پراڈکٹ     |



| Term                   | Meaning                                                                                                | اردو                 |
|------------------------|--------------------------------------------------------------------------------------------------------|----------------------|
| Viable                 | Usable enough to actually solve the main problem.                                                      | قابلِ عمل            |
| Market demand          | Whether enough people want and will use or pay for a product.                                          | بازار کی طلب         |
| Assumption             | Something we believe to be true but have not tested yet.                                               | مفروضہ               |
| Riskiest assumption    | The belief that would make the whole business fail if it turned out to be wrong.                       | سب سے خطرناک مفروضہ  |
| MVP plan               | A plan stating problem, target users, main feature, design, cost, marketing, feedback and improvement. | ایم وی پی کا منصوبہ  |
| Number-based feedback  | Feedback in counts, e.g. how many tried the MVP or would use it again.                                 | عددی رائے            |
| Comment-based feedback | Feedback in words explaining likes, dislikes or confusion.                                             | تبصراتی رائے         |
| Observation            | Watching users to see where they face difficulty.                                                      | مشاہدہ               |
| Beachhead market       | A small, focused group of first users with a clear need who are likely to try the product early.       | ابتدائی مخصوص مارکیٹ |
| Rubric                 | A marking guide describing levels of quality for each criterion.                                       | جانچ کا معیار نامہ   |
| Privacy                | A person's right to control their personal information.                                                | رازداری              |
| Equity                 | Making sure people with fewer resources or language difficulty can also use a solution.                | مساوی رسائی          |
| Startup                | A new, small business built around a new product or service idea.                                      | نیا کاروبار          |

## Practice: multiple choice

1. An entrepreneur is best described as a person who:
  - a. only invests money in big companies
  - b. identifies a problem and tries to turn a solution into a product or service
  - c. always works from a large office
  - d. only writes computer programs
2. In prototyping, "fidelity" means:
  - a. the cost of the prototype
  - b. the number of users
  - c. how close the prototype is to the final product
  - d. the speed of the internet
3. Which of these is a low-fidelity prototype?
  - a. Hand-drawn app screens on paper
  - b. A working demo website of an order form
  - c. Clickable screens in Figma
  - d. An app published on the Play Store
4. A mid-fidelity prototype is mainly useful for testing:
  - a. online payments
  - b. screen-to-screen navigation
  - c. market demand over three days
  - d. investor interest
5. The correct order of the prototype development cycle is:
  - a. Build, Design, Iterate, Test
  - b. Test, Build, Design, Iterate
  - c. Design, Test, Build, Iterate
  - d. Design, Build, Test, Iterate
6. Creating, testing, improving and testing a prototype again is called:
  - a. iteration
  - b. incubation
  - c. marketing
  - d. assumption
7. In MVP, the letter "V" stands for:
  - a. Visual
  - b. Valuable
  - c. Viable
  - d. Virtual
8. Which is the best MVP for a canteen pre-order idea?
  - a. A full app with login, payment, delivery tracking and discounts
  - b. A Google Form used for real orders for three days
  - c. Paper sketches shown to five friends
  - d. A colourful poster of the menu
9. A prototype mainly answers which question?
  - a. Does it look right and make sense?
  - b. Will people pay for it every month?
  - c. How big is the national market?
  - d. How much profit will we make?
10. The riskiest assumption is:
  - a. the cheapest feature to build
  - b. the colour scheme of the app
  - c. the name of the company
  - d. the belief that, if wrong, can make the whole business fail
11. Which is the best beachhead market for a new study app?
  - a. All students in Pakistan
  - b. All online shoppers
  - c. Grade XII students preparing for board exams in one college
  - d. All transport users
12. Which is NOT a characteristic of a good beachhead market?
  - a. Specific and narrowly defined
  - b. Reachable by the startup team
  - c. Users have a strong, immediate need
  - d. As large as possible from the first day
13. "30 out of 40 said they would use it again" is which type of feedback?
  - a. Number-based
  - b. Comment-based
  - c. Observation
  - d. Rubric-based
14. Which is an ethical practice when collecting user feedback?
  - a. Collect CNIC numbers of all users
  - b. Share users' phone numbers in the class group
  - c. Collect only necessary information and share nothing without permission
  - d. Change results so the idea looks better
15. To support users with weak internet and language difficulty, a design should use:
  - a. heavy videos on every screen
  - b. English-only instructions
  - c. features that need the latest phone
  - d. large buttons, simple Urdu/Sindhi instructions and low data usage

### Practice: short questions

1. Define entrepreneur and entrepreneurship. How are they different?
2. What is meant by entrepreneurship in the digital age? Give two local examples that need no coding.
3. Write the five questions you should ask to find a problem in daily life.
4. Why should a student choose a small problem at the start? Give one example of a too-large and one of a suitable problem.
5. Define prototype and give three reasons why it is useful.
6. Differentiate between low-, mid- and high-fidelity prototypes, with one example of each.
7. Define MVP and explain the words "Minimum" and "Viable".
8. What is the riskiest assumption? Give the risky assumption and its test for the college van update system.
9. An MVP was tried by 40 students and 30 said they would use it again. Calculate the repeat-use rate and name two other kinds of feedback to collect.
10. State the three characteristics of a good beachhead market and give one weak and one better first market for an Ajrak catalogue.

### Practice: long / problem questions

1. Explain the prototype development cycle with a labelled diagram. Use the ClassBuddy class activity to show what happens at each stage, and explain why negative feedback does not mean the prototype has failed.
2. Compare a prototype and an MVP under five headings (purpose, working status, users, cost, example). Use the canteen pre-order system as your example and state the key question each one answers.
3. **Plan an MVP for a college van update system** for a college in Hyderabad, where students and parents often do not know when the van will arrive. Fill in all eight elements of the MVP planning checklist, and also state the riskiest assumption, how to test it, and the beachhead market.
4. **Plan an MVP for a small Sindhi topi and Ajrak shop in Hala** that wants online customers. Describe a low-fidelity prototype, the MVP, its riskiest assumption and test, the beachhead market, the marketing method, and two ethical or equity points the shop must follow.
5. A team tested a Lost and Found MVP (Google Form + WhatsApp group) in a college of 60 Grade XII students for two weeks. 45 students tried it; 36 of them said they would use it again; 12 of the users said they did not know where to collect found items; the team observed that many reports had no photo. (a) Calculate the percentage of the class who tried it, the repeat-use rate among users, and the percentage of users confused about collection. (b) Classify each piece of feedback by type. (c) Decide which improvement to make first and justify it.

## Answer key

---

### Multiple choice

1. b
2. c
3. a
4. b
5. d
6. a
7. c
8. b
9. a
10. d
11. c
12. d
13. a
14. c
15. d

## Short questions — model answers

1. An **entrepreneur** is a person who identifies a problem, thinks of a solution and tries to change that solution into a product or service. **Entrepreneurship** is the process of planning, starting, testing and improving a business idea, involving creativity, problem-solving, teamwork, communication and calculated risk. The entrepreneur is the person; entrepreneurship is the process.
2. It means using digital tools such as mobile phones, social media, websites, online forms, digital payments, maps, videos and simple apps to start, test, improve and grow a business idea. Examples: a student selling handmade cards through Instagram; a shopkeeper making a WhatsApp catalogue for Ajrak.
3. What problem do people face? Who faces this problem? How often does it happen? What do people currently do to solve it? Can technology help in a simple way?
4. Small, clear problems are easier to test and improve. Too large: "improving the whole education system". Suitable: "helping Grade XII students track practical file submission dates".
5. A prototype is an early model of a product, service, website or app that shows how the idea may look or work; it is not the final product and is made to test the idea and collect feedback. Useful because it makes the idea visible, helps find mistakes early (hard-to-find buttons, too much text, missing features), and saves time and money before final coding.
6. Low-fidelity: quick, rough paper model showing main idea and flow, e.g. hand-drawn homework-reminder screens. Mid-fidelity: clearer model in PowerPoint/Canva/Figma showing layout and navigation without final graphics, e.g. clickable PowerPoint screens. High-fidelity: detailed model close to the final product with realistic colours, buttons, sample data and working features, e.g. a working demo of a small order-form website.
7. An MVP (Minimum Viable Product) is the simplest working version of a product with only the most important features, used to test the idea with real users. Minimum: only the essential features, no extras. Viable: usable enough to actually solve the main problem.
8. It is the belief that can make the entire business fail if it turns out to be wrong. For the van update system: "Parents and students will check van updates daily." Test: create a trial WhatsApp broadcast group for one route.
9. Repeat-use rate =  $(30 \div 40) \times 100 = 75\%$ . Other feedback: comment-based (e.g. "What was confusing?") and observation (e.g. where users faced difficulty, such as forgetting to enter class name).
10. Specific and narrowly defined; reachable by the team; users have a strong, immediate need. Ajrak catalogue: weak = all online shoppers; better = existing customers of one local shop.

## Long questions — model answers

1. **Prototype development cycle.** Draw four boxes in a loop: Design → Build → Test → Iterate → back to Design. Prototype development is not one-time; repeating the cycle is called iteration.
  - **Design:** understand the problem and plan what the prototype shows. ClassBuddy: interview 10 students; 7 of 10 forget tasks, so show a daily task list; reminders are needed before leaving home, so add a morning notification; students want simplicity, so use large buttons and clear labels.
  - **Build:** create the first version to make the idea visible (not beautiful). ClassBuddy: three paper screens (Today's classes, Add homework, Show reminder) joined by arrows.
  - **Test:** show it to users and observe. ClassBuddy: five classmates try "add one homework task" while the observer watches silently.
  - **Iterate:** improve from feedback. ClassBuddy: larger, centred Add Homework button; subject name and due date fields; a Weekly View button.

Negative feedback is information that shows what to fix; the prototype fails only if the team ignores useful feedback.

2. **Prototype vs MVP.** Purpose: prototype shows how the idea may look or work; MVP tests whether users will actually use it. Working status: prototype may not work (sketch or model); MVP must work at least in a basic way. Users: prototype, a small group giving design feedback; MVP, early real users in daily life. Cost: prototype very low; MVP low but may need basic development. Example: prototype = paper screens for a canteen order system; MVP = Google Form used for real canteen orders for three days. Key questions: prototype, "Does it look right and make sense?"; MVP, "Will people actually use it and find value in it?"

3. **MVP plan: college van update system (Hyderabad).**

- Problem: students and parents do not know when the van will arrive, so students wait outside for a long time or miss the van.
- Target users: students and parents on one van route.
- Main feature: the driver or a route monitor sends a short "Van left / 5 minutes away" message.
- Basic design: one WhatsApp broadcast message with route stop name, time and status; voice note option for parents who cannot read.
- Cost: free, using a WhatsApp broadcast list on the monitor's phone.
- Marketing: a message in the class WhatsApp group and a printed slip for parents with the number to save.
- Feedback: after one week, a 3-question Google Form or phone call: Did you check the updates daily? Did you wait less? Would you continue?
- Improvement plan: if messages arrive too late, send them earlier; if many parents do not read, switch to voice notes; then expand to a second route.
- Riskiest assumption: parents and students will check van updates daily. Test: trial broadcast group for one route for one week, counting how many people read or reply to messages each day.
- Beachhead market: students and parents of ONE specific college van route.

4. **MVP plan: topi and Ajrak shop in Hala.**

- Low-fidelity prototype: paper catalogue pages, one per product, with a sketch, price, size/type and "Message to order"; shown to 5 customers to see if information is clear.
- MVP: a WhatsApp Business catalogue with real photos and prices for 10–15 best-selling items; orders by chat; payment by cash on delivery or JazzCash/Easypaisa.
- Riskiest assumption: customers will actually place orders through WhatsApp. Test: share the catalogue with 30 existing customers and record actual orders for two weeks.
- Beachhead market: existing customers of this one shop (people who have bought before and whose numbers the shop already has with permission).
- Marketing: broadcast message to existing customers, a QR code at the shop counter, posts in local Facebook groups.
- Ethics: share no customer numbers or photos without permission; give truthful prices, sizes and photos. Equity: simple Urdu/Sindhi descriptions and compressed photos for low data use.
- Improvement plan: drop designs nobody orders, add more of the popular ones, then expand to customers in Hyderabad and Karachi.

## 5. Lost and Found MVP analysis.

- o (a) Tried:  $(45 \div 60) \times 100 = 75\%$  of the class. Repeat-use rate:  $(36 \div 45) \times 100 = 80\%$ . Confused about collection:  $(12 \div 45) \times 100 = 26.7\%$  (about 27%).
- o (b) "45 tried" and "36 would use again": number-based. "Did not know where to collect": comment-based (12 users said it). "Many reports had no photo": observation.
- o (c) Fix the collection confusion first: it is a repeated problem (more than a quarter of users) and it stops the system's main purpose, which is returning items to owners. Add "Collect found items from the library desk" at the top of the form and pin it in the WhatsApp group. Next, make the photo field strongly encouraged (or required for found items), since photos help owners recognise items. Re-test for another week and compare the numbers.

### Self-assessment: I can...

- |                                                                                                            |                                                                                                                |
|------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> define entrepreneur and entrepreneurship and describe an entrepreneur's habits.   | <input type="checkbox"/> explain how digital tools let me start and test an idea cheaply, with local examples. |
| <input type="checkbox"/> use the five questions and the steps of Table 6.2 to turn a problem into an idea. | <input type="checkbox"/> define a prototype and list its benefits and possible forms.                          |
| <input type="checkbox"/> compare low-, mid- and high-fidelity prototypes.                                  | <input type="checkbox"/> draw and explain the Design-Build-Test-Iterate cycle.                                 |
| <input type="checkbox"/> turn user feedback into problems and improvements, as in Table 6.5.               | <input type="checkbox"/> define an MVP and explain how it differs from a prototype.                            |
| <input type="checkbox"/> identify the riskiest assumption of an idea and design a test for it.             | <input type="checkbox"/> fill in an MVP plan and analyse MVP test results with percentages.                    |
| <input type="checkbox"/> choose a specific, reachable beachhead market.                                    | <input type="checkbox"/> collect feedback ethically and design for equal access.                               |

### FINAL REVISION

## One-page recaps and a mock paper

Use these pages in the last week before the exam. Read one recap aloud from memory, then sit the mock paper in 2 hours 30 minutes with no notes.

امتحان سے ایک ہفتہ پہلے خلاصے زبانی دہرائیں، پھر ماک پیپر بغیر نوٹس کے حل کریں۔

## Chapter 1 recap — Human Computer Interaction

- HCI = study of how people and computers work together so systems are usable, useful and satisfying.
- Traditional interaction = keyboard, mouse, menus. Natural interaction = touch, voice, gesture, gaze.
- Components: user, computer, interaction (input + output + feedback) inside an environment.
- UI = how it looks and the controls. UX = how the whole experience feels. Wireframe → prototype (Figma) → test.
- Evaluate with usability tests, heuristic evaluation, A/B tests and automated checks. Golden: accessibility, HCI problems, UI vs UX, testing methods.

## Chapter 2 recap — Algorithms and Data Structures

- Correctness: trace table (variable values after every step) or stepwise reasoning.
- Clarity: modularity (small named parts) and readability (names, comments, indentation).
- Efficiency: count steps, conditions, repetitions. Big O:  $O(1)$  constant,  $O(\log n)$  binary search,  $O(n)$  linear,  $O(n \log n)$  good sorts,  $O(n^2)$  nested loops.
- Linear DS: array, linked list, stack (LIFO, push/pop), queue (FIFO, enqueue/dequeue). Non-linear: tree (one root, parent-child), graph (nodes + edges).
- Pick the structure from the problem: undo → stack; printer jobs → queue; hierarchy → tree; maps → graph; fast index → array.

## Chapter 3 recap — Python Data Structures, Functions and Files

---

- list [] ordered, mutable. tuple () ordered, immutable. set {} unordered, unique. dict {k:v} key-value, mutable.
- Functions: def, parameters, return. Local vs global scope. Built-in vs user-defined.
- Files: "r" read, "w" overwrite, "a" append. Always prefer with open(...) as f so the file closes itself.
- Exceptions: try / except / else / finally. Catch specific errors (ValueError, FileNotFoundError).

## Chapter 4 recap — Data and Analysis

---

- SQLite in Python: connect → cursor → execute → commit → close. Always commit writes.
- Pandas DataFrame = table. load CSV/Excel, isnull, dropna, fillna, sort, filter, groupby.
- Charts: bar (categories), line (trend over time), pie (parts of a whole, few slices), histogram (distribution), scatter (relationship of two variables), box (spread and outliers).
- Mean = average. Median = middle after sorting. Mode = most frequent. Range = max–min. Variance / std-dev measure spread. pandas .std() uses sample n–1.

## Chapter 5 recap — AI, Security and Collaboration

---

- AI ⊃ Machine Learning ⊃ Deep Learning. A neuron: weighted sum + bias, then an activation function. Deep = many hidden layers.
- Secure collaboration: authentication (passwords, MFA), access control (who may do what).
- Protect data: encryption, strong passwords, 3-2-1 backup, firewall. Threats: malware, phishing, DoS/DDoS, social engineering.
- Equity: everyone can take part (devices, language, disability, bandwidth). Tools: shared docs, video, LMS, versioning.

## Chapter 6 recap — Entrepreneurship in the Digital Age

---

- Entrepreneur = person who turns a problem into a venture, taking risk. Digital tools make testing cheap (WhatsApp, Forms, a Facebook page).
- Prototype = experimental model. Low / mid / high fidelity. Cycle: Design → Build → Test → Iterate.
- MVP = smallest product that tests the riskiest assumption with real users. Prototype ≠ MVP (MVP is used in the real market).
- Beachhead market = the first, narrow group you can win. Measure retention, not only sign-ups. Use data ethically.

## Mock paper — Computer Science XII

---

Time: 2 hours 30 minutes | Total: 75 marks | Answer all of A and B; attempt three from C.

## Section A — Multiple choice (15 × 1 = 15)

1. HCI stands for:
  - a. Human Computer Interaction
  - b. High Code Interface
  - c. Hardware Control Input
  - d. Hybrid Computing Internet
2. A wireframe is:
  - a. the final coloured UI
  - b. a simple layout sketch of screens
  - c. a compiled program
  - d. an encryption key
3. Which is  $O(\log n)$ ?
  - a. Linear search
  - b. Nested for-loops
  - c. Binary search
  - d. Reading one array cell
4. A stack is:
  - a. FIFO
  - b. LIFO
  - c. random access only
  - d. unordered unique items
5. Which Python type is immutable?
  - a. list
  - b. dict
  - c. set
  - d. tuple
6. The file mode that overwrites an existing file is:
  - a. r
  - b. a
  - c. w
  - d. x+
7. `with open("t.txt") as f` is useful because:
  - a. it encrypts the file
  - b. it closes the file automatically
  - c. it creates a DataFrame
  - d. it skips exceptions
8. After INSERT in sqlite3 you must:
  - a. rollback always
  - b. commit
  - c. drop the table
  - d. print the cursor
9. A pie chart is a poor choice when:
  - a. you have 3 parts of a total
  - b. you have 20 tiny slices
  - c. you show percentages of a whole
  - d. you label each slice
10. pandas `df['x'].std()` uses:
  - a. population n
  - b. sample n-1
  - c. median
  - d. mode
11. Deep Learning is a subset of:
  - a. only statistics
  - b. Machine Learning
  - c. only HCI
  - d. only networking
12. MFA means:
  - a. Many File Archives
  - b. Multi-Factor Authentication
  - c. Main Firewall Access
  - d. Minimum Function API
13. A Caesar cipher is a form of:
  - a. phishing
  - b. encryption
  - c. a firewall
  - d. a prototype
14. An MVP is:
  - a. the most polished version of a product
  - b. the smallest product that tests the riskiest assumption
  - c. a paper sketch only
  - d. a full company registration
15. A printer job list is best modelled as a:
  - a. stack
  - b. queue
  - c. binary tree
  - d. set



## Section B — Short questions (10 × 3 = 30)

1. Differentiate UI and UX with one example.
2. What is a trace table? Why is it better than "it looks right"?
3. Draw a 3-node stack after push 10, push 20, pop. Label TOS.
4. Write one difference each: list vs tuple vs set vs dictionary.
5. Write a Python function `area(l, w)` that returns  $l*w$ , and a call that prints it.
6. Name the five steps of a database connection in Python.
7. When would you draw a scatter plot rather than a line graph?
8. Compute mean, median and range of 4, 8, 6, 10, 2.
9. List three signs of a phishing message.
10. Differentiate a prototype and an MVP.

## Section C — Long questions (3 × 10 = 30). Attempt any three.

1. A college wants a new student-portal. Describe need analysis, a low-fidelity wireframe of the login + timetable screens, three accessibility principles you would apply, and one usability test you would run.
2. Trace this algorithm with a table for  $n = 5$ :  $total=0$ ; for  $i$  in  $1..n$ :  $total = total + i$ ; return  $total$ . State its Big O and rewrite it as a formula that is  $O(1)$ .
3. Write a Python program that reads a text file of lines "name,marks", stores results in a dictionary, and writes "name: grade" to an output file using `with`. Grade A if marks  $\geq 80$  else B. Include exception handling if the file is missing.
4. Using the dataset 12, 18, 12, 20, 15, compute mean, median, mode, range, population variance and population standard deviation, showing every step. Then state which chart you would use to show the distribution and why.
5. A Karachi canteen wants a pre-order app. Identify the problem, the riskiest assumption, an MVP, a beachhead market, two test metrics, and one ethical rule for student data.

## Mock paper — answers

---

### Section A

1. a
2. b
3. c
4. b
5. d
6. c
7. b
8. b
9. b
10. b
11. b
12. b
13. b
14. b
15. b

## Section B

1. UI is the screens and controls (buttons, colours). UX is the whole journey (time to complete a task, frustration, trust). Example: a JazzCash send-money screen (UI) vs how safe and quick the payment felt (UX).
2. A trace table lists each variable after every instruction. It proves correctness on a chosen input; "it looks right" misses off-by-one and uninitialised variables.
3. After push 10, TOS=10. After push 20, TOS=20 (10 below). After pop, 20 removed, TOS=10.
4. list: ordered mutable. tuple: ordered immutable. set: unordered unique. dict: keyed pairs.
5. `def area(l, w): return l * w then print(area(3, 4))` → 12.
6. connect, cursor, execute, commit, close.
7. Scatter: two measured variables (height vs marks) with no single time axis. Line: one quantity changing over an ordered time or sequence.
8. Sorted: 2,4,6,8,10. Mean =  $30/5 = 6$ . Median = 6. Range =  $10-2 = 8$ .
9. Unknown sender, urgent threat ("account will close today"), a link that does not match the real bank, request for OTP/password, poor Urdu/English, unexpected attachment.
10. A prototype is a model used to learn (often internal). An MVP is a real, usable product with just enough features to test the riskiest assumption with actual customers.

## Section C — outline

1. Need analysis: who (students, teachers), tasks (view timetable, submit leave), constraints (mobile, slow internet). Wireframe: simple boxes for logo, fields, button, a second screen with a 5-day table. Accessibility: large tap targets, colour not the only signal, Urdu/English labels, screen-reader names. Test: five students think-aloud while logging in; record errors and time.
2.  $i=1$  total=1;  $i=2$  total=3;  $i=3$  total=6;  $i=4$  total=10;  $i=5$  total=15. Loop body runs  $n$  times →  $O(n)$ . Formula  $n*(n+1)/2$  is  $O(1)$ .
3. try: with open(path) as f: for line in f: name, m = line.strip().split(","); d[name]=int(m) except FileNotFoundError: print("missing"). Then with open(out,"w") as g: write grades. Show one sample dictionary.
4. Sorted 12,12,15,18,20. Mean  $77/5=15.4$ . Median 15. Mode 12. Range 8. Population variance: deviations -3.4,-3.4,-0.4,2.6,4.6; squares 11.56,11.56,0.16,6.76,21.16; sum 51.2;  $/5 = 10.24$ . std = 3.2. Histogram to show distribution (or bar of frequencies).
5. Problem: long canteen queues between periods. Riskiest assumption: students will pre-order 10 minutes before break. MVP: WhatsApp number + Google Form, pickup tokens, no app. Beachhead: one shift of first-year students. Metrics: % of orders picked up, time-to-collect, repeat use. Ethical: do not publish phone numbers; delete form responses after the term.